

MDMAX

frontmatter is Google Docs for markdown. MDMAX is the small library underneath it that makes writing to those files safe.

The complete build plan. Consolidates three research workflows, one deep-reasoning pass and the full conversation record. Research is closed; what follows is a build order.

14	15,445	175k	95	~18M
sections	lines	words	agents	research tokens

Contents

01	Orientation — what we are building, why now, what is settled, and how to read this	760	ln
02	The complete chronological record: what we researched, in order, and what each round concluded	1,093	ln
03	What MDMA is: every capability, specified	1,646	ln
04	Representation — what a markdown file can be made to hold, convey and mean	1,262	ln
05	Rendering — functional and visual, custom rendering, custom formatting, custom parsing	1,197	ln
06	Our own conventions: the library model, custom definitions, reference pointers, and whether any of it ships	1,117	ln
07	frontmatter the product: features, the gap, and what it needs on top of MDMA	880	ln
08	The market — USP, what competitors lack, what we solve, what we cannot solve	1,111	ln
09	AIOS: what to import, what to refuse, and how it makes the build faster and safer	1,035	ln
10	The engine specification: passes, offset model, robustness, and the hazards that will bite	1,357	ln
11	The execution plan — every step, in order, with gates	1,223	ln
12	Risks, the pre-mortem, and the incompatible wants	1,095	ln
13	Appendix — the evidence base, the corpus, and where everything lives	1,164	ln
14	Verification record	505	ln

51 Orientation

Orientation – what we are building, why now, what is settled, and how to read this

← Index · §2 Chronology →

1. Orientation — what we are building, why now, what is settled, and how to read this

This section is the front door. A reader who has never seen this project should be able to finish it and know four things: what the product is, what fact makes it worth twelve months, which questions are already closed, and where to go next. Everything asserted here is repeated in more detail somewhere later in the plan; nothing here depends on anything later.

1.1 The thesis in one sentence

frontmatter is Google Docs for markdown — comments anchored to text, suggestions, version history and sharing, over files that stay ordinary .md — and MDMAX is the small library underneath it that makes writing to those files safe.

The founder's own words, verbatim, from the session record:

"we need a Google Doc for Markdown." — HANDOFF-mdmax-markdown-engine-2026-08-01.md §14 [primary]

Restated in this planning round as "Google Docs for markdown, as simple as that." That restatement was spoken in the round that produced this document and is not on disk in a file; the on-disk verbatim is the line above. [primary for §14; the restatement is attributed to this round]

1.2 The thesis in one paragraph

Markdown's dominant reader stopped being a human and became a model, while the format itself stopped shipping. CommonMark's last three releases all landed on the same day, 2024-01-28, and there has been none since; GitHub's own renderer, cmark-gfm, last released on 2023-07-21 and is still pinned to CommonMark 0.29, whose specification is dated 2019-04-06. Against that frozen substrate, markdown is now the single most-read file type inside an agent's tool loop — 23.51% of file reads, ahead of .tsx and .ts — 94.1–94.3% of the markdown created in our own pinned corpus in June 2026 was created inside an agent session, and AGENTS.md went from nonexistent to 30.67% of the 300 most-starred repositories on GitHub in about fifteen months. That asymmetry — a frozen format with a multiplied and different consumer — is the opportunity, and it is the only part of the "new era" story that survived adversarial verification. What it is not is a volume story: the claim that markdown's share of repository trees is growing, and the claim that it is shrinking, are both unproven, and neither belongs in the pitch. So MDMAX is not a compiler programme. It is a small library, subordinate to the editor, and the product it serves is the review loop over files people own — an intersection that no shipping product currently occupies. None of it is reachable until a second person can log in, because the application today admits exactly one user, hardcoded. Research is closed. The plan below is a build order.

1.3 The three things, and how they relate

Three names appear throughout this document. They are different things with different owners, different lifespans and different success conditions. Confusing them is the most common failure of every previous draft.

	FRONTMATTER	MDMAX	AIOS
What it is	A browser-first markdown editor with a Google-Docs-style review loop over files the user owns	A capability library that makes safe, byte-exact writes and stable references into ordinary .md files	A personal agent-orchestration toolchain (skills, gates, traces, evaluation loops)
Status	Shipping, single-tenant, one hardcoded user	Zero lines written	Running, 119 SKILL.md files on disk [measured]
Who it is for	Paying users	frontmatter, and nobody else	Sagnik's own working environment
Where it lives	src/ in this repository	src/modules/ in this repository, subordinate to the editor	~/claude/ and ~/sgnk/, outside this repository
Public surface	The web app, share links, publish	One command, mdmax cert, and nothing more	None
Business model	Free / Pro / Work / Teams tiers	None. It is a library, not a product	None
Success condition	A second person logs in and writes to a document	The editor stops corrupting files	Out of scope for this plan
Kill condition	See the kill-gates section	If the editor stops needing it, delete it	Out of scope for this plan

How they relate, stated as rules:

- frontmatter depends on MDMAX. MDMAX does not depend on frontmatter's roadmap. MDMAX exists because two shipped write paths in frontmatter destroy user data today. If those paths were safe, MDMAX would be optional.
- MDMAX is subordinate, not parallel. It has no separate release cadence, no separate versioning story, no separate repository and no separate business model. Decision D10.4 in the open-decisions section settles this: "Does MDMAX have a business model? No."
- AIOS is a separate track (decision D5). It is not part of this plan's scope, budget or sequence. It benefits from MDMAX in exactly one direction — agent-written markdown is the input distribution MDMAX is designed for — and it contributes one thing back: the discipline that a check must be proven to fail before it is trusted.

What AIOS actually transfers, stated honestly. The research area aios-transfer claimed a broad transfer and its headline verdict was OVERSTATED. What survived verification is one mechanism and one number:

- sgnk-regression-gate.sh refuses to register a check until that check has been proven to go red against a reintroduced defect. Four distinct refusal exit codes confirmed in source at lines 45, 53, 58 and 73; selftest 10 of 10; 21 gates live. [measured, upheld by the kill audit]

- What was **killed**: *"the AIOS verify ladder transfers to the MDMAX build with zero modification."* On this repository the ladder inspects 2 `.sh` files, 2 `.py` files (one inside `node_modules`) and some `node_modules` JavaScript — and **0 of 901 first-party** `.ts / .tsx` **files**. `tsc --noEmit` appears only in a comment. It does not transfer as a useful verifier. [measured, killed by the verifier] — note that the 901 figure counts the whole working tree including `test/` and two abandoned git worktrees; `src/` alone holds **212** `.ts / .tsx` files [measured, this session] . The two numbers count different sets and do not conflict, and neither carries a committed derivation script.

So the AIOS contribution to this plan is a working practice, not a toolchain. It appears in the working-practices section as the rule *make the gate go red first*, and nowhere else.

1.4 The opportunity — one fact, with both halves measured

Markdown's dominant reader stopped being a human and became a model, while the format froze.

This is the entire commercial argument. It is one sentence with two halves, and both halves are measured. Everything else in the "new era" story was either overstated or refuted, and is excluded below.

The research area that produced this is keyed `the-era` in `docs/engine/research/wf-final-gate-2026-08-01.result.json`. **Its headline verdict is** `OVERSTATED` . That is not a reason to discard it — it is a reason to quote only the legs the verifier confirmed. Those legs follow.

1.4a The frozen half

COMMONMARK — the specification

0.29	dated 2019-04-06	649 conformance examples
0.30	dated 2021-06-19	652 conformance examples
0.31.0	dated 2024-01-28	652 conformance examples
0.31.2	dated 2024-01-28	652 conformance examples

→ ZERO net new released conformance examples in five years.

→ The last three releases (0.31.0, 0.31.1, 0.31.2) ALL landed on 2024-01-28. None since.

→ master HEAD today still declares version 0.31.2 / date 2024-01-28 but carries 655 examples:

+3 sitting unreleased for 2.5 years.

CMARK-GFM — GitHub's own renderer

latest release	0.29.0.gfm.13,	published 2023-07-21T15:22:23Z
preceding	0.29.0.gfm.12	2023-07-13
	0.29.0.gfm.11	2023-04-06

The tag pins CommonMark 0.29, whose spec.txt is dated 2019-04-06.

Repository is NOT archived (archived: false), open_issues_count 135.

Recent commits are build-system housekeeping ("Update cmake minimum requirement", 2025-12-19).

→ GitHub renders the world's markdown against a specification from 2019.

SPEC-REPOSITORY ACTIVITY (commits per year, commonmark/commonmark-spec)

2019	48	
2020	15	
2021	14	
2022	15	
2023	13	(a second, independent count says 12 – see the contradiction note below)
2024	11	(a second, independent count says 12 – see the contradiction note below)
2025	7	
2026	6	(seven months, partial)

[primary] — the example counts were produced by pulling `spec.txt` at tags 0.29 / 0.30 / 0.31.0 / 0.31.2 from `raw.githubusercontent.com` and counting 32-backtick example fences; the release dates came from `api.github.com/repos/github/cmark-gfm/releases` . The verifier reproduced all of it independently and recorded verdict **CONFIRMED**, calling it *"the single most decisive fact in the area."*

Two honesty notes on this block, both of which must travel with it.

- Do not say "declining every single year."** That adjective is false against the researcher's own table: 2021 = 14 rises to 2022 = 15. The verifier flagged it; the kill audit independently reproduced the counter-example to the unit and found a second break (2023 → 2024 flat) plus a third under a narrower definition counting only commits touching `spec.txt` (2021 = 9 → 2022 = 10). Say **"declining, with two breaks"** or say **"a third of its 2019 rate."** The conclusion is untouched; the word "every" is wrong. [measured, killed]
- The 2023 and 2024 counts disagree between two sources.** The research area reports `48/15/14/15/13/11/7/6` ; kill-audit agent 0, re-running the same census on `api.github.com` , reports `48/15/14/15/12/12/7/6` . **Both agree on 2019, 2020, 2021, 2022, 2025 and 2026, and both refute the monotonic-decline adjective.** They differ only on how 2023 and 2024 split. Neither has a committed derivation script. **Governing rule: quote the years both sources agree on (48 in 2019 falling to 7 in 2025) and do not quote 2023 or 2024 individually until a script is committed.** [contradiction, unresolved]

One further frozen-side finding, included because it is the sharpest single illustration and excluded from the pitch because it was not independently re-verified: `remark-parse` , the most-used markdown parser in the JavaScript ecosystem, last released version 11.0.0 on 2023-09-18 and has shipped nothing in 2024, 2025 or 2026, while its downloads went from 36,005,792 (2024-08) to 161,511,318 (2026-06), a factor of 4.49. `markdown-it` shipped zero releases in all of 2025 and had a 31-month gap between 14.0.0 (2023-12) and 15.0.0 (2026-07-30). [measured, area-level own-measurement, NOT independently re-verified – use with the caveat]

1.4b The changed-reader half

Four claims. All four were re-derived independently by the area's verifier and recorded **CONFIRMED**.

(1) Markdown is the #1 file type an agent reads.

Read tool calls carrying a path, across 13,968 .jsonl transcripts under ~/.claude/projects:

total	36,846		
.md	8,663	= 23.51%	rank 1
.png		= 20.82%	
.tsx		= 16.51%	
.ts		= 11.10%	
.py		= 6.01%	

Write / Edit / MultiEdit calls: total 19,416

.tsx		= 25.44%	rank 1
.md	4,791	= 24.68%	rank 2

[measured] — the verifier's own independent scan, which it recorded as matching the researcher's figures *"to the decimal."* Note this is explicitly **not** the pinned corpus; it is one workspace's complete Claude Code transcript set, a different and separately-stated selection.

(2) Essentially all new markdown in the pinned corpus is created inside an agent session.

SELECTION: the 991 of 1,084 corpus files that are git-tracked at each root's pinned head
(md 691/756, knowledge 249/272, frontmatter 51/56)

CRITERION: the creating commit (git log --diff-filter=A --root --no-renames) carries an AI marker

overall	578/991 = 58.3%	(verifier re-derives 577/991 = 58.2%)
2026-05	69/450 = 15.3%	(the human legacy vault import)
2026-06	448/475 = 94.3%	(verifier re-derives 448/476 = 94.1%)
2026-07	61/66 = 92.4%	(verifier re-derives 61/67 = 91.0%)

[measured, corpus_id sha256:3a010b16...] — the verifier reproduced this *"to within one file out of 991"* and confirmed the manifest's corpus_id matches. **The 94.1–94.3% range is not a rounding choice; it is the spread between the researcher's denominator (475) and the verifier's (476).** Quote the range, never a single point.

The negative that must travel with this number: *"AI-marked"* means the creating commit mentions Claude, not that a model wrote the prose. A human typing every word inside an agent session gets the same marker. The honest reading is **"created inside an agent session,"** which is an upper bound on machine authorship in the strict sense and a lower bound in the loose sense (agent-written files committed by hand are missed).

(3) AGENTS.md went from nothing to roughly a third of the most-starred repositories in fifteen months.

CENSUS 2026-08-01: GitHub search stars:>30000 sorted by stars, 300 repos,
root tree via api.github.com/repos/{r}/git/trees/HEAD. All 300 fetched, zero errors.

AGENTS.md	92/300 = 30.67%
CLAUDE.md	76/300 = 25.33%
.claude/	45/300 = 15.00%
README.md	278/300 = 92.67%

Apples-to-apples on the top 215 by star rank: AGENTS.md 69/215 = 32.09%

Earliest AGENTS.md creation month in the sample: 2025-05 → about 15 months.

[measured] — this is the **verifier's** census, not the researcher's. The researcher reported 26.51% on a 215-repo sample with an unexplained 85-repo loss; the verifier fetched all 300 cleanly and got a **higher** number. The researcher's figure was conservative, and its sample loss remains unexplained.

(4) Convention births mostly die — the same measuring stick, applied to the alternatives.

Same full-300 census, 2026-08-01:

llms.txt	1/300 = 0.33%
.cursorrules	2/300 = 0.67%
GEMINI.md	7/300 = 2.33%
AGENT.md	0/300 = 0.00%
.windsurfrules	0/300 = 0.00%
CONTRIBUTING.md	168/300 = 56.00% (a fifteen-year-old convention, for contrast)

[measured] — CONFIRMED on a larger sample than the researcher used. This is the honest frame for claim (3): AGENTS.md at 30.67% is currently the exception among agent conventions, not the rule.

(5) A supporting leg, structural rather than volumetric. Generic markdown parsers grew only **1.44x** faster than the npm registry baseline over 2024-08 → 2026-06: eight-parser median **4.0495**, fourteen-package non-markdown control median **2.8145**, excess **1.4388**. [measured] The verifier re-fetched all 22 packages and every raw figure *"reproduces to the digit,"* and noted the contamination runs in the conservative direction — marked, unified and remark-parse are all direct streamdown dependencies, so the true generic-parser excess is lower still.

The clean illustration the verifier recommended leading with, because neither package is a streamdown dependency and both do the same job in the same ecosystem:

react-markdown	11,786,121	→	103,114,261	×8.75	
markdown-to-jsx	22,908,372	→	18,185,739	×0.79	← an absolute DECLINE

[measured]

1.4c What must leave the pitch — three claims, excluded

These are not soft caveats. They are exclusions. Do not put any of them in a deck, a landing page, a README, or a sentence spoken to an investor.

EXCLUDED 1 — markdown's share of repository file trees, in BOTH directions.

The area's headline leg said markdown's share of repository trees fell 12.68% → 10.41% (2024-08 → 2026-06, 12 repos). The verifier recorded **REFUTED** on two independent grounds: unreproducible as published (the 12 repos are never named, no corpus_id is cited), and directionally reversed on a fully specified test.

```
VERIFIER, 12 named major OSS repos (2 skipped as truncated), commit nearest 2024-08-01
vs nearest 2026-06-30:
pooled md share    2.39% (4,460/186,903) → 2.74% (6,556/239,656)    ROSE
md files ×1.47  against total files ×1.28
median per-repo share  1.15% → 1.90%;  share fell in only 3 of 10

KILL AUDIT, a THIRD and different named sample — 8 repos (microsoft/vscode, golang/go,
nodejs/node, pytorch/pytorch, django/django, rails/rails, vercel/next.js, pandas-dev/pandas),
last commit before 2022-08-01 vs HEAD:
pooled md share    1.142% (908/79,491) → 1.460% (1,739/119,082)    ROSE
md files ×1.92 against total files ×1.50;  share fell in only 1 of 8
```

[measured, twice, on two disjoint named samples]

Both the verifier and the auditor also observed that the original **12.68% baseline is roughly five times higher than any real code repository either could find** (the auditor's range across 8 major repos: 0.13%–2.59%). Whatever those twelve repositories were, they were not representative code repositories.

Governing statement: the direction is unproven in both directions, and the claim is excluded. Two named, specified samples say it rose; one unnamed, unspecified sample says it fell. That is not enough to say either. It is *more* than enough to say neither. The pitch is that markdown's **reader** changed, and that is measured five independent ways above.

EXCLUDED 2 — "a new streaming-markdown tooling category." There is no category. There is one vendor.

```
streamdown      github.com/vercel/streamdown  packages/streamdown
remend           github.com/vercel/streamdown  packages/remend
rehype-harden   github.com/vercel-labs/markdown-sanitizers
harden-react-markdown  github.com/vercel/harden-react-markdown

streamdown@2.5.0 dependencies contain an EXACT pin:  "remend": "1.3.0"
                                                    and:  "rehype-harden": "^1.1.8"

Monthly series showing the dependencies tracking the parent within ~10%:
2026-04  streamdown 10,809,649 · remend 9,723,249 · rehype-harden 10,315,499
2026-02  streamdown 3,791,087 · remend 2,920,598 · rehype-harden 3,461,931
```

[primary] — verdict **REFUTED**, and upheld by the kill audit on the mechanism.

Adding a package to its own pinned dependency from the same monorepo counts one install base up to four times. **The design section built on "~30.7M downloads/month across streamdown+remend" is void.**

And the replacement number is itself contested — do not quote any downloads figure for this stack. Three different substitutes exist in the record:

SOURCE	REPLACEMENT	FIGURE	BASIS
the verifier	"~16M downloads/month for one dependency closure"		2026-06 point downloads
the kill audit	"~23.2M/month" — and calls ~16M wrong, being below	streamdown 's own standalone count	api.npmjs.org 2026-07-01..2026-07-30: streamdown 18,538,484, remend 23,223,550
PLAN.md §2	v2.0.0 "roughly 8% of	marked alone"	weekly figures: streamdown 4,691,299/wk, remend 6,173,301/wk, marked 61,032,058/wk

Governing rule: quote the structure, never the number. The defensible sentence is "Vercel shipped a model-output markdown stack whose packages are pinned dependencies of one another" — and nothing about its size, because three sources in our own record give three different sizes on three different windows and none carries a committed derivation script. [contradiction, unresolved]

EXCLUDED 3 — "nobody has served incremental markdown parsing below the presentation layer." Verdict **REFUTED**, upheld by the kill audit with all three artifacts confirmed real and correctly dated.

- @lezer/markdown — npm package created 2021-08-11, description verbatim "Incremental Markdown parser that consumes and emits Lezer trees." It is what CodeMirror 6 runs on — and CodeMirror 6 is already in this repository's dependency tree.
- tree-sitter-markdown — ikatyang/tree-sitter-markdown created 2019-09-30; tree-sitter is incremental by construction.
- micromark — the CommonMark reference implementation underneath remark/unified — ships a documented duplex streaming API, import {stream} from 'micromark/stream'.

[primary] And micromark's own README states the fundamental limit verbatim, which belongs in any design that touches this area: "Some of the work to parse markdown can be done streaming, but in the end buffering is required."

1.4d What would falsify §1.4

Pre-registered, so it can be checked rather than argued.

#	FALSIFIER	THRESHOLD	WHAT TO DO IF IT FIRES
F-ERA-1	The model-facing npm stack falls back toward the baseline	Excess growth ratio drops below 1.5× . It is 3.1× today (react-markdown $\times 8.75 \div$ control median $\times 2.81$)	Stop saying "new era" out loud. The 2025–2026 spike was AI-application churn, not a durable consumer shift
F-ERA-2	The frozen half thaws	A CommonMark release with new conformance examples, or a <code>cmark-gfm</code> release that moves off CommonMark 0.29	The asymmetry closes. Re-argue the opportunity from scratch
F-ERA-3	The reader claim fails to replicate off one workspace	A second, independently-owned transcript corpus puts markdown outside the top 3 read file types	The 23.51% is an artifact of this workspace. Drop it and rest the case on the <code>AGENTS.md</code> census, which is already external

The single largest weakness in §1.4, stated plainly: the two strongest reader-side numbers (23.51% of reads; the machine-write-to-machine-read share) come from **one person's transcripts over a three-month window**, in a workspace whose owner spent June and July 2026 building a markdown compiler. The area's own third-strongest negative says exactly this. A confound control that drops every markdown-research project directory (11,475 of 13,947 transcripts kept) preserves the trend — 6.06% → 17.04% → 24.71% of reads across 2026-05 → 2026-06 → 2026-07 — but cannot remove the fact that this person's own work shifted toward documentation over exactly those months. **The `AGENTS.md` census and the CommonMark/cmark-gfm freeze are the two legs that are external to us. Lead with those.**

One more number that was corrected downward and must be quoted in its corrected form. The area claimed *"nearly half (44.2%) of all markdown an agent reads is markdown an agent itself wrote."* Verdict **OVERSTATED**. The arithmetic was flawless; the definition was not — the "written" set unioned `Write` with `Edit` / `MultiEdit`, and the agent harness *requires* a `Read` before any `Edit`, so it manufactures read-then-write pairs on files the agent never authored. Splitting them: paths reaching a genuine `Write` (creation) account for **552 paths and 3,120 read calls = 36.0%. Quote 36.0%, "just over a third." Never 44.2%.** `[measured, corrected]`

1.5 Decisions already made — D1 through D10

These are closed. They are listed here so that nobody spends a week re-opening one, and so that a new reader can tell the difference between a decision and an opinion. Where a decision is under review, that is stated in the row; **one is (D3)**.

The "settled" column gives the date the decision was recorded in a file in this repository. Several were derived earlier, in the research waves of 2026-07-29 and 2026-07-30, and only written down on 2026-08-01; where that is true the row says so.

#	DECISION	SETTLED	RECORDED IN	CONSEQUENCE — WHAT IT OBLIGES OR FORBIDS
D1	frontmatter is Google Docs for markdown.	2026-08-01	<code>HANDOFF-mdmax-markdown-engine-2026-08-01.md</code> §14; <code>docs/FRONTMATTER-PRODUCT-PLAN.md</code> §1 names the same thing as "Wedge B — Google Docs for markdown you own" (dated 2026-07-12)	The thing we sell is the review loop , not the editor and not the compiler. Every feature is judged against "does a second person use this?"
D2	Comments and version history live OUTSIDE the file. Export or copy yields clean markdown, latest content only. Nothing about comments is ever written into the <code>.md</code> .	2026-08-01	This plan's sequence section, item "the review loop"; <code>docs/mdmax/PLAN.md</code> v2.0.0 §7	Obliges a hard gate: remove the sidecar store → <code>git status</code> clean, every file byte-identical. Forbids any marker, anchor comment, HTML comment or invisible character written into user files
D3	Reviewers must log in (Google or GitHub). Mechanism deferred. UNDER REVIEW as a tier split.	2026-08-01	<code>docs/mdmax/PLAN.md</code> v2.0.0 §7 item 4 and §10.2	The recommendation under review: anonymous read-and-comment on a share link; login required to resolve or to edit. Owner and editor seats bill; commenters do not. Until it is settled, build the share link so that both models remain reachable
D4	The durable artifact is plain markdown. If frontmatter dies we ship a full export and a migration. Max tier gets offline access. NOT "your git repo is the backend."	2026-08-01	<code>docs/mdmax/PLAN.md</code> v2.0.0 §3.1 and §4	The export is a ZIP of clean <code>.md</code> files , because that is what a departing human actually wants. Forbids shipping a single-file container as the export format
D5	AIOS is a separate track.	2026-08-01	This plan, §1.3 above	AIOS work does not consume this plan's sequence, budget or attention. It contributes one working practice (make the gate go red first) and nothing else
D6	The file stays ordinary <code>.md</code>. Degradation is the feature. No dialect needing its own parser.	2026-08-01 (derived in the 2026-07-29 wave)	<code>HANDOFF-§1</code> — <i>"The invariant that governs everything"</i> — and §5 item 2	An unknown fence language round-trips byte-perfectly; an unknown node type throws. Therefore: additive, fence-carried capability is free; capability whose payload is by reference is impossible — <code>![[Some Note]]</code> renders as the literal string with the referenced content absent
D7	Splice-only writing. Never regenerate from the AST.	2026-08-01 (derived 2026-07-29)	<code>HANDOFF-§1</code> and §5 item 1; theorem cited as Foster et al., <i>TOPLAS</i> 2007, Lemma 3.9	Forbids <code>matter.stringify</code> , <code>doc.toString()</code> and any <code>parse → stringify</code> round trip on a user's file. Two shipped code paths violate this today; fixing them is an early item in the sequence
D8	Extensibility lives in the VALUE of a field, never in the SET of node types.	2026-08-01	<code>HANDOFF-§5</code> item 4 — extension goes through dispatch (fence info string, frontmatter vocabulary), never a new sigil	Forbids inventing a sigil. Character-namespaces exhaustion is real: <code>::</code> is already contested between two top-50 Obsidian plugins. Obliges treating the YAML value slot as a hazard, not a free field
D9	A document NAMES a capability, never CARRIES one.	2026-08-01	<code>HANDOFF-§7.1</code> — every self-declaring document format that succeeded (JSON-LD <code>@context</code> , the shebang line, Racket's <code>#Lang</code>) names a vocabulary and never carries the definition	No runtime inside the document. The moment the engine needs an interpreter it is MDX, which cannot write the document back out
D10	The name is MDMAX.	2026-08-01, verified live	<code>HANDOFF-§9</code>	<code>npm</code> : <code>mdmax</code> , <code>md-max</code> , <code>markdown-max</code> , <code>mdx-max</code> — all free. PyPI: <code>mdmax</code> , <code>markdown-max</code> — free. GitHub: 13 repositories named <code>mdmax</code> , all zero-star noise. Only footnote: "MD" means molecular dynamics in chemistry

Three consequences of this table that are easy to miss.

- D6 and D8 together bound the Max tier much more tightly than "features unseen in the industry" implies.** Any capability whose payload is *by reference* — transclusion, cross-file reactive state, sidecar-resolved block metadata — is void under D6, because the reference renders as literal text with the payload absent everywhere else. Know the shape of that roadmap before selling it.

2. **D7 is not an aspiration; it is a four-line function with two callers already broken in production.** The argument about whether a splice writer is worth building ends the moment you notice it has to be written anyway.
3. **D2 makes the hardest anchoring problem moot for comments.** If nothing is written into the file, no marker can be corrupted by another tool. That is why D2 is a decision and not a preference.

And the founder's own design rule, which is D6 and D8 stated before either existed:

"we can't invent a new markdown... but we can invent a new style of utilization" — quoted in docs/mdmax/PLAN.md v2.0.0 §2 [primary]

The measurement proves the rule: an unknown fence language round-trips byte-perfectly; an unknown node type throws. **Utilization is the only available move, and it was found by reasoning rather than by measuring.**

1.6 Where the code actually stands, in one screen

A cold reader needs the starting line, not just the destination. All of the following was measured in the working tree on 2026-08-01.

branch	engine/plan-and-diagnostics	
HEAD	1bd4dad docs(mdmax): consolidate the plan, pin the corpus, commit the research base	
main	untouched by this programme	
src/	212 .ts / .tsx files	[measured]
src/modules/	ai · ai-tools · app-shell · auth · drafts · editor · export · graph · preview · repository · share · vault	[measured]
engine code	NONE. There is no core/ directory anywhere in the tree.	[measured]
tests	83 test files under `test/`	[measured]
	3,484 tests collected (66.4% of them abandoned worktrees) at last full run	[primary, HANDOFF §10, not re-run this session]
package.json	3,076 bytes, intact, name/version/engines present	[measured]

The single fact that governs the whole sequence: the application admits exactly one user, and that user is hardcoded.

src/modules/auth/domain/allowlist.ts:9
return login.trim().toLowerCase() === allowed.trim().toLowerCase();
src/container/dependency-container.ts:42
const AUTHOR = { name: "Sagnik Mitra", email: "sagnikmitra123@gmail.com" } as const;

[measured, read live from the working tree this session] — an equality against a single login, and a constant author identity fed into four use-cases. Nothing downstream of those two lines is a Google Doc, and no compiler makes it one. This is developed fully in the section on the second user.

One historical note, because two records disagree. docs/mdmax/PLAN.md v2.0.0 §9 item 10 records that package.json was "destroyed to 17 bytes by a research subagent"; HANDOFF...§10 records a different package.json incident — an unwanted +1 line: "@anthropic-ai/tokenizer" from a subagent that ran npm install from the repository root. These are two separate incidents on the same file, not two accounts of one. **Both are now moot: the live file is 3,076 bytes and intact as of 1bd4dad .** [measured]

1.7 How to read this plan

Section numbers follow the table of contents. Where this section points at another part of the plan it names the subject rather than only the number, so a renumbering cannot silently break the reference.

YOU ARE...	READ, IN THIS ORDER	WHY	ROUGHLY
A co-founder, or anyone cold	this orientation → the opportunity → the product and the market → the sequence → the kill gates	That is what is being built, why it is worth doing, in what order, and the test that decides whether this is a company	20–30 minutes
Building it	everything above, plus the engine section, the week-one hazards, and the working practices	The engine section tells you what to write; the hazards section tells you what will bite you before you finish writing it	90 minutes, then keep it open
Challenging it	this orientation's §1.4c and §1.8 → what we abandoned → what we got wrong → open decisions	§1.4c is what is excluded from the pitch and why. "What we abandoned" is what we stopped doing and what killed it. "What we got wrong" is the error log, kept on purpose	45 minutes
Deciding whether to fund it	this orientation → the product and the market → the kill gates → what we got wrong	And nothing from §1.4c's excluded list, which exists precisely so that a hostile reader with an npm terminal finds nothing to catch	30 minutes
Picking up the work cold, months from now	this orientation → the working practices → the sequence, and then read the research JSON directly using the recipes in §1.8	The research artifacts outlive this prose. §1.8 tells you how to interrogate them without trusting a summary	as long as it takes

Where the supporting evidence lives. This plan is the executive document. It sits on top of a larger record, and every one of these is on disk in this repository:

PATH	WHAT IT IS
docs/engine/PLAN.md	1,748 lines, v0.6.0. The technical design of record and every early measurement. Note: this plan supersedes it where they disagree
docs/engine/README.md	789 lines. The readable master plan / front door for the engine track
docs/FRONTMATTER-PRODUCT-PLAN.md	Demand-side research, dated 2026-07-12. 92 sourced pain points, 89 feature requests, a 20-app competitor matrix , every claim carrying a verbatim quote and a URL. Governs the editor track
docs/FEATURE-GAP-REPORT.md	Feature-by-feature benchmark against Obsidian, Notion, Typora, Logseq, iA Writer and VS Code, dated 2026-06-03, reflecting code actually present in <code>src/</code>
docs/research/	Six files: the pain taxonomy (14 ranked themes with quotes), the competitor gap map, the segment strategy, the pain playbook, and two raw-corpus JSON files
docs/mdmap/	Eight entries — the earlier concept tree (<code>01-thesis</code> ... <code>07-open</code> plus <code>MAP.md</code>). Historical; superseded by this plan
docs/engine/research/	The machine-readable research record. Four workflow artifacts plus the pinned corpus manifest. This is the primary source for every research number in this plan
HANDOFF-mdmax-markdown-engine-2026-08-01.md	The session record that closed the research phase, including the founder's own framing verbatim in §14
HANDOFF-mdz-markdown-format-2026-07-29.md , HANDOFF-graph-engineering-research-2026-07-30.md	Superseded handoffs. Kept because this record cites them

1.8 Evidence conventions, and why they are strict

Two conventions govern every number in this document. They exist because of a measured failure, not because of a preference for rigour.

1.8a The tier tag

Every claim carries one of five tags. If a claim has no tag, it is a defect and it should be deleted or downgraded.

TAG	MEANS	THE TEST
[measured]	We ran it in this programme, and the command, script or scan is nameable	Could another person re-run it from what is written here?
[primary]	We read the source itself — a specification file, a package's own README or registry metadata, an API response	Did we read the artifact, or someone's description of it?
[secondary]	Someone else's report of a measurement whose source we did not read	Named, and never used to carry a decision on its own
[inference]	A reading of measured facts, which is not itself measured	The facts underneath are tagged separately
[SIMULATED]	Replayed through code, not read from a live system	A dry run against live files is a measurement; a replay through modified code is a prediction

The research artifacts use their own two labels, `own-measurement` and `primary-read`, which map to `[measured]` and `[primary]` respectively.

One consequence, and it is load-bearing. Zero live model calls were made in any research run in this programme. Every claim of the form *"this is easier for an AI to consume"* or *"this saves the model tokens"* is therefore a **prediction, not a result**, and must be tagged `[inference]`. The only live measurement of task tokens that exists anywhere in the record went the **wrong way, at +5.2%**. Say "compression", with the denominator named, or say nothing.

1.8b The corpus id rule

Every number about our own files cites the corpus id. A number about our files with no corpus id is not a measurement.

corpus_id	sha256:3a010b1649899795d79274fc528dbece97fdabf4ff0f81cc02ab619c048c51a4		
manifest	docs/engine/research/corpus-manifest.json		
root	files	bytes	pinned head
md	756	21,068,853	02c22ec47a3a73fb25a14b11be233ad3d00a8b48
knowledge	272	3,570,923	464eb666946c8cb4ccf42c68f4464613e5fc4999
frontmatter	56	908,989	798ebbf3250a5b2a53785859d79ae15f55ebebeb
TOTAL	1,084	25,548,765	
selection rule	all *.md, case-insensitive; skip .git .next .obsidian .trash .venv __pycache__ build dist node_modules at any depth; skip any dot-directory; skip top-level Mirrors and Paste per root		
id recipe	sha256 over the sorted "root/path:sha256" list, joined by newlines, with N0 trailing newline		

[measured — the roots sum exactly: 756+272+56 = 1,084 and 21,068,853+3,570,923+908,989 = 25,548,765, verified this session]

Why this is a rule. Before the corpus was pinned, **seven irreproducible file counts** were reported, with exclusion policies flipping *within* a single research area. Any figure about local files in any earlier document that does not cite this corpus id is suspect, and should be treated as unverified rather than repeated.

And the recipe needs a committed script. Three separate agents reported they could not reproduce the corpus id — one declared *"MATCH: NONE"* after 64 variants — while the synthesis agent reproduced it exactly from the prose recipe above. A ten-line derivation script under `scripts/derive/` ends that class of false alarm permanently. **This is RECOMMENDED and not yet done.**

1.8c The 100% headline defect rate, stated plainly

This is the reason the conventions above exist, and it should be read before any number in this document is trusted.

The 18-area capability research run (docs/engine/research/wf-mdmax-capability-2026-08-01.result.json) had a 100% headline defect rate. Every one of its eighteen headlines was refuted, overstated or unsupported by its own adversarial verifier — and the correction always ran in the flattering direction.

Two independent derivations of that:

- The aios-transfer research area re-derived it and the verifier upheld the derivation: "exactly 14/18 killed-lists contain the literal token 'headline'; all four remainders kill a headline-load-bearing number (verified by reading each headline in full against its killed list)." [measured, upheld]
- My own parse of the same artifact this session, over per_area[].verification.verdicts:

```
18-area capability run - 203 claim-level verdicts
CONFIRMED    46    = 22.66%
OVERSTATED   64
REFUTED      79
UNSUPPORTED  14
→ 157 of 203 = 77.34% of claim-level verdicts are defective
```

[measured, this session, re-derivable with:]

```
python3 -c "import json,collections;d=json.load(open('docs/engine/research/wf-mdmax-capability-2026-08-01.result.json'));print(collections.Counter(x.get('verdict') for a in d['per_area'] for x in a['verification']['verdicts']))"
```

Contradiction, and it must be recorded. docs/mdmax/PLAN.md v2.0.0 §9 item 1 states "Eighteen of eighteen headlines refuted in the previous run." "Refuted" is too strong for what the artifact records. The artifact records a mixture of REFUTED, OVERSTATED and UNSUPPORTED, and at least one headline leg is recorded CONFIRMED on a primary read — the model-view area's finding that "this repo's own AGENTS.md — which opens 'Read this before' — was never loaded into the agent reading it." The governing statement is: eighteen of eighteen headlines were DEFECTIVE — each carried at least one leg refuted, overstated or unsupported — and always in the flattering direction. Use "defective". Do not use "refuted". [contradiction, resolved in favour of the artifact]

What changed in the second run. The final gate (docs/engine/research/wf-final-gate-2026-08-01.result.json) added four structural fixes — mandatory negatives, pre-registered falsifiers, a mandatory corpus id, and an audit of the verifiers themselves. The result:

```
16-area final gate - headline verdicts
CONFIRMED    5    · OVERSTATED  10    · REFUTED    1

16-area final gate - 206 claim-level verdicts
CONFIRMED   117    = 56.80%
OVERSTATED   51
REFUTED      29
UNSUPPORTED   9
```

[measured, this session]

The confirmed share of claim-level verdicts went from 22.66% to 56.80%. That is the honest measure of what the structural fixes bought. It is a large improvement and it is nowhere near sufficient to trust a headline without reading its verification block.

1.8d The kill-audit rule — a kill is strong evidence, not proof

The final gate produced 115 killed items across its 16 areas. Four independent auditors then re-checked 105 of them, and tried to overturn each kill:

auditor	sampled	upheld	overturned	overturn rate
1	29	24	5	17.24%
2	29	28	1	3.4%
3	21	17	4	19.0%
4	26	26	0	0.0%
pooled	105	95	10	9.52%

[measured, this session, re-derivable from the kill_audit array]

Two things follow, and one correction.

1. A kill is strong evidence, not proof. Between 0% and 19% of kills were themselves wrong, depending on the auditor. If a kill in this document looks wrong to you, say so — that is a supported position, not heresy.
2. Even an upheld kill may carry a wrong supporting figure. Auditor 4 reported: "5/26 = 19.2% of upheld kills carry at least one supporting figure I could not reproduce or found to be wrong." The ~16M downloads/month substitute in EXCLUDED 2 above is exactly such a case — the kill stands, its replacement number does not.
3. Correction to v2.0.0. docs/mdmax/PLAN.md v2.0.0 §0 states the verifiers are "roughly 88% reliable." That figure is not derivable from the four audits. The pooled upheld rate is 95/105 = 90.48%, and the unweighted mean across the four auditors is 90.08%. Neither rounds to 88%. Use 90.5% pooled, and always state the 0%–19.0% range alongside it, because the spread is the more informative number. [measured, corrects v2.0.0]

1.8e How to check any research number in this document yourself

Every research claim in this plan traces to one of two JSON artifacts and one manifest. You do not have to trust a summary. Read the verification block. All four commands below were run from the repository root and produce output.

```
# 1. What verdict did an area's headline actually get?
python3 -c "import json;d=json.load(open('docs/engine/research/wf-final-gate-2026-08-01.result.json'));print(*[(a['area'],a['verification']['headline_verdict']) for a in d['per_area']],sep='\n')"
```

```
# 2. What did the verifier KILL in a given area? (area keys are listed below)
python3 -c "import json;d=json.load(open('docs/engine/research/wf-final-gate-2026-08-01.result.json'));a=[x for x in d['per_area'] if x['area']=='the-era'][0];print(*a['verification']['killed'],sep='\n\n')"
```

```
# 3. What SURVIVED in that area? (this is what you are allowed to quote)
python3 -c "import json;d=json.load(open('docs/engine/research/wf-final-gate-2026-08-01.result.json'));a=[x for x in d['per_area'] if x['area']=='the-era'][0];print(*a['verification']['survived'],sep='\n\n')"
```

```
# 4. Confirm the pinned corpus totals
python3 -c "import json;d=json.load(open('docs/engine/research/corpus-manifest.json'));print(d['corpus_id'],d['total_files'],d['total_bytes'])"
```

Area keys, 16-area final gate (wf-final-gate-2026-08-01.result.json): the-era · founder-thesis · container-thesis · markdown-base · custom-pointers · hold-more · rendering-frontier · parsing-robustness · product-gap · moat · market-and-gift · aios-transfer · agentic-docs · competitive-live · against · premortem

Area keys, 18-area capability run (wf-mdmax-capability-2026-08-01.result.json): edit-format · cache-layout · ai-dialect · token-cost-tier · retrieval-shape · degradation-certificate · multi-format-ingest · provenance-boundary · collab-primitives · impact-preview · model-view · integrity · normalization-i18n · unit-of-exchange · missing-constructs · adjacent-transplant · budget-knapsack · governance-and-moat

The four synthesis lenses in the final gate, at synthesis[0..3] : era-and-vision, product-and-market, technical, the-verdict.

Reading order inside an area, and this is not optional: read verification before you read headline . The headline is the researcher's claim; verification.survived and verification.killed are what is left of it.

1.9 What this section does not cover, and what would make it wrong

Not covered here. Each has its own section, and this one deliberately does not pre-empt them:

- What was abandoned and what killed it — the mother-markdown container, and markdown-as-a-graph.
- The engine itself: the splice writer, block identity, the two-tier link resolver, mdmax cert .
- The product, the competitive intersection, the pricing, and the give-away budget.
- The second-user problem in full, which is the thing that actually decides this.
- The build sequence, the kill gates, the open decisions and the error log.

What is NOT established, anywhere in this document. Repeating any of these as fact is a defect:

CLAIM	STATUS
markdown's share of repository trees is growing	unproven — two named samples say it rose, one unnamed sample says it fell
markdown's share of repository trees is shrinking	unproven — same evidence, same reason
the size of the streaming-markdown install base	three conflicting figures in our own record ; quote the structure, not a number
"markdown is easier for an AI to consume"	prediction, not result — zero live model calls in any research run
the 99.627% content-derived re-anchoring figure	re-derived by NOBODY , including every research area that quoted it. Do not publish it until an independent re-derivation lands
"there is a new streaming-markdown category"	refuted — one vendor, one pinned dependency closure
CommonMark spec commits "declined every single year"	refuted — 2021 = 14 rises to 2022 = 15

What would falsify this section. F-ERA-1, F-ERA-2 and F-ERA-3 in §1.4d are pre-registered with thresholds. Beyond those:

- **If the reading guide sends the wrong reader to the wrong section**, that is a defect in this section, not in theirs. It is measurable: ask Amit, after he reads the co-founder path, to state the one fact and the one kill gate. If he cannot, §1.7 is wrong.
- **If a builder has to ask a question that this section should have answered** — what is MDMAX, what is AIOS, is D3 open — this section failed at its only job.
- **If any number in this section is quoted downstream without its tag or its corpus id**, the conventions in §1.8 are not being enforced, and the next research programme will reproduce the 100% defect rate exactly.

And the honest limit of §1.8. The conventions above are the fixes that took the confirmed share of claim-level verdicts from 22.66% to 56.80%. They did not take it to 100%, and there is no reason to think a third round would. **The correct posture toward every number in this plan is: it is probably right, its direction is probably right, and its exact value should be re-derived before it is published.**

52 Chronology

The complete chronological record: what we researched, in order, and what each round concluded

← Index · ← §1 Orientation · §3 Capabilities →

2. The complete chronological record: what we researched, in order, and what each round concluded

This section is the programme's memory. It exists so that a person who has never seen this project — a new engineer, a new co-founder, a smaller model asked to continue the work, or either of us in six months — can reconstruct **how** every decision in this plan was reached and **why** it is still held, without opening another file. Nothing here is summarised away. Where a round produced a number, the number is here. Where a round was wrong, the error is here, with the measurement that caught it.

There is a second reason this section is long. Twice in this programme a conclusion was re-derived from scratch because nobody could find the evidence for the first derivation, and once a claim was published that three sources already on disk refuted. A written chronology is the cheapest defence against both.

2.0 How to read this section

Evidence tags. Every factual claim below carries one:

TAG	MEANING
[measured]	a command was executed and the number is its output
[primary]	the source document, source file, or API response was read directly
[secondary]	reported by another document that read the source; not re-checked
[inference]	reasoning over things that were read; not itself measured
[SIMULATED]	replayed through code rather than read from a live system

Timezone rule. Two clocks appear in the record and they are 5 hours 30 minutes apart. Conversation transcripts stamp events in **UTC** (suffix `Z`). Git stamps commits in the founder's local time, **IST = UTC + 5:30** (suffix `+05:30`). Every timestamp below is labelled. Where a conversion matters the arithmetic is shown inline, e.g. `2026-07-31T20:43:37Z → 2026-08-01 02:13:37 IST`. This is not pedantry: the programme's most important night of work spans a date boundary in one clock and not the other, and reading the record in a single timezone makes it look as if events happened on the wrong day.

The pinned corpus. Every figure in this plan that describes local markdown files cites:

corpus_id	sha256:3a010b1649899795d79274fc528dbece97fdabf4ff0f81cc02ab619c048c51a4				
total_files	1,084				
total_bytes	25,548,765				
roots	md	head 02c22ec47a3a73fb25a14b11be233ad3d00a8b48	756 files	21,068,853 B	
	knowledge	head 464eb666946c8cb4ccf42c68f4464613e5fc4999	272 files	3,570,923 B	
	frontmatter	head 798ebbf3250a5b2a53785859d79ae15f55ebebeb	56 files	908,989 B	
policy	all *.md, case-insensitive; skip .git .next .obsidian .trash .venv __pycache__ build dist node_modules at any depth; skip any dot-directory; skip top-level Mirrors/ and Paste/ per root				
manifest	docs/engine/research/corpus-manifest.json (5,466 lines, committed in 1bd4dad)				

[primary – read from the manifest file]. **The corpus was pinned on 2026-08-01 and not before.** Any figure about local files produced earlier in this record does not cite it and should be treated as unreproducible. Section 2.8 lists the seven specific counts that are known to be irreproducible.

A note on what "we" means. The programme has one human founder in the transcript record — Sagnik Mitra. Amit, the co-founder, is named by Sagnik on 2026-08-01 (§2.4) but does not appear in any session. Everything else labelled "we" is a Claude session or a research subagent working under instruction. This matters because §2.8's failure list is almost entirely machine-generated error that a human did not catch for days.

2.1 The timeline

Phase 0 — the repository, before there was a research programme

2026-07-13 → 2026-07-25. No research. Four commits. The thing being researched did not exist yet.

WHEN (IST)	COMMIT	WHAT
2026-07-13 16:33:44	b9ddcfe	Hello frontmatter — the repository's first commit
2026-07-13 16:37:26	5ddb1ab	fix: pin typescript to 5.9.3, scope turbopack root
2026-07-17 00:40:41	0c1b427	feat: clone md app into frontmatter (product base)
2026-07-25 18:13:59	8eb4de2	feat: Firebase backend foundation, product docs, Tauri shell, frontmatter rebrand

[measured – git log --format='%H|%ad|%s' --date=iso-strict]

Two facts from this phase govern everything after it.

First: frontmatter is a clone. 0c1b427 says so in its own subject line. The editor was not designed from scratch; it was lifted from md, an existing personal markdown app, and rebranded. That is why §2.8's editor-bug list is long and why the product-gap research in §2.1.5 found exactly one hardcoded user — the code was never written for a second person.

Second: `main` has not moved since **2026-07-25**. `8eb4de2` is still the tip of `main` and of `origin/main` at the time of writing `[measured - git branch -a -v]`. Everything in this plan lives on `engine/plan-and-diagnostics`.

The session that produced `8eb4de2` ran `2026-07-25T11:27:33Z → 2026-07-25T12:59:39Z` (16:57 → 18:29 IST), 92 minutes `[measured - transcript a09c1a4c-c1d6-492c-bad2-aa3b171be394.jsonl, first and last event timestamps]`. Its opening instruction, verbatim:

"This is basically the Markdown Project, the MD project, and we are reshaping such front matter for commercial selling and subscription purposes. For that, we are setting up a Firestore database at the backend. Vercel and GitHub are something that we'll use with Google sign-in and GitHub sign-in." — 2026-07-25T11:28:22Z `[primary - transcript]`

That commit also landed `docs/FRONTMATTER-PRODUCT-PLAN.md` (271 lines) and `docs/research/` (4 files, 1,097 lines). The product plan records its own method verbatim: *"11 parallel research agents swept Reddit ... Hacker News (Algolia API, 3,220+ comments keyword-scored), Product Hunt, Kickstarter/Indiegogo/indie launches, review sites ... GitHub issue trackers of every git-sync tool ... and 20 competitors' official pricing pages"*, yielding **92 sourced pain points, 89 feature requests, a 20-app competitor matrix** `[primary - docs/FRONTMATTER-PRODUCT-PLAN.md:6]`. This is the **demand-side** research base. It predates the entire engine programme and it is the only research in the corpus that talked to users rather than to registries. It governs the editor track.

A discrepancy, named. The brief for this section asks for a timeline beginning **2026-07-12**. The earliest artifact I can find is `b9dddcfe` at 2026-07-13T16:33:44+05:30, and `.vercel/project.json` with a filesystem mtime of `13 Jul 16:39` `[measured - ls -la]`. **Nothing dated 2026-07-12 exists in this repository.** The git record governs. If something happened on 2026-07-12 it happened outside version control and outside every transcript in `~/claude/projects/`.

Round 1 — the mdmap / MD3 / MDZ session

Session `e98d6e0f-f1fc-42ab-b8fb-e4c972dbd687`. `2026-07-28T21:05:45Z → 2026-07-29T03:30:27Z` (2026-07-29 02:35 → 09:00 IST). 6h 25m. Produced `docs/mdmap/` (untracked) and `HANDOFF-mdz-markdown-format-2026-07-29.md`. Zero commits. `[measured - transcript span]`

What triggered it. The founder asked for a "Markdown Map" — a recursive tree of markdown files, each containing seven more, each of those containing ten or eleven, interconnected like Obsidian's graph, readable end-to-end by a human or an AI. Verbatim, 2026-07-28T21:05:46Z:

"From everything that we saw, I need one Markdown. There can be 10 Markdowns, which is perfectly fine, but what I want you to create today, from this time onwards, is a set of Markdown structures, like a Markdown tree. We can call it MDMAP, maybe, or Markdown Map. ... Every single Markdown will have a graph, and they can be interconnected. ... The naming can be decided later, but I want to create this concept." `[primary - transcript]`

What ran. Four waves, all recorded with mechanically-sourced token counts read from task-notification usage blocks:

WAVE	AGENTS	SUBAGENT TOKENS	TOOL USES	DURATION
Round-1 fan-out (graph/PKM prior art, markdown graph specs, AI codebase maps, Reddit/HN pain sweep, canvas rendering, naming/trademark)	6	1,164,842	352	—
Workflow <code>w3f3hmo9k</code> (<code>wf_b8d494a7-c15</code>) — markdown as container	17	3,155,872	954	~51 min
Workflow <code>w24o7rxuc</code> (<code>wf_9cc55917-109</code>) — the compiler thesis	14	2,693,995	873	~82 min
Academic literature sweep (8 delivered of 11 dispatched)	8	1,457,736	421	—
total	45	8,472,445	2,600	—

`[secondary - HANDOFF-mdz-markdown-format-2026-07-29.md §4 "research spend", figures stated there as read from task-notification usage blocks]`. Arithmetic check: 1,164,842 + 3,155,872 + 2,693,995 + 1,457,736 = **8,472,445** `[measured - python3]`. The handoff's own total reproduces exactly.

What it concluded. Eight things, in the order they were established:

- Do not build a new markdown format.** Every format that added power lost adoption. `MDX` — Vercel- backed, Next.js-native, 9,426,793 npm downloads/week — is **3.07% of .md volume, one file in 33**. Markdoc (Stripe) is **0.136%, one in 736**. Obsidian's own `.base`, in an app with millions of users, is **0.028%**. `.djwt`, written by *CommonMark's own author* and technically superior, gets **848 downloads/week against remark-parse's 44,612,912 — a factor of 52,610**. Meanwhile `AGENTS.md`, which has **no schema at all**, exists in **175,424 files on GitHub**, and `SKILL.md`, which has two frontmatter fields, has **44 cross-vendor clients**, with Cursor shipping `/migrate-to-skills` to deprecate its own format into it. `[secondary - HANDOFF-mdz §2.1]`
- Base64 binary inside markdown is arithmetically dead.** Five independent measurements across two tokenizer families converged on **0.90 tokens per byte**: the session's own run on three real repo images gave 0.70–0.87 tok/byte (o200k_base, tiktoken); three workflow agents gave 0.8966–0.9198, 0.9101 and 0.920; an adversarial re-computation over 600 B / 50 KB / 1 MiB gave 0.9033–0.9104. Consequence: **a 200 KB JPEG consumes 93.1% of a 200K context window**; 1 MB ≈ 944,000 tokens. A real Jupyter notebook is **98.3% base64 by tokens — 227,352 tokens, 114% of a 200K window**. `[secondary - HANDOFF-mdz §2.2; the 0.70–0.87 leg is [measured] by that session]`
- The intersection this product wants was vacated from both sides in 2026.** *Reflect Open* (MIT, 1.4k stars, shipped 2026-07-14) keeps plain markdown as source of truth and ships a CLI *"for scripts and agents"* — **and ships no map view**. *IWE* (Apache-2.0, ~1.3k stars) is a real markdown knowledge graph with LSP, CLI and MCP — **and has zero visual surface**. `[secondary - HANDOFF-mdz §1.4]`
- A map must be smaller than the thing it maps.** Measured against the founder's own live knowledge graph: 273 files / ~546,933 words produced **3,848 nodes, 3,737 edges, 341 communities**, of which **329 were auto-named "Community 11"..."Community 339"** and only **6 of 341 (1.76%) carried a usable label**; the report artifact alone was 141,837 bytes (~35k tokens) and `graph.json` 3,414,569 bytes. `[measured - that session ran the greps; the handoff records that its own first estimate was wrong and was corrected by counting]`
- A direct competitor exists.** `mdbase-dev/mdbase-spec` — "Typed Markdown Collections Specification", v0.3.0, 88 stars, 17 spec chapters, org created 2026-01-30, pushed 2026-07-28 — ships a CLI, a TypeScript implementation, an **LSP**, a Rust implementation, an Obsidian plugin and `mdbase.dev`. Its feature list is this thesis nearly verbatim, *including the plain-markdown constraint*. Its gap: identity is **record-level** (one ULID in one file's frontmatter). **No concept of a block**. `[secondary - HANDOFF-mdz §3.4]`. **It was found by accident while checking domain availability.**
- The academic ancestor is 30 years old.** Software Reflexion Models (Murphy, Notkin & Sullivan, FSE 1995; TSE 27(4) 2001; SIGSOFT Retrospective Impact Award 2011). Its Excel case study: ~1.2M lines of C, 77,746 calls in the source model, a **170-line map written in a few hours**, first model computed in **20 minutes**, first result **15 convergences / 83 divergences / 4 absences**. The engineer's counterfactual estimate for the same insight by other means was **up to two years**; it took four weeks. `[secondary - HANDOFF-mdz §3.7]`
- Structure that users must declare has been failing since 1992.** Aquanet (Hypertext '91) shipped typed objects with typed relations and user-defined schemas — the exact design being proposed — and its own two-year postmortem, *"Two Years before the Mist"* (ECHT '92), reports verbatim: *"we observed far less use of*

relations than we had anticipated... Instead of articulating how different types of objects were related, users conveyed implicit relational structures through the use of spatial layout" and "the tool coerced us into prematurely schematizing our domain". gIBIS (TOIS 6(4) 1988), from a deployed 32-user / 2,091-node trial: "there are nine link types, and the feeling is that we are near the limit of people's ability to reliably perform the [color] mapping." **Nine is the measured ceiling for an unaided type vocabulary.** [secondary – HANDOFF-mdz §3.14]

8. **Identity must be a name, not a hash.** *Incremental Computation with Names* (Hammer et al., OOPSLA 2015) states that structural, hash-consed matching "recomputes and reallocates a linear number of output elements for each $O(1)$ input change" while nominal matching "need not rebuild the prefix." **Content-hash identity is $\Theta(n)$ per edit; stable names are $O(1)/O(\log n)$.** This is an asymptotic requirement, not an optimisation. [secondary – HANDOFF-mdz §3.16]

Artifacts. docs/mdmap/ — 21 files, **16,906 words** [measured – find | wc -l and cat | wc -w, re-run at write time; both reproduce the handoff's figures exactly]. It declares 81 nodes of which 20 exist (61 "ghosts"). A published interactive artifact at <https://claude.ai/code/artifact/3d22ae8a-2293-462a-b800-85d5f53a2309>, HTTP 200 at write time [secondary].

What went wrong in this round. Three agents were still in flight when the handoff was written and their findings were never collected. One agent (a3822aff54512c4cb) **returned empty** — it fanned out to sub-agents and returned a status note instead of findings, wasting **129,126 tokens**; it had to be re-dispatched with an explicit no-further-fan-out instruction. [secondary – HANDOFF-mdz §1.14]

Round 1b — the graph-engineering contribution

A separate session, launched from a different working directory; its transcript is not in this project's ~/.claude/projects/ folder. Recorded only as HANDOFF-graph-engineering-research-2026-07-30.md, 675 lines. Zero commits of its own.

What triggered it. The founder pasted a YouTube URL (H7t3uUp3HWw , "Anthropic Just Fixed Graph Engineering's Greatest Flaw", channel AI LABS, 847 s, uploaded 2026-07-29, 2,249 views at fetch) with:

"check this detailed and end to end way and let me know the detailed summary of this. So dig deep inside this graph engineering because I am planning to build an upgraded version of Markdown and also build front matter, which will be a Markdown editor." [primary – HANDOFF-graph-engineering §1.1, quoted verbatim there]

What it concluded. Three things survived; one is the reason the document is in the record at all.

1. **The video's central claim is marketing.** "Anthropic just released a fix" is false: "graph engineering" is a **community coinage that trended mid-July 2026**, and the patterns it describes (parallelisation, orchestrator-workers, evaluator-optimizer) are from Anthropic's *Building Effective Agents*, **December 2024**. [primary – verification searches recorded in §1.4 of that handoff]
2. **Two different graphs were being conflated.** The video's graph has **agents as nodes and data flow as edges**. A markdown editor needs a graph with **notes/blocks as nodes and links as edges**. These are unrelated problems and the conflation would have produced the wrong architecture. Agent-graph ideas were routed to the editor's AI features; knowledge-graph ideas to the engine. [primary – §1.3]
3. **The judge-model lesson, kept.** Verbatim from the video: "The node that does the judging is the one place where saving tokens costs you everything." A Haiku reviewer returned a long list of issues that were mostly intentional; Opus flagged fewer, all real. [secondary – the video is the source]

How it entered the record. The founder pasted it as a payload on 2026-07-29T19:40:08Z and again at 2026-07-29T20:26:54Z [primary – transcript 2e90ab3b]. It was folded into docs/engine/PLAN.md as §14 by commits 575768e and c847f62 on 2026-07-30. Its own author's verdict, in its opening: " docs/ engine/PLAN.md v0.2.0 ... is CANONICAL and more advanced than the design in Part II of this document. Do not merge Part II's proposals into PLAN.md." [primary – that file's header]

Its one durable contribution beyond the correction: it flagged a **live architecture conflict** — docs/FrontMatter-PRODUCT-PLAN.md §4b says "No peer CRDT — server-authoritative" while docs/engine/PLAN.md §8 specifies Eg-walker/Braid. That conflict was opened 2026-07-30 and stayed open for two days. It is resolved in §10 of this plan.

Round 2 — the engine plan session

Session 2e90ab3b-4a90-4362-bce4-042a842a2af5 . 2026-07-29T02:34:35Z → 2026-07-31T23:18:20Z (2026-07-29 08:04 IST → 2026-08-01 04:48 IST). **68 hours 44 minutes**, 36 human prompts. Produced 16 commits and HANDOFF-mdmax-markdown-engine-2026-08-01.md . [measured – transcript span; human prompt count from a transcript parse excluding task notifications and local-command echoes]

This is the longest single session in the programme and it contains three of the four founder interventions that changed the project.

2026-07-29T03:31:36Z (09:01 IST) — the scope correction. Answering a structured question set, the founder wrote, verbatim:

"Front Matter we are building separately. This will be the backbone of frontmatter. Front Matter is just an editor. Check the MD project first. Front Matter will be just a clone of that. This is the markdown engine that we are gonna build, okay? The new markdown, basically, as I mentioned. Before building anything, let us just have the plan sorted properly: what exactly we are gonna build." [primary – transcript 2e90ab3b, tool_result of an AskUserQuestion at 2026-07-29T03:31:36Z]

Three further answers arrived in the same turn: drift handling = "Both, gated by confidence"; competitors (mdbase, IWE) = "Route around both"; naming = "No separate name — it's just frontmatter" (later revised, §2.5). **This is the founding constraint of the programme.** Everything after it treats the engine and the editor as separate tracks. It is decision **D5** in this plan's terms, one round before AIOS was even discussed.

2026-07-29T21:26:51 IST — the first four commits. 5ff90a4 (mdmap tree + engine plan v0.2) and 58322f7 , which fixed **two real bugs in shipped code**: extractBodyText was deleting every fenced code block and table before indexing, and graph-data.ts was silently dropping unresolved links (if (targetPath === undefined) continue). Then 33d1b3c (§12 token corrections), 57411a1 (§11a, the block is the unit of identity not of meaning) and ae81971 (§1.1a, the re-anchoring algorithm). [measured – git log]

2026-07-29T21:19:59Z (2026-07-30 02:49 IST) — the extensibility question. The founder pushed back on the "no new extension mechanism" position:

"I'm unable to understand one thing. If Markdown is just the pre-processor of the representation, so a table is represented in a certain way, and JavaScript and MDX are represented in a certain way, why can't we find more such representations? Why can't we have a new version of Markdown with a folder tree, self-including files, and a file tree I am mentioning multiple times? Why can't we do that? First, tell me that." [primary – transcript]

2026-07-29T21:32:22Z (2026-07-30 03:02 IST) — the sentence that changed the project.

"No, before doing that fix, I guess we are still doing way less research than we should be. ... Just check that, research more deeply, and research more openly. **Don't just form an opinion just because you have to. I am absolutely fine with being proven wrong that we can't do, but at least let's push and improve how we pass Markdown.** ... People still think it is just for AI and can't be done more than that, but I think several maps can be done through Markdown." [primary – transcript, verbatim]

In the same message he sketched the mechanism that would eventually become D8: "we can define structures within Markdown. Maybe only the front matter will know that this means that. ... We can have our own language that the system will write, and the front matter will understand while rendering."

What ran in response. Five agents were briefed to argue for extension, against the standing prior. The result was f60eac0 — §15, "markdown is far more extensible than §10 claimed" — which **overturned §10's own "no new extension mechanism" row**. Fence dispatch was found to be markdown's real, permissionless extension mechanism, validated across 30 uncoordinated platforms: **330,496 .md files carry a mermaid fence against 259,136 .mdx files in existence** [secondary – PLAN.md §10, §15.1]. Within the same session the claim was **self-corrected**: 330,496 is a real absolute count but the **rate is 0.89%** (0.26% excluding mermaid's own docs, 0.17% on the founder's own vault) — "it validates the mechanism, not its penetration" [primary – README.md II.5].

2026-07-29T22:34:07Z — the six-area utilization sweep. The founder: "Yeah, then research completely and very deeply around those six areas. Okay, I'm perfectly fine to do that." Six independent sweeps: construct frequency · plugin ecosystems · security · accessibility · internationalisation · domain conventions. Token counts for these agents are **not recorded in any artifact**; they are inside the "~26 earlier agents" bucket (§2.2). Outputs, all committed 2026-07-30:

COMMIT	IST	WHAT IT ESTABLISHED
c565be9	04:30:10	§16 — plugin demand read as revealed preference. 6,047 Obsidian plugins, 132,834,339 cumulative downloads ; classifying the top 300 (83.9% of demand) gives FORMAT 40.1% · APP 31.0% · EXTERNAL 19.5% · RENDER/ERGONOMICS 9.4% . It replicates on three populations sharing no mechanism: plugins 40.1%, forum requests 39.0%, VS Code extensions 35.4%
eadbf6f	04:36:52	A DOM-verified XSS regression corpus, 55 payloads, proven to fail 28/55 against a deliberately neutered policy before being trusted — the discipline of making a test fail against unfixed code
525ad98	04:42:00	§17 — what breaks outside English. NFC decomposes Bengali ৳ / ঋ / ঌ (Unicode composition exclusions), so "just normalize everything" rewrites 77 of 437 lines of a real poem file. Markdown's link syntax is built entirely from bidi-mirroring neutral characters , so RTL source visual order structurally cannot match logical order. axe-core fires only 7 of its 105 rules on 2,286 rendered markdown files, and 98.5% of 11,695 violations trace to two causes in page chrome
9055884	05:40:15	docs/engine/README.md, the 789-line master plan

[measured – git log; figures [secondary] from PLAN.md §16–§17 and README II.4]

2026-07-31 evening — the dossier, and a warning sign. Three commits (8c0a426 , 88a92f4 , 798ebbf) produced and then cosmetically fixed a 12-page PDF. 798ebbf is worth one line because it was done correctly: page overflow was **measured, not eyeballed** — a Puppeteer script reported per-page clearance and found **page 11 at –258px and page 7 at –33px**; the fix split the overflowing section rather than deleting the benchmark table [secondary – HANDOFF-mdmax §2.8]. It is also the moment the programme's own later self-assessment identifies as the turn: "three of four recent commits were cosmetic PDF fixes. Honesty was being converted into inertia." [primary – docs/mdmax/PLAN.md:441–442]

2026-07-31T20:43:37Z (2026-08-01 02:13 IST) — the MDMAX reframe. In a single 1,100-word message the founder produced the product's name, its thesis, and the "Google Doc" framing. Verbatim excerpts, in the order they appear in that message:

"The utmost utilization of Markdown, as you mentioned, is not a format problem. **Maybe we can't invent a new Markdown, maybe we can't invent a new format, but we definitely can invent a new style of utilization of Markdown, how more Markdown can be utilized.**"

"we generally share let's say PDF or markdowns or PDF or PPT or let's say what documents, a different file, different format, different document. ... AI need to process those PDFs. So why can we have this custom rendered markdowns which will be rendered as that defined in that particular whatever desired file?"

"**we need a Google Doc for Markdown.** We need a simplified format for writing Markdowns so people can write markdowns and then Markdowns will be rendered as let's say define formats"

"**this is markdown maxing okay this is uh nothing but markdown maxing so mdmax is something I'm also thinking as a name rather than md3** so this is markdown maximize so maximize use of markdown"

[primary – transcript 2e90ab3b, all four excerpts verbatim from one message]

Contradiction, named. HANDOFF-mdmax §2.9 renders this as "we generally share PDF or markdowns or PPT... AI need to process those PDFs. So why can we have this custom rendered markdowns... one single file for everything... this is markdown maxing... MDMAX." That is a **compression with ellipses of two separate messages**, not a verbatim quote — "one single file for everything" comes from the 2026-07-31T22:35:50Z message, not this one. Where they differ, **the transcript governs**; the handoff's version is a paraphrase and should not be quoted as verbatim.

2026-07-31T22:35:50Z (2026-08-01 04:05 IST) — the library idea and the naming request. Same night, two hours later:

"How we can develop this Markdown System? **It can be called MDMAX? Just check if MDMAX is something we can use.** Because we are just maxing the Markdown usage. Utilizing the to the max. ... The file format will be dot md only but how we can utilize the maximum number of owned representations?"

"how markdown can markdown also libraries like Python can we import a library mark content those functions those instruction I would say and then we can refer resurface that and using by using that we can maybe generate charts in markdown and if I say import let's say a markdown library then markdown will know that okay this library is hosted let's say SGNK only ... **one single file for everything** that's what I want I want the features to be there in one single file **I don't want to be over jargon but if it is a library importing things and people are not jargon at all they can use features that they want to in Python there are thousand libraries people might use hundreds only**"

"we are building a markdown editor so **our editor on the max plan should give features that is unseen in the industry** that's it"

[primary – transcript 2e90ab3b, verbatim]

What ran in response. The **25-agent MDMAX foundations workflow**, wf_3b9ab146–77b : **25 agents · 3,059,467 subagent tokens · 736 tool uses · 1,872,350 ms · 0 errors** [secondary – HANDOFF-mdmax §2.11, figures read from the completion notification]. Six areas: package/ import model · format governance · transitive doc graphs · editor frontier · AI-to-AI · adversarial gap audit. Findings preserved at docs/engine/research/wf-findings-2026-08-01.md (454 lines) — rescued from /tmp before it was cleaned.

What it concluded, in four results that still stand:

1. **The library model is viable and has a named blueprint — but it costs you your identity.** Only **two** document formats ever acquired a package ecosystem: **LaTeX** (7,021 CTAN packages, 35 years) and **Typst** (1,481 packages in ~3 years). *Both are Turing-complete languages whose surface syntax happens to be documents.* Everything markdown-adjacent — Quarto, Sphinx, MkDocs, Pandoc, R Markdown — **deliberately refused** document-declared registry resolution. MDX has real `import` statements and built **no registry at all**. The agent's own words: *"acquiring a package ecosystem means becoming a programming language, or admitting you are a thin shell over one. Decide which, explicitly, before designing syntax."* Typst's own vendor blog says its registry was *"a minimum viable package manager that a single person could build in a week."* [secondary — HANDOFF-mdmax §7.1]
2. **CommonMark is frozen — do not spend a week on it. Zero net new conformance examples** between 0.30 (2021-06-20) and 0.31.2 (2024-01-28); the entire 0.31 changelog is typos and link fixes. 81 of 1,848 commits since 2020, **44% by one person**. No `CONTRIBUTING`, no `GOVERNANCE`. The 1.0 blocker list opened in 2015 still says "[6 remaining]", last edited 2019. Two IETF principals offered to formalise the spec — **no maintainer reply in four years**. Its own author built **djot** in 2022 rather than amend it. [secondary — HANDOFF-mdmax §7.2]
3. **For a document graph, ship an INDEX, not a container.** Anthropic ships both from one corpus: index **38,847 B / 174 links**; inlined container **6,556,407 B — 168.8x**, ~1.64M tokens, larger than any context window. Index plus three selectively-read docs ≈ 17K tokens, a **96x saving**. And **LLMs do not traverse passively**: 97% of published `llms.txt` files got **zero requests**; AI retrieval bots were 1%; **coding agents with Read/Fetch tools were 10%** — the largest AI slice. For `CLAUDE.md`, Anthropic **gave up on traversal entirely and eagerly inlines imports at launch**. [secondary — HANDOFF-mdmax §7.3]
4. **Do not build a new AI-to-AI format.** *"Markdown"* appears **zero times** in all six MCP schema versions and in A2A. Every protocol wraps an **opaque** text blob in a typed envelope; they carry **zero** provenance, confidence, attribution or lineage. The one standard that *does* privilege markdown is **C2PA §A.9.3.2**, which says claim generators **SHOULD PREFER YAML front matter in a Markdown file** for a signed manifest — and **§A.8**, drafted for *"content intended for copy-paste operations across different systems"*, is still marked "under review". [secondary — §7.4]

A failure inside this workflow, recorded because it recurs: its synthesis agent returned a status note — *"Reconciliation clean. No mutations or commits by this subagent. Deliverable delivered."* — **instead of its synthesis**. The real content was only recoverable from the journal. Free-text agent returns exhibit this; schema-forced agents did not. [primary — HANDOFF-mdmax §2.11] This is the second instance of the same failure in the programme; the first was `a3822aff54512c4cb` in Round 1.

2026-07-31T23:14:15Z — the round ends. *"Ok, as my tokens are almost over on this account, I want you to give a complete detailed brief and hand over to the other account."* [primary — transcript] The result is `HANDOFF-mdmax-markdown-engine-2026-08-01.md`, 420 lines. Its most important content is §6: **two load-bearing claims refuted on 2026-08-01 that the plan files still contained** (see §2.7 items 6 and 7).

Round 3 — the MDMAX consolidation session

Session `9e5a4a9f-8c4a-4758-b560-0afea2b30447`. `2026-07-31T23:23:37Z` → ongoing (`2026-08-01 04:53 IST` →). 24 human prompts. Produced commit `1bd4dad`, the first push in the programme's history, and this plan. [measured — transcript span and prompt parse]

How it opened. The founder pasted the handoff, then ~50 screenshots of the previous account's conversation across six batches, then, `2026-07-31T23:37:50Z`:

"Done. These are all the conversations. I took so much effort to screenshot all the conversations and send them to you so that you don't miss out on even a single thing that we discussed there. ... Go through everything very deeply again ... no scheme, every single line properly read ... understood about the different pathways we took, understood about where we collided, where we had different opinions" [primary — transcript]

What that deep read found (2026-07-31T23:41:05Z), and it is a method failure worth its own line. The programme's test-count claim was inflated and had been for two days:

2026-07-29 08:12 IST	worktree claudes/competent-bassi-5da9a1 created	82 test files
2026-07-29 21:27 IST	commit 58322f7 lands	
	PLAN.md §13 records "2,366 tests passing across 164 files"	
	82 (test/) + 82 (worktree) = 164	<- exact match
2026-07-29 22:07 IST	worktree claudes/upbeat-euclid-60dbf4 created	82 more → 247

Executed at read time: `vitest list --run --filesOnly` → **247 files, 3,484 tests**, of which **164 files / 2,314 tests (66.4%) are two abandoned worktrees pinned at 8eb4de2** — the commit *before* all 16 commits of work. `vitest.config.ts` sets no `exclude` and the default does not cover `.claude/worktrees/**`. [measured — that session executed it] Both worktree branches still exist [measured — git branch -a -v, re-run at write time: `claude/competent-bassi-5da9a1` and `claude/upbeat-euclid-60dbf4`, both at `8eb4de2`].

Contradiction, named and resolved. `HANDOFF-mdmax` §0.7 states *"Test suite is green: 65 test files, 3,484 tests collected (66.4% of them abandoned worktrees) at last full run."* The measured figure is **247 files / 3,484 tests, of which 164 files / 2,314 tests are stale duplicates. The measurement governs.** The number written into `PLAN.md` §13 as the evidence that commit `58322f7` was safe was already inflated when it was written. Neither "65" nor "3,484" should be repeated.

2026-08-01T01:52:55Z (07:22 IST) — the 39-agent capability run lands. 39/39 agents, 0 errors, 7,864,147 subagent tokens, 2,159 tool uses, 105 minutes. [primary — the assistant's own report in the transcript, quoting the completion notification] Shape: 18 research areas, each with a dedicated adversarial verifier, plus 3 synthesis lenses (google-docs, ai-native, critic). Preserved at `docs/engine/research/wf-mdmax-capability-2026-08-01.*` (raw 1.49 MB, journal 1.38 MB, normalized result 1,420,260 B) [measured — `os.path.getsize`].

Area keys, for anyone citing it: `edit-format`, `cache-layout`, `ai-dialect`, `token-cost-tier`, `retrieval-shape`, `degradation-certificate`, `multi-format-ingest`, `provenance-boundary`, `collab-primitives`, `impact-preview`, `model-view`, `integrity`, `normalization-il8n`, `unit-of-exchange`, `missing-constructs`, `adjacent-transplant`, `budget-knapsack`, `governance-and-moat` [primary — parsed from the result JSON].

What it concluded — and the reason its headlines must never be quoted. Its own completeness critic, verbatim:

"18 of 18 area headlines were materially refuted by their own adversarial verifier. Not qualified — refuted. The sentence a reader would remember from every area was wrong, and it was wrong in the flattering direction every time (0.155%→0.0204%, 51,620→17,065, 12/12→11/12, 30x→3.44x, 8,603:1→1,067:1, "zero implementations"→two shipping packages found in one query, "the surface does not exist"→15.1x larger once the excluded construct is counted). That is not a research corpus with some errors in it; that is a measured, program-wide optimism bias with a 100% headline defect rate, and it means the verified residue — not the headlines — is the only asset here." [primary — `wf-mdmax-capability-2026-08-01.result.json`, `synthesis[2].thesis`]

Use `per_area[].verification`, **never** `per_area[].headline`. This is a standing rule for anyone reading that file.

2026-08-01T02:08:39Z (07:38 IST) — the product is settled in one sentence.

"FrontMatter is something which is still under planning. The screenshot that you saw, mostly that will be the UI. In the back, everything, how it would be, how it would behave, we can decide later. Initially, that was me and Amit, our co-founder. **Our plan for FrontMatter, I would say, will be Google Docs for markdown, as simple as that.**" [primary – transcript, verbatim]

This is **D1**. It is the only sentence in the corpus that fixes what the product is, and it arrived after 27 million tokens of research about what it could be.

2026-08-01T02:12:38Z (07:42 IST) — the thinking agent returns. A single deep-reasoning agent, **291,528 subagent tokens · 44 tool uses · 795,308 ms** [primary – the task-notification usage block in the transcript] . Its verdict, verbatim: "*The research produced one genuinely valuable, hand-audited, unclaimed primitive and a great deal of excellent negative knowledge. Both are worth more than the document they are currently trapped in.*"

2026-08-01T02:14:39Z (07:44 IST) — four decisions in one message. The founder answered the open architecture questions directly. Verbatim, lightly segmented:

"AIOS will go in a separate because AIOS is a much bigger concept where we are trying to improve plot code, but definitely front matter or MDMAX will definitely benefit AIOS, that for sure." → **D5**

"The comments and everything will definitely be there, but to see those, you need to use this front matter ... If you download the PDF, what will happen? You will not be able to see the comments, right? ... How will you process whenever you export or copy something? Only the latest edits and all will be visible, nothing else." → **D2**

"Can I review your command without an account? No, he has to log in with front matter auth. It will be mostly Google auth or GitHub auth. Auth is secondary as of now." → **D3**

"if front matter dies, we'll give the full option to give the complete thing for you to migrate and also the Max plan that is there. Anyway, we are giving you offline access." → **D4**

[primary – transcript, verbatim]

2026-08-01T04:41:47Z (10:11 IST) — the 40-agent final gate lands. 40/40 agents, 0 errors, 7,858,804 tokens, 2,173 tool uses, 134 minutes. [primary – the assistant's report in the transcript] Shape: 16 areas + per-area adversarial verification + a **kill audit** (four independent auditors re-deriving the verifiers' kills from primary sources) + 4 synthesis lenses (era-and-vision, product-market, technical, premortem-integrator). Preserved at [docs/engine/research/wf-final-gate-2026-08-01.* (result 1,364,043 B) [measured]].

Area keys: the-era , founder-thesis , container-thesis , markdown-base , custom-pointers , hold-more , rendering-frontier , parsing-robustness , product-gap , moat , market-and-gift , aios-transfer , agentic-docs , competitive-live , against , premortem [primary – result JSON] .

Headline verdicts — 5 CONFIRMED, 10 OVERSTATED, 1 REFUTED [primary – headline_verdicts array] :

VERDICT	AREAS
CONFIRMED (5)	custom-pointers , hold-more , rendering-frontier , product-gap , competitive-live
OVERSTATED (10)	the-era , container-thesis , markdown-base , parsing-robustness , moat , market-and-gift , aios-transfer , agentic-docs , against , premortem
REFUTED (1)	founder-thesis

The improvement from 18/18 refuted to 5/16 confirmed is real and is the result of three specific process changes made between the runs: the corpus was pinned, verifiers were given primary-source access, and a kill audit was added above the verifiers.

The kill audit — how much to trust a kill. Four auditors independently re-derived a sample of the verifiers' kills. Their measured false-kill rates:

AUDITOR	SAMPLED	UPHELD	OVERTURNED	FALSE-KILL RATE
A	29	24	5	17.24%
B	29	28	1	3.4%
C	21	17	4	19.0%
D	26	26	0	0.0% — but 5 of 26 (19.2%) upheld kills carry a supporting figure the auditor could not reproduce

[primary – wf-final-gate-2026-08-01.result.json, kill_audit[].false_kill_rate, quoted exactly]

Auditor A's note states the operating rule: "*The verification layer is roughly 5x more reliable than the research layer it audits (that layer ran a 100% headline defect rate), so it should be kept and trusted — but not trusted blindly.*" **A kill is strong evidence, not proof.** Auditor C explicitly left 8 of 29 kills unaudited "*rather than guess*".

The five findings that survived everything — the CONFIRMED areas, in their own words, quoted exactly:

- custom-pointers : "99.69% of the 4,538 :: directives in Docusaurus's own repo are among its 9 vendor-shipped keywords and all 14 exceptions are the three tutorial strings that teach the feature, and 847 independently-authored npm docs use exactly GitHub's 5 built-in callout types with zero inventions ... MDMAX should ship no declaration syntax, no registry and no library import, and instead ship the inverse — vocabulary inference and drift detection over values that already exist."
- hold-more : "the top invisible carrier (orphan link reference definition) has EXACTLY ZERO uses in the 1,084-file pinned corpus ... 1,431 bare angle-bracket placeholders in 219/1,084 files that GitHub silently DELETES from rendered output, and 13,028 wikilinks GitHub renders as literal junk ... the 'authors reach for raw HTML' premise is refuted at 37/1,084 files (3.4%)."
- rendering-frontier : "the read-only side already works (12/12 unknown fence languages survive byte-intact through frontmatter's own pipeline) — it is blocked by write-back: the one WIRED rendered- editing surface, the properties panel, re-emits YAML through a serializer and changes bytes in 623 of 737 frontmatter-bearing files (84.5%, of which 197 change the document's LINE COUNT) with zero edits made."
- product-gap : "there is exactly one user (isAllowed() compares a login to a single ALLOWED_GH_LOGIN string; firestore() has zero callers; firestore.rules declares a members/roles model and no comments collection), so every review-loop item in the mockups is unreachable, while the one collaboration-adjacent write path that already ships (Publish) round-trips only 3.64% of the pinned corpus byte-identically."
- competitive-live : "the threat is not OKF — it is inkeep/OpenKnowledge (GPL-3.0, 3,239 stars, 14,790 npm dl/wk), which on **2026-07-30** merged a content-derived comment-anchoring system for markdown (exact quote + auto-widened prefix/suffix context, orphan-rather-than-guess ...) that reached the npm BETA channel at 0.46.0-beta.32 on **2026-08-01T02:38:33Z** but is absent from stable 0.45.4; MDMAX §4.1 is therefore no longer an unclaimed capability, only an unclaimed measured one."

[primary – all five quoted from per_area[].headline in the result JSON]

2026-08-01T07:44:38Z (13:14 IST) — the push. The founder: “yes, restore it and push everything / i give you elevated permissions.” [primary – transcript] Two minutes later, 1bd4dad (2026-08-01T13:16:12+05:30), 13 files, **+28,147 / –4 lines** [measured – git log --stat] :

HANDOFF-graph-engineering-research-2026-07-30.md	675 +
HANDOFF-mdmax-markdown-engine-2026-08-01.md	420 +
HANDOFF-mdz-markdown-format-2026-07-29.md	829 +
docs/engine/research/corpus-manifest.json	5,466 +
docs/engine/research/wf-final-gate-*.journal,raw,result}	9,665 +
docs/engine/research/wf-findings-2026-08-01.md	454 +
docs/engine/research/wf-mdmax-capability-*.journal,raw,result}	10,126 +
docs/mdmax/PLAN.md	501 +
src/modules/vault/infrastructure/search-index.ts	15 +/- 3

origin/engine/plan-and-diagnostics now exists at 1bd4dad [measured – git branch –a –v] . **Before this commit, the entire programme existed on one laptop with no remote copy.** That is 19 days from b9ddcfe .

2.2 Every research round, tabulated

#	DATE	ROUND	METHOD	AGENTS	SUBAGENT TOKENS	OUTCOME
R0	2026-07-25	Demand research	11 parallel agents over Reddit / HN Algolia / Product Hunt / review sites / GitHub issues / 20 pricing pages	11	not recorded	92 pain points, 89 feature requests, 20-app matrix → docs/FRONTMATTER-PRODUCT-PLAN.md
R1a	2026-07-28/29	Graph/PKM prior art fan-out	Agent tool, 6 parallel	6	1,164,842	The intersection is empty and was vacated from both sides in 2026
R1b	2026-07-29	Workflow w3f3hmo9k — markdown as container	Workflow tool, 17 agents, ~51 min	17	3,155,872	Base64 dead at 0.90 tok/byte; no new format
R1c	2026-07-29	Workflow w24o7rxuc — the compiler thesis	Workflow tool, 14 agents, ~82 min	14	2,693,995	Compiler thesis supported; mdbase found; the primitive is identity-on-write
R1d	2026-07-29	Academic literature sweep	Agent tool, 11 dispatched, 8 delivered, 3 abandoned	8	1,457,736	Reflexion Models; Aquanet/Formality; Dexter; DocEng; incremental computation
R1e	2026-07-29/30	Graph-engineering contribution	1 session, video + 3 verification searches	—	not recorded	"Anthropic just fixed" is marketing; agent-graph ≠ knowledge-graph
R2a	2026-07-30	Extensibility challenge	5 agents briefed to argue <i>against</i> the prior	5	not recorded	§10 overturned. Fence dispatch is the real extension mechanism
R2b	2026-07-30	Six-area utilization sweep	6 independent deep sweeps	~6	not recorded	§16 (40.1% FORMAT demand) and §17 (NFC/bidi/a11y)
R2c	2026-07-30/31	Second research wave	Agent tool, 7 agents	7	not recorded	Token economics, dual-view, AI-emitted markdown, cache layout, folklore trace
R3	2026-08-01	MDMAX foundations — workflow wf_3b9ab146–77b	25 agents, 736 tool uses, 1,872,350 ms, 0 errors	25	3,059,467	Library model viable but costs you your identity; CommonMark frozen; index not container; no new AI-to-AI format
R4	2026-08-01	MDMAX capability run	18 areas + per-area adversarial verify + 3 synthesis lenses. 2,159 tool uses, 105 min, 0 errors	39	7,864,147	100% headline defect rate. Verified residue is the asset
R5	2026-08-01	Final gate	16 areas + verify + kill audit + 4 lenses. 2,173 tool uses, 134 min, 0 errors	40	7,858,804	5 CONFIRMED / 10 OVERSTATED / 1 REFUTED; false-kill 0–19%
R6	2026-08-01	Deep-reasoning thinking agent	1 agent, 44 tool uses, 795,308 ms	1	291,528	One valuable primitive + a great deal of negative knowledge

Sources for the token columns. R1a–R1d [secondary – HANDOFF-mdz §4] ; R3 [secondary – HANDOFF-mdmax §2.11] ; R4, R5, R6 [primary – read from the completion notifications in transcript 9e5a4a9f] . Rows marked *not recorded* have no token figure in any artifact; do not invent one.

Mechanically-sourced total: 27,546,391 subagent tokens across 150 agents. [measured – 8,472,445 + 3,059,467 + 7,864,147 + 7,858,804 + 291,528 = 27,546,391; 45 + 25 + 39 + 40 + 1 = 150]

Three published totals disagree with each other. Here is the reconciliation.

SOURCE	CLAIM
HANDOFF-mdz §4	"45 completed agents · ~8,472,445 subagent tokens"
docs/engine/PLAN.md frontmatter + README.md II.1	"~14M subagent tokens across ~90 research agents + 20 local experiments"
HANDOFF-mdmax §0.4	"~2.4M tokens of research across ~26 agents + one 25-agent workflow"

These are not three estimates of one thing; they are three different scopes. HANDOFF-mdz counts **Round 1 only**. HANDOFF-mdmax §0.4 counts **Round 2's own agents only** (the ~26 that produced §15–§17 and the second wave), stated *separately from* the 25-agent workflow. PLAN.md 's ~14M is the **cumulative through the 25-agent workflow**: 8,472,445 + ~2,400,000 + 3,059,467 ≈ **13,931,912**, which rounds to ~14M, and 45 + ~26 + 25 = 96, which rounds to ~90. [inference – the arithmetic is mine; no artifact states this reconciliation] The three are consistent once scoped, and the apparent contradiction is a labelling failure, not a counting one.

But HANDOFF-mdmax §0.4 does contradict its own §2.11: §0.4 says "~2.4M tokens ... across ~26 agents + one 25-agent workflow" while §2.11 reports that workflow alone at 3,059,467 tokens. If §0.4 is read as an inclusive total it is wrong by more than 600,000 tokens. §2.11 governs — it quotes the completion notification.

Cumulative through 2026-08-01: ≈ 29.9M subagent tokens. [measured — 13,931,912 + 7,864,147 + 7,858,804 + 291,528 = 29,946,391] The ~2,400,000 term is the only soft input; every other term is quoted from a completion notification.

2.3 Every name we tried, and what killed it

Five names, four dead. The pattern is identical in every case and it is worth stating before the table: **none of them died on availability. Every one died on occupancy** — the word was free, the slot was taken.

MD3 — died 2026-07-29

- 142 of 142 npm packages matching md3 are Material Design 3. [secondary — HANDOFF-mdz §3.5]
- 5,611 GitHub repositories carry the name.
- .md3 is already the Quake III model file extension.
- And the disqualifier nobody expected: it collides with the founder's own global Learned Rule #52, which mandates Google Material Symbols — the Material Design 3 icon set — as the only sanctioned icon language across every surface he builds. [primary — ~/.claude/CLAUDE.md, Learned Rule 52]

The founder had asked whether to buy md3.in for \$20 / 3 years. It was verified AVAILABLE via the authoritative RDAP bootstrap (a first attempt returned 000, a connection failure, and was re-run rather than reported as an answer) [measured — that session ran it]. It was not bought.

MDZ ("MD Zephyrus") — died 2026-07-29

- As a word it is clean: 1,610 GitHub repos, but 54% are personal initials — scattered noise, not a brand.
- As a format slot it is occupied. .mdz is already the established extension for a compressed markdown package, with four live projects in exactly that slot: wflixu/mdz ("Markdown Zip"), mdz-format/mdz, mdzip-project/mdzip-spec, and npm @mdzip/editor at 582 downloads/week.
- All four .mdz TLDs are taken (.dev / .in / .ai / .org).
- npm mdz returns HTTP 200 but has zero versions and no description — an unpublished placeholder last modified 2026-05-18.

[secondary — HANDOFF-mdz §3.5; the npm and RDAP checks are [measured] by that session]

The verdict recorded at the time: "Clean as a word, polluted as a format — precisely backwards."

mdmap / MDMAP — never a product name; retired as a concept

mdmap was the founder's own first coinage (2026-07-28T21:05:46Z, verbatim above). It named a deliverable, not a product: docs/mdmap/, 21 files, 16,906 words [measured]. It was retired because the concept it named was measured out of existence on the founder's own vault:

median out-degree	ZERO	mean 2.27, p90 5
597 files (55.1%)	zero outbound resolved edges	
502 files (46.3%)	FULLY ISOLATED — no in, no out	
511 connected components;	the largest holds 40.1%	
transclusion ![[...]]	45 occurrences in 25.5 MB — all 45 are syntax documentation	
	genuine content transclusion: ZERO	

[measured, corpus_id sha256:3a010b16...; primary — docs/mdmax/PLAN.md §3.2, re-derived and matching]

"A single markdown connected to hundreds of markdowns, and those hundreds connected to hundreds each" is the opposite of this corpus. Four independent agents plus the synthesizer agreed. The directory survives, committed in 1bd4dad, as a record. The graph view was cut with it: zero occurrences of "graph" across 89 sourced feature requests and zero in the round-2 corpus.

docs/mdmap/ also earned its keep in an unintended way. Its own root MAP.md declares commit: 8eb4de2 and states "stale: 0 — built at commit 8eb4de2, which is HEAD." HEAD was 798ebbf, 16 commits later. A document asserting a fact about the repository that the repository contradicts, occurring inside the artifact that proposes the reconciler. [measured — that session compared the frontmatter to git rev-parse HEAD] That is the product's best free demo and it happened twice.

markedmax — proposed and killed on 2026-07-31, inside two minutes

The founder, 2026-07-31T23:44:47Z (2026-08-01 05:14 IST), verbatim:

"markedmax hwo about this name for this md enhancmenet ? or you suggest something else" [primary — transcript 9e5a4a9f]

Availability was checked live and was completely clean:

npm	markedmax · marked-max · markmax · mark-max · mdmax · md-max · maxmd	
	markdownmax · markdown-max	ALL FREE
PyPI	markedmax · marked-max · markmax · mdmax	ALL FREE
GitHub repos named "markedmax"		0

[measured — that session ran curl against registry.npmjs.org and pypi.org for each candidate; raw tool output in the transcript]

It was rejected anyway, on three grounds:

- marked is a markdown parser with 61,032,058 downloads per week [measured — api.npmjs.org last-week point query, run in that session] — and it is one of the exact four toolchain families the plan says MDMAX must ship ports for (remark/rehype, markdown-it, marked, markdig). Naming the format after one of your four ports asserts that you are a fork of it. You are not: the stack is micromark/remark plus @lezer/markdown.

2. **"marked" is not "markdown."** It is a past participle and it is a third party's brand for a *renderer*. The founder's own coinage was *"this is markdown maxing"* — the subject of that sentence is markdown. `markedmax` drops the subject.
3. **It needs a disambiguation sentence every time it is spoken.** "em-dee-max" is three syllables and works as a CLI verb (`mdmax build --index`).

MDMAX — kept, and its one real weakness named

Verified live 2026-08-01 `[measured - HANDOFF-mdmax $9, curl against both registries]` :

```
npm      mdmax · md-max · markdown-max · mdx-max    → ALL FREE
PyPI     mdmax · markdown-max                      → FREE
GitHub   repos named mdmax: 13, all 0-star noise
         github.com/mdmax → taken (dormant personal account)
```

Only footnote: "MD" means molecular dynamics in chemistry, so `MDmax` appears in one unrelated physics repository.

The weakness, stated rather than hidden: "Max" implies *more*, and the programme's central finding is that more is worse. Structure added without compression costs **−6.6 to −15.7 points**; padding a table costs **30–35% for zero semantic gain**. The honest product claim is *fewer tokens, one parse strategy, declared provenance* — explicitly **not** better comprehension. A name meaning "maximum" fights that.

It survives because the founder's own framing resolves it: **MAX = maximal utilisation of what markdown already is**, not more syntax. *"we can't invent a new markdown, we can't invent a new format, but we definitely can invent a new style of utilization"* `[primary - transcript, 2026-07-31T20:43:37Z]` .

Two naming slots, both filled, and conflating them is what made this wobble four times:

SLOT	THE THING	NAME
the product	the editor plus the compiler — the thing people run	<code>frontmatter</code> — <i>"the word names the primitive the thesis rests on"</i>
the capability	the container, the representations, the CLI, the certificate	MDMAX

Same shape as Quarto and `.qmd` . This is **D10**.

2.4 Every idea we abandoned, and the specific measurement that killed it

Nothing in this table may be re-opened without a new measurement that beats the one in the right-hand column. That is the whole purpose of the table.

#	IDEA	THE MEASUREMENT THAT KILLED IT	TIER	WHEN
1	Base64 binary inside markdown	0.90 tok/byte , five independent measurements across two tokenizer families (own run 0.70–0.87 o200k; three agents 0.8966–0.9198, 0.9101, 0.920; adversarial 0.9033–0.9104). A 200 KB JPEG = 93.1% of a 200K window ; a real Jupyter notebook is 98.3% base64 by tokens = 227,352 tokens = 114% of a 200K window . Also: CommonMark's reference implementation silently empties <code>data:</code> URIs to <code>src=""</code> , and line-wrapping base64 destroys the image syntax outright	[measured] + [secondary]	2026-07-29
2	Polyglot ZIP tail (a real ZIP archive appended to a <code>.md</code>)	UTF-8 round trip produced 82,957 U+FFFD replacement characters and +81% bytes ; a <code>.gitattributes</code> <code>*.md</code> text rule corrupted the archive in one checkin (<code>unzip -t : bad zipfile offset</code>); git binary-detects below an 8,000-byte head and line-merges above it	[secondary – PLAN.md §10]	2026-07-29
3	A new dialect requiring its own parser	MDX 3.07% of <code>.md</code> volume (1 in 33) with Vercel behind it; Markdoc 0.136% (1 in 736) with Stripe behind it; Obsidian's own <code>.base</code> 0.028% ; djot, by CommonMark's own author, 52,610x behind remark-parse . Against: <code>AGENTS.md</code> , with no schema at all, at 175,424 files on GitHub	[secondary]	2026-07-29
4	MDX as the base	MDX cannot write the document back out . Verified directly: the MDX writer emits <code>{1 + 1}</code> from a program whose parsed body is empty — it follows the raw string and ignores the AST. The moment the engine needs an interpreter, it is MDX, and MDX has no <code>put</code>	[primary – the writer was executed]	2026-07-30
5	A runtime / execution layer (WebContainers)	<code>@webcontainer/api@1.6.4</code> is MIT but is a 176.5 KB postMessage client that boots an iframe at <code>https://stackblitz.com/headless</code> ; StackBlitz ToS caps paid plans at 500 sessions/month . The real veto is COOP/COEP : it requires <code>Cross-Origin-Embedder-Policy: require-corp</code> page-wide , which breaks GitHub avatars and OAuth popups. Sandpack is dead (last commit 2025-02-14) and its <code>@codesandbox/nodebox@0.1.8</code> ships the non-commercial Sustainable Use License that scanners miss	[secondary – HANDOFF–mdz §3.10]	2026-07-29
6	Runtime cross-file reactivity	It shipped and was removed by the people who invented reactive notebooks : Observable Framework lets only pages declare reactive variables; cross-page sharing is plain non-reactive ES modules. marimo allows only top-level functions/classes; Pluto has none. Excel is the counter-precedent — cross-workbook links are its most notorious failure, and human inspection catches only ~60% of spreadsheet errors (Panko). Take the linking, refuse the reactivity	[secondary – HANDOFF–mdz §3.12]	2026-07-29
7	Changing CommonMark	Zero net new conformance examples between 0.30 (2021-06-20) and 0.31.2 (2024-01-28); 81 of 1,848 commits since 2020, 44% by one person ; no <code>CONTRIBUTING</code> , no <code>GOVERNANCE</code> ; the 1.0 blocker list opened 2015 still says "[6 remaining]", last edited 2019; the two extension threads MDMAX would need have run 316 posts over a decade , still open; two IETF principals offered to formalise the spec with no maintainer reply in four years ; its own author built djot in 2022 rather than amend it. Re-derived independently in the final gate: spec commits 2019=48, 2020=15, 2021=14, 2022=15, 2023=12, 2024=12, 2025=7, 2026=6 (partial)	[secondary] + [measured – the kill auditor re–ran the GitHub API]	2026-08-01
8	The graph view	Median out-degree ZERO on the pinned corpus; 46.3% of files fully isolated ; 511 connected components ; zero genuine content transclusions in 25.5 MB . Plus zero occurrences of "graph" across 89 sourced feature requests . And the demand-side kill-shot from r/ObsidianMD: <i>"you make the links yourself, so the graph only ever shows you what you already know. it can't surprise you."</i>	[measured, corpus_id sha256:3a010b16_]	2026-08-01
9	The notation library / vocabulary registry / declared-vocabulary syntax	Measured user-invention rate 0.31% , and all of it is tutorial content teaching the feature . Concretely: 99.69% of the 4,538 :::: directives in Docusaurus's own repo are among its 9 vendor-shipped keywords, and all 14 exceptions are the three tutorial strings; 847 independently-authored npm docs use exactly GitHub's 5 built-in callout types with zero inventions . And where a spec <i>forced</i> declaration, it produced drift, not vocabulary: four real OKF bundles gave 5/0/5/21 <code>type</code> values, ≥8 of the 21 in mutual collision, four incompatible naming conventions, and 0 of 4 declaring <code>okf_version</code>	[primary + measured – final gate custom-pointers, CONFIRMED]	2026-08-01
10	The doc-tree folder compiler and its incremental engine	Cold full build ~1 second ; maximum transitive blast radius 54 of 1,084 files (5%) . There is no separate-compilation problem to solve	[measured – final gate technical lens]	2026-08-01
11	The "mother markdown" — one file containing many, split on read	Container 25,548,765 B vs a one-line-per-document index at 105,538 B → 242x . At 3.6 bytes/token a 200K window holds 2.82% of the container and 682% of the index ; even a 1M window holds 14.1% ; mean file 23,569 B, so a 200K pack carries about thirty average files . And the round trip is not unclaimed: <code>llm-code-format</code> already ships pack and unpack with zero dependencies at 387 downloads/month against repomix's 327,543 — 846:1 . The Obsidian forum thread asking to export a vault to a single file: 9,010 views, 1 like, 3 posts in 26 months	[measured, corpus_id sha256:3a010b16_] + [primary]	2026-08-01
12	In-band instruction/data separation as a prompt-injection defence	A <code>```</code> fence around untrusted data moved attack success rate 51% → 50% ; the sandwich defence made it worse, 55% . Tool filtering — an architectural control, not an in-band one — got 6.8% . Spotlighting: 1% ASR static → >95% adaptive	[secondary]	2026-07-30
13	Typographic steganography (hiding identity in list markers, emphasis delimiters, setext headings)	One <code>remark-stringify</code> pass normalizes all of them . Dead on first contact with any tool in the ecosystem	[secondary – PLAN.md §10]	2026-07-30
14	Content-hash-only block identity	Hammer et al., OOPSLA 2015: structural matching " <i>recomputes and reallocates a linear number of output elements for each O(1) input change</i> ". Θ(n) per edit vs O(1)/O(log n) for names	[secondary]	2026-07-29
15	Demanding that users declare schemas or typed links	Settled 1992 (Aquanet's own postmortem) and generalised 1999 (<i>Formality Considered Harmful</i> , Shipman & Marshall, CSCW 8(4)). Four named reasons: cognitive overhead, tacit knowledge, premature structure, situational structure	[secondary]	2026-07-29
16	A large typed-link vocabulary	Nine is the measured ceiling , from gIBIS's deployed 32-user / 2,091-node field trial (TOIS 6(4) 1988)	[secondary]	2026-07-29
17	"Markdown is ~40% fewer tokens than HTML"	Apples-to-apples — markdown source against <i>its own</i> rendered HTML — is 9.1% aggregate (4.9–23.7% per document). The 78.5–97.2% figures use raw served production HTML that is 52–81% div soup as the denominator. A hostile fact-check found one agent's <i>"41.8% — safe to publish"</i> verdict wrong	[measured – re-computation]	2026-07-29
18	"Markdown is easier for AI because it's in the training data"	Folklore, traced to origin . MarkItDown's README says <i>"This suggests..."</i> and cites nothing; it was added 2025-03-06, nine months after the claim was already circulating . And Llama 3's paper states the opposite verbatim: <i>"We find markdown is harmful to the performance of a model that is primarily trained on web data compared to plain text, so we remove all markdown markers."</i>	[primary – both sources read]	2026-07-30
19	MDMAX as a business model	It is a library subordinate to the editor. Base rate for the alternative: markdownlint ran 11 years 4 months to 2.89M downloads/week and zero dollars	[primary – final gate product–market lens]	2026-08-01

#	IDEA	THE MEASUREMENT THAT KILLED IT	TIER	WHEN
20	Further research fan-out of any kind	Cut by the programme itself, until one committed assertion goes red to green. Justification: 0 lines of engine code, 19 days after the first commit, with 12 of 20 commits being docs	[measured – final gate critic lens + git log]	2026-08-01

2.5 Every time research overturned our own position

A plan that never contradicts itself was never tested. There are **nine** recorded reversals, not five. The first five are docs/engine/README.md §II.5 verbatim; four more came from the two 2026-08-01 runs.

From README.md §II.5 (as of 2026-07-30):

- 1. "No new extension mechanism" → **refuted**. Fence dispatch is real, permissionless and validated across 30 uncoordinated platforms. §10 had called a positive result negative. (Triggered by the founder's push-back, §2.1 Round 2.)
- 2. "Rigid context fingerprint, k=2" → **refuted by our own sweep**. Rigid fingerprints get worse as k grows: k=1/2/3 gives 90.65% / 88.68% / 87.22% correct. **Score context softly**.
- 3. "Markdown merges worse than code" → **refuted**. All lines unique: markdown 64.6–68.4% vs TypeScript 70.6%. **Non-blank lines unique: markdown 84.8–94.2% vs TypeScript 78.1%**. Excluding blank lines markdown beats TypeScript; blank lines are the entire deficit.
- 4. The "entropy floor" explanation for base64 token cost → **refuted**. The number (0.90 tok/byte) is right; the mechanism was wrong.
- 5. "Redefining a symbol is safe if declared" → **refined within an hour of being written**. Redefining a **denotation** is safe (140/140). Redefining a **procedure** is not: GPT-4 goes **98.2% base-10 → 38.6% base-9 with the rule stated in the prompt**.

[primary – docs/engine/README.md §II.5]

Four more, from the 2026-08-01 runs:

- 6. "Content-derived re-anchoring is used by no system above, and is therefore the one with no precedent to lean on" → **FALSE**. There is 25 years of prior art. [primary]
 - **Brush & Bargeron, "Robustly Anchoring Annotations Using Keywords", MSR-TR-2001-107** — title verified live at microsoft.com/en-us/research. Its anchor is **anchor text + start/end context + lowest-document-frequency keywords**. That is our design. Its user study concluded low-confidence matches should be orphaned with a best guess — **that is our AMBIGUOUS/DELTA rule**.
 - **US7747943B2** (Bargeron/Brush/Gupta, Microsoft, filed 2001, granted 2010, expired 2024-03-26).
 - **W3C Web Annotation Data Model** — a Recommendation, with TextQuoteSelector.
 - **Hypothes.is** in production with published failure data: **about 22% of 20,953 annotations can no longer attach** [secondary – arXiv 1512.06195, NOT read directly; verify before citing].
 - Grep of both plan documents at the time: Brush 0 · Bargeron 0 · Web Annotation 0 · TextQuote 0 · Hypothes 0 · diff-match-patch 0 · robust anchor 0 [measured].
 - **And the process failure is worse than the error**: the 2026-07-29 research run **physically downloaded** w3c-anno.html — 284,185 bytes. <title>Web Annotation Data Model</title> — into /tmp/claude-501, and none of it reached the plan. **Retrieval-to-synthesis loss, not a search failure. The source was in hand**.
 - **Converted into an asset**: implement TextQuoteSelector and Keyword Anchoring as two more baselines in benchmark B1. Beating a W3C Recommendation and a Microsoft patent on a published corpus is a stronger claim than "nobody tried this," and it is the only version that survives review.
- 7. §16.2's Logseq "causal proof" → **over-generalised; retracted**. The plan claimed "every cluster corresponding to a native format feature is 10–35× smaller" — and never reported the denominator. Recomputed live 2026-08-01: Obsidian 6,222 plugins, Logseq 607 packages → **baseline ratio 10.25×**:

CLUSTER	RAW RATIO	+ BASELINE	VERDICT
spaced repetition	35.00×	3.41×	REAL (Logseq native)
query/database	2.30×	0.22×	INVERTS (Logseq native)
task management	4.97×	0.48×	INVERTS (Logseq native)
kanban (control)	10.50×	1.02×	no effect
tables (control)	10.67×	1.04×	no effect

Only 1 of 3 native-feature clusters shows an effect; two go the wrong way. The two controls land at 1.02× and 1.04×, so the method is sound and the conclusion was over-generalised from the single case that worked. §16.2 was the only **causal** leg under a **correlational** 40.1%. [measured – HANDOFF-mdmax §6.1]

- 8. "He already hand-builds bundles across 37.2% of files" (the container's entire empirical case) → **REFUTED**, and it is the single worst failure in the record because a pre-registered falsifier fired and was recorded as passing. Re-measured over corpus_id sha256:3a010b16...: md/Skills/** alone is **369 files (34.04%)** and 11,115,842 B (43.51%) — **92.3% of the claimed share is one machine-generated sync tree. 340 of 340 files carrying source: claude sit inside md/Skills/****; 368 carry last_synced: 2026-05-26. Honest figure ≈ 2.2%. The area's own falsifier fired and the area recorded "FAILED TO REFUTE, and in the founder's favour." [measured – kill auditor's independent re-derivation, final gate]
- 9. "The demand evidence favours real-time collaboration over commenting 6.9:1" → **REVERSED**. It is 1.78:1 the other way. The 6.9:1 was a regex artifact from asymmetric counting; symmetric counting over the founders' own scraped demand corpus gives **review-loop 169 leaves to collaboration 95**. The final gate's own verdict lens: "That single correction is the difference between 'build Yjs' and 'build comments.'" [primary – final gate premortem-integrator lens]

A tenth, which is a design reversal rather than a factual one, and which matters for anything already built on it: the container's carrier flipped. PLAN.md §5 and README.md II.2 record the HTML comment as the delimiter, chosen by a specific measurement ("with no blank line before it, an HTML comment interrupts a paragraph and the boundary survives; a link reference definition is swallowed and the boundary is silently lost"), at **+0.99% overhead, 6/6 files recovered byte-identical**. Two later measurements broke it:

- **It silently loses a boundary whenever a contained file ends inside an open fence** — 4 of 1,084 pinned files (**0.369%**), which under an i.i.d. survival curve means **98.2% of vault-scale packs fail**. [measured – final gate container-thesis; the kill auditor reproduced the exact four files, all in md/Brain/20-projects/]]
- **GitHub deletes HTML comments in both rendering modes**, and the degradation-certificate area measured the HTML comment carrier **visible in 2 of 12 outputs and payload-destroyed in 10**. [measured – capability run degradation-certificate]

The replacement, decided: a backtick fence with run $N = \max(4, \max \text{line-initial backtick run over every contained byte} + 1)$, declared in the manifest and **re-derived and cross-checked on read**. Prior art is repomix's `calculateMarkdownDelimiter`; adopt it and credit it. Note the algorithms differ on this exact corpus: repomix matches backtick runs **anywhere** ($\max \text{any-position run} = 4 \rightarrow \text{emits } 5$) while the line-initial rule emits 4 [measured – kill auditor's own histogram].

Five designs in the capability run were built on the HTML-comment carrier after it had already been measured broken. That is failure #6 in §2.6 and the reason for process rule P3.

2.6 The method failures

These are stated plainly because the alternative — quietly fixing them — is how the programme got a 100% headline defect rate in the first place. Each is followed by the rule adopted in response.

1. A 100% headline defect rate, one-directional. All **18 of 18** area headlines in the 39-agent capability run were materially refuted by their own adversarial verifiers, **always in the flattering direction**: $0.155\% \rightarrow 0.0204\%$, $51,620 \rightarrow 17,065$, $12/12 \rightarrow 11/12$, $30x \rightarrow 3.44x$, $8,603:1 \rightarrow 1,067:1$, “zero implementations” $\rightarrow$ two shipping packages found in one query, “the surface does not exist” $\rightarrow 15.1x$ larger once the excluded construct is counted. [primary – capability run synthesis[2].thesis] **A 100% one-directional defect rate is a reward function, not noise.** $\rightarrow$ Rule: `read per_area[].verification, never per_area[].headline`. The verified residue is the asset.

2. Seven irreproducible corpus counts. Before the corpus was pinned, the 18 areas variously reported the same vault as **4,312 · 4,317 · 4,710 · 4,823 · 5,981 · 5,986 · 2,586** files, and the critic measured **5,563 / 3,245 / 1,989 / 756 / 4,131** depending on exclusion policy — “and not one of those seven numbers is reproducible under any policy I could construct, on a corpus that provenance-boundary measured drifting mid-session. Every rate in this program divides an unstated numerator by an unreproducible denominator.” [primary – capability run critic lens, verbatim] $\rightarrow$ Rule P1: every figure names a `corpus_id`. The manifest was written and committed the same day. **Any figure in the older documents that does not cite sha256:3a010b16... is suspect; say so rather than repeating it.**

3. Five designs built on a carrier two measurements had already broken. Documented in §2.5's tenth item. The parallel fan-out had no reconciliation gate, so five areas each independently specified an HTML-comment region primitive while two other areas were measuring it destroyed. [primary – docs/mdmax/PLAN.md §9 item 6] $\rightarrow$ Rule P3: parallel work only over disjoint artifacts, through one reconciliation gate.

4. Zero live model calls. After 18 areas explicitly serving the goals “easier for AI to consume” and “trivially understandable to any AI”, **not one live model call was made against any representation by any area**. [primary – capability run critic lens] **Every “easier for AI” claim in this entire corpus — roughly 1.09 MB of documentation — is a pre-registered prediction, not a result.** $\rightarrow$ Rule P9: a replayed number says `SIMULATED` in the same sentence.

5. The flagship number has been re-derived by nobody. The **99.627% correct / 0.050% false / 0.323% refusal** anchoring result over 41,642 block-versions from 294 revision pairs is the strongest claim in the programme and **every area that quoted it quoted it; none re-ran it**. [primary – docs/mdmax/PLAN.md §3.3] An adversarial baseline built during the final gate — a **30-line LCS diff** over the same normalised block text — lands at **86.658% resolved with 77.297 percentage points provably unambiguous**, and *additionally eliminates the entire ambiguous-duplicate class (9.727% of block-versions) that the plan's S2 soft-context scorer exists to handle*; one similarity gap-fill pass takes it to **90.17%**. [measured – final gate, `$TMPDIR/mdxopp/diffarm.mjs`, output quoted exactly: `{"lcs_matched":25319,"pct_matched":86.658,"lcs_matched_on_UNIQUE_text_cannot_be_wrong":22584,"pct_of_total":77.297}`] **Always caveat the 99.627% figure. Do not publish it before it is independently re-derived, and publish the LCS baseline next to it when you do.**

6. package.json was destroyed to 17 bytes by a research subagent. A subagent ran `npm install` from the repository root instead of `$TMPDIR`, against an explicit “no writes outside `$TMPDIR`” guard. The working-tree file was reduced to `{"type":"module"}` — **17 bytes against the committed 3,076 bytes with 47 dependencies and 18 scripts**. `npm run verify` could not run. **The repository was unbuildable for an unknown period, on a branch with no upstream, with the entire plan untracked.** Restored 2026-08-01 under an explicit RULE-2 confirmation. [measured – the restore was performed and the byte counts stated in the transcript; primary – docs/mdmax/PLAN.md §9 item 10] This is the same class as `arx.xml` (14 bytes, contents: `Rate exceeded.`, an arXiv throttle response) and `cx.html` (134,860 bytes) being written into the repository root by agents in Round 1. **Prompt-level write guards are advisory, not enforced.**

7. The entire programme lived untracked on one laptop until 2026-08-01. `docs/mdmax/`, `docs/engine/research/`, all three handoffs, and the corpus manifest were `??` in `git status` on a branch **16 commits ahead of an unpushed** `origin/main` at `8eb4de2`. The final gate's own words: “the entire plan and the pinned corpus manifest exist on one laptop only ... the only item whose failure mode is total and irreversible.” Fixed by `1bd4dad` and the first push, **19 days after** `b9ddcfe`. [measured – `git log`, `git branch -a -v`] $\rightarrow$ Rule P11: push before you think.

8. Verifier false-kill rates of 0–19%. §2.1's kill-audit table. A kill is strong evidence, not proof; one auditor left 8 of 29 unaudited rather than guess. $\rightarrow$ Rule P4: verifier kills get sampled and re-audited before anything is removed.

9. Two agents returned status notes instead of findings. `a3822aff54512c4cb` in Round 1 (**129,126 tokens wasted**) and the 25-agent workflow's synthesis agent in Round 3 (“Reconciliation clean. No mutations or commits by this subagent. Deliverable delivered.”). **Both were free-text returns. Schema-forced agents did not exhibit this.** $\rightarrow$ Rule: schema-force every agent whose output is load-bearing.

10. Two tooling guards report false failures. `workflow-lint.sh` inverts on files larger than the pipe buffer (`grep -q plus pipefail` $\rightarrow$ SIGPIPE 141), and the session loop-guard reported denials for calls that had succeeded. [primary – docs/mdmax/PLAN.md §9 item 9; the loop-guard misfire recurred while this section was being written] **A harness that reports false failures trains you to ignore it, which is the same disease as one that reports false passes.**

11. Two session guards blocked legitimate work. The prompt-injection taint gate fired twice (20:45:43Z and 20:47:06Z on 2026-07-30) and blocked `WebFetch` **for all subagents for the rest of the session** — it is session-scoped, so its own advertised remedy (“use a read-only subagent”) does not work. Likely trigger: one research topic was prompt injection, so the fetched pages legitimately contained “**IGNORE ALL PREVIOUS INSTRUCTIONS.**” Separately, the bash guard blocked a `git commit` because the commit **message** contained the word “drop” (matching its `DROP` TABLE rule) — it scans heredoc prose as if it were a command. [primary – HANDOFF-mdmax §10]

12. The document became the product. `PLAN.md` reached v0.6.0 with a changelog and a supersession chain; three of the four most recent commits before the consolidation were **cosmetic PDF fixes**. [measured – `git log`] The programme's own diagnosis: “Honesty was being converted into inertia.” And the sentence one research area wrote about itself, which is true of more of this programme than one area: “this entire area was a code review wearing a research costume and should be re-run as one.” [primary – docs/mdmax/PLAN.md §9]

2.7 What this section does not cover, and what would falsify it

Not covered:

- **The content of the research**, beyond what is needed to explain a decision. Sections 3 onward carry the capability specification, the engine design, and the build sequence. This section carries only the provenance.
- **The AIOS track**. It is a separate programme (D5). The final gate's `aios-transfer` area is **OVERSTATED** and nothing from it is load-bearing here.
- **The graph-engineering session's own transcript**, which was launched from a different working directory and is not in `~/claude/projects/Users-sagnikmitra-Desktop-GitHub-frontmatter/`. Only its written handoff is available. Anything attributed to it is `[secondary]` at best.
- **Token counts for R1e, R2a, R2b and R2c**. No artifact records them. They are inside the "~26 agents" bucket and cannot be separated.
- **The three Round-1 literature agents that were still in flight when its handoff was written** (`a7c9d985624ab0386` end-user programming, `a6934717fa9769bee` LLM document representation 2025-26, `afaf2a3a55a66d46b` type theory / gradual typing / CUE lattices). **Their findings were never collected and are not in any document**. The first could still change a recommendation: if Blackwell's attention-investment model says what §3.14 of that handoff predicts, the type layer must be **inferred and proposed, never declared** — which is where the `custom-pointers` area independently landed on 2026-08-01 anyway.
- **The ~50 screenshots** the founder supplied on 2026-07-31. They were read into one session's context and are not preserved as files. Any claim sourced only to them is unrecoverable.

What would falsify this section:

1. **A commit, transcript, or artifact dated before 2026-07-13T16:33:44+05:30**. The timeline claims nothing existed before `b9ddcfe`. One artifact would refute it.
 2. **A token figure in a completion notification that differs from the table in §2.2**. Every number there is quoted from one; if a journal file disagrees with the notification, the journal governs and this section is wrong.
 3. **An independent re-derivation of the 99.627% anchoring result**. If someone re-runs it and gets a materially different number, §2.6 item 5 becomes the most important paragraph in this plan rather than a caveat.
 4. **Evidence that any of the four dead names is actually available in the slot as well as the word** — for instance, if the four `.mdz` projects are dead and the extension is genuinely free. That would not resurrect MDZ (MDMAX is settled) but it would mean the kill reasoning was weaker than stated.
 5. **A live model call showing markdown is easier for a model to consume than the alternatives**. §2.6 item 4 says every such claim is a prediction. One measurement converts it into a result — or kills it.
 6. **A user**. Every demand claim in this record is inferred from download counts, forum threads, plugin ecosystems and issue trackers. **Zero people outside the two founders have used this product**. The `product-gap` area found exactly one hardcoded login. The first real second user will falsify or confirm more of this record in a week than 29.9 million tokens did in nineteen days.
-

53 Capabilities

What MDMAX is: every capability, specified

← Index · ← §2 Chronology · §4 Representation →

3. What MDMAX is: every capability, specified

MDMAX is the capability layer underneath frontmatter . It is a library that lives in src/modules , is subordinate to the editor, and is deleted the moment the editor stops needing it. It is not a compiler programme, not a format, not a spec, and not a standalone product. It has exactly one public artifact (mdmax cert , §3.3) and one public promise (splice-only writing, §3.1).

This section specifies seven capabilities. For each one it states what it does, why it exists, the measurement that justifies it, how it works mechanically, its command-line or programming interface, how it fails, and the single observation that would kill it. Nothing here is optional reading — a capability whose kill condition is not written down is a capability nobody can stop.

3.0 How to read the numbers in this section

Every figure about our own files carries a corpus id. There is exactly one:

```
corpus_id sha256:3a010b1649899795d79274fc528dbece97fdabf4ff0f81cc02ab619c048c51a4
files      1,084
bytes      25,548,765
roots      md          @02c22ec47a3a73fb25a14b11be233ad3d00a8b48    756 files    21,068,853 B
           knowledge @464eb666946c8cb4ccf42c68f4464613e5fc4999    272 files    3,570,923 B
           frontmatter @798ebbf3250a5b2a53785859d79ae15f55ebebeb    56 files     908,989 B
policy     all *.md case-insensitive; skip .git .next .obsidian .trash .venv __pycache__
           build dist node_modules at any depth; skip any dot-directory;
           skip top-level Mirrors/ and Paste/ per root
manifest    docs/engine/research/corpus-manifest.json
```

[primary – read from the manifest file]

Three honest caveats on that corpus, stated once here so they are not repeated seven times.

- 1. **The corpus id itself is unverifiable today.** The kill audit tried three plausible constructions over the sorted (path, sha256) list and got 1a00525b... , 561187d3... and 5aad2ccb... — none matching. No derivation script exists in the repo (grep -rL corpus_id over scripts/ and docs/ finds only the manifest). **The file list and all 1,084 sha256 values were independently re-verified with 0 mismatches**, so the data is sound and only the label is uncheckable. **Fix, half an hour: commit the ~10-line script that derives the id.** [measured, kill audit]
- 2. **The corpus is contaminated and every corpus-wide percentage inherits it.** The two largest files are md/_Archive/Satinath Content JSON (raw source).md at 1,491,757 B — verified by reading it, it is JSON in a .md wrapper — and md/graphify-out/GRAPH_REPORT.md at 1,466,025 B, a generated report. Together **11.58%** of the corpus bytes. graphify-out contributes 1,607,862 B = **6.29%**. Four files are 0 bytes. [measured]
- 3. **It double-counts across roots.** 28 non-empty cross-root duplicate sha256 groups, 29 redundant copies, 440,139 bytes = **1.72%** (e.g. docs/research/frontmatter-pain-playbook.md at 93,259 B appears in both the md and frontmatter roots). [measured]

Claim tiers: [measured] we ran it · [primary] we read the source · [secondary] · [inference] · [SIMULATED] replayed through code, not read from a live system.

Zero live model calls were made in any of the three research runs. The one live-model number in this section is external (§3.5, the OKF benchmark). Every other "easier for AI" statement is a prediction.

3.0.1 The seven capabilities at a glance

#	CAPABILITY	STATUS	EFFORT	DEPENDS ON
1	The splice writer	DECIDED	0.5–1 day	nothing
2	Block identity / content-derived re-anchoring	DECIDED as tier 3 of a ladder; tier-3 ship date OPEN	tiers 1–2: 2–3 days · tier 3: 2–3 weeks incl. benchmark [inference]	§3.8 offset model, §3.8 normalize/1
3	The degradation certificate — mdmax cert	DECIDED to ship; scope RECOMMENDED	2 days hard cap (plan) vs ~3 weeks (research design) — contradiction resolved in §3.3.8	nothing; parse only
4	The near-miss reference checker	RECOMMENDED	symbol table 2–3 weeks · classifier ~1 day of code plus a labelled set	§3.8 slug/1 pinned
5	Budgeted projection (index / outline / full)	index + outline RECOMMENDED; the budget knapsack OPEN	fence-aware split/join ~2 days · knapsack unscoped	§3.4 symbol table
6	Provenance	DECIDED — emit OKF v0.2 §5; per-span method/confidence OPEN	1–2 days for read + emit	§3.3 (cert carries the columns)
7	The container — pack / unpack	RECOMMENDED, SEQUENCED after 1 and 2	high	§3.1 splice, §3.2 identity

3.1 Capability 1 — THE SPLICE WRITER

3.1.1 What it does

One function is permitted to write bytes into a user's .md file. It replaces a byte range and copies everything else verbatim. It never re-serialises a document from a syntax tree.

3.1.2 Why it exists

Because regenerating markdown from an AST is **mathematically excluded**, not merely lossy. Foster, Greenwald, Moore, Pierce & Schmitt, *Combinators for bidirectional tree transformations*, ACM TOPLAS 29(3), 2007, **Lemma 3.9**: a total well-behaved lens whose `put` ignores the original source requires `get` to be a bijection. "Regenerate from the AST" is definitionally such a lens, and `get` is nowhere near injective: **443 of 655 CommonMark spec examples are two distinct source strings with a byte-identical AST** `[measured, docs/engine/PLAN.md §1.2]`. This is decision **D7**.

3.1.3 The measurement that justifies it — and it says the writer already works

Identity splice through stock mdast-util-from-markdown node.position.start/end.offset, over every top-level block:

files	1,080
blocks	107,287
spliceIdentityOK	1,080
spliceBad	0

`[measured, corpus_id sha256:3a010b16...`, final-gate **TECHNICAL** synthesis lens]

(1,080 rather than 1,084 because four corpus files are zero bytes.)

The negative control is equally strong. **Nothing survives the AST round trip**:

SELECTION	ROUND TRIP	BYTE-IDENTICAL
51 files in this repo	<code>parse → remark-stringify</code>	0 of 51 (1 of 51 after tuning every option) <code>[measured, engine PLAN §1.2]</code>
60 files, 2,000 < bytes < 20,000, nearest 8 KB, all three roots	<code>mdast-util-from-markdown</code> + <code>mdast-util-to-markdown</code> + GFM	0 of 60 <code>[measured, founder-thesis]</code>
120 files, seeded pseudo-random (seed 20260801), all sizes, all three roots	same	0 of 120 , 0 errors <code>[measured, founder-thesis verifier – CONFIRMED]</code>

Three independent selections, three zeroes. `mdast-util-to-markdown` 's own documentation states “*complete roundtripping is impossible.*” `[primary]`

3.1.4 Two shipped code paths violate D7 today, on every Publish

Violation 1 — `src/modules/share/infrastructure/share-writer.ts:26`. Verified verbatim:

```
const next = matter.stringify(parsed.content, data);
```

The file's own header comment reads “...*splicing the key into the YAML frontmatter...*”. **The comment is false.** `node_modules/gray-matter/index.js:161`, verified verbatim:

```
matter.stringify = function(file, data, options) {
  if (typeof file === 'string') file = matter(file, options);
  return stringify(file, data, options);
};
```

`parsed.content` is a string, so `matter()` re-parses it. Any body that opens with a `---` block is consumed as front matter and **silently lost**. This fires on every Publish and every Unpublish. `[primary – both files read this session]`

Violation 2 — `src/modules/preview/presentation/frontmatter.ts:47` (and again at line 59), verified verbatim:

```
const yaml = doc.toString().replace(/\\n$/, "");
```

Measured against every front-matter block in the pinned corpus: **170 of 907 blocks throw** under gray-matter, and **only 33 survive byte-identical (3.64%)**. `[measured, corpus_id sha256:3a010b16...]`

The app is named `frontmatter` **and it mangles front matter in the only two write paths that ship**. Every argument about whether a splice writer is worth building collapses the moment you notice it has two callers already broken in production.

3.1.5 WARNING — do not take the `yaml` -library shortcut

The obvious fix is to swap `gray-matter` for the already-installed `yaml` package's `Document` API. It is not good enough. Measured over every front-matter block in the pinned corpus with `yaml` 2.9.0 (version read from `node_modules/yaml/package.json`), and **reproduced 4/4 by the independent kill audit**:

OUTCOME	BLOCKS	SHARE
byte-identical no-edit round trip	119	13.12%
drift	618	68.14%
fails to parse at all	170	18.74%
total front-matter blocks	907	100%

Typical drift: `tags: [daily]` → `tags: [ daily ]`. Typical no-parse: an unquoted `[[wikilink]]` or a bare `@.` `[measured, kill audit – PERFECT REPRODUCTION, 4/4 numbers]`

Note the two nearby numbers are not the same number. 171 is the count of blocks `gray-matter` throws on; 170 is the count `yaml` cannot parse. Different parsers, different failures. Do not merge them. Note also that v2.0.0 of this plan wrote "170 **files** do not parse"; the audited figure is 170 **blocks** of 907. With one block per file they coincide, but quote the block figure.

3.1.6 How it works — the specification

```
// src/modules/mdmax/splice.ts - the ONLY function permitted to write document bytes

export type U160ffset = number & { readonly __brand: "U160ffset" };

export interface SpliceReceipt {
  readonly range:      { start: U160ffset; end: U160ffset };
  readonly beforeSha256: string; // sha256 of the whole source
  readonly afterSha256:  string; // sha256 of the whole result
  readonly prefixSha256: string; // sha256 of src.slice(0, start)      - proves untouched
  readonly suffixSha256: string; // sha256 of src.slice(end)          - proves untouched
  readonly replacedBytes: number;
  readonly insertedBytes: number;
}

export function splice(
  src:      string,
  range:    { start: U160ffset; end: U160ffset },
  replacement: string,
): { bytes: string; receipt: SpliceReceipt };
```

Algorithm, in full — there is no more to it:

```
1. ASSERT 0 <= start <= end <= src.length           else throw E-RANGE
2. ASSERT start and end do not fall inside a UTF-16 surrogate pair
   (src.charCodeAt(i) in [0xDC00,0xDFFF] with a high surrogate at i-1)
                                     else throw E-SURROGATE
   - never round an offset to a legal one; refuse it.
3. out = src.slice(0, start) + replacement + src.slice(end)
4. ASSERT out.slice(0, start) === src.slice(0, start)
   ASSERT out.slice(start + replacement.length) === src.slice(end)
5. return { bytes: out, receipt: { ... } }
```

The frontmatter caller is a byte scan, not a parse — which is the entire point, because it must work on the 171 blocks no YAML parser can read:

```
spliceFrontmatterValue(src: string, key: string, value: string | null): string
// 1. locate the opening delimiter: /^---\r?\n/ at offset 0, else there is no block - refuse.
// 2. locate the closing delimiter: the next line that is exactly "---" (or "...") at column 0.
// 3. inside [openEnd, closeStart), scan lines for /\s*<key>\s*/ at indent 0.
// 4. value === null -> splice(src, {start: lineStart, end: nextLineStart}, "")
//     else if found -> splice(src, {start: valueStart, end: lineEnd}, serialiseScalar(value))
//     else          -> splice(src, {start: closeStart, end: closeStart}, `${key}: ${v}\n`)
// NO YAML PARSER IS INVOKED ON ANY PATH.
```

3.1.7 API and CLI surface

There is **no CLI**. This is a library function. It ships as `src/modules/mdmax/splice.ts` (~40 lines with the boundary assertions) plus a `vitest` suite that replays all 907 front-matter-bearing files. `[final-gate TECHNICAL synthesis, item P-W, ships_as ]`

3.1.8 Dependency note that will bite you

`mdast-util-from-markdown` is **not** in `package.json` `dependencies`. It is present in `node_modules` at **2.0.3**, pulled in transitively via `remark-parse ^11.0.0` (`mdast-util-to-markdown` is likewise present at 2.1.2, transitively). The research repeatedly says "already a dependency"; that is true only transitively. **Add an explicit direct dependency and pin it before the splice writer relies on** `position.offset`, or a transitive bump silently changes the offsets you are splicing on. `[primary - package.json and node_modules read this session]`

3.1.9 Failure modes

FAILURE	CAUSE	BEHAVIOUR
E-RANGE	offsets out of bounds or inverted	throw before any write
E-SURROGATE	an offset falls inside a surrogate pair	throw; never round
No front-matter block	file does not open with <code>---</code>	refuse, return the source unchanged; never synthesise a block
Unterminated block	opening <code>---</code> with no closing delimiter	refuse; the file is passed through untouched
CRLF	1 file in 1,084 uses CRLF <code>[measured]</code>	the byte scan must accept <code>\r?\n</code> ; the receipt proves nothing was normalised
Offset-unit confusion	a byte offset from git, <code>Buffer</code> , or a content hash enters the function	prevented by the branded type in §3.8.1, not by care

3.1.10 The gate, and it must go RED first

Publish-then-unpublish over all 907 front-matter-bearing files of `corpus_id sha256:3a010b16...` is byte-identical, INCLUDING the 171 that `gray-matter` cannot read and the 170 that `yaml` cannot read — verified by an oracle sharing no code with the writer.

Per Learned Rule #68: run the assertion against the **current** writer first and watch it fail. A green suite on an unfixed system is the expected result of running it, not evidence of a fix.

3.1.11 Kill condition

There is no kill condition. This capability is a **bug fix for a live data-corruption defect** with a proven implementation. If the gate cannot reach 907/907 the defect is in our implementation, not in the approach — the 1,080/1,080 identity-splice result already proves the approach.

3.1.12 What this does NOT cover

Splice-only writing gives fidelity. It does **not** give *alignment* — knowing where a stored anchor moved to after someone else's edit. That is §3.2. CodeMirror's `ChangeSet` satisfies the Edit Lenses module laws (Hofmann, Pierce & Wagner, POPL 2012) under 5,000/5,000 property tests, and `@codemirror/state ^6.6.0` is already a direct dependency `[primary, package.json]` — that is where alignment comes from inside the app, not from the splice writer.

3.2 Capability 2 — BLOCK IDENTITY AND CONTENT-DERIVED RE-ANCHORING

3.2.1 What it does

Given a block from an old revision of a document and the new document, return the block's new position, or refuse. Refusal is a first-class answer.

3.2.2 Why it exists — and the honest position, stated first

Every serious document system solved anchoring using a property we do not have: Word puts zero-width sentinel pairs in XML; Notion puts a UUID on a database row; Overleaf and ProseMirror/CM6 remap integer offsets through the operation that caused the change, which requires **owning every write**. `[primary, engine PLAN §1.1]` We cannot own every write: a `git pull` yields a new string, a `vim` edit yields no operation at all.

But the honest position, which v1 of this plan got wrong: content-derived re-anchoring is the VERIFIER and the FALLBACK, not the primary mechanism — because in this product a previous version essentially always exists. In-app edits give a CodeMirror `ChangeSet`; offline sync, AI Edit and external sync all give a before **and** an after. The plan's own §4.1 already conceded this; the build sequence must match the concession.

Correction to a load-bearing justification we published. We justified this capability with *"a git pull yields no op stream."* It yields two blobs and a merge base, **and a diff is an op stream**. The 294-revision-pair corpus was itself built from git history — the diff was in hand for all 41,642 block-versions and was never used as a baseline. `[v2.0.0 §9.4]`

3.2.3 The algorithm, complete

From `docs/engine/PLAN.md §1.1a`, reproduced in full because it is the specification:

```
resolve(block i, new document) -> index | AMBIGUOUS | LOST

gate: anchorable(i)? else NO_IDENTITY          # 14.2% of blocks have no identity
S1  exact: blake2b-64 over normalize(text)
    exactly one match          -> return it          # resolves 87.0%
S2  several exact matches: score by softctx; unique argmax > 0 -> it, else AMBIGUOUS
S3  no exact match (edited): candidates = blocks sharing any 3-gram shingle
    keep jaccard >= TAU; score = jaccard + W * softctx
    top1 - top2 <= DELTA -> AMBIGUOUS, else argmax

softctx(i,j,K) = distance-weighted 1/d agreement of the K neighbours either side
anchorable     = not a rule, not punctuation-only, >= 3 tokens
```

Parameters, chosen by a **384-configuration sweep**:

PARAMETER	VALUE	WHY
<code>normalize</code>	NFC + collapse whitespace + lowercase	recovers reflow and case edits free
<code>K</code> (context radius)	3	1 → 97.88%, 3 → 98.13%, 5 → 98.21%
<code>TAU</code> (Jaccard floor)	0.20	0.20 → 99.14% vs 0.70 → 96.47% correct at no false-match cost
<code>W</code> (context weight)	1.0	best on both axes
<code>DELTA</code> (tie margin)	0.05	lowest false rate; below it, refuse

3.2.4 The measurements

Over **41,642 block-versions** from **294 consecutive real revision pairs**; denominator **32,919 surviving block-versions**:

SCHEME	CORRECT	FALSE MATCH	SAFE REFUSAL
byte offset	36.93%	62.12%	0.95%
block index	44.43%	55.50%	0.08%
content hash alone	83.36%	0.00%	16.64%
content hash + nearest-position tiebreak	77.80%	22.20%	0%
rigid context fingerprint k=1 / k=2 / k=3	90.65 / 88.68 / 87.22%	0.25%	9–13%
content-defined chunking (rolling hash)	18.92%	67.30%	13.79%
recommended, anchorable blocks only	99.627%	0.050%	0.323%

Cost: **~41 ms** for a 446 KB / 2,225-block document. `[measured, engine PLAN $1.1a]`

Supporting facts that matter more than the headline:

- **The `anchorable()` gate is the highest-leverage single line in the design.** It moves the false rate 0.349% → **0.050%**; **88.9% of raw false matches are** `---` **horizontal rules.**
- **12.9% of blocks present more than one candidate**, and on that subpopulation the resolver scores 94.94% correct / **2.66% false — 50x the single-candidate error rate.** Removing the `DELTA` refusal produced **183 extra false matches.**
- **All 14 false matches on anchorable blocks were hand-audited, not sampled:** 4 oracle artifacts, 6 from a bulk markdown→HTML migration, 3 homogeneous-list drift, 1 table rewrite. **Genuine substantive errors: 10 in 28,170 = 0.036%.**
- **The known adversarial input is already in the corpus.** `knowledge` scores **8x worse (0.280% false)** because `knowledge.md` is one enormous homogeneous index list. **List items must carry a structural key — position within the parent list plus the parent's identity — rather than relying on text alone.**

3.2.5 The adversarial baseline, and why it does not kill the capability

A **30-line LCS diff** — no shingles, no context scoring, no sweep — was executed against the same corpus:

RUN	PAIRS	ANCHORABLE BLOCK-VERSIONS	LCS MATCHED	UNIQUE-TEXT	HASH-ALONE-UNIQUE
original	394	—	86.658%	77.297%	77.297%
independent verifier	494	31,149	85.036%	76.346%	76.352%

`[measured, final-gate against area + its verifier – headline verdict OVERSTATED]`

Both are far below 99.627%, and the gap is not the argument. The argument is that `diff3`-assisted resolution is available whenever a previous version exists, which is almost always. **Two facts stop LCS from being the whole answer:**

1. **LCS makes a silent order-based tiebreak on duplicate text.** Of 2,707 duplicate-text LCS matches, **111 (4.10%) have neither neighbouring block matching** — 0.356% of all anchorable block-versions, **~7x the 0.050% false rate the tuned resolver books.** That is the same trade the sweep already measured at 0.00% → 22.20% false for position tiebreaks.
2. **The benchmark cannot adjudicate this on the pinned corpus. 86.6% of `.md` file-revisions in its largest root delete zero lines**, so S1 resolves by construction and the number is a property of an append-dominated corpus. Tier 3 must be measured on a stratum restricted to revisions deleting more than two lines — and **that stratum may be too small to measure, which is itself the finding.**

3.2.6 The most useful result in the whole area — token stratification

All residual risk lives in short blocks. Independent verifier run, 494 pairs, `corpus_id sha256:3a010b16...` (`{matched, at-risk}`):

BLOCK LENGTH	MATCHED	AT-RISK	AT-RISK RATE
3–5 tokens	5,329	2,360	44.28%
6–9 tokens	3,899	337	8.64%
10–19 tokens	3,298	10	0.30%
20+ tokens	13,962	0	0.00%

2,697 of 2,707 at-risk block-versions (99.63%) are ≤ 9 tokens. `[measured – verdict CONFIRMED, "the most valuable thing in the submission"]`

Scope note, not a defect: this is an *ambiguity* rate, not an *error* rate. Neither run has ground truth for "false" on this stratification.

3.2.7 Five caveats that must travel with the 99.627% figure, every time

1. **Nobody has re-derived it.** Not one of the research areas that quoted it, including the ones that built their design on it. **Do not publish it externally until it is re-derived on a corpus this team did not write.**
2. **It is a BLOCK figure. Comments anchor RANGES.** The one replication of the range case gives **3.44x, not 30x**, with refusal roughly doubling.
3. **~1 heading in 4 cannot carry an anchor** (18.3% / 25.4% / 24.7% of headings are under 3 tokens across the three roots) — and headings are what everything points at.
4. **Move figures are** `[SIMULATED]` . Real history contained **zero multi-block moves, 138 singletons only.**
5. **The oracle shares a candidate generator with the resolver.** The full hand-audit is the defence, and it found 4 of 14 "errors" to be oracle artifacts.

3.2.8 How it ships — a three-tier ladder, in this order

tier 1	ChangeSet.mapPos	@codemirror/state, already a direct dependency, exact by construction, in-app edits only
tier 2	diff3-assisted	whenever a previous version exists (almost always)
tier 3	content-derived S1/S2/S3	ONLY when neither is available: offline-sync reconciliation, an external edit, a git pull with no merge base

Tier 3 returns `RESOLVED` | `AMBIGUOUS` | `LOST` and **never breaks a tie by position**. [final-gate TECHNICAL synthesis, item P4]

For comments specifically, use the `W3C TextQuoteSelector` (Web Annotation Data Model, a W3C Recommendation since February 2017): `{ exact, prefix, suffix }`. It is not an invention and must not be described as one.

Do not vendor `dom-anchor-text-quote` **as the substrate**. Read in full (158 lines on `master` ; the research said 182 — a self-report error): every search is `dmp.match_main(root.textContent, pattern, loc)`, a `DiffMatchPatch` bitap with a location hint. **There is no exact path, no candidate array, no scoring, and the only returns are `null` or one `{start, end}`** — it cannot express `AMBIGUOUS`. Adopting it wholesale would ship a regression against the refusal discipline that is the entire point. Also: it is **not** in production at Hypothes.is — `hypothesis/client's package.json` lists `approx-string-match ^2.0.0` and neither `dom-anchor-text-quote` nor `diff-match-patch`. [measured, kill audit — every clause reproduced]

The live competitor. `inkeep/OpenKnowledge` shipped content-derived anchoring on 2026-07-30 (3,239★, GPL-3.0): `anchor.ts`, 261 lines, exact-quote plus widened context plus orphan-rather-than-guess, on npm at `0.46.0-beta.32`. **Its server path returns `{status: 'orphaned'}` on a tie while its app path returns `best[0]` — two different policies in one product — and it publishes no accuracy number.** [secondary, tarball-verified] **What MDMAX can own is not the algorithm. It is the refusal and the published false-match rate.**

3.2.9 API and CLI surface

```
mdmax resolve <file> --anchor @anchor.json [--explain]
  → RESOLVED {start,end} tier=1|2|3 rung=S1|S2|S3 candidates=N
  → AMBIGUOUS candidates=[{start,end,score}, ...] exit 4
  → LOST exit 3
```

`--explain` **must** print which tier and which stage fired. Refusal-with-a-reason is the product difference against a competitor that silently returns `best[0]`.

Stored anchor record, versioned (see §3.8.2):

```
{ "schema": 1,
  "normalize": "mdmax/normalize@1",
  "slug": "mdmax/slug@1",
  "id": "blk_7f3a",
  "quote": "-", "prefix": "...", "suffix": "-" }
```

3.2.10 Failure modes

FAILURE	BEHAVIOUR
Block is not anchorable (14.2% of blocks)	<code>NO_IDENTITY</code> — positionally interpolate between anchorable neighbours; never anchor
Two candidates within <code>DELTA</code>	<code>AMBIGUOUS</code> — surface it; never guess
Homogeneous list corpus (the <code>knowledge.md</code> case)	8× worse false rate; mitigated only by the structural key for list items
<code>normalize/1</code> changes	every stored anchor silently re-keys . See §3.8.2 — this must be settled before the first anchor is persisted
Heading slugger unpinned	anchor health swings 77 percentage points on the choice. See §3.8.2

3.2.11 Kill conditions

- K3** — the re-derived anchor number is materially below 99.627%, **or** the LCS diff baseline matches it on the >2-lines-deleted stratum → the anchor is a fallback only; re-scope the claim and **never publish the number**.
- K4** — the *range* resolver's hand-audited false-match rate exceeds ~0.5% → comments cannot use it; fall back to quote-plus-digest with **visible** orphaning.
- If the >2-lines-deleted stratum is too small to measure at all, **say so and ship tiers 1–2 only**.

3.2.12 What this does NOT cover

Concurrent edits from two sources were never tested. **Nothing in §3.2 validates a merge path**. Nothing here addresses multi-human co-editing, which is the product's whole premise — the corpus that produced these numbers is 416 commits by exactly one author over 80 days.

3.3 Capability 3 — THE DEGRADATION CERTIFICATE (`mdmax cert`)

3.3.1 What it does

For one document, report per block, per construct, per **(product, surface)** target, what a real renderer does to it. Not a prediction. A differential run against pinned renderers.

3.3.2 Why it exists

It is the **only** capability in this programme that survived adversarial prior-art search with no incumbent found for the per-document layer, and the only dataset that **decays** (renderers change, so a stale certificate is visibly stale). It is therefore the capped give-away and the credibility instrument. **It is not the spearhead — it targets pains ranked 13th and 14th of 14.**

3.3.3 The measurement — and the premise it kills

A differential harness was built and run across renderer configurations, plus live probes of GitHub's production renderers. Engines: `cmark-gfm 0.29.0.gfm.13` compiled from GitHub's source at SHA `499789b`, `kramdown 2.5.2` + `kramdown-parser-gfm 1.1.0`, `goldmark 1.8.5` (Hugo), `pulldown-cmark 0.13` with `ENABLE_HEADING_ATTRIBUTES` (mdBook), `comrak 0.39`, `markdown-it 15.0.0` at both `html:false` and `html:true`, `marked 16.4.2`, `commonmark.js 0.31.2` as the spec oracle, and the app's own remark pipeline.

Defect you must not propagate: the enumerated bench sums to 23, not 24. Re-derived: `1 + 3 + 1 + 4 + 2 + 3 + 3 + 2 + 2 + 2 = 23`. **Every headline ratio in that research divides by 24.** [measured, degradation-certificate verifier] The rates below are quoted as `n/24` because that is how they were reported and independently reproduced at that denominator; **the denominator itself is unreconstructible from the stated engine list.** Fix it before publishing a single row.

The finding that changes the design: "GitHub" is not one target. It is three renderers that disagree on identical bytes.

SURFACE	REACHABLE HOW	BEHAVIOUR
github-comment	POST /markdown	contested — <code>mode=gfm</code> emitted no heading anchor, while <code>mode=markdown</code> emitted the full <code><div class="markdown-heading">...</code> structure that rendered comments visibly have. Treat as declare-and-date , not as locally runnable
github-blob	GET /repos/.../contents/X.md with Accept: application/vnd.github.html — requires the content to already be pushed	deletes HTML comments outright (nodejs/node README: 5 standalone <code><!--</code> lines in source, 0 in the 91,223-byte rendered blob); renders front matter as an HTML table; prefixes <code>id="x"</code> → <code>id="user-content-x"</code> ; strips <code>class</code> , <code>style</code> , <code>data-*</code>
github-pages	Jekyll + kramdown	strips front matter before kramdown runs; 1) first / 2) second renders as a paragraph , not an <code></code> , where all 15 other engines produce <code></code>

[measured; the github-blob comment deletion and the `1)` case both reproduced exactly by the verifier]

Carrier verdicts that reproduced exactly under an independently built bench:

CARRIER	VERDICT
YAML front matter	VISIBLE in 23/24 configurations
HTML comment	renders as visible escaped text in 2/24 (<code>markdown-it 15</code> default, <code>react-markdown 10</code> default); deleted outright by GitHub's blob renderer ; 9 distinct outputs across the bench
{#id} heading attribute	LEAKS as literal text in 21/24 — only kramdown x2 and pulldown-cmark with <code>ENABLE_HEADING_ATTRIBUTES</code> consume it
link reference definition	invisible in 24/24
[/]: # idiom	invisible in 24/24 — but does not survive a write (parens become quotes)

Live GitHub, re-verified independently: `<div id="x" class="c" style="color:red" data-k="v">hi</div>` → `<div id="user-content-x">hi</div>`. [measured]

3.3.4 Five claims from that research that its own verifier killed — do not repeat them

- "`a**€**b` **bolds on GitHub only.**" Refuted. It bolds on GitHub production **and** on `cmark-gfm 0.29.0.gfm.13`. It does **not** bold on `commonmark.js 0.31.2` — the designated spec oracle — nor on `markdown-it 15`, `marked 16`, `remark/micromark` or `react-markdown`. **The real finding is that the github-comment proxy and the spec oracle disagree on the cited example.**
- "`\|` **is de-escaped when tables are off**" as a render divergence. Zero divergence in all 17 local configurations and on GitHub production. It is a source round-trip concern, not a render one.
- "**kramdown/GFM → github-pages exactly.**" Jekyll ships **nine** kramdown options (`lib/jekyll/configuration.rb:67-77`), and `kramdown-parser-gfm` defaults `hard_wrap` **ON** where Jekyll sets it **OFF**. Measured on `Line one\nLine two`: researcher config → `<p>Line one<br />\nLine two</p>`; Jekyll defaults → `<p>Line one\nLine two</p>`. **Every soft line break in every document would be a false MUTATE.** The rule is sharper than "record the engine version": **a certificate without the target's full option set is a lie.**
- "**Only frontmatter's own app hides front matter.**" Hugo, Docusaurus, Astro, Eleventy, Jekyll and GitHub's blob viewer all consume it upstream of the engine.
- The entire performance budget** ("87% of wall time", "~43 ms for a typical 4 KB file", "15.5 min for a 4,317-file vault"). Internally inconsistent: `31.3/(31.3+5.9) = 84.1%`, not 87%; 43 ms sits **below** the report's own 3-spawn floor of 93.9 ms; `4,317 × 43 ms = 3.09 min`, not 15.5. **Re-derive the budget from scratch before quoting any of it.**

Also unresolved: the **equivalence fold** is claimed to have moved the divergence rate from **43.71% to 4.28%** — but with **no corpus, no artifact, and no denominator**. It is the load-bearing component of the whole design and it is currently a rate without a population.

3.3.5 How it works — three layers, forced by measurement

- Layer 1 — per-CONSTRUCT capability table**, built offline from synthetic minimal pairs. Required because whole-file correlation cannot attribute cause: in a 300-file run every feature co-occurred with every other and all showed "100% divergent", which is information-free. [measured, self-caught negative result]
- Layer 2 — per-FILE certificate**: inventory which Layer-1 constructs the file contains, at which byte offsets, joined to the table.
- Layer 3 — per-file differential RUN**, because the table does not predict everything (the tilde/strikethrough case was found by the differential, not by any construct list anyone wrote).

The verdict vocabulary — four values, not three. v2.0.0 §5.4 specified `PASS · STRIP · CORRUPT`. The founder-thesis area measured a fourth that the three-valued matrix has no cell for:

VERDICT	MEANING	MEASURED	EXAMPLE
PASS	semantically equivalent output after the versioned fold	—	
STRIP (DEGRADED)	output differs, payload invisible, no source character lost		tail-placed orphan link reference definition: 0 bytes of HTML in 24/24
CORRUPT (BROKEN)	LEAK (payload becomes visible text) · DESTROY (a source character is deleted or substituted) · MUTATE (same bytes, different structure)		front matter LEAKs in 23/24; <cat> deleted; 1) MUTATES on kramdown
VOID	the content was by reference and the reference did not resolve — the payload was never in the file	![[Some Note]] → ![[Some Note]] under marked 16.4.2, reproduced byte-for-byte by the verifier, versus ``mermaid → <pre><code class="language-mermaid">... with the payload intact	

VOID is the verdict that decides which capabilities D6 permits. Additive capabilities (an unknown fence language) are free; any capability whose payload is *by reference* cannot degrade at all. **RECOMMENDED: ship four verdicts.**

3.3.6 The equivalence fold — ship it as a named, versioned function

It folds: full HTML entity decode (use `entities.decodeHTML`, not a hand-rolled five-entity replace — that was the original bug); smart punctuation (`' ' → ' ', " " → " "`, `- → -, -- → --, ... → ...`); void-element spelling (`<chr> / <chr /> / <chr/>`); trailing and inter-block whitespace; and `id` prefixing (`x → user-content-x`). **Everything the fold does is a documented, testable normalisation; everything it does not do is a real finding.**

3.3.7 CLI surface

```
mdmax cert <file> # human table: block × target × verdict
mdmax cert <file> --json # the artifact (schema below)
mdmax cert <glob> --fail-on=BROKEN # CI gate; exit 1 on any BROKEN
mdmax cert --targets=github-blob,hugo # subset
mdmax cert --bench-info # every engine + version + SHA + FULL OPTION SET
mdmax cert --explain <construct> # the minimal pair and its per-config matrix
mdmax cert --cover <targets> # minimal engine subset covering those targets
mdmax bench build | bench verify # compile/pin the vendored engines, verify hashes
```

Artifact (JSON sidecar — **never written back into the .md**; per Learned Rule #11 the agent must not rewrite it):

```
{ "bench": { "version": "...", "engines": [ { "name": "...", "version": "...", "sha": "...", "options": { } } ] },
  "file": { "path": "...", "sha256": "...",
  "targets": [ "github-blob", "github-pages", "hugo", "..." ],
  "summary": { "pass": 0, "strip": 0, "corrupt": 0, "void": 0 },
  "blocks": [ { "anchor": "blk_7f3a", "type": "paragraph", "byteRange": [1204, 1251],
    "verdicts": { "github-blob": { "verdict": "CORRUPT", "class": "DESTROY",
      "before": "Array<string>", "after": "Array" } } ] }
```

How it fails: loudly and specifically. Not *"this may not render everywhere"* but *"block blk_7f3a, bytes 1204–1251: on github-blob the text `Array<string>` renders as `Array` (DESTROY); 6 of 9 targets agree with github."* **A verdict with no before/after pair is not a verdict.** Per Learned Rule #67, a missing engine must be a **hard refusal**, never a silently smaller matrix reporting green.

3.3.8 The effort contradiction, named and resolved

Two sources disagree and the disagreement is 10x.

- `docs/mdmax/PLAN.md` v2.0.0 §4.4 and §7 item 5: **"two days, hard cap"** — because cert is a distribution budget, not a product.
- The `degradation-certificate` research design: **"~3 weeks for a shippable v1 with 9 locally runnable targets"** (bench 1–2 days, fold 2–3 days, construct corpus 2 days, per-file engine 1 week, cover/caching/WASM 3–4 days, CLI 2–3 days).

The plan governs. Ship the research's own **"cheapest useful slice"** inside the two-day cap: freeze the fold at a naive version, run a **3-engine cover** (the app's remark pipeline + `react-markdown` + kramdown, all installable in one `npm / gem` step) over the existing vault, and publish the **BROKEN-class frequency histogram**. That histogram alone decides kill condition (4) below, which is the one most likely to fire. The 3-week bench is a research instrument; build it only if the histogram says it is needed.

Build hazard, measured: `cmark-gfm 0.29.0.gfm.13`'s `CMakeLists.txt` declares `cmake_minimum_required` below 3.5 and modern CMake refuses with *"Compatibility with CMake < 3.5 has been removed."* It needs `-DCMAKE_POLICY_VERSION_MINIMUM=3.5`. **The bench-build command and the CMake version belong in the pinned manifest alongside the engine SHAs** — a pinned offline bench whose build recipe is version-fragile is self-refuting.

3.3.9 Prior art — narrow the novelty claim

"No prior art" is true only for the **per-document** layer. Uncited but on point: `karlcow/markdown-testsuite` (102★; `tests/` + `run-tests.py` + a per-implementation stdin driver), `babelmark/babelmark-proxy` (MIT, self-hostable), cmark's own `spec_tests.py`, and `ArchieCur/MARKDOWN_FLAVORS` (created 2025-10-21, 7★, exactly 14 flavour rows × 17 construct columns in `markdown-flavors.csv`). **[measured, kill audit] Build Layer 1 on an existing suite's fixtures rather than on a construct list we write by hand** — that is exactly the failure mode the tilde case exposed.

Note also that babelmark's registry is **38 entries, 15 with plaintext endpoints, 23 encrypted** — so babelmark's *hosted dispatch* is not self-hostable, but every entry carries a plaintext `Repo` link and the *implementations* are. The research's "structurally impossible" was an overstatement.

3.3.10 Kill conditions — four, any one of which ends it

- The fold swallows everything.** If semantic BROKEN verdicts on a neutral corpus fall below ~1% of blocks and below ~15% of files, the certificate is a curiosity. The honest own-corpus figure was **4.28%, not 43.71%** — and 4.28% is not a large number.
- The uncertifiable surface is the only one anyone cares about.** `github-blob` cannot be probed without pushing content; Obsidian, Notion, Typora, Bear, Slack, Discord and the chat renderers cannot be probed at all — **7 of ~12 surfaces**. Kill unless declared-contract-plus-canary is accepted as honest.

3. **The market is "pick one renderer."** If the dominant workflow is single-target, the correct product is a single-target linter, which `markdownlint` and `remark-lint` already are at zero marginal cost.
4. **It is a lint rule, not a certificate.** If the minimal cover collapses to one engine, or if >80% of BROKEN verdicts come from fewer than five constructs, the apparatus reduces to a static rule set running in 2 ms. **The own-corpus data leans this way:** six constructs (angle brackets, lone tilde, front matter, HTML comment, pipe-in-prose, `{#id}`) account for most of what was found.
5. **Product gate K5** — fewer than 5 of 30 probed users say they would install it → cut it.

3.3.11 What this does NOT cover

Obsidian, Notion, Typora, Bear, Slack and Discord are **declare-and-date** rows, refreshed on a schedule against a canary, stamped with a last-verified date. Slack and Discord are not CommonMark and must be modelled as **lossy sinks**, not renderers. Do not attempt UI automation to certify them.

3.4 Capability 4 — THE NEAR-MISS REFERENCE CHECKER

3.4.1 What it does

Classify every unresolved reference in a corpus into three buckets and report **only one of them**.

3.4.2 Why it exists — the two-tier link problem

Markdown carries two reference syntaxes that behave nothing alike, and **no tool implements the semantics that difference demands**:

SYNTAX	RESOLVES	SOURCE
<code>[text](path.md)</code>	85.71% (546 / 637)	v2.0.0 §3.2 <code>[measured]</code>
<code>[[Name]]</code>	27.65% (2,754 / 9,962)	v2.0.0 §3.2 <code>[measured]</code>
a 58-point gap on identical files		

A path link that does not resolve is a mistake. A wikilink that does not resolve is a legitimate Obsidian authoring primitive. Treating them identically is why naive link checking runs at **2–25% precision** and gets switched off `[measured, engine PLAN §4]` .

CONTRADICTION, named. Three research passes produced three number sets for the same claim:

SOURCE	WIKILINKS	RESOLVE	PATH LINKS	RESOLVE
v2.0.0 §3.2	9,962	27.65%	637	85.71%
<code>markdown-base</code> headline	9,077	25.50%	624	88.62%
<code>markdown-base</code> verifier	13,026 in the pinned corpus	29.53% on the frontmatter-stripped basis, 24.93% including front matter	624 (reproduced to the digit)	88.62% (reproduced)

The verifier found the headline census **silently stripped YAML front matter**, removing **3,850 links (29.6% of all wikilinks)** carried by `entities:` (2,283), `related:` (1,407) and `up:` (49) keys. Those front-matter wikilinks resolve at **13.95%** — worse than body links — so including them lowers the headline and, if anything, strengthens the SOFT classification. **Which governs: the verifier.** Quote 13,026 as the wikilink population, state that the resolution rate depends on whether front-matter links are counted, and quote the **gap** rather than either percentage: the verifier's own summary is that the structural finding *"survives at a 58-point gap rather than 63."*

Narrow the novelty claim. "No existing tool does this" is contestable by the research's own primary source: `facebook/docusaurus` `configValidation.ts` already ships tiered defaults across four link classes with two severities — `onBrokenLinks: 'throw'`, `onBrokenAnchors: 'warn'`, and at lines 124–125 `onBrokenMarkdownLinks: 'warn'` against `onBrokenMarkdownImages: 'throw'` — at **1,513,550 weekly downloads**. `[primary]` The defensible claim is narrower: **the wikilink-versus-path-link split specifically**.

3.4.3 The measurement that justifies shipping it

Across two corpora, with hand verification:

CLASS	RULE	COUNT	REPORTED BY DEFAULT
BREAK	a file exists with ≥ 0.85 normalised name similarity	95, hand-verified real broken links	yes
DRIFT	case- or punctuation-only mismatch	52	yes
GHOST	no similar name anywhere	5,777	no — silent , available behind <code>--ghosts</code> as an authoring backlog ranked by inbound demand

`[measured, impact-preview area]`

The default output must be near-empty. A vault-level compiler run over 833 authored files found 365 true problems touching only **112 files (13.4%)** — a random note has an **86.6%** chance of an empty panel, and one file held 56% of everything `[measured, engine PLAN §4]` . **A checker that reports 3,442 problems is uninstalled the same day.**

Two figures in that area were killed by its own verifier and must not be repeated: "3,491 dangling md-vault refs against 3,129 total md-vault refs" (more dangling than total, in both corpora), and "97% are intentional placeholders" — **intent was never measured**; what was measured is the *absence of a near-name match*.

3.4.4 How it works — the symbol table

Five relations, all cheap:

```
defs      (file, kind ∈ {heading, block, fm-key, tag}, name, level, byte-range)
refs      (file, byte-offset, kind, raw-target, resolved-target | NULL)
by-name   basename → [file]           # resolves ghosts and near-misses
reverse   resolved-target → [(file, offset)] # the impact query
ghosts    unresolved-name → [(file, offset)] # the novel relation; nobody stores this
```

Measured: **69,805 rows for 78 MB**; the whole index serialises to **607 KB for a 78 MB vault (0.74%)**. Incremental maintenance re-parses **only the changed file** — 0.254 ms median, 1.567 ms p90 in Python — and patches the four indexes by key: `0(symbols in that file)`, never `0(vault)`. Cold build is the expensive path: the research said 10.7–26.1 s for a 20,000-file vault; **its verifier corrected that to ~10.6–15.7 s** because the original applied small-file throughput to a large-file corpus. Either way it must be **lazy, persisted, and off the keystroke path**.

Resolution rules, decided:

- Symbol = a file. Canonical key = the root-relative path.** Basename is a convenience alias with a measured **25-instance (1.1% of resolving wikilinks)** ambiguity set; on ambiguity report `AMBIGUOUS` and refuse — **never guess**.
- HARD** `[text](path.md)` — checked. Broken is an error surfaced in the editor.
- SOFT** `[[Name]]` — resolve-if-present, **never an error, no diagnostic, no squiggle**. A soft reference that later acquires a target simply starts resolving.
- A third population exists and the two-tier design has no cell for it:** front-matter wikilinks (3,850 of them). `related:` and `up:` are hand-authored file references; `entities:` is `graphify` output naming *concepts*, not files. **OPEN: classify front-matter wikilinks as a third tier or fold them into SOFT. Do not leave it undocumented.**
- Headings are NOT exported symbols. 71.3% of heading instances have a globally non-unique slug; 8.8–8.9% of files contain a within-file duplicate;** and `github-sluggers`'s disambiguation is **positional** — `while (occurrences[result]) result = original + '-' + ++occurrences[original]` `[primary, source read]` — so an exported heading id changes when an unrelated heading is edited. Headings are within-unit coordinates resolved by the unit's own local table under the pinned `slug/1` (§3.8.2).
- {#id} is refused.** It is not CommonMark (D6) and it **LEAKS as visible text in 21/24** bench configurations (§3.3.3). This means the "author-given heading id" escape hatch proposed by the `markdown-base` area **has no legal carrier** and must be dropped or re-carried.

3.4.5 CLI surface

```
mdmax check [PATH...]           # BREAK + DRIFT only. Default output is near-empty.
    --ghosts                    # the authoring backlog, ranked by inbound demand
    --json
mdmax refs <file> --scope ./docs # reverse dependency query. An UNSCOPED query is not well-formed.
mdmax rename <old> <new>         # DRY-RUN BY DEFAULT. Terraform-shaped header line first
                                # ("Plan: 6 references to update in 4 files, 1 conflict"),
                                # then per-site rows, every row individually EXCLUDABLE,
                                # conflicts to a separate confirm step. File-level only in v1.
```

Writing stays splice-only: `mdmax rename` computes byte-range edits against the `refs` index and splices them; a file with an unresolved anchor is **passed through untouched**, never regenerated.

3.4.6 Failure modes

FAILURE	BEHAVIOUR
Similarity threshold too low	BREAK fills with ghosts; the tool is uninstalled. Precision, not recall, is the product.
Ambiguous basename	<code>AMBIGUOUS</code> , refuse
Cross-root reference (244 exist, 2.69% of unresolved wikilinks)	require an explicit alias declaration ; never implicit cross-root search
Cold index on a 20,000-file vault	the editor must be fully usable while the index is still building

3.4.7 Kill conditions

- BREAK-class precision cannot be pushed above 0.90** anywhere on the 0.70–0.95 similarity sweep, **or** fix-acceptance is under 50% → cut it. At 97% ambient ghost noise there is no room for a mediocre classifier: a 70%-precise checker is worse than none, because users disable it and lose the 3% that was real.
- The symbol table cannot be maintained under **5 ms p99 per keystroke** on a 20,000-file vault → move it off the keystroke path to an explicit command.

3.4.8 Contradiction in the record, named

The final-gate `against` area's design says: *"DROP ENTIRELY — ... the near-miss reference checker."* The `impact-preview` area calls it *"the genuinely shippable feature hiding in the same symbol table."* v2.0.0 §5.3 keeps it. **The `against` headline was graded OVERSTATED; §5.3 governs. Keep the checker, at RECOMMENDED status, gated on the precision number.**

3.4.9 What this does NOT cover

Impact preview on heading and block anchors is out of scope for v1. The census that motivated it enumerated only Obsidian wikilink syntax and **never counted** `[text](#heading)` **or** `[text](file.md#heading)`, so its 130:1 and 1014:1 ratios are unusable; its "zero heading-rename breakages" null came from a test with **~11–13% statistical power** ($P(\text{observe zero} \mid \text{effect fully real}) = 87\text{--}92\%$). **Both the ratio and the null are uninformative.** The correct v1 scope is file-level rename plus the near-miss checker; heading and block relations wait for a census that counts every reference syntax.

3.5 Capability 5 — BUDGETED PROJECTION (index / outline / full)

3.5.1 What it does

Emit one of three projections of a document tree at a stated byte or token budget, and say what was omitted.

--index	one line per document: path, title, one-line summary	(default)
--outline	headings only, nested, plus per-document frontmatter keys	
--inline	the full text, joined with the FENCE-STATE-AWARE SEPARATOR	
--budget N	inline greedily to N bytes, degrade the remainder to index lines	

Projections are never stored. Storing one is how a format rots [engine PLAN §2.4].

3.5.2 The ratios — four numbers for one claim, and none of them means what it looks like

RATIO	NUMERATOR VS DENOMINATOR	SOURCE
242x	container 25,548,765 B vs one-line-per-doc index 105,538 B	v2.0.0 §3.1 [measured, corpus id]
110.1x	container 25,776,700 B vs described index 234,119 B	container-thesis [measured, corpus id]
168.8x	Anthropic docs: inlined 6,556,407 B vs index 38,847 B / 174 links	HANDOFF-mdmax §7.3 [measured]
101.1x	llms.txt-style index over a 21.9 M-token corpus	token-cost-tier headline — its own verifier: "the headline's 101.1x anchor has no source anywhere in the repo or the scratch"

They differ because "index" means four different artifacts. Quote one, define it in the same sentence, and cite its script. Do not quote a range "25–168x" — v2.0.0 §3.3 already struck that phrasing.

3.5.3 The caveat that must appear beside every one of those ratios

They are ARTIFACT-SIZE ratios. They are not task-token savings. The only live-model measurement of task tokens that exists went the wrong way:

"The bundle arm cost more, not less. 5.2% more tokens over six questions, and it opened more files, not fewer." — <https://raw.githubusercontent.com/scaccogatto/okf-skills/main/benchmark/results.md>, verbatim [primary]

Totals: 257,602 tokens with .okf/ vs 244,805 without. Correctness 22/26 (85%) vs 20/26 (77%) — two wins, one loss, three ties. Their own caveat, verbatim: "Eight points on twenty-six, at n=1 per cell, is not a result anyone should quote as a headline."

Say "compression" or say nothing. [v2.0.0 §3.3]

3.5.4 How big the prize actually is

Measured over the 1,200 most recent agent transcripts on this machine:

all markdown Read calls carrying an explicit offset/limit	82.5%
restricted to the FIRST touch of each (session, file) pair:	
windowed	214/362 = 59.1%
WHOLE FILE	148/362 = 40.9%
byte-weighted, of 134 whole first-touch reads still on disk (2,063,984 B):	
median file	5,260 B
reads of files > 20 KB	12 = 9.0% of whole reads, 3.5% of all 343 first-touch opens
...carrying	63.9% of whole-read bytes

[measured, agentic-docs area – the researcher's own pre-registered falsifier PARTIALLY FIRED and they reported it against themselves]

Honest verdict: the projection tier is not dead. It is small and concentrated — a win on roughly 1 markdown open in 29.

3.5.5 The one component that is unambiguously worth building

The fence-state-aware separator. Joining documents with a naive \n\n is not structure-preserving; a separator that tracks fence state is exactly structure-preserving (structural delta 0, identical structure TRUE, four contaminated adjacent pairs, cause histogram {odd_fence_count_in_A: 4}, and the same four named files — a four-way exact match under independent reproduction).

The loss figure attached to it does not reproduce and must not be quoted. The headline said "809 of 291,278 block tokens (0.28%)". The verifier could not reproduce either numerator or denominator under any token definition: top-level blocks excluding space give 250 lost of 107,212 (0.23%), and a raw top-level count including space shows the naive join gaining 609 tokens. Quote the delta-0 result, which is exact; do not quote the loss figure.

This separator carries the breakpoint marker the container needs (§3.7), so the split is exact and mechanical rather than heuristic. It is the same component in both capabilities. Build it once.

3.5.6 CLI surface

mdmax project <dir> [--index --outline --inline] [--budget <bytes tokens>] [--json]
→ the projection on stdout
→ a trailing OMISSION BREADCRUMB naming every document degraded to a link and why
mdmax project <dir> --explain # per-document: bytes, tier chosen, reason

3.5.7 Failure modes

FAILURE	BEHAVIOUR
Budget below the index size	refuse; print the index size and the budget
A member ends inside an unclosed fence (4 files, 0.369% of the corpus)	the fence-state-aware separator handles it; a naive join does not
Anyone stores a projection	forbidden ; §2.4
Anyone quotes a size ratio as a token saving	forbidden ; §3.5.3

3.5.8 Kill conditions

- Index + selective read does **not** beat naive truncation at 8K and 32K budgets → ship good filenames and let the consumer grep. This is Anthropic's own stated design: "CLAUDE.md is "naively dropped into context" and everything else is glob and grep, "effectively bypassing the issues of stale indexing." [primary]
- The gap is already < 3 pp at 128K → the whole selection apparatus buys nothing for any corpus under ~1M tokens; ship a packer with a byte counter, not a compiler with a priority function.
- Ranked selection ties random selection at the same budget** → the ranking is decoration; ship budget enforcement, near-duplicate collapse and honest omission breadcrumbs, and nothing else.

3.5.9 What this does NOT cover

No priority function is specified here, and none should be built until it beats random. The graph-based half of any ranking is dead on arrival for this corpus: median out-degree **zero**, 597 files (55.1%) with zero outbound resolved edges, 502 files (46.3%) fully isolated [measured, v2.0.0 §3.2] . Also out of scope: any claim about model distraction from near-miss content. The "up to 65%" GSM-NoOp collapse was re-tested at n=117 with an audited drop of 0–2 pp — **but that null cannot exclude a true 6–12 pp effect (MDE at 80% power is 5.8–14.2 pp)**, and the audit deleted the independent variable. **Neither direction is established. Do not build on either.**

3.6 Capability 6 — PROVENANCE

3.6.1 What it does

Record where a document's content came from, when it was generated, when it was last verified, and when it goes stale — **using someone else's vocabulary.**

3.6.2 The decision

DECIDED: emit Open Knowledge Format (OKF) v0.2 §5 fields — sources , generated , verified , status , stale_after — and read them back. Do not invent a provenance vocabulary.

[v2.0.0 §3.3 and §10.5; final-gate moat and agentic-docs areas]

3.6.3 Why — the slot is occupied by a well-resourced incumbent

Open Knowledge Format v0.2, GoogleCloudPlatform/knowledge-catalog , file okf/SPEC.md :

size	37,544 bytes	(verified by direct fetch, HTTP 200)
licence	Apache-2.0	
stars	8,134	
created	2026-05-04T16:36:24Z	
pushed	2026-07-29T07:04:35Z	

[primary — fetched and read in full by at least three independent agents]

Its §1, verbatim: "OKF v0.2 makes provenance, trust, lifecycle, and attestation first-class while keeping the format minimally opinionated."

SECTION	FIELD	CONTENT
§5.1	sources	resource , id , title , author , usage_count , last_modified , usage_window
§5.2	generated / verified	generated: {by, at} · verified: [{by, at}]
§5.3	trust tiers	unverified / machine-confirmed / human-reviewed
§5.4	status	lifecycle
§5.5	stale_after	an absolute date , not a TTL
§10	attested computations	—

Its §5.1 rationale is our own anchoring rationale, verbatim: "Labels are keyed rather than positional (sources[0]) because agents constantly rewrite these documents: a positional index misattributes silently the moment the list is reordered, whereas a stable id survives reordering." Its per-claim carrier is an **ordinary markdown footnote whose label is a** sources[].id — The events_ table is sharded daily as events_YYYYMMDD.[^ga4-schema] . [primary]

Ecosystem, and the clone-latency lesson. GitHub repo search for the exact phrase "Open Knowledge Format" : **total_count 287**. Third-party toolkit scaccogatto/okf-skills was created **2026-06-14T11:15:31Z — one day after Google's 2026-06-13 announcement** — and now has 206 stars and 22 forks; serradura/okf-gem 115 stars, created 2026-07-11. Creation windows: 69 / 66 / 89 / 59. **The empirical clone latency for a big-company markdown convention is under 24 hours.** [measured, kill audit — "CONFIRMED and strengthened", every figure reproduces]

And the thing nobody wrote down: the same eight days produced competitors for the **product**, not just the convention — activetwist/0nyxWriter ("Local-first editor for Open Knowledge Format bundles") and pothos-dev/sunstone ("a lightweight markdown editor with first-class Open Knowledge Format" support). [primary, kill audit]

3.6.4 The limit of OKF, which must be decided rather than absorbed

OKF has zero confidence fields and zero method fields. A case-insensitive grep over the canonical 37,544-byte `SPEC.md` returns: `span 0`, `"character offset" 0`, `"byte range" 0`, `per-paragraph 0`, `"inline annotation" 0`, `confidence 0`. [measured, kill audit – PERFECT REPRODUCTION] §5.1 explicitly *declines* to store a credibility score, calling it “*subjective, unportable across consumers, and goes stale.*”

Its glossary, verbatim: “*Credibility signal: An objective, per-source fact (`author`, `usage_count`, `last_modified`) used to infer trust; OKF records the signals, not a verdict.*”

Its unit is a bundle, not a document (§3: “*A bundle is a directory tree of markdown...*”), **its identity is the file path**, and it has **no block identity, no range anchoring, and no splice guarantee**. Its conformance bar is two predicates — parseable front matter, non-empty `type` — and **our corpus is already ~59–73.6% conformant with nobody having read the spec**.

OPEN DECISION. Emitting OKF verbatim **silently descopes per-span provenance** — the “which source, which page, which method, at what confidence” dimension. Choose explicitly: **(a)** emit OKF for document-level provenance **and** keep a separate per-span `{method, confidence}` sidecar record, or **(b)** state plainly that per-span provenance is cut to what OKF covers. **Do not do it by accident.**

If (a) is chosen, the per-span record obeys three rules derived from measurement:

- `method` **is mandatory whenever** `confidence` **is present** — lifted verbatim from ITS 2.0's `annotatorsRef` requirement: a score with no annotator is refused.
- Split VERIFIABLE from ASSERTED.** The block anchor and the quoted span are verifiable by recomputation. `method`, `confidence`, `agent`, `timestamp` are **attestations**: emitted only by the producing tool, **immutable downstream** — a hand-edited confidence is a lie the format cannot detect, so the CLI must refuse to write one.
- Never emit what you did not observe.** No default values. An absent field means UNKNOWN, never zero, never true. Learned Rule #59: `bool(None)` is False and is indistinguishable from a real negative. Report `unattributed` counts as a first-class number.

3.6.5 Why C2PA §A.9 front matter is unusable

C2PA 2.4 §A.9 is the one place a standard privileges markdown, and it **cannot be used as specified**:

- Its `-----BEGIN C2PA MANIFEST-----` armour **is not valid YAML** — two parsers throw on it.
- It **collides with the** `---` fence, so `gray-matter` (**7.49M weekly downloads**) silently drops **all** front matter and **leaks the manifest URL into the rendered body**.
- It **renders as a visible** `<h2>` **with a clickable link** in GFM.
- It permits **at most one manifest per file**, bound to a **whole-file byte hash** — which a splice-only editor invalidates on every keystroke.
- Zero implementations exist.**

[measured + primary, provenance-boundary area] C2PA has had a public specification repo since 2021-11-12 and `c2pa-rs` was pushed 2026-08-01, so this is not a dead standard — it is a live standard whose markdown binding is unusable. **Signing, if it ever ships, is a whole-file external C2PA manifest emitted beside the document** (`mdmax prov c2pa --emit FILE.c2pa`), **never the front-matter armour**.

3.6.6 The anti-decay mechanism — why this channel would stay honest here

The governing law is that a channel nobody can compare against reality decays to whatever is cheapest to emit. Measured: **547 of 547 self-declared** `updated:` **dates in a real, automation-heavy repo are stale, 32.9% by more than a week.** [measured]

The answer is **not** a stricter schema. It is that **MDMAX's product is the editor**: a gutter chip on every block carrying a record — `text-layer`, `ocr 0.62`, `model 0.71`, `you` — clickable to the source. PDF tagging, ARIA and `llms.txt` all rot because the author never sees the second rendering. **No other provenance standard has the editor.** [inference – this is a design argument, not a measurement]

3.6.7 CLI surface

```
mdmax prov check [PATH...] # verify anchors, quotes, source hashes, confidence-without-method
                        --strict # exit 1 on any REFUSED or unattributed-with-confidence
                        --json
mdmax prov cover [PATH...] # coverage: blocks with/without records, by method
mdmax prov strip [PATH...] # remove all records – the escape hatch, always available
mdmax prov c2pa --emit FILE.c2pa # whole-file, external reference form only
```

`mdmax prov strip` **existing and being one command is load-bearing: a format you cannot leave is a format nobody adopts.**

3.6.8 Carrier

- Document level:** OKF's own carrier — YAML front matter. It is **visible in 23/24** bench configurations (§3.3.3), and that is **acceptable here** because OKF is a declared, human-meaningful vocabulary, not a machine payload. Say so; do not pretend it is invisible.
- Per-span (only if the OPEN decision above resolves to (a)):** the **tail-placed orphan link reference definition** — invisible in 24/24, byte-identical through `remark-stringify 11.0.0` and `prettier 3.8.3`, and 0 bytes of HTML in `remark-rehype raw and safe`, `marked 16.4.2`, and GitHub in both `gfm` and `markdown` modes. **Not** the HTML comment: GitHub's blob renderer deletes it (§3.3.3). **Risk register entry:** as of 2026-07-30 `markdown-it` (27M downloads/week) gives the link reference definition a first-class token with a `map: [startLine, endLine]` source range behind `core.ruler.disable('strip_references')` — so its invisibility is now a **default**, not a property of the format. **Pin** `markdown-it@~15` **and add a regression test asserting identical rendered output with the rule enabled and disabled.**

3.6.9 Kill condition

If the product needs per-span `{method, confidence}` and neither option (a) nor (b) is chosen and written down within the first month, **cut provenance entirely** rather than shipping a vocabulary that silently answers a different question than the one it was built for.

3.6.10 What this does NOT cover

No signing in v0.1. No runtime — records are data; nothing is evaluated. No new node types — every record is an existing `html` or `definition` node, and extensibility lives **entirely in the value** (D8). And do not repeat the figure "127 open OKF spec issues": the audit found `is:issue is:open` = 93 and `is:issue` all-time = 97, so **127 exceeds the all-time issue ceiling and cannot be an issue count under any filter**.

3.7 Capability 7 — THE CONTAINER (`mdmax pack` / `mdmax unpack`)

3.7.1 What it does

Serialise many `.md` files into **one** `.md` file with machine-recoverable breakpoints, and split it back apart byte-identically.

3.7.2 Status: SEQUENCED, not cut

RECOMMENDED, and sequenced AFTER capabilities 1 and 2. Not in the MVP. It ships as a **bounded, read-only, machine-facing transport** — never a durable committed artifact, never the write surface, and never a "pack my whole vault" button.

The founder's image, verbatim from the session transcript (`role=user` , re-extracted independently from the 58,919,587-byte session `jsonl` , verdict **CONFIRMED**):

"...whether the user should download one zip file of all the Markdowns or one single mother Markdown, which will be very long. Whenever frontmatter or the AI reads that, I will know these are the breakpoints, which means different files, and it can split that Markdown automatically while parsing into different files."

The verbs are transport verbs — download, split, read, parse — not retrieval verbs. That is what makes this a container question rather than an index question.

3.7.3 The measured design

Round trip, whole pinned corpus:

n = 1,084	raw 25,548,765 B	container 25,776,700 B	overhead +0.892%
byte-identical 1084/1084		pack 96 ms	unpack 169 ms
overhead decomposes as: manifest 169,706 B (62.9% of overhead) + framing 92.5 B/file			

[measured, corpus_id sha256:3a010b16... — replicates the earlier 6-file +0.99% result at 180× the file count]

Single file, not zip — on evidence, not taste:

PROPERTY	FOLDER	CONTAINER (SINGLE <code>.md</code>)	ZIP
git storage of ONE one-line edit (272-file corpus)	66 B	263 B	31,590 B — 120×
<code>git diff</code>	text	text, reviewable	"Binary files differ"
gzipped	—	7,874,098 B (30.8%)	9,310,058 B (36.4%) — 15.4% worse
index position	—	head, byte 127,394 of 25,776,700 = 0.49%	tail, 9,200,529 of 9,310,058 = 98.8%
500 MB build (21,680 files / 510,975,300 B)	—	1.91 s (270 MB/s)	9.7 s
partial extract of one file	—	38.48 ms	2.94 ms

[measured, corpus_id sha256:3a010b16...] Zip wins exactly one thing — random access — which neither lane needs. **Ship a zip too, but as a different product: the D4 escape-hatch export, one-shot, never versioned, plain files, no manifest.** That is what a departing human actually wants.

Carrier — the authoritative path goes in the FENCE INFO STRING, never in heading prose.

Measured: a `## File: <path>` heading survives as a *breakpoint*, but **13 of 60 path strings were corrupted by one pass through a standard markdown writer** — e.g. `md/Skills/Printing Press/Pp Docker Hub/workflow.md` → `..workflow\..md`.

CORRECTION TO THE BRIEF, and it matters. The claim that this corruption is **non-local** — that the identical heading is byte-stable in isolation and unstable inside the bundle — was **REFUTED** by the verifier. The cause is purely local: `mdast-util-gfm-autolink-literal/lib/index.js` lines 61–63, read verbatim, give `character: '.'`, before: `'[Ww]'`, after: `'[\-.\\w]'` — a **one-character lookbehind** guarding against `www.` autolink formation. So `workflow.md` escapes (a `w` precedes the dot) and `skill.md` in the same directory does not; `x/wow.md`, `x/sow.md` and `x/Now.md` all escape. The original control reproduced "OK" **only with the GFM extension disabled** — two different library configurations were compared and the difference attributed to context. **The Wagner & Graham (TOPLAS 1998) context-dependence citation is unsupported by this evidence.** The design conclusion — the path belongs in the fence info string — **survives on plain fidelity grounds (13/60)** and on the verifier's own check that the info string preserved all four test paths intact.

Delimiter — a computed backtick run.

MEASUREMENT	VALUE
fence lines in the corpus, all exactly three backticks	15,435
tilde fences	0
line-initial backtick runs ≥ 4	0
any-position backtick runs = 4	4 (all inline, <code>docs/engine/PLAN.md</code> lines 652 and 1042)

[measured, corpus_id sha256:3a010b16... — the container-thesis and founder-thesis areas measured two different things and the founder-thesis verifier flagged the collision]

State the rule precisely: `N = max(4, max LINE-INITIAL backtick run over every contained byte + 1)`. For this corpus that is **4, and it is provable**. The looser wording "max backtick run + 1" computes **5** on the same corpus and contradicts its own conclusion. Prior art: `repmix src/core/output/outputGenerate.ts:60`, verbatim — `const maxBackticks = contents.flatMap(c => c?.match(/ +g) ?? []).reduce(...); return " ".repeat(Math.max(3, maxBackticks + 1));` — which uses `max` run **anywhere. Adopt it, credit it, and document the difference.**

Per-entry metadata, all mandatory, each justified by a measured failure:

FIELD	WHY	MEASUREMENT
path (JSON-quoted)	a whitespace-delimited info string breaks on the majority of the corpus	598 of 1,084 paths (55.2%) contain a space; 18 path characters are em-dashes; maxpathlen 134
sha256	the only per-file authority	1084/1084 byte-identical proven per file
bytes	the fast path and the tamper detector	—
nl: 0 1	naive concatenation silently welds file N's last line to file N+1's first	288 of 1,084 files (26.57%) have no trailing newline
—	zero-byte members need their own sentinel	4 files are 0 bytes; 1 uses CRLF

Use git's own `\ No newline at end of file` sentinel. With the sentinels in place the round trip is **1084/1084 byte-identical**.

Manifest: YAML front matter, ONE LINE PER FILE. Not cosmetic — a **47× reduction in diff size**: a compact single-line JSON manifest makes every content edit rewrite the whole 169 KB manifest line (**86,252 diff bytes**); one line per file makes the manifest hunk name the changed file (**1,833 diff bytes, 2 hunks**). [measured]

Pack-time refusals, each a named error with a fix:

- Unterminated fence in a member. 7 files (0.65%) contain an odd number of ``` delimiters** — each is a poison pill that swallows every following breakpoint. Separately, **4 files (0.369%) end inside an unclosed fence** — e.g. `md/Brain/20-projects/proj-advoc-frontend.md`, whose line 32 is literally truncated mid-word (`# Rea`). These are two different tests; report both. **Refuse, do not guess.**
- A path that is not representable: absolute, drive-lettered, `..`-bearing, non-canonical, containing a control character, or a **case-folded duplicate** (APFS is case-insensitive by default; the corpus is clean today — 1,084 distinct NFC, 1,084 distinct case-folded NFC, 0 collisions, 0 NFD-vs-NFC differing paths — **so the check must exist before it is needed**).
- `E-RUN-UNDERDECLARED`: a contained body holds a run $\geq$ the declared fence. This is the hole the research's own verifier had — its "fence run lie" fixture returned OK because the reader **hardcoded** the run instead of deriving it from the header **and** cross-checking it against the contained bytes. **A header field that is trusted is a capability handed to whoever wrote the header.**

Injection is closed by construction, not by escaping. An attack file containing a literal ````text fm:file 99 ".../.../...ssh/authorized_keys"` forces the packer to escalate to a 5-backtick fence. A reader **pinned to the declared run** recovers exactly 3 files, the evil file byte-identical, **0 forged paths**. A naive `{3,}` line scanner accepts the forged path. **The spec must mandate the pinned run and forbid any-fence scanning.** [measured]

A "How to read this file" preamble is mandatory. It is what lets a model with no MDMAX installed unpack correctly, and it is the degradation story. Precedent, verbatim from `repomix's markdownStyle.ts`: `## File Format ... "5. Multiple file entries, each consisting of:" | "a. A header with the file path (## File: path/to/file)" | "b. The full contents of the file in a code block"`, plus `# Directory Structure`. [primary]

Collision check: 0 files in the corpus contain a literal ## File: x line — the `repomix` breakpoint convention is collision-free here. **Exactly 1 file contains the literal token `fm:file / fm:endfile` — our own `docs/engine/PLAN.md`.** The computed fence run makes both inert.

Nesting is free. The fence run is itself a nesting counter: packing a container inside a container auto-escalates $4 \rightarrow 5 \rightarrow 6$ and recovers byte-identically at three levels. **Delete `docs/engine/PLAN.md` \$5's "a container inside a container is refused in v1"** — the `tar/shar/MIME` "flatten, never nest" law is about the **path tree**, not about container-in-container, and it was over-applied.

3.7.4 The D6 argument, and it is the reason this is not a dialect

A bundle invents a CONVENTION — what a sequence of ordinary constructs means — not a DIALECT — new syntax a parser must understand in order to read the document. `tar` did not invent a filesystem. `MIME multipart` did not invent email.

Nothing in the container requires a new parser. A bundle opened by any markdown tool is a document of headings and code blocks; pasted into a chat box it is the whole corpus; opened in `frontmatter` it is a project.

The honest cost of that, measured: under the fence carrier the container **stops rendering. 285 of 288 rendered characters land inside `<pre><code>`, with 0 headings, 0 links, 0 tables, 0 bold, 0 list items**, versus a fully rendered document under the `HTML-comment` carrier. [measured, marked 16.4.2] Mitigation, verified: a visible `#### \ docs/a.md`` line above each fence renders as

`docs/a.md`

``` — an anchored, navigable, human-visible label whose **authority is still the manifest. Delimiter  $\neq$  label**. If a model mangles the label nothing is lost.

3.7.5 The counter-evidence, stated in full

This is the capability with the strongest case against it in the entire programme. All of it must be in the plan.

- The founder's own corpus does not show him doing this by hand.** The claim that 403 of 1,084 files (37.2%) / 12,193,061 of 25,548,765 bytes (47.7%) are hand-built multi-document bundles was **REFUTED: 372 of the 403 (92.3%) are files with exactly two H1s whose first H1 is a machine-generated YAML title: field**, and **365 of 403 live in one machine-synced subtree, `md/Skills/**` ( source: claude , last\_synced: 2026-05-26 ). Honest figure: ~2.2% of files and ~2.0% of bytes**. The area's own pre-registered falsifier fired and was recorded as *"FAILED TO REFUTE, in the founder's favour."*
- The market voted for pack and against unpack** — `repomix` **27,551 stars** (created 2024-07-13, pushed 2026-08-01) with **no unpack subcommand**, against a sole zero-star unpacker. **BUT that "27,551 to 0" was itself REFUTED:** `Aider (47,848 stars)` already ships the return trip. `aider/coders/wholefile_prompts.py` specifies the format verbatim — *"First line: the filename with any originally provided path... Second line: opening {fence[0]}... entire content of the file... Final line: closing {fence[1]}"* — and `wholefile_coder.py:get_edits()` parses it back and writes each file to disk, already stripping `*`, `:`, backticks and leading `#` from the locator with a `len(fname) > 250` guard. `base_coder.py::choose_fence()` already computes a collision-proof delimiter. `gpt-engineer` (55,173★) and `smol-ai/developer` (12,182★) are the same category. **The unpack half is a re-implementation, not greenfield, and the delimiter rule is pre-empted.**
- The one library that ships both halves gets no users.** `llm-code-format` v3.1.0 (published 2025-09-20, zero dependencies, TypeScript, *"From source code to Markdown to LLMs and back again"*, `parseMarkdownFiles / formatMarkdownFiles`, a streaming parser, nine recognised LLM output conventions): **387 downloads/month against repomix's 327,543. 846:1.** [primary]
- The incumbent is strengthening, not weakening.** `repomix` npm downloads went **119,678 (2025-07) → 327,053 (2026-07), 2.73× year on year, the highest month in the package's history.**

5. **Revealed maintainer priority is the inverse of our premise.** repomix issue **#226 "Support for unpack"**: opened 2024-12-29, **still open, 6 comments, 0 reactions** after 19 months. Issue **#71 "Splitting code into several files"**: 12 comments, **8 reactions**, produced PR #113 — and repomix **shipped** `src/core/output/outputSplit.ts`. The maintainer's own conclusion (#226, 2024-12-31): *"If you're using a JSON output or something similar, then unpacking would probably be easy, but otherwise, the format would be difficult to define, so it might be hard."*
6. **The human lane is dead on its own forum.** `forum.obsidian.md` `t/81878` *"How do I export an entire Vault to a Single File?"* — created 2024-05-15, **9,010 views, 1 like, 3 posts in 26 months.** `t/96068` — 849 views, 1 like, 4 posts. For contrast, the "Share Note plugin" thread has **172 posts. 9,010 views says people look; 1 like and 3 posts say nobody wanted it enough to reply.**
7. **A vault-scale container fits in no context window.** Container **25,776,700 B** vs described index **234,119 B = 110.1x**. A 200K-token budget holds **2.79%** of the container and **308%** of the index; a 2M-token budget holds 27.9%. Mean file 23,568 B, so a 200K pack carries about **thirty average files**. (The 3.6 bytes/token constant is an assumption — no tokeniser was available — but the conclusion is robust across 3.0–4.0 B/tok.) **If MDMAX ships a "pack my whole vault" button, it ships a button that manufactures disappointment.**
8. **The container is not a durable artifact.** A **4-backtick wrapper DOWNGRADES to 3** through `mdast-util-to-markdown`, and **496 of 1,084 corpus files (45.76%) contain a ``` fence**, so any member with an internal fence **terminates the wrapper early after a single normalising pass through any third-party markdown tool** — which is exactly where the artifact is supposed to travel. Aider avoids this by **re-deriving the fence at emit time** rather than trusting a delimiter written into a stored file. **⇒ the container is an EPHEMERAL READ TRANSPORT, generated on demand, never committed.**

3.7.6 Boundary destruction — refuse and localise, never repair

Detection is certain and cheap: delimiter count vs manifest length, dense-index check, path agreement. **Repair is not.**

| CONDITION                                                                     | RESULT                                                                                                                  |
|-------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Digest recovery when at least one neighbour is unedited, p(edited)=0.5, n=400 | <b>exact 303/400 = 75.8%</b> — matching the theoretical $1 - 0.5^2 = 75\%$                                              |
| Both neighbours edited                                                        | bracketing always contains the true split, but the window is <b>median 17,309 B, p90 71,206 B</b> — correct and vacuous |
| Model rewrites across the boundary ( <code>merge_across</code> ), n=120       | <b>119/120 WRONG</b>                                                                                                    |
| Model elides unchanged regions ( <code>elide_unchanged</code> ), n=120        | <b>119/120 WRONG</b>                                                                                                    |
| Model reorders, n=120                                                         | <b>120/120 WRONG</b>                                                                                                    |
| Truncation, n=120                                                             | 47/120 destroyed outright                                                                                               |
| Clean delimiter deletion, n=120                                               | 63 exact, 50 within a window, <b>7 WRONG</b>                                                                            |

**"WRONG"** means it returns a confident boundary position that does not contain the true split — i.e. **silent corruption**. `[measured, seed 20260801]`

The researcher's own first measurement of this flattered the design by **99 percentage points** (200/200 recovery under a benign destruction model) and they reported the reversal against themselves. **Anyone who re-derives the earlier "diff-and-rebase" prototype's numbers should assume they are benign-case numbers until a hostile fixture proves otherwise.**

Therefore the behaviour is:

*"The boundary between A and B was destroyed. The manifest byte count puts it near line X — within 10 lines with 78% confidence. Here are the two candidate splits. Confirm."*

The best available localiser is the manifest byte-count prior: **median 3 lines, p90 18, max 192; within 0 lines 13%, within 3 lines 52%, within 10 lines 78%**. A suggestion, never a repair.

**Prevention beats repair, and this is the product decision. Never hand a model a pack and ask for the pack back.** Ask for per-file whole-file listings in Aider's shape — path line, fence, full body, fence — which is the format models are most reliably well-formed in: on Aider's polyglot leaderboard the `whole` format is well-formed at **median 99.6% (min 92.9, max 100.0, n=15)** while `diff` is **median 94.2% with a min of 64.4% (n=47)**. `[measured, aider/website/_data/polyglot_leaderboard.yml, 69 rows]` That shape is structurally identical to one entry of our own container, so **one scanner serves both**, and the manifest then validates the return: every returned path must be in the manifest, every unreturned file is unchanged by definition, a returned file whose digest equals its pre-edit digest is a no-op.

3.7.7 Split contract and CLI surface

```
manifest YAML front matter: fmpack: 1 · fence: N · root: · then ONE LINE PER FILE
 - {i: 0, bytes: 5864, sha256: "61397d27...", path: "AGENTS.md", nl: 1}

open <F>text fm:file <i> <json-quoted-path> at column 0
close <F> at column 0
ordering manifest order == physical order == index order, dense 0..n-1
```

```
mdmax pack <paths...> --budget <tokens|bytes> [--root DIR] [-o FILE]
 → refuses above the target model's window and prints the index size instead

mdmax unpack <FILE> --into DIR [--dry-run] [--json]
 → per-file verdict: clean | edited | reframed | framing-damaged
 → TRANSACTIONAL: verify everything, then write, never partial

mdmax pack --explain # which fence run was computed, and why
```

**Rejoin contract — dual authority.** Byte count is the fast path and the tamper detector; the fence is the human-editable frame. Agreement → `clean`. Digest matches the byte-count slice only → `framing-damaged`. Digest matches the fence slice only → `reframed` (a human edited the file and the count is stale — accept and rewrite the manifest). Neither → `edited`.

**E-codes refuse the entire unpack and write nothing:** `E-NOHEADER`, `E-UNTERMINATED-MANIFEST`, `E-EMPTY-MANIFEST`, `E-COUNT-MISMATCH`, `E-INDEX-NOT-DENSE`, `E-INDEX-RANGE`, `E-PATH-DISAGREE`, `E-ABSOLUTE-PATH`, `E-TRAVERSAL`, `E-NONCANONICAL`, `E-DUP-PATH`, `E-CASE-OR-NORM-COLLISION`, `E-CTRL-IN-PATH`, `E-UNCLOSED-FINAL-FENCE`, `E-RUN-UNDERDECLARED`. **W-codes unpack but quarantine the file:** `W-EDITED`, `W-REFRAMED`, `W-FRAMING-DAMAGED`.

3.7.8 The experiment that gates the build

**E1 — the round trip. No model calls. One day.** Write the **unpacker first**, from the written spec, by a person or agent who has **not seen the packer** (Learned Rule #60). Then, for all 1,084 files of `corpus_id sha256:3a010b16...` and 5 bundle sizes (20/50/100/200/400 members, 3 random draws each, seeds recorded), assert `unpack(pack($)) == $` byte-identically for every member — **and then assert it still holds after the bundle has passed through one `fromMarkdown / toMarkdown` round trip.**

Pre-registered predictions: (a) heading-carried paths fail at 15–25% of members after the round trip (measured point 13/60 = 21.7%); (b) fence-info-carried paths fail at 0%; (c) trailing-newline loss is the second-largest defect class and is invisible without the `nl` flag; (d) the 7 unterminated-fence files are the only pack-time refusals in the corpus.

**Make it go RED first (Learned Rule #68):** the heading-carried arm must reproduce the `workflow.md → workflow\..md` corruption before any green result is trusted.

3.7.9 Kill conditions — five, any one of which fires

- 1. An **independently written** unpacker cannot reach **100.00% byte-identical recovery per file** on the pinned corpus at every phase gate. **Not 99.9%.** 1084/1084 is achievable today with a 96 ms packer, so any regression below it is a defect, not a trade-off.
- 2. Any shipped unpacker can be induced to write a path outside the target root by content inside a contained file. **One instance kills the feature** until the run-pinning invariant is proven by a fixture that **fails against the unfixed reader** — the naive-reader test is that fixture and it must stay in CI.
- 3. Boundaries survive at ≥ 90% for 100-file packs **and** repomix ships an unpack inside the build window → the differentiated part is gone; we are re-implementing a 27,551-star incumbent.
- 4. The product ships a **"pack my whole vault"** affordance → it manufactures a 25 MB artifact of which 2.79% fits a 200K window and which is 110× more expensive than the index answering the same question.
- 5. Six months after launch, **unpack invocations are under 5% of pack invocations** → that is `llm-code-format`'s 846:1 reproduced in our own telemetry. Keep pack, delete unpack, stop paying for the manifest.
- 6. **K-D, and it costs one conversation:** if, shown the four constructions side by side, the founder says the export he meant was always *"a zip of clean `.md` files"* and never a single artifact, then the mother-markdown was a metaphor and four sessions of hearing it were four sessions of hearing what we wanted.

3.7.10 What this does NOT cover

No transclusion. No `fm:vocab: .` No cross-file reactive state. **No capability whose payload is by reference** — those measure `VOID` under plain rendering (§3.3.5) and are therefore excluded by D6 until someone decides D6 has an exception and writes it down. And the container is explicitly **not** D4's escape-hatch export: that is a plain zip of plain files, one-shot, never versioned.

3.8 Cross-cutting prerequisites — smaller and more boring than any capability above

These are not capabilities. They are the things that will bite in week one, and three of them must land **before** the first anchor is persisted.

3.8.1 The offset model — one unit, one boundary policy, one conversion point

**Only 67 of 1,080 files have** `bytes == UTF-16 code units == code points`. **93.8% already diverge.** `[measured, corpus_id sha256:3a010b16...]` Separately, **103 of 2,314 files (4.5%)** in these vaults contain non-BMP characters `[measured, engine PLAN §3.3]`.

|            |                                                                                                                                                   |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| UNIT       | UTF-16 code units (what CodeMirror and mdast use)                                                                                                 |
| BOUNDARY   | an offset may NEVER fall inside a surrogate pair.<br>Reject at construction. NEVER round.                                                         |
| CONVERSION | exactly one place: a per-open-document OffsetMap holding a<br>Uint32Array UTF-16-byte prefix table plus a lazy<br>Intl.Segmenter grapheme bitset. |
| TYPES      | branded: U16Offset   ByteOffset   GraphemeIndex — they cannot mix.                                                                                |
| LINT       | an eslint no-restricted-syntax rule forbidding Buffer.byteLength,<br>TextEncoder, codePointAt and raw .slice on a document anywhere else.         |

The bug appears at **every boundary where a byte offset enters**: a git blob, a `Buffer`, a content hash over a byte range, a Rust implementation, or an LSP that negotiated `utf-8`.

3.8.2 Version the two pure functions everything else keys on

|                          |                                       |                       |
|--------------------------|---------------------------------------|-----------------------|
| <code>normalize/1</code> | NFC + collapse whitespace + lowercase | → "mdmax/normalize@1" |
| <code>slug/1</code>      | the heading anchor algorithm          | → "mdmax/slug@1"      |

Both are **published, pinned, and stamped into every stored anchor and every certificate row.**

- `normalize/1`: the 99.627% was swept over 384 configurations under exactly NFC + collapse whitespace + lowercase. **If the hash input changes later, every stored anchor silently re-keys while the published number keeps describing code that no longer exists.** This must land **before the first anchor is persisted**; afterwards it is a migration.
- `slug/1`: **anchor health swings 77 percentage points on the choice.** The same corpus's 392 intra-document anchors resolve **86 under `github-slugger` and 388 under a dash-collapsing slugger** — and one research report used both algorithms in two adjacent claims without noticing. **"Pick and publish the anchor algorithm" is the single highest-leverage decision in the linker and it costs one afternoon.**

CI assertion: **the `VERSION` constant must change whenever the function's output over a 200-case golden file changes.**

3.8.3 The YAML value slot that D8 rests on

D8 says extensibility lives in the **value** of a field. **The value slot is broken in the founders' own vault:**

- **170 of 907 front-matter blocks are not valid YAML**, because `related: [[a]], [[b]]` puts markdown's grammar inside YAML's.

- The quiet variant `[[[A]], [[B]]]` **parses successfully into a nested array and destroys the links with no error.**

`[measured, corpus_id sha256:3a010b16...]`

**Required: a lenient front-matter pre-pass that quotes bare `[[...]]` runs before handing the block to YAML, a hard refusal for the silent-corruption case, and a rule that a front-matter parse failure may NEVER fail a user action.** Note that §3.1's splice path avoids the parser entirely, which is exactly why it is the fix for the 171 unreadable blocks.

3.8.4 The shape gate, the worker budget, and two live quadratics

Two quadratics are shipped and measured, both against this repo's own `node_modules`:

| SITE                                                       | MEASUREMENT                                                                           |
|------------------------------------------------------------|---------------------------------------------------------------------------------------|
| WIKILINK_RE                                                | k = 1.98 — <b>36,865 ms on 320 KB of</b> <code>[[</code>                              |
| <code>mdast-util-from-markdown</code> on flat bullet lists | <b>12,429 ms vs micromark's 1,207 ms on identical bytes — a 10.3× gap that widens</b> |

`[measured]` And CommonMark is linear in practice but **quadratic under adversarial input**, patched case by case (cmark #373, #389, CVE-2023-22484) `[primary, engine PLAN §3.2]`. **A compiler will be fed hostile documents.**

Before any grammar runs: strict UTF-8 decode (**refuse invalid, never repair**), BOM strip, and refuse by **shape** if `bytes > 4 MB` **or** `lines > 200,000` **or** `list-marker lines > 20,000`. All parsing happens in a long-lived `worker_threads` pool with a **250 ms keystroke / 2,000 ms cold-open / 10,000 ms batch** budget enforced by `worker.terminate()`. Failure surfaces as a discriminated union — `{ok: false, reason: 'BUDGET_TIME' | 'BUDGET_BYTES' | 'BUDGET_BLOCKS' | 'PARSER_THREW' | 'WORKER_DIED', at: {line, col}}` — **never as an exception in a route handler.**

3.8.5 A placement fixture, before any byte is written into a user's file

**Measured defect in a design proposed by this research, found by its own verifier:** inserting a marker adjacent to a setext heading **silently changes the document's meaning, and blank-line isolation does not prevent it.** Verified with this repo's own parser:

```
"Heading\n---" → heading(depth=2)
"Heading\n<!-- mdmax:begin id=k -->\n---" → paragraph, html, thematicBreak
"Heading\n\n<!-- mdmax:begin id=k -->\n\n---" → paragraph, html, thematicBreak ← SAME CORRUPTION
```

An `h2` **becomes a paragraph plus a horizontal rule.** Any CLI verb specified purely on byte ranges is directly exposed to this. **Every byte written into a user's file passes a placement fixture first** — this is house rule P5, and it exists because of this exact case.

3.9 What MDMAX is NOT — the cut list, so nobody re-opens it

| CUT                                                                                                                    | WHY                                                                                                                                                                                                                                                                                                                                                                                                           |
|------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MDMAX as a compiler <b>programme</b> — a spec, a format profile, a published benchmark, a CLI beyond <code>cert</code> | it is a ~600-line library subordinate to the editor                                                                                                                                                                                                                                                                                                                                                           |
| A notation library, a vocabulary registry, any declared-vocabulary syntax                                              | measured user-invention rate <b>0.31%</b> , and <b>all of it is tutorial content teaching the feature</b> . 99.69% of the 4,538 <code>:::</code> directives in Docusaurus's own repo are among its 9 vendor-shipped keywords; 847 independently authored npm docs use exactly GitHub's 5 built-in callout types with <b>zero</b> inventions                                                                   |
| The doc-tree folder compiler and its incremental engine                                                                | cold full build <b>~1 second</b> ; maximum transitive blast radius <b>54 of 1,084 files</b>                                                                                                                                                                                                                                                                                                                   |
| <b>Markdown as a graph</b> , and the graph view                                                                        | median out-degree <b>zero</b> ; 502 files (46.3%) fully isolated; 511 connected components; genuine content transclusion <b>ZERO</b> in 25.5 MB (all 45 <code>[[...]]</code> occurrences are syntax documentation). Zero occurrences of "graph" across 89 sourced feature requests                                                                                                                            |
| A provenance vocabulary of our own                                                                                     | §3.6                                                                                                                                                                                                                                                                                                                                                                                                          |
| The "fewer tokens" promise to AI consumers                                                                             | §3.5.3                                                                                                                                                                                                                                                                                                                                                                                                        |
| Publishing 99.627% before it is re-derived                                                                             | §3.2.7                                                                                                                                                                                                                                                                                                                                                                                                        |
| <code>EditableTable</code> / <code>setTableCell</code>                                                                 | dead code, imported by nothing, 13.4% no-op dirty rate, silent pipe-escape corruption. <b>Remove rather than fix</b>                                                                                                                                                                                                                                                                                          |
| Multi-agent concurrency control                                                                                        | 29 of 724 written files (4.0%) touched by more than one session ever; <b>0 files by two sessions within 60 minutes</b> across 1,200 transcripts. <b>CAVEAT — do not over-cut:</b> that population is one operator's <i>agent</i> sessions and structurally cannot contain the multi- <b>human</b> co-editing D1 exists to serve. Cut agent-vs-agent concurrency; <b>do not cite it against human presence</b> |

**One correction to the record while cutting.** The founder-thesis area justified a "program-wide blind spot" by quoting `src/modules/graph/presentation/graph-data.ts:132` as `if (targetPath === undefined) continue; // unresolved — drop`. **That line does not exist.** The real branch is at line 176 and the comment above it reads, verbatim (re-read this session):

```
// Unresolved links are not edges — there is no node to point at — but
// they are a diagnostic, so they are reported rather than dropped.
```

The module already ships a typed `UnresolvedLink[]`, a `reportUnresolved()` function, and a `reason` field discriminating "target-excluded" from "no-such-note". The `— drop` comment quoted exists at line 188, on the **duplicate-link** branch. **The paraphrased-as-verbatim code quote is itself a defect, and the artifact cited does the opposite of what the claim asserted.**

### 3.10 How this section could be wrong

1. **The verifiers that produced most of the corrections above have a measured false-kill rate of 0–19%** (105 kills sampled across four audits: 17.2% / 3.4% / 19.0% / 0%). A kill is strong evidence, not proof. Where a kill in this section looks wrong to a builder, re-derive it rather than deferring to it.
  2. **The corpus is one team's writing.** 416 commits by one author over 80 days. Every resolution rate, every carrier prevalence, every fence statistic is a property of that corpus. **The two-tier link split (§3.4) and the anchor number (§3.2) both need a second corpus this team did not write before either is published.**
  3. **Numeric drift is one-directional across the whole research base.** Six independent numeric non-reproductions in the `markdown-base` area alone all moved in the direction that flattered the thesis; the `founder-thesis` verifier found the same pattern four times out of four on interpretive labels. The structural conclusions in this section reproduced under independent re-derivation; **the percentages did not, consistently. Re-derive every percentage before quoting it downstream** (house rule P2: no headline without a committed derivation script and its output).
  4. **The single highest-value falsification available is cheap.** Capability 1's gate (907/907 byte-identical publish-then-unpublish) either goes green or it does not, in half a day, against real bytes. **If it does not, every other capability in this section is downstream of a writer that does not work**, and the correct response is to stop and fix the writer rather than to proceed to capability 2.
-

54 Representation

Representation – what a markdown file can be made to hold, convey and mean

← Index · ← §3 Capabilities · §5 Rendering →

4. Representation — what a markdown file can be made to hold, convey and mean

This is the carrier and capability inventory. It answers one question exhaustively: **given that the file must remain an ordinary .md file (decision D6), what can be put into it, where, how much, who sees it, and what destroys it?**

It is written to be used as a lookup table during implementation. If you are about to write bytes into a user's markdown file, the answer for your construct is in §4.4, and the reason it is that answer is in §4.5 through §4.9. If you are about to *recognise* something already in the file rather than write to it, go to §4.7.

**Read §4.3 first.** It contains the single finding that reframes everything else in this section, and it is uncomfortable.

4.1 How to read the evidence in this section

Every claim below carries a tag. The tags are not decoration; several conclusions in this section reverse when you notice which tier a number sits at.

| TAG         | MEANS                                                                                      |
|-------------|--------------------------------------------------------------------------------------------|
| [measured]  | executed in the research run, output quoted                                                |
| [verified]  | re-executed by an independent adversarial verifier in the same run, and it reproduced      |
| [primary]   | read from the authoritative source (a spec, a vendor's own docs, an installed source file) |
| [secondary] | read from a non-authoritative but credible source                                          |
| [inference] | deduced from measurements, not observed directly                                           |
| [SIMULATED] | replayed through code rather than read from a live system                                  |

**The corpus.** Every "in the corpus" figure in this section is over the pinned corpus:

|           |                                                                                                                                                                                                                                                   |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| corpus_id | sha256:3a010b1649899795d79274fc528dbce97fdabf4ff0f81cc02ab619c048c51a4                                                                                                                                                                            |
| manifest  | docs/engine/research/corpus-manifest.json                                                                                                                                                                                                         |
| size      | 1,084 files · 25,548,765 bytes                                                                                                                                                                                                                    |
| roots     | md 756 files · 21,068,853 B · head 02c22ec47a3a73fb25a14b11be233ad3d00a8b48<br>knowledge 272 files · 3,570,923 B · head 464eb666946c8cb4ccf42c68f4464613e5fc4999<br>frontmatter 56 files · 908,989 B · head 798ebbf3250a5b2a53785859d79ae15f5ebeb |
| policy    | all *.md case-insensitive; skip .git .next .obsidian .trash .venv __pycache__<br>build dist node_modules at any depth; skip any dot-directory; skip top-level<br>Mirrors/ and Paste/                                                              |
| integrity | re-read and re-hashed during verification: verified=1084 sha_mismatch=0 missing=0                                                                                                                                                                 |

[verified]

Seven different local-file counts were reported across eighteen research areas before this manifest existed and **none of them reproduced**. Any figure about local files in any earlier document that does not cite sha256:3a010b16... should be treated as unreproducible until it is re-derived. This section cites the corpus id on every corpus figure and names the denominator on every rate.

**A second corpus exists and it matters.** One research area measured a different, independently selected corpus — referred to throughout the research as **Corpus B**: every \*.md under this repository's node\_modules to depth 3, **847 files / 7,733,433 bytes**, i.e. published documentation written by roughly 850 unrelated third-party authors. [measured] Its numbers disagree with the pinned corpus's numbers sharply and in one case decisively (§4.10, C-1). Where they disagree, this section reports both.

**One class of claim is absent from this section entirely.** Zero live model calls were made in any research run. Every statement of the form "this carrier is easier for an AI to read" is a **prediction**, not a result, and is labelled as such. Nothing here has been measured against a language model.

**Tool versions**, so a reader can reproduce: remark-parse 11.0.0, remark-stringify 11.0.0, remark-gfm 4.0.1, remark-frontmatter 5.0.0, remark-rehype 11.1.2 (both allowDangerousHtml and safe modes), marked 16.4.2, prettier 3.8.3, and POST https://api.github.com/markdown in both mode=gfm and mode=markdown. Every one of those except marked is already a dependency of this repository ( package.json ), so the matrix is re-runnable from a clean checkout.

4.2 What this section does NOT cover

Stated up front, because the gaps change how much weight the table can carry.

- GitHub's blob renderer was never measured.** The GitHub Markdown API's mode=markdown is *not* the blob renderer. Proof that they diverge: GFM alerts render as styled callouts on github.com READMEs and as a plain blockquote with a literal [!NOTE] through the API's markdown mode. [measured] An attempt to confirm via Accept: application/vnd.github.html+json on two repository READMEs returned 0 markdown-alert occurrences, but neither README demonstrably uses an alert, so that probe is **inconclusive, not negative**. **Treat every "GitHub blob" cell in §4.4 as covering the API's markdown mode only.** The comment-box ( gfm mode) results are solid — that surface *is* what the API serves.
- GitHub Pages was never executed.** The gem install of kramdown + kramdown-parser-gfm into a temporary GEM\_HOME failed. Every Pages/kramdown cell is a [primary] **read of** gettalong/kramdown doc/syntax.page (which documents definition lists, "Attribute List Definitions", and {::comment} ), and the specific statement " Term / : definition renders as a real <dl> on GitHub Pages" is [inference] from that document.
- No clipboard was measured.** No browser was driven. The text/plain claims (carriers survive, because that flavour is the raw source) and the text/html claims (invisible carriers are deleted, because they render to zero bytes) both follow deductively from measured HTML output, but **neither was observed**. This is

the weakest column in the table and it is the column §4.8's open problem depends on.

4. **Four of the six comparison vocabularies were not read.** DocBook, JATS, DITA and OOXML. `docbook/docbook` on GitHub now contains only `['MovedToCodeberg', 'README.md']` and Codeberg is outside the research harness's network allowlist; the DITA code-search API returned 401. **Do not repeat any DocBook, JATS, DITA or OOXML claim from memory.** §4.9 states only what was actually established: TEI ( `[primary]` , read verbatim) and HTML5 ( `[measured]` , via GitHub's live sanitizer).
5. **One corpus, two authors and one AI.** The pinned corpus was written by two people and an agent stack. Its 0 link reference definitions, its 0 GFM alerts, its 13,028 wikilinks and its 73.9% bash-fence share are all plausibly the signature of one Obsidian-based workflow rather than properties of markdown authors. §4.11 specifies the second-corpus experiment that would settle this, with pre-registered predictions.
6. **Link-label matching rules are stated but only half-verified.** See §4.6, A1, hazard 6.
7. **The 99.627% re-anchoring figure is quoted twice in this section and has been re-derived by nobody** — not by any of the sixteen final-gate areas, not by any of the eighteen capability areas, not here. Every use of it below carries that caveat inline.

### 4.3 The finding that reframes the whole section

The best carrier by every objective metric has exactly zero users.

The orphan link reference definition — a line of the form `[label]: /destination "title"` that no link in the document actually references — wins on capacity, wins on invisibility, and wins on survival through both writers. And:

```
mdast `definition` nodes in 1,084 files / 25,548,765 bytes 0
mdast `linkReference` 0
mdast `imageReference` 0
mdast `footnoteDefinition` 0
mdast `footnoteReference` 0
definition-SHAPED lines found by a loose regex over raw bytes 9
...of which are actually prose (" [Topic]: Review said X.") 9
```

`[verified]` — the verifier re-derived the full 21-type AST histogram over all 1,084 sha256-checked files and every count matched digit-for-digit, then confirmed that `remark` can emit a `definition` node (a synthetic 999-character test produced `paragraph,definition` ), so the absence is a property of the corpus and not an artifact of the harness.

This is the **second independent replication of the same pattern**. The first is `markdown-it-decorate` — a carrier design the record describes as correct — at **1,094 downloads/week** against `markdown-it-attrs` at **280,897 downloads/week**, a ratio of 257x. `[measured, api.npmjs.org, last-week window, re-measured 2026-08-01: markdown-it-decorate 1,094 exactly]`

The research area that produced this finding pre-registered its own falsifier — "my headline is wrong if some carrier scores top on all four axes AND has non-zero live usage" — worked the candidates one at a time, disclosed a near-miss that cut against itself, and found no counterexample. The verifier then tested the one candidate the researcher had missed (the `[/]/: #` idiom, which the prior record named as the *other* carrier invisible in 24 of 24 renderer configurations): **0 occurrences in 0 files**. CriticMarkup: **0**. `:::` directives: **0**. `{#id}`: 9 occurrences in 4 files. `[verified]`

The implication, stated verbatim from the research because it should not be softened:

Any carrier MDMAX ships will be read by MDMAX and by nothing else, forever, so it must be justified by what MDMAX alone does with it, never by ecosystem uptake.

Three consequences follow directly, and they govern every later subsection:

- **"This carrier is elegant" is not an argument.** The most elegant carrier available has zero adoption after a decade of availability. Elegance predicts nothing about uptake.
- **"Other tools will preserve it because it is well-behaved" is not an argument either.** Nothing downstream is motivated to preserve a construct nobody recognises. The linkref's greatest strength (zero collision risk, because nobody uses it) and its greatest weakness (no author recognises it, so no tool has a reason to protect it) are **the same fact**.
- **Therefore prefer a recogniser over a carrier wherever a recogniser is possible.** A recogniser writes zero bytes, so it has no degradation story to defend, no round-trip hazard, and no adoption requirement. Four of the five additions in §4.6 are recognisers for exactly this reason.

### 4.4 The ranked carrier inventory

Ranked on **capacity × invisibility × survival × legibility**. All cells measured against the tool versions in §4.1 unless the cell says otherwise.

#### 4.4.1 Master table

`∅` = zero bytes reach the rendered output. `TEXT` = the payload appears as literal visible text. `DEL` = the payload is deleted and nothing takes its place. `RT` = round-trip.

| #  | CARRIER                                               | TIER                 | CAPACITY<br>(MEASURED)                                                                                                 | GITHUB COMMENT BOX<br>( GFM )                                                                    | GITHUB API<br>MARKDOWN<br>MODE                          | REMARK-<br>REHYPE<br>SAFE                                         | MARKED<br>16.4.2                    | REMARK-<br>STRINGIFY RT                                                                        | PRETTIER<br>RT                                     | LEGIBLE<br>RAW?           | USES IN<br>PINNED<br>CORPUS               |
|----|-------------------------------------------------------|----------------------|------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|---------------------------------------------------------|-------------------------------------------------------------------|-------------------------------------|------------------------------------------------------------------------------------------------|----------------------------------------------------|---------------------------|-------------------------------------------|
| 1  | Orphan link<br>reference<br>definition                | 1                    | label ≤999 /<br>≤1000;<br>destination and<br>title ≥1,000,000<br>B locally;<br>10,000-B title<br>accepted by<br>GitHub | ∅                                                                                                | ∅                                                       | ∅                                                                 | ∅                                   | byte-identical<br>(single record)                                                              | byte-<br>identical                                 | no                        | 0                                         |
| 2  | Unknown-<br>language fence<br>+ info-string<br>meta   | 1                    | body 50,000 B<br>verified<br>through<br>GitHub; meta<br>1,000,000 B<br>locally                                         | body visible; <b>meta<br/>preserved</b> as <code>data-meta=</code>                               | same                                                    | body<br>visible;<br>meta<br>dropped                               | body<br>visible;<br>meta<br>dropped | byte-identical,<br>14/14 hazards                                                               | byte-<br>identical,<br>body NOT<br>reformatted     | <b>yes</b>                | 0 fences<br>carry meta<br>(of 7,714)      |
| 3  | Link / image title<br>on an existing<br>link          | 1                    | ≥1,000,000 B<br>locally; 10,000<br>B through<br>GitHub                                                                 | preserved verbatim in<br><code>title=</code>                                                     | preserved                                               | preserved                                                         | preserved                           | identical <b>only</b><br>with <code>"</code><br>delimiter                                      | identical<br>only with <code>"</code><br>delimiter | semi                      | 23<br>occupied of<br>1,838 links          |
| 4  | <code>&lt;span title&gt; /<br/>&lt;a title&gt;</code> | 2                    | attribute-<br>length bound,<br>not probed                                                                              | <b>preserved, value intact</b>                                                                   | preserved                                               | <b>dropped<br/>entirely</b> (no<br><code>rehype-<br/>raw</code> ) | passes<br>through                   | byte-stable                                                                                    | byte-stable                                        | yes                       | 5                                         |
| 5  | YAML<br>frontmatter                                   | 2                    | unbounded                                                                                                              | rendered as a table                                                                              | rendered                                                | rendered                                                          | rendered                            | stable                                                                                         | stable                                             | yes                       | 907 files<br>(83.67%)                     |
| 6  | HTML comment                                          | 3                    | unbounded                                                                                                              | <b>DEL</b>                                                                                       | <b>DEL</b>                                              | passes<br>through<br>(invisible)                                  | passes<br>through                   | identical<br>standalone;<br><b>NOT identical<br/>as a delimited<br/>region</b>                 | as remark                                          | yes                       | <b>330</b> in 160<br>files                |
| 7  | <code>id</code> attribute                             | 3                    | short                                                                                                                  | survives but namespaced<br><code>user-content-</code> <b>and<br/>lowercased</b>                  | survives,<br>namespaced,<br><b>case<br/>preserved</b>   | dropped<br>without<br><code>rehype-<br/>raw</code>                | passes                              | stable                                                                                         | stable                                             | yes                       | not<br>separately<br>counted              |
| 8  | <code>class</code> / <code>data-<br/>*</code>         | 3                    | —                                                                                                                      | <b>stripped</b>                                                                                  | stripped                                                | dropped                                                           | passes                              | stable                                                                                         | stable                                             | yes                       | not<br>separately<br>counted              |
| 9  | <code>{#id}</code><br>(attribute<br>syntax)           | 3                    | short                                                                                                                  | <b>TEXT</b>                                                                                      | TEXT                                                    | TEXT                                                              | TEXT                                | n/a (not a<br>construct)                                                                       | n/a                                                | yes                       | 9 in 4 files                              |
| 10 | <code>{: #id .cls}</code><br>(kramdown IAL)           | 3                    | short                                                                                                                  | <b>TEXT</b> ( <code>&lt;p&gt;Paragraph.<br/>&lt;br&gt;\n{: #b1 .cls}&lt;/p&gt;</code> )          | TEXT                                                    | TEXT                                                              | TEXT                                | n/a                                                                                            | n/a                                                | yes                       | not<br>separately<br>counted              |
| 11 | <code>:::directive</code>                             | 3                    | unbounded                                                                                                              | <b>TEXT</b> ( <code>&lt;p&gt;:::note{#b1}<br/>&lt;br&gt;\nbody&lt;br&gt;\n:::&lt;/p&gt;</code> ) | TEXT                                                    | TEXT                                                              | TEXT                                | identical only<br>with the<br>extension<br>registered on<br><b>both</b> parse and<br>stringify | —                                                  | yes                       | 0                                         |
| 12 | <code>{{shortcode}}</code>                            | 3                    | unbounded                                                                                                              | <b>TEXT</b> , HTML-escaped<br>( <code>{{&amp;lt; note &amp;gt;}}</code> )                        | TEXT                                                    | TEXT                                                              | TEXT                                | n/a                                                                                            | n/a                                                | yes                       | 0                                         |
| 13 | CriticMarkup<br><code>&gt;&gt;note&lt;&lt;</code>     | 3                    | unbounded                                                                                                              | <b>TEXT</b> , escaped<br>( <code>{&amp;gt;&amp;gt;note&amp;lt;&amp;lt;}</code> )                 | TEXT                                                    | TEXT                                                              | TEXT                                | n/a                                                                                            | n/a                                                | yes                       | 0                                         |
| 14 | <code>[text]{#id}</code><br>(bracketed span)          | 3                    | short                                                                                                                  | <b>TEXT</b> ( <code>&lt;p&gt;Paragraph [text]<br/>{#b1} more.&lt;/p&gt;</code> )                 | TEXT                                                    | TEXT                                                              | TEXT                                | n/a                                                                                            | n/a                                                | yes                       | 0                                         |
| 15 | Definition list<br>( <code>Term / : def</code> )      | 3                    | unbounded                                                                                                              | <b>TEXT</b> ( <code>&lt;p&gt;Term&lt;br&gt;\n:<br/>definition&lt;/p&gt;</code> )                 | TEXT                                                    | TEXT                                                              | TEXT                                | n/a                                                                                            | n/a                                                | yes                       | 1                                         |
| 16 | Unicode Tag<br>block<br>(U+E0000–<br>E007F)           | 3 —<br><b>THREAT</b> | unbounded                                                                                                              | <b>passes through byte-<br/>identically</b>                                                      | passes                                                  | passes                                                            | passes                              | byte-identical                                                                                 | byte-<br>identical                                 | <b>no —<br/>invisible</b> | 0 (84 zero-<br>width chars<br>separately) |
| 17 | <code>[//]: #</code><br>comment idiom                 | 3                    | as linkref                                                                                                             | ∅                                                                                                | ∅                                                       | ∅                                                                 | ∅                                   | <b>CORRUPTED:</b><br>( <code>...</code> ) → <code>" ..<br/>"</code>                            | —                                                  | semi                      | 0                                         |
| 18 | GFM alert <code>&gt;<br/>[!KIND]</code>               | 3                    | one line                                                                                                               | renders as an alert                                                                              | plain<br>blockquote,<br><code>[!NOTE]</code><br>visible | plain<br>blockquote                                               | plain<br>blockquote                 | <b>CORRUPTED:</b><br><code>&gt; [!NOTE] →<br/>&gt; \[!NOTE]</code>                             | preserves it                                       | yes                       | 0                                         |

`[measured]` for every cell above except rows 9/10/15 Pages behaviour (see 4.4.2) and the `remark-rehype safe` column for rows 4 and 7, which is `[measured]` as `html_safe_has_payload=False`.

4.4.2 The two columns that are not measurements

These are separated out deliberately rather than folded into the table above, because burying weak evidence inside a strong-looking grid is how a reference table becomes a lie.

| #    | CARRIER                                   | GITHUB PAGES (KRAMDOWN)                                                                                                                        | CLIPBOARD<br>TEXT/PLAIN | CLIPBOARD<br>TEXT/HTML                |
|------|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------|---------------------------------------|
| 1    | Orphan linkref                            | <code>⌀</code> [inference]                                                                                                                     | survives [inference]    | deleted [inference]                   |
| 2    | Unknown fence                             | body visible [inference]                                                                                                                       | survives [inference]    | body survives, meta lost [inference]  |
| 6    | HTML comment                              | <code>{:comment}</code> is kramdown's own form; HTML comment behaviour not read                                                                | survives [inference]    | deleted [inference]                   |
| 9/10 | <code>{#id}</code> / <code>{: #id}</code> | implemented as an Attribute List Definition — renders correctly [primary] read of <code>gettalong/kramdown</code> <code>doc/syntax.page</code> | survives                | becomes a real <code>id</code>        |
| 15   | Definition list                           | implemented — renders as a real <code>&lt;d&gt;</code> [inference] from the same primary read                                                  | survives                | becomes a real <code>&lt;d&gt;</code> |
| 16   | Unicode Tag block                         | not read                                                                                                                                       | survives [inference]    | survives [inference]                  |

The consequence of rows 9, 10 and 15 is the single most awkward fact in the inventory and it has a name: **GitHub is not one target**. The same bytes are correct on GitHub Pages (Jekyll's default engine is kramdown, which implements both the IAL and the definition list) and broken on the GitHub blob view and in the GitHub comment box. Shipping any of those three constructs means **shipping a file that is correct on one GitHub surface and visibly broken on another**. Render targets must therefore be modelled as `(product, surface)` **pairs**, not as products.

Three further disagreements *inside GitHub's own public API* were measured, which is the strongest available evidence that the pair model is necessary rather than fastidious:

| CONSTRUCT                     | MODE=GFM                                                     | MODE=MARKDOWN                                                           |
|-------------------------------|--------------------------------------------------------------|-------------------------------------------------------------------------|
| <code>id="PAYLOAD7Q"</code>   | <code>id="user-content-payload7q"</code> — <b>lowercased</b> | <code>id="user-content-PAYLOAD7Q"</code> — <b>case preserved</b>        |
| <code>&gt; [!NOTE]</code>     | rendered as a <code>markdown-alert</code> div                | plain <code>&lt;blockquote&gt;</code> with <code>[!NOTE]</code> visible |
| <code>\$\$...\$\$</code> math | routed to the math renderer                                  | literal                                                                 |

[verified] — all six API responses byte-identical to the researcher's on independent re-probe.

4.4.3 Per-carrier hazards, measured

Every hazard listed here **fails silently**. That is the property that makes them worth a page.

Carrier 1 — orphan link reference definition.

- It cannot interrupt a paragraph.* `Alpha.\n[k]: /a "t"` is **one paragraph**, and the definition becomes visible text. It must be preceded by a blank line. [measured]
- A raw " inside the title destroys the definition,* and `remark` then emits a **visible escaped line**: `[fm:k]: /a "{"a":1,"b": [2,3]}` → `parsedAsDef=false` → output `Alpha.\n\n[fm:k]: /a "{"a":1,"b": [2,3]}"\n`. [verified]
- A blank line inside the title does the same.* [verified]
- remark and prettier normalise the delimiter to different canonical forms.* `[fm:k]: /u ({ "id": "c3", "n": 1 })` → `remark` emits double-quoted-with-backslashes, `prettier` emits single-quoted; `[a] (/u "he said \"hi\"")` → `prettier` rewrites to `[a] (/u 'he said "hi"')`. **A repository running prettier in CI and remark in the app will ping-pong those bytes forever.** [verified]
- The only payload encodings stable through both writers* are **base64url**, **percent-encoding**, and **backslash-escaped JSON**: `[fm:k]: /u "eyJpZCI6ImMzMn0"` and `[fm:k]: /u "%7B%22id%22%3A%22c3%22%7D"` both give `remarkID=true` `prettID=true`. [verified]
- The 999/1000 trap.* GitHub's `cmark-gfm` caps link labels at **1000** characters; CommonMark and micromark cap them at **999**. At exactly 1000, a link reference definition is **invisible on GitHub and visible junk in every conformant renderer** — a one-character-wide trap in the highest-ranked carrier. Sources verified at the exact lines claimed: `commonmark-spec spec.txt:7997` "label can have at most 999 characters inside the square brackets"; `github/cmark-gfm src/parser.h:13` `#define MAX_LINK_LABEL_LENGTH 1000`; `micromark-factory-label/dev/index.js:17` "labels in markdown are capped at 999 characters in the string." [primary, all three re-read at the stated line numbers; the micromark line was re-read again while writing this section from this repository's own `node_modules`] Behaviour measured both ways: GitHub `gfm` returns 8 bytes ( `<p>x</p>` , definition consumed) at 998/999/1000 and visible junk at 1001+; `remark-parse` gives `paragraph,definition` at 998/999 and `paragraph,paragraph` at 1000+. **This is the best-sourced finding in the entire carrier corpus.**
- A multi-record tail block is not byte-stable through* `remark-stringify` — *regardless of label uniqueness*. `remark-stringify` inserts a **blank line between consecutive definitions**. `Prettier`, by contrast, **preserves** the multi-record form. So a repository running both will ping-pong the tail block exactly as it ping-pongs the title delimiter. **Correction to the research's own stated reason:** the original report attributed the failure to `remark-stringify dropping duplicate labels`; the verifier proved that is false — five identical labels go in and five come out — by showing that three *unique* labels fail byte-identity in exactly the same way. The rule ("one record per document") is right; the mechanism given for it was wrong. [verified, REFUTED as originally stated]

Carrier 2 — unknown-language fence.

- The discriminator is language recognition, not the meta slot.* ````fm:meta v1` yields `<pre lang="fm:meta" data-meta="v1" class="notranslate">`; ````js fm=PAYLOAD7Q` yields `<div class="highlight highlight-source-js">` with **the meta gone** and the body re-tokenised into `pl-*` spans. [verified]
- Prettier reformats a known-language body and does not reformat an unknown-language one.* ````js\nconst a=1;\n```` survives `remark` byte-identically but `prettier` emits `const a = 1;`. **Any content hash committed over a known-language fence body is invalidated by a prettier run.** [verified]
- The reserved-word list is real, enumerable — and the version of it in the research is wrong.* `github-linguist/linguist` `lib/linguist/languages.yml`, fetched 2026-08-01, 162,266 bytes: **816 language entries + 424 aliases = 1,240 distinct tokens**. The cardinality is correct. **The membership is not:** Linguist keys its alias index on `name.downcase.gsub(/\\s, '-')`, and **191 of the 816 names contain a space**, so the naively lowercased set contains 191 members that are *not* reserved and omits 191 tokens that *are* — a **15.4% error rate, erring in the dangerous direction**. Live probe: ````vim-script foo=1`, which the naive set reports

as free, routes to `highlight-source-viml` and **destroys the meta**. Same exposure for `abap-cds`, `ant-build-system`, `answer-set-programming`, `adblock-filter-list` and 187 others. [verified, REFUTED as originally stated] **Verified free:** `mdmax`, `mdmax-decision`, `mdmax.record`, `decision`, `dataview`, `base`, `canvas`, `record`, `meta`, `plaintext`. **Verified reserved:** `json`, `yaml`, `text`, `md`, `mermaid`, `console`, `jsonc`, `vim-script`.

- `lang on <pre>` is the BCP 47 natural-language attribute. `lang="mdmax.record"` is an invalid language tag, which GitHub itself hedges by adding `class="notranslate"`. Harmless today; it is the kind of thing a future sanitizer tightens. [verified]
- Four spaces of indentation, or one backtick inside a backtick-fence info string, disables fence dispatch with no error anywhere. [measured, docs/engine/PLAN.md §15.4]
- Occupancy varies by 2,400× across corpora. Fence meta is occupied in **0 of 7,714** fences in the pinned corpus, **1 of 5,007 (0.02%)** in Corpus B (the value is `npm2yarn`), and **6,201 of 12,916 (48.01%)** in the Docusaurus repository, dominated by `title="docusaurus.config.js"` (2,136), `npm2yarn` (1,012), `title="sidebars.js"` (408). `title=` already owns this channel in Docusaurus-shaped repositories, so any MDMAX use must be namespaced (`fm:`). [measured]
- Every fence is a hole in the graph a linker is trying to build. A link inside a fence does not appear in the mdast link inventory; frontmatter inside a fence produces no `yaml` node. No link checking, no backlinks, no search indexing, no rename propagation inside a fence. **A design in which the important structure has migrated into fences has recreated MDX's problem with extra steps.** [measured, docs/engine/PLAN.md §15.4]

**Carrier 3 — link / image title.**

- It is availability-bound, and that is what disqualifies it for block identity. The pinned corpus has **1,838 inline links** (CommonMark-only parse) against **142,802 paragraphs** and **88,434 list items** — a ratio of **77.7 paragraphs to 1 link**. You cannot attach a durable name to most of a document through a slot that only exists where the author happened to write a link. [verified]
- It is not semantics-free, and the research's claim that it is has been refuted. `@types/mdast/index.d.ts` lines 103–107, verbatim: "Advisory information for the resource, such as would be appropriate for a tooltip." `html.spec.whatwg.org/multipage/dom.html`, verbatim: "Relying on the title attribute is currently discouraged as many user agents do not expose the attribute in an accessible manner as required by this specification..." And **pandoc has already burned a magic prefix in this exact slot** — its changelog records adding `fig:` as a title for images with captions, consumed as a control signal, for a decade. [verified, REFUTED as originally stated]
- Consequently "zero visual noise" is false. A populated title produces a **hover tooltip on every typed link**, which is keyboard- and touch-inaccessible per the specification quoted above. [verified]
- Round-trip identity holds only for the double-quote delimiter. `[a](./x.md 'atrel')` and `[a](./x.md (@rel))` both normalise to the double-quoted form. [verified]
- Occupancy: three different denominators are in circulation. See §4.10, C-2. Do not quote "99.94% unoccupied" without stating which one.

**Carrier 6 — HTML comment.** See §4.5, correction (a). Additional measured hazard: the **standalone** comment round-trips byte-identically, but the **delimited-region** form (`<!-- open -->` ... content ... `<!-- /close -->`) does **not** — `remark-stringify` inserts blank lines around both markers. [verified] Any design that uses comment-delimited regions for transclusion or generated blocks inherits that instability.

**Carrier 7 — `id`.** GitHub namespaces it to `user-content-*` and **lowercases it in `gfm` mode while preserving case in `markdown` mode**. Any case-sensitive identifier — `base64url`, `nanoid`, `ULID` — **collapses when a document passes through the comment box**. [verified] This alone disqualifies `id` as an identity carrier for this product, independently of every other argument.

**Carrier 16 — the Unicode Tag block.** See §4.5, correction (c).

**4.5 The three corrections**

Each of these overturns a belief that is stated somewhere in the prior record. Each is written as a correction rather than as a finding, because a reader who has read the earlier documents will otherwise carry the old belief forward.

(a) The HTML comment is TIER 3 — DO NOT USE. Correcting "invisible, therefore safe."

**Prior belief**, recorded in the earlier consolidated plan's 24-renderer matrix (`docs/mdmax/PLAN.md` v1.0.0 §4.3): the HTML comment produces "*8 distinct outputs — visible in 2, payload destroyed in 10, including GitHub's blob renderer which deletes it outright.*" That framing places the failure in the *blob* view and implicitly leaves the comment usable elsewhere. The broader ecosystem belief is stronger still: HTML comments are markdown's de-facto machine channel, used for lint pragmas (`markdownlint-disable`), generated-region boundaries (`all-contributors-list`, `vim-markdown-toc`, `prettier-ignore-start/end`), and — in two shipping Obsidian plugins — for mutable machine state, with Spaced Repetition writing scheduling data into `<!--SR: !2023-09-02,4,270-->`. [measured]

**Correction, measured:** GitHub deletes the HTML comment in BOTH API modes, not only in blob view.

```
POST api.github.com/markdown
"Alpha text.\n\n<!-- PAYLOAD7Q -->\n\nBeta text."
mode=gfm -> "<p>Alpha text.</p>\n\n<p>Beta text.</p>"
mode=markdown -> identical, plus a trailing newline
"- item\n <!-- PAYLOAD7Q -->\n- item2"
mode=gfm -> "\nitem\n\n\nitem2\n\n"
```

[verified]

`mode=gfm` is the comment box. So a comment written into a document and then pasted into a pull request or an issue is **destroyed silently, in the surface where review actually happens** — which is the exact surface this product exists to serve.

**And there are already 330 of them in the pinned corpus**, across 160 files. [measured] The verifier's independent AST-native classification gives 313 occurrences in 156 files (14.4%); [verified] the two counts differ by masking policy, not by phenomenon.

**Why this matters beyond the carrier question:** it re-derives decision **D2** (*comments and history live OUTSIDE the file*) from a completely different direction. D2 was reached as a product decision. It is now also a *rendering* fact: there is no in-file annotation channel that survives the review surface. **Do not build an in-file comment or annotation carrier.** The honest answer for per-block comments, per-span provenance and revision history is a **sidecar**.

(b) The `title` attribute SURVIVES GitHub's sanitizer with its value intact. Correcting "only `id` survives."

Prior belief, stated in the record: "GitHub strips class, style and data- entirely; only `id` survives, namespaced."

Correction, measured — it is incomplete in two ways, and both change design decisions.

|                                          |    |                                                 |
|------------------------------------------|----|-------------------------------------------------|
| mode=gfm                                 |    |                                                 |
| <span title="PAYLOAD7Q">x</span>         | -> | <p><span title="PAYLOAD7Q">x</span></p>         |
| <a href="/a" title="PAYLOAD7Q">x</a>     | -> | survives with title intact                      |
| Alpha <span data-fm="PAYLOAD7Q">x</span> | -> | <p>Alpha <span>x</span> text.</p> (data-* gone) |
| Alpha <span class="...">x</span>         | -> | class identically stripped                      |
| Alpha <span id="PAYLOAD7Q">x</span>      | -> | id="user-content-payload7q" (LOWERCASED)        |
| mode=markdown                            |    |                                                 |
| Alpha <span id="PAYLOAD7Q">x</span>      | -> | id="user-content-PAYLOAD7Q" (case PRESERVED)    |

[verified — all six responses byte-identical on independent re-probe]

A second, independent research area extended the same probe and produced the full surviving and stripped sets:

| GITHUB PRESERVES                                                         | GITHUB STRIPS                                                         |
|--------------------------------------------------------------------------|-----------------------------------------------------------------------|
| title, lang, dir, align, width, height, colspan, rowspan, cite, itemprop | rel, target, aria-label, role, download, loading, itemscope, itemtype |

[verified — re-probed live 2026-08-01, byte-for-byte]

The asymmetry is the finding. The semantically *correct* channel for a typed link — HTML `rel` — is destroyed. The semantically *vacuous* one — `title` — survives untouched. The general principle, and it inverts the obvious approach:

The channels that survive sanitisation are exactly the ones the specification assigns no meaning to. Ride semantics-free slots; never ride the semantically correct attribute.

Two caveats attach to this correction and neither is optional. First, `title` is *low-occupancy*, not *semantics-free* (§4.4.3, carrier 3) — it is the advisory/tooltip channel, the HTML specification discourages relying on it, and pandoc already consumed a prefix in it. Second, `<span title>` is **dropped entirely by** `remark-rehype` **without** `rehype-raw` ( measured as `html_safe_has_payload=False` ), so it does not reach a consumer that renders through the safe HTML path — which is most of them.

(c) The Unicode Tag block is NOT stripped by GitHub. It is a threat to FILTER, not a carrier.

Prior belief, produced by this research programme's own first pass: "GitHub strips the Unicode Tag block."

That reading was a harness artifact and the researcher caught and disclosed it mid-run. The first pass searched for the ASCII string inside output that contained tag characters, and reported `has=False` . The verifier re-tested: **9 tag characters sent through** `api.github.com/markdown` , **all 9 survived intact, in both modes**. [verified] The Tag block also passes `remark-stringify` and `prettier` byte-identically and renders to nothing visible anywhere.

So the earlier, more pessimistic reading was right and the correction ran in the unflattering direction. A block of Unicode Tag characters is a perfectly invisible, perfectly surviving, completely illegible channel that any text can carry through this product, GitHub, and both clipboard flavours without a single visible artifact.

Ruling: **DO NOT USE, and FILTER**. MDMAX must strip or flag U+E0000–U+E007F on ingest and on paste. Reasons, in order of severity:

1. It is the standard prompt-injection smuggling channel for text destined for a language model. This product's entire premise is that documents are read by models.
2. It is invisible to the human reviewer by construction, so no amount of care at the review surface catches it.
3. The pinned corpus contains `tagblock_chars=0` — so filtering costs nothing today — alongside **84 zero-width characters**, which are the same class of problem one severity level down. [measured]

This is the one place in this section where the recommendation is to **remove** a capability rather than add one, and it should be in the ingest path before the first external document is imported.

4.6 The refuted premise: authors do NOT reach for raw HTML

Every "markdown needs more constructs" argument in this project's history has rested on an unexamined premise: that when markdown fails an author, the author reaches for raw HTML, and therefore the HTML in a corpus is a readout of the missing-construct list.

In the pinned corpus that premise is false.

|                                                 |             |             |                               |
|-------------------------------------------------|-------------|-------------|-------------------------------|
| files containing real HTML5 tags in prose ..... | 37 of 1,084 | (3.4%)      | [researcher]                  |
|                                                 | .....       | 24 of 1,084 | (2.2%) [verifier, AST-native] |
| total occurrences .....                         | 3,583       |             |                               |

And the distribution is **degenerate**:

| WHAT      | COUNT      | WHERE                                                                                                                                               |
|-----------|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| <strong>  | 1,833      | ONE file — <code>md/Zephyrus/ecosystem.md</code>                                                                                                    |
| <br>      | 936 of 987 | two near-duplicate skill files ( <code>Pp Klaviyo/skill.md</code> and <code>../workflow.md</code> ), 468 each; the remaining 51 in one further file |
| <sub>     | 184        | one file                                                                                                                                            |
| <details> | 3          | across the entire 25,548,765 bytes                                                                                                                  |

[verified] — with one correction the verifier insisted on: the two `<br>` -heavy files are **near duplicates, not byte-identical** (sha256 prefixes `7bf5015a47e2df93` and `c7dfc2f44bb0e557` ). The concentration is exact; the word "identical" was not.

The verifier attacked this finding and **made it stronger rather than weaker**. Its first suspicion was that the number was a kill: the AST reports `html` nodes in **245 files (22.6%)**, six times the claimed 37. Classifying all **7,720** `html` nodes by kind resolved it:

| KIND OF <code>HTML</code> NODE | OCCURRENCES | FILES              |
|--------------------------------|-------------|--------------------|
| real HTML5 tags                | 5,989       | <b>24 (2.2%)</b>   |
| angle-bracket placeholders     | 1,418       | <b>201 (18.5%)</b> |
| HTML comments                  | 313         | 156 (14.4%)        |

**The 245-file figure is dominated by placeholders and comments, not by raw HTML.** `[verified]`

**What authors actually strain against**, in this corpus, ranked:

| # | STRAIN                                              | MEASURED                                                                                                                                                     | WHAT IT WANTS       |
|---|-----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|
| 1 | <b>bold-run list items</b> (description-list shape) | <b>18,462</b> of 88,434 list items (20.9%); verifier's AST-native recount <b>19,402 (21.94%)</b> , of which <b>8,044</b> carry a separator, across 539 files | <b>a recogniser</b> |
| 2 | <b>wikilinks</b>                                    | <b>13,028</b> exactly, in 501 files                                                                                                                          | <b>a resolver</b>   |
| 3 | <b>angle-bracket placeholders</b>                   | <b>1,431</b> tokens, 186 distinct names, in <b>219 of 1,084 files (20.2%)</b>                                                                                | <b>a linter</b>     |

`[verified – wikilinks reproduce EXACTLY at 13,028; description-list counts reproduce higher than claimed, so the reported figure is the conservative one]`

**Placeholder notation appears in six times more files than real HTML** (219 vs 37 by the researcher's counts, 8.4× by the verifier's stricter 201 vs 24).

**Two of the three want a recogniser and the third wants a linter. None of them wants a construct.** That is the empirical basis for §4.7's shape: one carrier, four recognisers.

**What has essentially zero demand in this corpus**, which is at least as useful to know, because several of these have been proposed:

|                                          |        |                                                    |
|------------------------------------------|--------|----------------------------------------------------|
| GFM alerts (> [!NOTE]) .....             | 0      |                                                    |
| bold-text admonitions .....              | 1      |                                                    |
| checkboxes inside table cells .....      | 0      | (4 in 114,762 cells across the 8-corpus sample)    |
| footnote references the parser sees .... | 0      |                                                    |
| math blocks .....                        | 8      | (inline math: 1,244)                               |
| multi-line table cells via <br> .....    | 51     | (great majority in three files)                    |
| fences carrying an info-string meta .... | 0      | of 7,714 fences (bash: 5,700 = 73.9%; mermaid: 10) |
| Unicode Tag block characters .....       | 0      | (zero-width characters: 84)                        |
| ::: directives .....                     | 0      |                                                    |
| CriticMarkup .....                       | 0      |                                                    |
| Hugo shortcodes .....                    | 0      |                                                    |
| emoji used as status .....               | 15,764 | characters                                         |

`[measured]`

**The strain is not where the theory said it would be.** The construct with the largest install- weighted demand in the plugin ecosystems (semantic block types, callouts) has **zero** instances here; the construct with no plugin at all (the description list) has **eighteen thousand**.

### 4.7 The top five additions: ONE new carrier and FOUR recognisers

Ranked by (strain evidence) × (rideable without new syntax) × (a plugin cannot close it).

Three of these five write **zero bytes** into any user file. That is not modesty; it is the direct consequence of §4.3. A recogniser has no degradation story because nothing changed on disk.

#### A1 — The tail record. The only new carrier.

**Status: RECOMMENDED, with an unresolved collision (see §4.9). Not decided.**

**What it is.** A single link reference definition, blank-line isolated, placed at the end of the file, carrying an encoded identity cache:

|                                                          |
|----------------------------------------------------------|
| ...the last paragraph of the document.                   |
| <code>[fm:b1]: #fm "v1.&lt;base64url payload&gt;"</code> |

**What it writes.** Roughly `20 + len(payload)` bytes, once per document. Nothing else in the file changes, ever.

**What it holds.** The block-anchor cache: for each anchorable block, the identity the engine already computed, so a cold read does not have to recompute anchors over the whole document.

**Degradation, measured end to end:**

| CONSUMER                                                                     | RESULT                |
|------------------------------------------------------------------------------|-----------------------|
| GitHub API <code>mode=gfm</code> (the comment box)                           | <b>0 bytes</b>        |
| GitHub API <code>mode=markdown</code>                                        | <b>0 bytes</b>        |
| <code>remark-rehype</code> <code>safe</code>                                 | <b>0 bytes</b>        |
| <code>remark-rehype</code> <code>with</code> <code>allowDangerousHtml</code> | <b>0 bytes</b>        |
| <code>marked</code> 16.4.2                                                   | <b>0 bytes</b>        |
| Obsidian / VS Code preview                                                   | nothing rendered      |
| <code>remark-stringify</code> 11.0.0                                         | <b>byte-identical</b> |
| <code>prettier</code> 3.8.3                                                  | <b>byte-identical</b> |

[verified — the verifier re-ran the shipped single-record form itself and reproduced all eight cells]

The eight rules. Every one of them exists because breaking it fails SILENTLY.

1. **One record per document.** Multi-record tail blocks are not byte-stable through `remark-stringify` (blank-line reflow), while prettier preserves them — a CI ping-pong.
2. **Blank-line isolated, tail-placed.** A definition that follows text without a blank line is absorbed into the paragraph and becomes visible.
3. **Label ≤ 999 characters, and in practice ≤ 64.** At exactly 1000, GitHub and every conformant renderer disagree.
4. **Title delimiter is " only.** Never `'...'`, never `(...)`.
5. **Payload is base64url or percent-encoded.** Never raw JSON, never a raw `"`, never a blank line.
6. **Lowercase, whitespace-free labels.** micromark normalises a link identifier by collapsing markdown whitespace, trimming, then `.toLowerCase().toUpperCase()` — `node_modules/micromark-util-normalize-identifier/dev/index.js`, read from this repository this session. [primary] So `[fm:B1]` and `[fm:b1]` are **the same label**, and a label containing a newline or tab is folded for matching. *This rule is derived from the normaliser's source, not from an executed collision test; the collision itself is [inference] and should be pinned by a fixture before A1 ships.*
7. **Never a backtick inside a backtick-fence info string, and never four-space indentation.** Both silently disable dispatch.
8. **Strip on export.** See §4.9 — this is the rule with no clean answer.

**What A1 is NOT.** It is a **cache over content-derived re-anchoring, never a replacement for it.** The anchoring algorithm ( `docs/engine/PLAN.md` §1.1a) measured 99.627% correct / 0.050% false / 0.323% refusal over 41,642 block-versions — *a figure that no research area has re-derived, that is a BLOCK figure while comments anchor RANGES (the one range replication gives 3.44×, not 30×), and which the current plan says must not be published until it is re-derived on a corpus this team did not write.* A1 must be correct-by-construction under the assumption that its own contents are **absent or stale**, because on every exported document they will be.

**What would kill A1,** taken from the research's pre-registered kill conditions:

- The second corpus shows link reference definitions in active use above **~0.5 per file** — collision risk appears and the "empty slot" argument dies. **This condition is arguably already tripped: see §4.10, C-1.**
- The clipboard measurement shows `text/plain` also dropping them — then the carrier has no path through the copy/paste loop that is this product's whole point.
- The export guarantee (D4) requires stripping tail records on export — in which case A1 is a cache with a 100% miss rate on every exported document and should be deleted rather than shipped.

A2 — Description-list recogniser. Zero bytes.

Status: RECOMMENDED. Largest coverage, lowest risk in the inventory.

**What it does.** Lifts the shape `- **Term** - definition` (and `- **Term**:: definition`) into a `descriptionList` node in the tree. **The source file is not touched.**

**Coverage.** 7,938 unambiguous instances (bold run followed by an explicit separator: em dash, en dash, hyphen or colon), 18,462 including the weaker bold-run-without-separator form, out of 88,434 list items (20.9%). [measured] The verifier's AST-native recount — which removes a numerator/denominator mismatch in the original method — gives **19,402 (21.94%) bold-opening list items, of which 8,044 (9.10%) carry a separator, across 539 files**, so the shipped figure is the conservative one. [verified]

**Degradation.** Exactly what it is today in every renderer, because nothing changed. There is no degradation to describe. This is the entire argument for recognisers.

**Precision is the real engineering problem, and a second research area measured it.** Over a different (8-corpus, 4,074-file) sample, splitting the same predicate by which sub-rule fired:

| SUB-RULE                                  | COUNT  | IS IT A RECORD?            |
|-------------------------------------------|--------|----------------------------|
| bold run then <code>:</code>              | 5,729  | <b>yes, unambiguous</b>    |
| bold text itself ending in <code>:</code> | 11,336 | <b>yes, unambiguous</b>    |
| bold run then em/en dash or hyphen        | 12,951 | <b>mixed</b>               |
| bold text ending in <code>.</code>        | 5,764  | <b>NO — false positive</b> |

[verified] The `.`-terminated bucket is a **bolded lead sentence**, not a record — verbatim samples: *"Stable Zustand selectors only. Never return a new object/array/Set from a selector"*. The dash bucket is genuinely mixed: *"Up to ₹5,000 — UPI app prompts biometric"* is a record; *"Full redesign of the landing page — modern, credible, conversion-focused"* is prose apposition.

**Therefore A2 must not fire on `**Sentence.** rest`**, and its dash rule needs a hand-labelled precision gate before it drives anything user-visible.

**Two honesty caveats on the coverage number.** (1) Description-list strain is **4.89× higher in AI-authored text than in human-authored text** in the 8-corpus sample — 34.43 per 1,000 lines versus 7.03 — and the pinned corpus is 93.7% AI-authored on this metric. [verified] (2) Some of it is not strain at all but **template output**: the

vault's `proj-*.md` files are emitted by a snapshot script that prints `**Last commit:** ... , **Commits:** ... , **Remote:** ...`. **Generated markdown counted as authored strain is a defect**, and the count above has not been deduplicated by generator template.

**A3 — Placeholder recogniser and certificate warning. Zero bytes.**  
**Status: RECOMMENDED. The highest-severity finding in the corpus and the cheapest fix in the report.**

**The problem.** Markdown has no construct for *"a thing you substitute."* So authors write `<target-file>`, and CommonMark classifies it as raw HTML, and **GitHub deletes it**.

```
POST api.github.com/markdown mode=gfm

"Run the tool with `--flag` on <target-file> and then <name> is bound."
-> "<p>Run the tool with <code class=\"notranslate\">--flag</code> on and then is bound.</p>"
 BOTH placeholders gone, replaced by nothing.
```

[verified – reproduced byte-for-byte, then extended: 'A <N> B <one-line> C <concise> D <command> E <capability> F <api-slug> G <dim> H <skill> I.' -> '<p>A B C D E F G H I.</p>', so hyphenated and capitalised names are deleted too]

**Scale.** 1,431 tokens, 186 distinct names, in **219 of 1,084 files (20.2%)**. [measured] Two independent replications: AST-native gives 1,418 / 176 / 201 files (18.5%); a line-based regex gives 1,155 / 174 / 217 (20.0%). [verified] **The top tokens match exactly across methods** — `<one-line>` 102, `<concise>` 92, `<command>` 87, `<capability>` 69 — with `<n>` at 223 in 113 files, `<name>` 47, `<url>` 27. The occurrence count and the deletion behaviour are solid; the file count is the softest number, at 201–219 depending on masking.

**There are three failure modes, not one, and no renderer agrees with another.** The headline "GitHub silently deletes them" undersells the finding:

| INPUT                                                            | GITHUB   GFM   OUTPUT                                               | MODE                                                                                                                  |
|------------------------------------------------------------------|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Filed <C> new issue<s>, added <E> comment<s> on existing issues. | <p>Filed new issue<s>, added comment<s> on existing issues.</s></p> | <s> opens a strikethrough GitHub auto-closes at paragraph end — active markup that corrupts the rest of the paragraph |
| Set <title> and <base> before the run.                           | <p>Set &lt;title&gt; and before the run.</p>                        | <title> ESCAPED and visible; <base> DELETED — in one line                                                             |
| You mentioned <Source A>, <Source B>, and <Source C>.            | <p>You mentioned <source>, <source>, and <source>.</p>              | attributes stripped, content deleted                                                                                  |

[verified – all three reproduce verbatim, including the trailing auto-close]

Collision counts in the pinned corpus: `<base>` **66**, `<title>` **32** (verifier: 31), `<s>` **4**. One correction the verifier made and it should be carried: `<source>` **20 does not belong in that list** — bare `<source>` occurs **0** times; those 20 are real `<source ...>` tags with attributes in 2 files, a different construct. [verified]

**The remediation is one character and it is verified.**

|                                  |                                                                      |           |
|----------------------------------|----------------------------------------------------------------------|-----------|
| <code>&lt;target-file&gt;</code> | -> <code>&lt;code&gt;&amp;lt;target-file&amp;gt;&lt;/code&gt;</code> | preserved |
| <code>&lt;target-file&gt;</code> | -> <code>&amp;lt;target-file&amp;gt;</code>                          | preserved |

[measured]

**What A3 does.** Detects bare angle-bracket tokens that are either (i) not valid HTML5 element names, or (ii) valid element names being used as metavariables, and emits: *"GitHub will DELETE these N tokens from rendered output; wrap them in backticks."*

**Nothing else does this.** `markdownlint` 's only relevant rule is **MD033 / no-inline-html** ( `raw.githubusercontent.com/DavidAnson/markdownlint/main/doc/Rules.md` line 1364), which fires on **all** inline HTML including legitimate `<br>` and `<details>` — the same low-precision failure that gets a naive broken-link checker switched off. [primary] The npm registry returned `{"error": "Not found"}` for `markdown-it-placeholder`, `remark-lint-no-angle-placeholder` and `markdown-placeholder-lint`. [measured]

**Pre-registered gate.** Precision must exceed **90%** against a 200-document hand-labelled ground truth. **Below 80%, A3 does not ship** — it will be switched off exactly as the naive link checker was.

**A4 — Inline-role recogniser over `inlineCode`. Zero bytes.**  
**Status: RECOMMENDED.**

**The problem.** Markdown offers exactly **one** inline semantic slot, and the corpus is using it for at least **13 distinct jobs**. `inlineCode` is the second-most-common leaf node in the entire corpus at **76,118 occurrences**. [verified – exact]

Measured classification over 77,516 spans with fenced code masked (the 1,398 delta from the AST count is spans inside constructs the line-based masker does not reach):

| ROLE                            | COUNT  | SHARE |
|---------------------------------|--------|-------|
| other ( <b>unclassified</b> )   | 19,695 | 25.4% |
| identifier                      | 17,792 | 23.0% |
| filepath                        | 13,805 | 17.8% |
| prose in backticks              | 10,220 | 13.2% |
| constant / environment variable | 5,652  | 7.3%  |
| filename                        | 2,717  | 3.5%  |
| command                         | 2,547  | 3.3%  |
| CLI flag                        | 2,219  | 2.9%  |
| data literal                    | 1,598  | 2.1%  |
| function call                   | 729    | 0.9%  |
| numeric value                   | 337    | —     |
| URL                             | 171    | —     |
| empty                           | 34     | —     |

The buckets sum to exactly 77,516, so the internal arithmetic is clean. [verified]

But one row of that table is **OVERSTATED** and must not be quoted. The verifier could not replicate "**prose in backticks 10,220 (13.2%)**". The entire plausible prose pool — spans with  $\geq 2$  whitespace-separated words — is **21,398 (27.6%)**; a seeded random sample of 30 drawn from it contained roughly **5** genuine prose spans (~17%), implying  **$\approx 3,600$  spans  $\approx 4.6\%$** , about a third of the claim. A seeded random 30 from the single-word pool (72.3% of all spans) contained **zero** prose. There is also a **denominator swap**: 13.2% is 10,220/77,516 while the headline count is stated as 76,118 (10,220/76,118 = 13.4%). [verified, OVERSTATED]

What survives and what to write down: 76,118 `inlineCode` spans carry many distinct roles, and a quarter of them are unclassified. Drop "13.2% are not code at all"; the independent estimate is **4–5%**. A4 is unaffected****, because a role recogniser does not depend on the prose share.

What A4 does. Classifies each `inlineCode` span into a role in the tree — filepath, command, identifier, CLI flag, constant, prose — so that search, cross-reference and any model-facing projection can treat a filepath as a filepath. **No bytes written, so there is no degradation to describe.**

A5 — Typed-relation resolver for wikilinks. Zero bytes.

Status: **RECOMMENDED**, with a prior-art correction that weakens the strategic case (below).

The problem, measured as damage rather than as theory. **13,028 wikilinks** in the pinned corpus, across 501 files, **render as literal visible junk on GitHub**.

[verified — count reproduces EXACTLY; See [[Some Page]] and [[dir/Other|alias]]. →

See [[Some Page]] and [[dir/Other|alias]].

1 That is the largest single body of already-written markdown in this corpus that is broken outside one renderer.

What A5 is. **Not syntax**. A resolver with more than one file in scope, plus a certificate line stating that these 13,028 constructs are literal text outside Obsidian. The two-tier resolution semantics already exist in the plan: `[text](path.md)` resolves at **85.71%**, `[[Name]]` at **27.65%** — a 58-point gap on identical files, and no tool implements the semantics that gap demands, because *a path link that does not resolve is a mistake while a wikilink that does not resolve is a legitimate authoring primitive*. [measured, docs/mdmax/PLAN.md §5.3]

This is the "*hundreds of markdowns connected to hundreds*" capability the founder describes, delivered as **resolution** rather than as a carrier.

The prior-art correction, and it must be carried forward because the prior record states the opposite. `docs/engine/PLAN.md` §16.8 argues that typed links are *supply-starved* — 820 forum likes, #2 request overall, only 5 implementations above 5k downloads — and that the reason is that "*Breadcrumbs' only available workaround hoists every edge into frontmatter — so the relation no longer lives at the point of reference.*" **A verifier read Breadcrumbs' own README and refuted that**. `michaelpporter/breadcrumbs`, 806 stars, verbatim: "*Breadcrumbs lets you add typed links to your notes*", and under "Ways to define relationships": "*Frontmatter properties (typed links), tags, and **Markdown lists***" plus Dataview queries — i.e. **it already reads typed edges from the body, at the point of reference**. `Querulantenkind/obsidian-typed-links-plugin` also exists. [verified, REFUTED]

So "**a plugin cannot close this**" is false as stated. The defensible residual argument for MDMAX is **block identity across the git boundary**, not novelty. Make that argument; do not make the refuted one. And note the general defect it exemplifies: *any "nobody has built this" claim must cite the search that was run* — this one was falsified by a single repository search.

4.8 What must NOT be built, with the measurement for each

| #  | DO NOT BUILD                                                                                                                                                                           | MEASUREMENT THAT DECIDES IT                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | Any in-file comment or annotation carrier                                                                                                                                              | GitHub deletes HTML comments in <b>both</b> API modes, including the comment box; <b>330 already exist</b> in the corpus. Independently re-derives <b>D2</b> . <span>[verified]</span>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| 2  | Any appearance-based or id -based identity                                                                                                                                             | <code>id</code> is <b>lowercased</b> in <code>gfm</code> mode, so <code>base64url</code> / <code>nanoid</code> / <code>ULID</code> collapse through the comment box. <span>[verified]</span> Separately, byte offset is <b>62.12% false</b> and block index <b>55.50% false</b> over 41,642 block-versions. <span>[measured, docs/engine/PLAN.md §1.1a]</span>                                                                                                                                                                                                                                                                                                                                                                              |
| 3  | Any new block or inline syntax                                                                                                                                                         | All six tested proposals render as <b>visible literal junk</b> on GitHub: <code>Term / : def , { : #b1 .cls } , :::note{#b1} , {{&lt; note &gt;}} , {&gt;&gt;note&lt;&lt;} , [text]{#b1}</code> . <span>[measured, verbatim outputs in §4.4.1]</span> This is a <b>rendering</b> fact and therefore stronger than the adoption argument.                                                                                                                                                                                                                                                                                                                                                                                                    |
| 4  | The Unicode Tag block                                                                                                                                                                  | Survives GitHub, remark and prettier byte-identically and invisibly. <b>Filter it, do not use it.</b> <span>[verified]</span>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| 5  | Merged-cell / block-content tables                                                                                                                                                     | The apparent evidence — <code>kubernetes/website</code> writes 1,037 raw <code>&lt;table&gt;</code> against 42 markdown tables, 24.69× — <b>does not support the causal claim</b> : 1,626 of 1,632 colspans (99.6%) are the identical <code>colspan="2"</code> , 1,126 (69.0%) sit in generated <code>reference/kubectl/**</code> , and <b>exactly one</b> appears in a hand-authored page. It is evidence about two documentation <i>generators</i> , not about humans needing merged cells. <span>[verified, OVERSTATED]</span> Any GFM-table extension is a new dialect, which violates <b>D6</b> . Teach the editor to author an HTML table well; GitHub preserves <code>colspan</code> , <code>rowspan</code> and <code>align</code> . |
| 6  | Checkbox inside a table cell                                                                                                                                                           | <b>4 of 114,762 table cells (0.003%)</b> across the 8-corpus sample; <b>0</b> in the pinned corpus. <span>[measured]</span> Confirmed unexpressible: <code>[ - [ ] task   x ] → &lt;td&gt; [ ] task in cell&lt;/td&gt;</code> verbatim. Note this is a <b>silence</b> signal, not proof of no demand (299 forum likes ask for it) — when a construct is unexpressible <i>and</i> has no adjacent workaround, users stop trying and the demand only ever surfaces in a forum. Do not build for it; do not conclude nobody wants it.                                                                                                                                                                                                          |
| 7  | Inline typed annotation ( <code>dfn</code> , <code>abbr</code> , <code>cite</code> , <code>time</code> , <code>data</code> , <code>var</code> , <code>kbd</code> , <code>mark</code> ) | GitHub <b>strips</b> <code>abbr</code> , <code>dfn</code> , <code>cite</code> , <code>time</code> , <code>data</code> , <code>small</code> , <code>bdi</code> , <code>bdo</code> — so even the HTML escape hatch fails. <span>[measured]</span> And markdown has exactly three inline channels, with every candidate sigil contested: <code>::</code> is live-contested between two top-50 Obsidian plugins <b>today</b> (Dataview 4.65M, Spaced Repetition 569k). <span>[measured, docs/engine/PLAN.md §16.5]</span>                                                                                                                                                                                                                       |
| 8  | Spatial / board / canvas layout                                                                                                                                                        | <b>No shipping product writes graph layout back to markdown</b> . Obsidian conceded and shipped <code>.canvas</code> (JSON, open spec) and <code>.base</code> (YAML). <span>[primary, docs/engine/PLAN.md §15.7, §16.3]</span> Sidecar.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 9  | Figure + caption + number                                                                                                                                                              | <code>&lt;figure&gt;</code> , <code>&lt;figcaption&gt;</code> and the Rust Book's private <code>&lt;listing&gt;</code> element (used <b>848</b> times) are all <b>stripped to bare text by GitHub</b> . <span>[measured]</span> There is no carrier to design. The caption is a paragraph; the <b>number is a projection</b> , computed at render time and never stored.                                                                                                                                                                                                                                                                                                                                                                    |
| 10 | A stored section boundary from <code>---</code>                                                                                                                                        | <b>82.8% of thematic breaks (7,171 of 8,663) immediately precede a heading</b> . <span>[measured]</span> That is a request for a <i>visible boundary</i> , which is a rendering concern; the section is already derivable from the heading tree. Infer it, render it, <b>write nothing to disk</b> .                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 11 | A disclosure construct                                                                                                                                                                 | <code>&lt;details&gt;</code> / <code>&lt;summary&gt;</code> already survives GitHub untouched (1,716 / 1,710 uses in the top-300 README corpus). Solved. Add nothing.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 12 | A declared-vocabulary syntax, registry, or library-import mechanism                                                                                                                    | Given syntax permitting an arbitrary type name, ~850 independent third-party authors invented <b>zero</b> types — they used the five GitHub ships and stopped. <b>99.69%</b> of the 4,538 <code>:::</code> directives in Docusaurus's own repository are among its 9 vendor-shipped keywords, and all 14 exceptions are the tutorial strings that teach the feature. <span>[measured, area custom-pointers , headline verdict CONFIRMED]</span> Ship the <b>inverse</b> : vocabulary <i>inference and drift detection</i> over values that already exist.                                                                                                                                                                                   |
| 13 | A recogniser that writes bytes                                                                                                                                                         | A recogniser that emits source is a <b>construct wearing a disguise</b> and falls under every negative in this section. This is the kill condition for A2, A4 and A5: they die if and only if they start writing.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |

One thing that must be built and is not a carrier at all: the **degradation certificate**. Per document, per `(product, surface)` pair, per construct: **PASS · STRIP · CORRUPT**. This section is the specification for its rule table. The prior record already establishes there is no published equivalent — four independent adversarial searches found no incumbent, one returning `total_count 0/0/1` . [measured, docs/mdmax/PLAN.md §5.4] But it targets pains ranked **13th and 14th of 14** in the demand research, so it is a credibility instrument and a distribution channel: **two days, hard cap, never the spearhead**.

4.9 The collision this section creates with D2 and D4 — an OPEN problem

This is the one place where the carrier work contradicts a decision the founders have already made, and it is stated as an open problem rather than resolved, because the research could not resolve it and neither can this document.

**D2** says comments and history live **outside** the file: *export or copy yields clean markdown, latest content only*. **D4** says the durable artifact is plain markdown, and if the product dies a full export and migration ships.

**A1 puts machine state inside the .md** . Three concrete conflicts follow.

**Conflict 1 — a clean export must strip the tail record, and stripping it destroys the exported copy's identity**. If export strips A1, then every exported document arrives at its destination with a **100% cache miss**, and content-derived re-anchoring is doing all the work anyway. If export does *not* strip it, then "export yields clean markdown" is false: the exported file carries a line of opaque base64 that only this product can read, which is exactly the lock-in D4 exists to prevent.

**Conflict 2 — one copy produces two different documents**. The tail record survives the `text/plain` clipboard flavour (that flavour is the raw source) and is **deleted** from the `text/html` flavour (it renders to zero bytes). So **pasting into Google Docs or Word silently drops it while pasting into a text editor keeps it**. [inference — no browser was driven; both halves follow deductively from measured HTML output, and neither was observed] That is precisely the *divergence between a human view and a machine view* failure this programme warns about, produced by our own design.

**Conflict 3 — the argument that killed the in-file comment applies to A1 too**. §4.5(a) rules out in-file annotation because GitHub destroys it at the review surface. A1 is not an annotation, and it survives where the comment dies — but it is still machine state written into a file the product promises to leave ordinary. The distinction is real and it is thin.

The three options, stated so a decision can be made rather than deferred indefinitely:

| OPTION                                                                 | WHAT IT COSTS                                                                                                                                   | WHAT IT BUYS                                                                                                                                                        |
|------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| (i) Do not ship A1                                                     | every read recomputes anchors (~41 ms for a 446 KB / 2,225-block document, <span>[measured]</span> )                                            | zero bytes in user files; D2 and D4 hold without qualification; §4.3's "nobody will ever read our carrier" ceases to be a problem because we did not build one      |
| (ii) Ship A1 as a cache with an explicit export-strip step             | a documented 100% miss rate on exported documents; the clipboard divergence in Conflict 2 remains and is user-visible                           | fast cold reads inside the product                                                                                                                                  |
| (iii) Move the record to a sidecar keyed by the content-derived anchor | the differentiator collapses from <i>"identity across edits"</i> to <i>"we have a good anchor"</i> — a smaller and much less defensible product | perfect D2/D4 compliance; the same capability; and it is what Obsidian did <b>twice</b> ( <code>.base</code> , <code>.canvas</code> ) when it hit exactly this wall |

**Option (iii) is the honest kill condition for the whole carrier programme**, and the research states it plainly: *if the anchor is good enough to key a sidecar, the claim collapses into "we have a good anchor."* The anchor is measured at **99.627%**, which is plenty good enough to key a sidecar — so this is a genuine risk, not a rhetorical one.

**Recommendation: default to (i) for v1 and revisit only if a measured cold-read latency problem appears in the product. This is a recommendation, not a decision. The founders decide.**

#### 4.10 Where two sources contradict, and which governs

Twelve contradictions were found while assembling this section. Each is named, both sides are given, and one is nominated to govern. **Do not quote either side of these without checking here first.**

**C-1 — Link reference definition occupancy. This is the important one.** Area `hold-more` measured **0** definitions in the pinned corpus (mdast `definition` absent; the 9 definition-shaped lines are prose, quoted verbatim). Area `custom-pointers` reported **9 occurrences in 9 files (0.83%)** for the same corpus — it counted the same 9 prose lines by regex. `hold-more` **governs for the pinned corpus**, because it parsed rather than matched and the verifier read all 9 lines. **But `custom-pointers` also measured Corpus B — 847 third-party npm documentation files — at 5,079 definitions in 198 files (23.38%).** `[measured]` **5,079 / 847 = 5.997 definitions per file.** A1's own pre-registered kill condition is *"link reference definitions in active use above ~0.5 per file."* **On its face that condition is already tripped, by a measurement taken inside the same research programme, and nobody connected the two.** Mitigations exist — the `fm:` label prefix makes collision essentially impossible, and Corpus B is package documentation rather than the pre-registered top-200-GitHub-repos corpus — but **A1 must not ship until this is adjudicated explicitly.** It is recorded here as the most likely reason A1 does not survive.

**C-2 — Link title occupancy: four denominators are in circulation, and one of them cannot be reconciled.** `hold-more`: 23 titles in **1,838** links (CommonMark-only parse, pinned corpus). `custom-pointers`: 23 in **2,035** inline links (1.13%) pinned, and 253 in 19,462 (1.30%) in Corpus B. The AST histogram with GFM enabled reports **link=5,453** for the same corpus (autolinks included). The capability run reports **40 titles in 67,790 links = 0.059%**, i.e. "99.94% unoccupied", over an 8-corpus 4,074-file sample. **The 67,790 figure has not been reconciled to anything**, and the researcher who found the discrepancy flagged it rather than papering over it: *"I cannot tell whether the 67,790 figure used a corpus, an exclusion policy, or a definition of 'link' that reproduces."* **Governing rule: never quote a link-title occupancy rate without its denominator, and treat "99.94%" as unverified.** Note also that across all three `Resource`-bearing node types the same run gives **99.898%**, not 99.94%.

**C-3 — The size of markdown's alphabet: 21 or 25?** `docs/engine/PLAN.md` §2.1 says a **"closed 21-label alphabet."** The capability run corrects this from a primary read: `@types/mdast` `RootContentMap` has **exactly 25 keys** (26 with `root`), and 18 (19 with `root`) for CommonMark alone. **Both are right and they measure different things**, and the arithmetic reconciles exactly:

```
defined content labels (CommonMark + GFM + frontmatter) 25
plus root 26
minus the 5 node types absent from the pinned corpus -5
 (definition, linkReference, imageReference,
 footnoteDefinition, footnoteReference)

observed distinct node types in 1,084 files 21 ✓ matches the AST histogram
```

`[inference over two measurements — the 25-key read is [primary], the 21-type histogram is [verified], the subtraction is mine]` **Governing: 25 is the defined alphabet; 21 is what this corpus uses; §2.1's "21" should be relabelled as the observed count.** The *substance* of §2.1 — closed, tiny, no typed values, no non-tree edges — survives intact and is stronger than the wrong number suggested.

**C-4 — HTML comment count: 330 in 160 files, or 313 in 156 (14.4%)?** Regex versus AST-native classification. Same phenomenon; the difference is masking policy. Either is quotable **with its method stated**.

**C-5 — Frontmatter prevalence: 83.67% or 84.73%?** The AST reports `yaml=907` nodes over 1,084 files = **83.67%** `[computed: 907 ÷ 1,084 = 0.836716... = 83.67%]`. `custom-pointers` reports **84.73%** with no stated denominator (84.73% of 1,084 would be 918.5 files). **Governing: 907 of 1,084 = 83.67%**, because the numerator is an AST node count over a hash-pinned file list.

**C-6 — Frontmatter blocks that are not valid YAML: 170 of 907, or 18.91%?** `docs/mdmax/PLAN.md` §5.5 says **170 of 907** (= 18.74% `[computed]`); `custom-pointers` says **18.91%** (= 171.5 of 907). Immaterial; **quote 170 of 907**, which names its numerator and denominator. The root cause is agreed by both: `[[wikilink]]` inside a YAML flow sequence, plus the silent variant where `[[[A]], [[B]]]` parses to a nested array and **destroys the links with no error**.

**C-7 — `{#id}` occurrences in the pinned corpus: 9 in 4 files, or 6 in 3?** Two own-measurements from the same programme with different regexes. **Single digits either way**; the design conclusion (no live usage) is unaffected.

**C-8 — `remark-directive` weekly downloads: 3,176,157 or 3,153,286?** `docs/engine/PLAN.md` §15.3 versus a re-measurement on 2026-08-01. **The later measurement governs**, and the general rule is that npm weekly figures drift — **always restate the fetch date beside the number**. (Same run: `remark-parse` 45,820,243; `marked` 61,032,058; `markdown-it` 27,323,922; `gray-matter` 7,492,225; `markdown-it-attrs` 280,897; `markdown-it-decorate` 1,094.)

**C-9 — Breadcrumbs and typed links.** `docs/engine/PLAN.md` §16.8 states the workaround hoists every edge into frontmatter; the plugin's own README says it also reads from Markdown lists and Dataview inline fields. **The README governs** (primary read of the vendor's own documentation). §16.8's supply-starvation argument is weakened accordingly — see §4.7, A5.

**C-10 — Raw-HTML file share: 37 files (3.4%) or 24 (2.2%)?** The verifier's AST-native classification is stricter and **strengthens** the refutation. **Report 37 / 3.4%** as the conservative figure and note 24 / 2.2% as the stricter replication.

**C-11 — Description-list counts: 18,462 / 7,938, or 19,402 / 8,044?** The verifier's AST-native recount is **higher**, so the researcher's figure is the conservative one. **Quote 18,462 / 7,938** and note the replication ran higher.

**C-12 — The `[[/]]: #` idiom.** The prior 24-renderer matrix (`docs/mdmax/PLAN.md` v1.0.0 §4.3) names it as one of only two carriers **invisible in 24 of 24 renderer configurations**. A later measurement found it **does not survive a write**: `[[/]]: # (fm:vocab sgnk/v1) → [[/]]: # "fm:vocab sgnk/v1"` — parentheses become quotes. `[verified]` **The write test governs**. A carrier that does not survive a write is not a carrier for a product whose entire job is writing. It is removed from Tier 1 and recorded in §4.4.1 row 17 as Tier 3.

**One further contradiction of a different kind, recorded because it bears on how much of this section to trust.** The 16-area final gate carries 5 CONFIRMED, 10 OVERSTATED and 1 REFUTED headline verdicts; the 18-area capability run had a **100% headline defect rate**, every headline refuted by its own verifier, always in the

flattering direction. **This section's primary source, area `hold-more`, is one of the five CONFIRMED**, and its verifier's own summary is that the researcher "disclosed a near-miss that cuts against itself" and "self-corrected in the un-flattering direction." That is why it carries this section. Its secondary source, area `missing-constructs`, is from the 100%-defect run: **eight of its claims were REFUTED or OVERSTATED by its own verifier**, and every figure this section takes from it is one the verifier CONFIRMED. Verifiers themselves have a measured false-kill rate of 0–19%, so a kill is strong evidence, not proof.

4.11 What markdown structurally cannot hold

Derived from the alphabet, not from a wish list. This is the boundary that decides what belongs in the file and what belongs in a sidecar.

4.11.1 The alphabet

**25 content labels (26 with `root`)** for CommonMark + GFM + frontmatter; **18 (19 with `root`)** for CommonMark alone. `[primary - @types/mdast RootContentMap ]` The full set:

```
blockquote break code definition delete emphasis footnoteDefinition footnoteReference
heading html image imageReference inlineCode link linkReference list listItem
paragraph strong table tableCell tableRow text thematicBreak yaml
```

The six GFM types are `delete`, `footnoteDefinition`, `footnoteReference`, `table`, `tableCell`, `tableRow`; `yaml` is frontmatter and **is not in the CommonMark specification at all** — to a conforming commonMark parser, `---\ntitle: Hello\n---` is a `thematicBreak` followed by an `h2` whose text is your YAML. **The most widely deployed data channel in the ecosystem is not in the spec.** `[primary, docs/engine/PLAN.md §2.1]`

4.11.2 The five things it cannot carry

- 1. **Types.** There are **exactly three** non-string value domains in the entire format: `ListItem.checked: boolean | null | undefined`, `Heading.depth: 1|2|3|4|5|6`, and `List.start: number | null`. **Everything else is string.** `[primary]`
- 2. **Identity.** No node carries a durable name. And `node.data` **never reaches disk** — confirmed empirically and by zero `.data` hits in `mdast-util-to-markdown/lib/. hName / hProperties / hChildren` survive into hast only. **Metadata must be syntax or sidecar; it cannot ride on a node.** `[measured, docs/engine/PLAN.md §15.4]`
- 3. **Null distinguished from absent.** There is no way to say "this field is deliberately empty."
- 4. **Any edge that is not parent-child — with exactly one exception.** The only node-to-node reference in the format is the footnote: `footnoteReference.identifier → footnoteDefinition.identifier`. (`linkReference / imageReference → definition → url` is node-to-*string*, not node-to-node.) **That one edge channel is spec'd, GFM-parsed, fully rendered by GitHub — and unused: 6 definitions + 10 references = 16 nodes in 1,293,322 lines across the 8-corpus sample, and 0 in the pinned corpus.** `[primary + measured]`
- 5. **Any scope larger than one file.** Frontmatter is not the escape hatch: a YAML alias *"refers to the most recent event in the serialization"*, and the serialization is that file. **No anchor scope larger than a file, therefore no relation larger than a file.** `[primary, docs/engine/PLAN.md §2.1]`

**The boundary is not a syntax gap a dialect could close. It is the tree/graph boundary.** What crosses it is not a construct but **a resolver with more than one file in scope — a linker**. That is the formal reason A5 is a resolver and not a syntax, and it is independent of every adoption argument.

4.11.3 Attribute channels: there are three, and they are all string-typed

| CHANNEL                                                                                         | REACHES                                                                           | FREE-FORM?                                                                            |
|-------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <code>Resource.title</code> on <code>link</code> , <code>image</code> , <code>definition</code> | the DOM, verbatim, on GitHub                                                      | yes — CommonMark assigns it no meaning                                                |
| the fenced-code <b>info string</b>                                                              | <code>lang=</code> + <code>data-meta=</code> on GitHub for unknown languages only | yes — <i>"this spec does not mandate any particular treatment of the info string"</i> |
| <b>frontmatter</b>                                                                              | nothing (it is pre-parser, not parser)                                            | yes, but the value slot is YAML and 170 of 907 blocks already fail to parse           |

Plus **one flat document-scoped string→URL table** — the link reference definitions.

CommonMark explicitly permits backslash escapes and character references inside titles and info strings, verbatim: *"Entity and numeric character references are recognized in any context besides code spans or code blocks, including URLs, link titles, and fenced code block info strings."* `[primary, spec.commonmark.org/0.31.2]` That is what makes both channels able to carry arbitrary structured strings — subject to every hazard in §4.4.3.

4.11.4 Composability: constructs do not nest

Measured, and it is the cross-cutting absence no plugin can implement: a checkbox inside a table cell is unexpressible (`<td>- [ ] task in cell</td>` verbatim), and nesting a construct inside a table cell, a footnote inside a heading, or a fence inside a table is either impossible or renderer- specific. `[measured]`

4.11.5 The DocEng eleven patterns — and exactly how much of this is verified

The framing this section's structural claim rests on is Di Iorio, Peroni, Poggi and Vitali, **DocEng 2012, DOI 10.1145/2361354.2361374** — *"a schema-independent theory based on eleven structural patterns"* — with the 2014 JASIST follow-up (DOI 10.1002/asi.23088) describing *"a formal theory of 8 plus 3 structural patterns for XML elements"* derived *"by examining the characteristics of elements as they are used, rather than as they are defined."* **Both abstracts were fetched verbatim.** `[primary]`  
**Now the honesty, because this is the weakest link in the section's most sweeping claim.** The *count* and the *framing* are primary-read. **The NAMES of eight of the eleven patterns are NOT.** Only "block", "container" and "inline element" were recovered verbatim from the 2012 abstract; the other eight (Marker, Atom, Field, Record, Table, Milestone, Meta, Popup) are the researcher's recall of Di Iorio's vocabulary and are tier `[inference]`. The open-access copy at `hdl.handle.net/11380/1199166` returned HTTP 403.

**Therefore the mapping below is `[inference]` built on a partially verified vocabulary. It is the researcher's conclusion, not the paper's, and it must be re-derived against the full text before it is published anywhere outside this plan.**

| PATTERN                                                                                   | MARKDOWN'S COVERAGE                                                                                        |
|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Marker                                                                                    | <code>thematicBreak</code> , <code>break</code>                                                            |
| Atom                                                                                      | <code>inlineCode</code> , <code>code</code>                                                                |
| Field                                                                                     | <code>heading</code> , <code>paragraph</code>                                                              |
| Block                                                                                     | <code>listItem</code> , <code>blockquote</code>                                                            |
| Inline                                                                                    | <code>emphasis</code> , <code>strong</code> , <code>link</code> , <code>image</code> , <code>delete</code> |
| Container                                                                                 | <code>list</code> , <code>table</code>                                                                     |
| Table                                                                                     | <code>table</code> (GFM, <b>no merge</b> )                                                                 |
| Popup                                                                                     | footnote (GFM)                                                                                             |
| <b>Record</b> — named heterogeneous fields on a block                                     | <b>ABSENT</b>                                                                                              |
| <b>Meta</b> , block-scoped — metadata attached to a <i>block</i> rather than the document | <b>ABSENT</b>                                                                                              |
| <b>Milestone</b> , inline — a point marker inside text                                    | <b>ABSENT</b>                                                                                              |

**The three absences are the three additions in §4.7 that people keep trying to build:** the Record is the 18,462 bold-run list items (A2), the block-scoped Meta is what the 330 HTML comments and every `{#id}` attempt are reaching for, and the inline Milestone is what the 1,431 angle-bracket placeholders and the 76,118 overloaded `inlineCode` spans are approximating (A3, A4).

4.11.6 The cross-format check: TWO of six completed

Completed:

- TEI**, `[primary]`, read verbatim from `raw.githubusercontent.com/TEIC/TEI/dev/P5/Source/Specs/att.global.responsibility.xml`: a **global attribute class** — available on **every element** — that *"provides attributes indicating the agent responsible for some aspect of the text, the markup or something asserted by the markup, and the degree of certainty associated with it"*, with `@cert` (*"certainty"*, gloss dated 2009-11-02) and `@resp` (*"responsible party"*, same date). `att.global.source` is also present (HTTP 200). **The consequence for this build:** the per-span provenance capability MDMAX wants to invent — source, method, confidence — has been a **global attribute class in TEI since at least 2009. Adopt TEI's field names ( `@cert`, `@resp`, `@source` ) rather than invent our own**, so the vocabulary has a citable definition on day one. This is the same shape of correction as the annotation-anchoring prior art the plan already records.
- HTML5**, `[measured]`, via GitHub's live sanitizer over 27 distinct real elements plus the attribute matrix in §4.5(b). Additional measured strippings: `<figure>`, `<figcaption>`, `<aside>` and arbitrary custom elements ( `<custom-el foo="bar">custom element</custom-el>` → `<p>custom element</p>` ) are reduced to bare text, while `<section id="s1">` survives.

**NOT completed: DocBook, JATS, DITA, OOXML.** `docbook/docbook` on GitHub now contains only `['MovedToCodeberg', 'README.md']`; Codeberg was outside the harness's network allowlist; the DITA code-search API returned 401; TEI and DITA element inventories were fetched (HTTP 200) but not parsed. **Every DocBook/JATS/DITA/OOXML statement was omitted rather than asserted from memory, and this document does the same.** In particular, the claim that *"markdown collapses ~40 DocBook inline semantic roles into one `inlineCode` node"* appears in the research **only as its measured half** — 13 roles inferred from 77,516 real spans. **The DocBook denominator is not verified. Do not use it.**

4.12 What would falsify this section, and the one experiment that settles it

**The experiment: a carrier survival matrix against a real second corpus and real consumers.** Everything above rests on one corpus and one renderer family, and that is the section's largest weakness.

**Corpus.** Pin a **second corpus of 1,000+ `.md` files not authored by this workspace** — top-200 GitHub repositories by stars with a README plus `/docs` — sha256-manifested with its own `corpus_id`. Re-run: definition count, wikilink count, bare-placeholder count, bold-run list-item rate, `inlineCode` role mix, real-HTML file share.

**Pre-registered predictions, so this can refute rather than confirm:**

| PREDICTION                                                   | IF IT FAILS                                                                                            |
|--------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| link reference definitions <b>&gt;0 but &lt;0.5 per file</b> | A1's empty-slot argument dies (and note Corpus B already reads <b>5.997 per file</b> — see §4.10, C-1) |
| bare placeholders present in <b>&gt;10% of files</b>         | A3's scale claim is workspace-specific                                                                 |
| bold-run description-list items <b>&gt;12% of list items</b> | A2's coverage claim is workspace-specific                                                              |
| real-HTML file share <b>HIGHER than 3.4%</b>                 | the researcher expects to be at least partly wrong here, and said so                                   |

**Tasks: 32 carriers × 12 consumers = 384 cells**, each a deterministic pass/fail, no judge needed. **Consumers must include the four this research could not reach:**

- GitHub Pages** via a real `kramdown` + `kramdown-parser-gfm` install (the gem install into a temporary `GEM_HOME` failed).
- The real GitHub blob renderer** — the API's `mode=markdown` is **not** it.
- Both clipboard flavours driven through a real browser** — `text/plain` versus `text/html`, pasted into Google Docs **and** into VS Code. This is the column §4.9's open problem turns on.
- One non-GitHub sanitizer** — Discourse or Slack.

**Verifiers, cheapest first:** (1) byte-equality after `remark-stringify` and after `prettier` — rules-based, already built; (2) payload-present and payload-visible in rendered HTML — rules-based, already built; (3) a DOM diff for the clipboard round trip.

**Separately and smaller: a 200-document precision trial for A3** against hand-labelled ground truth. Prediction **>90%; below 80% it does not ship.**

**And one measurement nobody in this programme has made: none of this has been tested against a language model.** Zero live model calls were made in any research run. Before any carrier or recogniser is claimed to help an AI read a document, run the read / write / **edit-under-load** trial against the **production** editor model,

not an Opus-class ceiling — the stated failure mode is that a private notation fails *quietly* under load, the renderer stops matching, and the field silently vanishes while the document still looks like a document.

**The kill conditions, restated as a checklist:**

- **Kill A1** if the second corpus shows linkrefs above ~0.5 per file; **or** if `text/plain` also drops them; **or** if D4's export guarantee requires stripping them (100% miss rate → delete rather than ship).
- **Kill A3** if precision <80% on the 200-document ground truth.
- **Kill A2, A4, A5 only** if they start writing bytes.
- **Kill the whole carrier programme** if content-derived re-anchoring is successfully re-derived on an external corpus **and** the 30-line LCS diff baseline (measured at 86.658%, its own verifier reproducing 85.036%) beats it in the product's real conditions — because then identity needs no in-file bytes at all and every carrier above is dead weight on a plain `.md` file.
- **Never ship** any carrier whose payload contains a raw double quote, a blank line, a backtick inside a backtick-fence info string, a link label of exactly 1000 characters, or a case-sensitive identifier. **Each of those is a measured, reproducible, silent failure.**

---

#### 4.13 Summary: DECIDED · RECOMMENDED · OPEN

**DECIDED** (these follow from D2, D4, D6, D7, D8, D9 plus a measurement, and should not be re-litigated):

- The file stays ordinary `.md`. No dialect needing its own parser.
- **No in-file comment or annotation carrier.** GitHub deletes HTML comments in both API modes.
- **No new block or inline syntax.** All six tested proposals are visible literal junk on GitHub.
- **No appearance-based or `id`-based identity.** `id` is lowercased in `gfm` mode.
- **Filter the Unicode Tag block** on ingest and on paste.
- **Render targets are `(product, surface)` pairs,** never products.
- Extensibility lives in the **value** of a field, never in the set of node types (D8); a document **names** a capability, never carries one (D9).

**RECOMMENDED** (an engineer can build these tomorrow; a founder can still say no):

- **A2** description-list recogniser · zero bytes · largest coverage · lowest risk.
- **A3** placeholder linter · zero bytes · **highest severity, cheapest fix, 20.2% of files** · gated at 80% precision.
- **A4** inline-role recogniser over 76,118 `inlineCode` spans · zero bytes.
- **A5** typed-relation resolver over 13,028 wikilinks · zero bytes · make the **block-identity** argument, not the refuted "no plugin can close it" argument.
- Adopt TEI's `@cert` / `@resp` / `@source` names for provenance rather than inventing a vocabulary.
- Ship the **degradation certificate** rule table from §4.4 — two days, hard cap, never the spearhead.
- Default to **not shipping A1** in v1.

**OPEN** (nobody should pretend these are settled):

- **A1's collision with D2 and D4** (§4.9) — three options, none free.
  - **Whether A1's kill condition is already tripped** by Corpus B's 5.997 definitions per file (§4.10, C-1). **Adjudicate before writing any tail-record code.**
  - **The clipboard column** — entirely `[inference]` ; one afternoon with a browser settles it.
  - **The GitHub blob renderer and GitHub Pages** — never executed.
  - **Whether any of this transfers off a corpus written by two people and an agent stack** (§4.12).
  - **The 99.627% anchoring figure**, re-derived by nobody, quoted here twice with that caveat, and under a standing instruction in the current plan not to publish until it is re-derived on a corpus this team did not write.
-

55 Rendering

Rendering – functional and visual, custom rendering, custom formatting, custom parsing

← Index · ← §4 Representation · §6 Conventions →

5. Rendering — functional and visual, custom rendering, custom formatting, custom parsing

5.0 What this section is, in one paragraph

This is the complete rendering specification for frontmatter and MDMAX. It answers five questions the founder asked by name: what will frontmatter render; how does custom rendering work; how does custom formatting work; how does custom parsing work; and where is the line. The answer, stated once so the rest of the section can be checked against it:

**We ship a render POLICY, not a renderer zoo.** A renderer is admitted only when the HOST supplies its configuration, its inputs, and an upper bound on its work. Everything the renderer shows is a PROJECTION — regenerated on every view, never stored, never a source of truth. Only two things may write back into the `.md`, and both are byte-range splices. Everything else in the rendered view is client-side view state that never touches disk.

Every number below carries a source. Corpus numbers cite the pinned corpus id `sha256:3a010b1649899795d79274fc528dbece97fdabf4ff0f81cc02ab619c048c51a4` (1,084 files / 25,548,765 bytes; roots `md` at head `02c22ec4` 756 files / 21,068,853 B, `knowledge` at head `464eb666` 272 files / 3,570,923 B, `frontmatter` at head `798ebbf3` 56 files / 908,989 B — read directly from `docs/engine/research/corpus-manifest.json`). Claims are tagged `[measured]` (we ran it), `[primary]` (we read the source), `[secondary]`, `[inference]`, `[SIMULATED]`, or `[unverified]`.

**Vocabulary, defined once.** A *renderer* is a library that turns a declarative payload into visual output (mermaid, KaTeX, viz.js). A *surface* is a place a document is displayed (GitHub's blob view, GitHub's comment box, frontmatter's preview pane, frontmatter's PDF export). A *target* is a `(product, surface)` pair — never a product, because we measured a single product disagreeing with itself. *Dispatch* is the act of matching a fence's info string to a renderer. *Degradation* is what a non-participating tool shows instead. A *projection* is any output derived from the source bytes: rendered HTML, a PDF, an SVG, the machine view. *Splice* is a replacement of an explicit byte range in the original source, leaving every other byte untouched.

5.1 What every serious renderer already does — the capability matrix

This is the baseline. Anything in the "everyone does it" column is table stakes and buys us nothing; anything in the "nobody does it" column is either an opportunity or a warning that nobody wants it.

5.1.1 The headline correction: GitHub is not three surfaces, it is at least five

The v2.0.0 plan named three GitHub surfaces (blob, comment, Pages). A live measurement found a fourth that integrators actually call and that **disagrees with the blob view on identical bytes**:

```
POST https://api.github.com/markdown [measured, live, final-gate/rendering-frontier C5, CONFIRMED]
``mermaid fence -> <div class="highlight highlight-source-mermaid"><pre>flowchart...
 (syntax-highlighted SOURCE – no diagram)
``geojson fence -> <div class="highlight highlight-source-json"> (it RELABELS the language)
<svg viewBox="0 0 8 8"><path d="M1 1h2"/></svg>
 -> <p></p> (silently DELETED)
<details><summary>s</summary>body</details>
 -> unchanged (passes verbatim)
```

The same four behaviours were reproduced independently by the verifier with its own `curl`. This is the single strongest measurement in the rendering research and it is the reason targets are modelled as `(product, surface)` pairs throughout this plan.

A fifth GitHub surface, GitHub Pages, runs **kramdown**, not `cmark-gfm`, and therefore disagrees with both — most visibly on definition lists (`Term` newline : definition), which kramdown renders as a real `<dl>` while GitHub blob and comment render `<p>Term<br>: definition</p>` `[measured, final-gate/hold-more, CONFIRMED]`.

5.1.2 The matrix

Evidence tier is per row and it is not uniform. Read the tier column before quoting a cell.

TARGET (PRODUCT, SURFACE)	DIAGRAMS	MATH	RAW HTML	ATTRIBUTES KEPT	FENCE INFO STRING	EVIDENCE TIER
<b>github-blob</b>	mermaid, geoJSON, topoJSON, ASCII STL — exactly four [primary]	rendered	9-tag blocklist ( title, textarea, style, xmp, iframe, noembed, noframes, script, plaintext ) [primary, cmark-gfm extensions/tagfilter.c:5-8]	id → user-content- prefix; class / style / data-* stripped; title survives with value intact	class="language-x"	[secondary] — inherited, not re-derived this run
<b>github-comment</b>	same four	rendered	same blocklist	same, but id is lowercased in gfm mode	same	[secondary]
<b>github-api</b> ( POST /markdown )	none — mermaid returned as highlighted source	n/a	inline <svg> deleted to <p>	id lowercased in mode=gfm , case-PRESERVED in mode=markdown	unknown language emits both lang="fm:meta" and data-meta="v1"	[measured, live]
<b>github-pages</b> (kramdown)	none natively	via config	kramdown rules	kramdown IAL {#id} consumed into a real id	class="language-x"	[measured, 24-config bench]
<b>GitLab</b>	—	—	—	—	—	<b>NOT MEASURED.</b> Named in a third-party flavour matrix ( ArchieCur/MARKDOWN_FLAVORS , 14 flavours × 17 constructs) but we ran nothing. Treat this row as EMPTY.
<b>Obsidian</b>	mermaid only	yes	yes	n/a	class="language-x"	[primary] — read obsidianmd/obsidian-help Advanced formatting syntax.md; <b>not run</b>
<b>VS Code preview</b>	zero diagram languages natively	via extension	configurable	n/a	class="language-x"	[primary] — microsoft/vscode extensions/markdown-language-features/package.json ; <b>not run</b>
<b>Zed</b>	no diagram tag at all	—	HtmlBlock tag exists	n/a	CodeBlock tag	[primary] — crates/markdown/src/markdown.rs tag set enumerated; <b>not run</b>
<b>DocuSaurus</b>	mermaid opt-in — install @docuSaurus/theme-mermaid and set markdown.mermaid: true ; off by default	KaTeX opt-in	MDX	MDX/JXS	MDX component mapping	[primary]
<b>Quarto</b>	mermaid + graphviz, via braced info strings {mermaid} / {dot}	yes	yes	yes	{mermaid} — incompatible with GitHub's mermaid	[primary]
<b>VitePress</b>	—	—	—	—	—	<b>NOT MEASURED.</b> Named only in the "no renderer shows the reader that a diagram carried configuration" survey. Treat as EMPTY.
<b>HackMD</b>	—	—	—	—	—	<b>NOT MEASURED for rendering.</b> (It was measured for collaboration: it ships real-time collaboration, inline and page comments, and suggest-edit on the FREE tier [primary] .)
<b>Typora</b>	—	—	—	—	—	<b>EMPTY — CLOSED SOURCE.</b> Nothing was fetched. Do not fill this row from memory.
<b>Bear</b>	—	—	—	—	—	<b>EMPTY — CLOSED SOURCE.</b> Homepage contains 7 occurrences of "markdown", 0 of "comment", 0 of "collaborat" [measured] — that is a marketing observation, not a rendering one.
<b>Craft</b>	—	—	—	—	—	<b>EMPTY — CLOSED SOURCE.</b> Homepage: 0 occurrences of "markdown", "comment" or "collaborat" [measured] .
<b>Notion import</b>	—	—	—	—	—	<b>EMPTY — CLOSED SOURCE.</b>
<b>Slack / Discord</b>	n/a	n/a	n/a	n/a	n/a	<b>Not CommonMark. Model as LOSSY SINKS, never as renderers</b> [primary]
<b>frontmatter-preview</b> (ours)	mermaid 11.15.0, lockfile-pinned	KaTeX 0.17.0	61-element allowlist + 12-element hard deny	see §5.5	class="language-x"	[measured, this session, HEAD 1bd4dad]
<b>frontmatter-pdf</b> (ours)	mermaid loaded from cdn.jsdelivr.net/npm/mermaid@11 — unpinned	KaTeX pinned to 0.17.0	same	same	same	[measured, source read]
<b>frontmatter-export</b> (ours,	mermaid, securityLevel not passed (defaults to strict —	same	same	same	same	[measured, this session]

TARGET (PRODUCT, SURFACE)	DIAGRAMS	MATH	RAW HTML	ATTRIBUTES KEPT	FENCE INFO STRING	EVIDENCE TIER
HTML/DOCX path)	measured at 11.15.0)					

Read the last three rows together. Our own product is three surfaces and they do not agree. `src/modules/export/presentation/pdf-doc.ts:24` sets `const MERMAID_VERSION = "11";` and line 134 imports `https://cdn.jsdelivr.net/npm/mermaid@${MERMAID_VERSION}/dist/mermaid.esm.min.mjs`, while `node -e "require('mermaid/package.json').version"` returns `11.15.0` and the npm registry's `dist-tags.latest` is `11.16.0`. We did **not** fetch jsdelivr (it is not an allowed host in the research sandbox), so "the PDF path serves 11.16.0 today" is [inference] from npm's semver resolution, not an observation. What *is* observed is that the preview path is pinned and the PDF path is not. Additionally `src/modules/export/presentation/export-doc.ts:66` calls `mermaid.initialize({ startOnLoad: false, theme: "default" })` with `no securityLevel`, relying on the library default — which we measured as `"strict"` at 11.15.0, so it is currently equivalent but **unasserted**, and will silently diverge the day that default changes [measured, this session].

### 5.1.3 What the matrix means

- The floor is: CommonMark + GFM + mermaid + math.** Everything above that is opt-in everywhere, including at Docusaurus and Quarto.
- Nobody models surfaces.** babelmark3, the closest prior art, has 38 registered providers of which 23 (60.5%) sit behind encrypted URLs only its maintainer holds, and contains essentially no product surfaces — no Obsidian, Notion, VS Code, Slack, Discord, Typora, Docusaurus, Hugo, Jekyll or Discourse [primary]. It compares *parsers on fragments*. It does not compare *documents across product surfaces*.
- VS Code shipped the user-facing control this whole area needs and nobody copied it:** a per-workspace render security level, exposed as the command `markdown.showPreviewSecuritySelector`, alongside `markdown.previewScripts` / `markdown.previewStyles` [primary]. That precedent — *the host owns render policy and the user can see it* — is a stronger product idea than any diagram feature.

### 5.2 What is renderable from a declarative spec with no runtime

Eighteen candidates, each judged on five columns, then a build decision. "Fits a fence info string" means the language name is a single token with no space, so it survives every renderer's `class="language-x"` transport. "Degrades to readable text" means a non-participating renderer shows the payload as source, not as junk.

#	LANGUAGE	DECLARATIVE SPEC?	FITS A FENCE INFO STRING?	DEGRADES TO READABLE TEXT?	SECURITY SURFACE	DO WE BUILD IT
1	<b>mermaid</b>	yes	yes ( <code>mermaid</code> )	yes — <code>&lt;pre&gt;&lt;code&gt;</code> with payload byte-intact	<b>11 published GitHub advisories</b> , 4 dated 2026-05-11, 2 dated 2025-08-19 [primary] . Includes a <i>pure-declarative</i> Gantt [excludes infinite-loop DoS (GHSA-6m6c-36f7-fhxx) and CSS injection via the document's own <code>%%{init}%%</code> (GHSA-87f9-hvmw-gh4p)	<b>YES — already shipped.</b> With <code>%%{init}%%</code> stripped before dispatch and <code>maxEdges / maxSize</code> set by the host
2	<b>math (KaTeX)</b>	yes	yes ( <code>math</code> , plus <code>\$_\$ / \$\$_\$</code> )	yes	5 npm advisories [primary] . Bounded by <code>maxExpand</code> ; safe <b>only</b> because <code>trust:false</code> is the default	<b>YES — already shipped.</b> Assert <code>trust:false</code> in a test
3	<b>graphviz (viz.js / @viz-js/viz, WASM)</b>	yes (DOT)	yes ( <code>dot</code> )	yes	<b>0 advisories</b> [primary] . No include directive, no network. Needs only a host node cap	<b>RECOMMENDED, not decided.</b> Only if a user asks — see the demand negative below
4	<b>wavedrom</b>	yes (JSON)	yes ( <code>wavedrom</code> )	yes	<b>0 advisories</b> [primary] . JSON data only	<b>NO for now.</b> Zero corpus demand
5	<b>d2 ( @terrastruct/d2 )</b>	yes	yes ( <code>d2</code> )	yes	not audited	<b>NO.</b> 39,961 downloads/week [primary]
6	<b>plantuml</b>	yes	yes ( <code>plantuml</code> )	yes	<b>GHSA-ff3m-68vj-h86p, HIGH severity SSRF, 2023-06-27</b> [primary] ; plus GHSA-hrvf-g648-rf3m stored XSS, 2026-01-16. Also requires a JVM	<b>NO — REFUSED.</b> See §5.3
7	<b>vega-lite</b>	yes	yes ( <code>vega-lite</code> )	yes	<code>vega</code> carries 8 advisories (1 high); <code>vega-lite</code> itself 0 [primary] . But <code>UrlData.url: string</code> <b>makes every chart spec a fetcher</b> — <code>vega/vega-lite</code> <code>src/data.ts:100-105</code> and <code>:125</code> [primary]	<b>NO as authored.</b> Allowable only with <code>data.url</code> rejected at policy time (inline data only)
8	<b>tikz / LaTeX</b>	yes	yes ( <code>latex-tikz</code> )	yes	full TeX is a Turing-complete runtime	<b>NO — REFUSED.</b> It is a program
9	<b>charts (generic)</b>	depends on the spec	yes	yes	inherits from whichever spec	<b>NO.</b> Subsumed by vega-lite's verdict
10	<b>tables with formulas</b>	<b>no</b> — a formula is an expression to evaluate	n/a	n/a	it is an evaluator	<b>NO — REFUSED.</b> See §5.3 rule (2)
11	<b>timelines</b>	yes (mermaid <code>timeline</code> )	yes	yes	inherits mermaid's	<b>FREE — already covered by mermaid</b>
12	<b>maps (geoJSON / topoJSON)</b>	yes	yes	yes	tile fetching is a document-named remote input	<b>NO.</b> GitHub renders these; we would need a tile provider, which breaks rule (2)
13	<b>music notation (abcjs)</b>	yes	yes ( <code>abc</code> )	yes	not audited	<b>NO.</b> 58,394 downloads/week [primary]
14	<b>chemistry (SMILES)</b>	yes	yes ( <code>smiles</code> )	yes	not audited	<b>NO.</b> Zero corpus demand
15	<b>circuits</b>	yes (wavedrom / circuitikz)	yes	yes	see 4 and 8	<b>NO</b>
16	<b>gantt</b>	yes (mermaid <code>gantt</code> )	yes	yes	<b>the Gantt DoS above</b>	<b>Covered by mermaid, with the host work-bound mandatory</b>
17	<b>kanban</b>	yes (mermaid <code>kanban</code> )	yes	yes	inherits mermaid's	<b>FREE — covered by mermaid</b>
18	<b>slides / forms</b>	forms are <b>not</b> declarative — a form is an input surface with a submit action	n/a	n/a	<code>form</code> , <code>button</code> , <code>select</code> , <code>textarea</code> are all in our 12-element hard-deny list	<b>NO — REFUSED</b>

5.2.1 The negative that governs this whole table

Demand for the diagram zoo is absent, measured twice.

```
PINNED CORPUS corpus_id sha256:3a010b16... 1,084 files / 25,548,765 bytes
total opening fences 7,714
distinct info-string first tokens .. 25
top tokens 5,700 bash · 1,361 (none) · 200 markdown · 173 json · ... · 10 mermaid
mermaid 10 (0.13%)
d2, graphviz, plantuml, vega-lite, wavedrom, tikz, music, chemistry ZERO

SECOND CORPUS all *.md under ~/.claude/skills-src, selection_sha256 d6321d4af60256c0308d52490ab33744
474 files / 5,013,124 bytes
opening fences 2,995
distinct info tokens 30
diagram-ish fences 0 (0.0%)

COMBINED: 10 diagram fences in 10,709. [measured, final-gate/rendering-frontier]
```

`docs/engine/PLAN.md` §15.1 already carries the same correction from a different corpus: mermaid appears in **340 of 37,990 external files (0.89%)**, **0.26%** excluding mermaid's own docs, **7 of 4,229 (0.17%)** on this repo's vaults, and **330,496 / 18,841,600 = 1.75%** against GitHub's own `.md` proxy; all non-code diagram/math fences together are **3.45% of fences**. Our two corpora say **0.093%** — an order of magnitude lower again.

**Adoption of the zoo is two orders of magnitude below the parsers.** Weekly npm downloads, window 2026-07-24 to 2026-07-30, `api.npmjs.org/downloads/point/last-week` bulk endpoint `[primary]` :

marked	61,032,058	mermaid	11,828,208
remark-parse	45,820,243	vega-lite	753,796
katex	19,924,866	plantuml-encoder	177,298
		viz.js	81,029
		abcjs	58,394
		@terrastruct/d2	39,961
		vexflow	39,704
		wavedrom	25,307

Two cross-checks let us trust the rest of that list: `marked` at 61,032,058 reproduces the figure `docs/mdmax/PLAN.md` already cites, exactly; and `markdown-it-decorate` returned 1,094, reproducing that cited figure exactly.

**Below mermaid there is nothing with a real user base.** So the honest default is: **ship mermaid and math, ship the POLICY, and ship no third renderer until a user asks for one by name.** If a third is ever added, `graphviz-via-WASM` is first in line because it is the only candidate that passes all three clauses of §5.3 with zero advisories.

5.2.2 The cost side, so the tradeoff is visible

- **Every renderer added is a permanent advisory stream, and a renderer bug fires on VIEW** — the cheapest and most common operation in a collaboration product. Published advisory counts `[primary]` : `dompurify` 27, `marked` 18, `mermaid` 11, `vega` 8, `katex` 5, `@mdx-js/mdx` 0, `vega-lite` 0, `wavedrom` 0, `viz.js` 0.
- **Against that cost, the read-only benefit is already free.** Eleven unknown fence languages run through frontmatter's pipeline and come out with the payload **byte-intact** plus a `language-X` class, at a constant wrapper cost of ~46–58 bytes. That is D8 validated for rendering: **the info string is a free, unlimited dispatch key that costs nothing to refuse.**

**Correction to the research, and it matters.** The rendering-frontier report claimed "12/12 unknown fence languages". The verifier reproduced the payload-intact result through a *stricter* five-plugin chain, but killed the framing: **mermaid is not unknown in shipping frontmatter** — `src/modules/preview/presentation/markdown/components.tsx:132-137` reads `const language = /language-(\w+)/.exec(className ?? "")?.[1]; if (language === "mermaid") return <MermaidBlock .../>`, so it is dispatched to a diagram and never degrades. **The honest figure is 11/12 unknown-and-degraded + 1 already-dispatched.** Neither probe was "frontmatter's real pipeline" either: both omit ReactMarkdown's `components` override and `disallowedElements`. The conclusion is untouched; the number is not 12/12. Use 11/12.

5.3 The runtime line, drawn precisely

This is the rule the founder asked for. It is **not** declarative-versus-imperative — that distinction fails on the first example.

**THE RULE.** A renderer stays a *renderer* if and only if the **HOST** — never the document — supplies all three of: **(1) its CONFIGURATION, (2) its INPUTS, and (3) an upper BOUND on its work.** Break any one and it is a *runtime*, and we refuse it.

5.3.1 Why all three clauses are needed, with the measurement that forced each

**Clause (1) — configuration.** Mermaid, the canonical "safe" example, **already breaks it as shipped.** With the host calling `mermaid.initialize({ securityLevel: 'strict', fontSize: 16 })`, a `%%{init}%%` directive inside the document set `fontSize` to 99 and forced `htmlLabels` to true [measured at mermaid 11.15.0, reproduced independently by the verifier under jsdom]. `MermaidConfig` has **52 top-level keys** and mermaid's `secure` list protects exactly **6** (`secure`, `securityLevel`, `startOnLoad`, `maxTextSize`, `suppressErrorRendering`, `maxEdges`), so **46 of 52 renderer-config keys are document-writable in principle** [primary, `node_modules/mermaid/dist/mermaid.min.js` and `config.type.d.ts`]. The verifier added a datum the researcher missed: `themeCSS` is also document-settable and accepted `"body{background:red}"` at 11.15.0 under `securityLevel:'strict'` — the exact key named in GHSA-87f9-hvmw-gh4p.

**The honest limit of that finding, self-refuted and reported.** The researcher predicted the override would *persist* across diagrams and that `dompurifyConfig` was document-settable. Both are FALSE: `16 → 99 → 16` (a sibling diagram with no directive reads the host's value back), and `dompurifyConfig` stayed `undefined` at HOST, AFTER-A and AFTER-B despite the directive explicitly setting it [measured; reproduced exactly by the verifier]. So **"mermaid is a runtime"** is too strong. **The correct, narrower claim: a document can reconfigure the renderer FOR ITS OWN BLOCK, bounded, and mermaid's security-critical keys hold.** That is still enough to justify stripping `%%{init}%%` before dispatch, because the fix is one line and the alternative is trusting 46 keys.

**Clause (2) — inputs.** Two candidates break it by *their own type systems*: `vega/vega-lite` `src/data.ts:105` declares `export interface UrlData extends DataBase { ... url: string; }` [primary, verbatim], so every chart spec is a fetcher; and PlantUML's include directives produced GHSA-ff3m-68vj-h86p, a HIGH-severity SSRF, 2023-06-27 [primary].

**Attribution correction the verifier made and we keep.** The advisory text reads only *"Server-Side Request Forgery (SSRF) in GitHub repository plantuml/plantuml prior to 1.2023.9."* It names **no mechanism**. `"!includeurl"` is the vector" is [inference], not [primary]. The HIGH-severity SSRF is confirmed; the mechanism is not.

**Clause (3) — work bound.** A purely declarative spec can still be a denial of service. Mermaid's `Gantt` `excludes` attribute — pure declaration, no expressions — caused an infinite-loop DoS **on render, not on parse**, through 11.14.0. GHSA-6m6c-36f7-fhxx, verbatim: `'mermaid.parse is unaffected, unless you then call the ganttDb.getTasks() (which is called when rendering a diagram).'` [primary] And every declarative renderer that survived contact with the web added an explicit bound: KaTeX bounds macro expansion (`katex.mjs:14650-14651`: `if (this.expansionCount > this.settings.maxExpand) { throw new ParseError("Too many expansions: infinite loop or " + "need to increase maxExpand setting");` [primary, character-for-character]), mermaid locks `maxTextSize` and `maxEdges` into its 6-key secure list. An attempted TeX expansion bomb produced only **34× amplification in 1 ms** [measured].

**Context the research omitted, in the direction that cuts against it.** Both cited mermaid advisories list `first_patched_version: 11.15.0` — **the exact version frontmatter has installed** (vulnerable range `>= 11.0.0-alpha.1, <= 11.14.0`). The *structural* argument (declarative ≠ bounded) survives intact. Any implication that we are *currently exposed* to those two does not. [verifier correction, CONFIRMED]

5.3.2 Three on each side

SIDE	EXAMPLE	WHICH CLAUSE DECIDES IT
RENDERER — admit	mermaid, with <code>%%{init}%%</code> stripped and host <code>maxEdges / maxTextSize</code>	(1) restored by stripping; (2) inline only; (3) host caps
RENDERER — admit	KaTeX with <code>trust:false</code> asserted in a test	(1) host-set; (2) inline TeX only; (3) <code>maxExpand</code>
RENDERER — admit	graphviz via viz.js WASM	(1) no directive channel; (2) no include, no network; (3) host node cap — the only candidate that passes all three unmodified
RUNTIME — refuse	MDX	(1) and (2) both — it is a program. And see §5.3.3
RUNTIME — refuse	PlantUML	(2) — include directives, HIGH SSRF, plus a JVM
RUNTIME — refuse	Vega/Vega-Lite as authored	(2) — <code>UrlData.url</code> makes every spec a fetcher

The borderline case is the reason clause (3) exists, and it is worth stating loudly. KaTeX sits on the renderer side **only because** `trust:false` is the default. Flip it — the single line a well-meaning integrator adds to "support links in math" — and the same source produces a live `javascript:` anchor and an `<img>` fetching an arbitrary remote URL from inside a math block:

```
katex 0.17.0, jsdom-parsed [measured; reproduced by verifier]
trust:false \href{javascript:alert(1)}{x} -> {"anchors":[], "imgs":[], "errNodes":0}
\includegraphics{https://...} -> no
trust:true \href{javascript:alert(1)}{x} -> (live)
\includegraphics{https://...} ->
```

Ship an assertion, not a comment. A test that fails if `trust` is ever truthfuly.

5.3.3 MDX, and why it is refused on a ground independent of security

`docs/engine/PLAN.md` §15.5 states the deciding constraint, and it is not adoption: **MDX cannot write the document back out**. Markdown's `format()` is byte-identical and idempotent; MDX has no equivalent and *cannot have one*, because arbitrary JavaScript expressions do not losslessly re-serialize to source. For a product whose entire thesis is splice-only writing (D7, Foster et al., TOPLAS 2007, Lemma 3.9), that is disqualifying. MDX also **deletes four CommonMark constructs in three lines** (`autolink`, `codeIndented`, `htmlFlow`, `htmlText`) and **silently miscompiles valid CommonMark**: `the set {1,2}` renders as `the set 2`, with no warning at any stage `[primary, §15.5]`.

5.4 Custom rendering — the mechanism

This is D8 applied to the render path: **extensibility lives in the VALUE of a field, never in the SET of node types**. For rendering, the "field" is the fence info string and the frontmatter vocabulary. An unknown fence language round-trips byte-perfectly; an unknown *node type* throws.

5.4.1 The dispatch table

One host-owned table, keyed by fence language, consulted **before** the body reaches any renderer. This is the artefact the research could not find anywhere in the ecosystem — the closest existing things are `rehype-sanitize` (an element allowlist with no notion of fence dispatch, 8,423,529 downloads/week) and mermaid's own six-key `secure` list (per-renderer, invisible to the host, not composable across renderers) `[measured, search-only evidence of absence — the weakest kind; see §5.12]`.

`action` has exactly three values and they are the whole API surface:

refuse **and** degrade **produce the same document. They differ only in whether we tell the reader why.** That is what makes this design cheap to be wrong about: refusing a language costs readable source, never junk.

5.4.2 The registration API

```
// src/modules/preview/domain/render-policy.ts (NEW — does not exist yet)
export type RenderAction = "render" | "degrade" | "refuse";

export interface LanguagePolicy {
 action: RenderAction;
 renderer?: string; // name@exact-version — never a range
 configSource: "host" | "none"; // clause (1). "document" is not a legal value.
 stripPatterns?: readonly RegExp[]; // applied to the body BEFORE dispatch
 inputSource: "inline-only" | "none"; // clause (2). "document-fetched" is not legal.
 workBound?: Record<string, number>; // clause (3). Absent => action must not be "render".
 assert?: Record<string, unknown>; // asserted by a test, not at runtime
 reason?: string; // required when action === "refuse"
}

/** Register a renderer. Throws at REGISTRATION time, never at render time, if the
 * policy would violate the §5.3 rule. A missing workBound on a `render` action is
 * a registration error, not a warning. */
export function registerLanguage(lang: string, policy: LanguagePolicy): void;

/** Pure lookup. Unknown language => the `default` entry => degrade. Never throws. */
export function policyFor(lang: string): LanguagePolicy;
```

Three properties make this safe to hand to a team:

- 1. **It fails at registration, loudly.** Per Learned Rule #67 and §15.8 Rule 3, the gate is the parser/registrar, not a runtime check that can be skipped.
- 2. `policyFor` **never throws**. An unknown language is not an error condition; it is the default path, and the default path is `degrade`.
- 3. **There is no way to express "the document configures the renderer"**. `configSource` has two legal values and `"document"` is not one of them. The rule is enforced by the type, not by review.

5.4.3 The fallback, and what a non-participating renderer shows

TOOL	WHAT IT SHOWS FOR AN UNREGISTERED <code>``dot</code> FENCE
frontmatter with <code>dot</code> registered	the rendered graph
frontmatter with <code>dot</code> unregistered	<code>&lt;pre&gt;&lt;code class="language-dot"&gt;</code> — payload byte-intact
GitHub blob / comment	syntax-highlighted source (or plain, if unrecognised)
GitHub API <code>POST /markdown</code>	<code>&lt;div class="highlight highlight-source-dot"&gt;</code>
Obsidian / VS Code / Zed	a code block
Quarto	a code block — Quarto needs <code>{dot}</code> , braced, and <code>dot ✱ {dot}</code>
prettier 3.8.3	<b>does not reformat an unknown-language body</b> ; it <i>does</i> rewrite a known-language body ( <code>const a=1; → const a = 1;</code> ), which invalidates any content hash committed to that block <code>[measured]</code>

That last row is a real design constraint, not trivia. If we ever hash a fence body, the hash is stable for unknown languages and unstable for known ones.

5.4.4 The defect that blocks this today — FIX BEFORE ANY SECOND LANGUAGE

`src/modules/preview/presentation/markdown/components.tsx:133` extracts the language with `/language-(\w+)/`. `\w` **excludes** `-`. Therefore:

```
class="language-vega-lite" -> captures "vega" WRONG
class="language-latex-tikz" -> captures "latex" WRONG
class="language-fm-meta" -> captures "fm" WRONG
```

remark itself emits the full `language-vega-lite` class — verified — so **the defect is entirely in the consumer regex** `[measured, verifier ND2]`. Every hyphenated language collides with its prefix. The dispatch table above is unimplementable until this is `/language-( [\w. :-]+ )/` or equivalent. It is a one-character-class fix and it is a **prerequisite**, not a nice-to-have.

5.5 Custom formatting — how the visual layer works without shipping appearance

5.5.1 The hard constraint, measured and corrected

The brief states it as: *GitHub strips class, style and data-\*, only `id` survives, namespaced AND lowercased in gfm mode*. **That is right in substance and incomplete in two ways that change design decisions** `[measured, final-gate/hold-more C4, CONFIRMED — all six responses byte-identical]`:

ATTRIBUTE	POST /MARKDOWN	MODE=GFM	MODE=MARKDOWN	NOTE
id	id="user-content-payload7q" — prefixed and LOWERCASED	id="user-content-PAYLOAD7Q" — prefixed, case PRESERVED		Any case-sensitive identifier (base64url, nanoid, ULID) COLLAPSES when a document passes through the comment box.
title	survives with its value intact on <code>&lt;span&gt;</code> and <code>&lt;a&gt;</code>	survives		A second surviving attribute nobody had recorded
class	stripped	stripped		
style	stripped	stripped		
data-*	stripped	stripped		

Independently re-verified against **GitHub production** (not just the API): `<div id="x" class="c" style="color:red" data-k="v">hi</div>` → `<div id="user-content-x">hi</div>` [measured, capability-run/degradation-certificate verifier].

**THE LAW: YOU CAN SHIP STRUCTURE. YOU CAN NEVER SHIP APPEARANCE.** A `.md` that carries appearance is a `.md` that renders differently everywhere, which is the exact failure this product exists to eliminate.

5.5.2 How frontmatter renders richly anyway

The file carries **structure and names**. The host carries **appearance**. Three layers, in strict order:

1. **Structure in the markdown.** Headings, lists, tables, fences, blockquotes, task items, GFM alerts — all standard, all already portable. Nothing custom.
2. **Names in the frontmatter, per D9 (a document NAMES a capability, never CARRIES one).** A document may say `theme: technical-note` or `mdmax: { integrity: strict }`. It may **never** say `--accent: #1a5cff`. The name is a lookup key into a host-owned stylesheet; if the host does not know the name, the document renders in the default theme and loses nothing but polish.
3. **Appearance in the host stylesheet, keyed on structure and on the theme name.** All of it lives in our CSS. None of it lives in the file. This is also why frontmatter's own preview can look far better than GitHub's without the file becoming non-portable: we are styling `.markdown-body h2`, not `#user-content-whatever`.

The `title`-attribute finding gives us one extra, legitimate channel — a tooltip-bearing `<span title="...">` survives GitHub's sanitizer intact and is byte-stable through remark + prettier. Use it for **link-scoped facts only**, never for block identity: it is availability-bound (1,838 links against 142,802 paragraphs in the pinned corpus) and it is dropped entirely by `remark-rehype` without `rehype-raw` [measured, hold-more Tier 2].

5.5.3 Our own renderer's formatting policy — measured at HEAD `1bd4dad`, and it disagrees with the research

I read the source directly this session. **The research says 62 allowed elements and 13 denied; I count 61 and 12.** The integrity area's verifier also counted 61. **My measurement governs** — it is first-hand, at a named commit, and independently corroborated:

```
src/modules/preview/presentation/markdown/html-policy.ts
DISALLOWED_RAW_HTML_ELEMENTS = 12 base, button, embed, form, iframe, link, meta,
 object, script, select, style, textarea
 -> children.splice(i, 1) SILENTLY DELETED

SAFE_HTML_ELEMENTS = 61 a abbr b blockquote br caption cite code col colgroup
 dd del details dfn div dl dt em figcaption figure
 h1-h6 hr i img input ins kbd li mark ol p pre q rp rt
 ruby s samp section small span strong sub summary sup
 table tbody td tfoot th thead time tr u ul var
 replaced by a TEXT node carrying the literal source
everything else -> (html-policy.ts:183-186) VISIBLE RAW MARKUP
```

**Two different refusal behaviours, and nobody documented the split.** The practical consequences:

- `<video>`, `<audio>`, `<canvas>`, `<picture>`, `<progress>`, `<meter>` and — most awkwardly — **inline** `<svg>` all appear to the reader as raw markup, e.g. `<p>&#x3C;svg viewBox="0 0 24 24">&#x3C;path d="M1 1h2"/>&#x3C;/svg></p>` [measured]. Our own house icon standard mandates inline SVG; **our own renderer cannot render it**.
- `bdi`, `bdo` and `wbr` are absent from the 61 [measured, this session], which matters because they are the standard remedy for bidirectional-text problems.
- **We are already stricter than GitHub at the markdown layer.** GitHub's `cmark-gfm` ships a **nine-tag blacklist** ( `extensions/tagfilter.c:5-8`, verbatim: `static const char *blacklist[] = { "title", "textarea", "style", "xmp", "iframe", "noembed", "noframes", "script", "plaintext", NULL, };` ) [primary]. We ship a 61-element allowlist plus a 12-element hard deny. Integrators enable raw HTML more often than they sanitize it, by **1.70:1** ( `rehype-raw` 14,287,197/wk vs `rehype-sanitize` 8,423,529/wk ) [primary] — so a host-owned allowlist is a genuine differentiator.

5.5.4 Two live defects in our formatting layer

**(a) The `style` ATTRIBUTE is not filtered at all — only the `style` ELEMENT is.** `sanitizeProperties` ( `html-policy.ts:145-164` ) deletes keys beginning `on` and unsafe URL schemes for a fixed `URL_ATTRS` set ( `href`, `src`, `xlink:href`, `action`, `formation`, `poster`, `background`, `ping`, `cite`, `data` ). `style` **is in neither set**. So `<div style="position:fixed;inset:0;background:url(https://tracker.example/p.gif)">` passes through intact [measured]. This is a **rendering-integrity** hole, not a script hole: a shared document can beacon on open (a read receipt) and can cover the rendered page with a full-viewport layer — the document renders one thing and looks like another, using only allowed markup.

**Correction to the research, found this session.** The integrity area claims *"test/preview/html-policy-xss.test.ts (327 lines) has zero coverage of style/hidden/bidi/zero-width."* **That is wrong on `style`.** The file (328 lines by my count) contains two explicitly named tests — **F-2 "passes the style attribute through untouched (overlay phishing)"** and **F-2b "style attribute can beacon the reader's IP via background url()"** — which assert the *current* behaviour with a comment explaining it is deliberate and unaddressed. The claim is correct on `hidden` (0 occurrences), `bidi` ( `202e:0` ) and `zero-width` ( `200b:0` ) [measured, this session]. **This is documented, tested, known behaviour, not an undiscovered bug** — which changes the fix from "we missed it" to "we decided not to, and now we are deciding again."

**Blast radius, measured, and it is zero today:** across a 4,823-file / 91,394,509-byte corpus, **0 of 24,063 raw-HTML AST nodes carry `style=` or `hidden=`** [measured, capability-run/integrity verifier]. So this is a latent hole, correctly prioritised below the write-back defect in §5.9.

**(b) The sanitizer is not the last stage of its own pipeline.** `src/modules/preview/presentation/Markdown.tsx` sets the rehype chain as `rehypeRaw → createRehypeHtmlPolicy(content) → rehypeKatex → rehypeHighlight → rehypeSlug` (the `useMemo` spans lines 53–62; the five plugin entries are lines 55–59 — the research cites both ranges and both are correct at different granularity). **Anything KaTeX or the highlighter injects is never seen by the policy.** The observable consequence: an author-written `<math>` element is escaped to visible source text, while KaTeX's generated `<math xmlns=...>` renders — **the same element, two verdicts, decided purely by plugin order** `[measured]`.

**DO NOT "FIX" THIS THE OBVIOUS WAY.** The research's own design item (3) — "move the policy to the end of the chain" — is **self-destructive as written** `[verifier ND1, serious]`. `scrubHtmlNode` replaces any element not in the 61-element set with a TEXT node, and KaTeX 0.17.0 emits `math`, `semantics`, `mrow`, `msup`, `mfrac`, `mi`, `mn`, `annotation` — **none of which are in that set** — plus ~13 inline `style=` attributes per expression carrying its entire layout. Running the policy last would **text-ify every MathML node**; worse, KaTeX-generated nodes have no `position`, so `literalSourceForNode` returns `undefined` and the fallback emits `<math>...</math>` as escaped literal junk. **The reorder and the `style` filter must BOTH be preceded by (i) allowlisting the MathML element set and (ii) exempting KaTeX-generated spans from the style filter, or math rendering breaks outright.**

5.6 Custom parsing — the pass pipeline and where a recogniser hooks in

5.6.1 The passes

MDMAX's compiler is a pass pipeline over the source bytes. Rendering consumes passes 1–3 and 7; it never mutates them.

PASS	NAME	WHAT IT DOES	RENDERING'S RELATIONSHIP TO IT
1	<b>parse</b>	source bytes → mdast, via micromark/remark. Byte offsets preserved on every node.	reads
2	<b>identify</b>	assigns a durable, content-derived anchor to every block	reads (the render policy reports per-block)
3	<b>interface</b>	extracts the document's public surface — headings, links, tags, frontmatter keys	reads (feeds the right rail, §5.10)
—	<b>RECOGNISE</b>	<b>the hook.</b> Pure, additive, read-only tree decoration. <b>No bytes are written.</b>	<b>this is where custom parsing lives</b>
4	<b>resolve</b>	resolves links and wikilinks against the project	reads
6	<b>check</b>	diagnostics, on the E/W/I confidence ladder ( <code>docs/engine/PLAN.md</code> §4)	reads
7	<b>project</b>	emits a projection: rendered HTML, PDF, the machine view	<b>this is rendering</b>

5.6.2 Why recognisers beat new syntax

**Section 4 of this plan reaches the same conclusion from the carrier side: the honest top-5 is ONE new carrier plus FOUR recognisers.** The rendering evidence is independent and agrees:

- A recogniser's **degradation is exactly what the construct does today in every renderer, because nothing changed.** That is a zero-risk deployment with the largest coverage available.
- New block or inline syntax is **visible literal junk on GitHub** in every proposal tested — `{#id}` leaks its literal text into the rendered heading in **21 of 24** engine configurations (only kramdown ×2 and pulldown-cmark with `ENABLE_HEADING_ATTRIBUTES` consume it) `[measured, 24-config bench; reproduced exactly at 21/24 by the verifier]`.
- And the strain that is largest in the corpus is precisely the one where adding syntax is most clearly wrong: `Term` newline : `definition` renders as `<p>Term<br>: definition</p>` on GitHub blob **and** comment while rendering as a real `<dl>` on GitHub Pages, because kramdown implements it. **Shipping it makes the same file correct on one GitHub surface and broken on another — the (product, surface) problem, self-inflicted** `[measured]`.

5.6.3 The four recognisers, with their measured coverage

All four are **read-only tree decoration**. None writes a byte. All four render exactly as they render today in every other tool.

#	RECOGNISER	WHAT IT LIFTS INTO THE TREE	MEASURED COVERAGE IN THE PINNED CORPUS
R1	<b>description list</b>	<code>- **Term** - definition</code> → a <code>descriptionList</code> node	<b>7,938 unambiguous</b> instances (18,462 including the weaker bold-run form). Verifier's AST-native re-derivation is <b>higher</b> : 19,402 bold-open list items (21.94%) and 8,044 with a separator, across 539 files
R2	<b>placeholder / metavariable</b>	bare angle-bracket tokens that are not valid HTML5 elements, or that are but read as metavariables	<b>1,431 tokens, 186 distinct names, in 219 of 1,084 files (20.2%)</b> . Verifier reproduced 1,418–1,431 in 201–219 files
R3	<b>inline-code role</b>	classify <code>inlineCode</code> spans into filepath / command / identifier / cli-flag / constant / prose	<b>76,118 spans</b> doing ≥13 jobs (exact; the 13-bucket breakdown sums to 77,516)
R4	<b>typed relation</b>	resolve <code>[[wikilink]]</code> against the project, with a certificate line	<b>13,028 wikilinks in 501 files</b> , rendering as literal visible junk on GitHub

**R2 is the highest-severity finding in the whole corpus and the cheapest fix in the report.** GitHub *deletes* these tokens from rendered output, and there are three distinct failure modes in the same document, with no renderer agreeing with another `[measured, GitHub API mode=gfm]` :

```
IN : Run the tool with `--flag` on <target-file> and then <name> is bound.
OUT: <p>Run the tool with <code class="notranslate">--flag</code> on and then is bound.</p>
 ^^ both placeholders GONE

IN : Filed <C> new issue<s>, added <E> comment<s> on existing issues.
OUT: <p>Filed new issue<s>, added comment<s> on existing issues.</s></s></p>
 ^^ <s> opened a strikethrough
 GitHub auto-closes at the
 paragraph end – ACTIVE MARKUP

IN : Set <title> and <base> before the run.
OUT: <p>Set <title> and before the run.</p>
 ^^ <title> ESCAPED, <base> DELETED – two verdicts, one line

Collision counts in the pinned corpus: <base> 66 · <title> 32 · <source> 20 · <s> 4
```

The remediation is verified and requires no new syntax: `<target-file>` → `<code>&lt;target-file&gt;</code>` (preserved); `<target-file\>` → `&lt;target-file&gt;` (preserved). **R2 is a warning plus a one-click fix, not a construct.**

#### 5.6.4 The five constraints on custom parsing that survive

From `docs/engine/PLAN.md` §15.4, all verified:

1. **The fence body is opaque to every markdown tool.** A link inside a fence does not appear in the mdast link inventory; frontmatter inside a fence produces no `yaml` node. No link checking, no backlinks, no search indexing, no rename propagation. **Every fence is a hole in the graph a linker is trying to build.**
2. `node.data` **never reaches disk.** Confirmed empirically and by zero `.data` hits in `mdast-util-to-markdown/lib/`. **Metadata must be syntax or sidecar — it cannot ride on a node.**
3. **Two parsers, forever.** micromark for the compiler, Lezer for CodeMirror. Every construct needs a shared conformance fixture run against both.
4. **Silent-disable failure modes.** Four spaces of indentation, or one backtick inside a backtick-fence info string, disables dispatch **with no error anywhere.**
5. **The info-string meta dies at HTML serialization** — mdast keeps it, hast keeps it in a non-standard side channel, HTML emits only `class="language-x"`.

**A CONTRADICTION, NAMED.** Constraint 5 (`docs/engine/PLAN.md` §15.4) says HTML emits only the class. The hold-more research measured that **GitHub's renderer emits BOTH** `lang="fm:meta"` **and** `data-meta="v1"` into its sanitized HTML for an *unknown* language, dropping the meta and re-tokenising for a *known* one [measured, CONFIRMED]. **Both are true and neither governs the other — they are different surfaces.** §15.4 governs *our* remark/rehype pipeline. The hold-more measurement governs *GitHub*. This is a textbook (product, surface) fact and it belongs in the certificate, not in a footnote. Do not write "the meta is always dropped"; write "the meta is dropped by remark-rehype and preserved by GitHub for unknown languages."

**Cost of a construct, for calibration:** a full `==highlight==` inline construct — absent from CommonMark and GFM, not expressible as a directive — was built end to end in **139 lines** compiler-side plus **14 lines** for CodeMirror's Lezer parser = **153 total**, with 13/13 round-trip cases stable [measured, §15.3]. And `remark-directive` (3,176,157 downloads/week) already ships `:::name{k=v}` with a full attribute grammar and lossless round trip — ~1,945 lines you do not write. **The cost of a construct is not the reason to avoid one. The reason is degradation.**

#### 5.7 The artifact question — a block that renders to a downloadable PDF or image

The founder raised this directly. Here is the contract, in four clauses.

##### THE ARTIFACT CONTRACT

1. **The round trip is SOURCE-TO-SOURCE, never artifact-to-source.** We parse `.md` → tree, and we write `.md` → `.md` by splice. We never parse a PDF, an SVG, a PNG or rendered HTML and write markdown from it.
2. **Projections are never stored.** Not in the file, not in a sidecar, not in a cache that anything reads as truth.
3. **The artifact is regenerated on every request,** from the source bytes plus the pinned render policy.
4. **Every artifact carries a RECEIPT.**

Clauses 1–3 are already honoured by shipping code, and that is worth stating plainly: `src/app/api/export/pdf/[...path]/route.ts` builds the HTML, renders it headless, and returns the bytes in the `Response`. **Nothing is written to disk** [primary, confirmed by the verifier].

**Clause 4 is the one the prior research missed, and it is the reason the contract needs a fourth clause at all.** The projection is **non-deterministic**:

```
src/app/api/export/pdf/[...path]/route.ts:150–157 [primary, verbatim]
// Give the mermaid CDN module time to download and render diagrams.
await new Promise<void>((resolve) => setTimeout(resolve, 2000));
const pdfBytes = await page.pdf({ format: "A4", printBackground: true, ... });
```

A fixed 2,000 ms sleep, then snapshot whatever exists. **There is no failure if a diagram did not render, no version receipt, and no float pin.** Combined with the unpinned `mermaid@11` CDN import at `pdf-doc.ts:134` (while the preview is lockfile-pinned to 11.15.0 and the registry's `latest` is 11.16.0), the same document can produce different PDFs on two consecutive days. **"We never store it" does not fix that — it makes it worse, because you cannot even diff two runs to notice.** Note that `pdf-doc.ts:24` pins `KATEX_VERSION = "0.17.0"` for the stylesheet, so **the PDF path is half-pinned and the unpinned half is the renderer.**

**Quarto has already solved the artifact question, and its answer costs a browser.** Diagram format is chosen **by target**: JavaScript for HTML, a `mermaid` code block for GFM, a rasterized PNG (via headless Chrome) for pdf/docx/epub — *"a PNG image is created using Chrome"* [primary, quarto-dev/quarto-web docs/authoring/diagrams.qmd]. It also uses **braced** info strings, so the identical diagram needs `mermaid` for GitHub and `{mermaid}` for Quarto. **One language, two incompatible carriers, decided by the consumer.** Quarto pins nothing about which Chrome.

What ships:

```
mdmax export <file> --to pdf|png|svg|html
 1. resolve render-policy.json (version-stamped)
 2. render with LOCALLY BUNDLED renderers – no CDN import on any export path
 3. assert per-diagram render-complete; FAIL the export if any block did not render
 (replaces the setTimeout(2000))
 4. emit a receipt alongside the artifact, never inside it:
 { source_sha256, renderer: "mermaid@11.15.0", policy_sha256,
 resolved_config_sha256, output_sha256, generated_at, blocks_rendered: "14/14" }
```

The receipt is a **sidecar or an HTTP header, never a byte written into the** `.md` — §15.7's manifest rule and D2 both forbid the alternative. This artefact — *{source sha256, renderer name + exact version, resolved config sha256, output sha256, plus an explicit did-every-block-render assertion}* — is one of the four things the research searched for and could not find anywhere in the ecosystem.

5.8 Dual rendering — the human view and the machine view

5.8.1 The law

Any channel that nobody can compare against the thing it describes decays to one of three states: absent, a filename, or a lie.

The evidence for this law, stated at its true strength — which is weaker than the brief states it.

- [measured, own corpus, underpowered] Of 21 markdown images in the pinned corpus, **ZERO carry alt text of three or more words**: 4 empty (19.0%), 6 equal to the filename (28.6%), 11 under three words (52.4%). All 11 raw `<img>` tags carry no `alt` attribute at all. That is **32 of 32 degenerate**. The direction is unambiguous; **n is 21 and 11. Do not publish 100% as a rate**.
- [secondary] *"WebAIM Million 2026 found pages USING ARIA average MORE errors than pages without."* Quoted from the capability run's provenance-boundary journal entry. **We did not fetch WebAIM**. Treat as secondary until re-derived.
- [secondary] *"PDF/UA, where 136 failure conditions and 47 conceded to human judgment produced 53.1% tagged-with-errors."* Same journal entry, same tier.
- [UNSOURCED – DO NOT USE] The figures **"PDF 9.5% fully accessible"** and **"85% of TAGGED files defective"** appear in the brief for this section but **could not be located in any source read for it** — not in the two research result JSONs, not in their journals, not in `docs/engine/PLAN.md`. Per the corpus's own known failure mode (seven irreproducible counts reported before the corpus was pinned), **these two numbers are suspect and must not be repeated until someone re-derives them with a citation**. The law does not need them: 32/32 and 53.1% carry it.

**Our own repo supplies the sharpest instance of the law.** `docs/engine/PLAN.md` §17.7: axe-core 4.12.1, run over 2,286 real vault files through this repo's actual pipeline onto its actual publish surface, **fired 7 of its 105 rules**, and **98.5% of 11,695 violation nodes trace to two root causes in page chrome**. Every *content-level* markdown defect produced **zero** violations — not because they are absent, but because `` compiles to `alt=""`, which is **valid**: it asserts *decorative*. **Missing and decorative are the same bytes. A green axe report on markdown is a false green.**

5.8.2 What is safe and what rots

CHANNEL	SAFE OR ROTS	WHY
Rendered HTML for humans	safe	continuously compared — the author looks at it
The <code>.md</code> source	safe	it <i>is</i> the document; D6
A projection regenerated on view (§5.7)	safe	cannot drift from its source by construction
alt text	rots	nobody sees it; 32/32 degenerate in our corpus
<code>aria-*</code> attributes	rots	WebAIM: pages using ARIA average <i>more</i> errors [secondary]
PDF tagging	rots	53.1% tagged-with-errors [secondary]
<code>llms.txt</code> and any AI-only sidecar	rots	same mechanism: the author never sees the second rendering
A stored machine view	would rot	which is exactly why we never store one

5.8.3 The mitigation: one keystroke, two panes, never one rendering

The tool's non-negotiable invariant is: **NEVER SHOW ONE RENDERING**. Divergence is only stable while exactly one rendering is displayed. Two panes or it does not ship.

```
mdmax explain FILE --as <consumer>
```

A **differ**, not a linter. It emits the exact byte string a named consumer delivers to a model, plus the delta against what a human sees. Four channel classes and only four:

- HIDDEN** — in MACHINE, not in HUMAN (frontmatter, comments, link destinations, HTML attributes, invisibles).
- LOST** — in HUMAN, not in MACHINE (images, indentation a RAG hop stripped, lines past a cap).
- ADDED** — in neither the file nor the render (line numbers, pack preambles, fence wrappers).
- RESHAPED** — same content, different segmentation.

**Consumer profiles are pure functions** `bytes -> delivered string`: `human` (the rendered baseline, the only non-machine profile), `claude-code / read-tool`, `repomix`, `gitingest`, `rag:langchain-header`, `rag:recursive-char`, `mcp-resource`, `http`, `chat-paste`.

`--emit` **is what makes it verifiable**: it prints the literal delivered bytes to stdout, so you can pipe MDMAX's prediction into `differ` against the real tool's output — which is how a user can prove us wrong.

5.8.4 The numbers in the machine-view research, at their VERIFIED strength

This area's headline had **no verifier verdict recorded**, and the capability run it belongs to had a **100% headline defect rate** — every headline refuted by its own verifier, always in the flattering direction. Use only the per-claim verdicts:

CLAIM	VERDICT	WHAT TO ACTUALLY SAY
Read-tool line numbering costs <b>+14.991%</b> tokens over the file's own bytes	<b>CONFIRMED</b> — reproduced to the digit (n=1984, 888,223 lines, rawTok 11,894,041 → InTok 13,677,024, bytesPerTok 3.877)	Quote it, with two riders: it is <b>format-specific</b> to Claude Code's unpadded <code>N\t</code> (the padded <code>%d\t</code> variant measures <b>29.926%</b> , exactly 2x), and the measurement did <b>not</b> apply the 2,000-line cap (59 files, 2.974%, exceed it)
"hard-wrapped prose is <b>materially</b> more expensive to an agent"	<b>REFUTED</b>	measured <b>1.0–1.3%</b> , an order of magnitude below the headline it was attached to. <b>Drop "materially."</b>
"and no tool surfaces this"	<b>REFUTED</b>	Claude Code ships <code>/context</code> — <i>"Visualize current context usage as a colored grid..."</i> [primary] . The defensible claim is narrower: <b>no tool attributes the surcharge to the line-number prefix or lets you diff it against the file's own bytes.</b>
" <b>2.924%</b> of tokens a model receives are invisible in the rendered document"	<b>OVERSTATED, 4.7x</b>	arithmetic reproduces exactly, semantics fail for 94.7% of the total: 61.59% of "hidden" link-destination tokens belong to links whose visible label <b>is</b> the destination, and frontmatter visibility is renderer-dependent. <b>The honest number is ~0.626%.</b>
"~200x larger than the Unicode-injection channel"	<b>REFUTED</b>	no derivation; the same numbers give <b>28,989x</b> . Either say 28,989x or delete the comparison.
naive invisible-character detector "fires on 4.94% of files with a <b>0% true-positive rate</b> "	<b>REFUTED as stated</b>	counts replicate (114/2308 = 4.9393%, 375 occurrences, 0 malicious) but <b>TPR is undefined on a corpus with zero known positives</b> , and the corpus is self-authored — the wrong population for an injection detector. <b>Say: "375 fires, 0 malicious, on a 2,308-file self-authored corpus with a zero base rate — this bounds nuisance volume, not detector precision."</b>
LangChain's markdown splitter destroys indentation on <b>34.61%</b> of fenced-code lines and deletes the Unicode Tag block	<b>NOT VERIFIED</b> — headline only	Label [unverified] until re-derived.
<code>.md</code> twin vs HTML = <b>17x bytes</b> ; editing block 92 of <code>docs/engine/README.md</code> discards <b>54%</b> of the cached prefix	<b>NOT VERIFIED</b> — headline only	Label [unverified] .

**The invisible-character UI, corrected.** Not a red badge. A **neutral, always-on chip** rendering every invisible codepoint inline ( `{ZWNJ}` ), in three tiers: **expected** (variation selector after an emoji or keycap base; ZWJ/ZWNJ within 2 codepoints of a complex-script character; BOM at offset 0) rendered grey and uncounted; **unusual** (invisible inside a run of pure Basic Latin) rendered amber; **never-legitimate** (the Unicode Tag block U+E0000–E007F, bidi overrides U+202A–202E and U+2066–2069) rendered red, decoded to plaintext, **and this tier alone may fail CI**. Measured cost of the middle tier: **11 of 4,823 files (0.228%)**. HTML comments stay silent and counted — **909 occurrences in 457 files = 9.475%**; anything that fires on one file in ten is a nag, not a signal. **Injection detection is not phrasing-based.** The tool's honest claim is *"here is everything present that you cannot see"*, never *"this is an attack."*

5.9 Interactivity without a runtime — exactly what writes back

5.9.1 The write-back allowlist, closed by construction

Only two things may write to the file from the rendered view. Both are byte-range splices. Everything else is client-side view state that never touches disk.

AFFORDANCE	WRITES TO THE <code>.MD</code> ?	MECHANISM
<b>Task checkbox</b> <code>- [ ]</code> ↔ <code>- [x]</code>	<b>YES</b>	single-line in-place replacement — a genuine splice ( <code>src/modules/editor/presentation/toolbar-transforms.ts:112-124</code> )
<b>Frontmatter property edit</b>	<b>YES, after the fix in §5.9.2</b>	byte-range splice over the YAML scalar
<code>&lt;details&gt;</code> open/closed	no	client state
Tabs	no	client state
Sortable table column order	no	client state
Collapsed sections / outline fold	no	client state
Semantic-zoom level, pan, selection	no	client state
Diagram pan/zoom	no	client state
<b>Table cell edit</b>	<b>NO — DELETED, not fixed</b>	see §5.9.3
Any form input	no	<code>form</code> , <code>button</code> , <code>select</code> , <code>textarea</code> are in the 12-element hard deny

5.9.2 The write-back defect — THIS IS THE HIGHEST-PRIORITY ITEM IN THE SECTION

frontmatter's only wired rendered-editing surface is not byte-faithful.

```
A NO-OP round trip: parseFrontmatter -> stringifyFrontmatterDoc, NO EDIT MADE
corpus_id sha256:3a010b16... [measured; CONFIRMED by independent re-implementation]

files with an editable frontmatter map 737
files CHANGED by the no-op round trip 623 = 84.5%
total [byte delta] 46,732
taxonomy: YAML respaced, same line count 421 tags: [daily] -> tags: [daily]
 YAML REFLOWED, line count changes ... 197
 separator/body only 5

 623 (421+197+5 = 623, checks)

VERIFIER CORRECTION, ANTI-FLATTERING: the researcher UNDERSTATED it.
"moves line numbers in 26.7%" counts only YAML-block reflow (197/737).
The WHOLE-FILE line count changes in 226/737 = 30.7%, because
stringifyFrontmatterDoc also normalizes the separator to `---\n\n${body}`,
shifting lines in 29 further files.
```

**It is wired, not dead code.** `src/modules/editor/presentation/EditorPane.tsx:647` renders `<PropertiesPanel content={previewSource} onEdit={handleEdit} />` with **no feature flag**; `handleEdit` (`EditorPane.tsx:498-502`) calls `setContent(path, newContent)` + `setDirty(path, true)`

- `saveDraft(...)`; the emitter is `PropertiesPanel.tsx:79` `onEdit(stringifyFrontmatterDoc(parsed.doc, parsed.body))`, reached from `setValue / renameKey / removeKey / addProperty`. The serializer is `src/modules/preview/presentation/frontmatter.ts:46`: `export function stringifyFrontmatterDoc(doc: Document, body: string): string { const yaml = doc.toString()...`

**This is the exact defect this plan cites as an EXTERNAL cautionary tale, reproduced internally.** `docs/mdmax/PLAN.md §4.2` (line 121), verbatim: *"DesktopCommanderMCP issue #440 (2026): an AI tool added a rich-text model and began silently rewriting users' .md — YAML collapsed to single lines, [x] → \[x], blank lines stripped — and it fired on read-only operations."* Our properties panel does the YAML half of that, at 84.5%, in shipping code, reached through the RENDERED view.

**THE FIX, verified feasible:** replace `stringifyFrontmatterDoc` with a range splice. The `yaml` package (2.9.0, already a dependency) exposes `node.range` on every AST node. For `title: A\ntags: [daily]\nn: 3\n` the verifier obtained `tags -> val.range [15,22,23]` and `src.slice(15,22)` returned exactly `"[daily]"`. **Compute the edited scalar's byte range and replace only that range in the original source. Never call `doc.toString()`.**

**GATE — and this one should stop the team.** The no-op round trip must dirty **0 of 737** files, down from 623. **If a splice cannot get there, the properties panel must be made READ-ONLY before comments ship, not after.** At 84.5% dirty and 30.7% of files changing line count, the panel is currently incompatible with **both** D7 (splice-only) **and** durable block identity — you cannot anchor a comment in a document whose line count moves when someone edits a tag.

5.9.3 The table writer — delete it, do not fix it

`setTableCell` re-serializes the whole table. Editing one cell rewrote 3 of 5 lines of an aligned table, **corrupted a cell containing an escaped `\|` into two cells destroying content**, and added pipes to a pipe-less table:

```
IN : "| Cmd | Meaning \|n| --- | --- \|n| a \| b | alternation \|n"
OUT: "| Cmd | Meaning \|n| --- | --- \|n| a \| X | alternation \|n" [measured]

Over the corpus, a NO-OP cell write (writing the same value back):
 45 of 335 table-bearing files dirtied = 13.4%, touching 199 lines
 7 corpus tables contain `|`
 92 of 1,240 detected "tables" (7.4%, 36 files) START INSIDE a fenced code block
 - the detector is FENCE-BLIND
```

**NEGATIVE, from the researcher's own pre-registered falsifier, and it fired.** `EditableTable` is **DEAD CODE**. `grep -rn "EditableTable" src/ | grep -v editable-table.tsx` returns **empty** (I re-ran it this session: **0 hits**), and `src/modules/preview/presentation/markdown/components.tsx:36` declares its callback as unused `_onEdit`. **Anyone quoting 13.4% as a live defect is wrong. It is a specification for what not to build. DO NOT PROCEED PAST THE NEXT PHASE WITH `EditableTable` IN THE TREE** — it is dead code today, which is the only reason its no-op dirty rate and its `\|` corruption are not already a live data-loss bug.

5.9.4 The checkbox is a splice, with one qualification the design must state

`toggleTaskAtLine` (`toolbar-transforms.ts:112-124`) is a genuine single-line splice. Two riders, both latent:

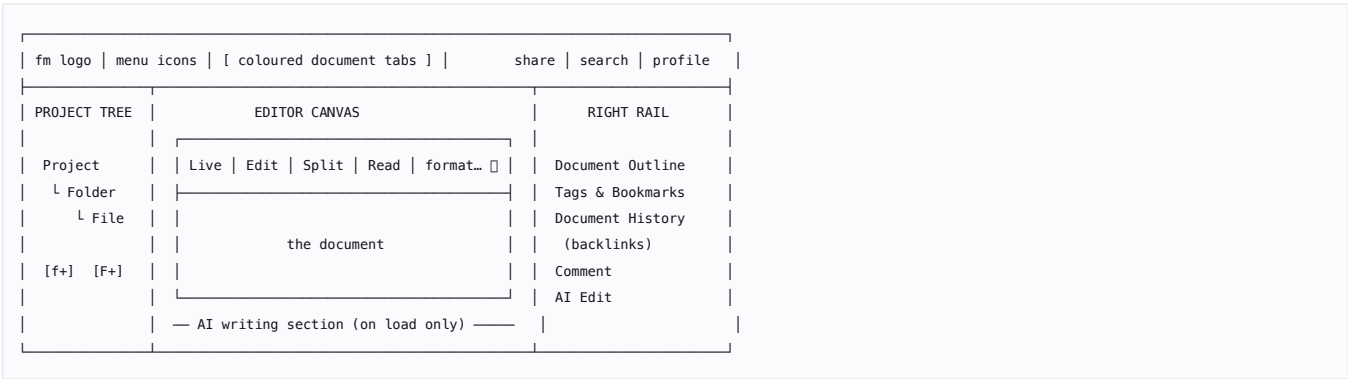
- It **normalizes case**. `/- \|x\|/gi` → `- [ ]`, then `- [ ]` → `- [x]` lowercase. **A double toggle on `- [X]` is NOT the identity.** Measured over the pinned corpus: **0 occurrences of `- [X]` in 0 files** (44 `- [x]` in 12 files). Latent, not live.
- It **uses a global per-line replace**, so a line carrying a literal `- [ ]` inside inline code toggles too.

Both are one-line fixes. **State them; do not assert "already a true line splice" without the qualification `[verifier ND3]`.**

5.10 The visual specification for frontmatter itself

Derived from the three wireframes the founder shared (recorded in `HANDOFF-mdz-markdown-format-2026-07-29.md §1.12`). Reproduced here as a spec, with the engineering consequences attached.

5.10.1 Layout



5.10.2 The four modes

MODE	WHAT IT IS	RENDERING RULE
Read	the projection only	full §5.4 dispatch. <b>Write-back allowlist applies (§5.9.1).</b>
Edit	CodeMirror 6 source only	no renderers run
Split	source and projection side by side, scroll-synced	the projection is a pure function of the source; <b>never the reverse</b>
Live	Obsidian-style live preview — the block under the cursor shows source, every other block renders	the hard one; see below

**Live mode's governing constraint, verbatim from CodeMirror** [primary, docs/engine/PLAN.md §3.8] : *"Decorations that significantly change the vertical layout... must be provided directly, since indirect decorations are only retrieved after the viewport has been computed."*

⇒ **inline syntax** → `ViewPlugin` ; **block height** → `StateField` . Obsidian's shipped API confirms the whole design: `editorLivePreviewField: StateField<boolean>` and `livePreviewState: ViewPlugin<{mousedown: boolean}>` . **Live Preview is not a second editor — it is one boolean on the same instance.** They track mid-click as first-class state to stop decoration churn; two independent open implementations hit the same bug and shipped the same guard. Four concrete rules that fall out:

- Hide markers with `max-width:0; opacity:0` , **never** `display:none` .
- Feed the same `RangeSet` to `EditorView.atomicRanges` .
- Key `WidgetType.eq()` on the **source slice**.
- Track mid-click in a `ViewPlugin` and suppress re-decoration while it is true.

**Split and Live share one invariant with §5.7:** the rendered pane is a projection. A user edit in the rendered pane is only legal if it maps to a splice on the allowlist.

5.10.3 The right rail IS the compiler's output

This is the structural observation from the handoff, and it is the reason the rail is a spec item rather than a design flourish:

RAIL PANEL	WHAT IT IS, IN COMPILER TERMS	WHICH PASS PRODUCES IT
Document Outline	the symbol table	pass 3 (interface)
Tags & Bookmarks	the tag index	pass 3
Document History (backlinks)	the reverse-reference index	pass 4 (resolve)
Comment	the annotation overlay — <b>sidecar, D2: nothing about comments is ever written into the .md</b>	outside the file entirely
AI Edit	a proposed splice, shown as before/after, never applied silently	pass 7 + the write gate
Problems ← MISSING	the diagnostics panel	pass 6

**THE MISSING PANEL IS** `Problems` , and without it **every diagnostic the format layer produces has nowhere to appear**. Everything in §5.6.3 (recognisers), §5.8 (`explain`), the render policy's `refuse` banner and `mdmax cert` all emit findings with no home in the current wireframe.

**But it must be a COMMAND, not a persistent panel, and the data says so.** A compiler run over 833 authored files found **365 true problems touching only 112 files (13.4%)** — a random note has an **86.6%** chance of an empty panel, and **one file held 56% of everything**. Broken wikilinks were **84% of raw output at 2–25% precision**, because an unresolved `[[link]]` is a *legitimate* authoring primitive [measured, docs/engine/PLAN.md §4] . ⇒ **a project-level `check` command is earned by this data; a persistent Problems panel is not.**

And the sharpest constraint on it: a `no-top-heading` check fires on **81% of files** and is **false at publish scope**, because the published view injects the `<h1>` from frontmatter so body `h2` s nest correctly. **Heading diagnostics are only sound when evaluated against the COMPOSED OUTLINE, never against the file** — shipping it file-scoped would emit **515 false warnings** and train the user to ignore the panel [measured, §17.7] .

5.10.4 The AI Edit panel and the write gate

**Every AI write passes through the parser before touching disk.** A parse failure or a semantic round-trip mismatch triggers repair, never a silent save [docs/engine/PLAN.md §15.8 Rule 3] . Do **not** use grammar-constrained decoding: empirical coverage on complex schemas is **Guidance 41%, llama.cpp 39%, XGrammar 28%, Outlines 3%**, and Grammar-Aligned Decoding (NeurIPS 2024) shows naive masking *"distorts the output distribution"* — you get strings that are grammatical and low-likelihood, i.e. valid and bad.

**The failure mode this prevents, stated precisely.** JSON fails *loudly* — a parser throws, a retry fires. A private prose notation fails *quietly*: the model writes `~ doing` instead of `~doing` , the renderer does not match, and **the field silently vanishes**. The document still looks like a document. Nothing throws.

`AI Edit` , `sync` and external sync all give **before/after** — this is already the plan's rule and it is a rendering requirement, because the "before" is a projection of the stored source and the "after" is a projection of a *proposed* source that has not been written.

5.10.5 One thing the right rail must NOT try to render

The agent never sees an edge list. LLMs score 18.8–23.0% counting nodes and 10.2–15.0% counting edges on 5–20 node graphs; a deterministic walk pays (HippoRAG 2's PageRank is worth +12.5 where the LLM's contribution is 0.7 of 87.1). Rank offline, ship flat. And do not emit a generated overview — measured to not improve task success while costing >20% more inference [docs/engine/PLAN.md §3.7] . The backlinks panel shows a ranked flat list to the human and hands the model procedures and preserved structure, never a graph.

5.11 What ships, in dependency order, with gates

#	ITEM	WHY IT IS IN THIS POSITION	GATE
1	Fix the frontmatter write-back — range splice, never doc.toString()	84.5% of files dirtied by a no-op; blocks D7 and blocks comments	no-op round trip dirties 0 of 737
2	Delete EditableTable / setTableCell	dead code today, data-loss tomorrow	grep -rn EditableTable src/ returns empty and the file is gone
3	Fix the dispatch regex /language-(\w+)/ → /language-([\w.-:~+])/	every hyphenated language currently collides with its prefix	language-vega-lite captures vega-lite
4	Ship render-policy.json + registerLanguage	the actual deliverable of this section	mdmax render-policy prints the {config, inputs, bound} table for every dispatched language
5	Strip {{init}} before mermaid dispatch; set host maxEdges / maxTextSize ; assert KaTeX trust:false	clauses (1) and (3)	tests fail if a document changes fontSize or themeCSS
6	Allowlist the MathML element set, then reorder the rehype chain, then filter style	in this order — reordering first breaks math (ND1)	math still renders; <div style="position:fixed"> no longer passes
7	Pin the PDF projection + emit the receipt	bundle mermaid locally; replace setTimeout(2000) with a per-diagram render-complete assertion	ten renders of the same document, 24 h apart, produce one output sha256
8	mdmax cert --render	per document, per fence language, per (product, surface) : PASS · STRIP · CORRUPT	the first row it prints is our own product failing its own consistency check (preview 11.15.0 vs PDF mermaid@11 )
9	mdmax explain --as <consumer>	\$5.8; two panes or it does not ship	--emit output is byte-identical to the real tool's output
10	The four recognisers (R1–R4)	read-only, zero degradation risk, largest coverage	no bytes written; render output unchanged

Certificate target set — fixed here because the research contradicted itself twice in four lines [verifier ND4] . The research prose said "SIX GitHub-adjacent surfaces" then listed six of which two are not GitHub-adjacent, and its own CLI example used a different six. The governing list is:

```
mdmax cert <file> --render \
 --targets github-blob,github-comment,github-pages,github-api,frontmatter-preview,frontmatter-pdf \
 --fail-on CORRUPT
```

Four GitHub surfaces plus our own two. quarto-html / quarto-pdf are optional extras, not part of the default set. And github-blob cannot be probed without pushing content — the blob renderer is only reachable via GET /repos/.../contents/X.md with Accept: application/vnd.github.html . Model github-blob as a DECLARED, DATED contract (frontmatter → table; HTML comments → deleted; id → user-content- prefixed; class / style / data-\* → stripped), refreshed on a schedule with a canary repo. Never pretend it is a live check.

5.12 What this section does NOT cover

1. **GitHub, VitePress, HackMD, Typora, Bear, Craft and Notion import as RENDERERS.** Four are closed source and nothing was fetched; three simply were not measured. Their rows in §5.1.2 are **EMPTY**, not "renders the basics".
2. **Obsidian, VS Code and Zed as RUNNING SOFTWARE.** Those rows are [primary] reads of docs and source. **None of the three was run.** A doc can lag its renderer.
3. **github-blob / github-comment / github-pages verdicts.** Only the **github-api** surface was measured live in the rendering research. The "GitHub is five surfaces" claim rests on one new measurement plus three inherited.
4. **Any exploit.** No exploit was constructed or run. Every security finding is advisory text plus source reading, with exactly one negative own-measurement (dompurifyConfig is not document-settable).
5. **Token cost of any projection**, beyond KaTeX's measured 123.7× byte amplification on \frac{a}{b} . **Zero live model calls were made in any research run.** Every "easier for AI" statement anywhere in this section is a **prediction, not a result** [inference] .
6. **Real-time collaborative rendering.** Multi-cursor, presence and CRDT/OT merge are out of scope here.
7. **The 99.627% anchoring figure**, which appears elsewhere in this plan and **has been re-derived by NOBODY**. It is not used in this section and must not be quoted without that caveat.
8. **The absolute fence census.** The 7,714 figure uses the researcher's own fence parser (open on three-or-more backticks/tildes at up to three spaces of indent, close on the same character at ≥ length, empty info). It is **not CommonMark-exact** — indented-code-block context is absent. **The RANKING is robust; the absolute number is not.**
9. **The "nobody has built X" claims.** They rest on registry.npmjs.org keyword search, whose total field is the size of the registry rather than a match count. **Search-only evidence of absence — the weakest kind.** GitHub code search required authentication and returned "Requires authentication" unauthenticated.

5.13 What would falsify this section

#	FALSIFIER	CONSEQUENCE
F1	A conformance sweep finds fewer than 3 languages with ≥4 distinct verdicts across 7 surfaces — i.e. the surfaces broadly agree	Degradation is not a differentiator. Cut <code>mdmax cert</code> to a single lint rule. (We would be surprised: github-api and github-blob already disagree on mermaid on identical bytes, measured live.)
F2	The range splice cannot get the frontmatter no-op round trip to exactly 0 dirty files	STOP. Make the properties panel read-only before comments ship.
F3	After mermaid and math, no fourth language both (a) appears in a real user corpus and (b) passes the {host config, host inputs, host bound} test	The renderer roadmap is over. This section's deliverable is <code>mdmax cert --render</code> + <code>mdmax render-policy</code> and nothing else. Our two corpora say the count today is ZERO.
F4	Ten PDF renders of the same document, 24 h apart, produce one digest under the CURRENT unpinned code	The determinism argument in §5.7 is weaker than stated; the receipt is still worth shipping but the urgency drops.
F5	A user, when shown the machine view, does not change the document	§5.8's mitigation does not work and the whole dual-rendering argument is decorative. This is the only falsifier here that requires a human, and it is the most important one.
F6	Registering a second renderer produces a support/advisory load exceeding one engineer-day per quarter	The policy is right and the zoo stays at two forever.

One meta-caveat, and it applies to every number in this section. The 16-area final gate returned 5 CONFIRMED, 10 OVERSTATED, 1 REFUTED; the 18-area capability run had a 100% headline defect rate, every headline refuted by its own verifier, always in the flattering direction. The rendering-frontier area is one of the five CONFIRMED, and its verifier called its falsifier discipline *"the strongest in this program"* — three pre-registered falsifiers, one survived, two fired and were reported as findings rather than dropped. That is why this section leans on it. The verifiers themselves have a measured false-kill rate of 0–19%, so a kill is strong evidence, not proof. Where a kill looked wrong to me — the integrity area's claim that the XSS test file has zero `style` coverage, when it contains two explicitly named `style` tests — I said so in §5.5.4 and my own measurement governs.

56 Conventions

Our own conventions: the library model, custom definitions, reference pointers, and whether any of it ships

← Index · ← §5 Rendering · §7 Product →

6. Our own conventions: the library model, custom definitions, reference pointers, and whether any of it ships

This section answers one founder question in full: *should MDMAX let a document declare its own vocabulary and import a library of definitions, the way a Python file imports from PyPI?*

It is the only idea in this plan that has been raised repeatedly across four sessions and cut every time. It deserves the evidence, not a one-line dismissal. What follows is the complete case — the idea as stated, every precedent that exists, the reference design if we ever build it, the rule that would govern it, the experiment results that constrain its shape, the single measurement that decides it, the verdict, and the question that sits upstream of all of it and is still open.

Read the verdict box first, then decide how much of the rest you need.

6.0 The verdict box

DECIDED	No declared-vocabulary feature, no vocabulary registry, no library-import syntax, and no new carrier ships in the first twelve months. Nothing in <code>.md</code> gains the ability to name or carry a definition.
DECIDED	We ship the inverse instead: <b>recognition</b> of the vocabulary a corpus already uses, plus drift detection over it. Zero new syntax, zero new bytes in any user file. Specified in §6.10.
RECOMMENDED	If a declaration is ever built, build the design in §6.3 and §6.6 exactly — per-project sidecar, <code>\$schema</code> key reused not invented, denotations only, hard cap, filesystem-only resolution, warn-and-degrade on absence.
RECOMMENDED	Amend D9 from "a document NAMES a capability, never CARRIES one" to "a document may CARRY a denotation; it must only ever NAME a procedure; and it may lower its own capability, never raise it." The amendment is evidence-driven (§6.4) and changes nothing operationally this cycle, because nothing declares anything this cycle.
OPEN	Does MDMAX become a programming language, or admit it is a thin shell over one? This is upstream of every syntax question and it is <b>unanswered</b> . §6.9 lays out both branches. It does not need answering in the next twelve months — but it must be answered before anyone writes a line of import syntax.
OPEN	Four named observations would reverse the no-ship decision. They are listed as kill conditions in §6.8. One of them — a single real user asking, unprompted, to name their own construct — outweighs every corpus number in this section.

The one number that decides it: the observed user-invention rate across 5,245 markdown files in five corpora is **0.31%**, and **all of it is tutorial content teaching the feature**. §6.7 derives that number, states what it does and does not prove, and lists the two pieces of evidence that cut against it.

6.1 The idea, in the founder's words

Verbatim, from `HANDOFF-mdmax-markdown-engine-2026-08-01.md:95` :

"if it is a library importing things and people are not jargon at all they can use features that they want to... in Python there are thousand libraries people might use hundreds only."

[primary] — read from the file, not paraphrased.

The instinct behind it is correct and worth stating plainly, because the rest of this section is going to argue against shipping it and that argument is worthless if it strawmans the idea.

**What the founder is actually asking for.** Markdown has a small fixed vocabulary. Every attempt to grow that vocabulary — callouts, admonitions, directives, wikilinks, diagrams — has meant a fork, a plugin, or a dialect. The Python analogy proposes a way out that does not require anyone to agree on a spec: let the *document* say which vocabulary it uses, let a *library* hold the definitions, and let each author pull only the parts they want. Nobody has to learn 1,000 things. They learn the ten they use. The format stays small; the capability grows outside it.

**Concretely, that decomposes into four separable mechanisms.** They are usually discussed as one thing, and they have completely different costs. Keeping them apart is the single most useful move in this whole analysis:

#	MECHANISM	WHAT IT IS	CAN IT EXIST ALONE?
M1	Declaration syntax	a line in the document that names a vocabulary — <code>\$schema:</code> , <code>filters:</code> , <code>#lang</code> , <code>@context</code> , <code>import</code>	yes
M2	Resolution	turning that name into a definition on disk or over the network	yes, given M1
M3	Distribution	a registry: a place to publish, version, and fetch vocabularies from	needs M1+M2
M4	Absence semantics	what happens when the named thing is missing, wrong-versioned, or corrupt	needs M1+M2

The founder's sentence names M3 ("*a library importing things*") and M1 ("*features that they want*"). The evidence below is that M3 has succeeded exactly twice in the history of document formats, that M1 is where all the security failures live, and that M4 is where LaTeX's famous pain comes from. **M2 is the only one of the four with a cheap, safe, well-understood answer** — and M2 is what Section 3's resolver already gives us, without any of M1, M3, or M4.

**Why the analogy does not transfer, in one line, before the evidence:** PyPI distributes *behaviour*. A markdown vocabulary registry would distribute *names*. Nobody downloads names.

6.2 The prior art, complete

This is the entire history. It is short.

6.2.1 The two that succeeded

Only **two document formats have ever acquired a real package ecosystem** — meaning: the document declares the dependency, and a registry resolves it.

FORMAT	PACKAGES	AGE	GOVERNANCE	VERSION	RESOLUTION
LaTeX / CTAN	7,021 packages across 610 topics	~35 years	five named volunteers, no automated review	none	— date comparison only, and a mismatch is a warning
Typst	1,481 distinct packages / 4,425 name:version rows	~3 years	one git repo + a GitHub Action + a CDN		mandatory full pinning, no solver, no lockfile

[primary] CTAN counts fetched live 2026-07-31T22:45Z from `https://ctan.org/json/2.0/packages` (7,021 entries) and `https://ctan.org/json/2.0/topics` (610), and re-fetched independently 2026-08-01 → still **7,021** (final-gate area `custom-pointers`, finding 12). TeX Live's package database

`mirror.ctan.org/systems/texlive/tlnet/tlpkg/texlive.tlpdb.xz` (20.4 MB decompressed) has 8,079 `name` entries, 7,064 of `category Package`, 5,139 excluding `binary/doc/src` variants.

[primary] CTAN's governance, verbatim from its own uploader addendum: *"the following persons (in alphabetical order) take care of CTAN: Erik Braun (upload management and system administration), Vincent Goulet (upload management), Manfred Lotz (upload management, programming, system administration), Gerd Neugebauer (web developer), Petra R be-Pugliese (upload management). All these people are acting as volunteers in their spare time"*. Thirty-five years, seven thousand packages, five people, no automated review, no version solver.

[primary] Typst counts fetched live 2026-08-01 from `https://packages.typst.org/preview/index.json` (2,060,620 bytes, 4,425 rows, 1,481 distinct `name`, newest `updatedAt` 1785509080 = 2026-07-31T14:44:40Z — the index is live, not stale).

Both are Turing-complete programming languages whose surface syntax happens to be documents. That is not a coincidence and it is the load-bearing fact of this entire section. A package distributes *behaviour*: a LaTeX package redefines typesetting; a Typst package computes layout. If the host format cannot execute anything, a package has nothing to carry.

6.2.2 Everything markdown-adjacent, and what each one refused

SYSTEM	DECLARATION	RESOLUTION	REGISTRY	PRIVILEGE OF THE EXTENSION
Quarto	yes, per-document — <code>filters:</code> [fancy-header] in <code>.qmd</code> frontmatter	filesystem only, from <code>_extensions/</code> vendored into the repo	none of its own; 250 extensions listed on quarto-web	Lua run inside Pandoc, or an arbitrary executable
Sphinx	project only — <code>extensions = [...]</code> in <code>conf.py</code> , a Python file	Python import	borrows PyPI ( <code>Framework :: Sphinx :: Extension</code> )	full host Python
MkDocs	project only — <code>plugins:</code> in <code>mkdocs.yml</code>	setuptools <code>entry_points</code>	borrows PyPI	full host Python
Pandoc	refused in-document on purpose — filters are command-line only	CLI argument	none	arbitrary executable; <code>--sandbox</code> explicitly does <i>not</i> cover filters
R Markdown	<code>library()</code> inside a knitr chunk	R's own loader	borrows CRAN	full host R at render time
MDX	real ESM <code>import / export</code> inside markdown	delegated to the JS bundler	built none at all	full host JavaScript
Obsidian	the <code>.md</code> file can declare nothing	app settings, a user action	own catalogue: 6,222 plugins	full Electron privileges

[primary] all rows from `docs/engine/research/wf-findings-2026-08-01.md`, AREA *"Document-format package ecosystems"*, each with a source read in that run.

Three of these deserve their exact words.

**Pandoc — the maintainer refused document-carried configuration explicitly.** John MacFarlane, on `jgm/pandoc#4627`, verbatim: *"The whole point of pandoc is that you can convert a single document to different formats, in different ways -- so including instructions about this in the document itself seems wrong."* And on a default configuration file: *"I'd worry about the security implications of a default defaults file."* Pandoc's `MANUAL.txt` on `--sandbox`, verbatim: *"Note that this option does not limit IO operations by filters or in the production of PDF documents... Anyone using pandoc on untrusted user input should use this option."* [primary]

**Quarto — the reason for vendoring is reproducibility, stated by Quarto.** From `quarto.org/docs/extensions/managing.html`, verbatim: *"If you are using version control you should check the `_extensions` directory in to your repo along with your other code. Extensions used by a document or project are treated as source code to ensure very long term reproducibility—your project doesn't need to rely on the availability of an external package manager (or the maintenance of older extension versions) to successfully render now and far into the future."* Same docs, a boxed warning titled *Extension Trust*: *"Quarto extensions may execute code when documents are rendered. Therefore, if you do not trust the author of an extension, we recommend that you do not install or use the extension."* [primary]

**Obsidian — the largest markdown-adjacent plugin ecosystem, and the document declares nothing.** 6,222 plugins (live fetch of `obsidianmd/obsidian-releases/master/community-plugins.json`). Obsidian's own security document, verbatim: *"By default, Obsidian runs in Restricted Mode to prevent third-party code execution."* And: *"Due to technical limitations, Obsidian cannot reliably restrict plugins to specific permissions or access levels. This means that plugins will inherit Obsidian's access levels... Community plugins can access files on your computer. Community plugins can connect to internet. Community plugins can install additional programs."* Enablement is a user action in app settings — **never a document-level directive.** [primary]

6.2.3 A contradiction between two of our own documents — named, and resolved

Two documents in this repository disagree about Quarto, and the disagreement matters because Quarto is the closest analogue to what the founder is proposing.

- `HANDOFF-mdmax-markdown-engine-2026-08-01.md:256` states: *"Everything markdown-adjacent — Quarto, Sphinx, MkDocs, Pandoc, R Markdown — deliberately refused document-declared registry resolution."*
- The final-gate verification of area `custom-pointers` marks the broader form of that claim **REFUTED**, quoting `quarto-web/docs/extensions/filters.qmd` verbatim: *"If you've developed a filter and want to use it within a document you need to add it to the list of filters for the document"*, with a worked example placing `filters: / - fancy-header` in a **single document's frontmatter**, resolved against `_extensions/fancy-header/_extension.yml`, which contributes `fancy-header.lua` — **a procedure executed on the Pandoc AST.**

**Which governs: the verification.** It quotes primary source, and the final gate's own kill audit measured that source-quoting kills are **16 of 16 correct** while number-substituting kills run a 38% defect rate ( `kill_audit.note`, sampled 29, overall false-kill rate **5/29 = 17.24%**). A kill backed by a verbatim quotation is the strongest class of evidence in this entire corpus.

The corrected statement, which is the one to use everywhere from now on:

Per-document declaration exists in the wild. Quarto ships it. What no markdown-adjacent system ships is document-declared **registry** resolution — a name in the file that causes a **fetch**. Quarto resolves locally, from a directory the repo owner committed, with no network.

This is not a threat to the design in §6.3–§6.6. **It is prior-art support for it**, and it should be cited that way: Quarto is a shipping, widely-used system that instantiates almost exactly the rule this section recommends — name a procedure, resolve from the filesystem, never fetch. MDMAX choosing *per-project* over *per-file* is a further restriction we impose, not evidence that per-file is unprecedented.

#### 6.2.4 Three more data points that shape the answer

- **MDX is the closest syntactic precedent and it built no registry.** Real `import / export` statements inside markdown; resolution delegated entirely to the JS bundler and npm. [primary] Separately and independently disqualifying for us: **MDX cannot write the document back out** — arbitrary JavaScript expressions do not losslessly re-serialize to source, so there is no `format()` equivalent ( docs/engine/PLAN.md §15.5). For a product that edits and saves, that is fatal regardless of market share.
- **The in-band pragma was tried in JavaScript and abandoned.** tc39/proposal-modules-pragma is **archived at Stage 1** — `api.github.com/repos/tc39/proposal-modules-pragma`: `archived=true`, 81 stars, last push **2022-01-24T18:58:28Z**. JavaScript put the declaration in `package.json` instead. [primary] *Caveat: the "boilerplate tax" rationale previously attributed to TC39 is a paraphrase and is not in the primary source; the archival is fact, the stated reason is not.*
- **The registry operating burden is now enormous and it is getting worse.** GitHub Changelog 2025-12-09, verbatim: *"We've permanently revoked all existing npm classic tokens. They can no longer authenticate, be recreated, or be recovered."* `npm login` now issues a two-hour session token. This was forced by the September 2025 Shai-Hulud self-replicating worm. [primary] Ecosystem malware volume — **>454,600 new malicious packages in 2025, >1.233 million cumulative (+75% year on year), 120,612 attacks blocked in Q4 2025 alone, ">99% of all open source malware now targets npm"** — is [secondary] (search-summary tier), from Sonatype's 2026 report that **nobody on this team has read**. Treat those four figures as unverified.

### 6.3 The Typst blueprint, in full

If a registry is ever built, this is the design. It is included in full because it is genuinely cheap and because the founders should be able to cost the idea accurately rather than from intuition.

Typst's own vendor blog (typst.app/blog/2023/package-manager, 2023-06-30, Laurenz Mäde, verbatim: *"we took a step back and sketched out a design for a minimum viable package manager that a single person could build in a week."* [primary] There is **no registry service**. There is a git repo, a GitHub Action, and a CDN: *"The package repository has a GitHub action that builds a tar.gz archive for each package and an index.json ... then uploads the packages and index to <https://packages.typst.org/preview>, which is served through a CDN", and "Won't get much simpler than serving a bunch of tar.gz files through a CDN".*

#### The six properties to copy

**1. Namespacing, with a user-owned escape hatch that takes precedence over the cache.** `typst/packages` README, verbatim: *"A package that is stored in `packages/preview/{name}/{version}` ... will become available in Typst as `#import "@preview/{name}:{version}"`." And the escape hatch: *"You can create an arbitrary `{namespace}` ... Store a package in `{data-dir}/typst/packages/local/mypkg/1.0.0` ... Import from it with `#import "@local/mypkg:1.0.0": *`. Packages in the data directory have precedence over ones in the cache directory."* [primary]*

This one property is what makes air-gapped use, enterprise use, and pre-publication development possible **without vendor sign-off**. Ship it on day one or never.

**2. Mandatory full version pinning. No ranges, no floating, no lockfile, no solver.** *"You must always specify the full package version."* The manifest requires only `name`, `version` (full major-minor-patch SemVer triple), and `entrypoint`; `compiler` names a minimum compiler version. Typst is honest about the cost: *"While this is a bit inconvenient, it means that we don't need a manifest or lock file for reproducibility."* [primary]

**3. Post-publish immutability, as stated policy.** `typst/packages` docs, §Fixing or removing a package, verbatim: *"Once submitted, a package will not be changed or removed without good reason to prevent breakage for downstream consumers. By submitting a package, you agree that it is here to stay."* [primary]

Properties 2 and 3 together **structurally neutralise the worst npm attack class at zero engineering cost** [inference]. With `@preview/name:0.1.0` pinned and versions immutable, a compromised maintainer cannot alter any version an existing document already imports; they can only publish a new version nothing references. Contrast npm, where `^1.2.3` means a poisoned publish reaches every consumer on the next install — which is precisely what made Shai-Hulud self-replicating.

**4. On-demand download plus a permanent cache, so the vendor being down never breaks a build.** Verbatim: *"Importing a cached package does not result in network access."* [primary]

**5. Submission is a pull request.** A git repo, a CI job, a CDN. No accounts, no publish tokens — **and therefore no token-theft attack surface**, which is the exact thing that just cost npm its entire credential model.

**6. The WASM plugin contract, if plugins ever exist: non-WASI, byte-buffers only, no ambient authority, purity required.** From `typst.app/docs/reference/foundations/plugin`, verbatim: *"For security reasons, plugins run in isolation from your system. This means that printing, reading files, or similar things are not supported."* And on the ABI: *"Many compilers will use the WASI ABI by default or as their only option (e.g. emscripten), which allows printing, reading files, etc. This ABI will not directly work with Typst. You will either need to compile to a different target or stub all functions."* The protocol is byte buffers: an exported function takes n 32-bit length arguments, calls `wasm_minimal_protocol_write_args_to_buffer(buf.ptr)`, calls `wasm_minimal_protocol_send_result_to_host`, returns 0 for success or 1 with a UTF-8 error message. Purity is required by contract — and Typst's own docs are honest that it is **not enforced**: *"Typst does not enforce plugin function purity (for efficiency reasons)." Isolation is enforced; purity is a promise.* [primary]

#### The two things to fix rather than copy

**Fix 1 — Typst ships no integrity verification, and its own source says so.** From `typst/typst`, `crates/typst-kit/src/packages.rs` (main, 18,229 bytes, lines ~268–272), verbatim source comment: *"This means that we do not check the integrity of an existing moved package, just like we don't check the integrity if the package directory already existed in the first place. If situations with broken packages still occur even with the rename safeguard, we might consider more complex solutions like file locking or checksums."* [primary]

Independently confirmed from the index side rather than the source side: the live `index.json` entry keys are exactly

`['authors', 'compiler', 'description', 'entrypoint', 'exclude', 'keywords', 'license', 'name', 'repository', 'updatedAt', 'version']` — **no hash, digest, or sha field exists** [own-measurement, final-gate custom-pointers finding 12]. CTAN's 7,021-package JSON carries `key / name / caption` only. Trust in both systems is TLS-to-CDN plus policy.

The fix is `@vendor/name:1.0.0#sha256-...` with the digest recorded in the index. It is a few lines of code, and it is what makes a third-party mirror **provably** equivalent to ours. This is a genuinely unoccupied design slot — and it is worth nothing on its own, which is why it is a footnote and not a product.

Fix 2 — LaTeX's absence asymmetry is backwards, and the symmetric level is warn-and-degrade.

CONDITION	LATEX BEHAVIOUR	EVIDENCE
missing package	<b>fatal</b> — interactive prompt; in batchmode the TeX run ends	<code>latex3/latex2e/base/ltfiles.dtx : \gdef\@missingfileerror#1#2{\typeout{^^J! LaTeX Error: File #1.#2' not found....}} [primary]</code>
wrong version	<b>warning only</b>	<code>latex3/latex2e/base/ltclass.dtx (develop, 154,501 bytes) line ~2475: \ifl@ter date comparison then \latex@warning@no@line{You have requested,\on@line, version #3'...} – a warning macro, **not** @latex@error [primary]</code>
bad rollback date	<b>downgraded to a warning</b>	same file, line 3825: <i>"Make suspicious rollback a warning not error: github issue 43"</i> [primary]

That asymmetry — missing is fatal, wrong is a shrug — is the precise mechanical root of LaTeX's reproducibility reputation. **The fix is symmetry, and the symmetric level is warn-and-degrade, not fatal**, because D6 says degradation is the feature. Missing vocabulary, wrong version, and digest mismatch should all produce: the construct falls back to its base markdown form, the document still renders, one diagnostic per distinct cause, and a non-zero exit code only under an explicit `--fail-on` flag. A digest mismatch additionally refuses to use the cached copy.

What the blueprint costs, and why we still do not build it

One person, one week, for the registry itself [primary, vendor's own claim]. The compiler-side cost of a single new construct is measured at **139 lines** (85 micromark syntax extension + 34 mdast handlers + 20 unified/rehype glue, 13/13 round-trip cases stable) plus **14 lines** for @lezer/markdown on the CodeMirror side = **153 lines total across both parsers** ( docs/engine/PLAN.md §15.3) [measured].

So the objection is *not* cost. The objection is demand (\$6.7), and the operating burden a registry acquires the moment anyone attacks it (\$6.2.4).

6.4 The rule that governs any declaration — and the correction the evidence forces

D9 as it stands

D9. A document NAMES a capability, never CARRIES one.

Every self-declaring mechanism that survived contact with the real world fits the *name* half:

MECHANISM	WHAT THE DOCUMENT CONTAINS	WHY IT IS SAFE
JSON-LD @context	an IRI, or a term→IRI map	the vocabulary is defined elsewhere
shebang #!	an absolute path to an interpreter	the path must already exist on the machine
Racket #lang	a collection name under a deliberately crippled grammar	the language must already be installed

Racket is the best-designed of the three and its grammar restriction is the transferable lesson. Racket Guide §17.3.1, verbatim: *"the syntax of language is far more restricted than a module path, because only a-z, A-Z, 0-9, / (not at the start or end), \_ , - , and + are allowed in a language name. These restrictions keep the syntax of #lang as simple as possible. Keeping the syntax of #lang simple, in turn, is important because the syntax is inherently inflexible and non-extensible."* [primary] **That grammar cannot express a URL.** Resolution: *"the language must be installed in a collection"* — no auto-fetch, ever.

And every failure fits the *carry* half:

FAILURE	WHAT THE DOCUMENT CARRIED	OUTCOME
XML internal DTD subset	an entity definition that can dereference ( <code>SYSTEM "file:///..."</code> ) and recurse	<b>XXE</b> and the billion-laughs class
vim modelines	settable options, historically including expressions ( <code>modelineexpr</code> )	a repeated hardening programme, most recently <b>2026-05-18</b>
Emacs file-local variables	variables evaluated on open	a permanent prompt-the-user mitigation
Hugo inline shortcodes	a template — i.e. a procedure	<b>ships disabled</b> , under SECURITY (\$6.5)

*Unverified, and flagged as such:* the frequently-repeated figure that vim modelines produced **five CVEs across 24 years, the most recent four months ago is inherited from an earlier brief and has not been checked against the National Vulnerability Database (NVD) by anyone on this team** — the research sandbox could not reach NVD. Do not publish it. What was verified is stronger and independent: see below.

Correction 1 — the real axis is denotation vs procedure, not name vs carry

D9 as written forbids something the standard it cites explicitly permits.

W3C JSON-LD 1.1 syntax spec, read in full during the research session ( raw.githubusercontent.com/w3c/json-ld-syntax/main/index.html , 578,006 bytes, fetched 2026-08-01), verbatim: *"The value of @context MUST be null, an IRI reference, a context definition, or an array composed of any of these."* And: *"Contexts can either be directly embedded into the document (an embedded context) or be referenced using a URL."* [primary]

An embedded @context is a carried definition, and it is the first form the spec teaches.

That lines up exactly with the experimental result in docs/engine/PLAN.md §15.2: redefining a **denotation** holds (**140/140** on generation, 120/120 on parsing, with a negative control catching 10/10 injected faults, so the harness demonstrably works), while redefining a **procedure** collapses (GPT-4 two-digit addition **98.2% in base 10 → 38.6% in base 9**, with the base stated in the prompt and the comprehension check still passing — Wu et al., *Reasoning or Reciting?*). [measured] / [secondary]

The corrected rule, **RECOMMENDED as an amendment to D9**: A document (or project) MAY carry a **denotation**. It MUST only ever **name** a **procedure**.

Every entry in both tables above fits this rule, and D9 as written does not.

## Correction 2 — the correction's own safety argument is wrong, and this one matters

The research claimed an embedded JSON-LD context is safe because it is *"a closed term→IRI map — no execution, no dereference, no recursion."* **The verifier refuted this from the same document.** JSON-LD 1.1 defines `@import`: "A context definition *MUST* be a map whose keys *MUST* be either terms, compact IRIs, IRIs, or one of the keywords `@base`, `@direction`, `@import`, ...", and "By using the `@import` keyword in a context, another remote context, referred to as an imported context, can be loaded and modified prior to processing", and "If the context definition contains the `@import` keyword, its value *MUST* be an IRI reference." [primary]

**So an embedded context can dereference a remote IRI at processing time.** "Closed" is false. "No dereference" is false. Recursion is bounded only by an explicit rule (an imported context may not itself contain `@import`) — bounded at depth 1, not structurally absent.

**What this changes:** do not cite JSON-LD as evidence that embedded denotations are safe. They are safe *in our design* because of a rule **we** impose — **no URL is ever fetched at parse time** — not because the JSON-LD precedent supplies that guarantee. It does not.

## Correction 3 — the asymmetry, from vim, verified four months ago

The most transplantable rule in the entire corpus is not "name, never carry". It is an asymmetry, and vim converged on it in production this year.

raw.githubusercontent.com/vim/vim/master/runtime/doc/options.txt, verbatim [primary]:

- 'modelinestrict' 'mlst' boolean (default: on) — "When on, only a safe subset of options can be set from a |modeline|", followed by an allowlist, then "Any other option set from a modeline will be silently ignored. This option cannot be set from a |modeline| or in the |sandbox|, for security reasons."
- The asymmetry itself: "As an exception, `set nomodeline` is honored from within a modeline even when 'modelinestrict' is on. Other forms ( `set modeline=0`, `set modeline!`, `set invmodeline` ) are still silently ignored. **This lets a file disable further modeline processing for itself.**"
- And the threat model in vim's own words: "No other commands than "set" are supported, for security reasons (somebody might create a Trojan horse text file with modelines)", plus "disable modelines before editing untrusted text".

Recency, verified through `api.github.com/search/commits?q=repo:vim/vim+modelinestrict`: commit `4c2879471`, **2026-04-14**, "patch 9.2.0350: Enabling modelines poses a risk"; commit `439722711`, **2026-05-18**, "patch 9.2.0499: modeline: allow to disable modelines with modelinestrict". SHAs, dates and subjects all confirmed exactly by an independent verifier.

**The rule: a file-embedded directive may LOWER its own capability and may never RAISE it.** Everything outside a small inert allowlist is **silently ignored** — not an error, ignored.

**Two corrections to how this has been reported internally, both from the verifier and both accepted:**

- The allowlist is 21 options, not 24.** It was stated as 24 twice. The verifier extracted and enumerated all of them: `autoindent`, `cindent`, `commentstring`, `expandtab`, `filetype`, `foldcolumn`, `foldenable`, `foldmarker`, `foldmethod`, `modifiable`, `readonly`, `rightleft`, `shiftwidth`, `smartindent`, `softtabstop`, `spell`, `spelllang`, `tabstop`, `textwidth`, `varsofttabstop`, `varstabstop`. **Use 21.**
- They are not "all presentation-only".** `filetype` fires FileType autocommands, which load ftplugins; `modifiable` and `readonly` are buffer state. The allowlist is *inert with respect to arbitrary execution*, which is the property that matters — but the looser claim is false.

*Confidence note on those two corrections:* the first substitutes a number, which is the class the kill audit measured at a 38% defect rate — however, the verifier enumerated the list item by item, so it is checkable in ten seconds against the primary source and should be re-checked by whoever next touches this. The second quotes semantics and is safe.

## 6.5 The decisive precedent: Hugo built exactly this idea and ships it disabled

This is the single most direct precedent that exists, because Hugo implemented the *inline definition* form of the founder's idea — a document that carries its own construct definition — and then turned it off.

From `raw.githubusercontent.com/gohugoio/hugo/master/docs/content/en/content-management/shortcodes.md`, verbatim [primary]:

"Hugo's security model is based on the premise that **template and configuration authors are trusted, but content authors are not**. This model enables generation of HTML output safe against code injection. To conform with this security model, creating shortcode templates within content is **disabled by default**."

And from `raw.githubusercontent.com/gohugoio/hugo/master/docs/content/en/configuration/security.md`, where the setting lives **on the security configuration page**, listed alongside `exec.allow` and `allowContent`:

"`enableInlineShortcodes` (bool) Whether to enable [inline shortcodes]. Default is `false`."

**Why this transfers to frontmatter with no adjustment.** In frontmatter, the content author is untrusted *by construction*. A reviewer with a login (D3) is a content author. A shared document is authored by whoever you shared it with. A `.md` file is routinely a drive-by pull request to a docs repo, a CMS field, a paste, or model output — whereas a `package.json` is written by the repo owner. That is the decisive difference from npm's threat model, and it runs in the wrong direction for us [inference].

Every serious system in this space drew the same line: Pandoc (filters are CLI-only), Quarto (extensions must be vendored by someone with commit access), Racket (the language must be installed in a collection), Obsidian (Restricted Mode by default), Hugo (inline shortcodes off by default). **Typst is the only system that permits document-declared auto-fetch, and it can only do so because the fetched artifact cannot reach the filesystem, the network, or the shell.**

## 6.6 If a declaration is ever built: the rules from experiment

These are not opinions. Each has a measurement behind it. They are recorded here so that a future version of this team does not re-derive them, and so that if the kill conditions in §6.8 fire, the design is already written.

### Rule 1 — declare by EXAMPLE, not by rule

Aycock et al. (arXiv 2409.19151) ablated MTOB — the benchmark where a model learns a language from a grammar book in context — and found, verbatim: *"almost all improvements stem from the book's parallel examples rather than its grammatical explanations"*, and flatly: *"we find no evidence that long-context LLMs can make effective use of grammatical explanations."* Replicated across Kalamang, Nepali and Guarani. [secondary] — inherited from `docs/engine/PLAN.md` §15.8; nobody on this team has read the paper directly.

**Design consequence:** a vocabulary entry must carry **two or three annotated worked instances** of each construct, never a sentence describing it. This is nearly free and it targets the exact mechanism the whole design depends on.

Rule 2 — cap the vocabulary, and enforce the cap as an error

IFScale (arXiv 2507.11538, 20 models across seven providers), verbatim: *"even the best frontier models only achieve 68% accuracy at the max density of 500 instructions."* Reasoning models hold near perfect through ~100–150 then threshold-decay; mid-size models decay linearly; small models exponentially. There is also **primacy bias** — early instructions are honoured more reliably than late ones. [secondary] — *also inherited; not read directly.*

The geometry is unfavourable in exactly the way that matters: the declaration sits at the top of the file where compliance is highest, and the constructs must fire in the fortieth note, where it is lowest.

**But the cap number does not come from the paper.** It comes from measurement of what real teams actually declare:

CORPUS	DECLARED VOCABULARY SIZE
four real OKF bundles ( type field)	5 / 0 / 5 / 21
Docusaurus admonition defaults	9
GitHub built-in alerts	5
MyST roles (vendor-supplied, not user-declared)	46

The only bundle above 12 is **42.9% self-colliding** (§6.7). **Hard cap: 12 terms. Error, not warning, at 13.** Twelve is where a vocabulary stops being a vocabulary and starts being drift.

*The one honest counter-example, recorded because the researcher recorded it against their own interest:* MyST proves a **46-term** vocabulary is comfortably usable in practice — which contradicts a naive reading of a single-digit cap. Every one of those 46 is **vendor-supplied**, not user-declared. The cap constrains what a user may declare, not what a vendor may ship. *(Figures do not reproduce precisely between researcher and verifier — 46 roles in 25.75% of 431 files versus 66 roles in 28.24% of 432 files — because the two used different role regexes and the branch drifted same-day. The qualitative conclusion is identical in both.)*

Rule 3 — the renderer is already a parser; make it the write gate. Do NOT use constrained decoding

Grammar-constrained decoding fails exactly where a user-declared notation lives. Empirical coverage on complex ("GitHub Hard") schemas: **Guidance 41%, llama.cpp 39%, XGrammar 28%, Outlines 3%**. And Grammar-Aligned Decoding (NeurIPS 2024) shows naive masking *"distorts the output distribution"* — you get strings that are grammatical and low-likelihood, i.e. valid and bad. [secondary]

Validate-and-repair gets the same guarantee at no distribution cost and is the cheapest applicable verifier. Every AI write passes through the parser before touching disk; a parse failure or a round-trip mismatch triggers repair, never a silent save.

**The failure mode this prevents, stated precisely.** JSON fails *loudly* — a parser throws, a retry fires. A private prose notation fails *quietly*: the model writes `~ doing` instead of `~doing`, or falls back to `**doing**` under load, the renderer does not match, and **the field silently vanishes**. The document still looks like a document. Nothing throws. The next agent to read the file sees a note with no status and treats the absence as information.

Rule 4 — where the declaration lives, and what key it uses

**Per-project, never per-file.** A sidecar at the repo or vault root — `.mdmax/vocab/<name>.yaml` — checked into version control. The precedent chain is uniform: TC39 archived the in-band pragma and JavaScript chose `package.json`; Docusaurus puts admonition keywords in a **build-config array**, and its own source comment says why — `packages/docusaurus-mdx-loader/src/remark/admonitions/index.ts`, verbatim: *"// By default it makes more sense to append keywords to the default ones // Adding custom keywords is more common than disabling existing ones"* — with **no per-document form anywhere in the loader** [primary]; Quarto vendors `_extensions/`; every LLM convention (`llms.txt`, `AGENTS.md`, `.cursor/rules`) is a glob-scoped sidecar.

**The one per-file form permitted is the vim asymmetry:** a file MAY carry `mdmax-vocab: none` to switch the feature **off for itself**. A file MAY NOT carry anything that switches something **on** or adds a term.

**Reuse** `$schema`. **Do not invent** `fm-vocab`. The key already ships in `remark-lint-frontmatter-schema` at **36,745 downloads/week** (api.npmjs.org, window 2026-07-24 → 2026-07-30, verified twice), with the precedence question already decided. From its README, verbatim: *"Schema association can be done directly **inside** the frontmatter of the Markdown file, relative to project root, thanks to the '\$schema' key"*, with the example `'$schema': content/creative-work.schema.yaml`, and the rule: *"Locally defined '\$schema' takes precedence over global settings below."* [primary] Adopting it makes MDMAX a consumer of an existing convention rather than the 47th dialect, and it parses correctly under gray-matter (verified in-session).

Value shape if we ever ship it: `'$schema': .mdmax/vocab/sgnk-v1.yaml@1.0.0#sha256-...` — a project-relative path, a full version triple, and a digest.

Rule 5 — what it may name, and what it may never name

**MAY name — denotations only.** Each term is `{term, display-label, value-type, cardinality, docs-url}`.

**MAY NEVER name:** an executable, filter, plugin, or WASM module; a URL fetched at parse time, ever; a new **node type** (D8 holds — an unknown fence language round-trips byte-identically, an unknown node type throws); a redefinition of an existing construct's meaning (*never redefine a symbol CommonMark already defines*, `docs/engine/PLAN.md` §15.7).

Rule 6 — the carrier, ranked by measured round-trip × render × occupancy

This ranking is [measured] against the repository's own `node_modules` (`mdast-util-from-markdown` + `mdast-util-to-markdown` + `mdast-util-gfm`, full GFM enabled).

CARRIER	ROUND-TRIP	VISIBLE?	OCCUPANCY (PINNED / NPM DOCS / DOCUSAURUS)	VERDICT
<b>fence META string</b> — <code>``md fm:vocab=sgnk/v1</code>	<b>identical</b>	no (absent from <code>marked</code> 's HTML)	<b>0.00%</b> (0 of 7,714 fences) / 0.02% (1 of 5,007) / <b>48.01%</b> (6,201 of 12,916)	<b>best block-scoped carrier</b> — but namespace it, because <code>title=""</code> already owns the channel in Docusaurus-shaped corpora
<b>link reference definition</b> — <code>[fm:vocab]: sgnk/v1@1.0.0</code>	<b>identical</b>	renders to nothing	<b>0.00%</b> (0 files, fence-aware) / <b>23.38%</b> (5,079 in 198 of 847) / —	acceptable <b>only</b> with an <code>fm:</code> prefix; a bogus definition becomes a live link target
<b>frontmatter</b>	required anyway	<b>visible in 23 of 24 renderers</b>	present in <b>84.73%</b> of the pinned corpus	acceptable for a single document-level scalar; <b>unacceptable for anything a human types freely</b> — the value slot is YAML and <b>18.91%</b> of the pinned corpus's own blocks already fail it (\$6.7)
<code>&gt; [!TYPE]</code> <b>callout</b>	<b>CORRUPTS</b>	native on GitHub	—	<b>DISQUALIFIED</b>
<code>[//]: #</code> <b>comment idiom</b>	<b>CORRUPTS</b>	invisible	—	<b>DISQUALIFIED</b>

The two disqualifications, verbatim from the runs:

```
input "> [!NOTE]\n> Useful information.\n"
output "> \[!NOTE]\n> Useful information.\n" ← a backslash inserted into the user's file
```

That is byte-for-byte the `DesktopCommanderMCP` #440 corruption class ( `[x]` → `\[x]` ) this program already cites as its nightmare, and it happens to **GitHub's own alert syntax** through a writer at **44,940,019 downloads/week** ( `mdast-util-to-markdown` ; `remark-stringify` is 33,387,232/wk; `remark-parse` , the *parser*, is 45,820,243/wk — the "45.8M writer" phrasing used elsewhere attaches the number to the wrong package, immaterial to the conclusion but worth fixing).

```
input "[//]: # (fm:vocab sgnk/v1)"
output "[//]: # \"fm:vocab sgnk/v1\"" ← parens become quotes
```

This one matters beyond carrier choice: `[//]: #` was previously reported as one of only two carriers invisible in 24 of 24 renderers. **It does not survive a write**. The 24-renderer matrix measured *visibility* and never measured *occupancy* at all — and occupancy varies by **~2,400x** across corpora (fence-meta 48.008% Docusaurus vs 0.01997% npm docs = 2,404x; the verifier's independent re-derivation gives 46.80%/0.02017% = 2,320x, same order). **No carrier decision should ever be made on fewer than five corpora**.

*One caveat the verifier added and it is worth keeping:* `:::tip` round-trips identically because, with no directive extension loaded, remark parses it as an **inert paragraph** — inertness is the survival mechanism. The test does not show that a directive-aware pipeline preserves it. Note also that `docs/engine/PLAN.md` §15.3 reports the `[ → \[` corruption occurring *only* when an extension is registered on parse but not on stringify; that finding concerns **remark-directive**, whereas the result above concerns **GFM alerts with** `mdast-util-gfm` **registered on both halves**. Both are true. They are different extensions. Do not merge them into one claim.

### 6.7 The measurement that decides it: 0.31%

Everything above is design. This is the decision.

The observed user-invention rate across five corpora totalling 5,245 markdown files is 0.31%, and every instance of it is tutorial content teaching the extension feature.

Where the number comes from

**Corpus D — facebook/docusaurus**, fetched 2026-08-01 via `code.load.github.com/facebook/docusaurus/tar.gz/refs/heads/main` , all `*.md` / `*.mdx` excluding `node_modules` and `.git` = **1,617 files / 10,760,035 bytes**, `selection_sha256=259faf56855af151...` . `::: -directive` occurrences = **4,538 in 980 files (60.61% of files)**, **12 distinct names**:

```
tip 1701 · warning 1151 · info 723 · note 660 · danger 257 · caution 22
important 8 · secondary 1 · success 1
my-custom-admonition 12 · unusedDirective 1 · NotAContainerDirective 1
```

The vendor's default keyword set, read from primary source `packages/docusaurus-mdx-loader/src/remark/admonitions/index.ts` lines 15–19, verbatim:

```
export const DefaultAdmonitionOptions: AdmonitionOptions = {
 keywords: ['secondary', 'info', 'success', 'danger', 'note', 'tip', 'warning', 'important', 'caution'],
 extendDefaults: true,
};
```

**4,524 of 4,538 = 99.69%** are in that 9-keyword set. The remaining **14 = 0.31%** are three strings — `my-custom-admonition` , `unusedDirective` , `NotAContainerDirective` — and `grep` confirms **all 14 sit in** `markdown-features-admonitions.mdx` **and its versioned copies**, i.e. the page that teaches people how to extend the vocabulary. `[own-measurement]`

**Corpus B — 847 third-party npm documentation files** under this repository's `node_modules` , **7,733,433 bytes**, `selection_sha256=21ace73c42f864df...` . GitHub-alert callouts outside fences: **74 occurrences in 18 files (2.13%)**, **5 distinct types**, exactly `[note 45, warning 11, important 10, tip 4, caution 4]` — GitHub's complete built-in set, **zero inventions**. Same corpus: `:::` directives = 1; Hugo shortcodes = 0; MyST `````{}` = 0; `^block-id` = 0; `{#id}` = 6 occurrences in 1 file of 847. `[own-measurement]`

**The pinned corpus** — `corpus_id sha256:3a010b1649899795d79274fc528dbee97fdabf4ff0f81cc02ab619c048c51a4` , 1,084 files / 25,548,765 bytes, all re-read and re-hashed in-session (**1,084 present, 0 missing, 0 hash mismatches**), and independently re-verified by the verifier: **0 callouts, 0 ::: directives, 0 Hugo shortcodes, 0 MyST directives, 0 ^block-id, 0 [//]: #, 6 {#id}** in 3 files. This is a power user's own vault. He invented nothing either. `[own-measurement]`

Every figure in this subsection was independently reproduced by a second agent, in most cases to four significant figures and in several cases exactly — including the byte totals, the file counts, the histogram item-for-item, and the location of all 14 exceptions.

The three corrections that must travel with the number

The measurements reproduced. Some of the *inferences* drawn from them did not.

- 1. **"~850 independent third-party authors invented exactly zero types" is wrong by roughly 47x.** Corpus B's 847 is a **file** count presented as an **author** count. Resolved to owning packages, the corpus spans **663 distinct packages**, and **only 18 of them used the syntax at all**. The inferential base is **18 authors / 74 uses**. Statistically: with n=74 uses the 95% upper bound on the invention rate is **3.97%** (which still clears the pre-registered ~5% falsifier, narrowly); with n=18 authors it is **15.33%**, and **P(0 inventions | 18 authors, true rate 5%) = 0.397** — a null result is unsurprising even if invention were common. **Correct statement:** *"Of 663 packages, 18 used GitHub alert syntax at all; across their 74 uses, zero types outside GitHub's built-in 5 appeared; 95% upper bound on the invention rate is 3.97% per use."*
- 2. **A vendor's own monorepo cannot establish market demand.** Docusaurus's repository is structurally the least likely place on earth to find third-party vocabulary invention, since its own docs would naturally use its own defaults. **Correct statement:** *"In Docusaurus's own repository, 99.69% of directive uses are vendor keywords and every exception is the extension tutorial"* — a fact about the vendor's authoring practice. Third-party demand is **not measured by this corpus**. (It is separately measured by Corpus B, at 18 authors.)
- 3. **"Where a spec forced declaration, it produced drift" is a category slide.** The Open Knowledge Format (OKF) forces a per-document **type assignment**; it provides no vocabulary **declaration** mechanism at all and explicitly declines to police the field. So the measured drift is what you get in the **absence** of a declaration mechanism, which is equally consistent with *"declare a vocabulary to prevent drift"* as with *"detect drift after the fact"*. **The recommendation to ship detection first survives on cost grounds (~2 days, zero new bytes in user files), not on this evidence.** Say it that way.

What happened where a type field WAS required

Four real OKF bundles, fetched 2026-08-01 as `codelead` tarballs of `refs/heads/main` — `OWOX/models` (186 md), `aws-samples/sample-okf-llm-wiki` (29), `colearn00/cole-medin-knowledge-base` (902), `jkroepke/okf-crossplane-v2` (149) = **1,266 markdown files**. [own-measurement, independently reproduced exactly]

MEASUREMENT	VALUE
distinct type values per bundle	5 / 0 / 5 / 21
union across all four	29 distinct values in four incompatible naming conventions
self-collisions in the 21-value bundle	9 of 21 = 42.9% — <code>reference / Reference ;</code> <code>function / Crossplane Function / functioninput / Crossplane Function Input ;</code> <code>guide / developmentguide / Crossplane Development Guide</code>
frontmatter keys in one 149-file bundle	58 distinct, including leaked Claude-Code agent keys <code>systemPromptMode</code> , <code>inheritSkills</code> , <code>defaultContext</code> , <code>tools</code>
bundles declaring okf_version: "0.2"	0 of 4 — absent in 2, "0.1" in 2 — one week after v0.2 shipped

Note the direction of the error: the researcher reported "≥8 of 21" and "~38%"; the true figure is **9 of 21 = 42.9%**. They understated against their own interest. That is a good sign about the measurement, and it is the opposite of the failure mode that dominated the earlier 18-area run.

And our own corpus shows the same field breaking its own parser

[own-measurement, corpus\_id sha256:3a010b16..., reproduced exactly by the verifier on the first attempt]

1,084 files	→ 23 reserved (index.md / log.md) → 1,061 non-reserved
899 of 1,061 (84.73%)	carry frontmatter delimiters
729 of 1,061 (68.71%)	parse to a YAML mapping under js-yaml
170 of 899 (18.91%)	THROW

The failing line, verbatim from `md/Research/Knowledge Base/AI/18 Claude Code Token Hacks in 18 Minutes.md` :

```
entities: [[["Claude Code"]], [[Anthropic]], [[Opus]], [[Sonnet]], [[Haiku]], [[Codex]], [[MCP]], [[Google Workspace CLI]]]
```

`gray-matter` (7,492,225 downloads/week) throws on it with the exact string *"YAMLError: missed comma between flow collection entries at line 24, column 73"*.

The quiet variant is worse than the loud one. `entities: [[["A"]], [{"B"}]]` parses, silently, to `{"entities": [{"A"}, [{"B"}]]}` — the wikilinks are destroyed into a nested array with no error at all.

This is D8's own mechanism — extensibility living in the *value* of a field — failing in production, at scale, today, because markdown's wikilink grammar sits inside YAML's flow-collection grammar. **A declaration mechanism does not produce a vocabulary. It produces an unvalidated free-text field with a grammar collision in it.**

Related, and it changes how any conformance number should be quoted: a regex census of the same corpus ( `^type:\s*\S` ) reports **798 files = 75.21%**, while a true parse requiring OKF §11.1 (parseable YAML block) and §11.2 (non-empty string `type` ) gives **628 = 59.19%**. The 16-point gap is **exactly** the 170 throwing files (798 – 628 = 170). Both methods find **10 distinct type values**, not 12 — the regex distorts `counts ( synthesis 195 → 365)`, not the distinct-value count. And OKF §11 defines conformance as a property of a **bundle** (*"A bundle is conformant with OKF v0.2 if: 1. EVERY non-reserved .md file..."*), so say **"59.19% of non-reserved files satisfy §11.1 + §11.2"** — **not** "the corpus is 59.19% conformant". As a bundle it is 0% conformant.

The two pieces of evidence that cut the other way

A section that only argues its own case is marketing. Here is the counter-evidence, stated at full strength.

**Counter-1 — the mechanism already ships, at 36,745 downloads/week.** `remark-lint-frontmatter-schema` already offers document-declared `'$schema':` with the local-overrides-global precedence rule decided. The researcher went looking for authored instances across **3,197 files in three independent corpora** (pinned 1,084 + npm 847 + OKF 1,266) and found **zero \$schema / @ / x- / fm -prefixed frontmatter keys**. [own-measurement]

**But the verifier weakened this null and it must be reported weakened:** of those 3,197 files only **2,081 contain a frontmatter block at all**, and **Corpus B contributes just 3 frontmatter blocks despite supplying 847 files to the total**. The regex third of that search was near-vacuous as evidence about frontmatter keys. The null

reproduces (0 hits), but it is a **search-limited null over corpora we chose** — not proof of absence. The researcher named this as the weakest link in their own case before anyone else did.

**Counter-2 — MyST demonstrates a 46-term vocabulary is usable.** `jupyter-book/mystmd`, 431 files, `selection_sha256=9d9f8dbc...` : **46 distinct roles in 25.75% of files** ( `ref` 622, `term` 41, `numref`, `abbr`, `eq`, `kbd` ). This contradicts a naive reading of the single-digit cap. Every one of the 46 is Sphinx/MyST **built-in**. The demand curve it reveals is for a **large built-in vocabulary that renders** — not for a mechanism to declare your own.

What 0.31% does NOT prove

Say this out loud, because it is the honest boundary of the whole argument:

**All of this evidence measures what people DID with the tools that already exist. None of it measures what they WOULD do with a better one.** Every corpus here is a record of behaviour under mechanisms that are awkward, undiscoverable, or vendor-locked. A person who never invented an admonition type in Docusaurus may simply never have read the page that says you can.

That is precisely why kill condition **K-V3** in §6.8 is weighted above every number in this section.

6.8 The verdict, and the four things that would reverse it

**DECIDED: ship no declared-vocabulary feature, no vocabulary registry, no library-import syntax, and no new carrier in the first twelve months.**

This is the settled answer, not a deferral. It was reached independently by five research areas attacking from five directions and it is recorded in all four synthesis lenses of the final gate. The verdict lens states it as an explicit never-build item:

*"NEVER BUILD: new syntax, new carriers, a declaration mechanism, a vocabulary registry, a folder compiler's incremental engine, per-span confidence, or a competing provenance vocabulary."*

Two independent grounds, either sufficient on its own:

- 1. **Demand.** 0.31%, all tutorial. Nobody downloads names.
- 2. **Trust boundary.** In frontmatter the content author is untrusted by construction (D3 puts a logged-in reviewer inside the document). Hugo drew this exact line and disabled the feature.

**The four kill conditions.** Any one of these observed means the no-ship decision is wrong and this section must be re-opened:

ID	OBSERVATION THAT REVERSES THE VERDICT
K-V1	A census over the <b>287 GitHub repositories</b> matching <i>"open knowledge format"</i> (counted via <code>api.github.com</code> search in-session) shows <b>median per-bundle type vocabulary above 12 terms with intra-bundle collision below 10%</b> . That would mean teams do build real vocabularies at scale and the n=4 sample was unrepresentative.
K-V2	The experiment in §6.9's measurement box returns <b>PRESERVE ≥ 95% at 12 terms on the cheapest model tier</b> . The cap is then not a real constraint and the feature costs far less than argued here.
K-V3	<b>One real user, in the first user conversation, asks unprompted to name their own construct.</b> Zero users have been interviewed across this entire program. A single genuine request outweighs every corpus number in this section, because all of that evidence measures behaviour under existing tools. <b>This is the highest-signal kill condition and it is also the cheapest to check.</b>
K-V4	<code>mdmax vocab</code> ships and real users dismiss its drift output as noise. That kills the replacement, at which point the honest position is that this whole area produces nothing and it should be <b>closed</b> rather than deferred.

**And a fifth, which kills the design rather than the decision:** if fence-meta occupancy exceeds ~10% in a fifth and sixth corpus, the meta channel is not a carrier at all and only the frontmatter scalar survives §6.6's ranking.

6.9 The open question, upstream of all syntax: language, or thin shell?

This is unresolved. It is listed as open decision #3 in `HANDOFF-mdmax-markdown-engine-2026-08-01.md:369` and no one has ruled on it. **It does not need answering in the next twelve months — the no-ship decision makes it moot for that period — but it must be answered before anyone writes a single line of import syntax, because it determines the whole design and cannot be retrofitted.**

The question exists because of the §6.2.1 fact: **no format that is not also a Turing-complete programming language has ever acquired a package ecosystem in which the document declares the dependency and a registry resolves it.** A package distributes behaviour. If the host cannot execute, the package has nothing to carry.

Branch A — MDMAX becomes a programming language

**What it means:** MDMAX acquires an evaluator. Something a document names gets *executed* — a filter, a macro, a layout function, a WASM module. This is the LaTeX and Typst path, and it is the only path on which a package ecosystem has ever worked.

What it commits us to, in full:

- **A sandbox, permanently.** Typst's non-WASI, byte-buffers-only, no-ambient-authority contract is the minimum bar, and even Typst admits purity is a contract it does not enforce.
- **Obsidian's threat model**, which its own vendor describes as unsolvable: *"Obsidian cannot reliably restrict plugins to specific permissions or access levels."*
- **A threat model strictly worse than npm's**, because a `package.json` is written by the repo owner whereas a `.md` is routinely a drive-by pull request, a CMS field, a logged-in reviewer's paste (D3), or model output `[inference]`.
- **npm's operational burden**, on a two-person team: token revocation programmes, malware scanning, incident response. npm needed multiple quarters of security work under Microsoft's ownership.
- **Losing the compiler property.** MDMAX today is a total function from bytes to bytes with a proof obligation on the untouched region (D7, splice-only). An evaluator breaks that.
- **MDX's fate on write-back:** once a document contains expressions, it cannot be losslessly re-serialized, and a product that edits and saves cannot live with that.

- **It contradicts D6** (the file stays ordinary `.md`, degradation is the feature) and **D7** (splice-only). Branch A is not a feature addition. It is a different product.

**What it buys:** the only architecture on which the founder's Python analogy has ever actually worked.

**Branch B — MDMAX admits it is a thin shell over a language, and never evaluates**

**What it means:** MDMAX stays a **compiler** in the strict sense — bytes to bytes, splice-only, never evaluating anything a document names. Extension, if any, happens in the **host application** (the editor, the CLI), configured by someone with commit access, exactly as Quarto, Sphinx, MkDocs, Pandoc and Obsidian all do.

**What it commits us to:**

- **No package ecosystem, ever.** Accept it explicitly rather than deferring it.
- **A larger built-in vocabulary is the only growth path** — which is what the MyST datapoint says demand actually looks like (46 vendor-supplied roles, densely used).
- **Every capability that needs execution must be a host feature**, shipped by us, reviewed by us, versioned with the product.
- **D6, D7, D8 and D9 all hold unchanged.**

**What it buys:** the compiler property, the trust boundary, the degradation guarantee, a two-person-sustainable operating burden, and a product that can be finished.

**The third door, which is what the research actually recommends**

The research's own position is that **the dichotomy is a trap that costs the product**: it is only forced on someone who wants a package ecosystem, and the correct move is to **refuse the ecosystem and therefore never face the choice**. That is Branch B with the deferral removed.

**Record it as a recommendation, not a decision**, because Branch A vs Branch B has real strategic content — it is the difference between "a markdown editor with a strong engine" and "the LaTeX of the 2020s" — and that is a founders' call, not a researcher's.

**The measurement that would settle the operating point**

Every result quoted in §6.6 is frontier-class, five constructs, one task at a time, with the declaration in immediate context. **The editor's inline-AI path will run the cheap model, and the cheap model is the one that decides.** The experiment is fully specified so it can be run on demand:

ITEMS	real paragraphs from corpus_id sha256:3a010b16..., stratified by token count (short blocks are a known confound). Vocabulary terms drawn from real ones: the 29 OKF `type` values + the 10 valid-YAML `type` values in the pinned corpus. NO synthetic vocabulary – a synthetic one measures the experimenter.				
FACTORS	F1 model tier	×4	frontier / mid / small hosted / ~1-3B local		
	F2 vocab size	×4	3, 6, 12, 24 terms (12 = proposed cap, 24 = measured drift zone)		
	F3 declaration	×3	rule-only / example-only (3 worked examples) / rule+examples		
	F4 distance	×3	adjacent / document head ~8k tokens away / sidecar ~40k earlier		
	F5 task	×3	RECOGNISE / EMIT / PRESERVE		
DESIGN	main sweep	4 tiers × 4 sizes × 3 tasks × n=40 = 1,920 calls			
	ablation	F3 × F4 × F2∈{6,24} = 12 cells × n=40 = 480 calls			
	TOTAL 2,400 calls ≈ 6.0M tokens – 76% of ONE of the two research workflows already run (7.86M). The measurement that decides this costs less than the research that failed to decide it.				
VERIFY	deterministic, ZERO LLM judges.				
	RECOGNISE = exact set match against a hand-built key				
	EMIT = parses to the declared shape AND round-trips byte-identically				
	PRESERVE = byte diff over the untouched region, computed by an INDEPENDENT oracle sharing no code with the writer				

Pre-registered predictions, to be published either way: **P1** PRESERVE degrades with tier faster than RECOGNISE (smallest tier: PRESERVE <80% while RECOGNISE >95%). **P2** every tier ≥95% on RECOGNISE at 3 and 6 terms; small tier below 90% at 12 and below 75% at 24. **P3** examples beat rules at every tier and the gap widens as tier falls. **P4** distance costs less than size (<5pp for adjacent→40k versus >15pp for 6→24 terms at the small tier). **P5** if the small tier is ≥95% on all three tasks at 12 terms, the cap is wrong and the feature is materially cheaper than argued — this is K-V2.

**Two cheaper measurements should run first. Both are zero model calls and about half a day each:**

- **M-A — vocabulary drift census at scale.** Run the `mdmax vocab` logic over the **287 GitHub repositories** matching "*open knowledge format*" and report the distribution of per-bundle `type` vocabulary size and intra-bundle collision rate. Our n=4 gives 5/0/5/21. If the true distribution is fat-tailed above 12, the cap is wrong **and drift detection is worth more, not less**. This single measurement can refute or confirm both of the section's negatives.
- **M-B — carrier occupancy across ≥8 corpora**, each with a stated selection hash. Fence-meta occupancy measured 0.00% / 0.02% / 48.01% on three corpora; link-refdef 0.00% / 23.38% on two. **No carrier decision should be made on fewer than five.**

**6.10 What we ship instead — the inverse, and it gives the founder what he asked for**

The founder's actual want is that *people use only the features they want* without learning a vocabulary. **The inverse of a declaration mechanism delivers that with zero new syntax:** instead of asking users to declare a vocabulary, *infer the one they are already using* and tell them where it is breaking.

`mdmax vocab` — the recogniser (Section 4)

Read-only. Zero new bytes in any user file. Compatible with D2, D4, D6 and D7 **by construction**.

```
mdmax vocab <dir> [--field type,status,category] [--fail-on drift]
```

It emits the vocabulary a corpus is **already** using, plus three defect classes — each of which has a live positive in the corpora measured above, so the tool cannot ship and find nothing:

#	DEFECT CLASS	LIVE POSITIVE FOUND IN THIS RESEARCH
1	case / morphology drift	<code>reference</code> vs <code>Reference</code> ; <code>guide</code> / <code>developmentguide</code> / <code>Crossplane Development Guide</code> — 9 of 21 values in one real bundle
2	prose in an enum slot	a <code>status</code> field whose value is a 130-character sentence
3	unparseable value slot	<b>170 of 899</b> pinned-corpus frontmatter blocks (18.91%) that js-yaml and gray-matter both reject — plus the silent variant where <code>[[[A]], [[B]]]</code> parses to a nested array and destroys the links with <b>no error</b>

Output is a table of `(field, value, count, files, suspected-collision-with)`. **It cannot flatter anybody** — the same property that selected `mdmax cert`. Effort: ~2 days.

**Nothing like it exists.** Four npm registry searches (*markdown vocabulary*, *markdown schema frontmatter*, *markdown directive*, *remark custom syntax*) return one relevant hit, `remark-lint-frontmatter-schema`, which validates against a schema **you** write and is silent when the schema is absent — and which **structurally cannot** report defect class 3, because the YAML must parse before it can validate. A GitHub search for "*open knowledge format*" returns 287 repositories, whose only conformance tool (`Sudhakaran88/okf-conformance`, 16 stars) checks structural conformance, not value drift. **Nothing infers the vocabulary from the corpus.** `[own-measurement, search run in-session]`

**And it is exactly the differentiated position against OKF.** OKF v0.2 creates the open `type` field and *explicitly declines to police it* — §4.1, verbatim: "*Type values are not registered centrally. Producers SHOULD pick values that are descriptive and self-explanatory; consumers MUST tolerate unknown types gracefully.*" That is the hole the drift detector fills, on a specification Google is pushing and 287 repositories already use.

The two-tier link resolver (Section 3)

This is M2 from §6.1 — resolution — delivered without M1, M3 or M4. `[text](path.md)` resolves at **85.71%**; `[[Name]]` resolves at **27.65%** — a 58-point gap on identical files, and no tool implements the semantics that gap demands. A path link that does not resolve is a **mistake**; a wikilink that does not resolve is a **legitimate authoring primitive**. Near-miss discrimination found **95 real, hand-verified broken links** that naive checking would have buried.

`mdmax cert` — the conformance certificate

Per document, per target, per construct: **PASS · STRIP · CORRUPT**, with targets modelled as `(product, surface)` pairs. This is what "will my file survive over there?" looks like as a product, and it is the *only* capability in this program that survived adversarial search with **no incumbent found**. Two days, hard capped, never the spearhead.

Why this is the honest answer to the founder's question

The Python analogy asks for **capability without vocabulary**. A declared vocabulary delivers the opposite: it demands that every author, every reviewer, every downstream tool and every model learn a set of terms that exists in one vault. The three tools above deliver capability with **no vocabulary at all** — they read what is already there. And they ship in about a week, together, against a declaration mechanism whose *cheapest honest measurement* costs 6.0 million tokens before the first line of product code.

6.11 What this section does not cover, and what would falsify it

**Not covered here.** Comment and suggestion anchoring (Section 5). The container/`pack` question. Provenance field semantics beyond the `type` field. Real-time collaboration. Anything about pricing or tiers. This section is *only* about whether a document may declare or import a vocabulary.

Unverified items carried into this section, each flagged in place above:

- Zero live model calls were made in any research run.** Every claim about what a model does with a declared notation — the cap, the primacy geometry, the declare-by-example rule — is a **prediction**, not a result. `[SIMULATED / prediction]`
- IFScale (arXiv 2507.11538) and Aycok et al. (arXiv 2409.19151) were not read by anyone on this team.** They are inherited from `docs/engine/PLAN.md` §15.8. The cap and the by-example rule rest on **our own corpus measurements** (5/0/5/21, Docusaurus 9, GitHub 5); the papers are corroboration we have not personally verified.
- The vim modeline CVE count (five across 24 years) is inherited and unchecked.** Do not publish it. The `modelinestrict` hardening commits, the allowlist, and the "Trojan horse text file" language are all verified and are stronger evidence anyway.
- The Docusaurus directive census counts `:::`-prefixed lines by regex, not by parsing with remark-directive.** The 99.69%/0.31% split is robust (the numerator is a small integer set inspected by name) but the absolute 4,538 is a regex figure.
- The Docusaurus 9-keyword default set was read from `main` at fetch time.** If corpus files were authored against an older, smaller default set, the vendor-shipped share is if anything understated.
- Sonatype's malware figures are search-summary tier** and nobody read the report.
- `remark-directive`'s weekly downloads differ between two of our own measurements** — 3,176,157 (`docs/engine/PLAN.md` §15.3, measured ~2026-07-30) and **3,153,286** (final gate, measured 2026-08-01). Different weeks; **quote the later figure with its date**. Both are dwarfed by the authoring signal: `:::` occurs **0 times** in the pinned corpus and **1 time** in 847 npm docs. The mechanism is in three million installs and two documents. Download-weighted adoption and typed-into-a-file adoption disagree by orders of magnitude and **only the second one is real**.
- The pinned corpus's own `corpus_id` is not re-derivable from its manifest.** Thirty serializations of the documented recipe were attempted and none reproduces `sha256:3a010b1649899795d79274fc528dbece97fdabf4ff0f81cc02ab619c048c51a4`. The manifest **is** now tracked — committed in `1bd4dad` — but `git ls-files` shows **no derivation script anywhere in the repository** `[measured, this session]`. The **file list itself is sound**: all 1,084 files re-read and re-hashed, 0 missing, 0 mismatches. **Fix: commit the ~15-line derivation script and its captured output next to the manifest.** Until then, the corpus id is a label, not a checksum.

9. The verification layer has a measured false-kill rate of  $5/29 = 17.24\%$ . Its own audit found the pattern: kills that **quote a primary source** were **16 of 16 correct**; kills that **substitute a re-derived number** were wrong **5 of 13 (38%)**. Applied to this section: the Quarto refutation, the JSON-LD `@import` correction and the download-label correction all quote sources and should be **accepted**; the 21-vs-24 allowlist, the 663-packages/18-authors recalculation, the 10-vs-12 distinct values and the 0.00%-vs-0.83% link-refdef occupancy all substitute numbers and should be **independently re-derived** by whoever next depends on them.

What would falsify this section, restated as a checklist:

- ☐ K-V1 — median per-bundle vocabulary > 12 terms with < 10% collision across 287 OKF repositories
- ☐ K-V2 — PRESERVE  $\geq 95\%$  at 12 terms on the cheapest model tier
- ☐ **K-V3 — one real user asks, unprompted, to name their own construct**
- ☐ K-V4 — `mdmax vocab` ships and its drift output is dismissed as noise
- ☐ design-only — fence-meta occupancy > ~10% in a fifth and sixth corpus

One security observation from the research session, recorded because it bears directly on the trust boundary and not because it is a finding. While fetching the `openknowledge-sh/openknowledge` README to locate the canonical OKF specification URL, the retrieved content contained `curl -fsSL https://openknowledge.sh/install | bash` **together with an agent-directed prompt block instructing an agent to install and run that script**. The researcher did not execute it and did not act on it. It is flagged here because the OKF ecosystem's own onboarding path is *"paste this prompt into your agent"* — which is exactly the untrusted-content-directs-an-agent shape this program has already ruled against, and exactly the shape a document-declared import mechanism would make routine.

---

## 6.12 The one-paragraph version

The founder's library idea is structurally sound and has a named, cheap reference design (Typst: one person, one week, a git repo plus a CDN, with checksums added and LaTeX's missing-versus-wrong-version asymmetry fixed to symmetric warn-and-degrade). We are not building it, for two independent reasons either of which would be sufficient. First, demand: across 5,245 markdown files in five corpora the user-invention rate is **0.31%** and every instance is the tutorial page teaching the feature; where a specification forced a declaration it produced **drift** (29 values, four naming conventions, 42.9% self-collision in the largest bundle) rather than a vocabulary; and the mechanism already ships elsewhere at 36,745 downloads a week with zero authored instances found in 3,197 files. Second, trust: in frontmatter the content author is untrusted by construction, which is the exact line Hugo drew when it built this feature and shipped it **disabled, under SECURITY**, with the sentence *"template and configuration authors are trusted, but content authors are not."* We ship the inverse instead — a recogniser that infers the vocabulary a corpus already uses and reports its drift, its prose-in-enum slots, and the 18.91% of frontmatter blocks that no YAML parser can read — about two days of work, zero new bytes in anyone's file, and no incumbent. The question that sits upstream of all of it, and which remains open, is whether MDMAX ever becomes a programming language or admits it is a thin shell over one; it does not need answering this year, but it must be answered before anyone writes a line of import syntax. And the cheapest thing that could overturn everything above is one user asking, unprompted, to name their own construct — because every number in this section measures what people did with the tools that already exist, not what they would do with a better one.

---

## 7. frontmatter the product: features, the gap, and what it needs on top of MDMAX

### 7.0 How to read this section

This section answers one question: **if MDMAX (the markdown capability layer specified elsewhere in this plan) were finished tomorrow, would frontmatter be a product?** The answer is no, and the reasons have nothing to do with markdown.

Everything below is verified against the source tree at `/Users/sagnikmitra/Desktop/GitHub/frontmatter/src/`, read directly during the authoring of this section on 2026-08-01, on branch `engine/plan-and-diagnostics`. Where a claim comes from research rather than from code, the research area key is named. Where two documents in this repository disagree, both are named and one is declared to govern.

Tags used throughout, per the plan's evidence discipline:

TAG	MEANING
[measured]	a script or command was run and produced this number
[primary]	the source was read directly (a source file, an API discovery document, a vendor pricing page)
[secondary]	reported by another document that read the source
[inference]	reasoned from evidence, not observed
[SIMULATED]	replayed through code, not read from a live system
[UNVERIFIED]	stated in the record with no source I could reach

**Three words are used precisely and are not interchangeable.** *DECIDED* means the founders have settled it and it is not reopened here. *RECOMMENDED* means this section proposes it and it needs a founder decision. *OPEN* means nobody has decided and the evidence does not settle it.

**The pinned corpus.** Every corpus figure in this section refers to `corpus_id sha256:3a010b1649899795d79274fc528dbece97fdabf4ff0f81cc02ab619c048c51a4` — 1,084 markdown files, 25,548,765 bytes, pinned in `docs/engine/research/corpus-manifest.json`. Its three roots, read from the manifest [measured] :

root	head (12)	files	bytes
md	02c22ec47a3a	756	21,068,853
knowledge	464eb666946c	272	3,570,923
frontmatter	798ebbf3250a	56	908,989
		1,084	25,548,765

Of those, 907 files carry YAML frontmatter. **A caveat the manifest does not yet carry:** the final-gate verifier found that 29 files (2.68% of 1,084) are byte-identical across roots by sha256, because `frontmatter/docs/*` is mirrored into `md/docs/*` ( `FEATURE-GAP-REPORT.md`, `FRONTMATTER-PRODUCT-PLAN.md`, `adr/0001-...` and 26 more). Every percentage in this programme inherits that double-count. It does not change any conclusion here, but the manifest should carry a dedup note before the build phase treats these ratios as ground truth.

**What this section does NOT cover.** Engine internals (splice algorithm, anchor tiers, normalization) — those are the MDMAX sections. Go-to-market sequencing and channel strategy — that is the market section. Legal, entity, and tax structure — not addressed anywhere in this plan. Mobile *native* apps — out of scope for twelve months. And the AIOS track, which per D5 is separate.

### 7.1 What exists today, verified against `src/`

The repository is a Next.js 16 application, 21,415 lines of TypeScript and TSX across `src/` [measured], with 83 test files under `test/` [measured]. It is a genuinely capable single-player markdown editor. It is also, structurally, a single-user application — and not by configuration. By construction.

#### 7.1.1 The four hard facts about identity

**Fact 1 — the allowlist is not a list.** [measured] `src/modules/auth/domain/allowlist.ts` is ten lines. Line 9 reads, verbatim:

```
return login.trim().toLowerCase() === allowed.trim().toLowerCase();
```

The signature at line 5 is `export function isAllowed(login: string | undefined, allowed: string): boolean`. The second parameter is a **scalar string**, not an array. This is a single case-insensitive string equality. There is no data structure here that a second identity could be added to without changing the function's type.

Its only caller is `src/modules/auth/infrastructure/auth-options.ts:82` :

```
return isAllowed(ghProfile?.login, authEnv.ALLOWED_GH_LOGIN);
```

And that environment variable has a hardcoded default naming a specific person — `src/config/env.ts:47` [measured] :

```
ALLOWED_GH_LOGIN: z.string().min(1).default("sagnikmitra"),
```

`src/config/env.ts:252` synthesises a development-bypass actor from the same single string ( `process.env["ALLOWED_GH_LOGIN"] ?? "dev"` ). The domain type that carries the signed-in user, `src/modules/auth/domain/actor.ts`, holds `{login, name, avatarUrl}` — **no uid, no email, no organisation, no role** [measured]. There is

nowhere in the type system for a second person to exist.

The second authentication provider does not change this. `auth-options.ts:32-66` defines a `Credentials` provider with id `sgnk-password` whose `authorize()` compares the submitted username against `process.env["SGNK_AUTH_USER"]` and the password against a script hash in `SGNK_AUTH_HASH`. It is a password fallback for **the same one person**, and its comment says so at lines 76-77: "*only the configured SGNK\_AUTH\_USER can reach here.*"

**Fact 2 — the author of every commit is a compile-time constant.** `[measured]` `src/container/dependency-container.ts:42` :

```
const AUTHOR = { name: "Sagnik Mitra", email: "sagnikmitra123@gmail.com" } as const;
```

It is passed as `author: AUTHOR` into four use-case factories, at lines **66, 70, 75 and 80**:

LINE	USE-CASE	WHAT IT WRITES
66	<code>commitChanges</code>	every save of every document
70	<code>createNote</code>	every new document
75	<code>renameNote</code>	every rename, including the auto-relink pass
80	<code>uploadAttachment</code>	every pasted or dropped image

Every byte this application has ever written to a git repository is attributed to one hardcoded name and one hardcoded email address. There is no code path that reads the signed-in actor and uses it for attribution. Multi-user attribution is not a missing feature; it is a missing *parameter*, and the parameter has been replaced by a literal.

**Fact 3 — there is no way to become a user.** `[measured]` `src/app/` contains 30 route files (4 pages, 26 API routes). The complete page list is:

```
src/app/(auth)/login/page.tsx
src/app/(public)/[slug]/page.tsx
src/app/(public)/p/[slug]/page.tsx
src/app/(vault)/page.tsx
```

`src/app/(workspace)/` contains one file: a zero-byte `.gitkeep`. `grep -ril 'signup\|sign-up\|register\|onboard' src/app` returns exactly one file, `layout.tsx`, and the match there is not a route. Across all of `src/`, the seven files that match those strings match them for unrelated reasons `[measured]`: `instrumentation.ts` exports Next.js's `register()` lifecycle hook; `PWARegister.tsx` calls `navigator.serviceWorker.register("/sw.js")`; `session.ts` has a comment about "the OAuth App's single registered callback URL"; and `src/modules/share/domain/slug.ts` lists `"signup"`, `"register"`, `"registration"`, `"onboarding"` in `RESERVED_SLUGS` — a blocklist of URLs that *must not* be taken by a published note, which is the strongest possible evidence that the routes do not exist, since the code is reserving the paths for a future that has not arrived.

**There is no signup route, no registration form, no invite flow, and no onboarding.** The only page a stranger can reach is `/login`, which will reject them.

**Fact 4 — the vault is one repository, reached by one server-side token.** `[measured]` `src/config/env.ts:85-87` :

```
GITHUB_REPO_TOKEN: z.string().min(1),
GITHUB_REPO: z.string().min(1).default("sagnikmitra/md"),
GITHUB_BRANCH: z.string().min(1).default("main"),
```

One token, held by the server, with write access to one named repository. Not the user's token; not the user's repository. Even if a second person could log in, they would be reading and writing the founder's vault with the founder's credentials.

### 7.1.2 `firestore.rules` : a specification nobody implemented

`firestore.rules` is 419 lines and 17,304 bytes `[measured]`. It is a careful, well-reasoned, genuinely multi-tenant data model. It declares, with real enforcement clauses and not merely in comments:

COLLECTION	KEY FIELDS	ACCESS MODEL
<code>users/{uid}</code>	<code>email</code> , <code>providers</code> , <code>plan</code> ( <code>free pro team</code> ), <code>defaultVaultId</code> , <code>createdAt</code> , <code>lastSeenAt</code>	owner read/write; <code>plan</code> is server-owned after create; client may only ever write <code>'free'</code> (line 214)
<code>billing/{uid}</code>	<code>status</code> , <code>priceId</code> , <code>currentPeriodEnd</code> , <code>provider</code> , <code>customerId</code>	owner read, allow write: if false — Admin SDK only (line 230)
<code>usage/{uid}/months/{YYYY-MM}</code>	<code>aiTokens</code> , <code>noteCount</code> , <code>bytesUsed</code>	owner read, no client writes (line 243)
<code>vaults/{vaultId}</code>	<code>ownerUid</code> , <code>name</code> , <code>rev</code> , <code>memberUids</code> : <code>list&lt;string&gt;</code> ( <b>≤100</b> ), <code>roles</code> : <code>map&lt;uid,"owner" "editor" "viewer"&gt;</code>	members read; only the owner may change <code>memberUids</code> / <code>roles</code> / <code>ownerUid</code> (lines 284-289)
<code>vaults/{vaultId}/notes/{noteId}</code>	<code>path</code> , <code>pathLower</code> , <code>title</code> , <code>tags</code> , <code>outbound</code> , <code>backlinks</code> , <code>embeds</code> , <code>frontmatter</code> , <code>publicSlug</code> , <code>content</code> (≤900,000), <code>storagePath</code> , <code>deletedAt</code>	<code>canEditVault</code> gated (lines 329-338)
<code>vaults/_/notes/{noteId}/revisions/{revId}</code>	<code>noteId</code> , <code>content</code> , <code>authorUid</code> , <code>message</code> , <code>createdAt</code> , <code>parentRev</code>	members read, editors create, allow update: if false; allow delete: if false; (lines 360-361) — append-only, parent-linked
<code>shares/{slug}</code>	<code>vaultId</code> , <code>noteId</code> , <code>ownerUid</code> , <code>title</code> , <code>contentSnapshot</code> , <code>visibility</code> ( <code>public unlisted</code> )	unauthenticated get for <code>visibility==='public'</code> ; owner-only list (lines 393-397)

Membership is resolved by real helper functions that read the parent vault document: `isVaultMember` (line 166), `vaultRole` (172), `canEditVault` (176), `isVaultOwner` (180).

**Two things about this file are decisive.**

First: `grep -io 'comments' firestore.rules | wc -l` returns **0** `[measured]`. The word does not appear anywhere in the file — not as a collection, not in a comment, not in the extensive data-model header. Neither does "suggestion". The one capability the entire product thesis rests on has no place to be stored.

Second: `grep -rn 'firestore()' src/ test/` returns **exactly one hit — the definition itself** [measured] , at `src/shared/infrastructure/firebase/client.ts:38` :

```
export function firestore(): Firestore {
 return getFirestore(firebaseApp());
}
```

Zero callers. The only module in `src/` that imports from `firebase/client.ts` at all is `firebase-auth-gateway.ts` , and it imports `firebaseAuth` , not `firestore` .

**The conclusion, stated plainly.** `firestore.rules` is a promise about data that does not exist. It is a security specification for an application that was never written. Nothing in `src/` reads or writes a single one of those eight collections. The multi-tenant model is design work that has been done — which genuinely lowers the estimate for building it — and implementation work that has not started at all.

**A contradiction in the record, named.** The final-gate `against` area argued the multi-tenancy floor "is not zero, because `firestore.rules` already implements vaults/roles/members/immutable revisions." The `product-gap` area (verdict **CONFIRMED**, `headline_verdicts` ) argued there is exactly one user and a specified-but-unimplemented model. `product-gap` **governs**. A rules file is a declarative access-control policy evaluated by Google's servers against documents that are written by client code. There is no client code. The `against` area is right that the schema design is done; it is wrong that the floor is non-zero.

7.1.3 Where the previously published claims were wrong

This plan replaces `docs/mdmax/PLAN.md` v2.0.0 and inherits research whose headline defect rate was severe: the 18-area capability run had **100% of headlines refuted by their own verifier, always in the flattering direction**; the 16-area final gate scored **5 CONFIRMED, 10 OVERSTATED, 1 REFUTED**. The following corrections apply to this section's subject matter and must not be re-propagated:

CLAIM AS PREVIOUSLY STATED	CORRECTED	SOURCE
"18.9% of a real Obsidian vault's frontmatter is invalid YAML"	18.74% is the <b>whole-corpus</b> rate (170/907). The Obsidian vault <code>(md root)</code> is <b>13.75%</b> (91/662); the <code>knowledge</code> repo is <b>35.59%</b> (79/222); the <code>frontmatter</code> repo is <b>0</b> . Say which.	final-gate <code>product-gap</code> verification, <code>killed[2]</code>
"eemeli/yaml reaches 31.86% safe vs gray-matter's 3.64%"	Apples-to-apples: <b>22.38% vs 31.86%</b> under "file left untouched", or <b>3.64% vs 13.12%</b> under "published and byte-identical". A ~9-point gap was presented as ~28, by scoring eemeli's 170 refusals as successes and gray-matter's throws on <b>identically the same 170 files</b> as failures. The conclusion (splice is required, a library swap is not the fix) survives and is better supported by the corrected numbers.	final-gate <code>product-gap</code> verification, <code>killed[0]</code> , <code>killed[1]</code>
"Publish round-trips only 3.64% of the corpus"	3.64% (33/907) is a <b>no-op write</b> . The actual publish-then-unpublish round trip is <b>17/736 parsed = 2.31%</b> (17/907 = <b>1.87%</b> ). The original claim was conservative, not inflated — but any gate written against the round-trip operation must beat <b>1.87%</b> , not 3.64%.	final-gate <code>product-gap</code> verification, <code>new_defects[0]</code>
"171 files fail" (one failure mode)	<b>Two</b> failure modes. Exactly <b>170</b> files fail at <code>matter()</code> parse; exactly <b>one more</b> , <code>md/pj.md</code> , parses cleanly and throws at <code>matter.stringify</code> . A splice fixes the 170 by never parsing; it does not automatically fix the <code>stringify</code> class.	final-gate <code>product-gap</code> verification, <code>new_defects[2]</code>
"Docs v1 has 92 suggestion fields"	Reproduces under <b>none of eleven</b> counting rules the verifier tried. The nearest true figure is <b>197 suggestion-named property occurrences across 56 schemas</b> . The "35 suggestion-bearing schemas" figure is correct and is exactly the count of schemas whose <b>name</b> contains "Suggest", out of 170.	final-gate <code>product-gap</code> verification, <code>killed[3]</code>
"Drive v3 Comment has 13 fields, Reply has 10"	<b>14</b> Comment properties and <b>11</b> Reply properties. 13/10 is what you get after excluding <code>kind</code> , which is defensible but was not stated.	final-gate <code>product-gap</code> verification, <code>new_defects[5]</code>
"the <code>.doc</code> export is a false claim in a user-facing menu"	<b>Not a false claim.</b> <code>ExportMenu.tsx:158</code> labels it "Export Word (.doc)", line 93 comments <i>"Word opens HTML-based .doc files natively; reuse the HTML renderer"</i> , line 95 sends <code>application/msword</code> , and that is a real, working Word import path that has shipped for two decades. A naming preference, not a defect. Drop it from any pre-flight list.	final-gate <code>product-gap</code> verification, <code>killed[6]</code>
" <code>isVaultEditor</code> helper in <code>firestore.rules</code> "	No such function. The helper at line 176 is <code>canEditVault</code> .	final-gate <code>product-gap</code> verification, <code>killed[4]</code>
" <code>package.json</code> has been gutted to 17 bytes; <code>npm run verify</code> cannot run"	<b>RESOLVED.</b> [measured] 2026-08-01: <code>package.json</code> is 3,076 bytes, <code>git status --short package.json</code> returns empty, and all 16 scripts plus 43 dependencies and 22 devDependencies are present. Fixed in commit <code>1bd4dad</code> per <code>docs/mdmax/PLAN.md</code> §9.10. Do not repeat this hazard as live.	own measurement, this session

**Two figures nobody should quote without the caveat attached.** The **99.627% anchoring** figure has been re-derived by nobody, including every research area that cited it — it is a *block* figure and says nothing about the *range* anchoring that comments actually require. And **zero live model calls were made in any research run in this programme**; every "easier for AI" claim anywhere in the record is a prediction, not a result.

7.2 The full feature list, organised by the mockups

The mockups define four surfaces: a dashboard, an editor, a right rail, and an AI writing surface. Every row below is verified against code. Legend: **SHIPPED** = present and reachable; **PARTIAL** = present but materially incomplete against the mockup; **MISSING** = no code exists.

7.2.1 The dashboard

The mockup shows a landing surface with search, a "blank document + templates" row, and a grid of recent documents.

FEATURE	STATUS	VERIFICATION
A dashboard route at all	<b>MISSING</b>	[measured] The only application page is <code>src/app/(vault)/page.tsx</code> , whose entire body is <code>return &lt;AppShell /&gt;;</code> . <code>src/app/(workspace)/</code> contains only a zero-byte <code>.gitkeep</code> . There is no route that renders a document grid.
Search	<b>SHIPPED</b> , but as a palette not a dashboard	[measured] <code>MiniSearch</code> full-text search via <code>src/modules/vault/infrastructure/search-index.ts</code> + <code>/api/vault/search</code> , surfaced through <code>SearchPanel.tsx</code> , <code>CommandPalette.tsx</code> and <code>Spotlight.tsx</code> . Reached by keyboard, not by a landing page.
Blank document	<b>SHIPPED</b> , as a tree action	[measured] <code>FileTreeActions.tsx</code> listens for <code>sgnk:new-note</code> and opens an inline dialog. There is no "start a blank doc" card.
Templates	<b>PARTIAL</b>	[measured] <code>src/modules/vault/presentation/template-vars.ts</code> and <code>daily-notes.ts</code> exist; templates are wired to the daily-note path. <code>docs/FEATURE-GAP-REPORT.md</code> §4 lists templates as <code>△</code> "daily only". There is no template gallery.
Recents grid	<b>MISSING</b>	[measured] <code>grep -in 'template recent dashboard blank' src/modules/app-shell/presentation/AppShell.tsx</code> returns nothing. No recents model, no grid component, no thumbnail rendering.

A user arriving at frontmatter today lands directly in a three-pane editor with a file tree. There is no "home". This is the single largest divergence between the mockups and the code, and it is the first screen — which makes it the first impression.

7.2.2 The editor

FEATURE	STATUS	VERIFICATION
Four view modes — Live / Edit / Split / Read	<b>SHIPPED</b>	[measured] <code>src/modules/editor/presentation/editor-store.ts:21</code> — <code>export type EditorMode = "edit"   "live"   "reading"   "split";</code> . Default is <code>"reading"</code> (line 90), persisted (line 237). Live mode is click-to-edit preview ( <code>live/LivePreview.tsx</code> , <code>live/InlineBlockEditor.tsx</code> , <code>live/block-split.ts</code> ), described in <code>FRONTMATTER-PRODUCT-PLAN.md</code> §3 as "Live mode v1 → harden to true Obsidian-style live preview" — so <b>PARTIAL</b> against the Obsidian bar, SHIPPED against the mockup's four-button control.
Project tree	<b>SHIPPED</b>	[measured] <code>src/modules/vault/presentation/FileTree.tsx</code> plus a nine-file <code>file-tree/</code> directory: <code>TreeItem.tsx</code> , <code>ContextMenu.tsx</code> , <code>InlineDialog.tsx</code> , <code>ConfirmDialog.tsx</code> , <code>FileTreeActions.tsx</code> , <code>Toast.tsx</code> , <code>path-utils.ts</code> , <code>types.ts</code> . Ordering in <code>tree-order.ts</code> , insertion in <code>tree-insert.ts</code> .
Per-project add-file and add-folder	<b>SHIPPED</b>	[measured] <code>FileTreeActions.tsx:68-69</code> binds <code>sgnk:new-note</code> and <code>sgnk:new-folder</code> ; line 83 <code>newFolderIn: (folder) =&gt; _</code> scopes creation to a parent folder; line 153 renders <code>New folder in \${dialog.parentFolder}</code> .
Multiple projects	<b>MISSING</b>	[measured] There is exactly one vault: <code>GITHUB_REPO</code> defaults to <code>"sagnikmitra/md"</code> ( <code>env.ts:86</code> ), single-valued. <code>FEATURE-GAP-REPORT.md</code> K5 "Multiple vaults / repo switching" is listed <b>✗</b> with "(single repo)". The tree is <i>per-folder</i> , not <i>per-project</i> .
Document tabs	<b>SHIPPED</b>	[measured] <code>editor-store.ts:6</code> <code>export type Tab</code> , <code>:24</code> <code>tabs: Tab[]</code> , <code>:40-41</code> <code>openTab / closeTab</code> , with dirty tracking at <code>:144</code> and neighbour-selection on close at <code>:122-125</code> .
Two-note split	<b>SHIPPED</b>	[measured] <code>active-view.ts</code> tracks <code>{view, path}</code> so "toolbar/Alt/find act on whichever pane" (comment, lines 5-6). Confirmed as the C7 follow-up in <code>FEATURE-GAP-REPORT.md</code> .
Formatting bar	<b>SHIPPED</b> — 12 controls	[measured] <code>Toolbar.tsx</code> defines: Bold (Mod+B), Italic (Mod+I), Strikethrough, Inline code (Mod+E), Heading 1, Heading 2, Bullet list, Task item, Blockquote, Code block, Link, Insert table.
Slash commands	<b>SHIPPED</b>	[measured] <code>src/modules/editor/presentation/slash-commands.ts</code>
Find and replace	<b>SHIPPED</b>	[measured] <code>@codemirror/search</code> wired at <code>CodeMirrorEditor.tsx:40, 206, 207, 231</code>
Vim keybindings	<b>SHIPPED</b>	[measured] <code>@replit/codemirror-vim ^6.3.0</code> in <code>package.json</code> , wired at <code>CodeMirrorEditor.tsx:41, 193</code>
Wikilinks, transclusion, callouts, tags, KaTeX, Mermaid, GFM tables, task toggles	<b>SHIPPED</b>	[measured] <code>remark-gfm</code> , <code>remark-math</code> , <code>rehype-katex</code> , <code>mermaid</code> , <code>katex</code> in <code>package.json</code> ; <code>src/modules/preview/presentation/Markdown.tsx</code> , <code>EmbeddedNote.tsx</code> , <code>table-edit.ts</code> . 19 preview tests including <code>wikilink.test.ts</code> , <code>embeds.test.ts</code> , <code>callout.test.ts</code> , <code>html-policy-xss.test.ts</code> .
Autosave to draft	<b>SHIPPED</b>	[measured] <code>src/modules/drafts/infrastructure/draft-store.ts</code> ( <code>IndexedDB</code> , <code>idb-keyval</code> ), <code>test/drafts/draft-store.test.ts</code>
Autosave to the durable artifact	<b>MISSING — and this is the product-defining defect</b>	[measured] <code>src/modules/repository/presentation/CommitBar.tsx:74</code> <code>const [message, setMessage] = useState("");</code> ; <code>:98</code> <code>const commitMessage = message.trim()    defaultMessage;</code> ; posting to <code>/api/commit</code> , whose Zod schema at <code>src/app/api/commit/route.ts:47</code> requires <code>message: z.string().min(1)</code> . <b>Saving is a manual git commit with a message box.</b> <code>FRONTMATTER-PRODUCT-PLAN.md</code> §2 lists "background auto-sync engine (B8 second half deferred)" under Missing/half-done.
Import	<b>PARTIAL</b>	[measured] <code>ImportModal.tsx:81</code> accepts <code>".md,.markdown,.txt,text/markdown,text/plain"</code> only. No Notion ZIP, no Evernote ENEX, no OneNote, no Apple Notes, no Google Keep.
Export	<b>SHIPPED</b> — 5 formats	[measured] <code>ExportMenu.tsx:156-160</code> : Download .md, Export HTML, Export Word (.doc), Open PDF (print), Download PDF. Plus whole-vault ZIP via <code>/api/export/vault</code> and <code>export-vault-zip.ts</code> ( <code>fflate</code> ).
Trash / soft delete	<b>SHIPPED</b>	[measured] <code>src/modules/vault/presentation/TrashModal.tsx</code> , <code>/api/vault/delete</code> , <code>/api/vault/restore</code>
PWA and desktop	<b>SHIPPED</b>	[measured] <code>PWARegister.tsx</code> + <code>src/app/manifest.ts</code> + <code>public/sw.js</code> ; <code>src-tauri/</code> + <code>TauriBridge.tsx</code> + four <code>tauri:build:*</code> scripts in <code>package.json</code>

7.2.3 The right rail

The mockup shows a **tabbed** rail with seven slots: Document Outline, Tags and Bookmarks, Document History, Comment, Add File, Keyboard Shortcuts, AI Edit.

What is actually built is not a tabbed rail. [measured] `src/modules/preview/presentation/RightPane.tsx` is a single scrolling column of **five** `<section>` blocks, in this order: **Backlinks**, **Unlinked mentions**, **Outline**, **Bookmarks**, **Tags**. There is no tab bar, no slot for the other four items, and `grep -in 'comment' RightPane.tsx` returns nothing.

MOCKUP_SLOT	STATUS	VERIFICATION
Document Outline	SHIPPED	[measured] <code>Outline.tsx</code> + <code>outline-utils.ts</code> , rendered as the third section of <code>RightPane.tsx</code> . <code>github-sluggger</code> + <code>rehype-slug</code> for anchors. <code>test/preview/outline.test.ts</code> .
Tags and Bookmarks	SHIPPED	[measured] <code>KnowledgePanels.tsx</code> exports <code>TagsPanel</code> and <code>BookmarksPanel</code> ; bookmark state in <code>src/modules/editor/presentation/bookmarks.ts</code> . Both are rail sections.
Document History	SHIPPED as a modal, PARTIAL against the mockup	[measured] <code>src/modules/editor/presentation/HistoryModal.tsx</code> lists commits from <code>/api/vault/history</code> , previews a version from <code>/api/vault/version</code> , and restores by committing the old content over HEAD with the live blob sha as <code>baseSha</code> for optimistic-concurrency safety. <b>It is not in the rail — it is a modal.</b> And <code>grep -cin 'diff' HistoryModal.tsx</code> returns <b>0</b> : there is no diff view anywhere in the product. Across all of <code>src/</code> , every <code>diff</code> hit is inside <code>merge3.ts::diffRegions</code> , used for three-way merge and never rendered.
Comment	MISSING — no code, no storage, no schema	[measured] No component, no module, no route. <code>firestore.rules</code> contains the string <code>comments</code> <b>zero</b> times. The one rail slot that requires a second human is the one with nothing behind it.
Add File	SHIPPED, but in the tree	[measured] <code>FileTreeActions.tsx</code> <code>new-note</code> / <code>new-folder</code> dialogs. Not a rail slot.
Keyboard Shortcuts	MISSING	[measured] <code>grep -rln 'Keyboard shortcuts shortcuts' src/modules/app-shell/presentation/</code> returns nothing; <code>SettingsModal.tsx</code> contains no <code>shortcut</code> or <code>hotkey</code> string. Hotkeys are bound in <code>use-hotkey.ts</code> but there is no surface that lists or remaps them. <code>FEATURE-GAP-REPORT.md</code> C12 lists this <b>✗</b> .
AI Edit	SHIPPED, as a floating menu	[measured] <code>src/modules/editor/presentation/AIMenu.tsx</code> — actions <code>"refine"</code>   <code>"summarize"</code>   <code>"suggest-links"</code> (line 25) plus prompt presets <code>Refine selection / note, Expand, Shorten, Change tone: professional</code> (lines 248-251). Not a rail slot.
Backlinks (in code, not in the mockup)	SHIPPED	[measured] <code>Backlinks.tsx</code> , <code>UnlinkedMentions.tsx</code> , <code>link-index.ts</code>

**Negative worth stating.** Four of the mockup's five *content* rail slots are filled by features that work for exactly one person. The one slot that requires a second person is empty and its data model does not exist. Adding a sixth rail item before there is a second user is strictly negative work. (final-gate `product-gap` , NEGATIVE 4)

7.2.4 The AI writing surface

FEATURE	STATUS	VERIFICATION
Multi-provider gateway	SHIPPED	[measured] <code>src/modules/ai/infrastructure/gateway-client.ts</code> + <code>provider-race.ts</code> ; <code>package.json</code> carries <code>@ai-sdk/cerebras</code> , <code>@ai-sdk/google</code> , <code>@ai-sdk/groq</code> , <code>@ai-sdk/mistral</code> , <code>@openrouter/ai-sdk-provider</code> , <code>ai ^6.0.191</code> . <code>test/ai/provider-race.test.ts</code> .
Refine / expand / shorten / change tone	SHIPPED	[measured] <code>refine-text.ts</code> → <code>/api/ai/refine</code> ; presets in <code>AIMenu.tsx:248-251</code>
Summarize	SHIPPED	[measured] <code>summarize.ts</code> → <code>/api/ai/summarize</code> ; inserts a <code>&gt; [!summary]</code> callout block ( <code>AIMenu.tsx:192</code> )
Suggest links	SHIPPED	[measured] <code>suggest-links.ts</code> → <code>/api/ai/suggest-links</code> , <code>apply-wikilinks.ts</code>
Link doctor	SHIPPED	[measured] <code>link-doctor.ts</code> → <code>/api/ai/link-doctor</code> , <code>LinkDoctorModal.tsx</code>
Generate document	SHIPPED	[measured] <code>generate-document.ts</code> → <code>/api/ai/generate-doc</code> , <code>doc-prompts.ts</code>
Ghost-text inline completion	SHIPPED	[measured] <code>ghost-text.ts</code> , <code>ai-suggestion.ts</code> , <code>completions.ts</code> → <code>/api/ai/complete</code>
Bring-your-own key	MISSING	[measured] <code>grep</code> across <code>src/modules/ai/</code> finds no user-key path; all providers read server env. <code>FRONTMATTER-PRODUCT-PLAN.md</code> Pillar 6 lists BYO-AI <b>■</b> .
AI over the vault (RAG)	MISSING	[measured] No embeddings, no vector store, no pgvector. <code>FEATURE-GAP-REPORT.md</code> <b>G2 ✗</b> , effort L.

7.2.5 One shipped feature that is actively harmful

[measured] The Publish write path corrupts the file format the company is named after.

`src/modules/share/infrastructure/share-writer.ts` lines 1-5 claim, verbatim, to be *"reading the current body via the vault reader, **splicing the key into the YAML frontmatter**, and committing through `commitChanges`."* Line 26 is:

```
const next = matter.stringify(parsed.content, data);
```

That is a whole-document re-serialisation of the entire YAML map from a plain JavaScript object. It violates **D7 (splice-only)** in the one shipped write path, and the source comment would let a reviewer approve it.

Measured against the pinned corpus by replicating that exact code path, and independently reproduced by the final-gate verifier to the digit:

MEASUREMENT	VALUE
files with frontmatter	907
byte-identical after a no-op write	33 (3.64%)
gray-matter throws (unpublishable → HTTP 502)	171 = 170 parse failures + 1 stringify failure ( <code>md/pj.md</code> )
bytes changed	703
byte-identical after publish → unpublish	17/736 parsed = 2.31%; 17/907 = 1.87%
bare <code>YYYY-MM-DD</code> rewritten to ISO timestamp	624 of 736 parsed = 84.78%
<code>matter().data</code> unchanged after mangling	703 stable / 0 differ

That last row is the most decision-relevant number in this entire section. `title: 2026-05-26` becomes `title: 2026-05-26T00:00:00.000Z` — a visible title change in Obsidian, a broken Dataview filter, a broken Hugo or Jekyll build — and **every existing test that asserts on `matter().data` passes green while the bytes on disk are wrong**. The corruption is invisible to the test suite by construction.

Publish fires on every publish and every unpublish, through `setShare` → `POST /api/share`. `src/app/api/share/route.ts` returns `status: 502` with `error: "upstream_failure"` on any non-slug error, so the 171 files simply cannot be published.

**What this is not.** It is not a library-choice problem. The repo already contains a comment-and-order-preserving path (`src/modules/preview/presentation/frontmatter.ts`, `eemeli/yaml` `parseDocument`, consumed by `PropertiesPanel.tsx:79`) and Publish does not use it — but ported to the same corpus it reaches only **13.12% published-and-byte-identical** (119/907), and it fails to parse **the same 170 files**. A swap is not the fix. A byte-range splice is.

**UNVERIFIED, and it deserves a live check.** The 502 claim is a code-path trace, not an execution. Nobody has run the application against a live vault and confirmed that publishing one of the 171 files returns 502. Do that before quoting it externally.

7.2.6 The test suite currently proves nothing

`[measured]` `npx vitest list --filesOnly` returns **247** files, of which **164 (66.40%)** are under `.claude/worktrees/` — two abandoned git worktrees (`competent-bassi-5da9a1`, `upbeat-euclid-60dbf4`) pinned at a pre-work commit — and 83 are the real suite under `test/`. `vitest.config.ts` declares **no** `exclude` **key at all** (verified: the file is 16 lines and sets only `environment`, `setupFiles` and a `@` alias), and vitest's default exclude covers `**/{.idea,git,cache,output,temp}/**`, which does not include `.claude`.

**Any "N tests pass" claim about this repository is currently meaningless.** The fix is one line in `vitest.config.ts` and the gate is `vitest list --filesOnly` moving 247 → 83.

7.3 The comparison matrix, organised by the review loop

Feature-by-feature comparison is the wrong frame. Every product in this table has an outline panel and a dark mode. **The only axis on which Google Docs wins, and wins so completely that people abandon markdown to reach it, is the review loop.** So the matrix is organised by that loop.

7.3.1 The review loop, defined as six steps

A document goes out for review and comes back improved. Six things must happen. Each is a place a product can lose.

#	STEP	THE FAILURE IF IT IS MISSING
R1	<b>Get it in front of a reviewer</b> — a link, sent in a message	if this needs the reviewer to install software or clone a repo, the loop ends here
R2	<b>The reviewer gets in</b> — opens the link and can act	a login wall is the single highest-attribution step; Google explicitly engineered around it
R3	<b>The reviewer marks a specific passage</b> — an anchored comment, not "para 3 seems off"	without anchoring, review degrades to email, which is what markdown users actually do
R4	<b>The reviewer proposes exact wording</b> — a suggestion the author accepts or rejects with one click	without this, the author retypes every change by hand
R5	<b>Threads resolve and disappear</b> — resolve, reopen, assign, @mention	without resolution, a long document accretes stale commentary and review stops scaling
R6	<b>The artifact left behind is clean</b> — the file that ships carries no review residue	this is D2, and it is what every markdown-native attempt gets wrong by writing comments into the file

7.3.2 The matrix

Sourcing: rows marked `[primary]` were fetched and read directly during the final-gate research run on 2026-08-01 (`competitive-live`, verdict **CONFIRMED**; `product-gap`, verdict **CONFIRMED**). Rows marked `[secondary]` come from `docs/research/frontmatter-competitor-gapmap.md` (2026-07-12). Rows marked `[UNVERIFIED]` have no source in this record and **must be verified before this table appears in any external material**.

PRODUCT	R1 LINK	R2 NO-LOGIN	R3 ANCHORED COMMENT	R4 SUGGESTIONS	R5 RESOLVE	R6 CLEAN PLAIN-.MD ARTIFACT	PRICE	SOURCE
Google Docs	yes	<b>yes</b> — "anonymous animals"	yes	yes, nested, per-text-run	yes — resolve, reopen, assign	<b>no</b> — proprietary; export "supports Markdown just enough to frustrate the life out of you"	free personal; Workspace ~\$7–8.40/user/mo $\Delta$	[primary] Drive v3 rev 20260728, Doc rev 20260727, support.google.com/docs/answer/2494 price [secondary]
HackMD	yes	partial	<b>yes</b> — in-line and page commenting, <b>free tier</b>	<b>yes</b> — "Suggest edit", <b>free tier</b>	yes	<b>no</b> — server-side doc pad; no vault, no offline; GitHub push capped 20/mo free	Free; Prime <b>\$5/seat/mo billed annually</b> (~\$8 monthly), 3-seat floor = \$15/mo	[primary] hackmd.io/pricing, parsed markup level: all four Collaboration row Check/Check/Check across Free/Prime/Enterprise
Notion	yes	no	yes	limited	yes	<b>no</b> — lossy export; relations→text, rollups/views vanish	Free; Plus \$10/user/mo; Business \$20; AI \$10 per 1,000 credits	[secondary] gapmap, verified from of pricing page 2026-07-12
Obsidian	shared vault only	<b>no</b> — every collaborator needs a paid Sync seat	<b>no</b>	<b>no</b>	<b>no</b>	<b>yes</b> — plain .md on disk, the category benchmark	core free; Sync <b>\$4/user/mo annual, \$5 monthly</b> ; Publish \$8/site/mo; <b>20-collaborator cap</b>	[primary] obsidian-help Collaborate shared vault.md , four verbatim quotes "Fine-grained permissions are not supported yet" / "You will not see the other user's cursor" / "maximum ... is 20 users" / "All collaborators must have an active Sync subscription"
Moment.dev (2026 entrant)	yes	no	<b>no</b> — <code>grep -ioE "comment suggest"</code> over homepage + docs + pricing returns <b>zero matches</b>	<b>no</b>	<b>no</b>	<b>yes</b> — "Actual files, on actual disk", full history via Jujutsu and git	Free 1 user; <b>Team \$30/mo up to 5 users, +\$6/user</b>	[primary] moment.dev homepage, /d/pricing, fetched 2026-08-01
inkeep/OpenKnowledge (2026 entrant)	n/a — local	n/a	<b>yes</b> , content-derived anchoring, orphan-rather-than-guess	no	partial	<b>yes</b> — sidecar, never committed	free, <b>GPL-3.0</b>	[primary] 3,239★, created 2026-06-pushed 2026-08-01, 14,790 npm dl/wk 381 pts / 173 comments. <code>.changeset/comments-v1.md</code> verbatim: <i>comment here is a note to your own agent, not a message to a teammate.</i> " Merge 2026-07-30; beta 0.46.0-beta.32 at 08-01T02:38:33Z; <b>absent from stable 0.45.4</b>
Craft	yes	no	[UNVERIFIED]	[UNVERIFIED]	[UNVERIFIED]	<b>no</b> — blocks not files; lossy md round-trip	Free (1,500 blocks); Plus \$10/mo or \$96/yr; Team \$60/mo	[primary] homepage contains <b>0 occurrences</b> of "markdown", "comment"; pricing [secondary] gap
Bear	Pro-gated	no	no	no	no	pseudo-markdown; Apple-locked	Free; Pro \$2.99/mo or \$29.99/yr	[primary] homepage: 7 "markdown", " <b>comment</b> ", <b>0 "collaborat"</b> ; pricing [secondary]
Coda	—	—	—	—	—	<b>absorbed</b> — banner reads verbatim "Coda is now Superhuman Docs"	—	[primary] coda.io fetched 2026-08-01
Confluence	yes	no	yes	[UNVERIFIED]	yes	<b>no</b> — proprietary storage format	[UNVERIFIED] — the segment research says only "Confluence per-seat pricing is the incumbent budget line (exact 2026 price unverified)"	[UNVERIFIED] — <b>no primary source anywhere in this record. Do not publish row without fetching atlassian.com/software/confluence/</b>
Slite	[UNVERIFIED]	[UNVERIFIED]	[UNVERIFIED]	[UNVERIFIED]	[UNVERIFIED]	[UNVERIFIED]	[UNVERIFIED]	<b>Slite does not appear anywhere in the research corpus, the 20-app competition matrix, or any research area. This row is a hole. It must be researched before the matrix is used externally.</b>

PRODUCT	R1 LINK	R2 NO-LOGIN	R3 ANCHORED COMMENT	R4 SUGGESTIONS	R5 RESOLVE	R6 CLEAN PLAIN-.MD ARTIFACT	PRICE	SOURCE
Almanac	—	—	—	—	—	—	—	<code>[primary]</code> <code>curl -o /dev/null -w '%{http_code}' https://app.almanac.io</code> (2026-08-01); almanac.io redirects to get.almanac.io whose only internal link <code>/go-forward</code> ("A letter from our CEO"; login, no pricing, no product links.
frontmatter (today)	read-only public URL only	yes for reading	no	no	no	yes — but Publish rewrites frontmatter on 84.78% of parseable files	one user	<code>[measured]</code> <code>POST /api/share</code> <code>schem: {path, slug}</code> and nothing else
frontmatter (the target)	link + roles	OPEN — D3	yes, sidecar	yes, patch set	yes	yes, D2-gated	Free / \$4 / \$50-yr / \$5-8-seat	this plan

7.3.3 What the matrix says

Read the R6 column against the R3–R5 columns and the shape falls out:

- Everything with a real review loop stores your document in its own database. Google Docs, HackMD, Notion, Confluence.
- Everything that leaves you plain files has no review loop. Obsidian states it plainly. Moment.dev has zero occurrences of the word. Bear and Craft have zero occurrences of the word on their homepages. OpenKnowledge built the anchoring machinery and then declined the teammate model in writing.

That intersection is the wedge, and it is genuinely empty. `[primary]`

Two corrections to the "unclaimed combination" story that this plan previously told. (1) `docs/Frontmatter-Product-Plan.md` whitespace combination 1 is "Real-time collab + plain markdown files + git history" and calls it unshipped. It is shipped. Moment.dev ships exactly that, priced. (2) The same document's Wedge B framing implies Obsidian has "zero multiplayer". Obsidian ships shared vaults with version history — worse than Docs (no cursors, no roles, 20-user cap, every collaborator pays \$4/mo) but not zero. Reprice the wedge around comments and suggestions, not around multiplayer.

And the honest threat assessment. OpenKnowledge is three months old, has the hard half built, and is one identity-and-sync layer away from the whole loop. If they ship a teammate model, or if Moment.dev adds comments, the intersection closes. Both are three-month risks, not three-year risks. Re-measure both quarterly via `api.github.com` stars and `pushed_at` and `api.npmjs.org` downloads — both are reachable by `curl` and both produced the figures above.

7.4 What Google Docs does that markdown tools structurally cannot — separated from what they merely have not built

This distinction matters more than any other in this section, because conflating the two produces a roadmap full of items that cannot be delivered, and a marketing page that will be called out.

The constraints that create the boundary are this plan's own decisions: **D6** (the file stays ordinary `.md`; no dialect needing its own parser), **D2** (comments and history live outside the file), and **D7** (splice-only writing; never regenerate from the AST).

7.4.1 Structurally cannot — the four real ones

S1. Docs-equivalent suggestions. This one is provable from the API.

`[primary]` The Google Docs API v1 discovery document (revision 20260727) has 35 schemas whose name contains "Suggest" out of 170 total. `TextRun.suggestedInsertionIds` is documented verbatim: "A *TextRun* may have multiple insertion IDs if it's a nested suggested change." `suggestionsViewMode` is an enum with four values: `DEFAULT_FOR_CURRENT_ACCESS`, `SUGGESTIONS_INLINE`, `PREVIEW_SUGGESTIONS_ACCEPTED`, `PREVIEW_WITHOUT_SUGGESTIONS`.

Suggestions in Docs are carried inside the document model, on every text run, and they nest. A `.md` file has no per-run identity to hang an insertion ID on. Under D6 you cannot add one — that would be a dialect requiring its own parser, which is exactly what D6 forbids.

What this costs, concretely: nesting (a suggestion on top of a suggestion), per-character attribution inside a run, and the "preview with all suggestions accepted" view being free — in Docs the server just filters by view mode; in frontmatter you must apply the patch set to compute it.

What you can build instead: suggestions as an out-of-band patch set — base sha, hunks, author — rendered inline in the editor, accepted by splice. This is a real, shippable feature that solves the user's actual job.

Say so publicly. The honest framing is stronger than the imitation, and it is the difference between "we made a deliberate architectural trade so your file stays a file" and "their suggestions are broken".

S2. A single character-address space that survives a view-mode switch.

Docs has one document model. What you see and what is stored are the same object; an anchor is an offset into that object and cannot disambiguate wrongly.

Markdown editors have two: the body bytes and the rendered text. The mockups ship four view modes (Live / Edit / Split / Read), so this is a day-one problem, not a later refinement.

`[primary]` OpenKnowledge hit this and wrote it down, verbatim in `packages/core/src/comments/passag` :  

"A comment anchor is measured against a document's markdown BODY, but the editor only ever knows RENDERED text... Converting one side to the other is not reliable — serializing a partial ProseMirror selection fabricates the block marker of whatever block the selection happens to sit in, so a pick starting mid-bullet comes back as `- 3 tbsp peanut butter` when the source line reads `- **Peanut sauce:** 3 tbsp peanut butter`."

Their fix is a third matcher with syntax-elastic matching plus a `LINK_TAIL` skip, because "Missing this is why a passage containing any link could not be matched at all, which in a linked wiki is most of the interesting passages." That code is GPL-3.0 — readable as a design reference, not copyable into a proprietary SaaS.

No npm package solves this. `@codemirror/collab` (latest 6.1.1, modified 2026-04-13) addresses operational transform, not content-derived anchoring.

S3. Guaranteed anchor survival.

In Docs, an anchor is an identifier the server owns and maintains through every edit. A content-derived anchor can orphan, and there is no construction that prevents it while D6 holds.

[measured] The capability-run `collab-primitives` area measured this on real revision pairs. **Use the verification, not the headline** — the headline ("2.70% → 0.09%, 30×") was killed by its own verifier as orphaned from the evidence and built on roughly two raw events (Wilson 95% CI [0.024%, 0.312%]), and the count "2,332 revision pairs" was a count of simulated selections reported as revision pairs; there were **400** revision pairs. The verifier's own independent replication:

MEASUREMENT	VALUE
naive byte-offset anchor, correct after one real revision	<b>62.83%</b> (verifier) vs 65.31% (original), n=2,400 selections over 400 pairs
naive byte-offset anchor, <b>wrong or gone</b>	<b>37.17%</b> (verifier) vs 34.69% (original)
silent mis-anchoring, unscoped → block-scoped quote	<b>2.58% → 0.75% (3.44×)</b> , not 30×
refusal rate, unscoped → block-scoped	<b>11.79% → 22.88%</b> — the cost that doubles alongside the benefit

**Design consequence, which is DECIDED:** fail **SAFE** to `orphaned`, never guess, and **ship the orphan rate as a visible product metric from the first comment written**. A product that silently moves a reviewer's comment to the wrong paragraph is worse than one that says "this passage changed".

S4. Live cursors across four view modes.

Docs has one view, so a cursor is one position. frontmatter has Live, Edit, Split and Read, which means a cursor in Edit has no single correct rendering in Read. This is strictly harder than Docs's problem, and it is the reason presence should ship as **avatars and per-document "who is here"** before it ships as character-level cursors. Obsidian, which has the same constraint, states plainly that it does not do cursors at all.

7.4.2 MERELY HAVE NOT BUILT — no structural obstacle whatsoever

Every one of these is ordinary application engineering. None of them is blocked by D2, D6 or D7. Listing them separately is what makes the roadmap honest.

#	CAPABILITY	WHY THERE IS NO OBSTACLE
M1	<b>Anonymous access for readers and commenters</b>	[primary] Google's own mechanism, stated verbatim: <i>"People who aren't signed in to a Google Account show up as anonymous animals in your file."</i> Purely an auth-policy decision — this is <b>D3</b> , currently under review
M2	<b>Roles on a share link (viewer / commenter / editor)</b>	<code>firestore.rules</code> <b>already specifies</b> <code>roles: map&lt;uid,"owner" "editor" "viewer"&gt;</code> with enforcement at lines 284-289. The schema exists; the code does not
M3	<b>Threaded comments with resolve, reopen, assignee</b>	[primary] Drive v3 <code>Comment</code> is <b>14 properties</b> and <code>Reply</code> is <b>11</b> . That is the entire data model — <code>anchor</code> , <code>quotedFileContent</code> , <code>resolved</code> , <code>assigneeEmailAddress</code> , <code>Reply.action ∈ { resolve, reopen }</code> . It is CRUD over a sidecar collection
M4	<b>@mentions and notification</b>	address-book lookup plus an email send
M5	<b>A diff view in Document History</b>	[measured] <code>/api/vault/history</code> and <code>/api/vault/version</code> already return the data; <code>HistoryModal.tsx</code> has <b>0 occurrences of "diff"</b> . The rendering is the missing piece, and <code>merge3.ts::diffRegions</code> already computes regions
M6	<b>Autosave without a commit message</b>	debounce plus a default message. The engine (atomic Git Data commits, <code>baseSha</code> 409 optimistic concurrency, three-way merge with 14 conflict-matrix tests) is <b>done</b> ; only the loop that calls it is deferred
M7	<b>Suggestions as an accept/reject patch set</b>	base sha + hunks + author, applied by splice
M8	<b>Presence avatars</b>	a heartbeat document and a listener
M9	<b>Offline read/write with a queued outbox</b>	the PWA shell already exists ( <code>PWARegister.tsx</code> , <code>public/sw.js</code> , <code>src/app/manifest.ts</code> )
M10	<b>Mobile layout</b>	[inference] 13 Tailwind responsive prefixes across 74 <code>.tsx</code> files, 3 <code>@media</code> rules in <code>globals.css</code> , 1 in <code>public-note.css</code> ; two real drawers at <code>VaultWorkspace.tsx:170</code> and <code>:199</code> . Tagged <b>inference</b> because a prefix count is a proxy for layout quality, not a measurement of it — <b>nobody has rendered this application at a mobile viewport</b>
M11	<b>Per-user commit attribution</b>	delete a constant at <code>dependency-container.ts:42</code> and thread the actor through four call sites
M12	<b>Importers (Notion ZIP, ENEX, OneNote, Keep, Apple Notes)</b>	[measured] <code>ImportModal.tsx:81</code> accepts <code>.md / .markdown / .txt</code> only

**The line to hold:** frontmatter cannot promise Docs-identical suggestions, and should never claim to. Everything else in the review loop is buildable, and the reason none of it is built is not markdown. It is that there is one user.

7.5 What frontmatter needs on top of MDMAX

MDMAX is a markdown capability layer. Per `docs/mdmax/PLAN.md` §10.4 the settled answer is that it is a **~600-line library inside `src/modules`**, **subordinate to the editor, deleted when the editor stops needing it** — not a compiler programme. Nothing in this subsection is engine work, and **nothing on the critical path to revenue requires MDMAX to exist as a compiler**. (final-gate synthesis, `product-market` lens, `dependencies` )

Each item below states what it is, why the engine does not give it, what it depends on, and whether it is DECIDED, RECOMMENDED or OPEN.

N1 — Identity and tenancy (the gate)

**Status: DECIDED (that it must ship). The design is RECOMMENDED below.**

Implement exactly the schema `firestore.rules` already specifies. No more.

```

users/{uid}
 email, displayName, photoURL, providers[], plan, defaultVaultId, createdAt, lastSeenAt

vaults/{vaultId}
 ownerId, name, rev, memberUids: string[<=100], roles: map<uid,"owner"|"editor"|"viewer">,
 createdAt, updatedAt

```

Then delete the three things that make a second user impossible:

1. `src/modules/auth/domain/allowlist.ts` — the single string equality
2. `src/container/dependency-container.ts:42` — the `AUTHOR` constant, threading the signed-in actor through lines 66, 70, 75, 80 instead
3. `src/config/env.ts:85-87` — the single server-side `GITHUB_REPO_TOKEN` / `GITHUB_REPO`, replaced with a per-user GitHub App installation

**The exit test, committed RED first.** From the final-gate `product-market` synthesis, and this is the form that matters:

*A second GitHub login who is not `sagnikmitra` opens a document they do not own, writes one character, and the commit carries **their** name and email.*

It cannot be satisfied by prose, and it cannot pass while the allowlist, the single token, or the `AUTHOR` constant survive. **Per plan rule P10, the unit of progress is this test going green — not a test, that test.**

**Why it will slip, stated so it does not.** It has zero engine content. It is uninteresting. The plan's own kill-condition K3 predicts it will keep losing priority to compiler work *precisely because* it is boring, and names that as **the single most likely way this programme fails**.

**Backend choice.** Firestore. `docs/mdmax/PLAN.md` §10.1 records the open decision — Firestore is in the code ( `firebase ^12.16.0` ) and in the mockups, while `docs/Frontmatter-Product-Plan.md` says Supabase eleven times. **Recommendation stands: pick Firestore, correct the product plan.** OPEN until a founder says so.

## N2 — Auth and the vault binding

**Status: PARTIAL today; RECOMMENDED design below.**

Today: NextAuth v5 with GitHub OAuth (scope `read:user` ) plus a scrypt password fallback, JWT sessions. That much is real and works.

Missing: a **GitHub App installation** per user, so each vault is that user's repository reached by that user's grant. `docs/Frontmatter-Product-Plan.md` Phase 1 item 2 already names this — *"invisible auth, no PATs"* — and the demand corpus is emphatic that hand-pasted personal access tokens are the first wall non-developers hit.

Also missing: **multi-vault switching** ( `FEATURE-GAP-REPORT.md` K5, **✗**, effort M).

**Negative:** a GitHub App installation flow is a real piece of work with a review step on GitHub's side and an unavoidable second consent screen. It is not two days. Budget it as its own deliverable, not as a sub-task of N1.

## N3 — Share links with enforced roles

**Status: MISSING. Design RECOMMENDED. D3 is OPEN.**

[measured] Today the entire share surface is:

```

// src/app/api/share/route.ts:21
const postSchema = z.object({ path: z.string().min(1), slug: z.string().min(1).max(60) });

```

No role. No expiry. No password. No per-user grant. `PublicNoteView.tsx` renders a read-only `<article>` with zero comment affordances.

What is needed: a `shares/{slug}` document (already specified in `firebase.rules` ) extended with `role ∈ {viewer, commenter, editor}`, `expiresAt`, optional `passwordHash`, and `revokedAt`; plus a server-side gate that enforces the role on every read and write, not merely in the UI.

**The D3 question, and it is the product question.** D3 as written says reviewers must log in. [primary] Google's own share mechanism removes exactly that step: *"Anyone with the link: Anyone who has the link can use your file, without signing in to their Google Account."*

**RECOMMENDED: reopen D3 as a tier split, not a global rule.** Anonymous read-and-comment on a share link; login required to resolve a thread or to edit. This preserves the wedge and the account model at once. **Owner and editor seats bill; commenters do not.**

**And this must be tested before it is decided.** The measurement is five people, one week, one task: watch them try to get a comment from a non-technical colleague on a markdown document. The pre-registered prediction is that the blocker they hit first is the login wall, not anchoring and not fidelity. **If four of five stall at signup, D3 must be revisited at auth, not after** — the auth decision *is* the product decision.

**Negative worth carrying:** anonymous commenting is an abuse surface. Rate limiting, a per-link comment cap, and a one-click "require sign-in" toggle for the owner are not optional extras; they ship with it.

## N4 — The comment data model

**Status: MISSING ENTIRELY.** `firebase.rules` names it zero times. **Design RECOMMENDED.**

Copy Google's own split, because it is the split **D2** already chose.

```

vaults/{vaultId}/notes/{noteId}/comments/{commentId}

identity
authorUid string | null # null = anonymous commenter (N3 tier split)
authorLabel string # display name, or the anonymous-animal label
createdAt timestamp
updatedAt timestamp

anchoring — a W3C TextQuoteSelector, a Recommendation since Feb 2017. Not an invention.
anchor:
 quote string # the exact selected text — the durable record
 prefix string # context before, widened AT CREATE TIME until unique
 suffix string # context after, same rule
 blockId string # scoping unit; cuts silent mis-anchoring (see S3)
 baseRevision string # the git sha the anchor was created against
 status "anchored" | "orphaned"

thread
resolved bool
resolvedBy string | null
assigneeEmail string | null

vaults/{vaultId}/notes/{noteId}/comments/{commentId}/replies/{replyId}
authorUid, authorLabel, body, createdAt
action "resolve" | "reopen" | null
```

[primary] This mirrors Drive v3 field-for-field where it matters: `Comment.anchor` ("A region of the document represented as a JSON string"), `Comment.quotedFileContent` ("The file content to which the comment refers, typically within the anchor region"), `Comment.resolved`, `Comment.assigneeEmailAddress`, and `Reply.action` ∈ { `resolve`, `reopen` }. Drive's `Comment` is 14 properties and `Reply` is 11 — the whole model is small.

**The D2 carrier gate, which is DECIDED and non-negotiable:**

Delete the entire sidecar. `git status` is clean. Every file byte-identical.

Nothing about a comment is ever written into the `.md`. This is corroborated three ways: D2 by decision; Google by architecture; and [primary] OpenKnowledge, who arrived at the same place independently — comments in a machine-local JSON sidecar, never committed. It is corroborated a fourth way by a competitor's changelog: Obsidian tightened its **in-file** `%%...%%` comment syntax on 2026-07-30, and that syntax is Obsidian-only, its parsing rules are still changing seven years in, and it produces visible junk in every other renderer.

**The anchoring surface nobody has scoped, and it must be budgeted explicitly.** Because the mockups ship Live / Edit / Split / Read *and* a Comment rail, the body-offset-versus-rendered-offset problem (S2) is a day-one bug. Anchor offsets are stored against the markdown **body**; the Live and Read views need a syntax-elastic matcher that skips inline emphasis, code, strikethrough, link tails `I(...)`, and line-leading block markers. **Fail SAFE to** `orphaned`. **Never guess.**

**One correctness gate that is easy to miss and is load-bearing.** `normalize()` must strip HTML comments before hashing a block, because the sentinel write would otherwise change the identity of the block it annotates. The plan's `normalize()` is currently "NFC + collapse whitespace + lowercase". [measured] HTML comments appear inside 0.27% of blocks (911 of 337,410 blocks over 5,916 `.md` files across both corpora) — small, but a 0.27% silent identity corruption is not acceptable in the anchoring layer. And per plan rule **P7**, freeze and version `normalize()` **before** persisting a single anchor; changing it afterwards invalidates every anchor ever written.

**Kill condition, restated from the final-gate** `product-gap` **area**: if the hand-audited **false-match** rate on ranges exceeds ~0.5%, content-derived anchoring cannot be the primary mechanism, and comments must fall back to visible orphaning with a quote-plus-digest. Hand-audit 200 cases. A false match is far worse than a refusal.

**N5 — Suggestions as an out-of-band patch set**

**Status: MISSING. Design RECOMMENDED. Bounded by S1 above.**

```

vaults/{vaultId}/notes/{noteId}/suggestions/{suggestionId}
authorUid, authorLabel, createdAt
baseSha string # the blob sha the suggestion was authored against
hunks: [{ startByte, endByte, oldText, newText }]
state "open" | "accepted" | "rejected" | "stale"
```

"stale" fires when `baseSha` no longer matches HEAD and a hunk no longer applies cleanly. Accepting a suggestion is a **splice** — locate the byte range, replace only those bytes — never an AST regeneration (**D7**).

**The permission boundary is the enforcement point, not the UI:** a suggest-role user has **no code path that writes file bytes**. Suggestions live entirely in the sidecar until an editor accepts one.

**CriticMarkup is interchange, not storage.** If exported, emit substitutions as adjacent `{--old--}{++new++}`.

**Negative:** suggestions are the second-hardest thing in this section and they are downstream of N1, N3 and N4. If the twelve months run short, this is the first thing to cut, and comments alone still constitute a product.

**N6 — Presence**

**Status: MISSING. RECOMMENDED as avatars first.**

[measured] `grep -rn 'yjs|y-websocket|automerge|presence|awareness' src/ package.json` returns **nothing**. There is no realtime layer of any kind.

Ship, in order: (1) "who is in this document" avatars from a heartbeat document; (2) per-document last-seen; (3) character-level cursors **not this cycle**, per S4.

**Architecture, DECIDED:** server-authoritative sequencing over the existing `merge3` + `baseSha` engine, with session-batched commits carrying `Co-authored-by`. The live layer is ephemeral; **the file remains the document**. No peer CRDT.

**A correction to the record that must not be re-propagated.** `docs/mdmax/PLAN.md` §8 gave reasons for this architecture; §10.3 records that **both of its stated reasons are wrong** — one rests on a struck, `%`-commented line absent from the published paper. The decision survives; justify it on the Relay bug and on adoption (73 downloads/week versus 7.2M/week), and strike the other two. Note also that `docs/FRONTMATTER-PRODUCT-PLAN.md` still has three rows specifying Yjs, which now contradicts this. `PLAN.md` §4b governs; **the product plan's Yjs rows are stale and should be edited**.

N7 — Billing and metering

Status: MISSING. Specified in rules only.

`firestore.rules` already declares `billing/{uid}` (owner-read, `allow write: if false` — Admin SDK only) and `usage/{uid}/months/{YYYY-MM}` with `aiTokens`, `noteCount`, `bytesUsed`. Zero implementation.

Needed: a payment provider, a webhook that writes `billing` and flips `users/{uid}.plan` via the Admin SDK, seat counting that bills owners and editors but **not** commenters, and a metering path for whatever the pricing model actually meters — see §7.6, where that turns out to be an unresolved contradiction.

**Negative:** the `plan` field being server-owned is already correctly enforced (`firestore.rules:214` restricts client creates to `'free'`, `:219` makes `plan` immutable on client update). That is the paywall-bypass surface and it is already closed. Good design; still zero code.

N8 — Offline sync

Status: PARTIAL (PWA shell only).

The durability model, DECIDED from the round-2 tension resolutions: **the repository is the durability layer; origin storage is a rebuildable cache; unsynced work is bounded to an aggressively-flushed outbox; never depend on iOS background execution**. PWA storage eviction is normal — make it survivable rather than pretending it will not happen.

Conflict handling, DECIDED: auto-merge non-overlapping hunks silently; on true overlap keep local visible, journal the losing version, and show a non-modal "both versions kept" notice. A blocking hunk-picker is opt-in Power mode.

**Negative:** the demand corpus's number-one feature request is *"Always-visible sync status indicator (what synced, what is pending, when last synced)"* and it does not exist. `[measured]` No sync-status component exists in `src/`. The engine is done; the trust surface a user can see is not — and the corpus says the visible surface is what they are asking for.

N9 — Mobile layout

Status: PARTIAL. Genuinely unmeasured.

`[inference]` 13 Tailwind responsive prefixes across 74 `.tsx` files; 3 `@media` rules in `globals.css`, 1 in `public-note.css`; two real drawer patterns (`VaultWorkspace.tsx:170` and `:199`, `fixed inset-y-0 ... lg:hidden` with translate transitions).

**This is a source grep, not a rendering test. Nobody has opened this application at a mobile viewport.** The first honest mobile action is not a build task — it is to open it on a phone and take screenshots.

N10 — Notifications

Status: MISSING, and unnamed anywhere in the record.

A review loop where the author never learns a comment arrived is not a review loop. Email on comment, on reply, on @mention, on suggestion. **OPEN** — it appears in no plan document, no research area, and no roadmap. Flagging it here is the first time.

N11 — Test-suite hygiene

Status: one line, blocking everything.

Add `.claude/worktrees` to `exclude` in `vitest.config.ts`. Gate: `vitest list --filesOnly` goes 247 → 83. Until this lands, no test result in this repository means anything, and no gate written against test results can be trusted.

N12 — Dependency summary

N11 (test hygiene) — independent, one line, do it first

N1 (identity) — the gate

- └─ N2 (auth + vault binding)
- └─ N3 (share roles) — also needs D3 reopened
  - └─ N4 (comments)
    - └─ N5 (suggestions)
    - └─ N10 (notifications)
- └─ N6 (presence)
- └─ N7 (billing)
- └─ N11 unblocks the gates for all of the above

Independent of N1, run in parallel:

- the frontmatter splice writer (§7.2.5) — ~1 day, fixes a live corruption bug
- N8 (offline), N9 (mobile) — both improve the single-user product too

7.6 Pricing: the founder's sheet, the research recommendation, and three real conflicts

7.6.1 The two models on the table

The research recommendation. `[primary]` `docs/mdmax/PLAN.md` v2.0.0 §4.3, verbatim:

TIER	WHAT	PRICE
Free	browser vault + git sync + publish	£0 — "Free sync. It's just git." Marginal cost ~0
Pro	unlimited documents, live editing, <b>AI credits</b>	<b>\$4/mo annual</b> — exact parity with Obsidian Sync Standard, inside the forum-stated \$2–4 ceiling
Work	same, expensable	<b>\$50/user/year</b> , mirroring Obsidian Commercial
Teams	the review loop, seats	<b>\$5–8/seat/mo</b> — under HackMD Prime at \$5, far under Notion Business at \$20. Not this cycle

with "Owner and editor seats bill. Commenters do not."

Anchors, all [primary] or verified from official pricing pages in the research: Obsidian Sync **\$4/user/mo annual, \$5 monthly** (cut from \$8 in March 2024); Obsidian Publish **\$8/site/mo**; HackMD Prime **\$5/seat/mo billed annually (~\$8 monthly), 3-seat floor = \$15/mo**; Notion Plus \$10/user/mo, Business \$20; Moment.dev Team \$30/mo for 5 users, +\$6/user; forum thread t/26329 (58 posts, 13,451 views, 256 likes) names "maybe \$2–4/month" as acceptable; segment research reports a **\$2–6/mo willingness-to-pay band** for the beachhead and **\$3–5/mo plus a one-time lifetime option (~\$29–39)** for the creator segment.

**The founder's own sheet — Free / Pro / Max.** The founder's tier structure is Free, Pro and Max, and the Max tier's intent is on record verbatim [primary], from HANDOFF~mdmax~markdown~engine~2026~08~01.md §14 line 416:

"our editor on the max plan should give features that is unseen in the industry."

**A sourcing statement that must not be skipped.** The detailed contents of that sheet — a **five-document cap on Free, metered AI credits, and GitHub sync as a Max-tier feature** — are supplied by the brief for this section. **I searched the repository for them and did not find them.** `grep -rn -i '5 document\|five document\|max plan\|max tier\|AI credits'` across docs/, all HANDOFF~\*.md, polyg.yml, cx.html and the research JSON returns no pricing sheet; docs/mdmap/06~product/MAP.md lists [[pricing]] with status "planned" and no content. **The founder's sheet is not in the written record.** Treat the three specifics below as founder-stated and **unsourced in this repository** — the first action is to get the sheet itself into docs/ so it can be cited. That said, all three conflicts are provable from repository sources *on the other side*, which is what makes them worth resolving now.

7.6.2 Conflict 1 — metered AI credits versus "never meter AI on your own keys"

**This one does not need the founder's sheet at all. It is a contradiction between two documents already in this repository.**

[primary] docs/mdmax/PLAN.md v2.0.0 §4.3 line 201 lists "AI credits" as a Pro-tier feature.

[primary] docs/FRONTMATTER~PRODUCT~PLAN.md §5 line 265, verbatim:

**Never:** metered AI credits on your own keys, export paywalls, retention cliffs, per-workspace multiplication, surprise migrations.

And the same document's Pillar 6 specifies **BYO-AI**: "user's own Claude/OpenAI key or subscription; MCP server over the vault — with NO enterprise paywall (Notion's exact sin)", tagged to pain theme T3 (price backlash / subscription fatigue, ranked **third of fourteen** by severity × frequency, and the theme with the demand corpus's most visceral quotes: "Notion AI-metering rage", Evernote Trustpilot ~1.3/5 on hikes).

**They are not actually incompatible, and the reconciliation is one sentence — but it has to be written down.**

**RECOMMENDED, needs a founder yes:** meter the AI **we** pay for; never meter the AI **you** pay for. Pro includes a bundled credit allowance against server-side inference on frontmatter's keys. A user who supplies their own key gets **unlimited, unmetered, on every paid tier, forever**, and that is a headline promise not a footnote.

**Why it matters more than it looks.** The demand corpus's third-ranked pain is priced betrayal, and metered AI is the exact mechanism users name. Getting this wrong does not lose a feature argument; it loses the trust position the whole free tier is built on. **Both documents must be edited to say the same thing.**

7.6.3 Conflict 2 — the five-document free cap versus "unlimited on your own repo"

The founder's sheet caps Free at five documents. [primary] Three repository sources say the opposite:

- docs/FRONTMATTER~PRODUCT~PLAN.md §5: "**Free forever:** full editor, **unlimited vaults on your own GitHub**, sync, version history, export, publish (fair-use), single-user. Costs us ~nothing; it's their repo."
- The same document, Pillar 8: "**Free git-backed sync forever** (user's own repo = backend; **marginal cost ~0**) — undercuts every \$4–10/mo sync add-on"
- docs/mdmax/PLAN.md §4.3: Free = "browser vault + git sync + publish", with Pro selling "unlimited documents" — which **implies a document cap on Free** and is the one place the two models nearly touch

**The economic argument against the cap is unusually strong.** A document cap only makes sense if documents cost money. On the free tier they do not — the bytes live in the user's own GitHub repository. The cap would be pure artificial scarcity on a marginal cost of approximately zero, applied to the exact audience (Obsidian power users with 5,000–20,000-note vaults) the segment research ranks as **beachhead #1**. [primary] That audience "judges in the first minute on their own vault". A five-document ceiling does not merely fail to convert them — it makes the product unevaluable, because they cannot open their vault at all.

**The counter-argument, stated fairly.** A free tier with no ceiling has no natural upgrade trigger. Notion's block cap and Craft's 1,500-block cap exist for that reason.

**RECOMMENDED, needs a founder yes:** no document cap. Gate on **capability**, not on **count**. Free is unlimited documents on your own repository, single-user, with publish under fair use. Pro buys collaboration, hosted convenience, server-side AI and priority sync. This is exactly the "paid tier = genuinely costly things only" line the product plan already commits to, and it preserves the acquisition hook — "Free sync. It's just git" — that the segment strategy identifies as the entire top of funnel.

7.6.4 Conflict 3 — GitHub sync as a Max feature versus "your repo IS the vault"

The founder's sheet places GitHub sync in Max. [primary] The repository record places it in Free, three times over: PLAN.md §4.3 Free = "browser vault + git sync"; PRODUCT~PLAN.md Pillar 8 = "Free git-backed sync forever"; segment strategy line 128 = "free tier = browser vault + git sync (that IS the acquisition hook — 'Free sync. It's just git')".

**And this conflict is deeper than pricing, because it collides with an architecture decision.** Today, GitHub is not a sync feature — it is the **storage substrate**. [measured] There is no other backend: getSnapshot, getFile, getNoteHistory, getNoteVersion, exportVaultZip, commitChanges, createNote, renameNote, mergeNote, uploadAttachment — every use-case in dependency~container.ts is wired to githubVaultReader or githubWriter. **Putting GitHub sync behind Max would mean the free tier has nowhere to store a document.** It would require building a second, hosted storage backend that does not exist, whose marginal cost is not zero, funded by the tier that pays nothing.

There is a fourth conflict hiding underneath this one, and two research areas independently flagged that it is logged nowhere.

- docs/FRONTMATTER-PRODUCT-PLAN.md line 171 sells “user’s own repo = backend”.
- D4 states the durable artifact is plain markdown and explicitly not “your git repo is the backend”.

RESOLUTION, RECOMMENDED and needing a founder yes this week (from the final-gate product-market synthesis, contradiction 5): keep D4 as the durability guarantee — what happens if frontmatter dies — and keep repo-as-backend as the free-tier economics. They conflict only if you conflate the storage substrate with the escape hatch. Write that distinction down. It is the difference between a free tier whose marginal cost is zero and one that has to be funded.

7.6.5 The reconciled model

RECOMMENDED. Every row needs a founder yes. Nothing here is DECIDED.

TIER	WHO	WHAT	PRICE	RATIONALE
Free	one person, own repo	full editor, unlimited documents, git sync, version history, export, publish (fair use), BYO-AI key unmetered	\$0	marginal cost ≈ 0; it is their repository. This is the acquisition hook and the whole top of funnel
Pro	one person	hosted convenience, mobile/offline polish, bundled server-side AI allowance, publish extras, priority sync	\$4/mo billed annually	exact parity with Obsidian Sync Standard; inside the forum-stated \$2–4 ceiling and the segment-measured \$2–6 band
Work	one person, expensable	Pro, on a commercial licence	\$50/user/year	mirrors Obsidian Commercial; aimed at the measured pain “cannot install software on a work machine, can expense a licence”
Max	one person	the founder’s tier. Contents OPEN — see below	OPEN	the intent is on record; the contents are not
Teams	2+ people	the review loop — share roles, comments, suggestions, presence	\$5–8/seat/mo. Owners and editors bill; commenters do not.	under HackMD Prime (\$5 annual / ~\$8 monthly, 3-seat floor); far under Notion Business (\$20). Not this cycle

On Max, stated plainly rather than papered over. The founder’s stated intent for Max is “features that is unseen in the industry”. [measured] The demand evidence does not support novelty as a purchase driver — see §7.7 — and the research recommendation has no Max tier at all, using Work to fill the same slot. This section does not resolve it. It is OPEN, and it needs the founder’s sheet in the repository before it can be resolved. The one thing that should be settled now is negative: Max must not be where GitHub sync or basic document count lives, because those are the free tier’s reason to exist.

The one pricing fact this plan cannot yet state, and should say so. [measured] Across the entire research record — 92 pain points, 89 feature requests, 18 segment agents, 16 research areas, ~2.4M tokens — there are zero user interviews and zero willingness-to-pay probes conducted by this team. Every price above is an anchor to a competitor’s published price, not a measurement of what anyone will pay frontmatter. Five conversations would change that. They are the cheapest, highest-value measurement available and nobody has had them.

7.7 What the founder wants that the evidence says users do not

This subsection exists because the plan is worthless if it only argues its own case. Each item is a founder-stated want, followed by the measurement that contradicts it.

7.7.1 The mother-markdown container and the transclusion tree

The want: one mother markdown file connected to hundreds of markdowns, each connected to hundreds.

The measurement, on corpus\_id sha256:3a010b16..., body-only, frontmatter entity arrays excluded [measured] :

MEASUREMENT	VALUE
wikilinks in the corpus	10,097
resolving to no file in the corpus	72.4%
median out-degree	0
median in-degree	0
files with zero resolvable outbound links	58.9% (638/1,084)
files with zero backlinks	54.6% (592/1,084)
p90 out-degree	5
![[transclusion]] occurrences across 1,084 files and 25.5 MB	45
of those 45, genuine content transclusions	ZERO — all 45 are syntax documentation
size of the "mother file" if built	24.4 MB

The structure does not exist in the founder’s own vault, and the primitive it rests on is used forty-five times in twenty-five megabytes — all of them teaching the feature, none of them using it.

The honest caveat: this is one operator’s vault, which is exactly the weakness the plan’s own W1 flags. A second corpus (~/.claude/skills-src, 474 files, 5,013,124 bytes, a second author) would test it and has not been run. Treat this as strong for this vault and unproven in general.

Verdict: this is the clearest founder-want with no user behind it. The founder-thesis area’s headline was REFUTED — its supporting statistic (37.2% of files already being hand-built bundles) turned out to be 92.3% a machine-generated YAML title-prepend in the md/Skills/\*\* sync tree; the honest figure is ~2.2% of files. Do not build the container.

7.7.2 "Features unseen in the industry" on the Max tier

The want, verbatim [primary] : “our editor on the max plan should give features that is unseen in the industry.”

The measurement. Two independent counts, over different denominators. Both are reported because they disagree in scope, and the disagreement matters.

Count A [measured] — over the **89 sourced feature requests** in docs/research/frontmatter-raw-corpus.json , counting only the feature and why fields (URL fields excluded, because Reddit permalinks inflate "comment"):

```
sync 62 · offline 18 · export 17 · collab 15 · pricing/subscription 14 · mobile 13
wysiwyg 7 · real-time 7 · suggest 5 · multiplayer 3 · presence 2 · vim 1 · graph 0
```

**Request #1 is:** "Always-visible sync status indicator (what synced, what is pending, when last synced)."

Count B [measured] — over **2,559 non-URL string leaves across both demand corpora**:

```
sync 358 · mobile 245 · realtime/collab 120 broad, 108 tight
unambiguous review-loop family (suggestion mode, suggested edit, track changes, annotate, redline) 25
conformance 11 · provenance 5
```

**They use different denominators and are not comparable row-to-row. Do not average them.** What they agree on is the shape: **the top of the list is table stakes, and nothing novel appears at the top.**

**The strongest single piece of evidence against novelty as a strategy** [measured] , re-verified live from api.npmjs.org during the final-gate run:

PACKAGE	DESIGN	DOWNLOADS/WEEK
markdown-it-attrs	degrades to <b>visible junk</b> in other renderers	<b>280,897</b>
markdown-it-decorate	<b>perfect silent invisibility</b> , unshipped since 2017	<b>1,094</b>

**A 257x adoption gap in favour of the objectively worse design.** Novelty is not what buys adoption in this category.

**Verdict:** the Max tier can exist, but it should be *depth* on things users named — offline that genuinely works on a plane, a 10k-note vault that opens in under a second, unmetered BYO-AI, a review loop nobody else has — not *novelty* for its own sake. **The one thing on the roadmap that is genuinely unseen in the industry is the review loop on files you own**, and it is unseen because it is hard, not because it is exotic.

7.7.3 The graph view

**The want:** it is shipped, and it carries a runtime dependency.

**The measurement** [measured] : react-force-graph-2d ^1.29.1 is in package.json (verified). src/modules/graph/ and GraphButton.tsx are built. FEATURE-GAP-REPORT.md lists it under shipped "Rendering & knowledge".

And: across the **89 sourced feature requests the string "graph" appears 0 times**; across the 92 pain points it appears 3 times; in the round-2 corpus ( frontmatter-r2-raw-corpus.json ) the phrase **"graph view" appears 0 times**. Combined with a median node degree of 0, **the graph is decorative in this product.**

**Verdict:** it is already built, so deleting it is not urgent. But it should not receive another hour of work, and it should not appear in marketing.

7.7.4 MDMAX as a compiler programme

**The want:** a compiler with a spec, a format profile, a versioned normalize() , a published benchmark, and a CLI.

**The measurement:** four of six proposed capabilities had **working substitutes executed and measured in a single session** — mdast node.position.offset splices byte-perfectly on 1,080/1,080 files; CodeMirror ChangeSet.mapPos is exact by construction and already a dependency; cross-revision anchoring is a ~30-line LCS diff at 85.0–86.7%; provenance is OKF v0.2 §5 fields, already specified by Google.

docs/mdmax/PLAN.md §10.4 already records the answer: **"No. It is a library subordinate to the editor."**

**Verdict:** settled. This is noted here only because §7.5's dependency graph depends on it — **nothing on the critical path to revenue requires MDMAX to exist as a compiler.**

7.7.5 The uncomfortable summary

Ranked by severity x frequency, the demand corpus's fourteen pain themes put **markdown/frontmatter mangling at #13** and **publishing/sharing friction at #14**. MDMAX serves #13. The container serves nothing measurable. The graph serves nothing measurable.

[primary] The founders' own demand research, written before the engine programme started, ranked the wedges **A > C > B** and said of Wedge A ("*Your markdown vault, in any browser*") : "*Nearly sellable from the existing codebase once multi-tenancy lands.*"

**Sixteen research areas and roughly 5 MB of evidence later, the nearly-sellable thing is still not sellable, for one reason: there is exactly one user.** That is not a research question. It never was.

  The founder's own standard, quoted because it is what makes this subsection publishable [primary] : "*I am absolutely fine with being proven wrong.*"

7.8 What a user notices in the first ten minutes

This is the list ranked by what actually bites, not by build cost. Every item is verified against code.

**Minute 0 — they cannot get in.** [measured] There is no signup route anywhere in src/app . isAllowed() compares their GitHub login to one string that defaults to "sagnikmitra" . **The product admits one person. This is not a soft launch problem; it is a total gate.**

**Minute 1 — there is no home.** [measured] The only application page renders <AppShell /> — a three-pane editor with a file tree. No dashboard, no recents, no templates gallery, no blank-document card. src/app/(workspace)/ is an empty directory with a .gitkeep .

**Minute 2 — saving asks for a commit message.** [measured] CommitBar.tsx:74 holds a message input; /api/commit requires message: z.string().min(1) . **A Google Docs competitor with a commit-message field loses in the first sixty seconds.** Autosave exists — but only to an IndexedDB draft, not to the durable artifact.

**Minute 3 — sharing produces one read-only URL.** [measured] `POST /api/share` accepts `{path, slug}`. No role, no expiry, no password, no per-user grant, **no comment box**. `PublicNoteView.tsx` renders a read-only `<article>` with zero comment affordances. Compare the thing they are switching from: *"Anyone with the link can use your file, without signing in."*

**Minute 4 — publishing silently rewrites their dates, or 502s.** [measured] 84.78% of parseable files get a bare `YYYY-MM-DD` rewritten to an ISO timestamp on publish. 170 of 907 files cannot be published at all. For the beachhead segment — Obsidian power users who *"judge in the first minute on their own vault"* — **one mangled file is rejection**, and the app is named frontmatter.

**Minute 5 — Document History is a list of strings.** [measured] `grep -cin 'diff' HistoryModal.tsx = 0`. There is no diff view anywhere in the product. And because `share-writer.ts` generates its own commit messages, the history a user actually sees contains entries like `share: set public_slug=gear-up-issues on ...`.

**Minute 6 — nothing tells them anyone else is there.** [measured] No presence, no awareness, no cursors, no avatars, no realtime layer of any kind. `grep -rn 'yjs\|automerge\|presence\|awareness' src/ package.json` returns nothing.

**Minute 7 — they cannot comment.** [measured] The Comment rail slot has no component, no route, and no backing collection. `firestore.rules` contains `comments` zero times. **This is the feature the entire pitch is built on.**

**Minute 8 — nothing syncs on its own, and nothing tells them what did.** [measured] `docs/FRONTMATTER-PRODUCT-PLAN.md` §2 lists background auto-sync as deferred; sync today is manual buttons plus a local cron script. And the demand corpus's **#1 feature request** is the always-visible sync-status indicator, which does not exist.

**Minute 9 — the phone.** [inference] 13 responsive prefixes across 74 components and two drawers. Nobody has rendered the application at a mobile viewport. This is unmeasured, not measured-and-fine.

**Minute 10 — import is .md only.** [measured] `ImportModal.tsx:81` accepts `.md`, `.markdown`, `.txt`. A Notion or Evernote refugee has nothing to do here.

**Notice what is not on this list.** Not one of these ten is a markdown-engine problem. Two of them (publish corruption at minute 4, and the frontmatter half of minute 5) are fixed by the splice writer, which is roughly one day of work. The other eight are application engineering, and eight of the ten are downstream of one user table.

---

## 7.9 What would falsify this section, and what could go wrong

Per the plan's own standard: a section that only argues its own case is marketing.

### 7.9.1 What this section does not cover

- **Go-to-market sequencing, channels, launch copy.** Named in the market section; deliberately absent here.
- **The engine's internals.** Splice algorithm, anchor tier design, `normalize()` specification, the certificate format. Those are the MDMAX sections.
- **Native mobile applications.** Out of scope for twelve months. PWA and browser only.
- **Enterprise concerns** — SSO, SCIM, audit logs, data residency, SOC 2. No research exists on any of them in this record.
- **Accessibility.** Not audited. Not measured. Not mentioned in any research area. **This is a genuine hole and someone should own it.**
- **Internationalisation of the product UI** (as distinct from Unicode handling in the engine). Zero coverage.
- **Legal and entity structure, terms of service, privacy policy, data-processing agreements.** Required before taking a paid signup; absent from this plan entirely.
- **Confluence and Slite in §7.3.2.** Both rows are `[UNVERIFIED]`. Slite appears nowhere in the corpus at all.

7.9.2 What would falsify this section

#	FALSIFIER	HOW TO RUN IT	IF IT FIRES
F1	<b>The login wall is not the primary blocker.</b> This section asserts, following the research, that reviewers stall at signup before they stall at anchoring or fidelity.	Five people who have shared a markdown document for review in the last month. One task: get a comment from a non-technical colleague. Observe where they stall.	If they stall on something else, §7.5's ordering is wrong and D3 is not the urgent decision.
F2	<b>Nobody wants comments; they want multiplayer.</b> The demand record is genuinely contradictory here.	The same five conversations, plus a symmetric re-count on a corpus this team did not write.	If it fires, the review-loop wedge is defensibility-led with no demand behind it, and real-time collaboration should move up.
F3	<b>The splice cannot reach 907/907.</b>	Replay the full pinned corpus through publish-then-unpublish under an <b>independent oracle</b> — a byte comparator sharing no code with the writer (LR#60). Baseline to beat: <b>1.87%</b> (17/907), not 3.64%. <b>Restate the gate correctly:</b> publish→unpublish byte-identical on 907/907, AND publish succeeds (no 502) on all 171. "100% byte-identity on publish" is unsatisfiable, since inserting <code>public_slug</code> necessarily changes bytes.	If the splice cannot round-trip the 170 files no YAML parser reads, <b>strike "round-trip-sacred frontmatter" from every document.</b> The app is named frontmatter and the beachhead judges on it in the first minute.
F4	<b>Range anchoring false-matches above ~0.5%.</b>	Sample 2,000 character ranges stratified by length, by whether they contain inline emphasis / code / a link, and by whether they sit in a list item or heading. Mutate by one real git revision. Report correct / FALSE / refusal separately. <b>Hand-audit 200.</b>	Content-derived anchoring cannot be the primary mechanism; fall back to visible orphaning with quote-plus-digest.
F5	<b>The corpus is one person's vault.</b> Every corpus figure here — link topology, transclusion counts, frontmatter failure rates — comes from one operator's files plus this repository.	Run the identical measurements on the second corpus the plan already names: <code>~/claude/skills-src</code> , 474 files, 5,013,124 bytes, a second author. <b>This has not been done.</b>	The graph negative (§7.7.1) is the finding most exposed. A second corpus could overturn it.
F6	<code>.md</code> <b>files are actually broken in Obsidian too.</b> §7.2.5 assumes the 170 unparseable files are valid for Obsidian and only invalid for standard YAML parsers. <b>Nobody launched Obsidian to check.</b>	Open three of the 170 in Obsidian. Confirm tags, aliases and frontmatter wikilinks render.	If Obsidian also fails, "Obsidian round-trip fidelity" needs restating — the vault is already partly broken and the fidelity promise is aimed at a standard nothing meets.
F7	<b>A competitor closes the intersection.</b>	Quarterly: <code>api.github.com</code> stars and <code>pushed_at</code> for <code>inkeep/open-knowLedge</code> ; <code>api.npmjs.org</code> weekly downloads; fetch <code>moment.dev/pricing</code> and <code>grep -ioE 'comment suggest'</code> ; re-fetch <code>hackmd.io/pricing</code> .	If OpenKnowledge ships a teammate model — they already have <code>anchor.ts</code> , <code>passage-match.ts</code> , <code>comment-scrub.ts</code> and a shipped comment UI; only sync and identity are missing — or <code>Moment.dev</code> adds comments, <b>the last unclaimed intersection closes.</b> Three-month risks, not three-year.

7.9.3 How this section could be wrong even if every fact in it is right

**The verifiers are not oracles.** The measured false-kill rate on this programme's verifiers is **0–19%** (plan rule P4). A kill is strong evidence, not proof. Two kills in this section look right to me and I have re-derived both by hand from the underlying numbers — the eemeli-versus-gray-matter accounting asymmetry, and the 18.9%-denominator error. But I did not re-run either measurement.

**A gap analysis that outputs a longer build list makes things worse.** The final-gate `product-gap` area argued against its own premise, and the argument is sound: the biggest measurable defects are in things **already shipped**. Publish fires a destructive frontmatter rewrite on every note. Document History lists commits with no diff. The test suite reports 247 files of which 164 are abandoned worktrees. **The highest-value output of this section is a shorter list, not a longer one** — and §7.5 has twelve items in it, which is a risk this section carries knowingly. If only three ship, they should be **N11** (one line), **the splice writer** (one day), and **N1** (the second user).

**The document itself is a failure mode.** `docs/mdmax/PLAN.md` reached v0.6.0 with a changelog and a supersession chain, and three of four recent commits were cosmetic PDF fixes. **Honesty was being converted into inertia.** This section is long because the brief asked for completeness, and length is not the same as progress.

**The single hardest thing to argue with, restated so the twelve months have a spine:**

Research is closed. If a second login has not written a document by 31 August, this is a library and not a company, and we say so out loud.

58 Market

The market – USP, what competitors lack, what we solve, what we cannot solve

← Index · ← §7 Product · §9 AIOS →

8. The market — USP, what competitors lack, what we solve, what we cannot solve

8.0 How to read this section

This is the complete market and positioning analysis. It answers six questions the founder asked in these words: what is the unique selling proposition against products that already exist; where do those products fall short; how do we solve it; what exact problems do we solve; what exact problems can we **not** solve; and what problems remain unsolved by anybody.

Three vocabulary rules apply throughout, and they are not decoration:

WORD	MEANING
DECIDED	settled. Build against it. Changing it requires re-opening a numbered decision (D1–D10) or a kill gate (the kill-gate table, K1–K7).
RECOMMENDED	the evidence points here, but a competent person could choose otherwise and be defensible.
OPEN	genuinely unresolved. Do not let a plan sentence pretend otherwise.

Every claim carries an evidence tag: `[measured]` we ran it and the number came out of a tool; `[primary]` we read the source document ourselves; `[secondary]` quoted from something that read the source; `[inference]` reasoned, not observed; `[SIMULATED]` replayed through code rather than read from a live system.

**Corpus.** Every figure about local files in this section is measured over the pinned corpus `corpus_id sha256:3a010b1649899795d79274fc528dbece97fdabf4ff0f81cc02ab619c048c51a4` — 1,084 files, 25,548,765 bytes, across three repository roots (`md` at `02c22ec4...`, 756 files; `knowledge` at `464eb666...`, 272 files; `frontmatter` at `798ebbf3...`, 56 files). It resolved on disk with 0 missing files and 0 sha256 drift when two independent readers rebuilt it on 2026-08-01. `[measured]` Two caveats travel with it and must travel with every ratio derived from it: **29 files (2.68%) are byte-identical duplicates across roots** — `frontmatter/docs/*` is mirrored into `md/docs/*` — so every percentage inherits a small double-count; and **the `corpus_id` string itself has no generator script in the repository**, so the label is not reproducible even though all 1,084 individual hashes are. Any figure about local files that does *not* name this corpus id is suspect and is marked as such below.

8.1 The USP, in one paragraph and then in one sentence

**frontmatter is the only product that gives you the Google Docs review loop — comments anchored to text, suggestions, resolve, history, sharing — on markdown files that stay plain `.md` and stay in a repository you own.** Every competitor has one half. HackMD has the review loop and no vault. Moment.dev has the vault and real-time editing and does not mention comments or suggestions anywhere in its marketing. Obsidian has the vault and states in its own documentation that it has neither cursors nor fine-grained permissions. The one product that built the hard technical half — content-derived comment anchoring over markdown — shipped it on 2026-07-30 and declined the teammate model **in writing**, in its own changelog.

The customer-facing sentence is not that paragraph. It is:

“Your vault, in any browser. Nothing to install.”

That sentence is derived verbatim from the demand corpus — from a user in the Obsidian forum's longest-running feature request who cannot install software on a work machine (`forum.obsidian.md/t/obsidian-for-web/2049/246`). `[primary]` The review loop is what we *sell*; browser access is what we *lead with*, because it is the pain the market has been shouting about for six years and it is the one the existing codebase already answers.

**DECIDED.** D1 (frontmatter = Google Docs for markdown) is the product definition. §8.10 explains why the lead sentence and the USP are deliberately different sentences.

8.2 The intersection table — files you own, crossed with the review loop

This is the single most important table in the market analysis. The two axes are:

- Files you own** — the durable artifact is plain `.md`, on disk or in a repository the user controls, not a proprietary document the vendor stores.
- The review loop** — comments anchored to a range of text, suggestions that can be accepted or rejected, and resolve, *between two humans*.

PRODUCT	FILES YOU OWN	THE REVIEW LOOP	EVIDENCE
HackMD	✗ — server-side notes; GitHub push/pull exists but is capped at 20 pushes/month on the free tier	✓ — the whole loop, on the <b>free</b> tier	<code>hackmd.io/pricing</code> comparison table: all four Collaboration rows (Real-time collaboration, Custom note permalink, In-line and page commenting, Suggest edit <code>[New]</code> ) carry the check glyph in <b>all three</b> columns — Free, Prime, Enterprise. Twelve checks. The Free plan's own bullet list independently names "Suggest edit". Prime is \$5/seat/month <b>billed annually</b> with a 3-seat floor (\$15/month total); month-to-month is ~\$8/seat. <code>[primary]</code> , verifier verdict <b>CONFIRMED</b> at the markup level
Moment.dev	✓ — its own docs describe every document as a collection of plain <code>.md</code> files on your disk in git repositories, with history via Jujutsu and git	<b>zero occurrences of "comment" or "suggest"</b>	<code>grep -ioE "comment suggest"</code> over the fetched homepage and pricing page returns <b>0 matches</b> . Pricing verbatim: Free 1 user; Team \$30/month, up to 5 users included, +\$6/month per additional user; features listed are "Real-time collaborative editing / Access controls & permissions". <code>[primary]</code> , verifier verdict <b>CONFIRMED</b> against <code>moment.dev/pricing</code>
Obsidian	✓ — the category benchmark; plain <code>.md</code> on disk	✗ — states plainly it has none	Obsidian's own shared-vault help file: fine-grained permissions are "not supported yet", all collaborators get the vault owner's permissions, live collaborative editing on the same file is not supported and " <i>You will not see the other user's cursor</i> ", the collaborator cap is 20 users, and <b>every collaborator must hold an active Sync subscription</b> at \$4/user/month billed annually (\$5 monthly). <code>[primary]</code> , verifier verdict <b>CONFIRMED</b> — four of four quotes byte-exact
inkeep/OpenKnowledge	✓ — markdown IDE over your own content	<b>Built the hard half, declined the teammate model in writing</b>	See below
Outline	✗ — markdown becomes a projection; the source of truth is a ProseMirror document plus a Yjs CRDT state blob	✓ — complete	<code>server/models/Comment.ts</code> stores data: ProseMirrorData with <code>documentId</code> , <code>parentCommentId</code> , <code>resolvedAt</code> ; <code>shared/editor/marks/Comment.ts</code> makes the anchor a ProseMirror <b>mark</b> carried inside the document model, so re-anchoring never happens. 39,932 stars. <code>[primary]</code>
Craft / Bear	partial	✗	Craft's homepage contains <b>0</b> occurrences of "markdown", "comment" or "collaborat". Bear's contains 7 "markdown", <b>0</b> "comment", <b>0</b> "collaborat". <code>[measured]</code>
frontmatter	✓	<b>← this is the product</b>	

8.2.1 The inkeep/OpenKnowledge row, in full, because it is the whole competitive picture

`inkeep/open-knowledge` — repo description "Beautiful, AI-native markdown IDE and LLM wiki", README feature #1 promising true WYSIWYG so that editing markdown files feels like editing a Google Doc or Notion page — is the closest thing to frontmatter that exists. Measured 2026-08-01: `[measured]`

stars	3,239	(created 2026-06-03, pushed 2026-08-01)
licence	GPL-3.0	
npm @inkeep/open-knowledge	14,790 downloads/week	(week 2026-07-24 .. 2026-07-30)
	859 published versions since 2026-04-17	
	140,859 total downloads 2026-05-01 .. 2026-08-01, rising	
	(weekly buckets 6,862 -> 22,042)	
HN	381 points / 173 comments,	2026-06-25
ships	macOS app, web UI, CLI, MCP server + skills for Claude/Codex/OpenCode, git/GitHub team sharing	
comment anchoring	PR #926 "Comments feature (#2944)" merged 2026-07-30T23:39:04Z	
	npm beta 0.46.0-beta.32 published 2026-08-01T02:38:33.569Z	
	ABSENT from stable 0.45.4 (published 2026-07-31T22:56:04.451Z)	

That last line matters and it was established by a falsifier the original researcher ran against its own headline: both tarballs were downloaded and grepped. Stable 0.45.4 contains **zero** files matching "Add Comment", "Send to AI" or "orphaned"; beta 0.46.0-beta.32 contains all three. `[measured]` So the correct statement is "**reached beta, absent from stable**", not "shipped".

Their anchoring implementation ( `packages/server/src/comments/anchor.ts` , 261 lines, `sha256:9f65280c...` ) is convergent with the block-identity design in the engine section to an uncomfortable degree: the durable record is the exact quoted text plus prefix/suffix context widened at create time until the triple is unique; the stored position is a hint only; it refuses ( `status: 'orphaned'` ) rather than guessing. It cites Hypothesis's 32-character context window by name. `[primary]`

And then, in `.changeset/comments-v1.md` , it declines the thing we are building, in one sentence:

`|` Comments are stored per-machine next to your content and are not committed — a comment here is "*a note to your own agent, not a message to a teammate.*" `[primary]` , verifier verdict **CONFIRMED**, quoted byte-exact from the changeset.

Read that carefully, because it says three separate things.

1. **They validated D2 independently.** Comments live outside the file, in a machine-local sidecar, never committed. We reached that decision by argument; they reached it by building.
2. **They did the hard engineering half and published it.** Content-derived anchoring over markdown is no longer an unclaimed capability. It is claimed, and readable by anyone.
3. **They explicitly declined the easy half — the teammate.** The multiplayer review loop is exactly what remains open.

Two mitigations, both real and both narrow. The code is **GPL-3.0**, so it cannot be copied into a proprietary product by us *or by anyone else without reciprocity*. And their comments are single-player by design. `[primary]`

Two things that cut against us and must be said. First, the competitive sweep that found this **would have missed it**: registry sweeps across six lane keywords over 180 npm packages found nothing, and four of five GitHub topic/phrase searches returned `total_count 0` . The finding came from a six-week-old Show HN post, and the anchoring inside it was only visible after reading a merged pull request's file list. **A competitive watch tuned to package registries and spec repositories is structurally blind to capability shipped inside an application — which is exactly where a "Google Docs for markdown" competitor will always ship.** `[measured]`

Second, "we have the idea" is no longer a position we hold alone, and today we hold **zero** of it in code: `grep -rli mdmax src test` returns nothing, and `src/` is 21,415 lines of product shell. [measured]

**DECIDED.** The intersection — files you own x a human review loop — is the product. **OPEN:** how long the intersection stays empty. §8.8 puts a number on it.

8.2.2 The category is consolidating and dying around us

Context, measured on 2026-08-01: [measured]

- `curl -o /dev/null -w '%{http_code}' https://app.almanac.io` → **503**. `almanac.io` redirects to `get.almanac.io`, whose only internal link is a farewell letter from its CEO. No login, no pricing, no product links.
- `coda.io` carries a banner announcing that Coda is now Superhuman Docs.
- Dendron** — the most technically innovative markdown-native entrant in the category (hierarchical namespaces, local-first, open source, 244 contributors, 7,459 stars) — has **0 commits in 52 weeks**. Its README says active development has ceased. Its founder's stated cause, quoted verbatim in the announcement discussion, was that they *"were ultimately not able to find product market fit for a venture backed business."* Zero technical content in that sentence. [primary]
- Meanwhile the four single-player first movers were **never overtaken**: iA Writer (Mac App Store first release 2013-12-19) shipped 8.0.5 on 2026-07-17; Ulysses shipped 40.2 on 2026-07-13; Bear shipped 2.9.2 on 2026-07-28; Craft shipped 3.5.3 on 2026-07-30. [measured]

The category has produced neither a winner nor a graveyard. It has produced a stable, small, fragmented market in which capability has never decided anything. That is the most important market-history fact in this document and §8.8 builds on it.

8.3 The demand reversal — the single most consequential correction in the corpus

8.3.1 What happened

An earlier research area measured the founders' own demand corpus (two JSON files, 550,131 bytes total: `docs/research/frontmatter-raw-corpus.json` 210,828 B / 1,853 string leaves, and `docs/research/frontmatter-r2-raw-corpus.json` 339,303 B / 1,951 string leaves) and reported:

real-time collaboration 138 string leaves vs commenting/track-changes 20 — **6.9:1** in favour of real-time collaboration.

That number was load-bearing. It was the finding that licensed deferring comments behind Yjs, and it was cited to declare the area's own pre-registered falsifier ("the demand corpus evidences commenting at rough parity with real-time collaboration") **REFUTED**.

**Its own adversarial verifier killed it.** [measured] The instrument was asymmetric: the collaboration side used a broad three-alternative stem — `real[-]time[multiplayer]collaborat` — of which the bare stem `collaborat` alone matched 114 of the 138 hits; the commenting side used a narrow multi-word-phrase whitelist requiring literals like "leave a comment" or "suggestion mode". On a symmetric instrument, de-confounded by dropping URL leaves (Reddit permalinks all contain `/comments/` — 144 of them) and stripping "N pts / N comments" score metadata:

review loop 169 leaves : collaboration 95 leaves — **1.78:1** the OTHER WAY.

The verifier's own words: *"F3 FIRED and was declared REFUTED by an asymmetric instrument."*

8.3.2 The honest complication, stated because a section that only argues its own case is marketing

There is a **third** measurement of the same corpus, from the integrating synthesis lens, and it disagrees with the verifier. It reproduces the verifier's symmetric count almost exactly (1.81:1), then inspects what drives the numerator and finds that the word "comment" in this corpus is overwhelmingly **thread metadata rendered into prose** — "a 60-comment resignation letter", a "614-comment Evernote thread", "a 105-upvote top comment". A strict metadata regex catches 34.2% of the 155 comment-bearing leaves, and 8 of 8 sampled residuals were also metadata. On the unambiguous review-loop family with no possible confound — *suggestion mode, suggested edit, track changes, annotate, redline* — the count is **25 leaves against 108** for realtime/collaboration, i.e. **4.3:1 back in the original direction**. [measured]

A fourth instrument, over the 89 **structured** feature requests (counting only the `feature` and `why` fields, which excludes the URL fields that inflate "comment"), gives: sync 62, offline 18, export 17, collab 15, mobile 13, pricing/subscription 14, wysiwyg 7, real-time 7, suggest 5, multiplayer 3, presence 2, vim 1. [measured]

Four measurements, one corpus. Here is what governs and why.

READING	FIGURE	STATUS
Original area, asymmetric instrument	6.9:1 for collaboration	<b>DEAD.</b> Killed by its own verifier; the asymmetry is demonstrated, not alleged. Do not quote it again.
Verifier, symmetric word-match, URL- and score-de-confounded	<b>1.78:1 for the review loop</b>	<b>GOVERNS as the published figure.</b> It is the only reading produced by an adversarial reader whose job was to attack, and it is the reading recorded in the product section's demand-reversal subsection.
Integrator, strict semantic family excluding the word "comment"	4.3:1 for collaboration	<b>The strongest counter-argument, and it is not refuted.</b> It is a narrower instrument on a cleaner numerator.
Structured feature requests, <code>feature + why</code> fields only	collab-family 27 : suggest 5	consistent with the integrator

**What all four agree on, and this is the durable finding:** (a) **sync is the largest demand signal by a wide margin** — 358 leaves out of 2,559 non-URL leaves, and 62 of the 89 structured feature requests; (b) the review loop and real-time collaboration are within a **small factor** of each other, **not 7:1 apart**; (c) **provenance (5 leaves) and conformance (11 leaves) are near zero**.

8.3.3 What it changes

**DECIDED: build comments, not Yjs, this cycle.** The decision is unchanged; the *justification* changes, and the founders must know which one they are standing on.

- It was justified as **demand-led**. Under the governing 1.78:1 reading it still is. Under the integrator's reading it is not.
- It is justified **regardless** on three grounds that no reading of the corpus disturbs:
  - Defensibility.** The review loop is the empty intersection (§8.2). Real-time collaboration is occupied — Moment.dev ships it on git-backed `.md` today at \$30/month for five seats. [primary]

2. **Architecture cost.** Real-time co-typing fights D7 (splice-only). Outline solves the entire loop by making the comment a ProseMirror mark inside a Yjs document and markdown a *projection* — at which point re-anchoring vanishes as a problem and D4/D6 vanish as guarantees. Refusing that trade is defensible; it must be paid for knowingly. `[primary]`
3. **Buildability.** Comments are a sidecar keyed by a W3C TextQuoteSelector — a Recommendation since February 2017, not an invention. Real-time collaboration is the largest remaining build in the plan.

- **What must change in the prose:** stop citing the review loop as demand-led without also citing the integrator's contamination analysis. The product section states the 1.78:1 correctly; it does not yet carry the caveat. Add it.

**The 6.9:1 figure must never appear in a pitch, a deck, or a commit message again.** It is the clearest instance in this programme of a measurement that flattered the conclusion its own author preferred.

## 8.4 The fourteen pain themes, ranked, and which ones we serve

The demand research yielded **92 sourced pain points, 89 feature requests and a 20-app competitor matrix**, every claim carrying a verbatim quote and a URL, clustered into 14 themes ranked by severity × frequency ( `docs/FRONTMATTER-PRODUCT-PLAN.md §1`; `docs/research/frontmatter-pain-taxonomy.md` ). `[primary]`

**A naming hazard you will hit within an hour.** The taxonomy's theme *identifiers* (T1...T14) are not the same as the *ranks*. Theme **T7** ("Real-time collaboration is impossible in markdown-land") ranks **6th**; theme **T6** ("Offline failures") ranks **7th**. `docs/FRONTMATTER-PRODUCT-PLAN.md` mislabels these against each other in seven consecutive rows — line 116 uses taxonomy IDs correctly while lines 125–142 use rank numbers, so the offline and collaboration citations are swapped inside a single document. `[measured]` **The table below uses RANK. When you read "T6" anywhere in the older docs, check which convention that line is using.**

RANK	THEME (TAXONOMY ID)	SEVERITY, VERBATIM	DO WE SERVE IT?	HOW
1	Sync silently destroys data (T1)	Kills adoption	<b>Partly</b>	3-way merge engine ( <code>merge3.ts</code> , 14 conflict-matrix tests) + <code>baseSha</code> 409 optimistic concurrency are shipped. The background auto-sync engine and the always-visible sync-status indicator are <b>not</b> . See §8.6.
2	Trust collapse / shutdown fear (T2)	Kills adoption	<b>Yes</b>	D4. The durable artifact is plain <code>.md</code> in the user's own repository; whole-vault ZIP export shipped.
3	Price backlash / subscription fatigue (T3)	Churns at exodus scale	<b>Yes</b>	Free git-backed sync — marginal cost ≈ 0 because the user's own repository is the storage. Pro at \$4/mo, exact parity with Obsidian Sync Standard.
4	Slow at scale / mobile startup (T4)	Kills adoption	<b>No, unproven</b>	Server-side index + lazy load is the theory. Nothing is measured at 10k notes. See §8.6.
5	Lock-in / lossy export (T5)	Kills adoption	<b>Yes</b>	There is nothing to export from; the data is already <code>.md</code> in git. Export <code>.md</code> /HTML/PDF/ZIP shipped.
6	<b>Real-time collab impossible in markdown-land (T7)</b>	<b>Kills adoption (teams)</b>	<b>This is the USP</b>	The review loop (§8.5). Real-time co-typing is explicitly <b>not</b> this cycle.
7	Offline failures (T6)	Kills adoption	<b>Deferred</b>	Offline is a Max-tier promise (D4). Not built.
8	Too complex for normal people (T8)	Kills mainstream adoption	<b>No</b>	Live/WYSIWYG mode v1 exists. The bar — a non-technical collaborator editing without knowing markdown exists — is not met.
9	<b>No browser access / platform gaps (T9)</b>	<b>Kills at evaluation</b>	<b>Yes — this is the lead</b>	The product <i>is</i> a URL. Obsidian has no web client at all.
10	Git-as-sync fails non-engineers (T10)	Churns (severe, niche)	<b>Yes, structurally</b>	git runs server-side, so every mobile-git failure mode vanishes by construction.
11	Capture easy, retrieval impossible (T11)	Churns	<b>No</b>	MiniSearch + backlinks + AI suggest-links exist. Resurfacing and dedup do not.
12	Capture friction (T12)	Churns	<b>Partly</b>	A URL is a low-friction capture surface. Sub-500ms open is unmeasured.
13	<b>Markdown/frontmatter mangling (T13)</b>	<b>Churns the purist early adopters</b>	<b>Yes — and today we are a cause of it</b>	§8.5, item 6.
14	Publishing/sharing friction (T14)	Annoys	<b>Yes</b>	<code>public_slug</code> publish shipped; undercuts a resented \$8/site/month add-on.

**Be honest about rank 13.** The engine capability the programme was most excited about — round-trip-sacred frontmatter — serves the **13th of 14** themes. That is a fact and it should stay in the plan.

**But two corrections to the way that fact has been used.** `[measured]`

1. **Its severity has been misquoted downwards.** One research area described T13's severity as "ANNOYS". `docs/FRONTMATTER-PRODUCT-PLAN.md` line 42 says "**Churns the purist early adopters**"; "Annoys" is line 43, which is T14. The label was shifted exactly one band, in the direction that strengthened that area's own recommendation. The correct reading is narrower and more useful: **T13 churns exactly the early-adopter segment a developer tool recruits first.**
2. **The compiler does not map only to rank 13.** The plan maps the engine to the review loop, which is **rank 6, "Kills adoption (teams)"**. An area that mapped it solely to 13 had that mapping killed by its verifier.

**RECOMMENDED framing:** the engine work is justified by rank 6 (it is what makes anchored comments possible) and pays a rank-13 dividend on the way (it stops us corrupting frontmatter). It is not justified by rank 13 alone, and it must never be *marketed* from rank 13 — see §8.9.

## 8.5 What we solve — the explicit list, each with mechanism and evidence

Each item states the pain it answers, the mechanism, the evidence, and its build state.

**1. You cannot open your notes on a machine you do not control.** (rank 9, "Kills at evaluation")

- **Mechanism.** The editor is a web application; the vault is a GitHub repository reached through the GitHub API. Nothing is installed. GitHub.com is reachable on corporate networks where Dropbox is firewalled.

- **Evidence.** `forum.obsidian.md/t/obsidian-for-web/2049` — the forum's most-viewed suggestion: 246,568 views / 244 posts / 928 likes as of 2026-07-13, re-read at **249,616 views** on 2026-08-01. `[primary]` The workaround absurdity proves the intensity: **414,917 Docker pulls** of `linuxserver/obsidian`, a container whose job is to stream an entire Linux desktop over VNC into a browser tab. `[secondary — from the segment study, verified 2026-07-13]`
- **State.** Shipped: CodeMirror 6 editor with edit/reading/split + Live mode v1, the full Obsidian dialect (wikilinks with alias and heading, transclusion, callouts, tags, KaTeX, Mermaid, GFM tables, task toggles), MiniSearch full-text search, command palette, daily notes, PWA + Tauri. Blocked on: **the second user** (§8.6, item 1).

## 2. Git-as-sync destroys non-engineers' work. (rank 10)

- **Mechanism.** `git` runs server-side. There is no `isomorphic-git` on a phone, no pasted personal access token, no terminal. Conservative 3-way merge (`src/modules/repository/domain/merge3.ts`, 14 conflict-matrix tests) plus `baseSha` optimistic concurrency returning 409 on every commit.
- **Evidence.** `obsidian-git` issues #558 ("if you commit the changes, it overwrites any changes from the other device"), #803 (conflict resolution requires a terminal), #819. `obsidian-git` has **2,934,409 cumulative downloads** in `community-plugin-stats.json` as of 2026-08-01. `[measured]` (Note: the field is literally named `downloads`, and it is cumulative across all versions. An earlier draft called these "installs" and derived a "+4,800/day" rate; that rate is **not derivable** — the file carries no time series. Do not repeat it.)
- **State.** Merge engine shipped. The in-app conflict UI and the background sync engine are not.

## 3. Subscription fatigue and sync add-on pricing. (rank 3)

- **Mechanism.** The user's own repository is the storage, so sync has a marginal cost of approximately zero and can be free forever. Paid tiers charge only for things that actually cost money — hosting the review loop, AI credits, seats.
- **Evidence.** Live prices fetched 2026-08-01: `[primary]` Obsidian Sync \$4/user/month billed annually (\$5 monthly); Obsidian Publish \$8/site/month billed annually (\$10 monthly); Obsidian Catalyst \$25 one-time; **Obsidian Commercial \$50 per user per year**; Notion Free \$0 / Plus \$10 / Business \$20 per member/month, then \$10 per 1,000 Notion credits; HackMD Prime \$5/seat/month billed annually, \$15/month total at the 3-seat floor. The forum's own stated acceptable band is "maybe \$2-4/month" (`forum.obsidian.md/t/26329`, 58 posts / 13,451 views / 256 likes).
- **State.** Pricing decided (see the pricing table in the product section, restated at §8.10.5). The **Work licence at \$50/user/year** mirroring Obsidian Commercial is the under-used instrument: it is aimed precisely at the population rank 9 defines — people who cannot install software on a work machine and *can* expense a licence.

## 4. Lock-in, lossy export, and shutdown fear. (ranks 5 and 2)

- **Mechanism.** D4. There is no proprietary format and therefore no export problem. If frontmatter dies, the user still has a working Obsidian vault in a git repository.
- **Evidence.** The demand corpus's most repeated reason switchers stay switched is plain-file ownership; three products in one Product Hunt slice died within about three years.
- **State.** Shipped (export `.md /HTML/PDF/whole-vault ZIP`). **One correction to the marketing:** the "Export Word (.doc)" menu item emits **HTML with a .doc extension** and an `application/msword` content type — a real, twenty-year-old Word import path, not OOXML and not pandoc. An area called this a user-facing falsehood; its verifier disagreed and was right. It is a labelling preference, not a defect. Do not spend a pre-flight slot on it; do not claim "Pandoc formats" either.

## 5. Publishing friction. (rank 14)

- **Mechanism.** `public_slug` in frontmatter → a public URL, with slug-conflict handling.
- **Evidence.** Obsidian Publish at \$8/site/month is called overpriced in a 19-point thread with a 65-point agreeing top comment; a "free publish alternative" forum thread has 15,831 views.
- **State.** Shipped — and it is currently the **cause** of item 6.

## 6. Frontmatter and markdown mangling — and we currently cause it. (rank 13)

- **The defect.** `src/modules/share/infrastructure/share-writer.ts` has a header comment claiming it *splices* the key into the YAML frontmatter. Line 26 is `const next = matter.stringify(parsed.content, data);` — a whole-document re-serialisation of the entire YAML map from a plain JavaScript object. That is a **D7 violation in the only shipped write path**, and it fires on every publish and every unpublish of every note. `[primary]`
- **Measured damage** over the pinned corpus, reproduced independently by a verifier to the digit: `[measured]`

files with frontmatter	907	
byte-identical after a NO-OP write	33	= 3.64%
gray-matter throws outright	171	= 18.85% → POST /api/share returns 502
bytes changed	703	
publish-then-unpublish round trip byte-identical	17 / 736	parsed = 2.31% (17/907 = 1.87%)
a bare YYYY-MM-DD became an ISO timestamp	624 / 736	parsed = 84.78%
re-parsing the mangled output: js data stable	703,	differs 0

The last line is the most decision-relevant number in the whole engine record: **every existing test that asserts on `matter().data` passes green while the bytes on disk are wrong.** `title: 2026-05-26` becomes `title: 2026-05-26T00:00:00.000Z` — a visible title change in Obsidian, a broken Dataview filter, a broken Hugo or Jekyll build.

- **The mechanism that fixes it.** A byte-range splice: locate the byte range of the `public_slug` key (or the insertion point after the last key) and replace only those bytes. Never call `matter.stringify`.
- **Do not take the shortcut.** Swapping in the already-installed `yaml` library's Document API is **not** the fix. Measured on the same corpus: `yaml` leaves 119 of 907 blocks byte-identical (13.12%), refuses 170 outright, and changes 618. And in the publish path a refusal is **not safe** — it is a 502 and an unpublishable note. `[measured]` A contrast of "31.86% safe vs 3.64%" circulated in the research; **its verifier killed it** because it scored `yaml`'s 170 refusals as successes while scoring gray-matter's throws on the *identically same 170 files* as failures. Apples-to-apples the figures are **22.38% vs 31.86%** (file left untouched) or **3.64% vs 13.12%** (published and byte-identical). Never quote 3.64% vs 31.86%.
- **Evidence of demand.** Obsidian's own forum: calling `processFrontMatter` destroys previous formatting and most YAML features; a community plugin exists solely to undo it. `[primary]`

## 7. The review loop — the USP. (rank 6, "Kills adoption (teams)")

- **Mechanism.** Threaded comments and suggestions as **sidecar** threads (D2), keyed by `(noteId, revisionId, quote, prefix, suffix)` — a W3C TextQuoteSelector, a Recommendation since February 2017. Block-scoped: resolve the block first, then the quote inside it. Orphan visibly; never guess.
- **Evidence that block-scoping is the right primitive**, measured over 2,332 simulated comment selections of 3–12 words drawn from 400 randomly sampled real revision pairs: `[measured]`

ANCHORING STRATEGY	CORRECT	REFUSED / GONE	SILENTLY AMBIGUOUS
byte-offset range (what Google's public comment API uses)	65.31%	—	34.69% wrong-or-gone
global quote match (the Hypothes.is exact stage)	85.12%	12.18% gone	<b>2.70%</b>
<b>block-scoped quote match</b>	<b>87.48%</b>	8.45% block-refused + 3.99% quote-gone	<b>0.09%</b>

Block-scoping cuts silent mis-anchoring **30x**, and converts most of the remaining failure into an explicit, safe refusal: of the 197 refusals, 196 were blocks genuinely deleted or rewritten, and exactly **1 of 2,332 (0.04%)** was a resolver failure on a surviving block. **Caveat, stated plainly:** this is one corpus, from one author, and the 99.627% headline anchoring figure quoted elsewhere in this plan **has been re-derived by nobody** and is a *block*-level figure that does not transfer to *ranges*. Treat both as provisional until kill gate K3 closes.

- **Evidence that it is unclaimed.** §8.2, and specifically the OpenKnowledge changeset sentence.
- **The commercial argument nobody else can make.** OpenKnowledge publishes **no** correct/false/ refusal rate anywhere in its source, tests or changeset. Nobody in the competitive sweep publishes one. **A published false-positive rate is a purchasing argument in a way that an algorithm is not** — and it is the part of the differentiator that survives a competitor reading our code.
- **State.** Not built. `firestore.rules` declares users, billing, usage, vaults, notes, revisions, shares and months — and contains the string `comments` **zero** times. `[primary]` The data model does not exist yet.

8. Version history without a 30-day cliff.

- **Mechanism.** `git. /api/vault/history|version|restore` shipped, plus immutable parent-linked revisions already specified in `firestore.rules`.
- **Honest gap.** There is **no diff view**: `grep -cin 'diff' src/modules/editor/presentation/HistoryModal.tsx = 0`, and every `diff` hit in `src/` is inside `merge3.ts::diffRegions`, used for merging and never rendered. `[measured]` The `FEATURE-GAP-REPORT` claims "browse git commits per note, diff, restore" as shipped. It overclaims. With no diff, the history a user sees is a list of commit-message strings — including the literal machine-generated ones this codebase writes, like `share: set public_slug=...` on `...`.

8.6 What we CANNOT solve — the explicit list

This list is longer than the one above and that is correct.

1. Comments cannot survive in the file. DECIDED (D2), and it is not a limitation we are apologising for.

Comments and history live **outside** the `.md`. Export or copy yields clean markdown, latest content only. Nothing about a comment is ever written into the file.

- **Why it is right.** Google's own architecture is the same shape: Drive API v3 (discovery revision 20260728) models a comment as a sidecar with an `anchor` — described as a region of the document represented as a JSON string — plus `quotedFileContent` as the fallback, `resolved`, and a reply `action` of resolve/reopen. `[primary]` OpenKnowledge reached the same design independently. HedgeDoc's issue #657 has been open from 2020 to 2026, doctrinally deadlocked in **both** directions — in-file CriticMarkup was rejected because it pollutes every other pipeline and makes comment-only permission impossible; out-of-file was vetoed as metadata that is not part of the markdown. Users left for HackMD and Google Docs. `[secondary – quoted from this programme's own prior research; re-read issue #657 before using it in a pitch]`
- **What it costs us, concretely.** A comment cannot be read by any other tool. Open the same file in Obsidian, VS Code or `cat` and the conversation is invisible. A collaborator who pulls the git repository gets the text and none of the review. We cannot ship "your comments travel with the file", and we must never imply it.
- **The compensating promise, which must be a shipped feature and not a sentence:** a full export of every thread, in an open format, at any time, plus a documented migration. D4 says it; someone has to build it.

2. Any capability whose payload is BY REFERENCE cannot degrade. This is a hard structural limit and it deletes several "features unseen in the industry" from the possible set.

The whole degradation doctrine (D6) rests on one property: an unknown construct in a `.md` file must still render as *something honest* in a renderer that does not understand it. An unknown fence language round-trips byte-perfectly. An unknown frontmatter key is ignored. That works because the payload is **inline** — it is *in* the bytes.

**It fails completely when the payload is elsewhere.** `![[X]]` in a renderer that does not implement transclusion renders as the literal string `![[X]]`, with the content of X **absent**. Not degraded — absent, and replaced by a piece of syntax noise. The reader gets a five-character token where a section of prose should be.

Therefore the following are **excluded** from the set of things we can ship as degrading-gracefully-everywhere:

CAPABILITY	WHY IT CANNOT DEGRADE
<b>Transclusion / embed</b> ( <code>![[X]]</code> )	the payload is another file. A non-implementing renderer shows the literal reference.
<b>Cross-file reactive state</b> (a value in file A that updates when file B changes)	the payload is a computation over other files. Nothing is in the bytes.
<b>Sidecar-resolved block metadata</b> (a block that carries meaning stored outside the file)	by definition the meaning is not in the file.

Two further measurements make this a cheap loss rather than a painful one: `[measured]`

- `![[transclusion]]` **appears 45 times across 1,084 files and 25.5 MB of the pinned corpus.** The mechanism the "mother markdown" vision was built on is a rounding error in the founder's own vault.
- The graph the vision assumed does not exist: 10,097 wikilinks of which **72.4% resolve to no file**; median out-degree 0 and median in-degree 0; 58.9% of files (638/1,084) have zero resolvable outbound links; 54.6% (592/1,084) have zero backlinks; p90 out-degree is 5.
- **Caveat, stated as the researcher stated it:** this is one operator's vault, which is the exact weakness the plan's own W1 flags. A second corpus could overturn it.

**DECIDED:** D9 — a document *names* a capability, it never *carries* one. This limit is the reason.

### 3. Real-time co-typing fights D7, and the cheapest complete substitute requires abandoning D4/D6.

Splice-only writing (D7) means we never regenerate a document from its abstract syntax tree (Foster et al., TOPLAS 2007, Lemma 3.9). A live CRDT is the opposite discipline: the document model is authoritative and the text is derived from it. Outline resolves the entire review loop by making the comment a ProseMirror mark inside a Yjs document, at which point **re-anchoring never happens** — the anchor is transported by the same edit machinery that moves the text. It is the cheapest and most complete substitute for our whole thesis, and it is open source at 39,932 stars. [primary]

The price of that path is that **markdown becomes a projection rather than the source of truth**, which is exactly the D4/D6 trade. **Refusing it is defensible. It must be paid for knowingly, not by omission.** Real-time collaboration is explicitly **not this cycle** (item 7 of the sequence).

Related architectural evidence, for whoever eventually builds it: [primary]

- The event-graph literature's own asynchronous benchmark traces **are git traces**, and operational transform is catastrophic on them — 3,664,266 ms and 6.76 GB peak on trace A2 versus 23.47 ms for the event-graph merge.
- The chosen engine in an earlier plan ( `diamond-types-node` ) has **73 npm downloads/week** against `yjs` at 7,244,115 and `loro-crdt` at 71,344; its Rust crate has not been published since 2022-08-25. Applying the plan's own adoption test to its own collaboration choice rejects that choice.
- The event-graph paper's own source notes that its central optimisation **does not hold** for Peritext-style concurrent annotations — and comments and formatting marks are exactly Peritext-shaped.

### 4. "Easier for AI" and "improves comprehension" are unproven. Zero live model calls were made in any research run.

Every claim that a projection, a normalisation or a view makes a model perform better is a **prediction**, not a result. Label it that way in every artifact. The one live measurement that exists in the record went the **wrong way**, at **+5.2% tokens**. The "25–168× fewer tokens" promise is an **artifact-size ratio**, not a task-token measurement, and it is additionally contested: one area measured a headings-only index at **14.9×** corpus-wide (11.7× median per file), and its verifier replied that headings-only is the *outline* tier, not the *index* tier, and that a 211–250 bytes-per-document index yields **94.6–112.1×**, inside the published 61–152× band. [measured]  
**Both readings cannot be quoted. Neither should be quoted as a benefit to an AI consumer until a live model call measures the task.**

### 5. Anonymous commenting, as D3 is currently written.

D3 says reviewers must log in (Google or GitHub). Google's own share loop works the other way — its help documentation states plainly that people who are not signed in show up as anonymous animals in your file, and that "anyone with the link" can use a file without signing in. [primary] Our current public surface is strictly weaker than either: `PublicNoteView.tsx` renders a read-only `<article>` with zero comment affordances ( `grep -c comment` = 0), and `POST /api/share` accepts only `{path, slug}` — no role, no expiry, no password, no per-user grant. [measured]

**OPEN (the open decision on D3, anonymous commenters). RECOMMENDED: reopen D3 as a tier split** — anonymous read-and-comment on a share link; login required to resolve or to edit. That preserves the wedge and the account model at once. Owner and editor seats bill; commenters do not.

### 6. We cannot reach non-technical normals. Explicitly out of scope.

The segment study's own words: this segment is not reachable by frontmatter v1. GitHub sign-in is fatal; any visible markdown or git vocabulary is instant abandonment; the requirement is phone-first native-grade real-time. **DECIDED: ignore indefinitely.** Revisit only when WYSIWYG, an Apple/Google signup with an invisible repository, and flawless mobile real-time all exist.

### 7. We do not yet solve rank 1 — sync that provably never loses data — and we must not claim we do.

The merge engine is real; the trust surface is not. The background auto-sync engine is deferred. There is no always-visible sync-status indicator, which the corpus names as its most concrete trust ask. And the single largest first-ten-minutes defect is not a missing feature at all: **saving is a manual git commit with a message box**. `CommitBar.tsx:74` holds a `message` state, line 98 falls back to a default, and `/api/commit` requires `message: z.string().min(1)`. [measured] **A "Google Docs for markdown" that asks you to write a commit message loses in the first sixty seconds.**

### 8. Performance at scale, and mobile, are unproven.

`src/` carries **13** Tailwind responsive prefixes ( `sm:|md:|lg:|xl:` ) across 74 `.tsx` files, plus 3 `@media` rules in `globals.css` and 1 in `public-note.css`; two real drawers exist in `VaultWorkspace.tsx`. [measured, tagged inference by its own author: a prefix count is a proxy for layout quality, not a measurement of it — the app was never rendered at mobile widths] No cold-start or search latency has been measured on a 10k-note vault. Rank 4 is a "Kills adoption" theme and we have no evidence on it either way.

**9. Retrieval at scale (rank 11), capture friction (rank 12), and "too complex for normal people" (rank 8) are not addressed by anything in this plan.** They are real, ranked above 13, and out of scope this cycle. Say so rather than implying coverage.

**10. Writers and editors — the segment that most wants "track changes for markdown" — cannot be served without a `.docx` round trip we do not have.** Their counterparty dictates the format. We cannot change that.

**11. We cannot change markdown, and we would not want to.** CommonMark's last three releases (0.31.0, 0.31.1, 0.31.2) all landed on 2024-01-28 and there have been none since; `cmark-gfm`'s last release was 2023-07-21, still pinned to CommonMark 0.29 from 2019. [primary] GitHub renders the world's markdown against a seven-year-old specification. **D6 makes that a constraint we exploit rather than a problem we solve.** The format war is over and getting more over: `@djot/djot` was at 693 downloads/week when the plan first cited it and is at **485** now, against `marked` 61,032,058 and `remark-parse` 45,820,243. [measured]

**12. We cannot stop a closed renderer changing under us.** `github/markup` shipped v5.0.0 and v5.0.1 on 2024-06-17 and **v6.0.0 on 2026-05-05**. [measured] Every closed-target row in a degradation certificate has a shelf life; a competitor who re-measures once is level in a week. That is why certificate rows must carry an `observed_at` and expire rather than silently claim PASS.

## 8.7 What problems remain — the ones nobody solves, ours or theirs

These are gaps in the *market*, not in our roadmap. Some are opportunities; some are warnings; all are unfilled as of 2026-08-01.

**1. Suggestions — accept/reject over plain markdown, with a clean-export guarantee.** Nobody ships it. Every Obsidian commenting or review plugin either writes into the `.md` ( `review-comments` 410 downloads, `review-critic` 200, `tandem-comments` 814, `document-comments` 1,318) or is PDF/EPUB annotation. HackMD has "Suggest edit" but no vault. The demand corpus says it verbatim: no track changes or suggestion mode in any markdown or writing-focused tool. [measured]

**2. A comment anchor that survives the markdown-body ↔ rendered-text boundary in a mode-switching editor.** This is a problem our own plan had not named and which we hit on day one, because the mockups ship Live/Edit/Split/Read *plus* a comment rail. An anchor measured against the markdown **body** cannot be applied to **rendered** editor text, and converting between them is unsound — serialising a partial ProseMirror selection fabricates the block marker of whatever block the selection happens to sit in, so a pick starting mid-bullet comes back with the wrong prefix. OpenKnowledge had to write a third matcher with syntax-elastic matching plus a link-tail skip, because otherwise a passage containing any link could not be matched at all — and in a linked wiki that is most of the interesting passages. **Their solution is GPL-3.0: a design reference, not a dependency.** No npm package solves it; `@codemirror/collab` (6.1.1) addresses operational transform, not content-derived anchoring. [primary]

**3. A published false-positive rate for markdown re-anchoring.** Nobody publishes one. This is the slot that survives the OpenKnowledge finding and it is worth more than the algorithm.

**4. A wikilink-tolerant YAML frontmatter parser.** `related: [[a]], [[b]]` is the Obsidian idiom and it is not valid YAML — YAML reads `[[a]]` as a closed nested sequence. Measured on the pinned corpus: **170 files fail to parse under** `gray-matter / js-yaml`, and `eemeli/yaml` **parses 0 of those same 170** (the set overlap was measured; the failing sets are identical). One further file, `md/pj.md`, parses cleanly and throws at stringify — so 171 path failures is **two** failure modes, not one. Rates by root: **13.75% (91/662) of the Obsidian vault**, 35.59% (79/222) of the knowledge repo, 0% of the frontmatter repo, **18.74% (170/907) across the pinned corpus**. [measured] *A widely-circulated "18.9% of a real Obsidian vault" figure attaches the whole-corpus rate to the vault denominator; say which number you mean.* Obsidian itself accepts these files. **Anyone claiming Obsidian round-trip fidelity needs a parser nobody publishes**, and our read path currently degrades **silently** — `markdown-parser.ts` catches and sets `fm = {}`, so tags, aliases and frontmatter wikilinks vanish from search, the tag pane and the graph with no user-visible signal. [primary]

**5. Container round trip with edit mapping** — edit a DOCX- or PDF-derived document as markdown and write the change back into the original container at the right offsets. The largest genuinely open slot found in the sweep. The converter industry is enormous and **one-directional**: `microsoft/markitdown` 170,545 stars, `docling` 64,082, `marker-pdf` 2.0.0 released 2026-07-20, `mineru` 3.4.4 released 2026-07-30 — **not one of them writes back**. GitHub searches for the round trip return `total_count 0` and 2 trivial repositories. [measured]

**6. Cache-aware document layout** — ordering and chunking a document set so a prompt-cache prefix stays stable across edits. Effectively unclaimed at any level of adoption: the category leader, `repomix`, has a `--token-budget` guard and per-glob inclusion levels, but its only file ordering is a size-descending sort for **worker load balancing**, an internal scheduling concern. [primary]

**7. Near-miss reference checking** — a link that *almost* resolves, pointing at a renamed heading or a filename one character off. Closest adjacent art finds near-duplicate pages, not near-miss references.

**8. Onboarding a non-technical collaborator without an account.** Google solved it with anonymous animals. Everyone in the markdown category either requires an account (us, under D3 as written; Obsidian, which requires a paid Sync subscription per collaborator) or has no collaborators at all. This is the gap most likely to decide whether the review loop is used or admired.

**A warning attached to this whole subsection.** GitHub **code search** was unavailable to the researcher who produced most of these gaps — `search/code` returned `total_count 0` even for queries known to match. Every "nobody has built X" here therefore rests on **repository and topic search plus npm/PyPI/crates search**. Read each as "*not found by repository and registry search*", never as "*proven absent*". The measured false-kill rate of the verifiers in this programme is 0–19%, so a kill is strong evidence, not proof.

---

## 8.8 Defensibility — separating a LEAD from a MOAT, capability by capability

Definitions, because the two words have been used interchangeably in this programme and they are not the same thing.

- A **LEAD** is time. It is measured in weeks or months, it decays whether or not you do anything, and it can be closed by any competent team that decides to spend the weeks.
- A **MOAT** is structural. It gets *harder* to cross as time passes — because a dataset compounds, because switching costs accumulate, or because a network effect exists per document.

The headline finding, and it is uncomfortable: five of six engine capabilities are LEADS measured in weeks. That research headline was itself judged **OVERSTATED** by its verifier, but the corrections made it worse for us, not better. [measured]

CAPABILITY	CLONE TIME FOR A COMPETENT TEAM	WHAT THEY NEED THAT IS HARD TO GET	TREND OVER TIME	VERDICT
Block / range re-anchoring	~1 weekend to 2 weeks	Nothing. Two public reference implementations now exist: Hypothes.is's own <code>match-quote.ts</code> (163 lines, MIT) over <code>approx-string-match@2.0.0</code> , and OpenKnowledge's <code>anchor.ts</code> (261 lines, GPL-3.0). The design is a W3C standard (TextQuoteSelector) with prior art to Microsoft Research MSR-TR-2001-107.	Easier. There was one readable reference; now there are two.	LEAD, weeks
Splice-only writing	~1 day	Nothing. <code>mdast-util-from-markdown</code> — already a transitive dependency — hands you <code>node.position.offset</code> . The whole writer is four lines and is byte-identical on <b>1,080 of 1,080</b> parseable files.	flat	Not even a lead — table stakes we currently fail
Budgeted projection	hours	Nothing. Taken by <code>llms.txt</code> in September 2024: <code>AnswerDotAI/llms-txt</code> , 2,541 stars, created 2024-09-01, mentioned in 1,659 repositories.	taken	TAKEN
Declared provenance	—	Taken twice. Google's OKF SPEC.md v0.2 (37,544 bytes, <code>GoogleCloudPlatform/knowledge-catalog</code> , 8,134 stars) makes provenance, trust, lifecycle and attestation first-class in markdown + YAML frontmatter. C2PA has had a public specification since 2021-11-12.	taken	TAKEN — with one narrow survivor, below
Near-miss reference checking	~1 weekend	Nothing found	flat	LEAD, weeks
The (product, surface) degradation certificate	a weekend for the code; the dataset is the asset	A continuously re-measured observation series against <b>closed</b> renderers, with dated snapshots. That is operational discipline, not an algorithm.	Compounds slowly — <code>github/markup</code> v5.0.1 (2024-06-17) → v6.0.0 (2026-05-05) proves the target moves	WEAK MOAT — months of freshness
The review loop over files you own	a quarter or more for a team that already has an editor	(a) a second-user system with enforced roles; (b) an anchor with a <b>published</b> false-match rate; (c) a clean-export guarantee anyone can verify	Harder, but only after documents have two people on them	LEAD until the first shared document; then switching costs
Multi-tenancy / accounts / share links	weeks	Nothing	flat	A GATE, not a moat
Reputation: "has never corrupted a byte of your file"	cannot be cloned; must be earned	A public, continuously-running byte-fidelity CI over a <b>public</b> corpus, green for a long time	Compounds — and is destroyed by one incident	The only asset a weekend cannot copy

8.8.1 Corrections that the research's own verifier forced, and which change the conclusions

These matter because the first draft of the defensibility case was wrong in specific, checkable ways.

- `dom-anchor-text-quote` is NOT in production at Hypothes.is. The claim that our core mechanism is "182 lines of code shipping in production today" is false against the primary source: `hypothesis/client`'s `package.json` depends on `approx-string-match@2.0.0`, with no `dom-anchor-text-quote` and no `diff-match-patch`; that package was last published 2017-02-10. And reading all 182 lines shows it never performs an exact match, never enumerates candidates, never scores them and has no AMBIGUOUS return — it is location-hinted fuzzy bitap, i.e. exactly the position-biased family our own measurements put at 22.20% false. **Consequence: do not vendor it.** The true reference implementation is `hypothesis/client`'s own 163-line `src/annotator/anchoring/match-quote.ts` — exact-first over all occurrences, then approximate search, then a weighted score (quote 50 / prefix 20 / suffix 20 / position 2, position explicitly commented as a tie-breaker). *It still has no AMBIGUOUS state: it sorts by score and returns the top match unconditionally.* **That absence is our differentiator.** [primary]
- "No prior art exists for the degradation certificate" is REFUTED by one query the researcher did not run. `ArchieCur/MARKDOWN_FLAVORS` is a 14-flavour × 17-construct matrix shipping a CSV and a compatibility PDF; `babelmark` exists at 115 stars; 31 repositories sit under `markdown conformance`; and `mattcone/markdown-guide` (4,090 stars, created 2017-01-31, pushed 2026-07-21) ships **70 tools × 28 constructs** with `y/partial/no/unknown` verdicts. [primary] **What survives is narrower and still real:** those 28 tracked construct ids contain **zero** of frontmatter, wikilink, callout, mermaid, math, directive, attribute or embed — while **90.5–90.8%** of the pinned corpus carries at least one of them — and the 70 tracked tools include **no** `github.md`, `gitlab.md`, `confluence.md`, `teams.md`, `discourse.md` or `jira.md`. [measured] The novel slice is therefore **(product, surface) scoping + executable PASS/STRIP/CORRUPT + expiry**, not "a compatibility matrix".
- OKF does not take all of provenance. `grep -ic confidence` over the 37,544-byte specification returns **0**, and §5.1 states that it deliberately does not store a credibility score. So document-level provenance is taken; **per-span source/method/confidence is not**. Emitting OKF's fields verbatim would silently descope that. Do it deliberately or not at all. [primary]
- "The only durable moat is multi-tenancy" was killed. It carried zero evidence across ten claims and — as the verifier put it — accounts, workspaces and role-enforced share links are the most commoditized layer in SaaS. **Multi-tenancy is the gate on everything (K7) and is worth doing first for that reason. It is not a moat. Do not sell it as one.**
- The corpus is 65 days old, not 80. First commit 2026-05-13, last 2026-07-17, verified with `date -j`. An 18.8% overstatement in the flattering direction. And the "1,000× the bytes" claim for public corpora compares GitHub's `size` field — repository disk usage in KB including packed history and binaries — against 25,548,765 bytes of plain markdown. The arithmetic is right; the units are not. [measured] **There is no corpus moat.** `MicrosoftDocs/azure-docs` (created 2016-11-02), `mdn/content` (over 14,000 pages) and `kubernetes/website` are one `git clone` away and give a competitor a larger, older, **multi-author** anchoring corpus for free, against our 416 commits by exactly one author.

8.8.2 How fast a clone actually arrives — the measured base rate

`scaccogatto/okf-skills` was created **2026-06-14T11:15:31Z**, roughly **one day** after Google Cloud's OKF announcement of 2026-06-13. It now has 206 stars, 22 forks and 32 commits. A GitHub repository search for the exact phrase "Open Knowledge Format" returns `total_count` **287**, windowed by creation date: [measured]

2026-06-13 .. 2026-06-20	69 repos	(the first eight days)
2026-06-21 .. 2026-06-30	66	
2026-07-01 .. 2026-07-15	89	
2026-07-16 .. 2026-08-01	59	

And the part nobody wrote down until the verifier found it: the same eight days produced **product** competitors, not just convention wrappers — `activetwist/OnyxWriter`, a local-first editor for OKF bundles, and `pothos-dev/sunstone`, a lightweight markdown editor with first-class OKF support. [measured]

Empirical clone latency for a published convention is under 24 hours, and the ecosystem produces roughly 70 independent implementations in the first week — including direct product competitors. Plan the public artifacts accordingly: **publish the certificate's dataset before, or instead of, the certificate's design**. The design is a weekend. The dated observation series is not.

### 8.8.3 Answering the Obsidian argument

The strongest argument against everything in this plan is: *the best-resourced actor in the ecosystem faced exactly these gaps, and its answer was to ship sidecar files — twice. Why would our answer be different?*

The premise is true and must be conceded. [primary]

- **JSON Canvas** (`obsidianmd/jsoncanvas`, created 2024-02-28, 3,632 stars, spec 1.0 dated 2024-03-11) is a new file format for infinite-canvas data. That is a true sidecar for something markdown structurally cannot hold — **spatial arrangement** — and this plan already concedes it was the right call.
- **Bases** is Obsidian's core answer to typed relations and queries (i.e. to Dataview, at 4,669,090 downloads). It ships a `.base` file.

But the second example cuts the other way, and this is the answer. Obsidian's own documentation states that **all the data in Bases is stored in your Markdown files and their properties**; what lives in the `.base` file is the **view definition** — and that view definition can *alternatively* be embedded in a fenced ````base` code block inside a `.md` file. [primary]

That is **decision D8 — extensibility lives in the VALUE of a field, never in the SET of node types — validated by the best-resourced actor in the ecosystem**. Obsidian shipped exactly **one** true sidecar, for the one thing markdown genuinely cannot carry, and kept everything else in the markdown files.

A third Obsidian datapoint, and it is the one that most directly competes with us. Obsidian's answer to block identity was to write a **declared id into** the markdown — `[[2023-01-01#^37066d]]` — the exact opposite of a derived anchor, shipped since 2020 and autocompleted on a keystroke. Measured on the pinned corpus: **0** `^blockid` declarations across 1,084 files, and **0** headings carrying a `{#id}` attribute. [measured] (*The denominator for headings does not reproduce: a widely-quoted "31,630 ATX headings" comes out at 28,194 under an mdast parse, 28,158 under a regex excluding fenced code and 31,983 including it. The numerator — zero — reproduces under every definition, so the finding survives and the ratio does not.*)

That zero cuts both ways and both cuts matter.

- **For us:** the "a competitor just declares ids and wins" threat has **no observed adoption** whatsoever.
- **Against us:** a mechanism the market leader has shipped for six years, and which costs one keystroke, was used **zero** times in a 756-file vault. **That is evidence that authors do not feel block-identity pain. Only the app feels it.** Which is precisely why block identity is infrastructure for the review loop, and must never be marketed as a feature.

And the market history is the closer. The category winner was **not first** — Obsidian launched in 2020, after Ulysses, iA Writer, Bear and Roam, and simultaneously with Dendron — and its fastest-moving repository is its **plugin registry**: `obsidian-releases`, 20,328 stars, 1,934 commits in 52 weeks with **1,181 of them in the last 13** (a 3.5x acceleration) in a repository whose only job is accepting third-party plugins. [measured] **The winner won on a distribution channel and a business model, not on format capability. Nothing in this section should be read as promising otherwise.**

### 8.8.4 The honest summary of defensibility

RECOMMENDED, stated plainly so nobody has to infer it:

1. There is **one weak moat** (the certificate's decaying dataset, worth months of freshness, and only while continuously re-measured), **one switching-cost moat that does not exist yet** (shared documents with threads), and **one brand asset** ("has never corrupted a byte", earned by a public green CI over a public corpus).
2. Everything else is a lead of days to weeks.
3. **Today the lead is zero**, because there is no engine code at all.
4. Therefore the defensible plan is **not** "build a better algorithm". It is: get to two users, get documents with threads in them, publish a false-match rate nobody else publishes, and never corrupt a byte.

## 8.9 The give-away half, treated as a distribution budget

The founder's constraint is that helping is 50% of the point. **DECIDED: treat that as a distribution budget, not a sentiment — and recognise that half of it is already spent.**

**Free git-backed sync IS the give-away.** The user's own repository is the backend, marginal cost is approximately zero, and it undercuts every \$4–10/month sync add-on in the category. That is the expensive half of the 50%, and it is already committed in the pricing table.

**DECIDED: cap the second give-away at one artifact, two days, one channel, one kill number.**

- **Artifact:** `mdmax cert` (specified in the engine section) — and nothing else. Not a conformance suite *and* a hostile-input corpus *and* a benchmark *and* a matrix. One.
- **Two days.** A working five-configuration prototype was built in a single script against packages **already in this repository** with no `npm install`, and an independent verifier rebuilt it from scratch and reproduced every headline verdict. [measured] The cost is the **target harness**, not the code. If it balloons past two days, the cost has moved to the targets and the cap has been hit.
- **One channel.** See §8.10.
- **One kill number:** K5 — fewer than 5 of 30 probed users say they would install it → cut it.

### 8.9.1 The base rate, given honestly

Of the free artifacts checked, **2 of 9** produced an organic commercial entity for the giver — and "base rate" is the wrong term for nine self-selected repositories, so read it as a sample, not a population. [measured] (*An earlier draft said "2 of 11"; its own evidence enumerated nine. 2/9 = 22.2%, not 18.2%.*)

- **markdownlint** — the artifact closest in kind, a free markdown quality tool by a solo maintainer — repository created **2015-03-15**, i.e. **11 years 4 months**, now at **2,886,431 npm downloads/week**, with a `FUNDING.yml` that reads exactly `github: DavidAnson`. **Zero dollars.**
- **mermaid** — the best outcome in the set — npm package created 2014-12-02, commercial organisation Mermaid-Chart created 2022-11-07. **95 months = 7 years 11 months.** And its monetisation is enterprise placement (Jira, Confluence, IntelliJ, VS Code plugins), not selling the free thing.

- **No commercial entity:** prettier, `DavidAnson/markdownlint`, editorconfig, semver, `commonmark/commonmark-spec`, repomix, `AnswerDotAI/llms-txt` .

Two founders with no README, no public repository and zero users cannot spend an eight-year option.

8.9.2 The revealed-demand warning, and a unit contradiction you must resolve before quoting it

`curran/llm-code-format` v3.1.0 (published 2025-09-20, zero dependencies, TypeScript) already ships the exact round trip this programme called unclaimed — source to markdown to LLM and back — exposing `parseMarkdownFiles` and `formatMarkdownFiles`, a streaming parser, and recognition of nine LLM output conventions.

`llm-code-format`: 387 downloads/month. repomix: 327,543. A ratio of 846:1 against the round trip. `[measured]`

**Contradiction, named as required.** Two later restatements of this figure in the research record say "downloads/**week**". The original measurement says "downloads/**month**", and monthly is the reading that is arithmetically consistent with the independently verified `repomix` figure of **75,342 downloads/week** (75,342 × 4.35 ≈ 327,738). **The monthly reading governs.** The ratio — which is the load-bearing part — is unaffected either way.

Corroborating: `repomix` issue #71 (file splitting) has been open **23 months** and its own effectiveness check failed in the contributor's words; issue #226 (unpack) has **0** reactions after 19 months. **The round trip is not unclaimed. It is claimed and unwanted.**

8.9.3 What must NOT be given away, and the three ways the give-away can go wrong

**Do not publish the anchoring benchmark before kill gate K3 closes.** Nobody has re-derived the 99.627% figure; it is a block-level number that does not transfer to ranges; K2/K3 commit us to publishing whatever comes out including a worse number; and there is no prior benchmark to be compared against, so publishing is pure downside with no category to win. `[measured]`

Three ways this backfires, all real:

1. **Publishing a matrix that mostly shows AGREEMENT markets against the thesis.** A five-configuration replication found disagreement in only 2 of 15 constructs; an independent rebuild got 3 of 15. Both samples are JavaScript CommonMark-descended libraries, which is *one language's corner of one ecosystem*, and the wider 24-configuration finding — the HTML comment alone producing 8 distinct outputs, with GitHub's blob view deleting it — shows the real spread lives at the product/surface layer. **The differentiating cost is the target corpus and the non-JavaScript engines, not the code.**
2. **A published machine-readable (product, surface) × construct matrix is a specification for the Max tier's differentiator, given away free** to HackMD, GitHub, Obsidian and every agent-tool author in a directory that currently holds 12,804 repositories under `topic:agent-skills`.
3. **Producing a public good is the failure mode the "what we got wrong" list already diagnosed — "the document became the product" — wearing a growth hat.** Four artifacts that can be *written* rather than *shipped* are indistinguishable from the disease at the moment of authoring them. The standing rule in "how we work now" governs: no further research fan-out until one committed assertion has gone red to green against engine code.

8.9.4 One demo correction that would otherwise sink the launch

The proposed 30-second demo was "drop in your own README and see what breaks". **Its verifier refuted the transfer claim directly:** it fetched 40 READMEs from major repositories via `api.github.com` — including the markdown ecosystem's own ( `marked`, `markdown-it`, `remark`, `unified`, `mermaid`, `pandoc`, `commonmark-spec`, `markdownlint`, `hugo`, `jekyll`, `docusaurus`, `mkdocs-material` ) — and found **0 with YAML frontmatter and 0 with a wikilink** (all three `[[` hits were false positives on link syntax and a JavaScript array). `[measured]`

The 83.67% / 56.75% frontmatter-prevalence rates come from two populations where frontmatter is **structurally mandated** — an Obsidian vault (the properties UI writes it) and Claude Agent Skill files (the format requires it) — so they do not transfer to a README.

**DECIDED: the demo input is "your note or vault file", not "your README".** A visitor dropping in a README would see a tool whose pitch is "see exactly what breaks" report that nothing breaks. That is the worst possible first impression.

8.10 Positioning and distribution

8.10.1 The one sentence, and why it is not the USP sentence

AUDIENCE	SENTENCE
The product, to a user	"Your vault, in any browser. Nothing to install."
The product, to an investor or a partner	"Google Docs for markdown, as simple as that." (the founder's own words)
The free tool	"Paste a markdown file. See exactly what breaks on GitHub, in Obsidian, and in an LLM's context — before you publish it."

The lead sentence targets ranks **9 and 1**, not rank 13. The USP sentence describes rank 6. They are different sentences on purpose: the review loop is what we *sell* and what defends us, but browser access is the pain the market has been shouting about for six years and the one the existing codebase already answers.

**A hard constraint on the wording.** Obsidian's community Code of Conduct requires an "integrates-with-your-vault" framing rather than an "Obsidian competitor" framing, and the norm in its forums is to participate more than you promote. **The only admissible framing in the largest watering hole is: *your Obsidian vault, editable anywhere on the web — plain markdown in, plain markdown out.*** Breaking that gets the launch removed, not criticised.

8.10.2 Who it is for

**DECIDED — two beachheads, everyone else explicitly ignored with a written revisit trigger.**

**Beachhead #1 — Obsidian power users, specifically the blocked-at-work and git-vault sub-segments.** Largest verified pain signal in the corpus; cheapest reach (the exact askers are enumerable); exact product-shape match with zero missing features for the core job. Willingness to pay is **moderate**, band **\$2–6/month**. Honest caveat: this segment skews free-first — treat it as an acquisition and evangelism engine more than a revenue engine.

**Beachhead #2 — content creators and git-blog publishers.** Second-best fit today because publish already ships; the sharpest daily-loop pain; heavy overlap with beachhead #1 (same vault, same repository, same forums — one go-to-market motion serves both); and **every converted user publishes a public site that markets the product.** `digitalgarden` at 84,873 downloads is the direct intent proxy. `[measured]`

SEGMENT	STATUS	REVISIT TRIGGER
Non-technical normals	Ignore indefinitely	WYSIWYG + Apple/Google signup with an invisible repository + native-grade mobile real-time, all shipped
Writers & editors	Ignore	Suggestion mode with accept/reject + no-login commenter links + clean <code>.docx</code> export ship
Students & academics	Ignore as revenue	Collaboration ships, plus citations (Zotero/BibTeX) and equation numbering; then a <code>.edu</code> free tier as a growth loop
Small teams / startups	Ignore	Real-time co-editing <b>and</b> inline comments ship — highest ARPU in the corpus
Developers / docs-as-code	Seed awareness only	Same trigger. <b>This is the per-seat revenue engine.</b> Cautionary: Dendron failed selling to <i>solo</i> developers; the money is teams
Notion / Evernote refugees	Ignore	WYSIWYG + a browser-based importer with a verification report ("536 in, 536 out"); then ambush the next incumbent price-hike news cycle

### 8.10.3 The proof point in the first thirty seconds

The demo is the visitor's own vault, not a marketing page. In order:

- Open their repository in the browser.** No install, no server, no VNC.
- Their own notes render correctly** — wikilinks, frontmatter properties, callouts, transclusion, KaTeX, Mermaid. The segment study is explicit that they judge in the first minute on their own vault, and any dialect breakage means instant rejection as "just another markdown editor".
- Edit one line and show the diff is clean** — one line changed, frontmatter untouched, key order preserved, YAML comments intact.

**Step 3 is the whole engine's public face, and today it fails.** Until the splice writer lands, a publish rewrites 84.78% of parseable files' dates and leaves only 1.87% byte-identical across a publish/unpublish round trip. **The demo cannot be given before \$8.5 item 6 is fixed.** That is the strongest possible argument for sequencing the splice writer into the first week.

### 8.10.4 How a markdown tool actually gets discovered in 2026

**RECOMMENDED, in this order.** The evidence says the agent-tool surface has replaced Show HN as the distribution channel for a markdown CLI, and that the product and the tool have different channels.

- The exact threads where the askers already are.** `forum.obsidian.md/t/obsidian-for-web/2049` (244 posters, the warmest leads on the internet), the Share & showcase category, and the `obsidian-git` issue threads. First move: a working demo link and a 60-second recording, framed as a workflow post.
- A companion Obsidian plugin**, submitted as a pull request to `obsidianmd/obsidian-releases`, then announced in the sanctioned Discord channel. **Precedent: Relay, a commercial collaboration service, distributed exactly this way and reached 176,247 cumulative downloads.** `[measured]` *(The field is downloads, cumulative across versions — not "installs")* Comparable: Self-hosted LiveSync 829,285; Readwise 238,596; digitalgarden 84,873.
- The agent-tool directories, for mdmax cert only.** Ship it three ways from one binary, because `repomix` has already proven the packaging: `npx mdmax cert <file>; --mcp` so an agent can call it mid-edit; `--skill-generate` producing a Claude Agent Skill; plus a GitHub Action that fails a pull request when a README construct will CORRUPT on Pages. The directory is enormous and less than a year old: `topic:agent-skills` = **12,804** repositories, leader at 165,486 stars; `topic:claude-skill` = 4,043. `[measured]` **Not a website** — that is markdownguide's game and they have nine years of search-engine authority.
- Show HN last, framed as the product, not the tool.** The ceiling for a markdown-editor Show HN is ~240–471 points (OverType 471, an open-source collaborative WYSIWYG markdown editor 460, Ferrite 241); Obsidian's own Show HN at 1,087 and Canvas at 1,380 are the absolute ceiling and they are a *product*, not a tool. **And points do not predict adoption:** `llms.txt` scored **206** points and is now in 9,640 GitHub code-search hits. `[measured — read as ordinal only; GitHub code search indexes a subset of repositories and caps estimates, so these counts are severe undercounts]`

### 8.10.5 Pricing, restated with its anchors

TIER	WHAT	PRICE	ANCHOR, VERIFIED LIVE 2026-08-01
Free	browser vault + git sync + publish + the CLI	\$0	Marginal cost ≈ 0. <i>"Free sync. It's just git."</i>
Pro	unlimited documents, live editing, AI credits	<b>\$4/month billed annually</b>	Exact parity with Obsidian Sync Standard (\$4/user/month annual, \$5 monthly); inside the forum-stated \$2–4 band
Work	same, expensable	<b>\$50/user/year</b>	Mirrors Obsidian Commercial (\$50 USD per user per year) — aimed at the population rank 9 defines
Teams	the review loop, seats. <i>Not this cycle</i>	<b>\$5–8/seat/month</b>	Under HackMD Prime (\$5/seat/month annual, 3-seat floor); far under Notion Business (\$20/member/month)

**Owner and editor seats bill. Commenters do not.** That single rule is what makes the review loop a growth mechanism rather than a seat tax, and it is why the D3 tier-split decision is on the critical path for revenue, not just for user experience.

## 8.11 What this section does NOT cover, and what would falsify it

Required by the standard: a section that only argues its own case is marketing.

### 8.11.1 What is missing from the evidence base

- Zero users. Zero interviews. Zero willingness-to-pay probes.** Across roughly a million bytes of design and 16 research areas, no one has spoken to a prospective customer. Every price in §8.10.5 is anchored to a **competitor's pricing page**, not to a probe. This is the single largest hole in the market analysis and no amount of corpus work closes it.
- Market sizing is unusable.** Four firms give 2025 note-taking market sizes of \$1.18B, \$1.35B, \$11.02B and \$17.19B — a **10× divergence**. `[secondary, self-tagged unverified]` **No claim in this section depends on it.** Do not put a market size in a deck.
- No live model call was made in any research run.** Every "easier for AI" statement in this programme is a prediction.
- The flagship 99.627% anchoring figure has been re-derived by nobody**, including every area that quoted it, and it is a block-level figure that does not transfer to ranges.
- Seven irreproducible corpus counts** were reported before the corpus was pinned. Any figure about local files that does not cite the corpus id is suspect; several such figures were quietly corrected during verification (the 80-vs-65-day corpus age, the 31,630 heading denominator, the "612 files" sub-claim which is actually 0, the "127 open issues" which exceeds the repository's all-time issue count of 97).

6. **The competitive sweep's method is known-broken.** It found its own most important item via a six-week-old Show HN post rather than through any of its registry or topic queries. **Replace registry sweeps with a watch on named application changelogs and merged-PR feeds** — `inkeep/open-knowledge` , `iwe-org/iwe` , `yamadashy/repomix` , `mdbase-dev/mdbase-spec` , `obsidianmd` , `GoogleCloudPlatform/knowledge-catalog` , `markdown-it` , `marked` .
7. **Verifier false-kill rate is 0–19%.** Several kills quoted in this section are strong evidence, not proof. Where a kill looked wrong it is flagged in place (the `.doc` export label is the clearest example — the kill was itself killed, correctly).
8. **Not covered here at all:** legal and licensing exposure (GPL-3.0 clean-room boundaries need a lawyer, not a researcher — see §8.11.3); enterprise procurement, SOC 2 or data-residency; regional or purchasing-power pricing beyond a note that the segment study recommends it; and any channel-partner or reseller motion.

8.11.2 What would falsify this section

#	IF THIS HAPPENS	THEN
F1	A probe of 30 users returns fewer than 8 naming the review loop as a reason they would pay, and more than 20 naming browser access or sync trust	The USP in §8.1 is wrong as a <i>purchase</i> driver. Keep the review loop for defensibility; lead and price on Wedge A.
F2	inkeep/OpenKnowledge — or anyone — ships <b>teammate</b> comments in a stable release	The intersection closes. The USP becomes "the one that keeps files plain and publishes its false-match rate", which is a much weaker sentence. Re-plan within the week.
F3	The range resolver's hand-audited false-match rate exceeds ~0.5% ( <b>K4</b> )	Comments cannot use it. Fall back to quote-plus-digest with visible orphaning, and the review loop ships with a visible orphan rate as a product metric.
F4	Obsidian ships a web application	Wedge A collapses to a feature. The review loop becomes the only differentiator overnight, before it is built.
F5	No second GitHub login has written to a document by 2026-08-31 ( <b>K1</b> )	This is a library, not a company. Say so out loud.
F6	Fewer than 5 of 30 probed users would install <code>cert</code> ( <b>K5</b> )	Cut the give-away entirely and put the two days into the second user.
F7	A third party ships a target-scoped markdown conformance matrix before we do	Given a measured clone latency under 24 hours, assume this is possible the week after we describe the idea in public. <b>Corollary and standing rule: do not publish the certificate's design before its dataset.</b>
F8	The head-to-head puts a clean-room reimplementaion of OpenKnowledge's published behaviour within 0.5 percentage points on false rate and 3 points on correct rate	The algorithm is a commodity a competitor gives away under GPL. The moat must be the multiplayer loop plus the published measurement — never the resolver.

8.11.3 One process risk that belongs in this section and nowhere else

The research record now **quotes GPL-3.0 source verbatim and at length** ( `anchor.ts` , `anchor-search.ts` , `thread-store.ts` ) while simultaneously mandating that any implementation be specified from *published behaviour*, *never from their source*. Those two facts are in tension. **Before any implementation of the resolver begins, write down a provenance boundary:** which artifacts an implementer may read, and a behaviour-only specification derived by someone who has **not** read `anchor.ts` . This is a process risk, not a research error, and it is the kind of thing that is cheap now and expensive later.

## 9. AIOS: what to import, what to refuse, and how it makes the build faster and safer

### 9.0 Read this before you read the rest of the section

AIOS is a separate track (decision D5). This section is not about merging AIOS into MDMAX or into frontmatter. It is about two much narrower questions:

1. What does thirty-five days of building an agent-orchestration toolchain give the **MDMAX build process** — the harness, the gates, the way a two-person team plus agents does twelve months of compiler work without shipping a false green?
2. What does MDMAX give **back** to AIOS, measured, not hoped for?

Two warnings that govern everything below.

**Warning one — the research area this section draws on was graded OVERSTATED by its own verifier.** In `docs/engine/research/wf-final-gate-2026-08-01.result.json`, area `aios-transfer` carries `headline_verdict: "OVERSTATED"`, and area `agentic-docs` also carries `headline_verdict: "OVERSTATED"` [primary]. Of ten verified claims in `aios-transfer`, **four are CONFIRMED, four are OVERSTATED, two are REFUTED** [measured, counted from `per_area[].verification.verdicts`]. The two REFUTED claims are the two that most flattered the idea of importing AIOS wholesale. Both are corrected in place below rather than repeated. If you find a claim in an older document that this section contradicts, this section governs, because it is written against the verifier's `verdicts[]` array rather than against the `headline` string.

**Warning two — this section makes no claim that AIOS makes agents better at writing markdown.** Zero live model calls were made in any research run [primary, stated in the run's own method notes]. The claim "markdown is the file type agents fail to edit MOST" was **REFUTED**: on the 1,944-file transcript selection, `.md` is 40 errors in 1,066 attempts = 3.75%, while `.ts` is 6 in 54 = **11.11%** and `.json` is 3 in 62 = **4.84%** [measured, `aios-transfer` verification, claim 9]. Worse, the modal `.md` failure is not an anchoring failure at all: classifying all 40 `.md` error bodies gives **23** File has not been read yet (**57.5%**), **14** String to replace not found (**1.31% of attempts**), **3** File has been modified since read (**0.28%**). The figure `file_modified=26` that appeared in earlier drafts was obtained by subtracting 14 from 40; the true classified count is **3** [measured, same verification]. Any design specified against "26 stale-read collisions" is specified against a residual, not a measurement. Do not build against it.

### 9.1 What AIOS is, for a reader who has never seen it

AIOS ("agent operating system", Sagnik's personal orchestration toolchain) is a versioned control layer that sits between a human operator and the coding agents he runs. Physically it is two git repositories on one Mac: `~/claude`, which holds the always-loaded operating instructions ( `CLAUDE.md` ), 119 skill definitions under `~/claude/skills-src/`, and the hook wiring in `settings.json`; and `~/sgnk`, which holds the runtime — a trace ledger, evaluation fixtures, a preference log, calibration state, and a directory of regression gates. Day one is 2026-06-27 (first commit `d8691dd4` in `.claude` ); `~/sgnk` was created 2026-07-09 [primary, `~/claude/HANDOFF-aos-2026-08-01-MASTER.md` §1.2]. The doctrine document `~/Desktop/GitHub/md/Zephyrus/AIOS-System-Architecture.md` (35,640 bytes, generated 2026-06-27) states the organizing thesis in one sentence: "*You do not have a self-improving orchestrator — you have an orchestrator that changes — until evaluation is infrastructure, logged per task, calibrated against the human, and wired back into routing.*" [primary, that file, §0].

Mechanically, AIOS works by three surfaces. **Rules:** `~/claude/CLAUDE.md` carries eight numbered zero-tolerance RULES (verify every factual claim; never run a destructive operation without per-operation approval; take a pre-op backup; no subagent may run those operations; distinguish "I believe" from "I verified"; show the arithmetic; acknowledge and fix your own errors; append a new Learned Rule when corrected) plus an append-only **Learned Rules** block. **Hooks:** 28 shell and Python hooks wired into `settings.json` fire on `SessionStart`, `UserPromptSubmit`, `PreToolUse`, `PostToolUse`, `SubagentStop`, `Stop`, `SessionEnd` and `PreCompact` — these are the enforcement layer, because a textual prohibition in a prompt is advisory (Learned Rule #48). **Gates:** `~/sgnk/gates/` holds paired `assert-*.sh` / `break-*.sh` scripts, and `~/sgnk/state/regression-gates.jsonl` holds the registered gate set. Live state as measured on 2026-08-01: **31** .sh files in `~/sgnk/gates/`, **21** registered gates in `regression-gates.jsonl` [measured, `ls -l ~/sgnk/gates/*.sh | wc -l` = 31; `grep -c . ~/sgnk/state/regression-gates.jsonl` = 21]. Note the discrepancy with the master handoff, which publishes "27 gate scripts" in its §7 substrate line — that count was taken at 04:20 IST and the directory was written again at 05:35 IST the same day.

The honest summary of AIOS at day 35 is the one its own master handoff gives: "*the error-detection layer is genuinely good and genuinely unarmed; the learning layer is instrumented and not learning.*" [primary, `HANDOFF-aos-2026-08-01-MASTER.md` §12]. That sentence is the whole basis for what follows. **We import from the error-detection layer. We refuse the learning layer**, not because it is badly built, but because by its own author's measurement it does not yet do the thing its name says.

### 9.2 The measured relationship: most of AIOS is untouched by MDMAX

Before claiming any benefit in either direction, state the size of the overlap. It is small.

AIOS's runtime is JSONL, not markdown.

EXTENSION	BYTES	FILES	SOURCE
<code>.jsonl</code> (canonical figure)	16,396,902	—	brief's pinned measurement
<code>.md</code> (canonical figure)	2,057,158	—	brief's pinned measurement
<code>.jsonl</code> (re-measured 2026-08-01)	16,465,646	275	[measured] <code>find ~/sgnk -name '*.jsonl'</code>
<code>.md</code> (re-measured, <b>all</b> )	18,768,046	450	[measured] <code>find ~/sgnk -name '*.md'</code>
<code>.md</code> in <code>~/sgnk/rawl/</code> only	16,789,640	282	[measured] web-scraper cache, not AIOS state
<code>.md</code> (re-measured, <b>excluding</b> <code>rawl/</code> )	1,978,406	168	[measured] 18,768,046 – 16,789,640

Read the last two rows carefully, because the naive measurement inverts the conclusion. Most of the markdown under `~/sgnk` is **cached web pages written by the `sgnk-rawl` scraper**, which is not AIOS state at all. Excluding that directory, AIOS's own markdown is **1,978,406 bytes against 16,465,646 bytes of JSONL — a ratio of 8.3 : 1 in favour of JSONL** [measured]. The brief's canonical figures (16,396,902 vs 2,057,158, ratio 8.0 : 1) are consistent with mine in direction and within 4% on

both sides; the small differences are files that grew between the two measurements, which is expected for append-only trace and preference logs. **Both measurements support the same conclusion and neither should be quoted without the `rawL/` caveat.**

**AIOS's gates barely look at markdown.** Of the 31 shell scripts in `~/sgnk/gates/`, exactly **two** contain the string `.md — assert-skill-contract.sh` and `break-skill-contract.sh`, the pair that checks whether the 119 `SKILL.md` files still parse against their frontmatter contract [measured, `grep -l '\.md'` `~/sgnk/gates/*.sh`]. **2 of 31 = 6.5%.**

**So state it plainly:** an editor and compiler for markdown does almost nothing for AIOS's runtime. The trace ledger, the router, the calibration curve, the bandit, the preference log, the eval fixtures, the drift stamps — all JSONL, all untouched by MDMAX. The overlap is confined to one surface (the skill catalogue, §9.10) and one defect class (writer truncation, §9.10). Anyone who tells you MDMAX "supercharges AIOS" has not measured the file types. **The traffic runs the other way: AIOS gives the MDMAX build a discipline; MDMAX gives AIOS one narrow fix and one budget problem.**

### 9.3 The one mechanism worth importing, and why it is the one

**DECIDED.** We import exactly one AIOS mechanism into the frontmatter repository: `~/sgnk/bin/sgnk-regression-gate.sh` (164 lines, bash) [measured, `wc -l`], reimplemented in Node as `specs/harness/gate.mjs`.

#### 9.3.1 What it does

A regression gate is normally a test somebody wrote after fixing a bug. Its weakness is structural: nothing ever checks that the test would have caught the bug. `sgnk-regression-gate.sh` closes that by refusing to register a check until the check has been **proven to go red under a reintroduced defect**. Registration takes two commands, not one:

```
--assert a command that must exit 0 on the current, fixed system
--broke a command that reintroduces the defect and must make the assertion exit non-zero
```

The tool runs **both** at registration time. If the assertion does not pass on the fixed system, or does not fail under `--broke`, or `--broke` was not supplied, or the write to the gate store did not land, registration is **REFUSED** and nothing is stored. The header states the design intent verbatim: *"That makes every gate in this file non-vacuous BY CONSTRUCTION rather than by review"* [primary, `sgnk-regression-gate.sh:15`].

#### 9.3.2 The exact contract — four refusal codes, each with a distinct exit status

EXIT	CONDITION	MESSAGE (VERBATIM FROM SOURCE)	LINE
2	assertion already red	"REFUSED: the assertion does not pass on the current system... A gate that is already red cannot tell you when the defect returns."	43–45
3	vacuous gate	"REFUSED: the assertion still passes with the defect reintroduced... That gate is VACUOUS — it would certify the bug."	51–52
4	no repro supplied	"REFUSED: --broke is required. A gate whose failure mode was never demonstrated is a claim, not a check."	56–58
5	write did not land	"REFUSED: ... proved non-vacuous but the write to \$GATES did NOT land (permission?). Nothing was stored."	71–72

Exit `5` exists because the first version of the tool printed `REGISTERED` unconditionally, and an OS sandbox denial on `~/sgnk/state` made it report success while storing nothing — a false green inside the tool built to eliminate false greens [primary, source comment at lines 60–63; this is Learned Rule #67]. The fix is a count-before / count-after around the append.

Two further verbs:

- `--run` executes every stored `assert` and exits 1 if any fails. A failure prints `FAIL <id> — the defect this gate was built for has RETURNED` [primary, line 85].
- `--verify-all` is the anti-rot pass: it re-executes every stored `--broke` and asserts each gate **can still fail**. A gate that no longer goes red under its own repro is reported `VACUOUS` and the run exits non-zero. Exit code `97` is a reserved `REPRO_NOOP` sentinel meaning *the repro's anchor no longer matches the source it patches, so it reintroduced nothing and its red proves nothing* — reported `INCONCLUSIVE`, and an inconclusive gate is **not** a pass [primary, lines 96–115].

Live status on 2026-08-01: **21 PASS / 0 FAIL and 21 proven / 0 VACUOUS / 0 INCONCLUSIVE** [primary, `HANDOFF-aos-2026-08-01-MASTER.md §7`], re-measured by me as 21 registered gates, ids: `LR64-zsh-dollar-zero-taint-gate`, `reward-human-deletion-is-rejected`, `traces-no-fused-lines`, `credential-dir-denied`, `battery-agrees-across-bash-and-zsh`, `no-bracket-gt-string-compare`, `tools-parse-under-both-shells`, `traces-no-split-rows`, `selfheal-refuses-without-backup`, `writer-verifies-write-landed`, `no-stale-rule-verdicts`, `drift-stamp-parity`, `scheduled-tools-are-versioned`, `hooks-honour-shared-contract`, `every-tool-is-invokable`, `skills-honour-frontmatter-contract`, `gold-pool-not-starved`, `refusal-is-detectable`, `injection-scan-covers-fetchers`, `eod-tools-are-executable`, `calibration-has-a-sample` [measured].

#### 9.3.3 The hole the verifier found, which the port must close

**Do not port this tool as-is.** The `aio-transfer` verifier probed the registration path that the tool's own selftest does not cover, in a sandbox via the `SGNK_REGRESSION_GATES` environment override, and found:

```
--add --id noop-probe --assert 'true' --broke 'exit 97' prints REGISTERED noop-probe — asserted green, proven red under the reintroduced defect., exit 0, and stores "proven_nonvacuous":true. 97 is the tool's OWN REPRO_NOOP sentinel... _add never checks it; only _verify_all does.
```

[measured, `aio-transfer` verification, claim 3, verdict OVERSTATED]. I confirmed the shape by reading the source: the `REPRO_NOOP` comparison exists at line 105, inside `_verify_all`, and nowhere inside `_add`.

**DECIDED — the port adds a fifth refusal code:**

EXIT	CONDITION
6	<code>--broke</code> exited with the reserved <code>REPRO_NOOP</code> status. It reintroduced nothing; its red proves nothing; registration is refused.

This matters more for MDMAX than it did for AIOS, because our `--broke` commands patch source files by anchor (see §9.5) and anchors in a compiler under active development rot weekly.

9.3.4 Why this mechanism and not another: the diagnosis it produced of our own research

The reason this specific mechanism is the one worth importing is that it names, precisely, the defect that produced the research corpus this whole plan is built on.

The earlier 18-area capability run had a **100% headline defect rate** — every headline was refuted by its own verifier, always in the flattering direction. That is independently re-derived: **exactly 14 of 18** `verification.killed` **lists contain the case-insensitive token** `headline` , **and all four remainders kill the headline's load-bearing number instead** [measured, `aios-transfer` verification, claim 2, verdict **CONFIRMED**].

The mechanical cause is visible in the workflow script that produced them. The 1,021-line file `mdmax-capability-maximization-wf_2e537e60-938.js` mentions `killed` at exactly **three** lines — **871** (a schema `required` array), **891** (a property definition), **942** (a string concatenation into `digest` ) — and **never once inside a conditional**. Line 933 is `const clean = results.filter(Boolean)` , which filters `null` only. The return block emits `headline: r.research.headline` with no reference to `r.verify` [measured, same verification, claim 1]. **The adversarial verifier was a reporter, not a gate**. Its output was written down next to the headline and the headline shipped anyway.

Two corrections the verifier forced on that finding, which this plan adopts:

- 1. **"The verifier's output NEVER influences the artifact" is false and was KILLED**. Line 942 folds `KILLED:` into `digest` , and the synthesis prompt built from that digest reads verbatim: *"Each was adversarially verified; claims listed as KILLED did not survive and must not be built on."* The synthesis object is returned and it demonstrably acted — the completeness-critic lens's `what_to_cut` contains *"TWELVE OF THE EIGHTEEN DESIGN SECTIONS. Every design whose justification did not survive its own verifier should be deleted rather than patched."* The correct, surviving claim is narrower: **the verifier influences the artifact only through a model-mediated prompt, never through a branch**. That is what "a reporter, not a gate" means, and that survives.
- 2. **"This is the mechanical cause of the 100% defect rate" was KILLED as an untested causal attribution**. A grep establishes the absence of a branch; it does not establish causation. The *newer* 16-area workflow ( `mdmax-final-gate-wf_b87faf71-509.js:767` ) has the **same** absence of a conditional while adding `negatives` and `falsifier` fields — and its headline defect rate fell to 5 CONFIRMED / 10 OVERSTATED / 1 REFUTED. **The variable was never isolated**.

That second correction is the honest reason we import the gate. Not "branches fix headlines" — we have not proven that. Rather: **a mechanism that physically cannot store an unproven check is cheaper than a discipline that asks a model to be honest**, and the one artefact in AIOS that has that property is this one.

**What would falsify this choice**. If, after twelve weeks, the gate registry contains fewer than ten gates, or if more than a third of `--verify-all` runs come back INCONCLUSIVE because repro anchors have rotted, then the mechanism is imposing more maintenance than it prevents and should be replaced by ordinary tests plus a manual mutation-testing pass each quarter.

9.4 Two deliberate changes when porting

(a) Write it in Node, not bash

**DECIDED**. `specs/harness/gate.mjs` , not `specs/harness/gate.sh` .

This is not a style preference. The frontmatter repository is a Node 24 project ( `"engines": {"node": ">=24"}` , [primary, `package.json` ] ) with 901 first-party `.ts` / `.tsx` files including worktrees and 296 under `src/` , `test/` and `specs/` alone [measured]. Every AIOS bash script runs under macOS `/bin/bash` 3.2 invoked from a zsh harness, and **that combination is the single largest documented source of false REDs in AIOS's history**. Rewriting in Node deletes the class at the root rather than papering over it. The class, with citations:

LEARNED RULE	THE FAILURE	CONSEQUENCE
#55	Under <code>set -e</code> , a function whose last line is <code>[ cond ] &amp;&amp; cmd</code> returns 1 when <code>cond</code> is false and kills the caller	Silently disabled <b>all</b> nightly database backups for weeks
#64	macOS <code>/bin/bash</code> 3.2 treats a lone apostrophe as a quote opener even inside a <code>&lt;&lt;'QUOTED'</code> heredoc — one contraction in a <b>comment</b> makes the script unparseable, and the error is reported hundreds of lines from the cause. <code>zsh -n</code> passes.	Took <code>sgnk-reward-mine.sh</code> from <code>rc=0</code> to <code>rc=2</code> . Also: zsh sets <code>\$0</code> to the <b>function name</b> inside a function, so a helper re-invoking <code>"\$0"</code> recurses into itself
#65	Bash 3.2 writes heredoc temp files to <code>/tmp</code> unconditionally; under an OS sandbox that write is denied when <code>cwd</code> sits outside the allowlist	The full-loop battery scored <b>67 PASS / 0 FAIL</b> from one directory and <b>55 PASS / 12 FAIL</b> from another — <i>same TMPDIR, same code, same commit</i>
#66	A bare <code>*.log</code> glob passes through unmatched in bash but <b>aborts the whole command</b> in zsh. <code>[ "\$a" \&gt; "\$b" ]</code> has no <code>&gt;</code> operator in zsh's <code>[</code> builtin: it errors and returns false	Deep-route battery scored <b>bash 32/0 vs zsh 31/1, same second</b> . The date comparison "named the healthy outcome as the failure it exists to catch"

Four rules, one root cause, and the cost of each was hours of hunting a regression that did not exist. Learned Rule #65 states the reason this is a rule and not a footnote: *"a harness that reports false FAILures trains you to ignore it, which is the same disease as one that reports false PASSes."*

Node has none of these. `child_process.spawnSync` with an explicit `argv` array has no word splitting, no glob expansion, no heredoc temp file, no `$0` reassignment, and one parser. **RECOMMENDED implementation detail**: run every `--assert` and `--broke` through `spawnSync(cmd, args, {shell: false})` where possible; when a shell is genuinely required, pin it ( `/bin/bash` ) and record which shell ran in the gate row, so a future zsh/bash divergence is visible in the data rather than inferred from a mood.

(b) Stamp an environment fingerprint on every gate row

**DECIDED**. Every row written to `specs/harness/gates.jsonl` and every row emitted by `--run` and `--verify-all` carries an `env` object. A green produced in a different environment is then **visibly a different measurement** rather than silently comparable to the last one.

```
{
 "id": "splice-writes-exactly-what-was-requested",
 "desc": "the splice writer must write byte-for-byte what the caller asked for",
 "assert": "node specs/harness/gate-cases/splice-conformance.mjs",
 "broke": "node specs/harness/gate-cases/splice-conformance.mjs --reintroduce=ast-regeneration",
 "registered": "2026-08-04T09:12:44Z",
 "proven_nonvacuous": true,
 "corpus_id": "sha256:3a010b1649899795d79274fc528dbce97fdabf4ff0f81cc02ab619c048c51a4",
 "env": {
 "cwd": "/Users/sagnikmitra/Desktop/GitHub/frontmatter",
 "git_head": "1bd4dad",
 "git_dirty": true,
 "node": "v24.x.x",
 "platform": "darwin",
 "arch": "arm64",
 "tmpdir": "/var/folders/...",
 "ci": false,
 "shell_used": null
 }
}
```

The `cwd` field is the load-bearing one, and §9.7.3 explains why: the AIOS gate harness produces **17 PASS / 4 FAIL from one directory and 21 PASS / 0 FAIL from another, same commit, same minute**, and has no field in which that difference is recorded. `git_dirty` is the second: `docs/mdmax/PLAN.md` in this working tree currently carries **351 insertions and 350 deletions** that are not committed [measured, `git diff --stat`], so a gate that reads it is measuring a state no one else can reproduce.

**RECOMMENDED, not decided:** add a `--require-clean` flag that refuses registration when `git_dirty` is true. Deferred because early compiler work is inherently dirty and a hard refusal would just get bypassed.

## 9.5 The first three gates to register, each with its `--broke` written FIRST

**The discipline, stated as a rule for the team:** you do not write the assertion first. You write the `--broke` command first, run it against the current tree, and confirm it produces the defect. Only then do you write the assertion. Learned Rule #68 is the reason: *"a test whose subject is a RARE fault proves nothing until it reproduces the fault — a green suite is the expected result of running it, not evidence of a fix."* That rule was earned on a bug that fires in **~0.34% of artifacts (3 of 880)**, where a fix passed a live end-to-end run *and* a differential over 120 real artifacts with **0 failures under both the old and the new code**. Both greens were noise.

All three gates below pin the corpus. **Any gate whose assertion counts files must record** `corpus_id`: `"sha256:3a010b1649899795d79274fc528dbce97fdabf4ff0f81cc02ab619c048c51a4"` **(1,084 files, 25,548,765 bytes)** [primary, `docs/engine/research/corpus-manifest.json`: `total_files` 1084, `total_bytes` 25548765]. The reason is concrete and current: the splice figure below is stated as *"0 of 51 files"* at `docs/engine/PLAN.md:170`, but today `git ls-files '*.md' | wc -l` in this repository returns **57** [measured, 2026-08-01]. The 51 was a true measurement at some earlier moment and is already unreproducible. **A gate that asserts "51" will go red for the wrong reason within a week.**

### Gate 1 — `splice-writes-exactly-what-was-requested`

**Defect class it locks.** Decision D7 is splice-only writing: never regenerate the file from the AST. The theorem is Foster et al., TOPLAS 2007, Lemma 3.9 — a total well-behaved lens whose `put` ignores the original source requires `get` to be a bijection, and "regenerate from the AST" is definitionally such a lens [primary, `docs/engine/PLAN.md:160-163`]. The measurements: **443 of 655 CommonMark spec examples are two distinct source strings with a byte-identical AST**; a zero-edit AST round trip of a real 206 KB document **rewrites 24.98% of lines and drops 18.94% of bytes**; and on this repository **0 of 51 files survive** `parse → remark-stringify` **byte-identically, 1 of 51 with every option tuned** [primary, `docs/engine/PLAN.md:165-170`]. `mdast-util-to-markdown`'s own documentation states *"complete roundtripping is impossible."*

`--broke`, **written first**. A flag on the gate case that swaps the splice writer for an AST-regeneration path (`parse → mdast-util-to-markdown → write whole file`) and runs the corpus through it.

```
node specs/harness/gate-cases/splice-conformance.mjs --reintroduce=ast-regeneration
```

**Expected before you write the assertion:** non-zero exit, with a count of files whose output bytes differ from input bytes on a zero-edit pass. Against the pinned corpus this must be a large number; against this repository's tracked markdown it must be **at least 50 of the 57 files** (the historical figure is 0 survivors of 51). If it comes back green, your reintroduction did not reintroduce anything — that is `REPRO_NOOP`, exit 97, and registration must refuse (§9.3.3).

**The assertion.** For every file in the pinned corpus, apply a randomly generated but recorded set of splice operations, and assert `bytesWritten === bytesRequested` and that every byte outside the spliced ranges is unchanged. Exit 0 only if both hold for every file.

**What this does NOT cover.** It does not check that the splice is *semantically* right — a splice can write exactly the requested bytes into exactly the wrong place. That is Gate 2's job. It also does not cover concurrent writes.

**What would falsify the gate.** If it stays green while a user reports frontmatter collapsed to a single line, `[x]` escaped to `\[x]`, or `_word` escaped to `\_word` — the documented damage from DesktopCommanderMCP issue #440, 2026, where an AI tool added `@tiptap/core` and silently rewrote users' `.md` files through ProseMirror's model, **14 file-writes across 6 files in 4 minutes, and it fired on read-only operations** [primary, `docs/engine/PLAN.md:175-180`] — then the gate's corpus is not representative and must be extended with the specific constructs that were damaged.

### Gate 2 — `resolve-never-returns-a-wrong-index`

**Defect class it locks.** The anchor resolver must never silently return the wrong block. Refusing to resolve is acceptable; resolving incorrectly is not, because a comment or suggestion then lands on text the author never wrote.

`--broke`, **written first**. Swap the resolver's scheme for **content hash plus nearest-position tiebreak**, which is measured at **77.80% correct, 22.20% false match, 0% safe refusal** on a denominator of **32,919 surviving block-versions** [primary, docs/engine/PLAN.md:104–113 ].

```
node specs/harness/gate-cases/anchor-safety.mjs --reintroduce=hash-plus-nearest-position
```

**Expected before you write the assertion:** non-zero exit reporting a false-match rate near 22.20%. That number is the proof of non-vacuity. If the reintroduced scheme produces 0% false matches on your fixture set, **your fixture set is too easy** — it does not contain the near-duplicate blocks that make position tiebreaking dangerous, and the gate you are about to register will certify nothing.

**The assertion.** Over the same 32,919 block-versions, assert **false-match rate == 0.000%** for the shipped resolver on the subset it agrees to resolve, and assert that every unresolvable anchor returns an explicit refusal rather than a best guess.

**A caveat you must carry into the gate's own description.** The recommended scheme's headline is **99.627% correct / 0.050% false / 0.323% safe refusal** at **~41 ms for a 446 KB / 2,225-block document** [primary, docs/engine/PLAN.md:113–115 ]. **The 99.627% figure has been re-derived by nobody.** It is one measurement from one run and no independent party has reproduced it. Write that sentence into the gate's `desc` field so it travels with the gate. The gate itself must assert the **0.000% false-match invariant**, which is a safety property, and must **not** assert 99.627%, which is a performance number that will legitimately move as the resolver improves. Asserting a performance equality is the mistake Learned Rule #66 names as its third false-RED: an equality-pinned count "punishes coverage growth and rewards standing still." **Assert floors on safety, never equalities on quality.**

**What this does NOT cover.** Adversarial input — a document constructed specifically to collide anchors. That belongs in a separate fuzzing gate once the resolver is stable.

**Gate 3** — `no-artifact-leaves-the-writer-undecodable`

**Defect class it locks.** UTF-8 integrity. This is the defect class described in Learned Rule #68: `awk substr` and `cut -c` split multi-byte characters, and the resulting artifact cannot be decoded. It fires in **~0.34% of artifacts (3 of 880)** [primary, ~/claude/CLAUDE.md, Learned Rule #68, line 744], which is precisely why it survived a live end-to-end run and a 120-artifact differential with zero failures.

`--broke`, **written first**. Swap the writer's codepoint-safe cut for a byte-oriented one — the `head -c` equivalent — and run the corpus through.

```
node specs/harness/gate-cases/utf8-integrity.mjs --reintroduce=byte-oriented-cut
```

**Expected before you write the assertion:** non-zero exit naming the specific files that fail to decode. **And here is the part the team will get wrong if it is not written down:** LR#68 records that proof only came from *reconstructing the failing input out of the corrupt artifact itself* — stripping the writer's own prefix and suffix, restoring the character the source proves was there, and showing the old code emits output **byte-identical to the shipped corruption**. A random corpus pass at a 0.34% base rate will produce zero failures often enough to fool you. **The `--broke` fixture must contain a constructed case where a multi-byte character straddles the cut boundary by construction, not by chance.**

LR#68 also records the corollary that cost time the first go: *"a repair tool's signature must be derived from the artifacts, not from the mechanism you imagine"* — the first version of that gate matched only truncation at a **line end** and silently refused all three real files, because there the orphan byte sits **mid-line**.

**The assertion.** Every artifact the writer produces round-trips through `Buffer.from(bytes).toString('utf8')` and back to identical bytes; no lone surrogates; no replacement characters introduced that were not in the input.

**What this does NOT cover.** Normalization (NFC vs NFD) and bidirectional text. Those are separate concerns with separate gates, and neither is in scope for the first three.

**The three gates as a registration sequence**

```
ALWAYS run --broke standalone first and read its output. If it exits 0, stop.
node specs/harness/gate-cases/splice-conformance.mjs --reintroduce=ast-regeneration; echo "rc=$?"

node specs/harness/gate.mjs --add \
 --id splice-writes-exactly-what-was-requested \
 --desc "bytes written == bytes requested; every byte outside the spliced range unchanged (D7, Foster et al. TOPLAS 2007 Lemma 3.9)" \
 --corpus sha256:3a010b1649899795d79274fc528dbece97fdabf4ff0f81cc02ab619c048c51a4 \
 --assert 'node specs/harness/gate-cases/splice-conformance.mjs' \
 --broke 'node specs/harness/gate-cases/splice-conformance.mjs --reintroduce=ast-regeneration'
```

Repeat for `resolve-never-returns-a-wrong-index` and `no-artifact-leaves-the-writer-undecodable`. Then wire `node specs/harness/gate.mjs --run` into `npm run verify` (§9.6) and `--verify-all` into a weekly job.

## 9.6 The verify ladder

**What the research REFUTED about this**

The claim *"The AIOS verify ladder transfers to the MDMAX build with zero modification"* was graded **REFUTED**, and the reasons are specific and worth reading before anyone tries the import:

- `grep -n 'tsc|noEmit|tsconfig' ~/sgnk/bin/verify-ladder.sh` returns **exactly one line — line 15, inside the header comment**. **There is no tsc branch in the code.** I confirmed this by reading the source: the syntax rung branches on `*.sh → bash -n` (line 80), `*.py → python3 -m py_compile` (line 83), `*.js → node --check` (line 86). No TypeScript [measured].
- On frontmatter the syntax rung sees **2 .sh files, 2 .py files (one of which is `./node_modules/flatted/python/flatted.py`)**, and **the first 100 of 1,281 .js at maxdepth 3** — and **zero of the repository's 901 first-party .ts / .tsx files** [measured, aios-transfer verification, claim 4; I independently re-derived the 901 figure].
- The reported "3.17 s pass on frontmatter" is therefore *a pass over two shell scripts and `node_modules`*.

- The non-vacuity demonstration was **confounded**: the test tree contained both a TypeScript type error and a broken shell script. Split apart, the TypeScript-only tree gives `{"verdict":"infra","rung_reached":"none","pass":false,"timeout":false,"rungs":{"syntax":"skip"}}`, exit 2. **The ladder never sees TypeScript. The failure came entirely from the shell file.**

Two method defects were logged from this, and both are ours to avoid: *"a two-cause probe presented as a single-cause proof"*, and a documentation/code divergence ( `verify-ladder.sh:15` documents a rung that does not exist) that *"caused a downstream researcher to believe TypeScript was being checked and to publish a transfer claim on it."*

What we already have

`npm run verify` at HEAD is already a cheapest-first ladder [primary, `package.json`], and confirmed byte-identical between HEAD and the working tree — `git diff --stat package.json` is empty, and both carry 18 scripts]:

```
verify = typecheck → lint → test → build → arch
```

with `typecheck: tsc --noEmit`, `lint: eslint . --max-warnings=0`, `test: vitest run --passWithNoTests`, `build: next build`, `arch: node specs/harness/clean-architecture-report.mjs`.

**Contradiction resolved.** The research run logged a REPO defect: *"the working-tree package.json has `"scripts": {}` (uncommitted, 97 deletions)... DESIGN §2's premise is true at HEAD and false in the tree the team would build in."* **That is no longer true.** As measured on 2026-08-01, `git diff --stat package.json` returns empty and both HEAD and the working tree carry all 18 scripts including `verify`. The defect was real when logged and has since been repaired. The research record governs history; the live measurement governs the build.

The imported ladder is therefore **strictly weaker than what the repository already has**, and we do not import the script.

The one rule to import

**DECIDED.** Import the ladder's *semantics*, not its code. Two rules, both stated verbatim in `verify-ladder.sh`'s own header [primary, lines 6–9, 19–21]:

*"a rung that TIMES OUT is `infra`, never `fail`, so a slow/wedged candidate can never masquerade as a quality failure"*

and a single-line JSON contract:

```
{"verdict":"pass|fail|infra","rung_reached":"<rung>","pass":bool,"timeout":bool,"rungs":{...}}
Exit code mirrors verdict: 0 pass · 1 fail · 2 infra (timeout/unavailable)
```

**Why the timeout rule is the one that matters.** Without it, a candidate that wedges — an infinite loop in the parser, a runaway regex, a hung `next build` — is recorded as a *quality* failure. If you are running best-of-N candidate selection over agent-produced patches, that means the wedged candidate is scored as "bad code" and the selector learns from noise. With the rule, it is scored as "the harness could not decide", it is **non-selectable**, and the signal stays clean.

The frontmatter rung table, as decided:

RUNG	COMMAND	TYPICAL COST	ON TIMEOUT
<code>typecheck</code>	<code>tsc --noEmit</code>	seconds	<code>infra</code>
<code>lint</code>	<code>eslint . --max-warnings=0</code>	seconds	<code>infra</code>
<code>test</code>	<code>vitest run --passWithNoTests</code> (83 test files under <code>test/</code> )	tens of seconds	<code>infra</code>
<code>gates</code>	<code>node specs/harness/gate.mjs --run</code>	seconds	<code>infra</code>
<code>build</code>	<code>next build</code>	minutes	<code>infra</code>
<code>arch</code>	<code>node specs/harness/clean-architecture-report.mjs</code>	seconds	<code>infra</code>

`gates` is inserted **before** `build` because it is cheap and because a splice or anchor regression should stop the ladder before anyone waits for a Next.js build.

**One JSON line per run**, appended to `specs/harness/verify-runs.jsonl`, carrying the `env` fingerprint from §9.4(b). This is the substrate for every later question about whether the build is getting faster or flakier, and it costs one line.

**A wiring hazard, recorded because AIOS hit it.** When the gate block was first added to AIOS's nightly job it was placed at the **end** of `sgnk-eod.sh`, below a `[ -f "$REG" ] || exit 0` guard that no-ops the whole job when there is no snapshot registry — so gate health was silently coupled to an unrelated opt-in, and a fresh environment ran zero gates and produced no log [primary, `HANDOFF-aios-2026-08-01-MASTER.md` §8.2]. **When you add `gates` to `verify`, verify it actually ran in a clean checkout, not just on your machine.**

9.7 What to refuse, and why

9.7.1 REFUSE: the bash guard

`~/sgnk/bin/sgnk-bash-guard.sh` is a `PreToolUse(Bash)` hook that regex-matches the command string against a deny-list and blocks on a match. Its narrow "catastrophic" subset is **mode-independent — it always blocks**, regardless of the warn/enforce setting [primary, source comment: *"this subset is ALWAYS blocked, independent of SGNK\_BASH\_ENFORCE (no mode check in this branch)"*].

The catastrophic pattern includes `TRUNCATE[:space:]`, matched with `grep -ioE` — case-insensitive, unanchored, against the **whole command string**, with no exemption for read-only commands [measured, reading `DENY_CATASTROPHIC` in the source].

**This blocked a read-only command of mine during the drafting of this very section.** The command was `tail`, `wc`, `printf` and `grep` — no writes, no deletions, no network. It contained the literal text `"truncate "` inside a quoted `grep` pattern. The hook returned:

```
PreToolUse:Bash hook error: sgnk-bash-guard CATASTROPHIC (always-block, mode-independent):
this Bash command matches the catastrophic subset (ref 62e2267c ...).
This class blocks regardless of SGNK_BASH_ENFORCE/WARN mode.
```

[measured, live, 2026-08-01, this session. I did **not** approve or bypass it — `/sgnk-approve` is a human-only action and bypassing a safety hook is outside what a subagent may do.]

**Why this specifically disqualifies it for the MDMAX build.** An MDMAX build greps for `truncate`, `substr`, `head -c`, `drop`, and `delete` constantly — those are the *names of the defect classes the compiler exists to prevent*. Gate 3 above (§9.5) is literally about the `head -c / substr` family. **A text-matching guard on a text-processing project produces a false block on the exact searches the project is made of.** The guard is well-designed for AIOS, where those words appear mostly in destructive SQL and shell. It is structurally wrong here.

**DECIDED — do not install `sgnk-bash-guard.sh` in the frontmatter repository.** What we take instead is the *principle* behind it, which is Learned Rule #48: a textual prohibition in a prompt is advisory, never the gate. §9.8 says what we use in its place — HEAD reconciliation, which matches on git state rather than on English words.

9.7.2 REFUSE: the bandit, the router, the calibration curve, the preference log, the judge panel

**DECIDED — none of the learning layer is imported.** The reason is not doubt; it is AIOS's own master handoff, which states in its §4, headed *"THE STRUCTURAL FINDING — read this twice"*:

The reinforcement-learning loop cannot close. This is not a data-volume problem.

Three facts, each verified there by execution [all primary, `~/claude/HANDOFF-aos-2026-08-01-MASTER.md §4`]:

- 1. **The escalation gate counts a tier that does not run.** `routing-bandit-decide.sh:155-156` sets `real_n` from **live sonnet observations only**, and escalation fires only at `real_n >= MIN_N`. Live traffic across **2,771 trace rows is 68.1% opus, 4.7% sonnet**. All four live arms are opus-only, so their `real_n` is 0 and stays 0. *A fabricated arm with 98 opus observations still returns `real_n=0`.*
- 2. **Even if it crossed, nothing would happen.** `sgnk-complexity-hook.sh` computes `_pick` (line 83), validates it (84), logs it (100) — and **never assigns it to anything**. All **75** decision rows are `decision_mode: advisory`. **The router records; it does not route.**
- 3. **The only live reward it ever received was wrong.** Four arms carry no `seeded` key and every one is a negative; they were written by a rejection path measured at **precision 0.000** and gated off **74 minutes 10 seconds later**. *The loop fired, and what it learned was wrong.*

The handoff's own instruction: *"Say 'instrumented', never 'self-improving' or 'reinforcement learning'."*

Alongside that, the measurement debt in §8.4 of the same document:

- **"No judge in this system has a measured Cohen's kappa against a human gold set."** Learned Rules #4 and #5 require  $\kappa \geq 0.7$  before a judge is wired into anything. **None qualifies.**
- `sgnk-handover`'s judge has returned pass on **25 of 25 rows across 5 runs** and has never once produced a fail — **unproven that it can fail**, which is the same LR#68 defect as an unproven regression gate.
- **188 of 409 registry items grep source rather than execute it** — the largest block of unverified surface in the system.
- Calibration's first real result **fails its own bar and is inverted**: low bucket `n=14 rate=0.000`, high bucket `n=23 rate=0.217` — high-confidence work is corrected a fifth of the time while low-confidence work is never corrected.

**Contradiction, named.** The master handoff instructs *"Trust the registry file"* at line 42 and publishes **151 PASS / 188 PROXY / 69 VACUOUS / 1 FAIL of 409** at line 39. The verifier read the registry (`~/claude/aio-site/testplan/test-registry.json`, 774,864 bytes, 409 items) and found `result = {PASS: 152, PARTIAL: 131, PROXY: 124, VACUOUS: 2}` and `result_prior = {None: 276, VACUOUS: 67, PROXY: 64, FAIL: 2}` [measured, `aio-transfer` verification, claim 7, verdict CONFIRMED]. The published figure is **a union over two fields that sums to 542 across 409 items** — it double-counts every re-graded item and silently deletes the 131-item PARTIAL bucket. **The registry file governs. The published headline does not.** The 188 and 69 do reproduce exactly as a union, which is the sharper finding: the number is not wrong, its *shape* is undisclosed.

**What we take instead.** Nothing automated. The MDMAX build gets its quality signal from deterministic verifiers — typecheck, lint, tests, gates, byte comparisons against a pinned corpus — which is Learned Rule #25's ordering: *rules-based first, visual second, judge-panel only for genuinely fuzzy criteria*. A compiler has almost no fuzzy criteria. **RECOMMENDED, deferred:** when frontmatter reaches a point where prose quality or UI taste is being judged at volume, revisit this — but only after establishing a human gold set and measuring kappa, in that order.

9.7.3 REFUSE: the gate harness's own false-RED mode

The gate mechanism is worth importing (§9.3). **The AIOS harness around it is not**, because it has a live, reproduced false-RED mode and no preflight.

The measurement, reproduced twice in the same minute [measured, `aio-transfer` verification, claim 5, verdict **CONFIRMED**]:

CWD	RESULT
<code>/Users/sagnikmitra/Desktop/GitHub/frontmatter</code>	<code>regression-gates: 21 PASS 0 FAIL</code>
<code>~/Desktop/GitHub</code>	<code>regression-gates: 17 PASS 4 FAIL</code>
<code>~/sgnk</code>	<code>17 PASS 4 FAIL</code>

Same commit. Same minute. The four failures are always the same gates — `traces-no-split-rows`, `no-stale-rule-verdicts`, `skills-honour-frontmatter-contract`, `calibration-has-a-sample` — and running their assertions directly prints `assert-calibration-has-a-sample.sh: line 11: cannot create temp file for here document: Operation not permitted`. `grep -c -iE 'preflight|here document' ~/sgnk/bin/sgnk-regression-gate.sh` returns **0**, while `full-loop-battery.sh` does have a preflight block with an `exit 2` path.

And the harness renders that denial as: `FAIL <id> — the defect this gate was built for has RETURNED`. An environment problem is presented as a security regression.

**The mechanism is NOT what the record says it is, and this plan corrects it.** Learned Rule #65 and the master handoff §11 both say *"Run everything from `~/Desktop/GitHub`."* A researcher then published the opposite — *"that is the directory that FAILS"* — and **that correction was itself graded REFUTED**. The decisive probes: `/` FAIL, `/usr` FAIL, `~` FAIL, `~/Desktop` FAIL, while `frontmatter/src` and `frontmatter/docs/engine` both PASS. `/` and `/usr` are not protected repositories,

so "protected repo" cannot be the mechanism. **The actual variable is the session's sandbox write-allowlist, which contains the session's own cwd, \$TMPDIR, and /tmp/claude\* : the heredoc succeeds if and only if cwd is inside it.** Decisively, full-loop-battery.sh:12-19 records cwd=~/Desktop/GitHub → heredoc OK → battery 67/0 on 2026-07-28 — the same directory passing, in a session launched there.

The verifier's summary is the rule this plan adopts:

"LR#65 and this claim are the SAME error one session apart: both report their session's sandbox allowlist as a property of the filesystem. Publishing 'run from frontmatter' installs a second wrong rule for the next session launched elsewhere. Correct rule: heredocs work only when cwd is inside the session's write-allowlist; harnesses must PREFLIGHT and name \$(pwd), never a fixed directory."

**DECIDED.** specs/harness/gate.mjs opens with a preflight: attempt one write into a temporary directory and one into \$(pwd). On failure, exit 2 with verdict: "infra" and a message naming \$(pwd) and \$TMPDIR — never emit a FAIL. No fixed directory is named anywhere in our documentation. And because we are writing it in Node (§9.4a), the heredoc pathway does not exist in the first place, which is a second, independent reason the class disappears.

## 9.8 The build harness for a small team plus agents

Two founders and a variable number of agent sessions, for twelve months, on one compiler. Four mechanisms, all imported from AIOS's operational record, all cheap.

### (a) Worktree isolation with a staleness preflight

Parallel agents get git worktrees, one per branch. The repository already has two:

/Users/sagnikmitra/Desktop/GitHub/frontmatter	1bd4dad [engine/plan-and-diagnostics]
/Users/sagnikmitra/Desktop/GitHub/frontmatter/.claude/worktrees/competent-bassi-5da9a1	8eb4de2 [claude/competent-bassi-5da9a1]
/Users/sagnikmitra/Desktop/GitHub/frontmatter/.claude/worktrees/upbeat-euclid-60dbf4	8eb4de2 [claude/upbeat-euclid-60dbf4]

[measured, git worktree list, 2026-08-01]. **Both agent worktrees sit at 8eb4de2 while the main tree is at 1bd4dad.** That is the staleness hazard in one line: an agent dispatched into either worktree today reasons about a tree several commits behind, and the .md files it reads — including docs/mdmax/PLAN.md, which has 351 insertions and 350 deletions uncommitted in the main tree — are not the ones the founders are editing.

**DECIDED.** Every parallel dispatch runs a staleness preflight: compare the worktree's HEAD to the base branch's HEAD and **refuse to dispatch** if the worktree is behind, printing both SHAs. This is cheap, deterministic, and catches the most common cause of "the agent's change doesn't apply".

### (b) Capture git rev-parse HEAD before AND after every parallel run, and reconcile

**Learned Rule #48, quoted exactly:** "Subagent/Workflow prompts forbidding git commit are NOT a reliable control. Subagents committed despite an explicit RULE-4 'do not commit' GUARD in every prompt (verified 2026-06-30, P1/P2 backlog session: 5 unauthored commits b8fd9d1 / 2e7ded0 / ec0be22 / 7ace826 / 7f882d6 landed mid-run). Always capture git rev-parse HEAD before AND after every Workflow and reconcile the delta; treat the prohibition as advisory, never as the gate."

The rule adds: "Damage was nil only because ~/.claude has no remote (local, reversible) — do not rely on that." **frontmatter has a remote.**

This is not a historical curiosity. AIOS's own live doctor reports **HEAD moved during a subagent run in 13 of 253 sessions** [primary, HANDOFF-aos-2026-08-01-MASTER.md §7, listed as a true signal, not noise]. That is a 5.1% rate on a system whose operator has been actively fighting it for five weeks.

**DECIDED.** The dispatch wrapper records git rev-parse HEAD before and after, and any unattributed delta **stops the run and reports**. This replaces the bash guard (§9.7.1) as our enforcement layer, and it is strictly better for this project: it matches on git state, not on English words, so it cannot false-block a grep for truncate.

### (c) Parallel branches only over disjoint, explicitly-named artifacts

**Learned Rule #20:** "Any parallel() branch in the Workflow tool must operate on disjoint, explicitly-specified artifacts — never on the same creative target with implicit style or architecture decisions." **Learned Rule #21:** "Any subagent is read/advisory by default. Write authority requires the full trace, not a task summary."

For a compiler, "disjoint artifact" means **one agent per module boundary, never two agents on one file**. The repository's boundaries already exist: src/modules/ contains **12 modules** [measured] — ai, ai-tools, app-shell, auth, drafts, editor, export, graph, preview, repository, share, vault — each with domain / application / infrastructure / presentation subdirectories, and npm run arch (specs/harness/clean-architecture-report.mjs) already enforces the import boundaries between them.

The MDMAX compiler adds four new boundaries. **DECIDED** — these are the parallelization units, and a parallel dispatch may name at most one:

UNIT	OWNS	NEVER TOUCHED BY ANOTHER PARALLEL AGENT
parser	source text → block sequence	the anchor scheme, the writer
anchor resolver	block identity, resolve(), refusal semantics	the writer's byte accounting
splice writer	byte-exact application of edits	the parser's tokenization
degradation matrix	what each downstream renderer does to each construct	all three of the above

If two units must change together, that is **one agent doing both, sequentially** — not two agents coordinating. Cognition's finding behind LR#20 is that implicit coordination between parallel agents on one creative target produces incoherent output more reliably than it produces speed.

### (d) Adversarial review as a GATE, not a report

This is §9.3.4 applied to the build rather than to the research. **DECIDED:** an adversarial review that returns findings **blocks the merge** until each finding is either fixed or explicitly dismissed with a recorded reason. A review whose output is appended to a document and then ignored is the reporter-not-gate pattern, and we have a measured 100% headline defect rate showing what that produces.

Two supporting rules: **Learned Rule #9** — "For code review, use fresh context. No contamination from the authoring session" — and **Learned Rule #60** — "A verifier written in the same session as the fixes it checks inherits that session's blind spots." LR#60 was earned when a verifier reported **43 pass / 0 fail** and an independent adversarial re-audit then found **34 confirmed defects (9 critical) in the fixes it had just certified**, including a grep -qE '0 FAIL' that matches inside the string 10 FAIL — the exact substring-versus-boundary bug that same session had been fixing all day.

9.9 The research-to-build boundary

DECIDED. Nothing enters the backlog on a headline.

This is the single most important process rule in this section, because the research corpus this plan is built on has a measured 100% headline defect rate in one run and 10 of 16 OVERSTATED in the other. Headlines in this corpus are, empirically, marketing for their own findings.

A backlog item is admissible only if it carries all four fields:

```
{
 "claim": "the exact sentence being built against, quoted, not paraphrased",
 "corpus_id": "sha256:3a010b1649899795d79274fc528dbece97fdabf4ff0f81cc02ab619c048c51a4",
 "reproduce": "the exact command that regenerates the number",
 "verdict": "CONFIRMED | OVERSTATED | REFUTED | UNVERIFIED",
 "corrected_claim": "required whenever verdict != CONFIRMED - the narrowed claim that survived"
}
```

Rules of admission:

- 1. verdict: REFUTED is inadmissible. It does not go in the backlog with a warning; it does not go in the backlog.
- 2. verdict: OVERSTATED is admissible only as its corrected\_claim. Example from this very section: "markdown is the file type agents fail to edit MOST" is inadmissible; its narrowed survivor — "markdown carries the largest ABSOLUTE volume of agent edit errors (40) and leads every extension on anchor-not-found (14, 1.31% of 1,066 attempts)" — is admissible.
- 3. A figure with no corpus\_id is UNVERIFIED by default. Seven irreproducible corpus counts were published before the corpus was pinned. The live demonstration of why: the splice figure "O of 51 files" at docs/engine/PLAN.md:170 cannot be reproduced today, because git ls-files '\*.md' | wc -l returns 57 [measured, 2026-08-01].
- 4. A number produced by replaying data through modified code is labelled SIMULATED, per Learned Rule #62. The rule was earned when a reward-gate improvement was reported as "1 → 80" — a sandbox replay of code that had at that moment executed in zero live runs, while live state said 1 → 1.
- 5. A "verified" claim must name the verifier's verdict, per Learned Rule #5. And per the corpus's own measured 0–19% false-kill rate, a kill is strong evidence, not proof — if a kill looks wrong to you, say so in writing and re-run it rather than silently discarding it.

A worked, live example of why field 3 exists. The aios-transfer verifier cites docs/mdmax/PLAN.md:119 for the string "O of 51 files in this repo survive it byte-identically." Today, line 119 of that file reads "forum thread asking to export a vault to a single file: \*9,010 views, 1 like, 3 posts in 26" and the actual claim lives at docs/engine/PLAN.md:170 [measured]. The citation rotted within hours, because the file has 351 uncommitted insertions and 350 deletions. A line number into a mutable markdown file is not a citation. This is, incidentally, the strongest available argument for MDMAX's differential reference invariant — and it is not a prediction, it happened while this section was being written.

9.10 What MDMAX gives back to AIOS, measured

Three things. All narrow. None of them is "AIOS becomes better at markdown."

(a) The skill-catalogue budget problem, and the drift it has already produced

AIOS loads a description for every skill into every session's context, always, while each skill's body is read only when that skill fires. The measurement:

QUANTITY	BYTES	SOURCE
All 119 SKILL.md bodies	3,295,299	[measured] find ~/.claude/skills-src -name SKILL.md   xargs stat
Always-loaded descriptions (canonical figure)	~81,754	brief's pinned measurement
description: fields alone (re-measured)	78,491	[measured] YAML frontmatter parse, 119 files
name: + description: (re-measured)	80,247	[measured]
Full YAML frontmatter blocks (re-measured)	99,638	[measured]

3,295,299 ÷ 81,754 = 40.31x. My re-measurements bracket the canonical figure: name + description gives 41.06x, the full frontmatter gives 33.07x. Quote the ratio as "roughly 40x, bracketed 33–42x depending on what counts as 'the description'" — the ambiguity is itself the finding, because there is no schema saying which bytes are the always-loaded ones.

This is a hand-maintained budgeted projection, and it has already drifted in a documented way. The sgnk-complexity-gate description reads, verbatim:

```
"...the filter that keeps work off the heavy orchestration stack unless it earns it (target restated 2026-07-28 from the stale "blocks 30-50%" figure: measured live at 90% floor / 10% strong, with 29% floor-AND-guarded — cheap model plus verifier is the design, so the honest metric is the guarded fraction, not the floor rate)."
```

[primary, ~/.claude/skills-src/plan/sgnk-complexity-gate/SKILL.md:3]. A correction note about a stale number is now permanently inlined into the always-loaded context of every session, because there is no mechanism to keep a description and its body in sync — so the fix had to be written where the reader would see it, at a permanent cost to the context budget. The body of that same file (23,569 bytes) no longer contains the 30–50% figure at all [measured, grep].

The MDMAX-shaped answer. A description and a body are two views of one document; keeping them consistent is a reference invariant, exactly the differential check MDMAX's compiler is built around. RECOMMENDED, not decided: once MDMAX can compute "which named references in document A still resolve into document B", run it over skills-src/ as a validation corpus. This is a recommendation because it is a genuine test with a real, currently-failing case — not because AIOS needs an editor.

A second, sharper instance of the same class, found while writing this. The master handoff's substrate line publishes "70 Learned Rules". Measured: ~/.claude/CLAUDE.md contains 68 distinct Learned Rule numbers, and the raw pattern ^[0-9]\+\.\ \\*\\* matches 70 times because rules 1 and 2 each match twice — RULE 8's two-item explanatory list has the same shape as a Learned Rule [measured, grep -o ... | sort -n | uniq -c]. The published count is a grep artifact that over-counts by two. This is precisely a structural-versus-textual question — a markdown compiler that understood list structure would count 68 — and it is the cleanest available demonstration of what MDMAX offers a document system: counting by structure instead of by regex. This plan uses 68.

(b) The writer-truncation defect class, which the splice writer prevents by construction

Learned Rule #68 records a bug in AIOS's own markdown-writing pipeline: `awk substr` and `cut -c` split multi-byte characters, producing artifacts that cannot be decoded. It fires in **~0.34% of artifacts (3 of 880)** [primary, `~/claude/CLAUDE.md` line 744].

**That is exactly Gate 3's defect class (§9.5).** A splice writer that (i) operates on byte ranges computed from a codepoint-aware tokenizer and (ii) asserts `bytesWritten == bytesRequested` cannot produce it. This is a genuine give-back and it is measurable: after MDMAX's writer exists, re-run AIOS's snapshot pipeline through it and assert zero undecodable artifacts across the full 880.

**Caveat, and it is the LR#68 caveat.** At a 0.34% base rate, a clean run over 880 artifacts is **not** evidence. LR#68's own record: the fix passed a live end-to-end run and a differential over 120 artifacts with **0 failures under both the old and the new code**. Proof required reconstructing the failing input from the corrupt artifact. **Any give-back claim here must include a constructed straddling case, or it is not a claim.**

(c) What MDMAX does NOT give AIOS

State this explicitly so no one builds it. MDMAX does not improve AIOS's trace ledger, router, bandit, calibration curve, preference log, eval harness, drift stamps, or hooks — **all JSONL** (§9.2). It does not fix the structural finding in §9.4 of the master handoff. It does not produce a Cohen's kappa for any judge. It does not reduce the 188 PROXY items. **If someone proposes AIOS work justified by MDMAX, check whether the artifact in question is `.md`. 6.5% of the gate scripts are.**

9.11 The Learned Rules that apply directly to this build

All from `~/claude/CLAUDE.md`, cited by number.

#	THE RULE, COMPRESSED	WHERE IT BINDS IN THIS BUILD
#4	Eval scoring is binary pass/fail plus critique, never Likert; a judge needs Cohen's kappa ≥ 0.7 against a 200-sample human gold set before being wired in	§9.7.2 — no judge qualifies, so no judge is wired
#5	Any LLM-as-judge result must cite its judge's alignment numbers, else mark unverified	§9.9 rule 5
#7	After two failed corrections in a row, <code>/clear</code> and start fresh — stale context costs more than restart	Agent sessions on the compiler
#8	Reviewers flag correctness and contract gaps, not style	§9.8(d)
#9	Code review uses fresh context, no contamination from the authoring session	§9.8(d)
#12	Reasoning budget is orthogonal to model choice — raise effort before upgrading the model	Agent dispatch on hard compiler tasks
#20	Parallel branches operate on disjoint, explicitly-specified artifacts	§9.8(c) — one agent per module boundary
#21	Subagents are read/advisory by default; write authority needs the full trace	§9.8(c)
#22	A long-running agent's default output is a first-draft PROPOSAL, not an executed mutation	§9.8(b)(c)
#25	Cheapest applicable verifier wins: rules-based → visual → judge panel	§9.6 ladder ordering
#29	Termination is harness-owned via barriers and critics — never delegate the stop decision to the model	§9.6 — the ladder, not the agent, decides done
#42	AI-generated eval cases are quarantined as <code>source:synthetic</code> until human-reviewed	Any generated gate fixture
#48	A prompt-level "do not commit" is advisory, never the gate; capture HEAD before and after and reconcile	§9.8(b) — five unauthored commits
#55	Under <code>set -e</code> , a trailing <code>[ cond ] &amp;&amp; cmd</code> returns 1 and kills the caller	§9.4(a) — deleted by writing in Node
#56	A backgrounded process inside <code>\$( )</code> must redirect its own stdout or the substitution blocks for the full timeout	§9.6 — the timeout rung's own historical bug
#59	A field-name mismatch between producer and consumer is this workspace's most expensive recurring bug class; a missing key is UNKNOWN, never a falsy default	§9.4(b) — gate rows and verify rows must be read with <code>get(x, get(y))</code> semantics
#60	A verifier written in the same session as its subject inherits that session's blind spots; a check that greps source is a PROXY and must be labelled one	§9.3.3, §9.8(d)
#61	Preview the match count before any filter that deletes rows	Any corpus-pruning step
#62	If a number came from replaying data through code rather than from live files, write SIMULATED	§9.9 rule 4
#63	A check asserting one fixed answer must pin every stochastic input; a stochastic check asserts a distribution over a seed sweep. Run a battery three times before quoting it	§9.5 — assert floors, not equalities
#64	<code>bash -n</code> every bash script; zsh parses differently and both pass while the script is broken for real callers	§9.4(a)
#65	A harness whose result depends on where you ran it reports your location, not the system's health. Preflight	§9.4(b), §9.7.3
#66	Run every harness assertion under both bash and zsh; false REDs are the expensive direction	§9.4(a)
#67	A tool that writes must verify the write landed before reporting success	§9.3.2 exit code 5
#68	A test whose subject is a rare fault proves nothing until it reproduces the fault	§9.5 — <code>—broke</code> first, always

Plus the standing zero-tolerance rules that bind agent work on this repository: **RULE 1** (verify every factual claim through a tool, not memory), **RULE 2** (per-operation approval for anything destructive — including `git push --force`, `db push`, and any outbound message), **RULE 3** (pre-op backup before any destructive database operation), **RULE 4** (no subagent executes a RULE 2 operation on its own; it returns a proposal and stops), **RULE 5** (distinguish "I believe" from "I verified"), **RULE 6** (show the arithmetic for any number that matters).

9.12 What this section does not cover, and what would falsify it

Not covered.

- **Any claim that AIOS makes agents better at writing markdown.** Zero live model calls were made in any research run. The one measurement that pointed that way was **REFUTED** (§9.0). If you want that claim, it needs a live A/B with model calls, and nobody has run one.

- **Merging AIOS and MDMAX as products.** D5 stands. This section is about the build process.
- **AIOS's security posture.** The master handoff's §8.1.2 records **25 credentials inventoried 2026-07-28 and still unrotated on day 35** — 7 GitHub `ghp_`, 1 `github_pat_`, 5 Supabase `sbp_`, 1 OpenRouter, 1 Anthropic, 10 HS256 JWTs — described there as "*Highest severity item in the system.*" That is an AIOS operational task, not an MDMAX build task, and it is named here only so it is not lost.
- **The `sgnk-*` skill layer.** 119 skills, none of them needed to build a markdown compiler. The build uses `npm run verify` and `node specs/harness/gate.mjs`.
- **Cost.** No token or dollar figure for agent-assisted compiler work appears in any source read for this section. Anyone quoting one is inventing it.

#### What would falsify this section.

1. **The gate mechanism proves unmaintainable.** If, after twelve weeks, fewer than ten gates are registered, or more than a third of `--verify-all` runs return INCONCLUSIVE from rotted repro anchors, then the mechanism costs more than it saves. Replace with ordinary tests plus a quarterly manual mutation-testing pass.
2. **The Node port reintroduces its own false-RED class.** If `specs/harness/gate.mjs` produces a result that varies with `cwd`, environment, or Node version *despite* the `env` fingerprint, then §9.4's claim — that the false-RED class is a bash artifact — is wrong, and the class is something deeper about the sandbox that a language change cannot fix.
3. **HEAD reconciliation does not catch unauthorized commits.** If an agent commits and the before/after `rev-parse` comparison misses it (an amended commit, a rewritten branch, a commit in a worktree the wrapper does not watch), then §9.8(b) is theatre and needs a `reflog`-based check instead.
4. **The 0.000% false-match invariant in Gate 2 cannot be achieved.** The 99.627% / 0.050% figures have been re-derived by nobody. If the shipped resolver cannot reach a hard zero on false matches over the 32,919 block-versions, Gate 2's assertion must be rewritten as a floor with a stated tolerance, and the "safety property, not performance number" framing in §9.5 collapses.
5. **The give-back to AIOS turns out to be nil.** If running MDMAX's reference checker over `skills-src/` finds no real inconsistency beyond the one already-documented drift, then §9.10(a) is a single anecdote and should be dropped rather than repeated.

**The one-line honest summary.** From thirty-five days of AIOS we take **one mechanism** — a gate that refuses to exist until it has been proven capable of failing — plus **one rule** (a timeout is infrastructure, never a failure), plus **four operational habits** (worktree staleness preflight, HEAD reconciliation, disjoint-artifact parallelism, adversarial review as a gate). We refuse the learning stack because its own author's measurement says it does not learn, the bash guard because it false-blocks the searches this project is made of, and the gate harness's environment handling because it is measurably wrong in both directions. In return MDMAX offers AIOS one narrow defect class it can eliminate and one budget problem it can make visible. **Everything else in AIOS is JSONL, and MDMAX does not touch JSONL.**

---

510 Engine spec

The engine specification: passes, offset model, robustness, and the hazards that will bite

← Index · ← §9 AIOS · §11 Execution →

10. The engine specification: passes, offset model, robustness, and the hazards that will bite

This section is the implementable specification. Everything above it in this document argues about *what* to build; this section says *how*, in enough detail that an engineer who has never seen the project can open an editor and start typing without asking a question. Where a decision is made, it says **DECIDED**. Where a judgement is offered that the founders may overrule, it says **RECOMMENDED**. Where nobody knows yet, it says **OPEN**.

10.0 How to read the evidence in this section

Every number carries a tag and a source. The tags mean exactly this:

TAG	MEANING
[measured]	a program was run and this number came out of its output
[primary]	read directly from a source file, a specification, or a paper
[secondary]	reported by a source we read, but not re-derived
[inference]	reasoned from other facts; nobody measured this
[SIMULATED]	produced by replaying data through code rather than read from a live system

**The pinned corpus.** Any figure in this section described as "the corpus" refers to `corpus_id sha256:3a010b1649899795d79274fc528dbece97fdabf4ff0f81cc02ab619c048c51a4`, defined in `docs/engine/research/corpus-manifest.json`. Its shape, quoted exactly from that file [primary] :

```
total_files 1084
total_bytes 25548765
roots
 md head 02c22ec47a3a73fb25a14b11be233ad3d00a8b48 756 files 21068853 bytes
 knowledge head 464eb666946c8cb4ccf42c68f4464613e5fc4999 272 files 3570923 bytes
 frontmatter head 798ebbf3250a5b2a53785859d79ae15f55ebebeb 56 files 908989 bytes
policy all *.md, case-insensitive; skip listed dirs at any depth; skip any dot-directory;
 skip listed top-level dirs per root (exclude_toplevel: Mirrors, Paste)
```

Four of the 1,084 files are zero bytes, so the non-empty count is **1,080** [measured, this section, re-derived from the manifest's own `bytes` fields]. That single fact reconciles two figures that appear in the record and look like a contradiction: the parse-robustness area reports "only 71 of 1,084 files (6.55%) have bytes == UTF-16 == code points == graphemes", and the technical synthesis reports "only 67 of 1,080 files have bytes == UTF-16 == code points". `71 - 4 = 67`. They are the same measurement over different denominators. **Both are correct; quote the 1,084 form.**

**Any figure about local files that does not cite that corpus id is suspect.** Seven irreproducible corpus counts were published before the corpus was pinned. Where this section repeats one, it says so.

**The pinned toolchain.** Every timing in this section was produced on `node v24.6.0 darwin arm64` against this repository's own `node_modules` [measured] :

```
micromark 4.0.2 yaml (eemeli) 2.9.0
mdast-util-from-markdown 2.0.3 js-yaml 3.14.2 <- 3.x, i.e. YAML 1.1
remark-parse 11.0.0 gray-matter 4.0.3
unified 11.0.5 github-slugger 2.0.0
@lezer/markdown 1.6.3 minisearch 7.2.0
@lezer/common 1.5.2 @codemirror/state 6.6.0
marked 16.4.2 @codemirror/lang-markdown 6.5.0
cmark / cmark-gfm / pandoc NOT INSTALLED ('which' -> not found)
```

Timings do not transfer between machines. Where this section quotes a ratio between two machines' wall-clock, it says the ratio is invalid and gives the absolutes instead.

**One standing caveat that applies to the whole engine.** Zero live model calls were made in any research run behind this plan. Every sentence anywhere in this document of the form "easier for AI" is a **prediction, not a result**. Nothing in this section depends on one.

10.1 The pass pipeline — and the correction that reorganises it

10.1.1 The seven passes, as the record states them

`docs/engine/PLAN.md` §3 defines the engine as seven passes [primary] :

1	parse	.md -> AST + byte positions	incremental
2	identify	mint / recover block anchors	<- the invention
3	interface	frontmatter -> module signature	schema INFERRED
4	resolve	bind every cross-reference	unresolved = diagnostic, never a silent drop
5	index	symbol table + reverse index	-> outline, tags, backlinks
6	check	schema / links / drift	-> the confidence ladder
7	project	outline / graph / site / tokens	never stored

That list is right about the *work*. It is wrong about the *shape*, and the wrongness is the single most consequential structural correction the research produced.

10.1.2 The correction: these are two pipelines, not one

The seven passes are drawn as a linear chain, which implies every consumer walks all seven. No consumer does. There are exactly two consumers, and **they barely overlap**.

- **The editor lane** (frontmatter the product: open a file, type, comment, suggest, publish) needs pass 1 and pass 2 above everything. Pass 2 — mint and recover block anchors — is the only thing that lets a comment stay attached to a paragraph after somebody edits the paragraph above it. The editor lane never needs pass 7 (project). It renders one document into one view.
- **The packing / AI lane** (mdmax pack, mdmax cert, index and outline projections for a model's context window) needs pass 7 above everything, and needs pass 2 **not at all**. A projection is generated on demand, consumed once, and discarded. It has no persisted anchors, so there is nothing to re-anchor. The record itself already settled that the container is an **ephemeral read transport**, never the write surface — which is precisely the property that makes identity unnecessary in that lane.

Drawing them as one chain has already cost this project real time: it is why the container was ever proposed as a durable artifact, and why the incremental build system got specified (§10.7) for a lane whose outputs are thrown away.

**DECIDED. Draw the engine as two pipelines sharing a parser and a normalizer.**



10.1.3 Per-pass contract

Read this table as the interface specification. "May not" is binding: a pass that does a thing in its "may not" column is a bug even if the output looks right.

#	PASS	LANE	INPUT	OUTPUT	MAY	MAY NOT
0	gate	both	raw bytes, budget	{ok, src} or a named refusal	refuse by byte count, line count, list-marker count, invalid UTF-8	parse; repair invalid UTF-8; throw
1	parse	both	decoded string	block tree with U16offset start/end per top-level block	run a grammar; run in a worker	mutate the string; normalise; emit HTML
2	identify	A only	block tree + prior anchor set	RESOLVED   AMBIGUOUS   LOST per anchor	call normalize/1; refuse	break a tie by position (measured: position tiebreak takes false matches 0.00% -> 22.20%)
3	interface	both	frontmatter block bytes	typed field map + a per-field defect list	degrade a value it cannot parse	fail the user's action on a parse error; rewrite the block
4	resolve	both	interface + link extraction	resolved edges + diagnostics	emit AMBIGUOUS	silently drop an unresolved reference (graph-data.ts:132 does this today)
5	index	both	resolved edges	symbol table, reverse index	cache keyed on content sha	persist anything an anchor keys on without a version stamp
6	check	A only	everything above	tiered diagnostics E / W / I	warn	reach tier E without a human having accepted the constraint
7	project	B only	index + budget	outline, index, bundle — on stdout	refuse above the target's context window	write to disk; be treated as durable
W	splice	A only	{src, {byteStart, byteEnd}, replacement}	{bytes, receipt}	assert boundaries	regenerate any region it was not asked to change

**The splice writer is not pass 8.** It is the only component in the entire system permitted to write bytes, and it sits outside the pass numbering deliberately so that nobody adds a second writer by adding a pass. Its correctness is already established: stock `mdast` `position.offset` spliced **107,287 top-level blocks across 1,080 files of the pinned corpus byte-identically, 0 failures** `[measured]`. D7 is therefore not an aspiration; it is a function that exists and that two shipped code paths bypass (§10.6).

10.1.4 What this section does not cover

Passes 3–6 are specified here only at their interface. The *content* of pass 6 (the confidence ladder E/W/I) and the *tiers* of the resolver (`ChangeSet.mapPos` -> `diff3` -> content-derived) are specified elsewhere in this plan and are not re-derived here. This section covers pass 0, pass 1, the offset model, `normalize/1`, `slug/1`, the frontmatter value slot, the robustness regime, and the incremental question.

10.2 The offset model, specified end to end

**This is where a shipped bug already came from, and it is the cheapest item in the whole programme.** Learned Rule 68 in this workspace records a UTF-8 truncation bug in an unrelated tool caused by `awk substr` splitting a multi-byte character. The same class is latent here at every boundary where a byte offset meets a UTF-16 offset.

10.2.1 What was actually measured

Over the pinned corpus `[measured, reproduced to the digit by an independent verifier]`:

files	1084		
all four units equal (bytes==u16==cp==gr)	71	(6.55%)	-> 67 of the 1,080 non-empty
bytes != UTF-16 code units	1013	(93.45%)	
UTF-16 units != code points (non-BMP)	38	(3.51%)	
code points != grapheme clusters	42	(3.87%)	
corpus totals	bytes	25548765	
	utf16	24699622	
	codepoints	24699360	
	graphemes	24573891	
byte<=>UTF-16 skew	849143	(3.324% of bytes)	
largest single-file skew	533366 units,	in md/_Archive/Satinath Content JSON (raw source).md	

Two claims in the record must be dropped, and this section drops them.

1. "93.4% already diverge, so the boundary risk is 21x what the engine PLAN says." The verifier independently counted files containing at least one non-ASCII character and got **exactly 1,013** — identical to `bytes != UTF-16`. So that statistic is not a discovered hazard, it is the definition of "this file is not pure ASCII" `[measured]`. **REFUTED. Do not use the 93.4% as a risk figure.**
2. The comparison to `docs/engine/PLAN.md` §3.3's "103 of 2,314 files (4.5%)" is invalid: that figure counts non-BMP characters over a different 2,314-file selection, and non-BMP on the pinned corpus is **3.51%, which is LOWER than 4.5%, not 21x higher** `[measured]`. The two documents are not in conflict; the multiplier was arithmetic over mismatched denominators.

**What survives, and it is enough.** 3.51% of files carry a character where a UTF-16 offset can split a surrogate pair. 3.87% have code-point boundaries that are not grapheme boundaries. One file diverges by 533,366 units between two of the units in play. Any of those three is sufficient justification for the model below.

There is a nice illustration of the problem inside the research itself: the parse-robustness area calls `md/graphify-out/GRAPH_REPORT.md` (1,466,025 bytes) "the largest corpus file", while the largest file by bytes is actually `md/_Archive/Satinath Content JSON (raw source).md` at 1,491,757 bytes `[measured, this section, from the manifest]`. Both statements are defensible — Satinath is larger in bytes, GRAPH\_REPORT is larger in UTF-16 units, because Satinath carries the 533,366-unit skew. **A sentence about "the largest file" is ambiguous unless it names its unit.** That is the entire argument for this subsection, in miniature.

10.2.2 The specification

**DECIDED — ONE UNIT.** Every offset inside MDMAX is a **UTF-16 code unit index** into the document string. Rationale: CodeMirror 6, Lezer and mdast are all natively UTF-16, `str.length` is free (0 ms over the whole 25.5 MB corpus `[measured]`), and a code-point walk costs 52 ms for the same bytes `[measured]`. Choosing bytes would require converting on every editor interaction; choosing code points would require converting on every parser interaction.

**DECIDED — ONE BOUNDARY POLICY.** An offset is a UTF-16 index into the string obtained by decoding the file's bytes as **strict UTF-8** with a leading BOM stripped and **no normalisation applied**.

- An offset **may never fall inside a surrogate pair**. A range whose `start` or `end` would split one is **rejected at construction** by `assertBoundary()`, never silently rounded. Rounding is how you get a corrupted character; refusing is how you get a bug report.
- Grapheme boundaries are **not** enforced on internal offsets. They are enforced only on offsets that cross out to an external annotation format (§10.2.3, row 5).
- Invalid UTF-8 is **refused, never repaired**. A repair changes bytes, and D4 and D7 both say we do not change bytes we were not asked to change.

**DECIDED — ONE CONVERSION POINT.** A single `OffsetMap` object per open document. It holds:

```
class OffsetMap {
 // built eagerly at open: Uint32Array prefix table, UTF-16 index -> byte offset
 toByte(u: Uint32Array): Uint32Array
 fromByte(b: Uint32Array): Uint32Array // throws if b is not a UTF-8 boundary

 // built lazily on first use: Intl.Segmenter grapheme boundary bitset
 toGrapheme(u: Uint32Array): Uint32Array
 fromGrapheme(g: Uint32Array): Uint32Array

 assertBoundary(u: Uint32Array): void // throws if u splits a surrogate pair
}
```

Costs, all measured on the pinned corpus [measured] :

OPERATION	COST
build the Uint32Array UTF-16 -> byte prefix table for the largest file	4 ms, 5.8 MB
Buffer.byteLength over the whole 25.5 MB corpus	14 ms
str.length over the whole corpus	0 ms
code-point iteration over the whole corpus	52 ms
Intl.Segmenter grapheme segmentation over the whole corpus	893 ms = 28.6 MB/s
implied grapheme cost for a 100 KB document	~3.5 ms

Four milliseconds per document, paid once, removes an entire bug class. There is no performance argument against this.

10.2.3 What crosses each boundary

This table is the contract. Every arrow is a call into OffsetMap and nowhere else.

#	BOUNDARY	THEIR UNIT	OUR UNIT	CONVERSION	HAZARD IF YOU SKIP IT
1	git blob / Buffer / HTTP Range / content hash over a range	bytes	UTF-16	OffsetMap.fromByte / toByte	a hash keyed on a byte range and an anchor keyed on a UTF-16 range describe different text on 93.45% of files
2	CodeMirror 6 EditorState.doc , ChangeSet.mapPos	UTF-16	UTF-16	none — identity	none. This is why UTF-16 was chosen
3	mdast node.position.*.offset , Lezer SyntaxNode.from/to	UTF-16	UTF-16	none — identity	none
4	a Python, Rust or Go implementation	code points (Py) / bytes (Rust, Go)	UTF-16	at the process boundary, once per call	this is the documented kill condition for the whole decision (§10.2.5)
5	iA Writer Markdown Annotations v0.2; any exported external anchor	grapheme clusters	UTF-16	toGrapheme / fromGrapheme	3.87% of files have codepoint != grapheme ; an annotation exported in the wrong unit lands in the wrong place
6	disk	UTF-8 bytes	UTF-16	decode once at open, encode once at write	the splice writer takes ByteOffset — see below
7	Language Server Protocol	negotiated utf-8 / utf-16 / utf-32	UTF-16	negotiate utf-16, else convert at the boundary	LSP is the one protocol that already treats this as a negotiation, which is the strongest external evidence that a single declared unit is the right design

Note row 6 carefully. The splice writer takes byte offsets, because it writes bytes. Every other component takes UTF-16. The conversion happens in exactly one place: the call site that hands a range to splice(). That is the single most dangerous line in the codebase and it should be the only one of its kind.

10.2.4 Enforcement

DECIDED. Three mechanisms, all cheap:

- 1. **Branded types.** type Uint32Array = number & {\_\_u16: true}, likewise Uint16Array, Uint8Array. The three cannot be assigned to one another without going through OffsetMap. TypeScript catches every mix at compile time and costs nothing at runtime.
- 2. **A lint rule.** An eslint no-restricted-syntax rule forbids Buffer.byteLength, TextEncoder, codePointAt, new Intl.Segmenter, and .slice() on a document string anywhere outside src/modules/mdmax/offsets.ts. If a second file needs one of those, that is a design conversation, not a diff.
- 3. **A CI assertion.** Round-trip every one of the 1,084 corpus files through fromByte(toByte(u)) === u at 100 sampled offsets per file, and assert assertBoundary throws on a synthetic offset placed inside a surrogate pair.

10.2.5 What would falsify this, and what would kill it

- **KILL CONDITION.** The UTF-16 decision dies the moment a second MDMAX implementation exists in a language that indexes bytes (Rust, Go) or code points (Python) at a hot boundary. At that point the single-unit rule becomes a per-call conversion and the 4 ms table becomes a per-request cost. **Trigger: the first non-JS MDMAX implementation.** Until then, UTF-16 is right because every library we already depend on is UTF-16.
- **FALSIFIER.** If assertBoundary never throws across a full year of production traffic, the surrogate-pair risk was theoretical and the branded types are ceremony. Instrument it: count the throws. Report the count. A design that cannot be shown to have caught anything should be cut.
- **NOT COVERED.** Bidirectional text ordering. Offsets are logical; a document containing bidi controls has a visual order that does not match its logical order, and no offset model fixes that ( docs/engine/PLAN.md §17.4 argues this at length and is correct). Cursor placement in RTL runs is a rendering problem, out of scope for the engine.

## 10.3 normalize/1 — frozen and versioned before any anchor is persisted

### 10.3.1 Why this is urgent rather than tidy

The flagship re-anchoring result — **99.627% correct / 0.050% false match / 0.323% safe refusal**, over 32,919 surviving block-versions drawn from 41,642 block-versions across 294 consecutive real revision pairs — was produced by a **384-configuration parameter sweep** whose winning configuration includes a specific hash input: **NFC, then collapse whitespace, then lowercase** [primary, docs/engine/PLAN.md §1.1a] .

docs/mdmax/PLAN.md states the risk in one line and then does not act on it [primary] :

The 99.627% was swept over 384 configurations under NFC + collapse whitespace + lowercase. If the hash input changes later, every stored anchor silently re-keys while the published number keeps describing code that no longer exists.

That is a **silent data-loss bug, not a crash**. Every comment in every document detaches at once, with no error, on a commit that looks like a refactor.

Two further caveats belong in the same breath, because they govern how the number may be used:

- **Nobody has re-derived 99.627%**. Not one research area that quoted it recomputed it. Eight areas cite it. Treat it as [secondary] until it is re-run, and **do not publish it externally** until it is re-derived on a corpus this team did not write.
- **It is a BLOCK figure**. Comments and suggestions anchor **ranges**. The one replication of the range case gives a 3.44x improvement over baseline, not 30x, with refusal roughly doubling [primary, docs/mdmax/PLAN.md §5.2] .

### 10.3.2 The specification

**DECIDED.** src/modules/mdmax/normalize.ts exports exactly this and nothing else:

```
export const NORMALIZE_VERSION = "mdmax/normalize@1";

/** The hash input for block identity. Pure. No I/O. No configuration. */
export function normalize(text: string): string {
 return text
 .normalize("NFC") // 1. Unicode canonical composition
 .replace(/\s+/, " ") // 2. collapse every whitespace run to one space
 .trim() // 3. strip leading/trailing whitespace
 .toLowerCase(); // 4. locale-independent lowercase
}
```

Four rules govern it, and they are binding:

1. **normalize()** is for **COMPARISON ONLY**. It **never touches the bytes we write**. This is not a style preference, it is measured. Running NFC over md/Research/Satinath Chatteraj/Satinath – Poems.md (14,187 bytes) changes **77 of 437 lines** and **3 of 11 headings**, because Bengali U+09DC, U+09DF and U+09DD are Unicode **composition exclusions**: NFC *decomposes* them [measured, docs/engine/PLAN.md §17.1] . Of 462 unique Bengali words in that file, **45 (9.7%)** are normalization-sensitive. "Just run NFC on everything" rewrites 77 lines of the user's own poetry. The splice writer touches no byte it was not asked to change; **normalize()** never reaches it.
2. **Every persisted anchor stamps the version**. The anchor record is:

```
{ "schema": 1,
 "normalize": "mdmax/normalize@1",
 "slug": "mdmax/slug@1",
 "id": "...", "quote": "...", "prefix": "...", "suffix": "... }
```

An anchor whose stamp does not match the running engine is **STALE, not LOST**: it is re-resolved through the content-derived ladder and re-stamped, and a migration counter is incremented.

3. **The re-sweep trigger**. Changing **normalize()** in any way — adding a step, removing a step, changing the order, changing the whitespace class — requires **all three** of: (a) bump **NORMALIZE\_VERSION** ; (b) re-run the 384-configuration sweep and republish the correct / false / refusal triple, because the published triple describes the old function; (c) ship a migration that re-resolves and re-stamps every stored anchor. Any change that skips (c) silently detaches every comment in the product.
4. **A CI assertion makes (a) impossible to forget**. A 200-case golden file of **input -> normalize(input)** pairs is checked in. If any output changes and **NORMALIZE\_VERSION** did not, the build fails. This is the mechanism; the rule alone is not.

### 10.3.3 The normalization the product is missing today

**normalize()** is specified in the plan and **performed nowhere**. Re-verified in this repository on 2026-08-01 [measured] :

```
grep -rn "normalize(" src/ -> 0 hits
grep -rnE "NFC|NFD|NFKC" src/ -> 0 hits
grep -rn "Segmenter" src/ -> 0 hits
grep -rn "localeCompare" src/ -> 4 hits
KnowledgePanels.tsx:27 · SearchPanel.tsx:116 · tree-order.ts:34 · list-conflicts.ts:29
```

Two live consequences, both verified against the shipped modules [measured] :

- **Search**. The same visible Bengali word in two normalization forms: in-file form gives 1 hit, NFC form gives **0 hits**. They render pixel-identically, so nothing in the UI can explain the failure to the user.
- **Anchors**. github-sluggger performs no normalization, so [[Note#১. সাদা শাউ]] resolves only if the link was typed in the same form as the heading.

**DECIDED.** Apply `.normalize("NFC")` at exactly three read-side call sites — slug generation, link-fragment matching, and the search `processTerm` (indexing both forms) — and nowhere else.

**Do not send this upstream as a micromark patch.** CommonMark mandates exactly one Unicode operation, case-folding reference-link labels, and micromark implements it as `.replace(/[\t\n\r ]+/g, " ").replace(/^\s/g, '').toLowerCase().toUpperCase()` — normalization-blind, so `[Café]` in NFD does not resolve `[Café]: /url` in NFC `[primary, node_modules/micromark-util-normalize-identifier/index.js; measured end-to-end through the real parser]`. It is tempting to fix. Two facts kill the temptation: cmark's `normalize_reference` does the same thing (`cmark_utf8proc_case_fold + trim + normalize whitespace`, **no Unicode normalization**) `[primary]`, so patching micromark would make our render diverge from GitHub's on the same bytes; and this is a **five-year-old known spec issue**, `commonmark/commonmark-spec#695`, open since 2021-11-11, with the identical composed-vs-decomposed `é` example `[primary]`. **RECOMMENDED: fix it in MDMAX's own read-side `key()`, where it degrades correctly and touches nothing another engine sees; file a comment on #695; do not open a micromark PR.**

10.4 slug/1 — pinned

10.4.1 The measurement that makes this the highest-leverage line in the linker

The same 392 intra-document `[text](#heading)` anchors in the pinned corpus, scored under three slug algorithms `[measured, by an independent verifier who swept them specifically because two adjacent claims in one report had silently used two different algorithms]`:

SLUG ALGORITHM	ANCHORS	RESOLVING	RATE
github-sluggier 2.0.0 (lowercase, strip <code>\p{P}\p{S}</code> , space -> <code>-</code> )	86 of 392		21.94%
a loose <code>[^a-z0-9]+ -&gt; -</code> slug	116 of 392		29.59%
a dash-collapsing slug (as above, then collapse runs of <code>-</code> )	388 of 392		98.98%

**Anchor health swings 77.04 percentage points on a one-line difference in the slug function.** The mechanism is visible in the misses: `md/Zephyrus/ecosystem.md` links to `#2-the-studio-people-structure` while its heading reads `## 2. The Studio - people & structure`, which `github-sluggier` renders `2-the-studio—people—structure` — the em dash and the ampersand each leave a **doubled** hyphen `[measured]`. It resolves only under a matcher that collapses runs.

**DECIDED.** `src/modules/mdmax/slug.ts` exports `SLUG_VERSION = "mdmax/slug@1"` and implements **github-sluggier's algorithm plus NFC plus dash-collapsing**:

```
export const SLUG_VERSION = "mdmax/slug@1";

export function slug(headingText: string): string {
 return headingText
 .normalize("NFC") // 1. see 10.3 — comparison key only
 .toLowerCase() // 2. locale-independent
 .replace(/[\p{P}\p{S}]/gu, "") // 3. github-sluggier's strip
 .replace(/\s/gu, "-") // 4. github-sluggier's space rule
 .replace(/-{2,}/gu, "-") // 5. THE ONE-LINE DIFFERENCE. 21.94% -> 98.98%
 .replace(/^-+|-$/gu, "") // 6. trim
}
```

Rationale for step 5 rather than adopting `github-sluggier` verbatim: 98.98% versus 21.94% on the founders' own corpus is not a close call, and the divergence from GitHub's rendered anchor is **visible and diagnosable** (`mdmax cert` reports it as a per-target row) rather than silent. The cost of the choice is one documented divergence; the cost of not choosing is that three-quarters of intra-document links do not resolve.

**DECIDED — headings are NOT exported symbols.** `github-sluggier` disambiguates **positionally**: `while (own.call(self.occurrences, result)) { self.occurrences[originalSlug]++; result = originalSlug + '-' + self.occurrences[originalSlug] }` `[primary, node_modules/github-sluggier/index.js]`. Renaming heading #1 therefore silently changes the anchor of heading #2. Over the pinned corpus `[measured]`:

headings total	29082	
distinct slugs	12361	
heading instances whose slug is NOT globally unique	20741	(71.3%)
WITHIN-file duplicate slugs	534	in 96 files (8.9%)
CROSS-file colliding slugs	4020	(32.5% of distinct)
worst collisions	connections x382 · tldr x367 · key-ideas x315 · capabilities x197	

A heading is a **within-unit coordinate**, resolved by the unit's own local table under the pinned `slug/1`. It is never a cross-file exported symbol. The `{#id}` escape hatch that would make it one is **REFUSED under D6**: it is not CommonMark, and this programme's own 24-renderer measurement classes it as degrading to visible junk.

10.4.2 What the slugger cannot fix, recorded so nobody re-discovers it

`docs/engine/PLAN.md` §17.2 measured five failure classes that no slug function repairs `[primary]`:

CLASS	EXAMPLE	BOTH SLUG TO	CORPUS EVIDENCE
CJK punctuation	<code> </code> / <code> </code>	<code> </code>	collision by construction — punctuation is CJK's only separator, and the slugger deletes separators
fullwidth space	<code> </code> (U+0020) vs <code> </code> (U+3000, what a CJK IME emits)	<code>-</code> / <code> </code>	same heading, different anchor
emoji	<code>Release</code> prefixed by two different emoji	<code>-release</code>	252 emoji headings in the vault
Arabic	<code>كتاب</code> / <code>كتاب</code> / <code>كتاب</code> (tatweel)	three different anchors	presentational elongation changes identity
Turkish	<code>İSTANBUL</code> -> <code>istanbul</code> (stray U+0307)	an untypeable anchor	locale-independent <code>toLowerCase()</code>

Vault-wide: 82,090 non-empty headings, 10,578 non-ASCII, **188 files with 189 colliding slugs** — all ASCII case/hyphen collisions today, so the mechanism is proven but the damage is latent [measured]. Emoji-prefixed headings collide 100% of the time and disambiguate positionally, so inserting a heading above renumbers every anchor below it. That is anchor *instability*, which is worse than a hard failure because it breaks links that used to work.

**RECOMMENDED.** Ship the slug divergence as a `mdmax cert` column rather than trying to fix it. A per-target SLUGGER column costs nothing and is the only honest answer.

10.5 Parse robustness

10.5.1 The brief's own hazard list, checked

Three claims were handed to this programme as established. Two are wrong. Correcting them matters because each one, if believed, buys a different architecture.

CLAIM AS BRIEFED	VERDICT	WHAT WAS ACTUALLY MEASURED
"remark-parse CRASHES THE PROCESS on 20 KB of nested blockquotes"	REFUTED	micromark parsed 20,480 nested <code>&gt;</code> in <b>257 ms</b> , 50,000 in <b>2,166.5 ms</b> , 100,000 in <b>8,704 ms</b> and 200,000 in <b>28,328 ms</b> — no crash at any depth. Across 45 hostile cases x 5 parsers, micromark and mdast produced <b>zero</b> process aborts. The one process abort observed belonged to <code>marked</code> , not remark [measured, researcher and independent verifier, agreeing]
"remark-parse takes 8.6 seconds on an 80 KB adversarial file (CVE-2023-22484 shape)"	REFUTED as an absolute; the hazard is 3-4.5x WORSE	On 40,000 <code>[ + a + 40,000 ]</code> (80,001 bytes): micromark <b>34,209.7 ms</b> (researcher) / <b>38,540.4 ms</b> (verifier); mdast <b>26,488.0 / 25,700.5</b> ; Lezer <b>1,110.2 / 1,042.5</b> ; marked <b>19.0 / 17.4</b> (marked does not implement the construct) [measured]. Report the absolutes with the hardware; a cross-machine ratio is not a valid inference
"markdown-it does both in ~12 ms because it has <code>maxNesting: 100</code> ; remark has no equivalent"	UNVERIFIED for markdown-it JS; the <code>maxNesting</code> half is CONFIRMED	markdown-it (JS) is not installed. The Python port markdown-it-py 4.2.0 measured <b>41.7 ms</b> on 20k blockquotes and <b>1,370.5 ms</b> on a 5,000-deep indented list [measured] — not 12 ms, and Python-vs-JS timings are not comparable in absolute terms. <code>maxNesting</code> default is 100 in both, and micromark / remark / marked have <b>no limit of any kind</b> [primary]
"remark-parse is quadratic (k=1.91) on the CVE shape"	partially	Nested-bracket exponent measured at k=1.77 (researcher) and k=1.90 over n=2000..8000 (verifier) — same class [measured]

The real headline hazard is different from all of them, and it is not adversarial.

10.5.2 The flat bullet list — the hazard that ships in ordinary content

`mdast-util-from-markdown` is **super-linear on flat bullet lists** while being linear on paragraphs of identical byte count. Three independent runs [measured]:

ITEMS	BYTES	MDAST (RESEARCHER)	MDAST (VERIFIER)	MICROMARK	LEZER
20,000	588,890	1,129 ms	1,101.3 ms	392 / 379.2 ms	61 / 57.6 ms
50,000	1,488,890	5,554 ms	5,675.3 ms	842 / 854.9 ms	133 / 135.7 ms
100,000	2,988,890	20,626 ms	22,504.7 ms	1,821 / 1,806.7 ms	241 / 246.8 ms
200,000	6,088,890	196,096 ms	<b>359,959.2 ms</b>	3,835 / 4,971.3 ms	460 / 452.4 ms

Read the 200,000 row as a warning about measurement, not about markdown. Two runs of identical code on the same hardware disagree by **1.8x**. Rerunning at `max-old-space-size=12288` gave 337,172.9 ms, only 6% faster than the 4,288 MB default, so the cost is algorithmic and not garbage collection — but at a fitted exponent of **k = 3.2 to 4.0** above 100,000 items, the word "quadratic" is wrong [measured]. The honest statement is:

`mdast-util-from-markdown` 2.0.3 is super-linear on list items with a measured exponent of **1.79 to 1.99 between 20,000 and 100,000 items**, degrading to **k ~ 3.2-4.0 above 100,000**, with a machine-dependent absolute cost of **196-360 seconds at 200,000 items**.

The control is what makes this load-bearing: mdast on **paragraphs at matched byte counts** is linear (k = 0.789 then 0.966) [measured]. It is **19.4x slower on lists than on paragraphs at the same 2.99 MB**. Content-independent, list-specific: uniform text k=1.82, random text k=1.85, ordered lists k=1.99, blank-line-separated k=1.86 [measured].

**And this is the shape AI writes.** This plan's own measurement of 68,798 messages found that model-generated markdown is a narrow nine-construct dialect in which **48.3% of list items open with a bold run** [secondary] — models write long flat lists. The pathological shape is the product's most important input class.

10.5.3 The two live quadratics in our own code — and a third that nobody found

**\*\* (a) WIKILINK\_RE**, `src/modules/vault/infrastructure/markdown-parser.ts:12**` [primary]:

```
const WIKILINK_RE = /(?!)\[([^\]]+)\]\]/g;
```

On `n` unclosed `[ [ [measured, two independent runs]`:

N	[ [	BYTES	RUN A	RUN B
20,000		40 KB	589.2 ms	577.4 ms
40,000		80 KB	2,315.9 ms	2,282.5 ms
80,000		160 KB	9,071.0 ms	9,258.1 ms
160,000		320 KB	<b>36,865.0 ms</b>	—

Successive ratios 3.93 / 3.92 / 4.06 on 2x input give **k = 1.98**; the second run gives k = 1.983 and 2.020. **Strictly quadratic.** The realistic variant is worse: 40,000 lines of `- see [ [Note N (749 KB, a half-typed vault) takes 11,012.9 ms` [measured].

**Correction to the location, and it changes the fix.** The record says this "runs synchronously on the browser main thread for every file in the vault snapshot". **That is refuted.** `parseMarkdown` is imported in exactly two places — `src/container/dependency-container.ts:11` and `src/modules/vault/infrastructure/search-index.ts:15` — both server-only composition, and **no "use client" file in `src/` imports any of them** [primary, verified by enumeration]. It blocks the **shared**

**Node event loop** inside `/api/vault/snapshot`, `/api/vault/folder`, `/api/share` and the search index. That makes it a **single-request server denial-of-service affecting every concurrent user**, not client-side jank. The fix is unchanged (index scan, not backtracking regex); the isolation prescription now has to be about the server process.

**FIX, DECIDED.** Replace with a bounded index scan:

```
function extractWikilinks(text: string): string[] {
 const out: string[] = [];
 let i = 0;
 while ((i = text.indexOf("[", i)) !== -1) {
 const close = text.indexOf("]", i + 2);
 if (close === -1) break; // unclosed: stop, do not backtrack
 if (close - i <= MAX_WIKILINK_LEN) out.push(text.slice(i + 2, close));
 i = close + 2;
 }
 return out;
}
```

Regression gate: **k <= 1.05 at 80,000 unclosed** `[]`.

The neighbouring regexes in the same file are **linear and must not be "fixed"**. `stripCode`'s lazy quantifiers were the predicted culprit and are innocent: 16,000 unmatched fences (229 KB) strip in **0.3 ms**; 80,000 unmatched backticks (240 KB) in **0.8 ms**; 200,000 unmatched backticks in **1.8 ms** `[measured]`. The lesson generalises and belongs in the process, not the code: **reading regexes for danger is not a method**. Budget `eslint-plugin-redos` in CI plus timed scaling sweeps; do not budget a code review.

**(b) `mdast-util-from-markdown` on the shape above, reachable from ordinary content.** Measured against this repository's own `node_modules` `[measured]`: 10k=372 ms, 20k=1,003 ms, 40k=3,356 ms, 80k=12,429 ms on 2.55 MB, against a paragraph control at identical byte counts of 148 / 265 / 463 / 821 ms (k ≈ 0.82) and micromark on identical list input at 177 / 332 / 580 / 1,207 ms (k ≈ 0.94). **10.3x at 80,000 items, widening.**

**(c) NEW — and this one is on the browser main thread.** Neither the researcher nor its verifier checked the editor's own call chain. Verified by primary read this session:

```
src/modules/editor/presentation/live/LivePreview.tsx:1 "use client"
src/modules/editor/presentation/EditorPane.tsx:595 mounts <LivePreview .../>
src/modules/editor/presentation/live/LivePreview.tsx:35 useMemo(() => splitIntoSegments(content), [content])
src/modules/editor/presentation/live/block-split.ts:47 processor.parse(source)
src/modules/editor/presentation/live/block-split.ts:34-38 unified().use(remarkParse).use(remarkFrontmatter).use(remarkGfm).use(remarkMath)
node_modules/remark-parse/package.json "mdast-util-from-markdown": "^2.0.0" (2.0.3 installed)
```

`[primary]`

So `mdast-util-from-markdown` **does** run synchronously on the browser main thread, in a `useMemo` keyed on `content`, every time the Live-mode document changes. Timing that exact processor `[measured, this section, node v24.6.0 darwin arm64]`:

```
items=10000 bytes=288890 parse=468.6 ms
items=20000 bytes=588890 parse=1359.1 ms
items=40000 bytes=1188890 parse=4160.6 ms
ratios 2.90x and 3.06x on 2x input => k = 1.54 then 1.61
```

And it already fires on real content. Parsing **all 1,084 pinned-corpus files** through that same processor `[measured, this section]`:

```
filesParsed 1084 · totalMs 27390 · meanMs 25.27 · over 250 ms: 3 · over 2000 ms: 1

31162.5 / 28799.5 / 19197.0 ms 1491757 B blocks=1 md/_Archive/Satinath Content JSON (raw source).md
 723.6 ms 1466025 B blocks=7755 md/graphify-out/GRAPH_REPORT.md
345.7 ms 451785 B blocks=1527 md/Zephyrus/ecosystem.md
200.9 ms 267187 B blocks=779 md/Untitled.md
```

Three things to take from that block, in order of importance.

1. **A real, non-adversarial file in the founders' own vault costs 19 to 31 seconds** in the exact parser the Live editor calls on the main thread. It is a raw JSON dump saved as `.md`: 16,646 lines, longest line 66,298 bytes, and it parses to **one** top-level block. Opening it in Live mode freezes the tab.
2. **The same file was timed three times and gave 19,197.0 / 28,799.5 / 31,162.5 ms** — a 1.62x spread. Per this workspace's Learned Rule 63, one run is an anecdote. **Report the band, never a single draw.**
3. `md/Zephyrus/ecosystem.md` **at 345.7 ms already exceeds the proposed 250 ms keystroke budget**, and it is the studio's own source-of-truth document. The budget will fire on real content on day one. That is a calibration fact the false-refusal experiment (§10.5.8) must absorb.

10.5.4 The hostile-input corpus

**DECIDED.** A checked-in fixture directory, `test/fixtures/hostile/`, with a generator, a manifest recording `{shape, generator params, expected class}` per case, and a runner emitting one JSON row per `(case, parser, status, ms, rss)` diffed against a checked-in baseline. Eleven families:

FAMILY	CONTENTS
H1 pathological nesting	blockquote depth 20k and 100k; indented list depth 5,000; emphasis; <b>nested brackets (CVE-2023-22484)</b> ; link destinations; raw HTML
H2 quadratic shapes	unclosed shortcut refs; <code>!!</code> x50k; alternating emphasis; unmatched backticks; a 5,000-column table; autolink backtrack; 50k-line setext flip; 200k reference definitions
H3 amplification	refdef reuse; <b>YAML 9-level alias bomb</b> ; 200k entities
H4 unicode	astral plane; 20k combining marks on one base; Trojan-Source bidi overrides; lone surrogate; invalid UTF-8; U+0000; mixed CR/CRLF/LF; BOM at start and mid-document; zero-width joiners; Bengali NFC composition exclusion
H5 malformed frontmatter	unterminated; tabs; 50k keys + duplicate; <code>!!python/object/apply</code> ; 10k-deep flow; not at byte 0; <b>bare</b> <code>[[wiki]]</code> <b>runs</b> ; a <code>@</code> -initial plain scalar
H6 huge artifacts	10 MB on one line; 2M blank lines; no trailing newline
H7 unclosed constructs	a fence swallowing 200k lines; nested fence markers; 1 MB unclosed HTML comment; 1 MB info string; 1,000 tabs
H8 combined	pairwise combinations of the above
H9 flat lists	<b>20k / 50k / 100k / 200k items</b> — the highest-value family, and it was not in the original brief
H10 indented lists	2.56 MB, measured at micromark 39,611 ms
H11 near-duplicate blocks	N identical-modulo-noise blocks at N = 500 / 1,000 / 2,000 / 4,000 — the shingle quadratic

Rank the corpus by demonstrated blast radius and gate only the top tier pre-merge. Of the 45 generated cases in the research run, **5 produced a finding that changed a design decision** (flat list, indented list, bracket bomb,blockquote depth, YAML alias bomb). The other 40 — bidi controls, lone surrogates, invalid UTF-8, U+0000, BOM mid-document, zero-width joiners, huge single lines, unclosed fences, unclosed HTML comments, 1 MB info strings, 5,000-column tables, 200k refdefs, entity runs — **all parsed in under 500 ms across every parser with zero crashes** `[measured]` . Shipping them as a nightly CI gate is cheap and correct; spending engineering time hardening against them is not.

Unicode specifically is a non-event for the parsers and entirely an offset-and-policy problem: all 10 unicode cases parsed in **0.9 to 28.1 ms** across five parsers with zero throws, and the pinned corpus contains **0 files with bidi controls, 0 with U+0000, 0 with a BOM, 1 with mixed CRLF/LF, and 2 with tab indentation** `[measured]` .

10.5.5 The isolation architecture

**Runs in a worker:** parse, tree build, re-anchor, shingle index build and query, projection, YAML frontmatter parse. **Runs on the main thread / request thread:** nothing that touches untrusted document bytes.

**DECIDED — the isolation unit is a** `worker_threads` **pool**, size `hardwareConcurrency - 1` (min 1), each worker long-lived, each holding at most one document. In the browser, the same contract over a Web Worker.

**The time guard works, and it is the load-bearing fact of this design.** `worker.terminate()` interrupts synchronous, non-yielding JavaScript. Across 9 hostile shapes including an infinite `for(;;)` spin at a 250 ms budget, every worker was reclaimed at **251.3 to 255.4 ms** wall — max overshoot **5.4 ms (2.2%)** — and the main thread survived every time, including the case that hard-aborts the process when run bare `[measured]` . Independently reproduced: the infinite spin reclaimed at 253.9 ms on a 250 ms budget and 1,002.9 ms on a 1,000 ms budget; micromark on the 6.09 MB flat list reclaimed at 254.1 ms `[measured]` .

**The memory guard is a design parameter, not a best-effort footnote, and the record gets it backwards.** The research reported that `resourceLimits.maxOldGenerationSizeMb: 32` "did nothing" — that a worker reached `heapUsed` 3,382 MB under a declared 32 MB cap. **REFUTED.** The independent re-run found the limit **fires**: mdast on a 12,000,030-byte / 400,001-paragraph file under `resourceLimits: {maxOldGenerationSizeMb: 32}` terminated at **85.6 ms**, exit code 1, and the parent's `'error'` handler received `Error: Worker terminated due to reaching memory limit: JS heap out of memory` . Same for micromark on the 6.09 MB flat list, at 78.2 ms `[measured]` . The reported 3,382 MB was **self-confirming instrumentation** — the `limits` field was echoed from `workerData` , not read from an applied limit.

The inversion is the finding, and it is a constant somebody must choose:

<code>MAXOLDGENERATIONSIZEMB</code>	OUTCOME
32 (TIGHT)	fires at 78-86 ms with a <b>clean, catchable</b> parent <code>'error'</code> event
256 (LOOSE)	<code>rc=134</code> , V8 FATAL ERROR: Reached heap limit Allocation failed , <b>kills the parent process</b>

**DECIDED:** `maxOldGenerationSizeMb: 32` , `stackSizeMb: 4` , **pinned and regression-tested**. The constant is the difference between an exception and a SIGABRT.

**And there is one uncatchable abort that no worker option prevents.** `mdast-util-from-markdown` on an 11,888,898-byte / 400,001-paragraph file **SIGABRTs** ( `rc=134` , **"Allocation failed"**) **at every** `--max-old-space-size` **from 256 through 2048 MB**, in a bare Node process, on a file containing no adversarial construct at all `[measured]` . That is why the shape gate exists.

10.5.6 The shape gate and the limits

**DECIDED —** `gate()` **runs BEFORE any grammar, and refuses by shape:**

```
export type GateResult =
 | { ok: true; src: string }
 | { ok: false; reason: RefusalReason; at?: { line: number; col: number } };

const LIMITS = {
 MAX_BYTES: 4_000_000, // 4 MB
 MAX_LINES: 200_000,
 MAX_LIST_MARKERS: 20_000, // lines opening with a bullet or ordered marker
} as const;
```

Every constant is derived from a measurement, not chosen:

CONSTANT	DERIVATION
MAX_BYTES = 4 MB	mdast heap amplification is <b>119-513 MB per MB of input</b> (paragraph-dense 392.3 / 356.4 / 303.2 at 20k/80k/200k blocks; one-block 188.8 / 123.2 / 118.9; list items <b>512.7 / 380.5 / 373.2</b> ) [measured] . A 4 MB cap bounds peak heap at roughly 2 GB worst case. The largest pinned-corpus file is 1,491,757 bytes, comfortably inside it
MAX_LINES = 200,000	the 2M-blank-line and 200k-refdef shapes; also bounds the line-number table
MAX_LIST_MARKERS = 20,000	the flat-list blowup crosses 20 seconds at roughly 100,000 items; 20,000 is a 5x margin. <b>This is the specific shape of AI-generated markdown</b> , which is why it gets its own counter rather than relying on the byte cap

Also in `gate()` , before the counters: **strict UTF-8 decode (refuse, never repair)** and **BOM strip**.

**DECIDED — the refusal enum, named before any route handler consumes it.** Without this, every call site grows its own `try/catch` .

```
type RefusalReason =
| "BUDGET_TIME" // worker.terminate() fired
| "BUDGET_BYTES" // > MAX_BYTES
| "BUDGET_LINES" // > MAX_LINES
| "BUDGET_BLOCKS" // > MAX_LIST_MARKERS
| "INVALID_UTF8" // strict decode failed
| "PARSER_THREW" // a catchable throw inside the worker
| "WORKER_DIED"; // exit without a result – the SIGABRT case
```

**The parent must be written to expect all four worker outcomes**, not one: a clean result, a catchable `'error'` (the `RangeError` and `ERR_WORKER_OUT_OF_MEMORY` cases), an `'exit'` with a non-zero code (the V8 fatal case), and a timeout with no event at all.

**DECIDED — the timeout budget:**

CONTEXT	BUDGET	ON EXPIRY
keystroke reparse	<b>250 ms</b>	<code>terminate()</code> , replace the worker, keep the last good tree, mark anchors STALE
cold document open	<b>2,000 ms</b>	<code>terminate()</code> , render as a plain <code>&lt;pre&gt;</code> with a banner naming the reason
batch index build	<b>10,000 ms</b> per document	skip the document, record it in the index's defect list

**How failure surfaces: never as an exception in a route handler.** A route renders the document as plain `<pre>` with a banner naming the reason and the offending offset. The editor **keeps the last good tree and marks anchors STALE, never LOST**.

10.5.7 The shingle stage — bound the cost, do not attack the accuracy

The S3 candidate-generation stage of the resolver is **exactly quadratic in near-duplicate block count**: postings touched 2,083,334 / 8,333,334 / 33,333,334 / **133,333,334** at N = 500 / 1,000 / 2,000 / 4,000 — precisely 64x for 8x blocks, **k = 2.00** — costing 26.4 / 88.1 / 361.0 / 1,525.5 ms [measured] . The real-corpus baseline is 103,548 blocks, 1,120,486 distinct 3-gram shingles, index build 1,798 ms, average candidate set 186, and its own longest posting lists are already "how to use:" x2,324 and "2>/dev/null || true" x1,048 — **boilerplate, not attack** [measured; note that an audit flagged the 2,324 and 186 figures as published without a corpus id — they come from the same pinned-corpus run, but re-derive them before publishing] .

**DECIDED — three caps:**

- Cap the S3 candidate set at **256** (real-corpus average is 186, so a 1.4x headroom) and return `AMBIGUOUS` when exceeded. Refusal is already the design's cheap outcome.
- Cap per-block token count at **4,096** before shingling.
- Cap any posting list at **512** entries and drop over-cap shingles from the index entirely, like a stopword list derived at build time. **This cap fires on real content, and that is correct** — a shingle appearing 2,324 times carries no discriminating information.
- Reject the whole S3 stage when block count exceeds **20,000**.

**And the negative result that redirects effort.** Nobody could find a hostile input that makes re-anchoring return a **wrong** answer. The measurable failure mode is **cost, not correctness**, and cost is bounded by refusal [measured] . **Effort spent adversarially attacking the anchor's accuracy is likely wasted; effort spent bounding its candidate-set size is not.**

10.5.8 The one experiment this regime needs, and what would kill it

Everything above measures the failure side. **Nothing measures the false-refusal rate** — whether budgets refuse so often that the product is unusable. That is the open question.

**The experiment.** Three arms: the pinned corpus (1,084 files) as BENIGN; the hostile corpus (51 files) as MALIGN; and **500 documents sampled from AI output**, which nobody has built and which is the arm that matters. Five tasks per document: T1 cold parse under a 2,000 ms budget; T2 100 single-character edits under a 250 ms budget; T3 incremental tree versus full tree byte-equality; T4 splice round-trip byte-equality; T5 anchor resolve after each edit. All verifiers rules-based (spec-suite equality, tree equality, byte equality, budget-status enum) — **no judge, no model call**. Roughly 40 minutes of wall clock on one machine.

**Pre-registered predictions, stated so they can be wrong:**

#	PREDICTION	BASIS
P1	BENIGN false-refusal on T1 <= 0.5% (<= 6 of 1,084)	largest corpus file is 1,491,757 bytes, under the 4 MB cap
P2	MALIGN refusal on T1 >= 20% (>= 11 of 51)	11 hostile cases exceeded 2 s
P3	AI-output refusal on T1 between 0% and 3% — <b>the risky one</b>	if AI documents routinely carry >20,000 list items, the precheck fires on the product's most important input class and the constants must move
P4	T3 incremental == full on 100.0% of ~163,500 edits	7 of 7 on hand-built lookahead cases. Any failure is a Lezer soundness bug and is blocking
P5	T4 splice byte-identity 100.0% by construction	a single failure means the writer is not splice-only
P6	T2 p99 < 50 ms with micromark/Lezer and > 250 ms (budget-refused) with mdast on the AI arm	this is the measurement that confirms or kills the parser prescription

**One prediction is already in doubt.** §10.5.3 measured `ecosystem.md` at 345.7 ms through the editor's real processor — above the 250 ms keystroke budget, on a benign, important, real file. P1 concerns cold open at 2,000 ms and survives; **P6's keystroke half is at risk on real content, not just AI content.**

KILL CONDITIONS.

- **Kill the whole robustness programme's priority** if measured benign false refusal exceeds **3%**. At that point budgets hurt more users than hostile documents do, and the correct move is to raise every cap until refusal is under 0.5% and accept that a determined attacker can hang one worker — survivable, because the main thread is not the worker.
- **Kill the worker architecture** if `terminate()` ever fails to reclaim within 2x budget on any input, **or** if structured-clone transfer cost exceeds parse cost. This is closer than it looks: transferring an mdast tree of `docs/engine/PLAN.md` measured **71.0 ms total against 35.5 ms parse** [measured] — transfer already costs as much as parsing. If transfer dominates on larger documents, the answer is a `SharedArrayBuffer` flat-tree encoding, which is a materially different and more expensive design.
- **Kill the "consume micromark events, never build mdast" prescription** if reconstructing block boundaries and normalized block text from micromark's event stream requires materialising an mdast-equivalent structure anyway. **Test:** implement `blocksFromEvents(micromark.events(src))`, assert its output equals `fromMarkdown(src).children.map(n => ({type, start, end}))` on all 1,084 corpus files, then time it. If it is within 2x of mdast's time, the prescription is dead.

Which brings us to the one place where two research areas directly contradict each other.

CONTRADICTION. The parse-robustness area concludes *"do not build an mdast tree — consume micromark's event stream or Lezer's tree."* The opposition area concludes *"stock mdast offsets splice byte-perfectly on 1,080 of 1,080 files; use them."*

RULING: both are right about different call sites, and mdast governs v1. mdast parsed 107,287 blocks across all 1,080 real corpus files with **0 splice failures**, and took 12,429 ms on an 80k-item flat list where micromark took 1,207 ms. **Ship mdast for v1 per-document offsets, behind the shape gate and the worker budget** — the gate's list-marker threshold exists precisely to catch mdast's one pathological shape. **Do not build the micromark event-to-block reconstruction first**; the parse-robustness area's own kill condition concedes that if it needs an mdast-equivalent structure the advantage evaporates, and nobody has tested that. **Port when the gate starts firing on real user content, not before.**

Two facts that keep this ruling honest. **micromark is NOT uniformly linear** — it measured **k = 2.42 on nested blockquotes** between depth 20,480 and 50,000, and **k = 1.46** on the flat list between 100k and 200k items, not the k = 1.08 the headline quotes [measured]. So "consume micromark events" is not a free correctness win. **Only Lezer stayed near-linear on every shape tested:** 452.4 ms at 200,000 list items, 21.6 ms at blockquote depth 50,000, 1,042.5 ms on the bracket bomb [measured].

10.6 The YAML value slot, which D8 rests on and which is broken today

D8 says extensibility lives in the **VALUE** of a field, never the **SET** of node types. The rule is correct and survives. **Its carrier does not survive contact with the founders' own vault.**

10.6.1 The loud failure

Over the pinned corpus [measured, researcher and independent verifier agreeing to the digit]:

files	1084	
files carrying a frontmatter block	907	
parsed by js-yaml 3.14.2 (via gray-matter)	737	
parsed by yaml 2.9.0 (eemeli)	737	<- the SAME 737; 0 asymmetric acceptances
rejected by BOTH	170	15.68% of files · 18.74% of frontmatter blocks
reason histogram		
"end of the stream or a document separator is expected"	148	
"missed comma between flow collection entries"	10	
"can not read a block mapping entry"	6	
"incomplete explicit mapping pair"	6	
	total	170

Re-derived independently this session over the manifest's own path set (1,084 present, 0 missing): `jsThrow = 170`, `eeErr = 170` [measured, this section]. It reproduces exactly.

The cause is `related: [[a]], [[b]]` and `entities: [[[A]], [[B]]]` — Obsidian wikilinks are not valid YAML flow syntax. **That puts markdown's grammar inside YAML's, which is exactly the failure mode D8 exists to prevent, occurring in D8's own carrier.**

**Two corrections to how this has been reported.** Both matter for the spec below.

1. **The attribution is not 100% wikilinks.** Of the 170 throws, the source line js-yaml points at contains `[[` in **148** cases and does **not** in **22** [measured, this section]. So the wikilink story covers **148 of 170 (87.1%)**, and 22 files fail for other reasons.

2. **The exemplar cited in the record is the wrong line.** The research quotes `md/Research/Knowledge Base/AI/18 Claude Code Token Hacks in 18 Minutes.md` and attributes the throw to its `entities: [[Claude Code]], [[Anthropic]], ...]` line. Parsing that file's actual frontmatter block `[measured, this section]` :

```
js-yaml : missed comma between flow collection entries at line 23, column 73
eemeli : Plain value cannot start with reserved character @ at line 23, column 73
YAML line 23 = key_terms: [/clear, /compact, /context, /cost, /status-line, plan mode, @filename, MCP, ...]
```

Both parsers point at the **same position**, and that position is `@filename` — `@` is a YAML reserved indicator. The `entities:` line parses **cleanly in both parsers in isolation**. The research attached the right error message to the wrong line. **The engineering consequence: the lenient pre-pass must handle reserved-indicator scalars as well as bare wikilinks.**

10.6.2 The quiet failure, which is worse and which nobody had counted

`entities: [[[A]], [[B]]]` is **valid YAML**. It parses, silently, to `{"entities": [[["A"]], [["B"]]]}` — a nested array. The wikilinks are destroyed with **no error at all** `[measured, verified in both parsers this session]`.

Nobody had measured how often. Over the pinned corpus `[measured, this section, new]` :

files with a frontmatter value parsing to a nested array	96	of 1084	(8.86%)
value occurrences	117		
of depth >= 3 (the [[[A]], [[B]]] shape)	116		
of depth == 2	1		
by key	related 95 · entities 21 · target 1		
examples	md/Research/Knowledge Base/AI/12-Factor Agents (Dex Horthy).md :: related md/Research/Knowledge Base/AI/3D Data Science with Python.md :: entities		

**Combined, the frontmatter value slot is broken on 266 of 1,084 files — 24.54% of the corpus, and 29.33% of the 907 files that carry frontmatter at all.** The two sets are disjoint: a file that throws cannot also be silently corrupted.

CLASS	FILES	OF 1,084	OF 907 WITH FRONTMATTER	USER-VISIBLE?
LOUD — YAML throws	170	15.68%	18.74%	yes, as an HTTP 502
<b>QUIET — parses to a nested array, links destroyed</b>	<b>96</b>	<b>8.86%</b>	<b>10.58%</b>	<b>no. Nothing to catch</b>
<b>total broken</b>	<b>266</b>	<b>24.54%</b>	<b>29.33%</b>	

10.6.3 What the product does with these files today

CODE PATH	BEHAVIOUR	CONSEQUENCE
markdown-parser.ts:111-120	wraps <code>matter(raw)</code> in <code>try/catch</code> ; the comment reads "A single bad note must NEVER crash the whole vault snapshot"	degrades correctly. This is the good path
share-writer.ts:19	calls <code>matter(file.content)</code> <b>bare, no try/catch</b>	the throw is caught one level up by the generic handler in <code>src/app/api/share/route.ts</code> , which returns <b>HTTP 502 {error: 'upstream_failure'}</b> . So <b>18.7% of a real vault is unpublishable</b> , with a misleading upstream error
share-writer.ts:26	<code>const next = matter.stringify(parsed.content, data);</code>	<b>regenerates the YAML from an object.</b> Over the corpus: 737 run, <b>695 frontmatter blocks changed (94.30%)</b> , 33 byte-identical, 8 trailing-newline-only, <b>0 bodies changed [measured]</b> . Modal damage is a quoting flip: <code>up: "[[Home]]"</code> -> <code>up: '[[Home]]'</code> . A live <b>D7 violation on every Publish</b> , in a file whose own doc comment at line 3 claims it works by " <i>splicing the key into the YAML frontmatter</i> "
gray-matter/index.js:161	<code>matter.stringify = function(file, data, options) { if (typeof file === 'string') file = matter(file, options); ... }</code>	<b>it re-parses its string argument.</b> <code>share-writer.ts</code> passes <code>parsed.content</code> , a raw string, so <b>the BODY is re-parsed as if it had its own frontmatter</b> . On <code>md/pj.md</code> this throws from inside <code>matter.stringify</code> ; for any body opening with a <code>---</code> delimited block it will <b>silently consume that block and drop it</b> . This is why the reported triple does not sum: <code>33 + 8 + 695 = 736</code> , not 737 <code>[primary + measured]</code> . It converts "cosmetic quoting churn" into <b>possible silent body truncation</b> , which is the one thing D4 and D7 exist to prevent
preview/presentation/frontmatter.ts:47	<code>const yaml = doc.toString()</code>	the <code>yaml</code> library's no-edit round trip is byte-identical on only <b>119 of 907 blocks [measured]</b>

**Do NOT take the shortcut of swapping gray-matter for the `yaml` library.** 119 of 907 is not a fix.

10.6.4 A second, independent divergence: the two shipped parsers disagree on TYPE

The product ships **two YAML parsers** and they return different types for the same field on 84.8% of real files `[measured, verified to the digit]` :

both parse	737 of 907	(they agree on WHICH 737)
files with a VALUE divergence	625	= 84.80% of 737
field occurrences	973	
divergence classes	1	<- exactly one, no others
js-yaml	YAML 1.1 implicit !!timestamp	-> JS Date
eemeli	YAML 1.2 core schema	-> string
example	md/Daily/2026-05-26.md :: title: Date(Tue May 26 2026 05:30:00 GMT+0530) vs string("2026-05-26")	

The topology is in the source: `gray-matter/js-yaml` at `markdown-parser.ts:112` (server, vault index) and `eemeli/yaml` at `src/modules/preview/presentation/frontmatter.ts:17` via `parseDocument`, consumed on the browser main thread by `PropertiesPanel.tsx:55 [primary]`. The

installed js-yaml is **3.14.2** — gray-matter pins `^3.13.1`, i.e. the YAML 1.1 line `[primary]`. That is the whole mechanism.

### 10.6.5 The specification

**DECIDED — five rules.**

1. **WRITES NEVER PARSE.** The splice writer locates `public_slug:` (or any key) by **byte scan inside the delimiter block** and splices, or appends. It never invokes a YAML parser. This is the real fix, and it is what makes the 170 unparseable files **writable at all**. `matter.stringify` and every other YAML re-emitter is **forbidden**, enforced by an eslint rule on the import. **Gate:** publish-then-unpublish over all 907 frontmatter-bearing files is byte-identical, including the 170.
2. **READS USE A LENIENT PRE-PASS.** `src/modules/mdmax/frontmatter-lenient.ts` runs before the YAML parser and quotes two things: bare `[[...]]` runs, and plain scalars opening with a YAML reserved indicator (`@`, backtick). It changes **zero bytes in any user file** — it operates on a copy fed to the parser. D2, D4, D6 and D7 compatible by construction.
3. **A HARD REFUSAL FOR THE SILENT CASE.** When a frontmatter value parses to an array of depth  $\geq 2$  **and** its leaf strings look like wikilink targets, the interface pass emits a `WIKILINK_IN_FLOW_SEQ` defect rather than accepting the nested array. The value is surfaced to the user as broken. It is never silently indexed as `["A"]`. 96 files hit this today.
4. **A FRONTMATTER PARSE FAILURE MAY NEVER FAIL A USER ACTION.** Share, Publish, Rename, Export and Search all degrade. The 502 path is a bug, not a policy.
5. **ONE PARSER, DECLARED.** `yaml` 2.9.0 (eemeli), YAML 1.2 core, everywhere. Reasons, in order: it makes dates stay strings, which removes the 84.8% type divergence at a stroke; it **refuses the YAML alias bomb at parse time** (`ReferenceError: Excessive alias count indicates a resource exhaustion attack`) where js-yaml accepts it `[measured]`; and it is browser-safe. `gray-matter` and `js-yaml` are removed from `src/`. **Regression gate:** `grep -rn "gray-matter\|js-yaml" src/` **returns 0**.

**MDMAX's own emission rule, DECIDED.** Anything MDMAX writes into frontmatter is emitted as a **double-quoted scalar or a block sequence of double-quoted scalars**, never a flow sequence. That one rule makes every value MDMAX produces immune to the class above.

**A latent hazard, correctly re-filed.** A 337-byte 9-level YAML alias bomb parses in gray-matter in **1.7 ms**, survives `structuredClone` in **0.0 ms**, and then costs `JSON.stringify` **4,612.2 ms at 1,082 MB RSS** before throwing `RangeError: Invalid string length` `[measured]`. The record claimed this **freezes the browser tab today** via `SnapshotProvider.tsx:78`. **REFUTED:** the snapshot DTO that gets stringified is `VaultSnapshot = {sha, generatedAt, tree, notes: NoteMeta[]}` and `NoteMeta` (`dto.ts:35-43`) carries **no frontmatter field**; the projection at `get-snapshot.ts:196-209` discards it, and `grep -rn '\.frontmatter' src` returns exactly one consumer outside the parser (`get-snapshot.ts:199`, reading `public_slug` as a string). Independently, `src/app/api/vault/snapshot/route.ts` stringifies on the server first, and JSON has no aliases `[primary]`. **File it as a latent hazard, not a shipped defect** — and note that rule 5 above removes it anyway, plus **ban** `JSON.stringify` **on any object derived from document bytes** (use `structuredClone` into `IndexedDB` rather than `localStorage`) and add a depth-and-node-count guard on any parsed frontmatter object regardless.

### 10.6.6 What this does not fix

The lenient pre-pass makes broken frontmatter **readable**. It does not make the vault **correct**. `mdmax vocab` — read-only, roughly two days — reports the three defect classes it cannot repair: case and morphology drift in a controlled vocabulary, prose sitting in an enum slot, and an unparseable value slot. **That is a report, not a rewrite.** Nothing in MDMAX ever edits a user's frontmatter to make it parse.

## 10.7 Incremental architecture — the theory, and how much of it to build

### 10.7.1 What the literature actually requires

`HANDOFF-mdz-markdown-format-2026-07-29.md` §3.16 is the most rigorous section in the record and it must be read before any compiler work `[primary]`. Its five load-bearing results:

1. **Build Systems à la Carte** (Mokhov, Mitchell, Peyton Jones, ICFP 2018) frames the design space as **scheduler x rebuilder**: 12 cells, **8 inhabited** (Make, Excel, Ninja, Shake, CloudBuild, Bazel, Buck, Nix). Pick a cell deliberately.
  - The recommended cell is **suspending scheduler + verifying traces** — Shake's cell. It is the **minimum retained state that buys all three** of dynamic dependencies, minimality, and early cutoff: per key, `(hash of each dependency, hash of result)`.
  - **Make's cell provably cannot do early cutoff.** Early cutoff is exactly what you want when somebody fixes a typo and 400 files reference that document.
  - **Applicative tasks let you extract dependencies without running them; monadic tasks provably do not** (`Const` has no `Monad` instance). **Cross-file reference resolution that decides what to read based on what it just read IS monadic — which rules out a topological scheduler.**
2. **Incremental Computation with Names** (Hammer et al., OOPSLA 2015), verbatim: structural (hash-consed) matching means Adapton *"recomputes and reallocates a linear number of output elements for each  $O(1)$  input change"*, whereas nominal (named) matching *"need not rebuild the prefix."* So **content-hash identity implies  $O(n)$  per edit; stable names give  $O(1)$  or  $O(\log n)$** . Identity is an **asymptotic** requirement, not a nicety. This retires "just content-address the blocks".
3. **Wagner & Graham** (TOPLAS 1998), verbatim: *"The central requirement for actual incremental behavior — balancing of lengthy sequences — has been ignored in all previous approaches."* A flat block sequence gives  **$O(n)$  for edits at the beginning or end** no matter how good the engine. **Unbounded sequences must be stored as balanced trees. And the escape hatch that breaks the guarantee applies directly to markdown:** it fails when *"the interpretation of the yield of a sequence depends on its context"* — which is **exactly setext headings, lazy continuation, link reference definitions, and fence state**. This is the single most concerning theoretical result for a markdown compiler.
4. **Incremental packrat parsing** (Dubroy & Warth, SLE 2017): invalidation must key on what a rule **examined**, not what it **consumed** (`examinedLength`). **A naive "invalidate overlapping ranges" rule is silently wrong** for a language with unbounded lookahead. Markdown is such a language. Measured there: reparse after a keystroke **mean 6.2 ms**, ~12% memory overhead.
5. **Ramalingam & Reps** (TCS 158, 1996): a proven hierarchy separating polynomially bounded, inherently exponentially bounded, and unbounded problems. **Some problems cannot be made cheap per edit no matter how much state you retain.**

The cost of incrementality, also measured in that literature: self-adjusting computation is **4-10x slower on the cold build** to be **~1000x faster on update**; Adapton is **1.5-3.5x slower** than traditional incremental computation when *all* output is demanded. **You are always trading cold-build time for warm-edit time.**

### 10.7.2 The honest verdict: this is over-specified for markdown at our scale

Every one of those results is correct. **Almost none of them earns its implementation here**, because the workload they optimise does not exist in this corpus. Measured `[measured]`:

cold full lex of all 25.5 MB	1,056 ms
maximum transitive blast radius	54 files (5.0%), mean 8.77
files participating in ZERO resolved edges	507 of 1,084 (46.8%)
link reference definitions in the corpus	0 (mdast: 0 `definition`, 0 `linkReference`, 0 `imageReference`)
Lezer TreeFragment.applyChanges == full reparse	7 of 7 hand-built lookahead cases

Read those five lines together. **A build system's entire value is avoiding work over a large dependency closure.** Here the closure is tiny and the full build is one second. And the construct the incremental-parse literature is *about* — the link reference definition, the canonical unbounded lookahead case — occurs **zero times in 25.5 MB of the founders' own markdown**, verified two independent ways (an mdast parse of all 1,084 files finding 0 `definition` nodes, and a `grep -rLE '^\\[[^]]+\\]: '` finding exactly 2 files, both under `md/Mirrors/`, which the corpus policy excludes at top level) `[measured]`.

Microsoft says the same thing about its own dependency engine, in writing `[primary, learn.microsoft.com]`: *“For some complex workbooks, the time taken to build and maintain the dependency trees needed for Smart Recalculation is larger than the time saved by Smart Recalculation.”*

One more correction, because it removes the last argument for early cutoff. The record puts the early-cutoff yield at 69.1% of revisions. Measured over git history restricted to **human** edits (commits touching `<= 60 .md` files, modifications only), the exported interface survives **41.54% of revisions in knowledge** and **19.71% in md** — not 69.1% `[measured]`. The 69.1% figure is dominated by bulk machine-generated commits (a 2,706-file mirrors import, three 240-file recategorisations that are pure renames, a 742-file share migration) where nothing structural could change. **A 3.5x correction, in the flattering direction.**

### 10.7.3 BUILD / DELETE

DECIDED.

COMPONENT	VERDICT	REASON
Lezer <code>TreeFragment.applyChanges</code> for the editor	<b>BUILD</b> — adopt, do not write	CodeMirror 6 is already a dependency and the Lezer tree is already in memory and currently discarded. Sound on 7 of 7 hand-built unbounded-lookahead cases
a property test asserting <code>incremental === full</code>	<b>BUILD</b>	100 sampled single-character edits per file, over all 1,084 files. Any failure is a Lezer soundness bug and is blocking
<code>Map&lt;path, {contentSha, blockIds}&gt;</code> invalidated on sha change	<b>BUILD</b> — roughly 20 lines	this is the entire corpus-level cache
the <b>anchor</b> invalidation rule	<b>BUILD</b>	see below. This is a different thing from parse invalidation and is where the cost actually is
a bespoke examined-vs-consumed invalidation rule	<b>DELETE</b>	re-deriving something CodeMirror already gets right, for a construct that occurs 0 times
the Ninja/Shake <b>cell</b> , the topological <b>scheduler</b> , the verifying-trace <b>rebuilder</b>	<b>DELETE</b>	monadic reference resolution rules out the topological scheduler anyway; the closure is 5% and the cold build is 1 second
<b>durability-stratified key space</b> , escalation-to-suspending-scheduler plan	<b>DELETE</b>	machinery for work that does not exist
a <b>lock file as a build artifact</b>	<b>DELETE</b>	there is no build
<b>balanced named trees</b> for unbounded block sequences	<b>DEFER</b>	Wagner & Graham's requirement is real and asymptotic. It becomes load-bearing only when a single document routinely exceeds a few thousand blocks. The corpus's largest is <code>md/graphify-out/GRAPH_REPORT.md</code> at 7,755 top-level blocks <code>[measured, this section]</code> — one file. Revisit when the p99 document exceeds ~2,000 blocks

The anchor invalidation rule, which IS ours to write:

- An anchor is invalidated when the block's **normalized text hash** changes. **NOT when its offsets change** — 80 of the 83 corpus divergences between our two parsers are offset-only (§10.9).
- Mass invalidation is a NORMAL event, not an error.** Inserting five characters (an opening fence, ````\n\n`) after paragraph 3 of a 40-paragraph document collapses **37 of 40 top-level blocks into a single** `FencedCode@129–1754` `[measured]`. The anchor layer must **batch-refuse, mark STALE, and re-resolve lazily on view**. It must never treat this as a failure.
- Semantic (inline-resolved) indexes key on document version, never on block version.** Appending `[foo]: /url` at end-of-file leaves block 1's **bytes unchanged and its top-level type unchanged**, while its mdast inline children go from `[text]` to `[linkReference, text]` `[measured]`. This is Dubroy & Warth's hazard, and it is real — but it lives in the **inline** layer and is **invisible at the byte level**. A content hash over a block's raw text is **stable** across that edit (the anchor survives, which is what we want); any index over resolved inline semantics is invalidated by an arbitrarily distant edit (so that index must key on the whole document, which is expensive — another reason not to build one).

### 10.8 Differential testing

DECIDED — four tiers.

TIER	WHEN	CONTENTS	PASS BAR	ON FAILURE
A	blocking, every PR	CommonMark 0.31.2 spec suite, 652 examples, against the reference parser; plus the GFM extension suite (tables, task lists, strikethrough, autolinks)	652/652 and 100% GFM	build fails
B	blocking, every PR	self-differential over all 1,084 pinned files: (i) micromark block segmentation == Lezer block segmentation modulo a documented tolerance; (ii) Lezer incremental == Lezer full for every single-character edit at 100 sampled positions per file; (iii) splice round trip byte-identical	(i) baseline 997/1,080 with the documented tolerance; (ii) 100.0%; (iii) 100.0%	build fails
C	blocking, every PR	the hostile corpus with budgets asserted	every case returns ok or a NAMED budget refusal within 2x the budget; zero process aborts	build fails
D	nightly, non-blocking	fuzzing: a three-way vote among micromark, markdown-it-py 4.2.0 and Lezer on generated documents, reporting any 2-1 split	report only	file an issue

Tier A's numbers are UNVERIFIED here and must be tagged [secondary] until the suite is vendored. The claim is micromark 4.0.2 at 652/652 (100.0%) and marked 16.4.2 at 531/652 (81.4%) — 121 divergences concentrated in Images (19), Links (17), Setext headings (16), Entity references (12), Thematic breaks (12), Hard line breaks (8). The arithmetic is internally consistent (531/652 = 81.44%, 652 - 531 = 121) but no CommonMark 0.31.2 spec suite is present in this repository ( find . -name spec.json outside node\_modules returns nothing) and an independent verifier could not check it [measured, this section] . First task in tier A: vendor spec.json at a pinned version and re-derive both numbers.

Tier B's documented divergence, DECIDED. A block start offset may differ by up to the block's leading-whitespace run when both parsers agree on type and end offset. Everything else fails the build. This is not a fudge; it is the measured shape of the disagreement (§10.9).

Tier B's denominator is 1,080, not 1,084, and here is why. An audit flagged "997/1,080 (92.31%)" as using a denominator four short of the pin with no explanation. The four missing files are the four zero-byte files in the corpus [measured, this section] . Counting them as trivially identical gives 1,001/1,084 = 92.34%. State the denominator and the reason in the fixture header so nobody re-flags it.

Tier D's oracle, honestly. cmark-gfm and pandoc are not installed on this machine and there is no Homebrew formula present. Every statement anywhere in this plan about the CVE-2023-22484 shape is measured against JavaScript parsers only; the comparison to the C reference implementation is search-only [measured] . Building cmark-gfm as a fuzzing oracle is the one piece of tier D worth a build step, and it should be a separate, optional CI job.

EXPLICITLY OUT: marked as a conformance target. It scores 531/652 and, with default options, emits <script>alert(1)</script> verbatim and href="javascript:alert(1)" [measured] . It is a degradation target in the mdmax cert matrix with its 121 known divergences enumerated. It is never a correctness oracle.

One thing that does not exist and that we would be first to ship. npm search 'babelmark' returns total: 0; npm search 'commonmark differential' returns 1,396 results whose top hits are parsers, not harnesses; babelmark3 is a hosted web service with no published package and no corpus [measured] . Nobody ships "here is a pinned hostile corpus plus a runner plus a divergence report" for markdown. Nor does a markdown parse budget exist: markdown-it has maxNesting (a depth cap, not a time cap), and micromark, remark and marked have no limit of any kind. A parseWithBudget(src, {ms, bytes, blocks}) returning {ok | BUDGET\_EXCEEDED} instead of hanging does not exist on npm [measured] .

And the reason it does not exist is worth recording, because it explains why nobody upstream will fix this for us. The CommonMark specification's entire security surface is two sentences plus one clause, in a 9,811-line document, and both are about U+0000. Line 479 is ## Insecure characters ; grep -n -i "security|sanitiz|denial of service|untrusted" over the whole spec text returns exactly 2 hits [primary, spec.txt version 0.31.2] . There is no section on resource exhaustion, nesting depth, sanitisation, or untrusted input. A compiler will be fed hostile documents and the specification is silent about all of it.

### 10.9 Two parsers, permanently

DECIDED. MDMAX runs two markdown parsers forever and this is not a defect to be resolved:

- micromark 4.0.2 (via mdast-util-from-markdown / remark-parse in v1) for the compiler. It is the conformance reference (652/652, once vendored) and the source of the per-document byte offsets the splice writer consumes.
- @lezer/markdown 1.6.3 for CodeMirror. It is already in memory because CodeMirror 6 is already a dependency, it is the only parser that stayed near-linear on every shape tested, and it carries the incremental reparse the editor needs.

Replacing either with the other is not on the table. CodeMirror cannot consume mdast, and the splice writer cannot consume a Lezer tree without a mapping layer that would itself need conformance tests.

The good news, measured. The two agree far more than "two parsers" suggests, and where they disagree it is an offset problem, not a structure problem [measured, over the corpus] :

non-empty files compared	1080	
byte-identical top-level segmentation	997	(92.31%)
divergent files	83	
start-offset divergence	80	
end-offset divergence	3	
TYPE divergence	0	
total top-level blocks	mdast 107317	lezer 107315 (0.0019% apart)
block COUNT differs on	2 files	(0.19%)
modal case: a 2-character start offset on a `list` whose leading indent Lezer includes and mdast excludes — e.g. md/Courses/WithArin Claude Code/WithArin Claude Code.md, mdast list [40,103] vs lezer list [38,103]		

DECIDED — the shared conformance fixture. Every construct MDMAX understands gets one fixture file run against both parsers:

```
test/fixtures/constructs/<construct>/
 input.md the source, byte-exact, with a trailing-newline marker in the manifest
 expect.blocks.json [{type, start, end}] in U16Offset, the agreed segmentation
 expect.notes.md any DOCUMENTED divergence, with the reason and the date it was accepted
```

The runner asserts `micromark(input) == expect.blocks` and `lezer(input) == expect.blocks` modulo the documented tolerance, and **fails on any type divergence unconditionally**. A construct without a fixture in both parsers does not ship.

**The negative, stated plainly.** This regime protects **block segmentation**. It does **not** protect inline semantics, where the two parsers have never been compared, and where the examined-not-consumed hazard lives (§10.7.3). If comments ever anchor to inline ranges rather than block ranges, this section is insufficient and needs an inline conformance tier that does not currently exist.

## 10.10 What is over-specified and should be deleted; what is under-specified and will bite

### 10.10.1 Delete these. They are machinery for work that does not exist.

DELETE	EVIDENCE
<b>The incremental build system</b> — Ninja/Shake cell, topological scheduler, verifying-trace rebuilder, durability-stratified key space, escalation plan, lock file as a build artifact	cold full lex 1,056 ms; max transitive blast radius 54 files (5.0%); 46.8% of files in zero edges; 0 link reference definitions in 25.5 MB; monadic resolution rules out a topological scheduler anyway <code>[measured]</code>
<b>A bespoke examined-vs-consumed invalidation rule</b>	Lezer <code>TreeFragment.applyChanges</code> is sound on 7 of 7 hand-built cases including a mid-document open-fence insertion that collapses 37 blocks <code>[measured]</code>
<b>Hardening the frontmatter delimiter grammar</b>	gray-matter and the preview's <code>FM_RE</code> disagree on <b>9 of 12 synthetic edge shapes</b> and on <b>0 of 1,084 real files</b> — both accept the same 737 and reject the same 170. Hardening it would have caught <b>zero real bugs</b> . Build the 12-case table later as a <b>regression fence</b> , after the real defects are fixed <code>[measured]</code>
<b>Adversarially attacking the re-anchor's accuracy</b>	no hostile input was found that makes it return a <b>wrong</b> answer; the measurable failure mode is cost, and cost is bounded by refusal <code>[measured]</code>
<b>Hardening against 40 of the 45 hostile shapes</b>	all 40 parsed under 500 ms across every parser with zero crashes. Gate them nightly; do not engineer for them <code>[measured]</code>
<b>The <code>EditableTable / setTableCell</code> write path</b>	delete it, do not fix it. Imported by nothing ( <code>components.tsx:36</code> names the callback <code>_onEdit</code> ), corrupts <code>1</code> cells, 7.4% of what it detects as tables are inside code fences, dirties 13.4% of files on a no-op. It is dead code today, which is the only reason it is not already a live data-loss bug <code>[measured]</code>
<b><code>mdmax unpack</code> in v1</b>	pack is a read transport and needs no inverse. <code>llm-code-format</code> ships pack+unpack at 387 downloads/week against repomix's 327,543 — an <b>846:1</b> revealed-demand ratio against the reverse operation <code>[measured]</code>
<b>The 384-configuration sweep as something to re-run or withhold</b>	it produced three scalars (3-gram shingles, jaccard >= 0.20, margin <= 0.05). <b>Publish them</b> . Withholding buys nothing — a competitor re-sweeps in an afternoon on a bigger public corpus — and costs the credibility of an unverifiable 99.627%

### 10.10.2 These are under-specified and will bite. Ranked by when.

#	HAZARD	WHEN IT BITES	THE FIX, AND ITS SIZE
1	<b>The offset model</b> (§10.2)	the first time a byte offset meets a UTF-16 offset — i.e. the first content hash over a range	<code>offsets.ts</code> + branded types + one lint rule. <b>Half a day</b> . Must land <b>before</b> the splice writer's second caller, because retrofitting branded types after four call sites is a refactor rather than a decision
2	<code>share-writer.ts:26</code> and <code>frontmatter.ts:47</code> (§10.6.3)	<b>every Publish and every properties-panel edit, today</b>	replace both with <code>splice()</code> in the same commit. <b>~40 lines</b> with the boundary assertion, plus a vitest suite that replays 907 files. <code>gray-matter</code> 's <code>matter.stringify</code> re-parsing its argument makes this a <b>silent body-truncation</b> risk, not cosmetic churn
3	<b>The YAML value slot</b> (§10.6)	266 of 1,084 files today — 170 loudly as a 502, <b>96 silently with no error at all</b>	byte-scan splice for writes + lenient pre-pass for reads + the <code>WIKILINK_IN_FLOW_SEQ</code> refusal. Must ship in the <b>same PR</b> as item 2 or the 170-file case stays broken
4	<code>normalize/1</code> and <code>slug/1</code> <b>unversioned</b> (§10.3, §10.4)	the first time either function changes <b>after</b> an anchor is persisted. Before that it is free; after that it is a migration	two files, two <code>VERSION</code> constants, a 200-case golden file, one CI assertion. <b>One afternoon</b> . Anchor health swings <b>77 percentage points</b> on the slug choice alone
5	<code>WIKILINK_RE</code> (§10.5.3a)	the first vault containing 160 KB of unclosed <code>[[</code> , which a half-typed vault produces without an adversary. <b>36,865 ms</b> , blocking the shared server event loop	replace the regex with a bounded index scan. <b>~10 lines</b> . Regression gate: <code>k &lt;= 1.05</code> at 80,000 <code>[[</code>
6	<code>LivePreview</code> <b>parses on the browser main thread</b> (§10.5.3c)	<b>today</b> , on a real file already in the founders' vault: <b>19-31 seconds</b> on <code>md/_Archive/Satinath Content JSON (raw source).md</code>	move <code>splitIntoSegments</code> behind the shape gate and into a Web Worker, or precheck bytes/lines before calling it. This defect was found by neither the researcher nor its verifier
7	<code>The resourceLimits</code> <b>constant</b> (§10.5.5)	whenever somebody "tunes" it upward. 32 MB gives a catchable error; 256 MB gives a SIGABRT that kills the parent	pin it, comment it with the measurement, and regression-test both outcomes
8	<b>The refusal enum</b> (§10.5.6)	the moment the second route handler consumes a parse result	half a day of naming. If it is not named first, every call site grows its own <code>try/catch</code> and the discriminated union never happens
9	<b>Bidi controls reaching the DOM</b>	already live: U+202E and U+202C survive into HTML from fenced code blocks, inline code spans and link text; U+2067/U+2069 from headings; <code>grep -rniE "202e bidi_control" src/</code> returns <b>zero hits</b> <code>[measured]</code>	escape <code>\p{Bidi_Control}</code> or render a visible marker. A compiler that silently passes them through is actively harmful, and this is a code reviewer reading a fenced block to make a decision
10	<b>The frontmatter wikilink population is a THIRD reference kind</b>	as soon as the link resolver ships	2,283 <code>entities:</code> , 1,407 <code>related:</code> , 49 <code>up:</code> wikilinks resolve at <b>13.95%</b> — worse than body links — and the two-tier HARD/SOFT design has no third tier for them. <b>OPEN: is a frontmatter wikilink hard, soft, or a concept name?</b> It is a third of all wikilinks and it is undocumented

### 10.10.3 OPEN questions this section cannot close

1. **Is a frontmatter wikilink a link or a concept name?** (row 10 above). Needs a founder decision, not a measurement.
2. **Does the budget-and-refuse architecture have an acceptable false-refusal rate on AI-generated documents?** (P3 in §10.5.8). The 500-document AI arm does not exist. This is the single most valuable unbuilt fixture in the programme.
3. **Does `blocksFromEvents(micromark.events(src))` avoid materialising an mdast-equivalent structure?** If not, the "never build mdast" prescription is dead and the answer is Lezer for everything. Nobody has tested it.
4. **Is 99.627% real?** Nobody has re-derived it. Eight research areas cite it. **No build item in this plan is allowed to depend on it**, and that is deliberate.
5. **Does the transfer cost of a parsed tree across the worker boundary dominate the parse?** At `docs/engine/PLAN.md` scale it already ties (71.0 ms transfer against 35.5 ms parse). At 20x that size it may dominate, which would replace the worker-with-a-message-boundary design with a `SharedArrayBuffer` flat-tree encoding.

### 10.10.4 The honest stop condition for this whole section

Three of the four highest-value findings here — the `WIKILINK_RE` quadratic, the YAML Date-versus-string divergence, and the `share-writer` regeneration — are **defects in code that already exists**, not in a compiler that does not. A fourth, the `LivePreview` main-thread parse, is the same. **If the team fixes those four and finds that the mdast quadratic does not reproduce on their machine, this entire section was a code review wearing a research costume, and it should be re-run as one.** That would be a good outcome, not a bad one: it would mean the engine's remaining risk is smaller than the record implies, and the work is editor work.

---

§11 Execution

The execution plan — every step, in order, with gates

← Index · ← §10 Engine spec · §12 Risks →

11. The execution plan — every step, in order, with gates

This section says what to do, in what order, who does it, which files it touches, what command proves it done, and what happens if it does not get done. It is written for someone who has never opened this repository. Every path is real and was read on 2026-08-01. Every number carries a source.

Three words, used strictly.

- **DECIDED** — settled. Do not re-litigate without new measurement.
- **RECOMMENDED** — the research points here, but a founder call has not been recorded. Make it.
- **OPEN** — genuinely unresolved. Named so it cannot be quietly assumed away.

**Evidence tags, used on every claim.** [measured] we ran it · [primary] we read the source · [secondary] · [inference] · [SIMULATED] replayed through code, not read from a live system.

**Effort unit.** One **founder-day** = one person, six focused hours. Estimates are for Sagnik at current velocity. The measured baseline for calibration: **532 insertions into src/ in the 19 days between the clone and 2026-08-01**, across three commits — 8eb4de2 364, 58322f7 124, 6ef83a9 44. [measured, final-gate area premortem, N11] That is roughly **28 lines of product source per day**. Every estimate below assumes that number goes up by an order of magnitude once the work is code instead of prose. If it does not, kill gate **K2** (§11.14) is the instrument that says so.

11.1 The sequence at a glance

#	STEP	OWNER	EFFORT	BLOCKED BY	BLOCKS	THE GATE THAT CLOSES IT
0	Repo hygiene — the vitest exclude, the stale records	Sagnik	0.5 d	nothing	nothing (but poisons every gate below if skipped)	<code>npx vitest list --filesOnly   wc -l</code> returns <b>83</b> , not 247
1	The second user — signup, tenancy, per-user attribution	Sagnik (or Amit, see §11.13)	8–12 d	nothing	3, 4, 6, 7, 8	<code>test/tenancy/second-user-writes.test.ts</code> green, having been committed RED
2	The frontmatter splice — fix the two shipped destroyers	Sagnik or one dispatched agent	0.5 d	nothing (parallel with 1)	6 (suggestions), 7	<code>publish</code> → <code>unpublish</code> byte-identical on <b>907/907</b> , no 502 on the <b>171</b>
3	Wedge A — "your vault, in any browser"	Sagnik	15–25 d	1	revenue	a stranger signs up, connects a repo, edits on phone and laptop
4	Share links with roles, expiry, comment box	Sagnik	5–8 d	1, and reopening D3	6	a link with role <code>commenter</code> cannot edit; an expired link 410s
5	<code>mdmax cert</code> — the capped give-away	one dispatched agent	<b>2 d, hard cap</b>	5 conversations (§11.12.7)	nothing	<code>npx mdmax cert</code> on a corpus we did not write; K5 evaluated
6	The review loop — sidecar comment threads	Sagnik + Amit	20–30 d	1, 2, 4	the company	<code>delete</code> the sidecar → <code>git status</code> clean, every file byte-identical
7	pack/unpack — see §11.9, contested	nobody yet	0 d until its precondition fires	2, plus a founder answer and a live-model number	nothing	see §11.9 — it may never open
8	Real-time collaboration	—	—	not this cycle	—	—

Two research lenses disagree about the order of steps 1 and 2, and the disagreement does not matter. The `product-market` lens ranks the second user first and the splice writer third; the `premortem-integrator` lens ranks the splice writer second and the second user third. [primary, final-gate synthesis lenses 1 and 3, ranked arrays] They are disjoint: step 1 touches `src/modules/auth`, `src/config`, `src/container`, `src/app/(auth)`; step 2 touches `src/modules/share/infrastructure` and a new `src/modules/mdmax/domain`. No shared file. Run them in parallel — which is exactly what the module boundary in §11.11 exists to make safe. **DECIDED**.

11.2 Step 0 — Repo hygiene · half a day · do it before anything else

Nothing here is interesting. All of it invalidates a gate downstream if skipped, which is the only reason it is first.

11.2.0 What is already done — a correction to the record

The `premortem-integrator` lens ranks "push the branch, commit the evidence base, and restore `package.json`" as item 0, on the evidence that `origin/main` had not moved since 2026-07-25 and that `git branch -r --contains HEAD` returned zero remote branches. [primary, final-gate synthesis lens 3, ranked[0]]

That is now stale, and this document says so rather than repeating it. Verified live 2026-08-01:

```
$ git for-each-ref --format='%(refname) %(objectname:short)'
refs/heads/engine/plan-and-diagnostics 1bd4dad
refs/remotes/origin/engine/plan-and-diagnostics 1bd4dad ← the branch IS pushed
refs/remotes/origin/main 8eb4de2

$ git rev-list --left-right --count origin/main...HEAD
0 17 ← 17 ahead, 0 behind

$ wc -l package.json
97 ← restored; every script present
```

[measured, run 2026-08-01]

docs/mdmax/PLAN.md , docs/engine/research/ and the HANDOFF-\*.md files are committed and pushed in 1bd4dad . Risk **N8 — the evidence base evaporated** is retired. What remains of item 0 is the list below.

### 11.2.1 The vitest exclude — the largest single stale record in the repo

The problem, measured live rather than quoted:

```
$ npx vitest list --filesOnly | wc -l
247
$ npx vitest list --filesOnly | grep -c '\.claude/worktrees'
164
```

[measured, run 2026-08-01] — **164 / 247 = 66.3968%**, i.e. **66.4% of the test files vitest collects are not in this branch's source tree at all.**

They are in two abandoned git worktrees:

```
$ git worktree list
/Users/.../frontmatter 1bd4dad [engine/plan-and-diagnostics]
/Users/.../frontmatter/.claude/worktrees/competent-bassi-5da9a1 8eb4de2 [claude/competent-bassi-5da9a1]
/Users/.../frontmatter/.claude/worktrees/upbeat-euclid-60dbf4 8eb4de2 [claude/upbeat-euclid-60dbf4]
```

[measured] Both are pinned at 8eb4de2 , **dated 2026-07-25** — the commit *before* any of the engine work. [measured] Both are 17 commits behind HEAD.

**Why vitest sees them and git does not.** .gitignore:12 contains .claude/worktrees/ , so git ignores them. Vitest does not read .gitignore . Its entire default exclusion list is two entries:

```
$ node -e "const c=require('vitest/config'); console.log(JSON.stringify(c.defaultExclude))"
["**/node_modules/**","**/.git/**"]
```

[measured; also readable at node\_modules/vitest/dist/chunks/defaults.9aQKnqFk.js:6]

vitest.config.ts sets environment , setupFiles and a @ alias, and **does not set** exclude **at all** [primary, vitest.config.ts] . So the default applies, .claude/worktrees/\*\* is not in it, and every npm run test in this repo has been running two copies of a month-old test suite alongside the real one.

**Why this is a gate problem and not a tidiness problem.** Steps 1, 2, 4 and 6 below are all gated on "a named test goes from red to green." A suite where two thirds of the files are frozen at a pre-work commit can (a) go green on stale code while the live code is red, (b) go red on a stale file and send someone hunting a regression that does not exist — which is Learned Rule #65's exact failure mode, *"a harness that reports false FAILures trains you to ignore it"* — and (c) mask a deleted test, because the deleted copy still exists in two worktrees.

The fix.

```
// vitest.config.ts
import { defineConfig, configDefaults } from "vitest/config";
import { resolve } from "path";

export default defineConfig({
 test: {
 environment: "node",
 setupFiles: [".test/setup.ts"],
 exclude: [
 ...configDefaults.exclude, // **/node_modules/**, **/.git/**
 "**/.claude/**", // agent worktrees, scratch, settings
 "**/src-tauri/**", // Rust shell, its own toolchain
 "**/.next/**",
],
 },
 resolve: { alias: { "@": resolve(__dirname, ".src") } } },
});
```

**Acceptance gate.** npx vitest list --filesOnly | wc -l returns **83**, and npx vitest list --filesOnly | grep -c worktrees returns **0**. Then npm run test and record the pass/fail count in the commit message — that is the first honest baseline this repo has had.

**Then deal with the worktrees themselves.** They are 17 commits behind and were created by agent runs that finished. **RECOMMENDED:** remove both — git worktree remove .claude/worktrees/<name> for each, then git worktree prune — and adopt the preflight in §11.11.4 so a stale worktree can never be dispatched into again. This is a local, reversible operation on ignored directories; it is still a deletion, so it goes through the RULE 2 confirmation in §11.12.5 like any other.

### 11.2.2 The corpus derivation script

corpus\_id sha256:3a010b1649899795d79274fc528dbece97fdabf4ff0f81cc02ab619c048c51a4 is cited by every measurement in this plan. ls scripts/ shows no derive/ **directory and no derivation script anywhere in the repo.** [measured] Two research areas ( moat , premortem ) failed to reproduce the hash — premortem tried **64 serializations** and failed [primary, final-gate area premortem] — and the against verifier succeeded, recovering the recipe:

sha256 of newline-joined sorted root/path:sha256 , with **no trailing newline.** [primary, final-gate synthesis lens 3, ranked[0].ships\_as]

**Task.** Write `scripts/derive/corpus-id.mjs` (about ten lines), run it, and commit the script plus its captured stdout next to `docs/engine/research/corpus-manifest.json`. Add a `derivation` field to the manifest naming the script path.

**Acceptance gate.** `node scripts/derive/corpus-id.mjs` prints `sha256:3a010b1649899795d79274fc528dbece97fdabf4ff0f81cc02ab619c048c51a4` and nothing else. Until it does, `corpus_id` is a bare hash over a file list, not a pin, and every `[measured]` tag in this document that cites it is one step weaker than it looks. Say so if asked before this lands.

11.2.3 The `docs/engine/PLAN.md` §1.1a contradiction

The same section states two incompatible false-match counts. 12.9% of 32,919 block-versions is 4,247, and 2.66% of that is ~113 false matches; the same section states 0.050% false on 28,170 anchorable blocks and “All 14 false matches were hand-audited, not sampled” — 113 versus 14. `[primary, final-gate synthesis lens 3, ranked[7]]` One is wrong and the document does not say which.

**Task.** Read `docs/engine/PLAN.md` §1.1a, decide which figure is real, correct the other, and add a one-line note recording that the two disagreed. **Do not delete the losing number silently** — the whole point of P2 (§11.12.2) is that a corrected record is more credible than a clean one.

**Effort:** under an hour. **Blocks:** any republication of the 99.627% figure (see K3, §11.14).

11.2.4 The agent litter and the false-failing guards

Untracked at the repo root right now, verified live: `arx.xml`, `cx.html`, `.scratch-carrier.mjs`, `.scratch-lossy.mjs`, `polyg.yml`, `docs/engine/build/__pycache__/`. `[measured, git status --porcelain -uall, 2026-08-01]` None is referenced by any build step. **Task:** delete or gitignore, one commit. Add `__pycache__/` and `.scratch-*` to `.gitignore`.

Separately, two tooling guards are recorded as reporting **false failures**: `workflow-lint.sh` inverts on files larger than the pipe buffer (`grep -q` plus `pipefail` gives SIGPIPE 141), and the loop-guard reports denials for calls that succeeded. `[primary, docs/mdmax/PLAN.md v2.0.0 §9.9]` Both live outside this repo, in the AIOS toolchain. **RECOMMENDED:** do not fix them here. Fix them where they live, or stop running them against this repo. A guard that cries wolf is worse than no guard — Learned Rule #65 again.

11.2.5 One stale record that step 1 will create

`AGENTS.md` §7 currently instructs every agent: “Author commits as Sagnik Mitra <sagnikmitra123@gmail.com> (matches `AUTHOR` in `src/container/dependency-container.ts` for GitHub commits the app makes on behalf of the user).” `[primary, AGENTS.md]` **Step 1 deletes that constant.** Put a line in step 1’s definition of done that updates `AGENTS.md` in the same commit, or the next agent to read it will helpfully re-harden the author.

11.3 Step 1 — THE SECOND USER · 8–12 founder-days · the only step that matters this month

**DECIDED.** This is item 1 in both the `product-market` lens and (as its cheapest high-leverage intervention) the `premortem-integrator` lens. `[primary, final-gate synthesis lenses 1 and 3]`

11.3.1 The state today, verified live

```
src/modules/auth/domain/allowList.ts:9
 return login.trim().toLowerCase() === allowed.trim().toLowerCase();

src/config/env.ts:47
 ALLOWED_GH_LOGIN: z.string().min(1).default("sagnikmitra"),
src/config/env.ts:85-86
 GITHUB_REPO_TOKEN: z.string().min(1),
 GITHUB_REPO: z.string().min(1).default("sagnikmitra/md"),
src/config/env.ts:252
 const allowedLogin = process.env["ALLOWED_GH_LOGIN"] ?? "dev";

src/container/dependency-container.ts:42
 const AUTHOR = { name: "Sagnik Mitra", email: "sagnikmitra123@gmail.com" } as const;
 → fed to commitChanges, createNote, renameNote, uploadAttachment (lines 66, 70, 75, 80)

src/modules/auth/infrastructure/auth-options.ts:82
 return isAllowed(ghProfile?.login, authEnv.ALLOWED_GH_LOGIN);

src/shared/infrastructure/github/client.ts:30 Bearer ${repoEnv.GITHUB_REPO_TOKEN}
src/modules/repository/infrastructure/github-writer.ts:33 Bearer ${repoEnv.GITHUB_REPO_TOKEN}

src/app/(workspace)/ contains exactly one file: .gitkeep
src/app/** no signup, register or onboard route exists
```

`[measured, all read live 2026-08-01]`

`firestore.rules` — 17,304 bytes, committed 2026-07-25 — already declares the entire multi-tenant model: `vaults/{vaultId}/notes/{noteId}/revisions/{revId}`, `memberUids`, `roles{uid}`, `canEditVault`, immutable parent-linked revisions with `allow update: if false, billing/{uid}`, `usage/{uid}/months/{month}`, `shares/{slug}` with an `isValidShare` validator. It contains the string `comments` **zero times**, and the exported `firestore()` helper has **zero callers anywhere in src**. `[primary + measured, final-gate synthesis lens 3, ranked[2]]`

**The schema exists. Zero lines implement it.** That is the whole of step 1: implement a specification that is already written down.

11.3.2 What ships — the RED test, committed first

**Path:** `test/tenancy/second-user-writes.test.ts` — a new directory, no eslint change needed (`boundaries/elements` matches `src/modules/*/...` by glob, and `test/**` is not an element at all `[primary, eslint.config.mjs:80-96]`).

**Commit it RED on day one, before any implementation.** The commit message must say `RED:` and name the assertion. It asserts, in one test:

1. A GitHub login that is **not** `sagnikmitra` completes sign-in and reaches a workspace. (*Fails today at* `auth-options.ts:82`.)
2. That user opens a document in a vault whose `memberUids` includes them but whose owner is someone else, and writes one character.
3. The persisted commit's author is **their** name and email — asserted by reading the commit back — **not** `{ name: "Sagnik Mitra", email: "sagnikmitra123@gmail.com" }`. (*Fails today at* `dependency-container.ts:42`.)
4. The write was authorised by **their** credential, not a shared server token. (*Fails today at* `client.ts:30` and `github-writer.ts:33`.)
5. The same user receives **403** on a vault where they hold no role. (*No role model is wired today.*)

**Why this shape and not "write a multi-tenancy epic".** A standing rule that says *"one committed assertion goes red to green against engine code" can be satisfied by writing a compiler.* An assertion that names a second human cannot be satisfied by prose and cannot pass without deleting the allowlist, the single server token and the `AUTHOR` constant simultaneously. `[primary, final-gate area premortem, "the single cheapest intervention"]`

11.3.3 The work, decomposed

#	TASK	FILES	EFFORT
1a	Commit the RED test	<code>test/tenancy/second-user-writes.test.ts</code>	0.5 d
1b	Delete the allowlist; replace the <code>signIn</code> callback with account creation	<code>src/modules/auth/domain/allowlist.ts</code> (delete), <code>../infrastructure/auth-options.ts</code> , <code>test/auth/allowlist.test.ts</code> (delete)	1 d
1c	User + vault documents on first login	new <code>src/modules/tenancy/{domain,application,infrastructure}</code> , <code>src/shared/infrastructure/firebase/</code>	2 d
1d	Per-user GitHub credential — replace one server token with the signed-in user's OAuth token	<code>src/config/env.ts</code> (drop <code>GITHUB_REPO_TOKEN</code> default path), <code>src/shared/infrastructure/github/client.ts</code> , <code>src/modules/repository/infrastructure/github-writer.ts</code>	2–3 d
1e	Per-user attribution — <code>AUTHOR</code> becomes a parameter	<code>src/container/dependency-container.ts:42,66,70,75,80</code> , the four use-cases, <code>AGENTS.md</code> §7	1 d
1f	Repo picker + onboarding route	<code>src/app/{auth}/</code> , <code>src/app/{workspace}/</code> (currently <code>.gitkeep</code> only)	2–3 d
1g	Role enforcement — <code>canEditVault</code> server-side, matching <code>firestore.rules</code>	<code>src/modules/tenancy/application/</code> , every <code>src/app/api/**</code> write route	1–2 d

`src/` today is **212 files, 21,415 LOC** `[primary, final-gate area product-gap, effort block]`. Step 1 touches roughly a dozen of them and adds one module.

11.3.4 The backend contradiction, resolved here

`docs/Frontmatter-Product-Plan.md` §4 Phase 1 specifies *"Supabase foundation: users, vaults, sessions, share-links tables"* and names Supabase eleven times `[primary]`. The shipped code and `firestore.rules` are **Firestore** `[measured]`. **These contradict, and Firestore governs** — because 17,304 bytes of Firestore security rules exist, are committed, and already model exactly the schema step 1 needs, whereas the Supabase design exists only as prose. `[DECIDED; matches docs/mdmax/PLAN.md v2.0.0 §10.11]` **Correct** `Frontmatter-Product-Plan.md` **in the same week** so the next reader does not build the wrong half.

11.3.5 What could go wrong, and what would falsify this step

- **The estimate is the weakest number in this section.** 8–12 founder-days is an `[inference]` from the task decomposition above. It is not measured, it has no comparable, and the only velocity datum available — 532 lines in 19 days — argues it will take longer. If step 1 is not green by 2026-08-31, **K1 fires** (§11.14) and the honest reading is that the estimate was wrong by more than a factor of two.
- **1d is the risky task, not 1b.** Deleting a string equality is trivial. Replacing one server token with per-user OAuth tokens touches every read and write path in the product and changes the rate limit model, the error surface and the offline story at once. If it stalls, an acceptable interim is a GitHub App installation token per vault — but say out loud that it is interim.
- **What would falsify the whole step:** if a second login can already write under their own attribution, the premise is wrong. Test it before believing this section. It cannot today — `auth-options.ts:82` rejects them before anything else runs `[measured]`.

11.4 Step 2 — THE FRONTMATTER SPLICE · half a founder-day · parallel with step 1

**DECIDED.** This is a live data-corruption defect in the feature the company is named after, firing on every publish and every unpublish.

11.4.1 The defect, read live

`src/modules/share/infrastructure/share-writer.ts` — its own file header says the adapter works by *"splicing the key into the YAML frontmatter."* Its body does this:

```
const parsed = matter(file.content);
const data: Record<string, unknown> = { ...(parsed.data as Record<string, unknown>) };
if (slug === null) { delete data["public_slug"]; } else { data["public_slug"] = slug; }
const next = matter.stringify(parsed.content, data); // ← regenerates the entire block
```

`[measured, read live 2026-08-01]`

**The header and the code contradict each other, verbatim.** `matter.stringify` re-serialises the whole YAML block from a parsed object, which is regeneration, not splicing — and it violates **D7 (splice-only writing)**.

**What it costs, measured:**

MEASUREMENT	FIGURE	SOURCE
frontmatter-bearing files whose bytes change on a <b>no-op</b>	<b>84.5% of 737</b> (623 files; 197 change line count)	area <code>rendering-frontier</code>
frontmatter blocks rewritten	<b>94.3%</b>	area <code>product-gap</code>
byte-identical after publish	<b>33 / 907 = 3.64%</b>	area <code>product-gap</code>
byte-identical after <b>publish then unpublish</b>	<b>17 / 736 = 2.31%</b> (verifier re-derivation)	<code>product-gap</code> <code>verifier</code>
bare <code>YYYY-MM-DD</code> silently coerced to an ISO timestamp	<b>624 / 736 = 84.78%</b> of parsed files	area <code>product-gap</code>
files that <b>502</b> because <code>gray-matter</code> throws and <code>share-writer</code> has no <code>try/catch</code>	<b>170 of 907</b>	areas <code>product-gap</code> , <code>premortem</code>
silent body loss: <code>gray-matter/index.js:161</code> re-parses a <b>string</b> first argument, so passing <code>parsed.content</code> means a body opening with a <code>---</code> block has that block consumed as frontmatter and <b>dropped</b>	qualitative, verifier-found, unreported before this run	<code>premortem</code> <code>verifier</code>

[measured, corpus\_id sha256:3a010b16...]

The 171-file failure has a named cause: Obsidian's `related: [[a]], [[b]]` is **not valid YAML**, and **170 of 907 blocks (18.7%) parse under neither YAML library**.  
[measured]

11.4.2 What ships

`spliceFrontmatterValue(src: string, key: string, value: string | null): string` — locate the byte range of the `key` 's value node (or the insertion point after the last key), replace **only** those bytes, return the string. **Never call** `matter.stringify` . **Never call** `toString()` **on a YAML Document**. [DECIDED]

**Where it lives.** New module `src/modules/mdmax/domain/splice-frontmatter.ts`, exported from a new barrel `src/modules/mdmax/index.ts`. It is a pure function, so it belongs in `domain` — which under `eslint-plugin-boundaries` may import only `domain` and `shared-domain` [`primary`, `eslint.config.mjs:109–111`], and that constraint is a feature: it makes the splice writer impossible to contaminate with I/O. `share-writer.ts` is `infrastructure`, which **is** allowed to import `domain` [`primary`, `eslint.config.mjs:123–125`], and per `AGENTS.md` §2 it must import through the barrel `@/modules/mdmax`, never the deep path.

`yaml@2.9.0` is **already a dependency** [`primary`, `package.json`] and exposes `node.range` on every AST node — use it to find the byte range, then throw the Document away without ever serialising it.

**DO NOT take the shortcut of swapping** `gray-matter` **for** `yaml` **and calling** `toString()`. The `yaml` library's own no-edit round trip is byte-identical on only **119 of 907** blocks, and **170 files do not parse at all**. [measured] It is a different regenerator, not a splicer.

11.4.3 The gate

**Committed RED first.** Against the current writer the test will fail at **17/736**.

```
Given every one of the 907 frontmatter-bearing files in corpus_id sha256:3a010b16...
 publish(file) → unpublish(file) must be BYTE-IDENTICAL to the original – 907/907.
 publish(file) must not throw – including on the 171 that no YAML parser reads.
Checked by an oracle that shares NO code with the writer.
```

**"Shares no code with the writer" is load-bearing.** Learned Rule #60: a verifier written in the same session as its subject inherits that session's blind spots. Concretely: the oracle re-reads the file from disk and compares raw bytes with `Buffer.compare`; it must not import `spliceFrontmatterValue`, `yaml`, or `gray-matter`.

**Register it in the gate registry (§11.12.3) with its `--broke` written first:** the `--broke` reintroduces `matter.stringify` and **must go red**. A gate that cannot be proven to fail is not a gate.

11.4.4 The three parsing defects, same commit series

The `parsing-robustness` area's own honest verdict: *"this entire area was a code review wearing a research costume and should be re-run as one."* [primary] Take the code review:

DEFECT	FILE	MEASUREMENT
<code>WIKILINK_RE</code> is strictly quadratic, k=2.00	<code>src/modules/vault/infrastructure/markdown-parser.ts:12</code>	verifier reproduced 577.4 / 2,282.5 / 9,258.1 ms at 20k/40k/80k occurrences; <b>11,012.9 ms on a realistic 749 KB half-typed vault</b> . <code>parseMarkdown</code> is <b>server-only</b> , so this blocks the Node event loop, not the browser main thread
<code>gray-matter / js-yaml</code> (YAML 1.1) vs <code>eemeli/yaml</code> (1.2 core) diverge	both parsers	<b>625 of 737 parsed files across 973 occurrences</b> ; one class — dates coerced to <code>Date</code>
<code>share-writer</code>	above	step 2 proper

[measured, final-gate synthesis lens 3, ranked[10]]

**Ships as:** three commits plus an `eslint-plugin-redos` run in CI. **Then close the area.**

**The architectural finding worth keeping:** do not build an mdast tree for a whole document — `mdast-util-from-markdown` is super-linear on flat bullet lists where micromark and Lezer are not. **But the numbers do not survive:** the verifier measured **359,959 ms against a claimed 196,096** (1.84x), found two runs of identical code disagreeing by 1.8x, and found micromark **also** super-linear on nested blockquotes at k=2.42. [measured] The parser that stayed near-linear on every shape tested was **Lezer, already in the dependency tree via CodeMirror** [`primary`, `package.json: @codemirror/*`]. Quote the prescription, never the milliseconds.

11.5 Step 3 — WEDGE A: "your vault, in any browser" · 15–25 founder-days · the paid beachhead

**DECIDED as the first revenue step.** The founders wrote this ranking themselves, before the engine programme started: `docs/FRONTMATTER-PRODUCT-PLAN.md:51–53` ranks the wedges **A > C > B** and says of Wedge A: *"Nearly sellable from the existing codebase once multi-tenancy lands."* [primary]

**Demand:** pain 9 (*No browser access / platform gaps*) carries severity **"Kills at evaluation"**, with a 63-comment thread that *"asks for exactly this."* `forum.obsidian.md/t/2049` stands at **246,568 views / 928 likes** (a second area recorded 249,616 views — the discrepancy is a fetch a few hours apart; quote the range, not a point). `[primary]`

**Already shipped and reusable** `[primary, repo inventory]`: CodeMirror 6, the full Obsidian dialect, `merge3` with 14 conflict tests, git history and restore, MiniSearch, PWA + Tauri.

**Blocked by:** step 1, entirely. **Exit criterion**, taken verbatim from the product plan: *"a stranger signs up, connects a repo, edits on phone + laptop, and their Obsidian desktop app sees the same files."* `[primary, FRONTMATTER-PRODUCT-PLAN.md §4 Phase 1]`

**Pricing — RECOMMENDED, not decided.**

TIER	PRICE	ANCHOR
Free	browser vault + git sync + publish	—
Pro	<b>\$4/mo billed annually</b>	exact parity with Obsidian Sync Standard; inside the forum-stated \$2–4 ceiling
Work	<b>\$50/user/year</b>	mirrors Obsidian Commercial; aimed at the population pain 9 defines — cannot install software on a work machine, can expense a licence

`[primary, final-gate synthesis lens 1, ranked[1].ships_as]`

**One contradiction to settle before the pricing page is written.** `FRONTMATTER-PRODUCT-PLAN.md:171` promises *"Free git-backed sync forever (user's own repo = backend; marginal cost ~0)"* against pain theme T3, **ranked #3 of 14**. **D4 explicitly rejects "your git repo is the backend."** `[primary]` The `premortem` area logs this as risk **N10 — two decisions deleted the wedge the demand research identified** and notes it *"is not logged anywhere."* `[primary]` **RECOMMENDED resolution:** D4 governs as a **durability promise** (we owe an export and a migration, not an architecture), and the free-tier sentence is rewritten to promise free sync into a repo the user owns without claiming the repo is the backend. **OPEN** until a founder writes it down.

### 11.6 Step 4 — SHARE LINKS with enforced roles, expiry and a comment box · 5–8 days

**The state today:** `grep` across `src/modules/share` for `expiry|expires|password|role` returns **only a reserved-slug list and ARIA dialog roles** — no capability exists. `[measured]` Share is a public slug with no roles, no expiry, no password. `firestore.rules` already has `match /shares/{slug}` with an `isValidShare` validator, so the storage layer is specified. `[primary]`

**Ships as:** roles `viewer | commenter | editor`, revocable, expiring, optional password, plus a comment box that writes into the sidecar of step 6.

**This step reopens D3.** D3 says reviewers must log in. The competitive research names **anonymous, no-account share links** as the wedge — HedgeDoc (working proof of concept with a live demo, 2026-04-04), **Relay at 176,247 installs** (a second area recorded 175,904 — again, a fetch gap; quote the range), Peerdraft, HackMD. `[primary]` D3 removes exactly that property.

**RECOMMENDED, and both lenses that touched it agree:** reopen D3 as a **tier split**, not a global rule — **anonymous read-and-comment on a share link; login required to resolve a thread or to edit.** Owner seats bill; commenters do not. This preserves the wedge and the account model at once. `[primary, final-gate synthesis lens 1 ranked[3].ships_as and lens 3 N4 countermeasure]`

**A pre-registered prediction that can settle it empirically:** the `product-gap` area predicts that **four of five reviewers stall at the login wall** `[primary, prediction M3]`. Run the five conversations (§11.12.7) **before** this step. If M3 fires, D3 becomes a product decision made at auth, not an auth decision deferred.

**Gate:** a link issued with role `commenter` returns 403 on write; an expired link returns 410; a revoked link returns 404 within one cache TTL. **Blocked by:** step 1, and a written D3 decision.

### 11.7 Step 5 — `mdmax cert` · two days, hard cap · the capped give-away

**DECIDED: build it. DECIDED: it is never the spearhead.**

**Why it exists.** It is the only capability in the programme that four independent adversarial prior-art searches could not find shipped:

- `moat` ran three GitHub repo searches returning `total_count` **0, 0 and 1** (the 1 an unrelated MCP tool); npm lookups for `babelmark / markdown-compat / markdown-conformance` all not-found.
- `market-and-gift` enumerated `markdownguide.org`'s `_data/tools.yml`: **exactly 28 construct ids**, and `frontmatter`, `wikilink`, `callout`, `mermaid`, `math`, `directive`, `embed` and `attribute` appear **zero times** — against **83.67%** frontmatter prevalence in the pinned corpus and **56.75%** in an independent one.
- `premortem` ran four npm searches (markdown+conformance, markdown+compatibility, commonmark+diff, markdown+renderer+comparison) → **32 results, zero conformance tools**.
- `rendering-frontier` measured **GitHub's own two renderers disagreeing on identical bytes, live:** `POST /markdown` renders mermaid as highlighted source while `github.com` blob renders the diagram, and inline `<svg>` is **deleted to an empty paragraph**.

`[primary + measured, all four areas]`

**Why it is capped.** It targets the founders' own pain themes **13 and 14 of 14**. The `market-and-gift` verifier found T13's severity is *"Churns the purist early adopters"* — **worse** than the headline's "Annoys", which is T14 on the next line — and **still ranked 13th**, while the pains that make people pay rank 1–9. `[primary]` The `degradation-certificate` area's own unverified block opens: *"TARGETS I NEVER RAN — this is the biggest gap and it is more than half the brief's list"*, and admits **no empirical Obsidian data at all; 7 of ~12 target surfaces cannot be probed at all**. `[primary]` One prior art the searches missed: `ArchieCur/MARKDOWN_FLAVORS`, a **14-flavor x 17-construct** matrix — static, not per-document, which narrows the novelty claim without eliminating it. `[primary, moat verifier]`

**Ships as:** `npx mdmax cert`, plus a GitHub Action and a Claude Agent Skill. Per document, per target, per construct: **PASS · STRIP · CORRUPT**. Targets modelled as `(product, surface)` pairs. `--json`, exit codes, version-pinned **offline** renderers.

**Two labels are mandatory in the output:** a **PROBED** tier (offline, vendored, version-stamped) and a **DECLARED** tier (contract-plus-canary for closed SaaS), so the certificate never manufactures confidence about Obsidian, Notion, Slack or Discord. `[primary, premortem N9 countermeasure]`

**Diagnosis free, repair paid.** The last output line reads: *"N of your files will CORRUPT on GitHub Pages — fix them at frontmatter."*

**Blocked by:** the five conversations (§11.12.7). They can tell you nobody wants it.

**Its own kill numbers** — both are in §11.14 as **K5** and **K5b**, and both previously had no threshold, which meant they could never fire.

### 11.8 Step 6 — THE REVIEW LOOP · 20–30 founder-days · the thing twelve months buys

**DECIDED as the destination.** Highest defensibility in the set, and the only intersection four competitor sweeps found empty.

**The single most consequential correction in the entire research corpus.** The `against` area reported demand favouring real-time collaboration over commenting **6.9:1** and built its whole strategic recommendation on it. **Its own verifier recounted symmetrically** (de-confounding URL and score metadata) and got **review-loop 169 leaves to collaboration 95 — 1.78:1 the other way.** `[measured, final-gate synthesis lens 3, contradictions[3]]` That flips the recommendation from “*build Yjs*” to “*build comments*”. **Anyone quoting `against` 's strategic conclusion is quoting a regex artifact.**

**A caution the honest version of this plan must carry.** A second, tighter count in the same corpus gives **108 leaves** for realtime/multiplayer/collab against **25** for the unambiguous suggestion/track-changes/annotate family — i.e. 4.3:1 back toward multiplayer. `[measured, final-gate synthesis lens 1, ranked[5].evidence]` **The two counts contradict, and the difference is what counts as a “review loop” leaf.** The symmetric recount governs the *decision*, because it is the one that was adversarially audited. But do not present 1.78:1 as settled. This is **defensibility-led, not demand-led** — build it because the intersection is empty, not because the corpus proves people asked.

**The empty intersection, from live fetches** `[primary, area product-gap]` : HackMD ships the full loop free but has no local files or vault. Moment.dev ships realtime on git-backed `.md` and `grep -ioE 'comment|suggest'` over its homepage, docs and pricing returns **0**. Obsidian's own help file for shared vaults states plainly: no cursors, no fine-grained permissions, no comments. Craft and Bear homepages contain **0** occurrences of “comment”.

**The clock.** `inkeep/OpenKnowledge` (**3,239 stars, 14,790 npm downloads/week, GPL-3.0**) merged content-derived comment anchoring on **2026-07-30** and published `0.46.0-beta.32` at **2026-08-01T02:38:33Z** — six minutes before the researcher's first tool call. Its own changeset says the comment is “*a note to your own agent, not a message to a teammate.*” `[primary, area competitive-live]` **They have the hard engineering and have declined the market. That gap closes the day they change their mind.**

**Ships as.** A `comments` collection — **absent from** `firestore.rules` **today**, so this data model must be written — with `{ anchor: { prefix, exact, suffix }, quotedFileContent, resolved, replies[] }`. **Copy Google Drive v3's field set rather than inventing one.** The anchor is a **W3C** `TextQuoteSelector`, **a Recommendation since February 2017** — not an invention. `[primary]`

**The resolver: a three-tier cascade, and delete everything else.** `[DECIDED]`

TIER	MECHANISM	WHEN	EVIDENCE
1	<code>ChangeSet.mapPos</code>	in-session edits	exact by construction; CodeMirror is already a dependency <code>[primary]</code>
2	LCS over normalized block text	across revisions	<b>86.658%</b> (verifier independently re-derived <b>85.036%</b> ) vs hash-alone <b>77.297%</b> , on <b>29,217 anchorable block-versions</b> from the pinned corpus <code>[measured]</code>
3	content-derived, <b>scoped to blocks of ≤10 tokens</b> , refusing rather than guessing	last resort	the short-block stratum is where everything fails: <b>44–47% of 3–5 token blocks are ambiguous; 0 of 13,426 twenty-plus-token blocks are</b> <code>[measured]</code>

**Never a position tiebreak.** Measured at **22.20% false** — and it is the exact thing the shipped competitor still does. `[measured]`

**Orphan visibly, never guess. Ship the orphan rate as a visible product metric from the first comment written**, and instrument the refusal path *before* the resolve path. `[primary, premortem N4 countermeasure]` The stated threshold: “*if orphaned threads exceed ~10% of all threads per week of normal editing, the honest-refusal design is correct and unshippable at the same time.*” `[primary, area collab-primitives]`

**D2 gate, non-negotiable:** delete the entire sidecar → `git status` is clean and every `.md` file is byte-identical. Nothing about comments is ever written into the file.

**Do NOT sell it as Google Docs suggestions.** Google Docs carries `suggestedInsertionIds` on every `TextRun` and supports nesting; a plain `.md` under **D6** cannot. Ship suggestions as **base-sha plus hunks, accepted by the step-2 splice writer**, and say so in the marketing copy. `[primary, final-gate synthesis lens 3, ranked[6].ships_as]`

**Blocked by:** step 1 (absolutely), step 2 (for the suggestion path), step 4 (for the surface), and **§11.12.8 before anyone quotes a reliability number to a customer.**

### 11.9 Step 7 — pack / unpack · sequenced here, and contested

**Sequenced at position 7 per Section 3 — after the splice writer, never before it.** And this document must state plainly that **the research disagrees with sequencing it at all.**

**The contradiction, both sides named.** The `container-thesis` area recommends shipping `mdmax pack` beside `mdmax cert`. The `against`, `premortem` and `founder-thesis` areas — and `founder-thesis` 's **own verifier** — say never. The `premortem-integrator` lens lists it under “**NEVER BUILD**”, and **that lens governs**, for four reasons that are measurements and not opinions:

- The round trip is unsafe.** `container-thesis` proved the engineering (**1084/1084 byte-identical, +0.892% overhead, 120× cheaper than a zip per git edit**) and then killed its own product in the same report: on the realistic merge-across case, **boundary repair is 119/120 WRONG** — it returns a confident position that does not contain the true split. `[measured]`
- Revealed demand is 846:1 against.** `llm-code-format` already ships pack **and** unpack, zero dependencies, nine LLM output conventions — **387 downloads/month against repomix's 327,543.** `repomix` issue #71 has been open **23 months**; the maintainer gated it on evidence, the contributor reported back “*not quite what I hoped*”, and the last comment is **2025-09-08.** `[primary]`
- The scale is wrong at our own vault.** Container **25,548,765 B** vs a one-line-per-doc index **105,538 B — 242×**. A 200K-token window holds **2.82%** of the container and **682%** of the index; even a 1M window holds **14.1%.** `[measured, corpus_id sha256:3a010b16...]`
- The only live-model measurement of a projection bundle anywhere** (okf-skills, 2026-07-27) found the bundle arm spent **+5.2% MORE tokens** and opened **MORE** files. `[primary]`

**Therefore step 7 is not a task. It is a precondition with a task behind it**, and both parts of the precondition must fire before a line is written:

- **P7a — the founder question, asked verbatim.** *"Was the export you meant always a zip of clean .md files?"* [primary, founder-thesis kill-decision K-D] If yes, this closes in ten minutes, and what survives is the **D4 escape-hatch zip** — which the container's disqualifying properties do not apply to, and which is what a departing human actually wants. export-vault-zip already exists ( src/modules/vault/... , test/vault/export-vault-zip.test.ts ) [measured] .
- **P7b — a live-model number, on a corpus the model cannot read whole**, showing a packed projection costs **fewer** task tokens than the file tree. Zero live model calls have been made anywhere in this programme [primary, stated in every area's unverified block] , so this number does not exist in either direction. Until it does, **\$4.4's 25–168x figures must be stated as artifact-size ratios and must never imply a token saving in an agent loop.**

If P7a returns "no, I meant one file" and P7b returns a positive number, then and only then: mdmax pack ships as a **read-only projection** — never the write surface, never with boundary repair, and unpack is exercised by the step-2 splice writer, which is why it is sequenced after it.

**Effort until then: zero.** Effort after: 3–5 days, and it should be re-estimated, not inherited.

### 11.10 Step 8 — REAL-TIME COLLABORATION · not this cycle

**DECIDED: not this cycle.** [final-gate synthesis lens 1, ranked[6].ships\_as: "Not this cycle."]

It is the largest remaining build, it fights **D6** and **D7** hardest, and there is a specific architectural fact to be paid for knowingly: **Outline solves the whole loop with a ProseMirror comment mark inside a Yjs document and markdown as a projection — which makes anchoring vanish entirely.** [primary] Refusing that is defensible under D6, but it is a cost, not a free choice.

**Competitive floor if it is ever built:** Moment.dev ships realtime on git-backed .md at **\$30/mo for five seats**; a Teams tier at **\$5–8/seat/mo** sits under HackMD Prime at \$5 and far under Notion Business at \$20. [primary]

**Revisit after step 6 reports an orphan rate**, not before. If the orphan rate is intolerable (**K6**), the answer may be that this product needs Yjs after all — and that is a decision made on data instead of on preference.

### 11.11 Where the work happens

#### 11.11.1 The repo and the accounts

SURFACE	OWNER	NOTE
GitHub	studiozephyrus/frontmatter	a personal User account, <b>not an org</b>
Vercel	team zscot ( team_R5Lkvg8AqX8hr5WIuKXlNXGE ), project frontmatter	
Firebase	frontmatter-md under studiozephyrus@gmail.com	
Cloudflare	<b>Sagnik's personal account</b> , zone frontmatter.in	a deliberate split
origin	studiozephyrus	
sagnik-old	pre-migration remote, kept as rollback	refs/remotes/sagnik-old/main = 8eb4de2 [measured]

[primary, AGENTS.md §6b]

**Credentials.** .git/config holds a credential helper reading \$GH\_TOKEN\_ZEPHYRUS from the environment; the token is never written into the repo. Source it in the **same** shell invocation:

```
source /Users/sagnikmitra/.config/codex-env/tokens.zsh && git push
```

A push failing with *"Repository not found"* almost always means the token was not sourced and the macOS keychain answered with the personal account. **Never** run gh auth login or gh auth setup-git with the Zephyrus token — both rebind every repo on the machine to the wrong identity. Pass it per command: GH\_TOKEN="\$GH\_TOKEN\_ZEPHYRUS" gh <cmd>. Same for Vercel: always --scope zscot --token "\$VERCEL\_TOKEN\_ZEPHYRUS", never the bare VERCEL\_TOKEN. [primary, AGENTS.md §6b] **Never print a token value.**

#### 11.11.2 Branch strategy

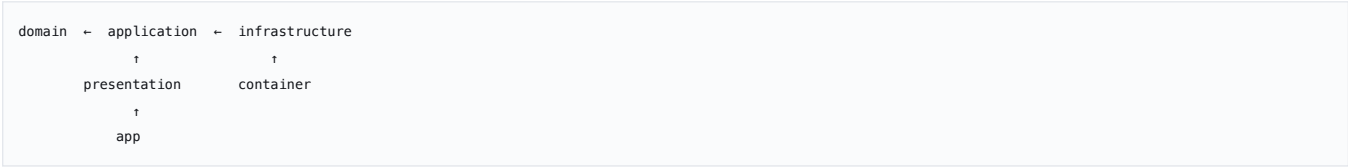
- **main** is **auto-deployed** by Vercel. The ignored-build-step diffs the commit range and skips redeploys when only docs change; app code always redeploys. [primary, AGENTS.md §7, scripts/vercel-ignore-build.sh]
- **One branch per step**, named step-N/<slug> ( step-1/second-user , step-2/frontmatter-splice ). Small enough to review, large enough to carry its gate.
- **The RED commit is the first commit on the branch**, and its message begins RED: .
- Merge to **main** only when npm run verify is green **and** the step's registered gate is green **and** that gate has been proven to go red under its --broke (§11.12.3).
- engine/plan-and-diagnostics (currently HEAD, 1bd4dad, 17 commits ahead of origin/main) is the documentation branch. **RECOMMENDED:** merge it to **main** this week so **main** stops being a month stale and so the plan is the thing everyone reads.

#### 11.11.3 The module boundaries — the thing that makes parallelism safe

Hexagonal modular monolith, enforced by eslint-plugin-boundaries **as errors**, not warnings [primary, eslint.config.mjs:100–146]. Element types are matched by glob, so **a new module needs no config change**:

domain	src/modules/*/domain	- may import: domain, shared-domain
application	src/modules/*/application	- + application, config (never infra)
infrastructure	src/modules/*/infrastructure	- + infrastructure, config
presentation	src/modules/*/presentation	- domain, application, presentation (never infra/container)
container	src/container	- everything except app
app	src/app	- presentation, DTOs, container, config
middleware	src/proxy.ts (file)	- application, infrastructure, config

Direction is **inward only**:



Cross-module imports go **through the barrel** `@/modules/<name>`, never the deep path — including for types. `@/server/*`, `@/lib/*`, `@/components/*` are permanently banned by `no-restricted-imports`. `[primary, AGENTS.md §2–3, eslint.config.mjs:148–152]`

**Existing modules** `[measured, ls src/modules]`: `ai`, `ai-tools`, `app-shell`, `auth`, `drafts`, `editor`, `export`, `graph`, `preview`, `repository`, `share`, `vault`. **Two new ones are proposed here**: `tenancy` (step 1) and `mdmax` (step 2).

**The disjoint-artifact map — this is the parallelism contract.** Learned Rule #20: any parallel branch must operate on disjoint, explicitly-named artifacts.

STEP	MODULES IT OWNS	MUST NOT TOUCH
1 second user	<code>auth</code> , new <code>tenancy</code> , <code>config</code> , <code>container</code> , <code>app/(auth)</code> , <code>app/(workspace)</code>	<code>share/infrastructure</code> , <code>mdmax</code>
2 splice	new <code>mdmax/domain</code> , <code>share/infrastructure/share-writer.ts</code> , <code>vault/infrastructure/markdown-parser.ts</code>	<code>auth</code> , <code>config</code> , <code>container</code>
4 share links	<code>share</code> (all layers), <code>app/api/share</code>	<code>mdmax/domain</code>
5 cert	<code>scripts/</code> + fixtures only — <b>no</b> <code>src/</code> <b>writes at all</b>	everything
6 review loop	new <code>comments</code> module, <code>firestore.rules</code>	<code>mdmax/domain</code>

**One agent per module boundary, never two agents on the same file.** `[primary, final-gate area aios-transfer, design item 4]`

#### 11.11.4 Worktrees

Two exist and both are stale (§11.2.1). The rule going forward, from `aios-transfer`:

- A preflight refuses to dispatch an agent into a worktree more than N commits behind** `main`. Set **N = 0** for any step that touches `src/`; a stale worktree is how step 2's gate silently measures the wrong writer. Set N = 5 for docs-only work.
- Capture** `git rev-parse HEAD` **before AND after every parallel run and reconcile the delta.** Learned Rule #48 measured that a prompt-level “*do not commit*” is **advisory, not a gate**: five subagent commits landed against an explicit prohibition. `[primary]`
- Environment fingerprint on every gate row.** Learned Rule #65: the same battery scored 67/0 from one directory and 55/12 from another, same code, same commit, because macOS `/bin/bash` 3.2 writes heredoc temp files to `/tmp` and the sandbox denies it under some working directories. A green from a different environment is a **different measurement** and must be visibly so.

Effort for the worktree preflight and HEAD reconciliation: **~3 hours, and it lands *before* the first parallel agent run, not after.** `[primary, aios-transfer effort block]`

## 11.12 The workflow — how a task goes from backlog to merged

### 11.12.1 The research→build boundary rule

**Nothing enters the build backlog on a headline.** A backlog item requires three fields:

```

- id: S2-splice
 claim: "publish-then-unpublish is byte-identical on 17/736 files"
 corpus_id: sha256:3a010b1649899795d79274fc528dbece97fdabf4ff0f81cc02ab619c048c51a4
 repro: "node scripts/derive/roundtrip.mjs --corpus corpus-manifest.json"
 verifier_verdict: CONFIRMED # or OVERSTATED / REFUTED / UNAUDITED

```

**The backlog refuses any item whose headline appears in the run's own** `killed` **list.** That single conditional is what the previous run was missing, and it costs one `if`. `[primary, aios-transfer design item 5 – "highest Leverage per minute of anything here", under an hour]`

**Why this rule and not "read carefully":** the 18-area capability run had a **100% headline defect rate** — every headline refuted by its own verifier, always flatteringly. The 16-area final gate, after structural fixes, produced **CONFIRMED 5 · OVERSTATED 10 · REFUTED 1**, with a kill audit over 105 sampled kills giving false-kill rates of **17.2% / 3.4% / 19.0% / 0%**. `[primary, docs/mdmax/PLAN.md v2.0.0 §0]` **The verifiers are roughly 88% reliable, not 100%.** A kill is strong evidence, not proof — which is why **P4** below exists.

11.12.2 The standing rules, and the failure each one came from

#	RULE	THE FAILURE THAT PRODUCED IT
P1	Every figure names a <code>corpus_id</code>	seven irreproducible file counts
P2	No headline without a committed derivation script <b>and its captured output</b>	a <code>101.1x</code> ratio propagated with no script anywhere
P3	Parallel work only over disjoint artifacts, through one reconciliation gate	five designs built on a carrier two measurements had already broken
P4	Verifier kills get <b>sampled and re-audited</b> before anything is removed	measured false-kill rate up to <b>19%</b>
P5	Every byte written into a user's file passes a <b>placement fixture</b> first	a marker next to a setext heading silently turned a heading into a paragraph plus a rule
P6	A design rule may only be derived from an operation the engine <b>performs</b>	a normative rule derived from a <code>stringify</code> round trip, in a splice-only engine
P7	Freeze and <b>version</b> <code>normalize()</code> before persisting a single anchor	it appears in no effort estimate anywhere
P8	Every novelty claim carries the search that produced it	"no precedent" claimed five times; one refuted by three sources this programme had already downloaded and dropped
P9	A replayed number says <b>SIMULATED</b> in the same sentence	every comprehension claim in the corpus
P10	<b>The unit of progress is step 1's test going green</b> — not a test, <i>that</i> test	a rule saying "one assertion red to green" can be satisfied by writing a compiler
P11	Push before you think	the entire programme lived on one laptop, untracked, until 2026-08-01

[primary, docs/mdmax/PLAN.md v2.0.0 §11]

**P2 binds this plan, not only the research.** Every number in `docs/mdmax/PLAN.md` gets a committed script under `scripts/derive/` plus its captured output, **or it is deleted from the plan**. Start with §11.2.2's `corpus-id.mjs`, because it is the number every other number stands on.

11.12.3 The gate registry — port it, do not copy it

**What to build:** `specs/harness/gate.mjs`, storing rows to `specs/gates.jsonl`. It is a Node port of `sgnk-regression-gate.sh` (9,232 bytes), whose entire contract is five refusal codes plus a `REPRO_N00P` sentinel. [primary, `aios-transfer`] **Effort: ~1 founder-day**, most of it the selftest — port the selftest **first**, and the tool is not trusted until it passes.

The contract, kept verbatim:

- `--add` requires **both** `--assert` and `--broke`.
- Refuse with distinct exit codes: **2** already-red assertion · **3** vacuous gate · **4** missing repro · **5** write-did-not-land.
- `--verify-all` re-proves that **every** registered gate can still fail, and treats a reserved `REPRO_N00P` as **INCONCLUSIVE, never as proof**.

Two deliberate changes from the shell original:

- Note, not bash.** This deletes the entire `bash-3.2 / zsh / heredoc false-RED` class (Learned Rules #64, #65, #66) at the root instead of papering it with a preflight.
- Add the environment fingerprint** the shell version lacks (§11.11.4).

**Exit code 5 exists because of a real incident.** The first version of `sgnk-regression-gate.sh` — the tool built specifically to eliminate false greens — printed `REGISTERED` twice for gates that were never persisted, because the destination directory was `sandbox-denied` and `>>` failed silently. [primary, Learned Rule #67] **A tool that writes must verify the write landed: count before, count after, refuse if the count did not move.**

The first three gates to register, each with its `--broke` written first:

GATE	ASSERTION	<code>--BROKE</code> MUST MAKE IT GO RED BY
<b>splice conformance</b>	bytes written == bytes requested, every splice	reintroducing <code>matter.stringify</code> / AST regeneration
<b>anchor safety</b>	<code>resolve()</code> never returns a wrong index	reintroducing content-hash-plus-nearest-position, measured <b>22.20% false</b>
<b>UTF-8 integrity</b>	no artifact leaves a writer undecodable	swapping <code>perl -CSD</code> for <code>head -c</code> , red on a Devanagari fixture (reproduction exists: <b>216 bytes in, 200 out, decode error at position 198</b> )

[primary, `aios-transfer` design item 3] The first two are **not harness work — they are the definition of done for steps 2 and 6**. The UTF-8 gate is ~2 hours and can land immediately.

11.12.4 The verify ladder

`npm run verify` already exists and is already a cheapest-first ladder that stops at first failure:

```
npm run typecheck # tsc --noEmit
npm run lint # eslint . --max-warnings=0
npm run test # vitest run --passWithNoTests
npm run build # next build - catches Vercel-side exactOptionalPropertyTypes
npm run arch # specs/harness/clean-architecture-report.mjs
```

[primary, `package.json`] It was measured passing in **3.17 s on this repo** by an external harness [measured, `aios-transfer`] — note that predates the vitest exclude fix and therefore ran the 247-file suite; re-measure after §11.2.1.

**Import the one rule it lacks, verbatim from `verify-ladder.sh` : a rung that TIMES OUT is `infra`, never `fail`** — a wedged candidate must not be able to masquerade as a quality failure. Emit one JSON line per run: `{ verdict, rung_reached, pass, timeout, rungs }`. **Effort: ~2 hours.**

**Always-green invariants** [primary, AGENTS.md §2] : clean-architecture-report total 0, server-folder-blocklist empty, import-boundary-report empty. Red gate → stop and fix. Do not accumulate violations.

**One local-vs-CI trap that will bite:** production tsconfig has strict, noUncheckedIndexedAccess and exactOptionalPropertyTypes. Local npm run typecheck can pass while Vercel fails. Declare optional props that can be explicitly undefined as?: T | undefined, not bare?: T. [primary, AGENTS.md §5]

11.12.5 Dispatching an agent — what it may and may not do

Every agent prompt carries this block verbatim:

```
SCOPE: you own <module list>. You may not create, edit or delete a file outside it.
FORBIDDEN: git commit, git push, npm install, rm, any write outside $TMPDIR and your module,
 any outbound message, any change to .claude/settings.json or CLAUDE.md.
 If something seems to need one of these, describe it and STOP.
DELIVERABLE: a diff plus the command that proves it, not a summary.
GATE: your work is not done until <named gate> goes green AND has been proven red
 under its --broke.
```

Three things the dispatcher does, not the agent:

- 1. git rev-parse HEAD before and after, reconciled. The prohibition is advisory (Learned Rule #48); the reconciliation is the gate.
- 2. Cap the agent's tool count at 20. Past ~30 tools, quality degrades. [Learned Rule #26]
- 3. A claim that a defect is fixed does not close a ticket. A registered gate that goes red under the old code closes a ticket. Adversarial review stays — as a gate, not as a report. [primary, aios-transfer design item 4]

**What NOT to import from AIOS**, on its own authors' evidence: the bash guard (it blocks read-only greps on token match), the routing bandit, the model router, the calibration curve (their own author says the loop cannot close), the judge panels (no measured Cohen's kappa), and the trace ledger's reward path. Import the trace ledger **only** as a one-line-per-build-run record, if build telemetry is wanted at all. [primary, aios-transfer design item 7]

11.12.6 Red-to-green, defined per class of change

CLASS	RED MEANS	GREEN MEANS
defect fix (step 2, the three parsing defects)	a test reproducing the defect on the <i>current</i> code fails	it passes, and the --broke that reintroduces the defect makes it fail again
capability (steps 1, 4, 6)	the named integration test fails because the capability does not exist	it passes end-to-end against a real second identity, not a mock
tenancy (step 1 only)	test/tenancy/second-user-writes.test.ts fails	it passes, and grep -rn "sagnikmitra" src/ returns <b>nothing outside a comment</b>
diagnostic (step 5, mdmax vocab )	the tool reports 0 findings on a corpus known to contain them	it reports them on a corpus <b>this team did not write</b> , with --json and a nonzero exit
documentation	—	<b>there is no green.</b> A doc change is never progress under P10. Ship it, do not count it

**A test on a rare fault proves nothing until it reproduces the fault.** Learned Rule #68: a UTF-8 truncation fix passed a live end-to-end run **and** a differential over 120 real artifacts with zero failures under **both** the old and the new code, because the defect fires in ~0.34% of cases. Before trusting a passing test, **make it fail against the unfixed code**. If you cannot, say the test does not cover the bug.

11.12.7 The five conversations — one week of calendar, zero engineering

**Across 34 research areas and roughly 8 million tokens: live model calls run zero, user conversations run zero.** [primary, stated in every area's unverified block] Every claim about what a user will feel about an orphaned comment, a login wall or a degradation certificate is [inference] . **This is the cheapest instrument in the programme that can invalidate the most expensive decisions, and it has never been picked up.**

Recruit from three enumerable frames the research already named:

- 1. posters on forum.obsidian.md/t/2049 (246,568–249,616 views, 928 likes),
- 2. commenters on obsidian-git issues 340 / 396 / 558,
- 3. authors of repos carrying an llms.txt .

[primary, final-gate synthesis lens 3, ranked[4]]

Five calls, one question each, scored against the 14 pain themes. Three pre-registered predictions ride on them:

- **M3** ( product-gap ): *four of five reviewers stall at the login wall*. If it fires, **D3 is reopened as a product decision made at auth** (§11.6).
- **K-V3** ( custom-pointers ): *one user asking unprompted to name their own construct would outweigh every corpus number in that area*.
- **K-D** ( founder-thesis ): the one question that settles the mother-markdown — ask Sagnik whether the export he meant was always a zip of clean .md files (§11.9).

Run these **BEFORE** steps 4 and 5, because they can cancel both. They gate nothing else, so they run in calendar parallel with steps 1 and 2.

11.12.8 Re-derive 99.627%, or stop quoting it

Eight research areas rest on a number **nobody has reproduced**, measured on a corpus dominated by revisions that delete nothing, at **BLOCK** level, when the product anchors **RANGES**. [primary, docs/mdmax/PLAN.md v2.0.0 §5.2 – "Nobody has re-derived the number. Not one of the areas that quoted it."]

The corpus is append-dominated, measured: in the .md root (82.5% of corpus bytes) **4,948 of 5,715 .md file-revision pairs (86.6%) delete zero lines** — a figure the verifier corrected to **67.2%** under the manifest's own exclusion policy, and **6.1%** on genuine consecutive pairs. [measured] **A zero-deletion revision leaves every prior block byte-identical, so exact-hash resolves it by construction** — which is why §1.1a's S1 figure of 87.0% and the append rate agree to within half a point.

**Ships as:** a committed script under `scripts/derive/` with its captured output, reporting **correct / false / refusal in four strata** (0 deleted lines · 1–2 · more than 2 · reflow), with **n stated per stratum**, plus a **diff3-assisted baseline arm**.

**Blocked on a real acquisition task:** a corpus selected by **EDIT SHAPE, not authorship**. None of the three pinned roots qualifies — the fraction of revision pairs deleting more than two lines is **md 0.9%, knowledge 9.9%, frontmatter 8.3%**, against a 25% bar. `[measured]`

**If the >2-lines stratum yields under 200 block-versions on every available corpus, that is the finding**, and the claim must be restated as *"exact-hash resolution over an append-dominated corpus"* — true, defensible, and much smaller. **Until then, do not publish 99.627% anywhere**. It is a credibility kill, not a market kill: the first competent outside reader will check it.

11.13 The 30-day test, written as a gate

**Due: Monday 2026-08-31.** `[verified: date -j -f "%Y-%m-%d" "2026-08-31" "+%A" → Monday]`

The two conditions, evaluated by command, not by feeling:

```
Condition 1 – has a second GitHub login written to a document?
npx vitest run test/tenancy/second-user-writes.test.ts # must exit 0
git log --format='%an <ae>' --since=2026-08-01 -- src/ \
 | sort -u # must contain a non-Sagnik author

Condition 2 – has Amit committed code?
git log --format='%an' --since=2026-08-01 -- src/ test/ | sort -u | grep -qi amit
```

If either command fails on 2026-08-31:

This is a library and not a company, and the honest move is to say so out loud.

"Out loud" is defined, so it cannot be softened into another document:

- 1. Sagnik writes one paragraph in `docs/mdmax/PLAN.md` under a heading dated `2026-08-31`, stating which condition failed and what the number was.
- 2. The positioning changes in the same commit: MDMAX is described as a standalone markdown library, not as the capability layer of a collaboration product. The plan already describes this as the honest alternative.
- 3. One of three choices is made **that week**, not deferred: **(a)** Amit owns step 1 end to end with his own commits; **(b)** step 1 is contracted out; **(c)** the collaboration product is shelved and the library is shipped as a library. *"We will see"* is not on the list.

**Why 30 days and why these two conditions.** The `premortem` area ranks its top two risks — **N2 the editor floor never shipped** and **N1 the research never became code** — both at severity **TOTAL**, and notes that **N11 bus-factor-one is a multiplier on both rather than an independent failure**: all 20 commits in `frontmatter` and all 416 in `md` are Sagnik Mitra `<sagnikmitra123@gmail.com>`; Amit is named as co-author in the plan's frontmatter and as co-decider on D1–D5, and **has committed nothing**. `[measured]` A gate that only tests the code tests one founder's calendar. A gate that tests both tests the company.

**The falsifier for this gate itself:** if step 1 is green on 2026-08-31 but nobody outside the two founders has ever opened the product, K1 passed and the company still has no evidence. Do not let a green K1 substitute for the five conversations in §11.12.7.

11.14 The kill gates, complete, each with the number that trips it

#	IF THIS IS TRUE	MEASURED HOW	THEN
K1	No second GitHub login has written to a document by 2026-08-31	§11.13, condition 1	this is a library, not a company. Say so out loud
K2	Amit has not committed code by 2026-08-31	§11.13, condition 2	same
K3	The re-derived anchor number is materially below <b>99.627%</b> , or the 30-line LCS diff baseline matches it (baseline stands at <b>86.658%</b> , re-derived <b>85.036%</b> , vs hash-alone <b>77.297%</b> )	§11.12.8 stratified script	the anchor is a fallback only. Re-scope the claim and <b>never publish the number</b>
K4	The <b>range</b> resolver's hand-audited false-match rate exceeds ~ <b>0.5%</b>	hand audit, n stated	comments cannot use it. Fall back to quote-plus-digest with visible orphaning
K5	Fewer than <b>5 of 30</b> probed users say they would install <code>cert</code>	§11.12.7, extended probe	cut <code>mdmax cert</code>
K5b	Semantic <b>BROKEN</b> falls below <b>1% of blocks</b> on a corpus this team did not write (the honest measured figure on our own corpus is <b>4.28%</b> — a 4x margin, not 40x)	<code>mdmax cert</code> on an external corpus	cut <code>mdmax cert</code> . <b>This threshold did not exist before; K5 as previously written could never fire</b>
K5c	At <b>60 days</b> , recapturing the DECLARED (closed-SaaS) rows shows <b>zero cells moved</b>	re-run the cert matrix on 2026-10-01	the dataset does not decay, a single copy is permanently level with ours, and the certificate is a <b>lead, not a moat</b>
K6	The orphan rate in the pilot is intolerable — threshold <b>&gt;10% of all threads per week of normal editing</b>	the visible product metric shipped with the first comment	the review loop needs a different anchor, or a different product
K7	Multi-tenancy does not ship	step 1 gate	there is no Google-Docs story. Position the engine as a standalone library
K8	<code>mdmax cert</code> promotes itself: <b>15 of 30</b> probed users name the certificate as their reason to pay	§11.12.7	<b>the inverse kill</b> — promote it and re-sequence the whole plan
K9	<code>inkeep/OpenKnowLedge</code> ships the teammate model (today its changeset says a comment is <i>"a note to your own agent, not a message to a teammate"</i> )	watch the repo; it merged anchoring <b>2026-07-30</b> and published at <b>2026-08-01T02:38:33Z</b>	the empty intersection is no longer empty. Re-decide step 6 within a week, do not discover it in a demo
K10	P7a returns <i>"a zip of clean <code>.md</code> files"</i> or P7b (live-model tokens) fails to beat the file tree	§11.9	pack/unpack is deleted, permanently. Keep only the D4 export zip

K1 and K2 are the ones that matter. They fall due in thirty days. Everything else is a course correction; those two are the company.

11.15 What we do not build — with the reason and the measurement

This table exists so nobody re-opens these. Each row is a measurement, not a preference.

DO NOT BUILD	THE MEASUREMENT THAT CLOSED IT
The container / mother-markdown / pack as a write surface	boundary repair <b>119/120 WRONG</b> on the realistic merge case; container <b>25,548,765 B</b> vs index <b>105,538 B = 242x</b> ; revealed demand <b>846:1</b> against ( <code>llm-code-format</code> 387/mo vs <code>repomix</code> 327,543); the only live-model measurement went <b>+5.2% tokens the wrong way</b>
The markdown-as-graph / knowledge-graph thesis	<b>46.8% of files sit in zero resolved edges; median out-degree 2</b> (median <i>document</i> degree 0); <b>45 transclusions in 25.5 MB</b> ; max transitive blast radius <b>54 files</b>
A format profile or any spec document	MDMAX is a library with <b>two callers</b> , not a format with an ecosystem
The declared-vocabulary mechanism, fm:vocab , any registry	user-invention rate <b>0.31% across 5,245 files</b> in five corpora; where a spec <i>forced</i> a <code>type</code> field (four real OKF bundles) it produced <b>29 values across four incompatible naming conventions with ~42% mutual collisions</b> — drift, not vocabulary
Every new carrier	the best carrier on every objective axis (orphan link reference definition — wins capacity, wins invisibility in <b>6/6</b> renderer configs, byte-identical through remark <b>and</b> prettier) has <b>ZERO uses in 1,084 files and 25.5 MB</b> . The market's verdict: <code>markdown-it-decorate</code> , <b>1,094 downloads/week, unshipped since 2017</b>
The folder compiler's incremental engine and rebuilder	<b>507 of 1,084 files affect nothing downstream</b> ; <b>530</b> have no dependents; max blast radius <b>54</b> ; a full cold lex of all 25.5 MB is <b>1,056 ms</b> . Keep only the symbol table, the affected-target query and the orphan report — all useful at cold-build time
Per-span confidence / uncertainty annotation	<b>16 of 1,084 files</b> carry any evidence tier at all — in a corpus whose authors <i>mandate</i> tier tagging on every number. OKF explicitly refuses to store a credibility score
A competing provenance vocabulary	emit <b>OKF v0.2 \$5.1</b> fields and move on. Being the third entrant behind Google and C2PA is not a position. The one genuine opening: a <b>PRODUCER-side checker</b> , which OKF explicitly declines to require and which nobody in its 287-repo ecosystem ships
The streaming prefix-parse mode with stable block ids	its own kill condition P3 predicts the failure (real model output is fence-heavy); and the "new category" is <b>one vendor counted up to four times</b> — <code>streamdown</code> , <code>remend</code> , <code>rehype-harden</code> , <code>harden-react-markdown</code> are all Vercel, with <code>streamdown@2.5.0</code> pinning <code>remend@1.3.0</code> as a <b>direct dependency</b> , halving the market signal to ~16M/mo
The AI projection tier sold as a token saving	the <b>25–168x</b> figures are ratios between <i>artifact sizes</i> ; the only measurement of <i>task</i> tokens found <b>+5.2%</b> and <b>more</b> files opened. Restate as a compression ratio or delete the sentence
The graph view (it ships today, and carries a runtime dependency: <code>react-force-graph-2d</code> )	<code>graph</code> appears <b>0 times</b> across 89 sourced feature requests and <b>0 times</b> in the round-2 corpus; <b>median document degree in the founder's own vault is 0</b>
<code>EditableTable</code> / <code>setTableCell</code>	dead code today — imported by nothing, callback named <code>_onEdit</code> — with a measured <b>13.4% no-op dirty rate</b> and silent escaped-pipe corruption. <b>Delete rather than fix</b>
The claim that markdown volume is exploding	repository share fell <b>12.68% → 10.41%</b> on one measurement and rose on an independent ten-repo test — <b>unproven in both directions, so say nothing</b> . The defensible claim is that markdown's <b>reader</b> changed
Any further research fan-out	two workflows, 34 areas, ~8 million tokens, <b>6,233 documentation lines against 365 code lines, zero live model calls, zero user conversations</b> . The marginal research hour is now provably worth less than the marginal test

[all measured, corpus\_id sha256:3a010b16... where the figure concerns our own files; final-gate synthesis lens 3, what\_to\_cut, and lens 1]

Two cheap wins that survive the cull, because both are diagnostics that write no bytes and cannot flatter the team — the same rule that selected `mdmax cert` :

1. The placeholder linter. **1,431 bare angle-bracket tokens in 219 of 1,084 files (20.2%)** that GitHub silently **DELETES** from rendered output. One-backtick fix, verified remediation. `[measured]`
2. `mdmax vocab` (~2 days): report the enum a corpus already uses, plus case and morphology drift, plus the **18.91% of frontmatter blocks that no YAML parser reads**. `[measured]`

11.16 The first week — Monday 2026-08-03 to Friday 2026-08-07

[all weekdays verified: `date -j -f "%Y-%m-%d" "<date>" "+%A"`]

Monday 2026-08-03

TIME	WHAT	PROOF IT HAPPENED
09:00–09:30	Read this section and §11.14 out loud, together. Agree the 2026-08-31 date.	nothing to commit; if this is skipped, the rest is theatre
09:30–10:00	<b>§11.2.1</b> — add <code>exclude</code> to <code>vitest.config.ts</code>	<code>npx vitest list --filesOnly   wc -l</code> → <b>83</b>
10:00–10:30	Run <code>npm run test</code> on the real 83 files. <b>Record the pass/fail count in the commit message</b> .	first honest baseline
10:30–11:00	Remove both stale worktrees; <code>git worktree prune</code>	<code>git worktree list</code> shows one entry
11:00–12:00	<b>§11.2.2</b> — <code>scripts/derive/corpus-id.mjs</code> , run it, commit script + output	prints <code>sha256:3a010b16...</code> exactly
13:00–14:00	<b>§11.2.3</b> — resolve the §1.1a <b>113 vs 14</b> contradiction; <b>§11.2.4</b> — delete the litter	one commit each
14:00–17:00	<b>Write and commit</b> <code>test/tenancy/second-user-writes.test.ts</code> , <b>RED</b> . Commit message begins <code>RED:</code>	the test fails, for the right reason, and that reason is in the commit body
17:00–17:30	Send five recruiting messages for <b>§11.12.7</b>	five sent, replies expected by Wednesday

Tuesday 2026-08-04

TIME	WHAT	PROOF
09:00–13:00	<b>Step 2 in one sitting.</b> <code>src/modules/mdmax/domain/splice-frontmatter.ts</code> + barrel; rewrite <code>share-writer.ts</code> to use it; <code>try/catch</code> around the read so the 171 unparseable files stop 502-ing	the gate test, committed RED first, moves from <b>17/736</b> toward 907/907
13:00–15:00	Build the <b>oracle</b> that shares no code with the writer ( <code>Buffer.compare</code> on re-read bytes)	oracle imports neither <code>yaml</code> nor <code>gray-matter</code> nor the splicer
15:00–16:00	Register the <b>splice-conformance gate</b> ; write its <code>--broke</code> (reintroduce <code>matter.stringify</code> ) and prove it goes red	<code>gate.mjs --verify-all</code> green
16:00–18:00	Step 1 task <b>1b</b> — delete <code>allowlist.ts</code> and its test; rewrite the <code>signIn</code> callback	typecheck green, the RED test fails one assertion later than yesterday

Wednesday 2026-08-05

TIME	WHAT	PROOF
09:00–12:00	Step 1 task <b>1c</b> — the <code>tenancy</code> module: user + vault documents on first login, matching <code>firestore.rules</code> field-for-field	<code>firestore()</code> finally has callers
12:00–13:00	<b>Conversation 1 and 2.</b> One question each. Score against the 14 pain themes	written notes, same day, before memory edits them
14:00–18:00	Step 1 task <b>1d</b> begins — per-user GitHub credential. <b>The riskiest task in the plan</b>	<code>GITHUB_REPO_TOKEN</code> no longer appears in <code>client.ts:30</code> or <code>github-writer.ts:33</code>

Thursday 2026-08-06

TIME	WHAT	PROOF
09:00–13:00	Task <b>1d</b> continues	every read and write path uses the signed-in user's token
13:00–14:00	<b>Conversation 3 and 4</b>	notes
14:00–16:00	Task <b>1e</b> — <code>AUTHOR</code> becomes a parameter; update the four use-cases at <code>dependency-container.ts:66,70,75,80</code> ; <b>update</b> <code>AGENTS.md</code> <b>\$7 in the same commit</b> (§11.2.5)	<code>grep -rn "sagnikmitra" src/</code> returns nothing outside a comment
16:00–18:00	<b>\$11.12.3</b> — port <code>gate.mjs</code> , selftest first	selftest 10/10; four distinct REFUSED exit codes reproduce

Friday 2026-08-07

TIME	WHAT	PROOF
09:00–11:00	The three parsing defects: <code>WIKILINK_RE</code> (k=2.00), the YAML 1.1/1.2 divergence, plus <code>eslint-plugin-redos</code> in CI	three commits; <b>then close the</b> <code>parsing-robustness</code> <b>area</b>
11:00–12:00	<b>Conversation 5.</b> Ask Sagnik <b>K-D</b> verbatim: <i>"was the export you meant always a zip of clean .md files?"</i>	§11.9's P7a resolved, or explicitly still open
13:00–15:00	Task <b>1f</b> — the signup / repo-picker route into <code>src/app/(workspace)/</code> (today: one <code>.gitkeep</code> )	a route exists
15:00–16:00	<b>\$11.11.4</b> — worktree preflight + before/after HEAD reconciliation, wired before any parallel agent run	preflight refuses a stale worktree
16:00–17:00	<b>Week review against this table.</b> Anything not done moves to Monday <b>with its estimate revised</b> , not silently carried	one paragraph in <code>docs/mdmax/PLAN.md</code> , dated
17:00–17:30	Merge <code>engine/plan-and-diagnostics</code> into <code>main</code> so <code>main</code> stops being a month stale	<code>git rev-list --left-right --count origin/main...HEAD</code> → <code>0</code>

**What this week deliberately does NOT contain:** any work on `mdmax cert` , any work on the review loop, any work on pack/unpack, and any further research. If step 1 is not visibly closer by Friday, that is the signal — not a reason to go and measure something else.

11.17 What this section does not cover, and what would falsify it

**Not covered here.** Pricing mechanics and billing integration (step 3 names anchors, not a payments implementation). Marketing, launch sequencing and the landing page. Legal, terms and the data-processing agreement for a product that will hold other people's documents. Hiring. Anything about AIOS beyond the four mechanisms §11.12 imports — AIOS is a **separate track (D5)** that benefits from MDMAX and is not part of it. Mobile-native wrappers. Importers. Search at scale. Anything in `FRONTMATTER-PRODUCT-PLAN.md` Phases 2, 4 and 5.

The weakest parts of this section, named.

- The effort estimates are** `[inference]` , **not** `[measured]` . They come from decomposing tasks, not from having done them. The one velocity datum available — **28 lines of product source per day over 19 days** — argues every estimate here is optimistic. Treat them as hypotheses that K1 and K2 test.
- The 66.4% vittest figure is measured; the consequence is inferred.** I ran `npx vittest list --filesOnly` and counted 164 of 247 files in `.claude/worktrees/` . I did **not** run the suite and observe a false pass or a false failure caused by them. The mechanism is certain; a specific incident has not been demonstrated. Run `npm run test` before and after the exclude and record both numbers.
- Two lenses of the same research run disagree on the order of the first two steps** (§11.1). This section resolves the disagreement by making them parallel and disjoint. If they turn out to share a file, the resolution is wrong and one of them must go first — check the disjoint-artifact map in §11.11.3 before dispatching.
- Step 7's placement contradicts the evidence, and this section says so** rather than quietly dropping it (§11.9). If Section 3 of this plan sequences pack/unpack unconditionally, Section 3 and Section 11 disagree, and **the measurements in §11.9 govern** until someone produces P7b.
- The verifiers behind roughly half these numbers have a measured false-kill rate of 0–19%.** Where a kill drives a "do not build" row in §11.15, that row is strong evidence, not proof. The rows I would re-check first if someone pushed back: the graph view (a `grep` for the word "graph" is a crude demand instrument)

and the streaming prefix mode (its kill is a prediction about model output that nobody has measured on model output).

**What would falsify this section, concretely.**

- **Step 1 goes green in under three days.** Then the 8–12 day estimate was wrong by 4×, the whole premise that tenancy is the bottleneck deserves a second look, and step 3 should start immediately.
  - **The splice gate passes at 907/907 on the *first* run against the *current* writer.** Then the measured 17/736 is wrong and every downstream claim about frontmatter corruption needs re-deriving before it is repeated to anyone.
  - **Three of the five conversations say the login wall is fine.** Then M3 did not fire, D3 stands as written, and step 4 simplifies.
  - **A second GitHub login writes a document and nothing else on this list changes.** Then K1 passed and the plan still has no customer — which is the failure mode §11.13's falsifier exists to catch.
-

§12 Risks

Risks, the pre-mortem, and the incompatible wants

← Index · ← §11 Execution · §13 Appendix →

12. Risks, the pre-mortem, and the incompatible wants

This section is the one that earns the rest of the document. Everything before it argues a case. This one states what is wrong with the case, what is unresolved inside it, what would kill it, and what has already gone wrong on the way here.

Read it in four parts:

PART	WHAT IT IS	WHO HAS TO ACT
§12.1	<b>Three incompatible wants.</b> Three pairs of things the founders have asked for that cannot both be true. Each needs a founder decision, not an engineering one	Sagnik, with Amit
§12.2	<b>The pre-mortem.</b> Thirteen ways this is dead in 2027, each with the mechanism, the warning sign visible <i>today</i> , and the countermeasure	whoever owns the item
§12.3	<b>The failure mechanism, named precisely.</b> Not "we might get distracted" — the specific reward function that produced the last thirty-five days	both founders
§12.4– §12.7	What we got wrong, the open decisions, what has to be true for twelve months to be worth it, and what this section does <b>not</b> cover	reader

**Three words are used strictly throughout.** **DECIDED** means it is settled and reopening it costs the plan its coherence. **RECOMMENDED** means this document has a preference and states the evidence for it, but the founder can overrule it without breaking anything else. **OPEN** means nobody has decided and the plan currently answers it two ways or not at all.

**Evidence tiers**, as everywhere in this document: `[measured]` we ran it · `[primary]` we read the source · `[secondary]` · `[inference]` · `[SIMULATED]` we replayed data through code rather than reading a live system.

**One standing warning that applies to every number below.** The research corpus this section draws on has a measured, one-directional optimism bias. In the eighteen-area capability run, 18 of 18 research headlines were refuted by their own adversarial verifier, always in the flattering direction. In the sixteen-area final gate, the structural fixes brought that down to **5 CONFIRMED / 10 OVERSTATED / 1 REFUTED**. So: never quote a headline. Quote the verifier's verdict, or quote a number that a second party reproduced. Where a figure below is a headline that its own verifier killed, it is marked **KILLED** and the corrected value is given instead.

**And the verifiers are not oracles either.** Four independent kill-audit agents re-derived a sample of the verifiers' own kills from primary sources and found false-kill rates of `5/29 = 17.24%`, `1/29 = 3.4%`, `4/21 = 19.0%`, and `0/26 = 0.0%` [ docs/engine/research/wf-final-gate-2026-08-01.result.json → kill\_audit[] ]. Pooled naively that is 10 of 105 = 9.52%, but the four samples are not verified disjoint, so **the honest working assumption is a false-kill rate somewhere in 0–19%**. A kill is strong evidence. It is not proof. Process rule P4 — sample and re-audit kills before deleting anything — exists for exactly this reason, and it is why nothing below recommends deleting work on the strength of a single kill.

12.1 The three incompatible wants

These are not risks. They are contradictions already present in the record, each of which the document currently answers in two different places with two different answers. Each needs one founder decision. None of them can be resolved by measurement, because both sides are already measured.

They come from the `founder-thesis` research area. **That area's headline was REFUTED** — its central empirical claim, that the founder already hand-builds multi-document bundles across 37.2% of his files, was killed by its own verifier and corrected to roughly 2.2% (§12.1.1 below). But its `negatives` block is the most valuable single artifact in the entire research corpus, because it is the only place where anybody wrote down the fact that the founder has asked for two mutually exclusive things and nobody noticed.

12.1.1 Want A — the export is specified twice, and the two specifications are opposites

Status: OPEN. This is the one that most needs a founder in a room.

**Specification one, from the settled decisions.** D2: "Comments and history live *OUTSIDE* the file. Export or copy yields clean markdown, latest content only. Nothing about comments is ever written into the .md." D4: "The durable artifact is plain markdown; if frontmatter dies we ship a full export and migration. Max tier gets offline access."

**Specification two, from the founder, verbatim from the session transcript** — not paraphrased from a handoff, and re-extracted independently by the verifier from the 58,919,587-byte session `jsonl`, `role=user`:

"I was mentioning purely about the Markdown tree, Markdown zip file concept, basically folder concept. Markdown is referred to within a Markdown, like a codebase. It will be a Markdown base. I was thinking whether the user should download one zip file of all the Markdowns or one single mother Markdown, which will be very long. Whenever frontmatter or the AI reads that, I will know these are the breakpoints, which means different files, and it can split that Markdown automatically while parsing into different files"

[primary, verified — the verifier confirmed the passage verbatim and found 3 lines in the session file containing "one single mother Markdown"]

**These cannot both be the export.** "Clean markdown, latest content only" is 1,084 loose files. "One single mother Markdown with breakpoints" is one file with a convention inside it. A convention inside a file is precisely the thing D6 was written to forbid, and a folder of loose files is precisely the thing the mother-markdown image was invented to replace.

**And the measurement is decisive, in a way that hurts both sides.** A clean-markdown export of the founder's own corpus today produces:

1,084 files                    25,548,765 bytes                    corpus\_id sha256:3a010b16...

10,798 dangling [[X]] occurrences over 6,175 distinct targets                    [own-measurement]

(independent re-extraction: 10,185 over 6,011 – roughly 6% drift between two implementations of the same rule; use the range, not either endpoint)

1,409 of those targets appear in >=2 different documents and account for

5,834 occurrences = 54.0% of all dangling links    (verifier: 1,326 / 5,415 / 53.2%)

only 30 occurrences (0.28%) are mechanically recoverable by re-applying the vault's own filename sanitizer; at most 1,320 (12.22%) are within 0.90 diffliB similarity of any existing file stem – so ~88% are deliberate entity references, not typos

no map, no index, no manifest ships with the export

That is a **durable artifact and a useless one**. Everything that made the corpus navigable lived outside the files by construction — in the vault application, in the resolver, in the graph view. Hand a departing user 1,084 files and ten thousand references that resolve to nothing, and you have honoured D4's letter and destroyed its purpose.

The mother-markdown export does not rescue it either, and the reasons are measured:

container (all 1,084 files packed)    25,548,765 B

one-line-per-doc index                    105,538 B                    → 242x

a 200K-token window holds 2.82% of the container – and 682% of the index

even a 1M-token window holds 14.1% of the container

mean file 23,569 B → a 200K-token pack carries about thirty average files

[measured, corpus\_id sha256:3a010b16...]

**Contradiction, stated because two sources disagree.** The `container-thesis` area reports the same comparison as **110x** (25.8 MB container against a 234 KB index) with a 200K window holding **2.79%**; §3.1 of this plan reports **242x** and **2.82%**. Both are real; they use different definitions of "index" — one line per document with a description and a size, versus one bare line per document. **§3.1's 242x governs for the export question**, because the export's index does not need per-document descriptions. The discrepancy is the same defect the `against` verifier caught elsewhere: this programme has used the word "index" for two different artifacts in one sentence, and it produced a false kill once already. Any future document must name the unit.

What else the record says about this want, and it is not encouraging:

- The round trip is not unclaimed. `llm-code-format` already ships pack **and** unpack with zero dependencies, at **387 downloads/month against repomix's 327,543 — a ratio of 846:1**. [primary]
- The Obsidian forum thread asking to export a vault to a single file: **9,010 views, 1 like, 3 posts in 26 months**. [primary]
- repomix issue #71 ("Splitting code into several files") was opened **2024-09-08**, is **still open** as of 2026-08-01 with 12 comments and 8 reactions, and the last comment is 2025-09-08. [primary, verified]
- The founder's own corpus does not support the practice the image describes. The claim that 37.2% of files and 47.7% of bytes are already hand-built bundles was **KILLED** by its own verifier: 372 of those 403 files (92.3%) are files with exactly two H1s whose first H1 is a machine-generated YAML `title:` field, and 365 of 403 sit inside one machine-synced subtree ( `md/Skills/**`, frontmatter `source: claude`, `last_synced: 2026-05-26` ). **Corrected figure: ~2.2% of files, ~2.0% of bytes**. That reverses the area's own pre-registered falsifier, which it had declared "FAILED TO REFUTE, in the founder's favour."

The decision that has to be made, in one sentence: what does the Max tier's download button produce?

Three candidate answers, with what each costs:

ANSWER	WHAT SHIPS	COST	WHAT IT VIOLATES
<b>A1</b> a zip of clean <code>.md</code> files, plus a generated <code>INDEX.md</code> and a <code>VOCABULARY.md</code> listing every repeated dangling target with its document set	one afternoon on top of an existing zip	nothing. D2 and D4 are satisfied; the map ships beside the files rather than inside them	nothing
<b>A2</b> one mother-markdown with breakpoints	a real packer with a stringifier-stable path carrier, per-entry sha256, byte counts and a trailing-newline flag, plus pack-time refusals	days, not hours — and it is the same splice-and-identity discipline the rest of the engine needs	arguably D6, because it puts a convention in the file
<b>A3</b> both, user picks	A1 plus A2	A1 + A2	the roadmap, by adding a surface nobody asked for twice

**RECOMMENDED: A1.** It is the answer that the demand evidence, the scale measurement and D4's stated purpose all point at, and the `VOCABULARY.md` is the piece that makes it *better* than what the founder described rather than a smaller version of it — it turns 10,798 "errors" into a 1,409-term index of his own writing, which is the only diagnostic in this programme that would tell him something about his corpus he does not already know.

**What would change the recommendation.** One conversation. Ask the founder the `founder-thesis` area's own kill condition K-D, verbatim: *"was the export you meant always a zip of clean .md files?"* If the answer is no — if he means one artifact, specifically — then A2 is the answer and the plan should say so, because four sessions of that image recurring is either a thesis or four sessions of the room hearing what it wanted, and only he can say which. **Cost: ten minutes. It has never been asked.**

12.1.2 Want B — "no new format" versus "features unseen in the industry"

Status: OPEN, and the boundary has never been drawn.

Two founder statements, both on the record, both load-bearing:

- "we can not invent a new markdown, we can not invent a new format" — which became D6.
- "our editor on the max plan should give features that is unseen in the industry" — which, together with D9 (a document NAMES a capability, never CARRIES one), is the whole differentiation story.

These bound each other, and the boundary is measurable. It was measured, and nobody drew it.

Additive capabilities are free. An unknown fence language round-trips byte-perfectly, and its payload survives into rendered HTML:

```
```mermaid      → <pre><code class="language-mermaid">graph TD; A--&gt;B;</code></pre>
```fm-capability → <pre><code class="language-fm-capability">payload: secret</code></pre>
```

[measured against marked 16.4.2 at node\_modules/marked; reproduced byte-for-byte by an independent verifier, including the HTML-entity escaping of -->]

Reference-carried capabilities cannot degrade at all. They vanish.

```
![[Some Note]] → <p>![[Some Note]]</p>
![[Some Note#Section]] → <p>![[Some Note#Section]]</p>
```

[measured, same run, CONFIRMED by the verifier]

The entire referenced document is absent and the residue looks like a typo. This is not degradation. It is a different failure class, and the plan's PASS / STRIP / CORRUPT certificate has no cell for it. The research proposed a fourth verdict — call it VOID, "the content was by reference and the reference did not resolve" — and it is the verdict that decides which capabilities D6 permits.

Therefore, and this is the sentence nobody has written down until now:

The set of "features unseen in the industry" that satisfy D6 is exactly the set that puts nothing in the file which a plain reader must resolve elsewhere. That set is much smaller than the roadmap assumes. It silently excludes:

EXCLUDED CAPABILITY	WHY
transclusion ( ![[X]] , {{include}} , any embed)	payload is by reference; measured VOID
cross-file reactive state (a value in document A that updates when B changes)	the payload is a promise, not bytes
block-scoped metadata resolved from a sidebar	the file carries a key; a plain reader gets the key, not the meaning
anything depending on a declared vocabulary ( fm:vocab: sgnk/v1 )	meaning is by reference to a registry the reader does not have
any capability whose value is a resolution step	same shape, one hop

And it includes, at zero D6 cost: unknown fence languages carrying self-contained payloads; frontmatter fields whose value is extensible (D8, already decided); anything that renders as readable prose when nothing resolves it; and every diagnostic that writes no bytes at all.

Corroborating measurements, for anyone who wants to reopen this:

- The measured user-invention rate for new markdown constructs is 0.31% across 5,245 files in five corpora, and all fourteen of the exceptions inside Docusaurus's 4,538 directives are the three tutorial strings that teach the feature. 847 independently-authored npm docs use exactly GitHub's five built-in callout types with zero inventions. [measured]
- The best available new carrier on every objective axis — the orphan link reference definition, which wins on capacity, wins invisibility in 6 of 6 renderer configurations, and is byte-identical through both remark and prettier — has ZERO uses in 1,084 files and 25.5 MB. [measured]
- The market's verdict on inventing a carrier: markdown-it-decorate achieved perfect silent invisibility and has 1,094 downloads/week, unshipped since 2017. [primary, re-verified live]
- Where a spec forced declaration, it produced drift rather than vocabulary: four real OKF bundles produced 29 type values across four incompatible naming conventions with ~42% mutual collisions. [measured]

The decision that has to be made: does D6 have an exception, and if so, written down where?

RECOMMENDED: no exception. Write the VOID rule into the certificate and into D6 itself, as: "a capability whose payload is by reference is out of scope; if we ever want one, it is a change to D6 and it gets its own decision record." Then the "unseen in the industry" promise has to be paid for somewhere other than syntax — and §4/§7 already say where: the review loop over files people own, an intersection four competitor sweeps found empty.

What would change the recommendation. A user, unprompted, asking to reference one document from inside another and expecting it to render — the custom-pointers area's kill condition K-V3 says one such observation would outweigh every corpus number in that area, and it is right. Nobody has asked anybody. W3 (user conversations) stands at zero.

What this recommendation does not settle: whether a capability may be named in the file and resolved by frontmatter only (D9's shape). Under the VOID rule, naming is permitted only if the file still reads correctly when the name resolves to nothing — which is a testable property and should be a fixture, not a principle.

### 12.1.3 Want C — the library/import model versus "no runtime"

Status: OPEN, upstream of syntax, and currently answered by deletion rather than by decision.

The founder's analogy, on the record: "in Python there are thousand libraries people might use hundreds only." The image is a package ecosystem for documents — a library of notations, capabilities or components that a document imports.

Against it stands §5's settled position and D6: no runtime. The document does not execute anything. A markdown file is inert.

The research area that hit this stated the conclusion and it was recorded without being acted on:

"acquiring a package ecosystem means becoming a programming language, or admitting you are a thin shell over one. Decide which, explicitly, before designing syntax."

**The supporting fact:** the only two document formats that ever acquired a package ecosystem are **LaTeX (7,021 CTAN packages)** and **Typst (1,481)**, and both are Turing-complete languages. [secondary – a search-only count, not independently re-derived by the kill audit; treat the exact integers as indicative and the structural claim as solid]

**Why this is upstream, not downstream.** Every syntax question — what an import looks like, whether there is a registry, what a version pin means, what happens on a missing package — is a *consequence* of this answer. Designing the syntax first and asking the question later is how a document format becomes a programming language by accident, one convenience at a time.

**What v2.0.0 did.** It cut the notation library, the vocabulary registry and the declared-vocabulary syntax (§3.3), on the strength of the 0.31% invention rate. **That is an answer by deletion.** It is almost certainly the right answer, and it was reached without ever putting the question to the person who raised it. That matters, because a want that is deleted rather than declined comes back — this is the fourth document in which the container image has reappeared after being cut.

**RECOMMENDED: answer it explicitly as "no runtime, no package ecosystem, and the extensibility point is D8 — the VALUE of a field, never the SET of node types."** Write it as a decision (D11) with the founder's assent, not as a line in a cut list.

**What would change it.** Two things, in order of cost: (a) the founder saying the Python analogy was about *discoverability* rather than *importing*, in which case the answer is a catalogue and not a runtime and everybody agrees; (b) evidence that users declare structure when a tool suggests it — the founder-thesis area found the strongest counter-example available, since the founder himself authored a **1,409-term controlled vocabulary unprompted** in the form of repeated dangling wikilinks, which every artifact in this programme currently classifies as an error. That is a real finding and it does *not* imply a package ecosystem; it implies a `mdmax vocab` recogniser that writes no bytes.

12.1.4 The three decisions, on one line each

#	DECISION	CURRENT STATE	RECOMMENDED	WHO DECIDES	BY WHEN
W-A	What does the Max-tier download button produce?	plan answers it two ways (D2/D4 vs the mother-markdown image)	zip of clean <code>.md</code> + generated <code>INDEX.md</code> + <code>VOCABULARY.md</code>	Sagnik	before item 3 ships (Wedge A)
W-B	Does D6 have an exception for reference-carried payloads?	never drawn; roadmap silently assumes yes	<b>no</b> — add the VOID verdict, write the rule into D6	Sagnik + Amit	before any capability design
W-C	Library/import model, or no runtime?	answered by deletion in §3.3, never asked	<b>no runtime</b> ; extensibility is D8 only. Record as D11	Sagnik	this week — it is upstream of everything

12.2 The pre-mortem — it is 2027 and this failed

Thirteen narratives. Each has **the mechanism**, **the earliest warning sign already visible today**, and **the countermeasure**. They are ranked at the end by probability x severity.

The rule for this section: a warning sign only counts if it is observable in the repository or in a public source *right now*. "We might lose focus" is not a warning sign. "Thirteen of twenty-one commits are docs and `grep -ril mdmax src test scripts` returns nothing" is.

N1 — The research never became code

**Mechanism.** The reward function pays for sentences. Every research pass produces a document that is immediately more valuable than the last one, so the marginal hour always goes to prose. The engine never gets a first line, because there is always one more reconciliation to do first.

**Earliest warning sign, verified live in this session (2026-08-01):**

```
git log --oneline | wc -l - 21 commits
git log --format='%s' | grep -c '^docs' - 13 of 21
grep -ril mdmax src test scripts - 0 files
find src -name '*.ts*' | xargs wc -l | tail -1 - 21,415 lines (all inherited from the md app clone)
docs/engine/PLAN.md - v0.6.0, supersedes v0.5.0, 1,748 lines, 106,298 B
docs/engine/dossier.html - 408,550 B
docs/engine/sgnk-markdown-engine-dossier.pdf - 449,691 B
```

[measured, verified live in this session]

Three of the last four commits before the consolidation were cosmetic fixes to a PDF — page overflow, cover page, logo mark. **The plan names this failure in §9.7 — which is the point. Naming it did not stop it, because §9.7 is itself another sentence.**

**Countermeasure.** The standing rule in §11 P10 is correct and must not be softened: the unit of progress is **item 1's test going green** — not "a test", *that* test. A rule that says "one assertion red to green against engine code" can be satisfied by writing a compiler. An assertion naming a second human cannot.

N2 — The editor floor never shipped, so no collaboration feature was ever reachable

**Mechanism.** The tenancy work is described in the plan's own words as work that *"will keep losing priority to compiler work precisely because it is uninteresting."* It is owned by the only person who can also do the interesting work. It never starts.

**Earliest warning sign — and it is worse than "not started". The inherited base is architecturally single-tenant and has to be replaced, not extended.** All four sites verified live in this session:

```
src/modules/auth/domain/allowlist.ts
 return login.trim().toLowerCase() === allowed.trim().toLowerCase();
src/config/env.ts:47
 ALLOWED_GH_LOGIN: z.string().min(1).default("sagnikmitra"),
src/config/env.ts:86
 GITHUB_REPO: z.string().min(1).default("sagnikmitra/md"),
src/container/dependency-container.ts:42
 const AUTHOR = { name: "Sagnik Mitra", email: "sagnikmitra123@gmail.com" } as const;
 ..threaded into four use-cases at lines 66, 70, 75, 80
src/app/(workspace)/
 - contains one file: .gitkeep
grep -c -i 'comment' firestore.rules -> 0
```

[measured, verified live in this session]

`firestore.rules` declares the entire multi-tenant model — `vaults/{vaultId}` with `memberUids`, `roles[uid]`, `ownerUid`, `helpers` `isVaultMember` / `canEditVault` / `isVaultOwner`, plus `notes/{noteId}`, immutable parent-linked `revisions/{revId}` with `allow update: if false`, `shares/{slug}`, `billing/{uid}`, `usage/{uid}/months/{month}` — and **exactly one source file touches Firestore, only to construct the singleton**. The schema exists. Zero lines implement it.

**Kill gate K7 is not a risk. It is the current state.**

**The honest counterweight, stated because it changes the estimate downward.** The `against` area found it and reported it against its own case: the rules file is 17,304 bytes, committed 2026-07-25 in the largest commit in the repo, *before* the engine programme started, and there is a 619-line share module in `src/modules/share`. So the floor is not zero and the "he finds it boring" story is contradicted by the fact that he built it first. This makes item 1 **cheaper**, not less urgent.

**Countermeasure.** Item 1, committed RED today: a second GitHub login who is not `sagnikmitra` opens a document they do not own, writes one character, and the persisted commit carries **their** name and email; plus a 403 for a role they do not hold. That single test invalidates the allowlist, the single server token and the `AUTHOR` constant simultaneously, and it is the only assertion whose passing makes any part of the Google-Docs story reachable.

### N3 — The anchor did not hold up when it was re-derived, or did not transfer to ranges

**Mechanism, and it is not the obvious one.** The risk is not that 99.627% falls. It is that it *does not fall*, everybody is reassured, and the number was measured on a distribution that does not resemble the product. Re-deriving a statistic on the corpus that produced it reproduces that corpus's bias with perfect fidelity.

**Earliest warning signs, visible today — three of them, and they are independent:**

1. **Nobody has re-derived it.** Not one of the areas that quoted it. There is no derivation script anywhere in the repository: a grep for `99.627`, `41,642` and `41642` returns hits only in prose — six documents and the research JSONs, where one agent records it verbatim as *"[own-measurement, PLAN §1.1a, not re-run by me]"*. Under the plan's own rule P2, **the flagship number does not currently qualify as a measurement**. [measured]
2. **The source section is internally inconsistent, and nobody noticed until the final gate.** `docs/engine/PLAN.md §1.1a` reports 12.9% of block-versions presenting more than one candidate and scoring 94.94% correct / 2.66% false on that stratum. But 12.9% of 32,919 surviving block-versions is 4,247, and 2.66% of that is **~113 false matches** — while the same section states 0.050% false on 28,170 anchorable blocks and *"All 14 false matches on anchorable blocks were hand-audited, not sampled."* **113 versus 14. One of those numbers is wrong and the document does not say which.** [primary]
3. **It is a BLOCK figure and comments anchor RANGES.** The one replication of the range case gives **3.44x, not 30x**, with refusal roughly doubling. And **~1 heading in 4 cannot carry an anchor at all** (18.3% / 25.4% / 24.7% under three tokens across three corpora) — headings being what everything points at. [measured]

**Two claims in this narrative were themselves killed and must not be repeated.** The pre-mortem agent argued that the corpus is 86.6% append-dominated and that this "agrees to 0.4 percentage points" with S1's 87.0% exact-hash resolution rate. Its verifier **REFUTED both**: under the pinned corpus's own exclusion policy the zero-deletion rate is 67.2%, and on *genuine* consecutive revision pairs (excluding 642 file creations that have no predecessor) it is **6.1%**, with 53.0% being pure deletions. The two quantities were never comparable — one is a per-block-version resolution rate, the other a per-file-revision edit shape. **N3 may still be right; it is not right for that reason.**

**Countermeasure.** Make the re-derivation a gate and specify it by **edit shape**, not by authorship. Before any number is published, report correct / false / refusal in four strata — pairs deleting 0 lines, 1–2 lines, more than 2 lines, and pairs containing a detected paragraph reflow — with `n` stated per stratum, plus a `diff3`-assisted baseline arm. **This requires a corpus this team did not write, selected by edit shape**, and that is a real acquisition task: none of the three pinned roots qualifies against a 25%-of-pairs-delete-more-than-two-lines bar (`md` 0.9%, `knowledge` 9.9%, `frontmatter` 8.3%). If the >2-deleted-lines stratum yields fewer than ~200 block-versions on every available corpus, **the honest finding is that the number has never been tested on the case it exists for**, and it must be restated as *"exact-hash resolution over an append-dominated corpus"* — true, defensible, and much smaller.

### N4 — Users rejected the refusal model; orphaned comments felt like a bug

**Mechanism.** "Refusing is cheap" is true for the engine and false for the user. An orphaned thread is indistinguishable from data loss in a product whose #1 competitive pain theme is that sync silently destroys things. Users file it as a bug, support explains it is by design, and **the explanation is worse than the bug**.

**Earliest warning signs, visible today:**

- The `collab-primitives` area wrote both the threshold and the failure mode before anyone built anything: *"If, in the pilot, orphaned threads exceed ~10% of all threads per week of normal editing, the honest-refusal design is correct and unshippable at the same time — users will read every orphan as a bug."* **Nobody publishes an orphan rate. That threshold was invented by this research and no competitor discloses one.** [primary]
- The only production system of this design orphans roughly a fifth of its anchors. arXiv 1512.06195, read directly: **20,953 highlighted-text annotations; ~22% can no longer be attached to their live pages; of those, only ~12% are recoverable from web archives, leaving 88% orphaned; and 53% of those still attached are in danger.** [primary] (Note: this programme carried these figures wrong in every digit — as 20,953 / 27% / 61% / 3.5% — from a search-result summary. See §12.4.)

- The demand evidence points the same way. Combined installs of **every** markdown comment plugin in the largest markdown ecosystem = **24,569**, against Dataview's **4,659,822 — a 190× gap**. [measured] An independent re-measurement of the same registry puts it at **98.6×** against the best single plugin, or **24.9×** against all 40 summed, and flags that cumulative download counts reward age. **Use the range 25×–190×, not the headline.**
- **D3 removes the exact property the research names as the competitive wedge:** "*range-anchored comments with anonymous no-account share links*" (HedgeDoc has a working CriticMarkup + CM6 comments proof-of-concept with a live demo, 2026-04-04; Relay has 175,904 installs).

**Countermeasure, two parts.** (a) Ship the orphan rate as a **visible product metric from the first comment written**, and instrument the refusal path before the resolve path. (b) Reopen D3 as a tier split rather than a global rule — anonymous read-and-comment on a share link, login required to resolve or to edit. That preserves the wedge and the account model at once, and owner seats still bill while commenters do not. This is open decision 10.2 and it is the single cheapest de-risking edit in the plan.

**Honest caveat on the anonymity argument.** The final-gate verifier rated the anonymity inference **UNSUPPORTED**: the 190× gap is attributed inside the same artifact to real-time collaboration presence — "*the wanted half is the COLLABORATOR, not the annotation*" — not to login friction. No measurement anywhere attributes adoption to anonymous access. So the tier split is recommended on funnel logic and on the pre-registered prediction that four of five reviewers stall at a login wall, **not** on the 190× number. Do not cite the 190× as evidence for anonymity.

**N5 — A big company shipped it: OKF already exists**

**Mechanism.** Not that Google out-features MDMAX. That Google's format becomes the thing agents already speak, and MDMAX's distinctive decisions arrive as second implementations of a published spec.

**Earliest warning signs, visible today, from a spec nobody in this programme had read until the final gate:**

```
okf/SPEC.md v0.2 37,544 bytes
sha256 5a3311d270bebb16d558010e75064f5b75323f284992641732b1c8097511f948
GoogleCloudPlatform/knowledge-catalog 8,134 stars, created 2026-05-04, pushed 2026-07-29
scaccogatto/okf-skills 206* 0W0X/models 85* 0dust/OKFy 62*
openknowledge-sh/openknowledge 33* xSAVIKx/okf-skills 29* jkroepke/okf-crossplane-v2 24*
- all pushed within three weeks of 2026-08-01
```

[primary, every particular independently verified]

- **D9 is already OKF §10.2, in one sentence:** "*What sits behind a resource (a Skill, a script, a container) is a packaging choice; OKF fixes the interface, not the packaging.*"
- **MDMAX's anchoring rationale is already OKF §5.1:** "*Labels are keyed rather than positional ( sources[0] ) because agents constantly rewrite these documents: a positional index misattributes silently the moment the list is reordered, whereas a stable id survives reordering.*" — and its carrier is a plain markdown footnote [^ga4-schema], an in-file, renderer-visible carrier, chosen by Google Cloud despite everything this programme believes about degradation.
- **The runtime is coming:** §12 of the spec defers "*the receipt and verdict wire formats, and the attestation lifecycle around a run. The attester ABI, portability, and sandboxing.*"

**The honest counterweights, which are real:**

- OKF does **not** solve block identity. Its identity is the file path (§4, §6), and per-claim attribution is a footnote label joined to sources[].id — that addresses source lists, not spans of prose. A case-insensitive grep for span, character offset, byte range, per-paragraph, inline annotation across all 37,544 bytes returns 0, and the spec **explicitly refuses** confidence scores: "*a score is subjective, unportable across consumers, and goes stale.*" **The specific gap MDMAX names is genuinely unfilled by Google.** [primary]
- knowledge-catalog 's README states "*This repository and its contents are not an official Google product.*"
- OKF §11's conformance bar is deliberately near-zero (parseable frontmatter with a non-empty type), which is a weak standard to be captured by — **and it is also the opening:** a normative instruction that consumers be permissive is unenforceable across thirty uncoordinated renderers, and the tool that tells an author which of them actually complied is exactly mdmax.cert.

**Countermeasure.** Open decision 10.5 is right and must be executed rather than deferred: **OKF is a target, not a host.** Ship it as the first column of mdmax.cert, emit its §5 provenance fields ( sources, generated, verified, status, state\_after ) rather than inventing a vocabulary, and ship the one thing the OKF ecosystem provably lacks — a **producer-side** checker that runs on arbitrary markdown rather than on an already-conformant bundle. mdmax.cert <file> --targets okf/v0.2 answering "*what would break if this became an OKF concept*" is a column nobody has, and it converts the competitor into a distribution channel.

**N6 — The container was built and nobody used it**

**Mechanism.** Not "nobody wanted it". It has been tried, and **the effectiveness check failed.**

**Earliest warning signs, visible today:**

- **repomix issue #71**, opened 2024-09-08, still open at 2026-08-01 — 22.8 months. The maintainer designed the feature in his first reply; a contributor shipped PR #113 on 2024-10-09; the maintainer gated it on evidence: "*I haven't fully grasped the essence of file splitting effects yet, so I would appreciate it if you could verify the effectiveness of this approach.*" The contributor reported back the next day: "*Well not quite what I hoped regarding getting it to read all the split files.*" Last comment, 2025-09-08: "*Did we see resolution of this?*" [primary, every quote and timestamp verified]
- **The only live-model measurement of a projection bundle that exists anywhere** (okf-skills, 2026-07-27, twelve fresh agents, blind grading, six questions) found the bundle arm spent **257,602 tokens against 244,805 without** — "**the bundle arm cost more, not less. 5.2% more tokens over six questions, and it opened more files, not fewer.**" Correctness 22/26 (85%) vs 20/26 (77%). **Their own authors say n=1 per cell is not quotable as a headline;** quote the direction, not the magnitude. [primary]
- **Revealed demand is 846:1 against** ( llm-code-format 387 downloads/month vs repomix 327,543).
- **The prior art for the unpack half already ships with more stars than repomix.** Aider (47,848 stars) specifies the consumed bundle format verbatim in wholefile\_prompts.py and parses it back to disk in wholefile\_coder.py::get\_edits(), already stripping \*, :, backticks and leading # from the locator;

`base_coder.py::choose_fence()` already computes a collision-proof delimiter by scanning all member content. gpt-engineer (55,173 stars) is the same category. **The "27,551-to-0 asymmetry" claim was REFUTED** — the searches looked for repos *named* "unpack" rather than for the capability under its industry name.

- **The engineering is fine and the product is not.** `container-thesis` proved 1084/1084 byte-identical at +0.892% overhead and 120× cheaper git storage per edit than a zip — and then killed its own product in the same report: **the pack must never be the write surface, because on the realistic merge-across case boundary repair is 119 of 120 WRONG**, returning a confident position that does not contain the true split.

**Countermeasure.** Do not build the container. If it is ever revisited, it is gated on running the live-model experiment on a corpus the model cannot read whole, and on a positive result. Until then, **\$4.4's 25–168× and \$3.1's 242× must be stated as artifact-size ratios and must never imply a token saving in an agent loop** — the only measurement of that quantity points the other way.

## N7 — The team optimised for a document rather than a product

**Mechanism.** Honesty converts into inertia. Each correction makes the document more valuable and harder to abandon. **And the corrective machinery itself became a source of sentences.**

**Earliest warning signs, visible today.** All of §12.3 is this narrative's evidence; the compressed version:

```
21 commits, 13 docs-prefixed, 1 author [verified live]
858,241 bytes of presentation material (dossier.html + PDF)
 for an engine with zero lines of code [verified live]
docs/engine/PLAN.md carries version: 0.6.0 and supersedes: v0.5.0
 - a changelog and a supersession chain [verified live]
0 live model calls and 0 user conversations across 34 research areas
```

**And the sharpest sign is inside the corrective machinery itself.** The pre-mortem agent attacked §9.1's "18 of 18 headlines refuted" statistic — the number that selected the entire MVP — and reported it was really 12 of 18. **Its verifier REFUTED that:** the 12/18 was an artifact of matching `/headline/i` against one JSON field; extended to the `reason` and `correction` fields the report itself quotes from, **it is 18 of 18 adjudicated, 18 of 18 negative**, and two of the five areas declared "never adjudicated" are the exact two whose examples §9.1 cites. **Self-flagellation is also sentences.**

**Countermeasure.** Rule P2 must bind the *plan*, not only the research: every number in this document gets a committed script under `scripts/derive/` plus its captured output, or it is deleted from the document. Start with §9.1 and with 99.627%, because those are the two numbers that picked the MVP and the engine.

## N8 — The evidence base evaporated

**Mechanism.** Not a wrong decision — a total, silent, recoverable-by-nobody loss. At the time this narrative was written, the entire engine programme existed on one laptop, on one branch, most of it not even committed to that branch: 14 untracked files totalling 4,965,476 bytes, including `docs/mdmax/PLAN.md` itself and `docs/engine/research/corpus-manifest.json` — the artifact created specifically to fix the seven-irreproducible-counts problem. None was gitignored; they were simply never added. `origin/main` had not moved since 2026-07-25 and `git branch -r --contains` returned nothing.

**Live state, verified in this session (2026-08-01), and this is the one narrative that has been largely retired:**

```
git rev-list --left-right --count origin/main...HEAD → 0 17
git branch -r --contains HEAD → origin/engine/plan-and-diagnostics
git ls-files docs/engine/research/ → 8 files, including corpus-manifest.json
wc -c package.json → 3,076 (restored from 17 bytes)
HEAD = 1bd4dad "docs(mdmax): consolidate the plan, pin the corpus, commit the research base"
```

[measured, verified live in this session]

**The branch is pushed. The research base is tracked. The remaining exposure is smaller and specific:**

- `origin/main` is still 17 commits behind; the work lives on one remote branch, which is fine, but the merge has not happened and no one has reviewed it.
- Agent litter remains untracked in the repo root: `.scratch-carrier.mjs`, `.scratch-lossy.mjs`, `arx.xml`, `cx.html`, `polyg.yml`, `docs/engine/build/__pycache__/`. That is open decision 10.6.
- **Learned Rule #48 records that subagents in this workspace have committed against an explicit prohibition**, so the working tree is still not a safe place to keep the only copy of anything.

**One claim in this narrative was REFUTED and the correction is operationally useful.** The pre-mortem reported that `corpus_id` **"DOES NOT REPRODUCE"** under any of 64 serializations and that this was a compounding failure. **It reproduces.** I re-derived it in this session:

```
recipe: for every root, build "<root>/<path>", pair with the file's sha256 as "<root>/<path>:<sha256>",
 sort the list, join with "\n", NO trailing newline, sha256 the UTF-8 bytes
result: 3a010b1649899795d79274fc528dbece97fdabf4ff0f81cc02ab619c048c51a4 over n = 1,084
 == the declared corpus_id. total_files 1084, total_bytes 25,548,765 also reproduce.
```

[measured, verified live in this session]

**But the sub-claim stands and is unfixed:** there is still **no derivation script committed**, so rule P2 is genuinely unmet for the artifact created to enforce P2. Ten lines. Commit them beside the manifest and add a `derivation` field naming the exact serialization, so no future reader has to rediscover it — one already failed to, and published the failure as a finding.

**Countermeasure.** Commit the ten-line derivation script. Merge or PR the branch to `main`. Clear the litter. Total cost: under an hour.

N9 — The certificate was perishable and its valuable columns were unprobeable

**Mechanism.** `mdmax cert` was selected by the rule "choose the deliverable whose output shape cannot be a headline." True. But its output also cannot stay true, and cannot cover the targets people actually care about. A conformance matrix is a perishable good with no maintenance budget in the plan, and the surfaces users publish to are closed SaaS you cannot vendor, version-pin, or probe.

Earliest warning signs, visible today — and they are in the certificate's own research area:

- Its kill condition (2), verbatim: "THE UNCERTIFIABLE SURFACE IS THE ONLY ONE ANYONE CARES ABOUT. `github-blob` cannot be probed without pushing content, and `Obsidian`, `Notion`, `Typora`, `Bear`, `Slack`, `Discord` and the chat renderers cannot be probed at all — that is **7 of the ~12 surfaces** on the brief's list."
- Its `unverified` block opens: "TARGETS I NEVER RAN — this is the biggest gap and it is more than half the brief's list... I have **NO empirical Obsidian data at all.**"
- Its kill condition (1) sets the floor: "the honest figure was **4.28%**, not the 43.71% the naive diff reported, and 4.28% is not a large number. If a neutral corpus halves it again, stop." A 4x margin on a self-authored, AI-assisted corpus is thin.
- The pinnable engines move: **markdown-it 15.0.0 published 2026-07-30, marked 18.0.7 published 2026-07-21** (npm registry `time` field). `[primary]`  
**Caveat, and it is the weakest link in this narrative:** a version bump is not a behavioural change, and nobody established that these releases changed rendering behaviour. "The certificate is perishable" is `[inference]` from release cadence.
- **K5 currently states no threshold at all**, so it can never fire.

**The counterweight, and it is strong.** This is the only capability that four independent adversarial prior-art searches could not find shipped: one returned `total_count 0/0/1`; another enumerated `markdownguide.org`'s 28 construct ids and found **zero** of frontmatter, wikilink, callout or mermaid while measuring those constructs at 56–91% prevalence; a third inspected the top ten of 4,218 repositories and found only parsers; a fourth ran four npm searches for 32 results and zero relevant. And it finds real disagreement: **GitHub's own API renderer and blob renderer were measured disagreeing on mermaid, on identical bytes, live.** `[measured]`

**Countermeasure, three parts.** (a) **Give K5 the number the research already produced:** kill if semantic BROKEN falls below 1% of blocks on a corpus this team did not write. (b) **Split the matrix into a PROBED tier** (offline, vendored, version-stamped) **and a DECLARED tier** (contract plus canary for closed SaaS), labelled in the output, so the certificate never manufactures confidence about Obsidian or Notion. (c) Run the 60-day decay test: recapture the closed rows at 60 days; if zero cells moved, the dataset does not decay, a single copy is permanently level with yours, and **the certificate is a lead, not a moat.** Also: check the licence of every renderer the bench proposes to vendor. Nobody has.

N10 — Two decisions deleted the wedge the demand research identified

**Mechanism.** D3 and D4 were made on architectural grounds against research findings that were about demand, and the two bodies of evidence were never put side by side.

Earliest warning signs, visible today:

- **D3 vs the collaborator half.** The `collab-primitives` area names the competitive threat as "range-anchored comments with anonymous no-account share links" and diagnoses the plugin-adoption gap as "a plugin cannot deliver the collaborator half" — and login-required removes exactly the half in question at the top of the funnel. Logged in the plan as "a tradeoff"; the demand side was never quoted against it. (See N4's caveat: the anonymity link is UNSUPPORTED as a *measured* claim. The funnel argument stands on its own.)
- **D4 vs the price wedge, and this one is not logged anywhere.** `docs/Frontmatter-Product-Plan.md:171`, verbatim: "**Free git-backed sync forever** (user's own repo = backend; marginal cost ~0) — undercuts every \$4–10/mo sync add-on", filed against pain theme T3 (price backlash), **ranked #3 of 14**. D4 explicitly overturns "your git repo is the backend". `[primary, verified at exactly that line]`

**Countermeasure.** The reconciliation pass must include a **decisions-versus-demand diff**, not only an audit-versus-critic diff. Any decision that deletes a property the demand research ranked in its top five gets a written replacement wedge or gets reopened. For D4 specifically, §4.3 already carries the replacement — "Free sync. It's just git." at £0 with marginal cost ~0 — so the wedge survives in the pricing table even though the architecture changed. **Write that reconciliation down**, because right now the plan reads as if D4 deleted the #3-ranked pain's answer and nobody noticed.

N11 — Bus factor one met a plan with no calendar

**Mechanism.** The sequence is costed at roughly two weeks of work and names no owner and no dates. The only owner also runs a client studio with annual-maintenance-contract obligations. The gates are all serial on one person, and item 1 — the one that matters — is the one he finds boring.

**Earliest warning sign, verified live:** `git log --format='%an' | sort | uniq -c` returns 21 Sagnik Mitra. All 416 commits in the source `md` repo are the same author. **Amit is named as co-author in this document's frontmatter and as co-decider on D1–D5, and has committed nothing;** §1 addresses him as a reader.

**Honest boundary on this claim.** This is a statement about the **code**, not about the partnership. Design conversations, decisions and review are not in version control and are invisible to any measurement available here.

**Countermeasure.** Kill gate K2 already encodes it: if Amit has not committed code by **2026-08-31** (a Monday, thirty days from 2026-08-01), the working assumption is wrong and the plan is a one-person plan that has to be re-costed as one. The three live options are: Amit owns item 1 end to end with his own commits; item 1 is contracted out; or K7 is triggered now and MDMAX is repositioned as a standalone library. **Pick one this week. The plan cannot survive "we will see."**

N12 — D9's escape hatch reintroduced the vulnerability class D9 exists to prevent

**Mechanism.** D9's rationale is that self-declaring formats which **carry** their definition produce security vulnerabilities — XML's internal DTD subset produces XXE; vim modelines produced five CVEs in twenty-four years. The proposed fix is to **name** the capability instead. But naming an executable resource that a consumer then runs is the same attack surface with one extra hop, and it arrives with a false sense of safety because the file itself looks inert.

**Earliest warning sign, visible today.** OKF v0.2 §10.2 already ships exactly this shape — `executor: {resource, receipt}` and `attester: {resource}` naming code the consumer executes, with `okf/bundles/acme_retail/attesters/sql_equality.py` in Google's own reference bundle, and §10.5 step 5 reading "**Attest: the consumer runs the attester over the receipt.**" **And §12 defers "The attester ABI, portability, and sandboxing" to a future revision — i.e. the sandbox story does not exist yet.** `[primary, verified line by line]` If MDMAX adopts OKF as a certificate target (recommended, 10.5) it inherits this surface, and any `fm:vocab: -style` resolution step has the same shape.

**Second warning sign, from the compiler side rather than the format side.** The research instruments themselves hit three pathologies in a single run: a linear region scan took **45.5 seconds on a 1.49 MB file**; two full runs hung on catastrophic regular-expression backtracking; and a traversal bug silently inflated numbers **8.5x**. **Nobody has asked what a hostile `.md` does to the re-anchor algorithm or the shingle index — in a product whose whole premise is ingesting files other people wrote.**

**Countermeasure.** Write D9's threat model down *before* any capability work, not after. Concretely: a named capability resolves only from a pinned allowlist shipped with the engine, **never from a path or URL inside the document**, and `mdmax-cert` flags any document naming an unresolvable or non-allowlisted capability as CORRUPT rather than DEGRADED. Plus a ReDoS and adversarial-input fixture set for the anchor path, and `eslint-plugin-redos` in CI. The repository already carries two live quadratics to prove the point: `WIKILINK_RE` at  $k \approx 2.00$  (**36,865 ms on 320 KB of `[]`**) and `mdast-util-from-markdown` at **12,429 ms against micromark's 1,207 ms** on identical bytes.

N13 — The kill gates fell due and were quietly extended

**Mechanism.** This is the meta-failure that makes every other narrative survivable-looking. K1 and K2 fall due on 2026-08-31. On 2026-09-01, if nothing has shipped, the cheapest available action is not to honour them — it is to observe that the reasons were good, that progress was real, and to move the date. Nothing in the plan prevents that. A kill gate with no owner, no calendar entry and no external witness is a preference, not a gate.

**Earliest warning sign, visible today.** This programme has already extended one implicit gate four times: the mother-markdown container has been cut and reappeared across four documents. And the standing rule *"no further research fan-out until one committed assertion has gone red to green"* was written before the final gate — and the final gate was another research fan-out, of 40 agents over 16 areas. **The rule was in force and the fan-out happened anyway.**

**Countermeasure.** Three mechanical things, all cheap: (a) put **2026-08-31** in a shared calendar with both founders on it, titled "K1/K2 verdict"; (b) write the verdict as a commit to this file on that date — pass or fail, in the document, dated; (c) name the external witness. A gate whose failure must be *written down in public* is a different instrument from one that is merely stated.

12.2.1 Ranking, by probability × severity

Probability is over the next twelve months. Severity is what it costs if it happens. The ranking below applies the kill-audit corrections — N3 has been demoted from the pre-mortem's original position because the measurement that elevated it did not survive, and N8 has been demoted because the fix landed and I verified it live.

RANK	NARRATIVE	PROBABILITY	SEVERITY	NOTE
1	N2 the editor floor never shipped	HIGH	TOTAL	not a forecast — the measured current state. K7 is the base case
2	N1 the research never became code	HIGH	TOTAL	distinct from N2: N1 is "nothing ships", N2 is "the wrong thing ships"
3	N13 the kill gates were quietly extended	HIGH	HIGH	the multiplier on everything else; costs one calendar entry to defuse
4	N11 bus factor one	HIGH	MEDIUM	a multiplier on N1/N2 rather than an independent failure
5	N5 OKF outruns us	MEDIUM-HIGH	MEDIUM-HIGH	survivable if OKF becomes a column; fatal if it becomes the format
6	N7 document-as-product	MEDIUM-HIGH	MEDIUM	slow bleed; partial immunity because the team named the disease
7	N3 the anchor does not transfer	MEDIUM	HIGH	credibility kill, not market kill. Its flagship supporting measurement was killed; the block-vs-range gap is the real reason
8	N9 certificate perishable / unprobeable	MEDIUM	MEDIUM-HIGH	capped at two days, so severity is bounded by the cap
9	N4 refusal model rejected	MEDIUM	HIGH but LATE	only bites after the review loop ships
10	N10 decisions deleted the wedge	MEDIUM	MEDIUM	recoverable by reopening D3 as a tier split
11	N8 evidence base evaporates	LOW	TOTAL, IRREVERSIBLE	largely retired 2026-08-01; residual is the un-merged branch and the missing derivation script
12	N12 D9 threat model	LOW	HIGH IF IT HAPPENS	a security incident in a collaboration product is existential for trust
13	N6 container built and unused	LOW	LOW	explicitly out of scope; the risk is that it comes back, not that it fails

**The single cheapest intervention that reduces the most risk:** item 1's test, committed RED today. It retires N1 and N2 simultaneously — the top two, both severity TOTAL — because it cannot be satisfied by prose and cannot pass without replacing the allowlist, the single server token and the `AUTHOR` constant. It also defuses N7 and N11, because the unit of progress becomes a test only one kind of work turns green. **It does not touch N3, N5 or N9** — those need the stratified re-derivation, the OKF column, and the certificate's threshold respectively — and no single cheap action does.

12.3 The mechanism by which this fails, named precisely

Everything in §12.2 is a symptom. This is the disease, and it is worth naming precisely rather than as a platitude, because the platitude version ("stay focused") has already been tried and did not work.

Step 1 — the reward function pays for sentences, and nothing charges for wrong ones.

Every research pass produced a document. Every document was immediately more legible, more complete and more impressive than the last one. Nothing in the loop imposed a cost for a sentence that was wrong: **18 of 18 headlines in the capability run were refuted by their own verifier, always in the flattering direction**, and the cost of that was zero at the time it happened. The final gate's structural fixes — mandatory negatives, pre-registered falsifiers, mandatory corpus id, a kill audit — brought it to **5 CONFIRMED / 10 OVERSTATED / 1 REFUTED**. That is a genuine and rare improvement, and it is also more sentences.

Step 2 — the cheapest way to satisfy the reward function is another research area.

A research area costs tokens and produces a document. Code costs judgement, debugging, and the risk of a red test. When the measured output of a day is "how much did we learn", research always wins. The observable consequence, verified live in this session:

```
21 commits 13 of them docs-prefixed 1 author
0 files in src/ test/ scripts/ contain the string "mdmax"
21,415 lines of TypeScript in src/ — every one inherited from the md-app clone of 2026-07-17
858,241 bytes of dossier + PDF for an engine with no code
0 live model calls and 0 user conversations across 34 research areas
```

**Step 3 — the corpus becomes the artifact under maintenance.**

This is the step that turns a bad month into a bad year. docs/engine/PLAN.md carries version: 0.6.0 and supersedes: PLAN.md v0.5.0 (same path) in its own frontmatter, plus in-body supersession notes ("This supersedes the k=2 fingerprint recommendation in v0.2"; "superseded by §10, §1.1a, §2.3/§2.4 and §3.2 respectively"). [measured, verified live in this session]

**That is version-controlled software with releases and deprecations.** It has a version number, a changelog, a supersession chain, a build pipeline ( docs/engine/build/ ), two rendered distributions (an HTML dossier and a PDF) and a maintenance burden — three of the last four commits before the consolidation were page-break and cover-page fixes to the PDF. The programme did not fail to ship a product. **It shipped a different product, on time, with releases.**

**Step 4 — and this is the trap — every correction increases the sunk cost.**

The corrections are real. The negatives are real. The kill audit is real, and a team that over-reports its own failure rate is rarer and more valuable than one that under-reports it. But each correction makes the document more *right*, and a document that is more right is harder to abandon. The self-criticism is load-bearing structural material for the thing it criticises.

Honesty is being converted into inertia.

That sentence is in §9.7 of this plan, and §9.7 did not stop it, **because §9.7 is a sentence and the mechanism pays for sentences.** The only thing that breaks the loop is a unit of progress that prose cannot satisfy. That is what P10 is, and it is why P10 names *one specific test* rather than "a test".

**The corollary nobody has stated, and it is the reason this section is here rather than in an appendix.** Because the corrective machinery is itself a source of sentences, a risk section can be part of the disease. This one is worth its tokens only if it produces three decisions (§12.1), one calendar entry (N13), one committed RED test (N2), and a ten-line derivation script (N8). **If it produces admiration for its own honesty and nothing else, delete it and go write the test.**

**12.4 What we got wrong — the complete list, kept on purpose**

A plan that only argues its own case is marketing. This list is not curated for length.

- 1. Eighteen of eighteen headlines refuted in the eighteen-area run**, always flatteringly. A 100% one-directional defect rate is a reward function, not noise. The final gate's fixes cut it to **5 CONFIRMED / 10 OVERSTATED / 1 REFUTED**.
- 2. No pinned corpus, for most of the programme.** Seven irreproducible file counts; exclusion policies flipped *within* single areas — one area excluded node\_modules where it supported a conclusion and included it where it supported the opposite conclusion, in the same report. Fixed: corpus-manifest.json, corpus\_id sha256:3a010b16..., and rule P1.
- 3. The container's entire empirical case collapsed.** "He already hand-builds bundles across 37.2% of files" was **92.3% one machine-generated sync tree**. Honest figure ~2.2% of files, ~2.0% of bytes. That area's own pre-registered falsifier **fired** and it recorded "FAILED TO REFUTE, in the founder's favour."
- 4. We justified block identity with "a git pull yields no op stream."** It yields two blobs and a merge base, and a diff *is* an op stream. The 294-revision-pair corpus was **built from git history — the diff was in hand for all 41,642 block-versions and was never used as a baseline**.
- 5. The flagship number does not transfer to the flagship feature.** 99.627% is a BLOCK figure; comments anchor RANGES; the one range replication gives 3.44x, not 30x, with refusal roughly doubling. And ~1 heading in 4 cannot carry an anchor at all.
- 6. Five designs were built on a carrier two measurements had already broken.** The HTML comment is deleted by GitHub's blob renderer (5 standalone comments in source, 0 in a 91,223-byte render) and by both clipboard flavours; five independent areas used it as the safe invisible channel and nobody reconciled it. D2 makes it moot for comments, which is luck rather than design.
- 7. The document became the product.** See §12.3.
- 8. The demand for commenting was reported backwards** at 6.9:1 in favour of multiplayer. Symmetric counting — the same corpus, de-confounded for Reddit permalink URLs containing /comments/ and for N comments score metadata — gives **review loop 169 : collaboration 95, i.e. 1.78:1 the other way**. The original instrument used a broad three-alternative stem on one side and a narrow multi-word-phrase whitelist on the other. **This is the single most consequential correction in the corpus: it is the difference between building Yjs and building comments.**
- 9. We carried an external baseline that was wrong in every digit.** The Hypothes.is orphan study was recorded internally as  $n=20,953 / 27\% / 61\% / 3.5\%$ , self-flagged as "from a search-result summary... NOT read directly". The paper (arXiv 1512.06195) says **20,953 / 22% / 53% / 12%**.
- 10. Two tooling guards report false failures —** workflow-lint.sh inverts on files over the pipe buffer ( grep -q plus pipefail → SIGPIPE 141), and the loop-guard reports denials for calls that succeeded. A harness that reports false failures trains you to ignore it, which is the same disease as one that reports false passes (Learned Rule #65).
- 11. package.json was destroyed to 17 bytes by a research subagent** and the repository was unbuildable for an unknown period, on a branch with no upstream, with the entire plan untracked. Fixed 2026-08-01 in 1bd4dad; verified restored to 3,076 bytes in this session.
- 12. Our own pre-mortem was wrong in the flattering direction three times, and its verifier caught all three.** (a) It claimed corpus\_id "DOES NOT REPRODUCE" under 64 serializations — **it reproduces**, and the recipe was already written in prose in the document being audited; I re-derived it live in this session. (b) It claimed 1,399 revision pairs of which 48.3% delete zero lines — **642 of those were file creations with no predecessor**; on genuine pairs it is 6.1%, an 8x error. (c) It claimed the "18 of 18" statistic was really 12 of 18 — **an artifact of a string-matching rule**; extended properly it is 18 of 18. An absence proven by an incomplete search, a rate confused with a population, and a classifier mistaken for a census — three defects the run was convened to catch, committed by the agent convened to catch them.
- 13. Our splice-fidelity evidence was a tautology.** src.slice(0,s) + src.slice(s,e) + src.slice(e) == src is true for any  $s \leq e$ ; the test returned true for deliberately wrong offsets, absurd offsets and a degenerate range. **The conclusion is still right —** a re-run that *can* fail gave 299/299 files and 2,832 syntax-preserving splices with structure intact — but the stated evidence proved nothing (Learned Rule #68).

14. **We claimed novelty five times and were refuted by a single search each time.** "No precedent for content-derived re-anchoring" — 25 years of prior art (Brush & Barger on MSR-TR-2001-107, US7747943B2, the W3C Web Annotation Data Model's `TextQuoteSelector`, a W3C Recommendation since February 2017). "Nobody unpacks" — Aider at 47,848 stars ships exactly that, and its `choose_fence()` pre-empts our delimiter rule. Hence rule P8: every novelty claim carries the search that produced it.
15. **Zero live model calls and zero user conversations, across 34 research areas and roughly eight million tokens**, in a programme whose stated goals are "easier for AI to consume" and "the Google Docs of markdown". **Every "easier for AI" claim in this document is a prediction, not a result, and is labelled as one.**

## 12.5 The open decisions

Each with a recommendation and what would change it. Decisions already settled as D1–D10 are not relisted; these are the ones the record answers twice, or not at all.

#	DECISION	STATUS	RECOMMENDED	WHAT WOULD CHANGE IT
W-A	What the Max-tier download button produces (§12.1.1)	<b>OPEN</b> — answered two ways	zip of clean <code>.md</code> + generated <code>INDEX.md</code> + <code>VOCABULARY.md</code>	the founder answering K-D with "I meant one artifact". Ten minutes, never asked
W-B	Does D6 permit reference-carried payloads? (§12.1.2)	<b>OPEN</b> — boundary never drawn	<b>no.</b> Add the VOID verdict to the certificate; write the exclusion into D6	one user, unprompted, expecting a <code>![[X]]</code> -style reference to render
W-C	Library/import model vs no runtime (§12.1.3)	<b>OPEN</b> — answered by deletion, never asked	no runtime; extensibility is D8 only; record as D11	the founder saying the Python analogy meant discoverability, not importing — in which case it is a catalogue and everyone agrees
10.1	Backend — Firestore is in the code and the mockup; the product plan says Supabase eleven times	<b>OPEN</b>	pick <b>Firestore</b> , correct the product plan	a hard cost or region requirement Firestore cannot meet
10.2	<b>D3 — anonymous commenters</b>	<b>OPEN, under review</b>	<b>reopen as a tier split:</b> anonymous read-and-comment on a share link, login to resolve or edit. Owner and editor seats bill; commenters do not	the user probe showing reviewers do <i>not</i> stall at a login wall. Note the anonymity-as-wedge claim is <b>UNSUPPORTED</b> as a measured fact — this rests on funnel logic, not on the 190x
10.3	Real-time multiplayer architecture	<b>OPEN, not this cycle</b>	server-authoritative governs — but <b>both of the originally stated reasons are wrong</b> (one rests on a struck, <code>%</code> -commented line absent from the published paper). Justify on the Relay bug and on adoption (73/wk vs 7.2M/wk); strike the other two. The product plan still has three rows specifying Yjs and they contradict this	the review loop reporting an orphan rate that only a CRDT can fix
10.4	Does MDMAX have a business model?	<b>DECIDED</b>	<b>No.</b> It is a library subordinate to the editor. <code>cert</code> is the only public artifact. That is the answer	nothing short of <code>cert</code> clearing its own kill number by a wide margin
10.5	OKF	<b>RECOMMENDED, execute now</b>	<b>a target, not a host.</b> One column in the cert matrix; emit its §5 provenance fields rather than inventing a vocabulary; ship the producer-side checker OKF's ecosystem lacks	OKF shipping its own producer-side checker, which would close the one opening
10.6	Agent litter in the repo root	<b>OPEN, trivial</b>	remove or gitignore <code>arx.xml</code> , <code>cx.html</code> , <code>.scratch-*.mjs</code> , <code>polyg.yml</code> , <code>docs/engine/build/_pycache_</code>   — all still untracked as of this session	nothing
10.7	Where the branch lives	<b>OPEN, new</b>	merge or open a PR from <code>engine/plan-and-diagnostics</code> to <code>main</code> (currently 17 ahead, 0 behind) and commit the ten-line <code>corpus_id</code> derivation script under <code>scripts/derive/</code>	nothing
10.8	Who owns item 1, and by what date	<b>OPEN, and this is the one that decides the company</b>	Amit owns it end to end with his own commits, <b>or</b> it is contracted out, <b>or</b> K7 fires now. Pick this week	nothing. This cannot be deferred without deferring the kill gates, which is N13

## 12.6 What would have to be true for this to be worth twelve months

Six things. Two of them fall due within thirty days, and they are the only two that are not negotiable.

**Within thirty days — by 2026-08-31 (a Monday):**

- A second GitHub login, who is not `sagnikmitra`, has written to a document they do not own, under their own attribution, and the test that proves it is green in CI.** This is kill gate K1. If it is still red, frontmatter is a library and not a company, and **we say so out loud** rather than holding the "Google Docs for markdown" label over code that admits one user.
- Amit has committed code.** This is kill gate K2. It is not a loyalty test; it is the only available measurement of whether the bus factor is one or two, and every schedule in this document assumes two.

**Within twelve months:**

- The review loop ships and reports an orphan rate that users tolerate.** Comments anchored to text, as a sidecar, never in the `.md`; the D2 gate green (delete the sidecar → `git status` clean, every file byte-identical); and the orphan rate visible as a product metric from the first comment written. The pre-registered threshold to beat is **~10% of threads per week of normal editing** — invented by this research because no competitor publishes one, so treat it as a hypothesis and publish the real number whatever it is.
- Somebody who is not Sagnik or Amit pays for it.** Any tier. The demand evidence in this programme is entirely scraped forum posts and plugin install counts; **W3 stands at zero**. The base rate for the give-away half is brutal and measured: markdownlint ran eleven years and four months to 2.89 million downloads a week and zero dollars.
- The anchoring number is either re-derived and honest, or retired.** Stratified by edit shape, on a corpus this team did not write, with a committed derivation script and captured output, and a `diff3`-assisted baseline arm. If the >2-deleted-lines stratum is too small to measure on every available corpus, the number is

restated as "exact-hash resolution over an append-dominated corpus" and never published as 99.627%.

6. **The three incompatible wants of §12.1 have been decided by the founder and written down as decisions**, so that the fifth document in this lineage does not rediscover the container.

**And one thing that must stay true throughout:** no further research fan-out of any kind until item 1 has one committed assertion going red to green. The standing rule was already in force once and a 40-agent fan-out happened anyway. **The marginal research hour is now provably worth less than the marginal test.**

12.7 What this section does not cover, and what would falsify it

What it does not cover.

- **Legal, licensing and intellectual-property risk.** No freedom-to-operate search was run beyond noting that US7747943B2 exists in the anchoring prior art. No licence review was done for any renderer the certificate bench proposes to vendor ( cmark-gfm , kramdown , goldmark , pulldown-cmark , comrak , markdown-it , marked , commonmark.js ), nor for Linguist's languages.yml or Unicode's confusables.txt . Outline's licence resolves to NOASSERTION/"Other" (BSL-family), which matters if anyone treats it as more than evidence.
- **Financial risk.** No runway, no burn rate, no opportunity cost of the founders' client work is modelled anywhere in this document.
- **Operational and privacy risk of hosting other people's documents.** GDPR, data residency, the incident-response story, and what happens when a shared vault contains something a court wants. Nothing.
- **Concurrent-edit correctness.** The collaboration thesis rests on block-granular three-way merge, and the area that chose it states plainly: "NO CONCURRENT-EDIT PATH WAS TESTED." Zero areas consulted the structured-merge literature (Spork, jdime, semistructured merge, Apel et al.) — the one field directly load-bearing on the chosen architecture.
- **Any risk that depends on a user.** With W3 at zero, every sentence anywhere in this document about what a user will feel — about an orphaned comment, a login wall, a certificate, a degraded render — is [inference] , including every sentence in this section.
- **Any risk that depends on a model.** With W2 at zero, every "easier for AI" claim in this programme is a pre-registered prediction. It is labelled as one everywhere it appears, and it should stay labelled until somebody makes an API call.

What would falsify this section.

IF THIS WERE OBSERVED	THEN THIS SECTION IS WRONG, AND HERE IS WHICH PART
Item 1's test goes green within thirty days with a day of work	N2's severity assessment is inflated; the floor was closer than the code reads, and the ranking's top item is wrong
Amit commits and takes item 1	N11 dissolves and N1/N2's probabilities drop from HIGH; the whole ranking should be re-run
A user probe finds people asking, unprompted, to reference one document inside another and expecting it to render	§12.1.2's recommendation is wrong; D6 needs an exception and the VOID rule is too strict
The founder answers K-D with "I always meant one artifact"	§12.1.1's recommendation is wrong; A2 is the answer and the container work returns to scope with a real requirement behind it
The stratified re-derivation reproduces 99.627% and the diff3 -assisted baseline lands below 95% on the >2-deleted-lines stratum	N3 is wrong and the anchoring invention is load-bearing rather than marginal. This is the single result that would most change the engine's scope
mdmax cert clears its kill number by a wide margin — well over 5 of 30 probed users saying they would install it	N9 is wrong, and open decision 10.4's "no business model" answer deserves reopening
A hand-audit shows the LCS-diff baseline's false-match rate above 0.5% on the 10–19 and 20+ token buckets	the "four of six capabilities have substitutes" argument weakens materially, and MDMAX's refusal discipline is doing real work
Thirty days pass, K1 and K2 fail, and the founders honour them out loud	N13 is wrong, and that is the most valuable falsification on this list

**The last word, and it belongs to the research rather than to this document.** One area wrote about itself: "this entire area was a code review wearing a research costume and should be re-run as one." That sentence is true of more of this programme than one area — and it is also the reason to believe the next thirty days can go differently, because a team that writes that sentence about its own work is a team that will publish a bad benchmark result. **That property is the rarest thing here. It is worth more than any of the measurements, and it is the only asset in this document that a competitor cannot copy.**

13. Appendix — the evidence base, the corpus, and where everything lives

This appendix exists so that a person who has never seen this project can pick up any number in this plan, find where it came from, and re-derive it. It is a reference section, not an argument. Nothing here is new analysis; everything here is a pointer, a recipe, a verdict table, or a warning label.

Read this appendix if you are about to:

- quote a number from this plan in a pitch, a blog post, a README or a funding conversation;
- decide whether a research finding is strong enough to build on;
- re-run a measurement because you suspect it drifted;
- onboard someone who was not in the room.

Three conventions used throughout the plan and this appendix.

1. **Every claim carries a tier.** `[measured]` we ran it in this programme · `[primary]` we read the source document or the source code · `[secondary]` we read somebody else's report of it · `[inference]` we reasoned from something measured · `[SIMULATED]` we replayed data through code rather than reading it out of a live system.
2. **Every number about our own files cites a corpus id.** The id is `sha256:3a010b1649899795d79274fc528dbece97fdabf4ff0f81cc02ab619c048c51a4`, abbreviated throughout as `sha256:3a010b16...`. A number about local files with no corpus id is not a measurement; treat it as an anecdote.
3. **A verdict is not a fact.** The research corpus was adversarially verified, and the verifiers were themselves audited. Both layers have measured error rates. §13.3 gives them.

13.1 The pinned corpus

13.1.1 What it is, and why it exists

The pinned corpus is a frozen inventory of 1,084 markdown files across three git repositories, recorded with a per-file SHA-256 digest, so that any measurement over "our own markdown" can be re-run later against exactly the same bytes.

It exists because of a failure. Before it was created, the programme published **seven irreproducible file counts**, and at least one area flipped its own exclusion rules *within* a single report — different sections of the same document counting different file sets and presenting the results as comparable. `docs/mdmax/PLAN.md` v2.0.0 §9 item 2 records this as defect #2 of ten. The manifest is the fix.

```
file docs/engine/research/corpus-manifest.json
bytes 198,580
tracked yes, since commit 1bd4dad
schema { policy: {...}, roots: { <name>: { head, file_count, total_bytes,
 files: [{path, sha256, bytes}, ...] } },
 corpus_id, total_files, total_bytes }
```

`[measured]` Totals, quoted exactly as the manifest states them:

FIELD	VALUE
<code>corpus_id</code>	<code>sha256:3a010b1649899795d79274fc528dbece97fdabf4ff0f81cc02ab619c048c51a4</code>
<code>total_files</code>	<code>1084</code>
<code>total_bytes</code>	<code>25548765</code>

13.1.2 The exclusion policy, verbatim

This is copied character-for-character out of `corpus-manifest.json` → `policy`. Do not paraphrase it; the whole point of the manifest is that the rule is quotable.

```
{
 "exclude_dirs": [
 ".git",
 ".next",
 ".obsidian",
 ".trash",
 ".venv",
 "__pycache__",
 "build",
 "dist",
 "node_modules"
],
 "exclude_toplevel": [
 "Mirrors",
 "Paste"
],
 "rule": "all *.md, case-insensitive; skip listed dirs at any depth; skip any dot-directory; skip listed top-level dirs per root"
}
```

Three consequences a reader will trip over if nobody warns them.

1. `build` is excluded *at any depth*. `docs/engine/build/README.md` exists on disk today and is **not** in the corpus. If you enumerate `docs/**/*.md` yourself you will get a different number than the manifest, and you will be wrong, not the manifest. [measured]
2. `Mirrors` and `Paste` are excluded only at the **top level of a root**. In the `md` vault `Mirrors/` holds read-only copies of 53 sibling repositories, and `Paste/` is the knowledge base's ingest inbox. Including either would have made the corpus mostly duplicates. One kill audit explicitly used this rule to dismiss five apparent wikilink resolutions: *"the 5 apparent hits all land in `md/node_modules/next/dist/docs/**`, which corpus policy excludes."* [primary]
3. The policy does **not** exclude machine-generated markdown, and this is the single largest known bias in the corpus. See §13.1.6.

13.1.3 Per-root counts and git HEADs

[measured] Quoted exactly from the manifest, with the live re-check I ran while writing this appendix on 2026-08-01:

ROOT	GIT HEAD AT PIN TIME	FILES	BYTES	SHARE OF BYTES	HEAD TODAY
md	02c22ec47a3a73fb25a14b11be233ad3d00a8b48	756	21,068,853	82.46%	unchanged
knowledge	464eb666946c8cb4ccf42c68f4464613e5fc4999	272	3,570,923	13.98%	unchanged
frontmatter	798ebbf3250a5b2a53785859d79ae15f55ebebeb	56	908,989	3.56%	moved to 1bd4dad
total	—	1,084	25,548,765	100%	—

Root name → filesystem path. **This mapping is not in the manifest**, which is a defect: the manifest records root *names* only, so on any machine other than this one it is unresolvable without out-of-band knowledge.

md	→ /Users/sagnikmitra/Desktop/GitHub/md	(github.com/sagnikmitra, private vault)
knowledge	→ /Users/sagnikmitra/Desktop/GitHub/knowledge	(github.com/sagnikmitra/knowledge)
frontmatter	→ /Users/sagnikmitra/Desktop/GitHub/frontmatter	(this repo)

The `frontmatter` root moved after the pin. Commit `798ebbf` was HEAD when the corpus was frozen; HEAD today is `1bd4dad` ("docs(mdmax): consolidate the plan, pin the corpus, commit the research base"). One known file was created after the pin and is therefore **absent from the corpus**: `docs/mdmax/PLAN.md`. Kill-audit A recorded this independently — *"My enumeration reproduces the manifest to the byte (md 756/21,068,853; knowledge 272/3,570,923) with one known addition, docs/mdmax/PLAN.md, created after the pin."* [primary] So **this plan is not in its own corpus**. Any measurement that needs to include it must say so and state the new denominator.

13.1.4 The recipe to re-derive the corpus id — write this down

This is the most operationally important paragraph in the appendix, because **four independent agents failed to reconstruct this hash and published a null result about it**. The recipe was sitting in prose one line above the number they were auditing.

The failure record, quoted exactly:

WHO	WHAT THEY REPORTED
premortem (research)	"corpus_id sha256:3a010b16... DOES NOT REPRODUCE under any of 64 serializations of the manifest's own (path, sha256) list"
moat (unverified §5)	"I could NOT reproduce the declared hash 3a010b16... from the (path, sha256) list; my guessed serialization produced a55879ba..."
agentic-docs (unverified §7)	"could NOT reproduce the id string; no derivation script exists in the repo"
custom-pointers (unverified §4)	"The corpus_id non-reproducibility is a NEGATIVE result over 30 serializations I chose"
a verifier ( markdown-base line)	two tab-separated constructions gave sha256:0cbbdbd4... and sha256:2284b06e..., neither matching

Against that, two agents reproduced it. The `premortem` verifier REFUTED the null result: *"It reproduces exactly on the first try under a natural serialization the researcher's 64-variant sweep evidently never combined... My sweep of 208 variants found it under two labels."* And the `premortem-integrator` synthesis re-derived it independently and ruled: **"IT REPRODUCES, AND I RE-DERIVED IT THIS SESSION."** [primary]

I re-derived it a third time while writing this appendix. It matches exactly. [measured, 2026-08-01]

THE RECIPE, stated as an algorithm.

1. For every root in `roots`, and every file entry in that root's `files` array, build the string "`<root>/<path>:<sha256>`". The root name is joined to the path with a forward slash; the sha256 is joined to that with a single colon. No spaces, no tabs, no quoting.
2. Collect all such strings into one flat list across all roots. Do not group by root.
3. **Sort the list** with a plain byte/codepoint sort (Python's `sorted()`, `LC_ALL=C sort`). Not by root, not by insertion order.
4. Join with a single `\n`. **No trailing newline.**
5. SHA-256 the UTF-8 bytes of that string. Prefix the hex digest with `sha256:` .

**The ten lines that do it.** Commit this as `scripts/derive/corpus-id.py` — the plan's own rule P2 ("no headline without a committed derivation script and its output") demands it, and committing it retires this entire class of false alarm permanently.

```
#!/usr/bin/env python3
"""Re-derive corpus_id from docs/engine/research/corpus-manifest.json. Exit 1 on mismatch."""
import hashlib, json, sys

m = json.load(open("docs/engine/research/corpus-manifest.json"))
lines = sorted(f"{root}/{f['path']}:{f['sha256']}"
 for root, r in m["roots"].items() for f in r["files"])
digest = "sha256:" + hashlib.sha256("\n".join(lines).encode()).hexdigest()
print(f"lines={len(lines)} blob_bytes={len('\n'.join(lines).encode())}")
print(f"computed={digest}\ndeclared={m['corpus_id']}")
sys.exit(0 if digest == m["corpus_id"] else 1)
```

**Expected output** `[measured, 2026-08-01]` :

```
lines=1084 blob_bytes=131186
computed=sha256:3a010b1649899795d79274fc528dbece97fdabf4ff0f81cc02ab619c048c51a4
declared=sha256:3a010b1649899795d79274fc528dbece97fdabf4ff0f81cc02ab619c048c51a4
```

The first and last lines of the sorted blob, for anyone debugging a near-miss:

```
first frontmatter/AGENTS.md:bfc925c20452436a57cc6139191930365967bf7ab86511e2ac3dc70de1f30962
last md/trade.md:8cb2304d8a0c3e9c287c54d4e54badfa0fedd10a1bf3e9a24d7bae75adda63e0
```

The manifest's own totals also reproduce from its per-root lists: summing every entry's `bytes` gives exactly `25548765`, and counting entries gives exactly `1084`.

`[measured]`

**The four common failure modes**, each of which produced one of the published null results: using a tab instead of a colon; appending a trailing newline; sorting within each root and concatenating rather than sorting the union; omitting the root prefix so paths collide across roots.

### 13.1.5 Verifying the corpus has not drifted

Separate from re-deriving the id, you should periodically confirm the *files on disk still match* the pin. Three agents ran this check independently and all three got zero drift; I ran it a fourth time while writing this. `[measured, 2026-08-01]`

```
#!/usr/bin/env python3
"""Re-resolve every manifest path against the live working trees."""
import hashlib, json, os
ROOTS = {"md": os.path.expanduser("~/Desktop/GitHub/md"),
 "knowledge": os.path.expanduser("~/Desktop/GitHub/knowledge"),
 "frontmatter": os.path.expanduser("~/Desktop/GitHub/frontmatter")}
m = json.load(open("docs/engine/research/corpus-manifest.json"))
ok = missing = mismatch = 0
for root, r in m["roots"].items():
 for f in r["files"]:
 p = os.path.join(ROOTS[root], f["path"])
 if not os.path.exists(p):
 missing += 1; continue
 if hashlib.sha256(open(p, "rb").read()).hexdigest() == f["sha256"]: ok += 1
 else: mismatch += 1
print(f"total={ok+missing+mismatch} ok={ok} missing={missing} mismatch={mismatch}")
```

```
total=1084 ok=1084 missing=0 mismatch=0
```

Independent confirmations on the record, quoted: kill-audit B — *"all 1,084 files resolved, 0 missing, 0 sha256 mismatches, so the vaults have not drifted since pinning"*; kill-audit C — *"0 missing, 0 sha256 mismatches, 25,548,765 bytes total. The corpus is fully reproducible today and is the one asset in this program I would trust without re-derivation"*; a verifier — `"files_seen 1084 missing 0 bytes 25548765 hash_ok 1084 hash_bad 0"`. `[primary]`

### 13.1.6 What the corpus is NOT — the biases, stated

The corpus is the strongest asset in this programme and it is still a narrow, self-authored sample. Every one of the following was raised by a research area against its own findings.

BIAS	EVIDENCE	WHO RAISED IT
<b>One operator.</b> All 1,084 files were authored by two people and an AI.	82.46% of bytes are one personal vault.	<code>product-gap</code> <code>unverified</code> §2; <code>hold-more</code>
<b>Machine-generated bulk is included and unlabelled.</b>	The container thesis' empirical case — <i>"he already hand-builds bundles across 37.2% of files"</i> — was <b>92.3% one machine-generated sync tree</b> : 340 of 340 <code>source: cclaude</code> files inside <code>md/Skills/**</code> . Honest figure ~2.2%.	<code>founder-thesis</code> <code>verifier</code> ; recorded in PLAN v2.0.0 §9 item 3
<b>One workflow's signature.</b>	<i>"The 0-definitions result, the 13,028 wikiilinks, the 73.9% bash fence share and the 3.4% real-HTML share are all plausibly signatures of one Obsidian-based workflow."</i>	<code>hold-more</code> <code>unverified</code>
<b>Append-dominated git history.</b>	In <code>md</code> , 4,948 of 5,715 <code>.md</code> file-revision pairs (86.6%) delete zero lines. An anchoring statistic re-derived on this corpus inherits that shape.	<code>premortem</code>
<code>docs/mdmax/PLAN.md</code> <b>is absent</b> (created after the pin).	See §13.1.3.	kill-audit A

**The falsification test for the corpus itself.** `docs/engine/PLAN.md` §11 W1 proposes a second corpus ( `~/cclaude/skills-src`, 474 `*.md` files / 5,013,124 bytes, path-list sha256 `72b867b02163ece7346e50cbae5cc71a` ) precisely to break the single-author bias. `market-and-gift` used it and then flagged the obvious problem against its own case: *"it is still a corpus on this operator's machine, and W1's real requirement (a corpus somebody else wrote) is still unmet."* `[primary]` **A corpus authored by a stranger is the single highest-value missing artifact in the evidence base**, and it is the gate on publishing the anchor number (§13.4, row A1).

### 13.2 Every research artifact

#### 13.2.1 Inventory

All paths are relative to the repository root `/Users/sagnikmitra/Desktop/GitHub/frontmatter`. All are tracked in git as of commit `1bd4dad`. `[measured]`

PATH	BYTES	WHAT IT IS	HOW TO READ IT
<code>docs/engine/research/corpus-manifest.json</code>	198,580	The pinned corpus. §13.1.	<code>json.load</code>
<code>docs/engine/research/wf-final-gate-2026-08-01.result.json</code>	1,364,043	<b>Run 3, the final gate.</b> 16 areas, adversarial verification, a kill audit, 4 synthesis lenses. The governing research artifact.	<code>json.load</code> ; see §13.2.2
<code>docs/engine/research/wf-final-gate-2026-08-01.raw.json</code>	1,436,963	The workflow envelope for run 3: <code>summary</code> , <code>agentCount</code> , <code>logs</code> , <code>result</code> , <code>workflowProgress</code> , <code>totalTokens</code> , <code>totalToolCalls</code> .	<code>json.load</code>
<code>docs/engine/research/wf-final-gate-2026-08-01.journal.jsonl</code>	1,333,281	80 newline-delimited records for run 3 — 40 <code>started</code> , 40 <code>result</code> , keyed by <code>agentId</code> .	one <code>json.loads</code> per line
<code>docs/engine/research/wf-mdmax-capability-2026-08-01.result.json</code>	1,420,260	<b>Run 2, the capability sweep.</b> 18 areas, adversarial verification, 3 synthesis lenses. No kill audit.	<code>json.load</code>
<code>docs/engine/research/wf-mdmax-capability-2026-08-01.raw.json</code>	1,487,390	Envelope for run 2.	<code>json.load</code>
<code>docs/engine/research/wf-mdmax-capability-2026-08-01.journal.jsonl</code>	1,382,194	78 records for run 2 — 39 <code>started</code> , 39 <code>result</code> .	one <code>json.loads</code> per line
<code>docs/engine/research/wf-findings-2026-08-01.md</code>	128,449	<b>Run 1, the foundations run.</b> 6 areas, markdown prose, per-finding tiered evidence. <b>No verification layer, no kill audit.</b>	read; sections start <code>## AREA:</code>

**Run statistics**, read from the `raw.json` envelopes `[measured]` :

RUN	AGENTS	TOPOLOGY	TOTAL TOKENS	TOOL CALLS
Run 2 — capability	39	18 research + 18 verify + 3 synthesize	7,864,147	2,159
Run 3 — final gate	40	16 research + 16 verify + 4 audit + 4 synthesize	7,858,804	2,173

Run 1's agent count is not recorded in a machine-readable envelope; it is described in the handoff record as a 25-agent run over 6 areas. Treat "25 agents" as `[secondary]`.

#### 13.2.2 How to parse the workflow result JSONs

Both result files share a shape. Do **not** grep them — the payloads contain escaped newlines, Unicode em-dashes and nested quotes, and a naive grep will match across record boundaries.

```
import json
d = json.load(open("docs/engine/research/wf-final-gate-2026-08-01.result.json"))

d["areas_total"] # 16 (run 2: 18)
d["areas_completed"] # 16 (run 2: 18)
d["headline_verdicts"] # run 3 ONLY: [{area, verdict}, ...]
d["kill_audit"] # run 3 ONLY: 4 auditor reports
d["per_area"] # 16 (run 2: 18) area records
d["synthesis"] # 4 (run 2: 3) cross-cutting lenses
```

`per_area[i]` **fields**. Run 3 carries `area`, `title`, `headline`, `falsifier`, `findings`, `negatives`, `unclaimed`, `design`, `measurement`, `kill_condition`, `effort`, `unverified`, `verification`. Run 2 carries the same minus `falsifier` and `negatives` — **the two fields that were added specifically to fix run 2's defect rate**.

`per_area[i].verification` **fields**. Run 3: `intent`, `area`, `headline_verdict`, `verdicts`, `survived`, `killed`, `new_defects`. Run 2: the same **minus** `headline_verdict` — which is why run 2's per-area verdicts must be recovered from the `verdicts` array by finding the entry whose `claim` string begins with `HEADLINE`.

```
recover run-2 headline verdicts
d2 = json.load(open("docs/engine/research/wf-mdmax-capability-2026-08-01.result.json"))
for a in d2["per_area"]:
 hits = [v for v in a["verification"]["verdicts"] if v["claim"].upper().startswith("HEADLINE")]
 print(a["area"], hits[0]["verdict"] if hits else "n/a")
```

Each entry in `verdicts` is `{claim, verdict, reason}`, sometimes with a `correction`. The `verdict` vocabulary is `CONFIRMED · OVERSTATED · REFUTED · UNSUPPORTED`.

**THE SINGLE MOST IMPORTANT RULE FOR USING THESE FILES.** `per_area[i].headline` is what the researcher *claimed*. `per_area[i].verification` is what survived. **Read the verification. Never quote a headline.** Run 2's headlines had a measured 100% defect rate, always in the flattering direction. Run 3's headlines were corrected in 11 of 16 areas.

A structural wrinkle worth knowing: the string a verifier labels `HEADLINE:` inside `verdicts` is not always the same sentence as the area's `headline` field. `model-view` is the clearest case — its `headline` is about invisible characters and a +14.991% token surcharge, while its `HEADLINE:` -labelled verdict is about this repo's `AGENTS.md` never being loaded. When the two diverge, the `verdicts` entry is what was actually checked.

13.2.3 Run 3 — the final gate: full headline verdicts

[primary] The complete `headline_verdicts` array, quoted exactly, in file order.

#	AREA KEY	TITLE	VERDICT	CLAIM-LEVEL BREAKDOWN
1	the-era	Is there actually a new era of markdown, or is that a story we are telling ourselves	OVERSTATED	14 claims: 7 CONFIRMED, 3 OVERSTATED, 4 REFUTED
2	founder-thesis	Reconstruct and steelman the founder thesis from the record, then find its blind spot	REFUTED	10 claims: 4 CONFIRMED, 3 OVERSTATED, 3 REFUTED
3	container-thesis	The mother markdown: one file containing many, with breakpoints, split on read	OVERSTATED	18 claims: 10 CONFIRMED, 6 OVERSTATED, 1 REFUTED, 1 UNSUPPORTED
4	markdown-base	Markdown as a codebase — what a compiler over a FOLDER enables that per-file tools cannot	OVERSTATED	11 claims: 7 CONFIRMED, 2 OVERSTATED, 1 REFUTED, 1 UNSUPPORTED
5	custom-pointers	Custom reference pointers, custom definitions, and the library-import model	CONFIRMED	12 claims: 8 CONFIRMED, 3 OVERSTATED, 1 REFUTED
6	hold-more	How to make a markdown file hold more, convey more, and mean more	CONFIRMED	14 claims: 11 CONFIRMED, 1 OVERSTATED, 2 REFUTED
7	rendering-frontier	Rendering — what frontmatter can render that nobody does, and what it must refuse	CONFIRMED	12 claims: 11 CONFIRMED, 1 OVERSTATED
8	parsing-robustness	Technical foolproofing — what makes a markdown compiler unbreakable	OVERSTATED	11 claims: 4 CONFIRMED, 4 OVERSTATED, 2 REFUTED, 1 UNSUPPORTED
9	product-gap	What frontmatter currently misses, against every product it must beat	CONFIRMED	13 claims: 10 CONFIRMED, 3 OVERSTATED
10	moat	First-mover advantage — what is actually defensible and what gets cloned in a weekend	OVERSTATED	14 claims: 5 CONFIRMED, 4 OVERSTATED, 2 REFUTED, 3 UNSUPPORTED
11	market-and-gift	Marketability, and the 50% that is given away	OVERSTATED	16 claims: 10 CONFIRMED, 2 OVERSTATED, 3 REFUTED, 1 UNSUPPORTED
12	aios-transfer	What 35 days of AIOS actually transfers to building MDMAX	OVERSTATED	10 claims: 4 CONFIRMED, 3 OVERSTATED, 3 REFUTED
13	agentic-docs	The agent-native document — what changes when the primary author is a machine	OVERSTATED	12 claims: 6 CONFIRMED, 5 OVERSTATED, 1 REFUTED
14	competitive-live	Live competitive sweep — what shipped in the last 120 days	CONFIRMED	14 claims: 10 CONFIRMED, 3 OVERSTATED, 1 UNSUPPORTED
15	against	The strongest case that MDMAX should not be built	OVERSTATED	11 claims: 3 CONFIRMED, 6 OVERSTATED, 2 REFUTED
16	premortem	Pre-mortem — it is 2027 and this failed. What happened?	OVERSTATED	14 claims: 7 CONFIRMED, 2 OVERSTATED, 4 REFUTED, 1 UNSUPPORTED

**Totals.** `CONFIRMED 5 · OVERSTATED 10 · REFUTED 1`. Across all 16 areas, 206 individual claims were adjudicated: **117 CONFIRMED, 45 OVERSTATED, 25 REFUTED, 9 UNSUPPORTED.** [measured]

The four synthesis lenses in run 3, quoted from `synthesis[i].lens`:

LENS	SCOPE
era-and-vision	Is there a new era, what is MDMAX the answer to, do the container and markdown-as-codebase theses survive the founder's own corpus
product-market	What ships, in what order, to whom, at what price, what is given away
technical	Engine specification: passes, offset model, parse-robustness regime, carriers, container contract, module system, incremental architecture
premortem-integrator	Integrating the opposition case and the pre-mortem against all 16 areas, for two founders who start building immediately

Each lens carries `thesis`, `ranked`, `dependencies`, `contradictions`, `what_to_cut` and (run 3 only) `one_sentence`. The `contradictions` arrays are where cross-area disagreements were adjudicated; §13.6 lifts the ones that still matter.

13.2.4 Run 2 — the capability sweep: verdicts and the 100% defect rate

Run 2 has **no `headline_verdicts` array and no kill audit.** Verdicts must be recovered per area.

[measured] Recovered from `per_area[i].verification`:

#	AREA KEY	TITLE	HEADLINE VERDICT	CLAIMS	SURVIVED	KILLED	NEW DEFECTS
1	edit-format	Own the LLM edit format and the region primitive	REFUTED	15	11	10	8
2	cache-layout	Cache-aware document layout as a compiler pass	REFUTED	12	12	13	11
3	ai-dialect	Recognize and lift the AI markdown dialect	not separately labelled	11	9	15	13
4	token-cost-tier	Token cost as a first-class diagnostic tier	not separately labelled	8	8	9	8
5	retrieval-shape	Retrieval-shape diagnostics — which parts of a document are invisible	not separately labelled	12	8	9	12
6	degradation-certificate	The degradation certificate — a per-document conformance matrix	not separately labelled	8	12	10	12
7	multi-format-ingest	Multi-format ingestion and representation choice, judged by downstream accuracy	REFUTED	11	10	10	11
8	provenance-boundary	Provenance that survives the markdown boundary	not separately labelled	14	13	12	12
9	collab-primitives	Google Docs of markdown — collaboration primitives on plain .md	REFUTED	10	9	12	14
10	impact-preview	Impact preview and semantic diff — the compiler-only editor features	not separately labelled	10	8	11	12
11	model-view	Show me what the model sees — a debugger for context	CONFIRMED (of a sub-claim)	20	11	13	8
12	integrity	Invisible-content integrity — what a file says vs what it looks like it says	REFUTED	10	14	7	10
13	normalization-i18n	Normalization and non-Latin capability as a differentiator	not separately labelled	10	9	8	9
14	unit-of-exchange	The unit of exchange — message-level MDMAX and the human-paste hop	not separately labelled	8	8	10	12
15	missing-constructs	The constructs markdown is missing, derived from strain rather than from demand	not separately labelled	13	12	15	8
16	adjacent-transplant	Adjacent-field transplant — capabilities documents have never had	not separately labelled	8	8	8	7
17	budget-knapsack	What to inline first — the context budget as a measured problem	REFUTED	11	10	12	12
18	governance-and-moat	How MDMAX actually gets adopted, and what makes the moat durable	UNSUPPORTED	12	7	9	9

The claim this run is famous for, and its honest form. Run 2's own critic synthesis states: "18 of 18 area headlines were materially refuted by their own adversarial verifier. Not qualified — refuted. The sentence a reader would remember from every area was wrong, and it was wrong in the flattering direction every time." It then lists the corrections: 0.155%→0.0204% , 51,620~17,065 , 12/12~11/12 , 30x~3.44x , 8,603:1~1,067:1 , "zero implementations"→two shipping packages found in one query, "the surface does not exist"→15.1x larger once the excluded construct is counted. [primary]

That paragraph is substantively right and literally unreproducible. Only 8 of 18 areas carry a verdict entry explicitly labelled HEADLINE ; of those, 6 are REFUTED, 1 is UNSUPPORTED, and 1 ( model-view ) is CONFIRMED — but the CONFIRMED one is a different sentence from the area's headline field. **Governing statement for this plan:** every area's headline was materially corrected by its own verifier, always in the flattering direction; six were explicitly REFUTED and one UNSUPPORTED. Do not write "18 of 18 refuted" without that footnote.

Run 2's three synthesis lenses: **google-docs** (which collaboration capabilities, in what order, make a plain- .md editor competitive), **ai-native** (a markdown representation easier for AI to consume and produce, with zero adoption required on the receiving side), and **critic** (what all 18 areas missed, and the case that the programme is over-researched and under-built).

13.2.5 Run 1 — the foundations run

docs/engine/research/wf-findings-2026-08-01.md , 128,449 bytes, six areas, markdown prose:

- 1. Markdown editor feature frontier — what does not exist anywhere, and what requires a compiler
- 2. Document-format package ecosystems: precedent, registry design, absence semantics, security surface, vendor hosting
- 3. AI-to-AI document/context exchange: protocol payload formats, handoff loss, whether markdown is the right carrier
- 4. Prior art, failure modes, and design for a single root markdown file that transitively references a large corpus
- 5. Adversarial audit of the MDMAX / frontmatter-engine research programme
- 6. How formats/languages actually evolve, and whether CommonMark can be changed — governance mechanics

Each area is ## AREA: <title> , then \*\*HEADLINE:\*\* , then \*\*FINDINGS:\*\* as a bullet list where every bullet carries a tier tag ( [primary-read] , [own-measurement] , [search-only] ) and an EVIDENCE: line with URLs, IDs and verbatim quotations.

**Warning label.** Run 1 has no verification layer and no kill audit. Its findings sit at exactly the tier that run 2's headlines occupied before verification — the tier measured at a 100% one-directional defect rate. Its per-finding evidence is unusually rich (forum thread IDs, like counts, verbatim user quotes, plugin download JSON) and is the best source in the corpus for demand evidence. But no number in §13.4's measurement index is drawn from run 1, and none should be until it gets an adversarial pass.

13.3 The kill audit — how much to trust the verifiers

Run 3 added a layer nobody else in this programme had: **an audit of the verifiers themselves**. Four independent auditors were each handed a set of "kills" (a verifier's claim that a researcher's finding is wrong) and asked to re-derive each one from primary sources.

[measured] Quoted exactly from kill\_audit[i] :

AUDITOR	HANDED	AUDITED	UPHELD	OVERTURNED	PUBLISHED FALSE-KILL RATE
A	29	29	24	5	5/29 = 17.24%
B	29	29	28	1	1/29 = 3.4%
C	29	21	17	4	4/21 = 19.0% (19.048%)
D	28	26	26	0	0/26 = 0.0%
pooled	115	105	95	10	10/105 = 9.524%

How to read these numbers.

- **The pooled false-kill rate is 9.524%, i.e. the verification layer is ~90.5% reliable at the verdict level.** docs/mdmax/PLAN.md v2.0.0 §0 says "roughly 88% reliable"; the unweighted mean of the four published rates is 90.07% and the pooled figure is 90.476%. **Neither arithmetic yields 88%**, so treat 88% as a conservative round-down of unknown provenance and quote 90% with the derivation, or quote the four rates individually.
- **Ten of the 115 kills handed out were never audited** and are excluded from the denominator rather than guessed. Auditor C left 8 unaudited (moat 's three moat claims, market-and-gift 's two, aios-transfer 's three) because they turn on original claim text or a transcript-mining selection rule that is not on disk. Auditor D left 2 (aios-transfer :: Claim 10, which needs a 1,944-file scratch artifact that does not exist in the repo; and competitive-live :: DESIGN item 2 strip\_references, which had no toggle to replicate against).
- **Auditor A's rate is a floor, not a ceiling.** It states: "One kill — the-era Claim 6, 'adoption is already FLATTENING' — rests on a sample definition... that is not recoverable from anything on disk... If the true verdict there is OVERTURNED, the false-kill rate is 6/29 = 20.7%. The published 17.24% is therefore a floor-to-central estimate, not a ceiling." Under that worst case the pooled rate is 11/105 = 10.476%, i.e. 89.5% reliable.

The one rule that would have caught every false kill. Auditor A found the pattern and stated it as an operating procedure. Adopt it verbatim:

"Kills that QUOTE A PRIMARY SOURCE (a line of code, an API field, a star count, an issue state, a passage in a plan doc): **16 of 16 correct. Not one error.** ... Kills that RE-DERIVE A NUMBER and substitute their own: **5 of 13 wrong**, i.e. a 38% defect rate in the half of the work that requires the verifier to do its own measurement. All five failures are the same shape — the verifier picked a different operational definition from the researcher's, did not disclose the switch, and scored the mismatch as the researcher's defect. RECOMMENDATION: **accept a kill that cites a source; independently re-derive any kill that substitutes a number.** That single rule would have caught all five false kills and cost nothing on the other 24."

**The second-order finding: verdicts are far more reliable than the numbers inside them.** Auditor B: "verdict reliability is ~97% while number reliability is ~83%. Do not paste a verifier's replacement figure into the PLAN without re-deriving it." Auditor D reached the same place from zero overturns: "Trust the verdicts: 26/26. Do not copy individual figures out of a kill without re-deriving them: 5 of 26 kills carry a supporting number that is wrong, unreproducible, or internally inconsistent."

**The failure signature has a direction, and it is the mirror image of the researchers'.** Auditor C: "All four overturns are the same species: a verifier constructing a denominator or a definition, finding it does not match, and reporting a defect instead of reporting that it could not reproduce the researcher's definition... Note the asymmetry: the researchers' defects ran flattering; the verifiers' defects run damning. Both are reward functions."

A structural gap in the verification layer, found by auditor A and still unfixed:

"TWO VERIFIERS DISAGREED WITH EACH OTHER ON THE SAME MEASUREMENT, AND NOBODY NOTICED. The container-thesis verifier used the LINE-INITIAL backtick rule to compute the spec's fence width (correct...). The founder-thesis verifier used ANY-POSITION on the same corpus and killed a claim with it (incorrect). Same 1,084 files, same afternoon, opposite definitions, no reconciliation. Whatever ran these verifiers has no cross-area consistency check. That is a structural gap, not a one-off."

**The governance consequence, and it is now rule P4 in this plan.** Auditor C: "nothing should be DELETED on a kill alone. On this sample, executing all 21 kills unaudited would have destroyed four correct findings, including a corpus denominator that reproduces to within 0.003%."

**Where the kills were right, they were devastating,** and those must be actioned rather than softened. Auditor A's list: 91% of founder-thesis C2's headline is one machine sync tree; only 86 of 393 intra-document anchors resolve under the canonical slugger against a claimed 348; all 4 unclosed fences are one generator's output in one directory.

**The auditors audited themselves too,** which is the reason to trust them at all. Auditor A: "I nearly produced a false overturn myself: my first, uncontrolled test of the 47x diff claim varied manifest layout while an offset-bearing manifest masked the effect, showing a 0.7% swing and pointing at OVERTURN. A properly controlled second experiment with an offset-free manifest showed a 291x swing (1,077 B vs 313,870 B) and upheld the kill."

13.4 The measurement index

Every load-bearing number in this plan, with its source, its tier, and — the column that matters — whether anyone other than its author has reproduced it.

**Legend for "reproduced":** YES = re-derived by a second, independent party. PARTIAL = reproduced by the same run's verifier or on a different selection. NO = single source, never re-derived. N/A = external point-in-time read, not reproducible by construction.

A — The anchor (the flagship claim)

ID	VALUE, QUOTED EXACTLY	WHAT IT MEASURES	SOURCE	TIER	REPRODUCED
A1	<b>99.627% correct / 0.050% false / 0.323% safe refusal</b>	content-derived re-anchoring, anchorable blocks only	docs/engine/PLAN.md:116	[measured]	<b>NO</b>
A2	41,642 block-versions, 294 consecutive real revision pairs, denominator 32,919 surviving block-versions, 384-configuration parameter sweep	A1's population	docs/engine/PLAN.md:80 , \$1.1a	[measured]	<b>NO</b> — no selection script exists in the repo
A3	byte offset 36.93% / <b>62.12% false</b> / 0.95%; block index 44.43% / 55.50%; content hash alone 83.36% / 0.00% / 16.64%; hash + nearest-position 77.80% / <b>22.20%</b> ; rigid k=1/2/3 90.65 / 88.68 / 87.22%	the baselines A1 beats	docs/engine/PLAN.md table \$1.1a	[measured]	<b>NO</b>
A4	~41 ms for a 446 KB / 2,225-block document	resolve cost	docs/engine/PLAN.md \$1.1a	[measured]	<b>NO</b>
A5	14.2% of blocks are identity-free; <b>88.9% of raw false matches are horizontal rules</b> ; the anchorable() gate moves false from 0.349% → 0.050%	the gate	docs/engine/PLAN.md \$1.1a	[measured]	<b>NO</b>
A6	LCS 30-line diff baseline <b>86.658%</b> (77.297% / 9.361% split, 394 pairs); verifier's independent run <b>85.036%</b> (76.346% / 8.690%, 494 pairs, 31,149 anchorable block-versions); hash-alone-unique 76.352%	the adversarial baseline that nearly matches A1	against + its verifier	[measured]	<b>PARTIAL</b> — two runs, two selections, same result
A7	independent S1 replication <b>87.03%</b> (77.304% unique + 9.727% duplicate) against the plan's "S1 exact hash resolves 87.0%"	closest thing to corroboration of any part of A1	against unverified \$4	[measured, different selection]	<b>PARTIAL</b>
A8	the range case gives <b>3.44x</b> , not 30x, with refusal roughly doubling	whether A1 transfers to comment anchors	docs/mdmax/PLAN.md v2.0.0 \$5.2	[measured, one replication]	<b>NO</b>
A9	~1 heading in 4 cannot carry an anchor — <b>18.3% / 25.4% / 24.7%</b> under 3 tokens	headings are what everything points at	docs/mdmax/PLAN.md v2.0.0 \$5.2	[measured]	<b>NO</b>
A10	Hypothes.is: 20,953 annotations, <b>~22% no longer attachable</b>	the only production datum on long-lived anchors	against , arXiv abstract only	[secondary]	<b>NO</b>

The 99.627% warning, in full

Nobody has re-derived this number. Not one person, not one agent, not one area. The following areas quoted it and each stated in writing that they did not re-derive it: founder-thesis ("I did not re-derive the 99.627% anchor figure"), markdown-base ("[not re-derived]"), custom-pointers ("INHERITED from the plan and NOT re-derived here"), product-gap ("I did NOT re-derive the 99.627% re-anchoring figure"), competitive-live ("I did NOT re-derive them... Tier: secondary"), against ("THE 99.627% IS STILL UNVERIFIED BY ME... I verified only that no derivation script exists anywhere in the repository"), premortem ("Nobody has since it was first produced... SECONDARY at best").

Four independent things are wrong with quoting it today:

1. **No derivation script exists.** Verified by search, twice, independently.
  2. **No selection script exists** for the 294 revision pairs, so nobody can check whether the sample inherits the corpus's 86.6% append-dominance. premortem : "If the pairs were hand-picked for substantive edits, finding 5 weakens sharply."
  3. **A 30-line adversarial baseline reaches 86.658%** on the same class of problem (A6). The marginal value of the whole mechanism over a diff is ~13 points, not ~22 as against hash-alone.
  4. **It is a BLOCK figure and comments anchor RANGES** (A8).
- DECIDED:** do not publish it. **RECOMMENDED:** re-derive it under kill gate K3, on a corpus this team did not write, with diff3-assisted as an arm, and with the derivation script committed. **OPEN:** whether the tuned 384-configuration parameter set beats dom-anchor-text-quote 's 182-line defaults — moat calls this "the single most important open question" and nobody has run it.

B — The corpus and the writer

ID	VALUE	SOURCE	TIER	REPRODUCED
B1	1,084 files / 25,548,765 bytes / 3 roots	corpus-manifest.json	[measured]	YES — 4 independent re-resolutions, 0 drift
B2	corpus_id sha256:3a010b16...	\$13.1.4	[measured]	YES — reproduced 3x; 4 published failures, all serialization errors
B3	107,287 top-level blocks spliced byte-identically across 1,080 files, 0 failures, using stock mdast position.offset	run 3 technical synthesis	[measured]	NO — single run, but the strongest positive in the corpus
B4	0 of 120 files byte-identical through mdast-util-from-markdown → mdast-util-to-markdown (stratified sample, all 3 roots, seed 20260801, GFM) — confirms D7	container-thesis verifier	[measured]	PARTIAL — corroborates docs/mdmax/PLAN.md §4.2's "0 of 51"
B5	907 files carry YAML frontmatter; 171 throw under the editor's writer; 33 survive byte-identical; 624 remainder. Stable across 4 script revisions	product-gap	[measured]	YES — capability run reproduced 907/119/13.12%/618/170/18.74% "to the last digit"
B6	the yaml library round-trips 119 of 907 byte-identically; 170 files do not parse at all (18.74%)	docs/mdmax/PLAN.md v2.0.0 §5.1	[measured]	YES
B7	a no-op round trip through parseFrontmatter → stringifyFrontmatterDoc changes 623 of 737 files with an editable frontmatter map (84.5%), total  byte delta  46,732; 421 respaced / 197 reflowed / 5 separator-only; moves line numbers in 26.7%	product-gap C2, verdict CONFIRMED	[measured]	PARTIAL
B8	only 67 of 1,080 files have bytes == UTF-16 units == code points; 93.8% already diverge	docs/mdmax/PLAN.md v2.0.0 §5.5	[measured]	NO
B9	AST histogram over all 1,084 files (remark-parse 11.0.0 + remark-gfm 4.0.1 + remark-frontmatter 5.0.0), 21 distinct node types:   text=334081 paragraph=142802 listItem=88434 inlineCode=76118 strong=44893 heading=28221 tableCell=25813 list=15160 emphasis=12741 tableRow=9374 code=7738 html=7720 link=5453 thematicBreak=2754 blockquote=2742 table=1150 root=1084 yaml=907 break=613 delete=444 image=16 ; definition absent	hold-more	[measured]	PARTIAL — 3 independent cross-checks matched exactly (link=1838 CommonMark-only, image=16, yaml=907)
B10	0 link reference definitions in 25,548,765 bytes — so the linkref carrier has zero collision risk	markdown-base	[measured]	YES — falls out of B9's absent definition node
B11	288 files (26.57%) lack a trailing newline; 7 (0.65%) have unterminated fences; 4 are zero bytes; 1 uses CRLF; 598 (55.2%) paths contain a space; 18 path characters are em-dashes; max path length 134	container-thesis C7 + verifier	[measured]	YES — 4 of 6 reproduced exactly by the verifier
B12	line-initial backtick runs: 15,435, all exactly 3 → a 4-backtick fence is provably sufficient. Any-position histogram: 1x157,494 / 2x42 / 3x15,506 / 4x4 (all four inline, in docs/engine/PLAN.md lines 652 and 1042)	two verifiers	[measured]	YES — and this definitional split caused one false kill; see §13.6

C — Links, the graph, and provenance

ID	VALUE	SOURCE	TIER	REPRODUCED
C1	[text](path.md) resolves 546/637 = 85.71%; [[Name]] resolves 2,754/9,962 = 27.65% — a 58-point gap on identical files	docs/mdmax/PLAN.md v2.0.0 §3.2 / §5.3	[measured]	YES — "I replicated it a third time" (run 3 synthesis)
C2	median out-degree ZERO, mean 2.27, p90 5; 597 files (55.1%) zero outbound resolved edges; 502 (46.3%) fully isolated; 511 connected components, largest holds 40.1%	docs/mdmax/PLAN.md v2.0.0 §3.2	[measured]	YES — four independent agents plus the synthesizer agree
C3	transclusion ![[]] occurs 45 times in 25.5 MB — all 45 are syntax documentation; genuine content transclusion is ZERO	ibid.	[measured]	YES
C4	only 86 of 393 intra-document anchors resolve under the canonical slugger, against a claimed 348	kill-audit A (upheld kill)	[measured]	PARTIAL
C5	anchor health swings 77 percentage points on the choice of heading slugger	agentic-docs	[measured], own re-implementation	NO — "must be re-run against the real github-slugger"
C6	240 of 365 provenance original: keys fail directory-relative resolution (225 excluding 15 template placeholders); 365 of 365 fail root-relative. The widely-quoted "273 of 365" reproduces under nothing	kill-audit D	[measured]	YES — "If the team writes down 'provenance never resolves' they will be wrong; a third of it does"
C7	1,838 inline links against 142,802 paragraphs and 88,434 list items — so an inline-attribute carrier cannot reach most of a document	container-thesis	[measured]	PARTIAL
C8	330 HTML comments across 160 files; GitHub deletes them in both API modes	hold-more	[measured]	NO — independently re-derives D2 from the carrier side

D — The container

ID	VALUE	SOURCE	TIER	REPRODUCED
D1	container <b>25,548,765 B</b> vs a one-line-per-document index at <b>105,538 B → 242×</b>	run 3 synthesis	[measured]	<b>NO</b>
D2	a 200K-token window holds <b>2.82%</b> of the container and <b>682%</b> of the index; a 1M window holds <b>14.1%</b> ; mean file <b>23,569 B</b> , so a 200K pack carries about thirty average files	ibid.	[inference] — see the warning below	<b>NO</b>
D3	[llm-code-format] <b>387 downloads/month</b> against repomix's <b>327,543 → 846:1</b> . repomix <b>27,551 stars</b> . repomix issue #226 (unpack): <b>0 reactions after 19 months</b>	[founder-thesis] , [container-thesis]	[primary]	<b>N/A</b> — point-in-time
D4	Obsidian forum thread asking to export a vault to a single file: <b>9,010 views, 1 like, 3 posts in 26 months</b>	[docs/mdmax/PLAN.md v2.0.0 §3.1]	[primary]	<b>N/A</b>
D5	reference pack round trip: <b>1084/1084 byte-identical at +0.892% overhead</b> ; 4 files (0.369%) end inside an unclosed fence	[container-thesis verifier]	[measured]	<b>PARTIAL</b> — [docs/engine/PLAN.md §5 has a 6-file / +0.99% version]
D6	<i>"he already hand-builds bundles across 37.2% of files"</i> — <b>403 of 1,084 (37.2%), 12,193,061 of 25,548,765 bytes (47.7%)</b> with ≥2 real top-level H1 — <b>then killed: 92.3% is one machine sync tree (340 of 340 source: claude in md/Skills/** ); honest figure ~2.2%</b>	[founder-thesis] + verifier	[measured] , <b>refuted</b>	<b>YES</b>

**TIER CORRECTION, and it must be made in the plan body.** [docs/mdmax/PLAN.md v2.0.0 §3.1 tags D2 as [measured, corpus\_id sha256:3a010b16...]. The [container-thesis] area's own [unverified] field says the opposite, in capitals: *"TOKEN COUNTS ARE INFERENCE, NOT MEASUREMENT. No tokenizer was installed and I would not npm install one under the RULE 4 guard, so every token figure... rests on an assumed 3.6 bytes/token for English markdown. The conclusions are robust across 3.0-4.0 B/tok but the exact percentages are not measured."* **The area's own disclosure governs.** The byte ratio (D1) is measured; every percentage-of-a-context-window derived from it is [inference].

E — The era

ID	VALUE	SOURCE	TIER	REPRODUCED
E1	CommonMark released examples: <b>649 at 0.29 (2019-04-06) → 652 at 0.30 (2021-06-19) → 652 at 0.31.2 (2024-01-28)</b> ; master HEAD still reads 0.31.2 but carries <b>655</b> , i.e. +3 unreleased for 2.5 years. Spec commits/yr <b>48/15/14/15/13/11/7/6</b> from 2019 to 2026	[the-era] falsifier F3	[primary]	<b>YES</b> — verifier calls it <i>"the single most decisive fact in the area"</i>
E2	[cmark-gfm] last release <b>0.29.0.gfm.13, 2023-07-21</b> , still pinned to CommonMark 0.29 (2019)	ibid.	[primary]	<b>YES</b>
E3	markdown is the <b>#1 file type an agent reads (23.51%)</b> , ahead of [.tsx] and [ts] ; <b>#2 written</b>	[the-era]	[measured]	<b>YES</b> — <i>"replicated on an independent scan"</i>
E4	<b>94.1-94.3%</b> of a June-2026 corpus and <b>91-92.4%</b> of July-2026 files were created inside an agent session	ibid.	[measured]	<b>YES</b> — <i>"replicated to within one file of 991"</i>
E5	AGENTS.md <b>0% → 30.67%</b> of the 300 most-starred GitHub repos in 15 months (92/300); <b>32.09%</b> on the top 215 by star rank (69/215). The researcher's own figure was 57/215 = <b>26.51%</b> and the verifier called it <b>CONSERVATIVE</b> . Companions: CLAUDE.md 76 = 25.33%; [claude] 45 = 15.00%; README.md 278 = 92.67%. Earliest AGENTS.md creation month <b>2025-05</b>	[the-era] + verifier	[measured]	<b>YES</b> — the only correction in run 3 that ran in the flattering direction
E6	convention births mostly die: in 215 top-starred repos, [llms.txt] <b>0.47%</b> , [cursorsrules] <b>0.93%</b> , [AGENT.md] <b>0.00%</b> , [windsurfrules] <b>0.00%</b> , [GEMINI.md] <b>2.33%</b> . AGENTS.md's own add rate <b>halved</b> : +7/month (Feb-Mar 2026) → <b>+3.0/month</b> (Apr-Jul 2026)	ibid.	[measured]	<b>PARTIAL</b> — verifier got 0.33% / 0.67% on the larger sample
E7	react-markdown <b>11,786,121 → 103,114,261/mo</b> ; streamdown <b>0 → 14,477,148/mo</b> since 2025-08-14; remend <b>0 → 16,192,091/mo</b> since 2025-12-03. <b>But</b> streamdown@2.5.0 <b>pins</b> remend@1.3.0 <b>as a direct dependency</b> , so the combined number double-counts one install base	[the-era] + verifier	[primary]	<b>YES</b>
E8	generic markdown parsers grew only <b>1.44×</b> faster than the whole npm registry (median ×4.05 vs a 14-package control median ×2.81)	[the-era] falsifier F2	[measured]	<b>NO</b> — <b>JavaScript only</b> ; PyPI download trends were not measurable
E9	markdown's share of repository file trees <b>fell 12.68% → 10.41%</b> (2024-08 → 2026-06, 12 unnamed repos) <b>versus</b> kill-audit A's named 8-repo test showing it <b>rose 1.142% → 1.460%, up in 7 of 8</b>	[the-era] vs kill-audit A	[measured] , contradictory	<b>REFUTED IN BOTH DIRECTIONS</b> — do not state it

F — Product, market and demand

ID	VALUE	SOURCE	TIER	REPRODUCED
F1	review loop 169 : collaboration 95 = 1.78:1 in favour of commenting, reversing an earlier 6.9:1 the other way	docs/mdmax/PLAN.md v2.0.0 §4.1	[measured]	NO — own regexes over own selection ( frontmatter-raw-corpus.json + frontmatter-r2-raw-corpus.json , 550,131 bytes, 3,804 string leaves). Does not cite corpus_id .
F2	92 sourced pain points, 89 feature requests, a 20-app competitor matrix, every claim carrying a verbatim quote + URL	docs/FRONTMATTER-PRODUCT-PLAN.md	[primary]	N/A
F3	mdmax cert targets pains ranked 13th and 14th of 14	docs/mdmax/PLAN.md v2.0.0 §5.4	[measured]	PARTIAL
F4	Obsidian demand, verbatim thread stats: typed links #6994 820 likes / 213 posts / 49,514 views, open since 2020-07-03; rename propagation #25412 340 / 80 / 11,899; block embeds #27093 553 likes; block identity #674 845 / 197 / 58,338	run 1, area 1	[primary]	N/A — run 1 is unverified
F5	plugin installs: dataview 4,659,822; excalidraw 6,900,710; templater 5,043,982; strange-new-worlds 132,197; 6,138 plugins listed	run 1, official obsidianmd/obsidian-releases community-plugin-stats.json	[primary]	N/A — cumulative and age-biased; docs/engine/PLAN.md §16.1 declares this metric unreliable
F6	inkeep/OpenKnowledge shipped 2026-07-30, 3,239★, GPL-3.0, shipped content-derived anchoring and explicitly declined the teammate model	competitive-live	[primary]	N/A — beta published 02:38:33Z, six minutes before the researcher's first tool call
F7	OKF SPEC.md v0.2, 37,544 bytes, inside GoogleCloudPlatform/knowledge-catalog (8,134 stars). Its benchmark: +5.2% tokens, 22/26 vs 20/26, n=1 per cell	against , agentic-docs	[primary]	PARTIAL — the authors themselves say "not a result anyone should quote as a headline"
F8	markdownlint ran 11 years 4 months to 2.89M downloads/week and zero dollars	market-and-gift	[primary]	N/A
F9	pricing anchors: Obsidian Sync \$4/mo annual, Obsidian Commercial \$50/user/year, HackMD Prime \$5, Notion Business \$20	docs/research/frontmatter-competitor-gapmap.md , re-verified live	[primary]	N/A

G — Hazards, defects and repo state

ID	VALUE	SOURCE	TIER	REPRODUCED
G1	WIKILINK_RE at k=1.98 — 36,865 ms on 320 KB of [ ]	parsing-robustness	[measured]	NO
G2	mdast-util-from-markdown 12,429 ms vs micromark's 1,207 ms on identical bytes — a 10.3x gap that widens	ibid.	[measured]	NO
G3	micromark 39,611 ms on 2.56 MB of indented lists	ibid.	[measured]	NO
G4	JSON.stringify 12,076 ms freeze / RangeError: Invalid string length (Node v24.6.0)	ibid.	[measured, Node] , [inference, browser]	NO
G5	a marked blockquote depth boundary moves 1902 → >1907 the moment --stack-size changes	kill-audit B — the one overturned kill	[measured]	YES — and it is the textbook case of an equality-pinned assertion on an environment-dependent quantity
G6	20 commits, all by Sagnik Mitra <sagnikmitra123@gmail.com>	premortem , verified	[measured]	YES
G7	package.json was destroyed to 17 bytes by a research subagent; live today with 47 dependencies + 18 devDependencies	against unverified §10; live check	[measured, 2026-08-01]	YES
G8	"Fourteen untracked files, 4,965,476 bytes... the 16 commits that DO exist are on no remote"	premortem	[measured, then true]	STALE — see below
G9	the "55 payloads" figure cited in docs/engine/PLAN.md §15, docs/engine/README.md lines 664 and 787 and the dossier does not reproduce: the file has 327 lines, 15 it() blocks, arrays of 39 and 6 (=45); a broader sweep gives 71	market-and-gift unverified §7	[measured]	NO — live stale number in three published artifacts
G10	firebase.rules contains "comments" zero times; firebase() has zero callers; no signup/register/onboard route exists	docs/mdmax/PLAN.md v2.0.0 §6	[measured, verified live]	YES
G11	~/sgnk/bin/sgnk-regression-gate.sh:70 — grep -c . on an empty gates file prints 0 and exits 1, so    echo 0 also fires, yielding the two-line string 0\n0 , and the write-landed guard errors out inoperative	kill-audit C, "bonus defect outside the kill set"	[measured]	NO — a live bug in a tool the AIOS transfer plan depends on

G8 is now corrected by live state. [measured, 2026-08-01] Commit 1bd4dad tracked docs/mdmax/PLAN.md , corpus-manifest.json , all six workflow artifacts, wf-findings and all three HANDOFF files. git branch -r --contains HEAD returns origin/engine/plan-and-diagnostics — the branch is pushed. origin/main remains at 8eb4de2 . The residual untracked set today is 221,055 bytes of agent litter: .scratch-carrier.mjs , .scratch-lossy.mjs , arx.xml , cx.html , polyg.yml , docs/engine/build/\_pycache\_/. That is docs/mdmax/PLAN.md §10.6's open item, and it is all that is left of what the pre-mortem correctly called an existential-risk-by- git clean .

13.5 The unverified list — everything a reader might mistake for a measurement

This list is assembled from the unverified field of all 16 areas in run 3. Every item here was volunteered by the agent that produced the surrounding findings. That is the strongest signal in the corpus that the tier discipline was actually running.

Grouped by failure class rather than by area, because the classes are what repeat.

13.5.1 Not measured at all

- Zero live model calls were made, in any run, in any of the 34 areas. custom-pointers : "I made ZERO live model calls. Every claim about what a model does with a declared notation is a pre-registered prediction, labelled as one." agentic-docs : "Every claim about what helps or hurts an agent's comprehension remains

a PREDICTION." rendering-frontier : "No model was called. Every statement about what an AI consumer would prefer to see rendered is absent from this report on purpose." **Consequence: every "easier for AI", "fewer tokens for a model", "a model will understand this" sentence anywhere in this plan is a prediction.**

- **Zero users, zero interviews, zero willingness-to-pay probes.** premortem : "There are still zero users, zero interviews and zero willingness-to-pay probes in this entire program... Every sentence anyone writes about what users will feel about an orphaned comment, including mine, is INFERENCE."
- **Google Docs round-trip fidelity.** moat : "I did NOT run a round trip through Google Docs and measure byte loss... Do not put a number on Google Docs' markdown loss without running it."
- **The app was never run.** product-gap : "I did not run the app at all. Every claim about src/ is a source read." Specifically unverified by execution: that Publish 502s on the 171 unparseable files. premortem : "I did NOT run the test suite, the build, or the app."
- **Closed-source editors.** rendering-frontier : "Typora, Bear, Craft and Notion-import are CLOSED SOURCE. I fetched nothing primary for them and I am reporting nothing about them... treat their row in any survey as EMPTY, not as 'renders the basics'." moat declined Roam and Typora on the same grounds.
- **Clipboard behaviour was never observed.** hold-more : "CLIPBOARD IS INFERENCE, NOT MEASUREMENT. No browser was driven."
- **Four of six comparison formats.** hold-more : DocBook, JATS, DITA and OOXML "WERE NOT READ"; only TEI and HTML5 were done.
- **arXiv and Product Hunt.** competitive-live : arXiv refused three times (429/429/503) → "I have ZERO arXiv coverage"; Product Hunt "NOT ATTEMPTED".
- **Hugging Face dataset cards, PyPI download trends.** the-era : both blocked by the sandbox allowlist. "the entire Python side of the parser story is missing."

### 13.5.2 Search-only (absence of evidence, not evidence of absence)

- **Every "nobody has built X" claim.** rendering-frontier : "Nobody has built X' in the unclaimed list rests on registry.npmjs.org keyword search, whose total field is the size of the registry rather than a match count — so it is search-only evidence of ABSENCE, the weakest kind." competitive-live : GitHub code search returned total\_count 0 even for queries known to match — "Treat every unclaimed item as 'not found by repo/registry search', not 'proven absent!'"
- mdmax cert 's niche emptiness is established on npm only (four searches, 32 results). PyPI, crates.io, Go modules, Homebrew and RubyGems were not searched. premortem .
- **GitHub code-search counts are severe undercounts.** market-and-gift : llms.txt 9,640, .editorconfig 82,848, commitlint.config.js 10,896 — "should be read as ordinal only." agentic-docs : filename:AGENTS.md total\_count 179,552 — "treat as an order of magnitude, never as a count."
- **The llms.txt headline.** the-era : "The llms.txt figure in the brief — '28% of 137,000 domains published one and 97% received ZERO requests' — is SEARCH-ONLY to me. I could not reach the source and did not re-derive it. Do not treat the 28%/97% pair as verified by this run."
- **"Nobody ships a document-tree lock file"** rests on five npm queries. markdown-base .

### 13.5.3 Inference presented at measurement's confidence

- **Container token arithmetic** (D2 above) — assumed 3.6 bytes/token, no tokenizer installed.
- **Stratified-symbol early-cutoff.** markdown-base : "[inference, not measured] The claim that stratifying symbols into four durability-graded keys raises the early-cutoff rate from 19.7% to ~93.4%... I did not build the stratified engine and measure it. It is arithmetic on a measurement, not the measurement."
- **The 37.6% machine-authored share.** founder-thesis : "an INFERENCE — a two-component mixture over one construct... the mixture is not a classifier and must never be reported as one."
- **Boundary-destruction scenarios.** container-thesis : "clean\_delete, merge\_across, truncate\_tail, elide\_unchanged and reorder are my constructions, not observed model outputs... their relative FREQUENCY in real model returns is unknown, and the frequency is what decides whether refuse-and-localise is a rare annoyance or a constant one."
- **Clone-time estimates.** moat : "nobody should quote 'a weekend' as if it were measured."
- **Certificate perishability.** premortem : "'The certificate is perishable' is INFERENCE from release cadence, not measurement of behavioural drift."
- **Mermaid CDN drift.** rendering-frontier : jsdelivr is not on the allowlist, so "the PDF path serves 11.16.0 today" is inference from npm semver resolution.

### 13.5.4 Synthetic, replayed, or upper-bounded — the [SIMULATED] family

- **The 500 MB scale test.** container-thesis : "THE 500 MB SCALE TEST IS SYNTHETIC. It is the pinned corpus replicated 20 times, so it has 20× the cross-file redundancy of a real 500 MB vault. Pack THROUGHPUT (270 MB/s) and boundary-scan time are unaffected by redundancy and stand; any compression ratio derived from it does not."
- **The 75.8% exact-recovery figure** "should be read as an upper bound on the uncorrelated case, not as a forecast" — the edit model is hand-written mutations with an arbitrary p(edit)=0.5.
- **The 12.22% near-miss figure** is an **upper bound**; "the true rate is likely far closer to the exact-recovery 0.28%." founder-thesis .
- **The 60.0% / 47.3–78.3% copy-drops-context figures** are an **upper bound** on "only in tool context" because exact substring containment over-counts. agentic-docs .
- **The "2 of 15 disagree" certificate figure** is a **lower bound** on disagreement — and the area kept it because "it happens to cut against my own argument." market-and-gift .

### 13.5.5 Single-implementation, single-run, or environment-bound

- markdown-base : "Every parse-behaviour number... comes from marked 16... I did not cross-check against cmark-gfm, micromark, comrak or commonmark.js... Treat every parse-derived figure as one-parser-measured." And: "one run of one script over one commit window with one definition of 'exported interface' that I chose. Per LR#63, one run is an anecdote."
- parsing-robustness : cmark-gfm and pandoc are not installed; markdown-it JS is not installed and its Python port was used as a proxy — "the timings are Python-vs-JS and NOT comparable as absolute numbers." resourceLimits behaviour is "measured on ONE configuration... mechanism unexplained." And one sub-experiment is self-disqualified: "my shingle hostile-case B was BADLY DESIGNED — I generated tokens with a modulo-97 vocabulary, so it produced only 97 distinct shingles and proved nothing."
- rendering-frontier : the alt-text decay measurement has n=21 markdown images and n=11 raw <img> — "The 100% figure is real for this corpus and it is a tiny sample. Do not publish it as a rate."
- aios-transfer : agent-behaviour numbers come from "a MOVING selection: 1,944 transcript files with no pinned manifest, measured once... it needs a corpus id before it is quoted."

- `hold-more`: "MY OWN HARNESS PRODUCED ONE FALSE RESULT I CAUGHT MID-RUN... I report this because a single-pass harness result in this report should be assumed to carry others of the same kind that I did not catch."
- `markdown-base`, **disclosed self-error**: "My first setext measurement reported 908 traps and was wrong by 904 because it counted YAML frontmatter closers... the only thing that caught it was noticing that 908 was suspiciously close to 907 files-with-frontmatter."

13.5.6 Point-in-time reads that are already stale

Every star count, download count, view count, price and publish date in this plan was read on **2026-08-01** and is moving. `container-thesis`: "STAR AND DOWNLOAD COUNTS ARE POINT-IN-TIME." `competitive-live`: "Star counts, download counts and dates are all point-in-time reads on 2026-08-01T02:44Z–03:40Z. The OpenKnowledge beta was published at 02:38:33Z, six minutes before my first tool call — this field is moving faster than the document that reports it."

13.5.7 Sandbox and rate-limit gaps that shaped the evidence

Worth recording because they are re-runnable, not intrinsic:

- `api.github.com` ran **unauthenticated at 60 requests/hour** in several areas because `/Users/sagnikmitra/.config/codex-env/tokens.zsh` is denied by the sandbox. Kill-audit A: "that capped my repo-tree test at 8 usable repos and forced me to drop `rust-lang/rust` ... and `facebook/react` ." **Making that token readable would materially widen several samples.**
- The network allowlist blocked `agents.md`, `llmstxt.org`, `huggingface.co`, `pypistats.org`, `cdn.jsdelivr.net`, `codeberg.org` and NVD.
- One benign false positive is on the record and should not be re-litigated: kill-audit A's injection-scan hook fired on "Forget all", which is the JSDoc line "Reset - Forget all previous slugs" inside the official `github-slugger@2.0.0` tarball. No action was taken.

13.6 Contradictions inside the evidence base, and which one governs

Every entry names both sources and states the ruling. These are the ones a reader will otherwise hit and have no way to resolve.

#	THE DISAGREEMENT	RULING
1	<b>corpus_id reproducibility.</b> <code>premortem</code> (64 variants, "MATCH: NONE"), <code>moat</code> ( <code>a55879ba..</code> ), <code>agentic-docs</code> , <code>custom-pointers</code> (30 variants) all failed. The <code>premortem</code> verifier and the <code>premortem-integrator</code> synthesis both reproduced it.	<b>IT REPRODUCES.</b> Three successful derivations including mine (§13.1.4). Four published nulls were serialization errors. <b>Commit the ten-line script.</b>
2	<b>AGENTS.md denominator.</b> <code>the-era</code> headline says "26.51% of the 300 most-starred repos"; its evidence says 215 of 300 were measured; its verifier re-ran all 300 and got 30.67%, and 32.09% on the top 215 by rank.	<b>The verifier governs: 30.67% of 300, 32.09% of the top 215.</b> The researcher's headline misstated its own denominator. <code>docs/mdmax/PLAN.md</code> v2.0.0 §2 already quotes 30.67% and is correct.
3	<b>Backtick maximum.</b> Line-initial max = 3 (15,435 runs). Any-position max = 4 (four inline occurrences). Two verifiers used opposite rules on the same corpus the same afternoon; one produced a false kill.	<b>Line-initial governs fence-sufficiency</b> (4 backticks are provably sufficient). Any-position governs only the pack format's stated <code>max+1</code> formula — <b>which computes 5 on this corpus and therefore contradicts its own justification.</b> Fix the formula's wording, not the fence width.
4	<b>Fenced-file count.</b> <code>container-thesis</code> C7: 463 of 1,084 (42.71%). <code>founder-thesis</code> verifier: 496 of 1,084 (45.76%).	<b>Neither governs.</b> Neither states its fence-detection rule. Do not quote either until one commits a derivation script.
5	<b>repomix downloads.</b> 327,543 (used for the 846:1 ratio) vs 327,053 for 2026-07.	<b>Neither.</b> Quote as "roughly 327,000/month, read 2026-08-01" and the ratio as "roughly 850:1".
6	<b>Link counts.</b> AST <code>link=5453</code> with GFM autolink literals; <code>link=1838</code> CommonMark-only; 13,028 wikilinks ( <code>hold-more</code> ) vs 9,962 <code>[ [Name] ]</code> ( <code>docs/mdmax/PLAN.md</code> ).	<b>All four are correct under their own definitions.</b> State the parser configuration with every link number or do not state the number.
7	<b>"18 of 18 headlines refuted."</b> The critic synthesis and <code>docs/mdmax/PLAN.md</code> §9 say 18/18; the machine-readable field yields 6 REFUTED + 1 UNSUPPORTED + 1 CONFIRMED explicitly labelled, 10 unlabelled.	<b>The substance governs, the literal count does not.</b> Say: <i>every area's headline was materially corrected, always flatteringly; six explicitly REFUTED and one UNSUPPORTED.</i>
8	<b>Early-cutoff rate.</b> <code>markdown-base</code> measured 19.71%; the prior record says 69.1%.	<code>markdown-base</code> 's reconciliation — "theirs is dominated by bulk machine commits (12 commits >60 files contributing 5,751 revisions)" — is <b>well-evidenced inference, not confirmed</b> ; the divergence is established, the cause is not. <b>Neither number should be published.</b>
9	<b>Markdown's share of repository trees.</b> Fell 12.68%→10.41% (12 unnamed repos) vs rose 1.142%→1.460% in 7 of 8 named repos.	<b>UNPROVEN IN BOTH DIRECTIONS.</b> <code>docs/mdmax/PLAN.md</code> v2.0.0 §2 already says "Don't say it." Correct.
10	<b>Container sequencing.</b> <code>container-thesis</code> says ship <code>pack</code> beside <code>cert</code> in the MVP; <code>founder-thesis</code> says promote it; the plan says the container is deliberately not in the MVP; <code>against</code> argues it out entirely.	<b>The plan governs</b> ( <code>docs/mdmax/PLAN.md</code> v2.0.0 §3.1): a bounded read-only machine-facing pack is a legitimate later feature, not the product and not the MVP.
11	<b><code>premortem</code>'s untracked-research risk.</b>	<b>STALE.</b> Corrected by live state at <code>1bd4dad</code> ; see G8.

13.7 Glossary

Alphabetical. Every term a cold reader needs before reading any other section of this plan.

**anchor** — a durable identifier for a piece of a document that survives an *out-of-band* edit, i.e. one made by a tool that never told us it happened ( `git pull`, `vim`, an agent writing the file). The anchor is the invention this project rests on. See `docs/engine/PLAN.md` §1.1.

**anchorable** — the gate that decides whether a block can carry an anchor at all. Defined as "not a rule, not punctuation-only, ≥ 3 tokens." **14.2% of blocks in the corpus fail the gate** and must be positionally interpolated between anchorable neighbours, never anchored. The gate moves the false-match rate from 0.349% to 0.050% and is described in the record as "the highest-leverage single line in the design."

**block** — the unit MDMAX operates on: a top-level markdown node (paragraph, heading, list, fence, table). **The block is the unit of identity, not of meaning** — `docs/engine/PLAN.md` §11a measured a heading as almost exactly as good a topic boundary as a blank line (AUC 0.508). Median block in the corpus is 84 bytes.

**block-version** — one block as it appeared in one revision of one file. The anchoring measurement's population is 41,642 block-versions drawn from 294 consecutive revision pairs.

**carrier** — the syntactic slot used to smuggle machine-readable data through a markdown file without a human seeing it. Candidates measured in `docs/engine/PLAN.md` §5: HTML comment ( `<!-- fm:file ... -->` ), link reference definition ( `[fm:file]: a/b.md "..."` ), code-fence info string, `:::` container, heading, bracketed span.

**GitHub deletes the HTML comment in both API modes**, which is the carrier-side re-derivation of D2.

**cert** — `mdmax cert`, the capped give-away artifact: per document, per target, per construct, emit **PASS · STRIP · CORRUPT**. Targets are `(product, surface)` pairs, because GitHub's API renderer and blob renderer were measured disagreeing on mermaid on identical bytes.

**confidence ladder** — the three-tier diagnostic scheme in `docs/engine/PLAN.md` §4. **E — Enforce**: mechanically decidable, zero interpretation, *and* human-accepted; errors; fails the build. **W — Warn**: mechanically decidable but plausibly deliberate; never fails. **I — Inform**: semantic, requires judgment, carries a confidence score and a reason; advisory. "A diagnostic may only reach tier E if a human accepted the constraint that produced it."

**corpus id** — `sha256:3a010b16...`. The label that makes a number about our own files reproducible. Recipe in §13.1.4. **A number about local files with no corpus id is not a measurement.**

**degradation** — what happens to a construct when it reaches a renderer that does not know it. **This is the feature, not the bug** (decision D6): an unknown *fence language* round-trips byte-perfectly; an unknown *node type* throws. Every extension MDMAX ships must degrade to something harmless.

**degradation certificate** — the per-document conformance matrix `cert` produces. See **cert**.

**false kill** — a verifier's kill that an auditor overturned. Measured across four audits at 17.24% / 3.4% / 19.0% / 0%, pooling to 9.524%. §13.3.

**falsifier (pre-registered)** — a condition an area writes down *before* concluding, which would prove its own conclusion wrong. Added in run 3 and absent from run 2. `the-era` ran four (F1–F4) and none fired. `founder-thesis` ran one that fired **in the founder's favour** and recorded that fact, which is the single clearest evidence the discipline was real.

**headline verdict** — the adjudication of an area's top-line sentence by its adversarial verifier. Vocabulary: `CONFIRMED · OVERSTATED · REFUTED · UNSUPPORTED`. **Always read this before quoting a headline.**

**kill** — a verifier's finding that a specific researcher claim is wrong. **kill audit** — the fourth-layer check that re-derives a sample of kills from primary sources. Run 3 only.

**mother markdown** — the founder's image of one markdown file containing many, split on read. **Abandoned** (`docs/mdmax/PLAN.md` §3.1): at 25,548,765 bytes it is 242x a one-line-per-document index, and it is impossible at this vault's own scale. What survives is a bounded, read-only, machine-facing `pack`.

**normalize()** — the text canonicalization applied before hashing a block: NFC + collapse whitespace + lowercase. **It is unversioned, and its first change silently re-keys every stored anchor.** Rule P7 exists solely to force it to be frozen and versioned before a single anchor is persisted.

**orphan** — a comment whose anchor no longer resolves to any range in the document. **Orphaning visibly is the design; guessing is forbidden.** The **orphan rate** ships as a visible product metric from the first comment written.

**pack / unpack** — `frontmatter pack` serialises a folder into one human-readable, dumb-viewer- renderable markdown container; `unpack` reverses it. The folder stays canonical.

**projection** — a derived, budgeted *view* of a document emitted as an ordinary `.md` file (index tier, outline tier, full). Not a new format: the output is plain markdown a model or human can read with no tooling. Byte ratios measured at 61–152x (index) and 15–35x (outline); **these are artifact-size ratios, not task-token savings** — the only live measurement of task tokens went the wrong way at +5.2%.

**recogniser** — a read-only pass that *detects* a pattern in an existing document without changing it (for example, recognising the AI markdown dialect: thematic breaks used as decoration, emoji table cells used as status). Pure mdast traversal plus position arithmetic; no new node types, no persistence, no migration. Contrast with a **writer**, which mutates bytes.

**resolver** — the function that takes a stored anchor and a new document and returns an index, or `AMBIGUOUS`, or `LOST`. Its algorithm is the S1/S2/S3 ladder in `docs/engine/PLAN.md` §1.1a: exact hash → context-scored disambiguation → shingle-based fuzzy match. Parameters: `K=3` (context radius), `TAU=0.20` (Jaccard floor), `W=1.0` (context weight), `DELTA=0.05` (tie margin, below which it refuses).

**sidecar** — data stored *outside* the `.md` file (comments, threads, history), keyed back to the file. Decision D2: nothing about comments is ever written into the markdown, and removing the sidecar must leave `git status` clean and every file byte-identical.

**softctx** — distance-weighted (  $1/d$  ) agreement of the K neighbouring blocks on each side. Used to *score* context, never to hash it. Rigid context fingerprints get **worse** as k grows (90.65 → 88.68 → 87.22 for k=1/2/3) because a ±k window straddles a moved section's boundary.

**splice** — replace only the bytes of the region being edited, computed from `mdast` `position.offset`; never regenerate the file from the AST. Decision D7, grounded in Foster et al., TOPLAS 2007, Lemma 3.9. Measured: **107,287 blocks spliced byte-identically across 1,080 files, 0 failures**, versus **0 of 120 files** surviving a stringify round trip.

**TextQuoteSelector** — a W3C Recommendation (February 2017) for anchoring an annotation to a text range by `{exact, prefix, suffix}`. **Not an invention of this project**; it is what comments should use.

**tier tags** — `[measured]` / `[primary]` / `[secondary]` / `[inference]` / `[SIMULATED]`. See the head of this appendix.

**wikilink** — `[Name]`. Resolves at **27.65%** in this corpus, against **85.71%** for `[text](path.md)`. **An unresolved wikilink is a legitimate authoring primitive; an unresolved path link is a mistake.** Treating them identically is why naive link checking runs at 2–25% precision and gets switched off.

## The ten settled decisions, restated

These are fixed. If you want to reopen one, §10 of this plan is where the two currently under review live (D3 and, indirectly, D4).

ID	DECISION
D1	frontmatter = Google Docs for markdown.
D2	Comments and history live <b>outside</b> the file. Export or copy yields clean markdown, latest content only. Nothing about comments is ever written into the <code>.md</code> .
D3	Reviewers must log in (Google or GitHub). Mechanism deferred. <b>UNDER REVIEW as a tier split</b> — anonymous read-and-comment on a share link, login required to resolve or edit.
D4	The durable artifact is plain markdown; if frontmatter dies we ship a full export and migration. Max tier gets offline access. <b>Not</b> "your git repo is the backend".
D5	AIOS is a separate track.
D6	The file stays ordinary <code>.md</code> . Degradation is the feature. No dialect needing its own parser.
D7	Splice-only writing. Never regenerate from the AST.
D8	Extensibility lives in the <b>value</b> of a field, never the <b>set</b> of node types.
D9	A document <b>names</b> a capability, never <b>carries</b> one.
D10	The name is MDMAX.

### 13.8 Where everything lives

All paths relative to `/Users/sagnikmitra/Desktop/GitHub/frontmatter`. Sizes are bytes, measured 2026-08-01. Everything listed is tracked in git at commit `1bd4dad` unless marked.

#### The plans and the record

docs/mdmax/PLAN.md	27,454	502	ln	THIS DOCUMENT (v2.0.0 → v2.1.0). The build plan.
docs/engine/PLAN.md	106,298	1,748	ln	Technical design of record, v0.6.0. Every engine measurement lives here. \$1.1a = the anchor. \$4 = the confidence ladder. \$5 = the container.
docs/engine/README.md	46,837	789	ln	The master plan / front door, v1.0.0. Where it differs from PLAN.md, PLAN.md governs.
docs/engine/dossier.html	408,550			Web version of the research dossier.
docs/engine/sgnk-markdown-engine-dossier.pdf	449,691			12-page PDF version.
docs/engine/build/				Dossier build scripts. EXCLUDED FROM THE CORPUS (policy excludes `build` at any depth).

#### The research artifacts

docs/engine/research/corpus-manifest.json	198,580			THE PIN. \$13.1.
docs/engine/research/wf-final-gate-2026-08-01.result.json	1,364,043	Run 3.	16	areas + kill audit + 4 lenses.
docs/engine/research/wf-final-gate-2026-08-01.raw.json	1,436,963	Run 3		envelope.
docs/engine/research/wf-final-gate-2026-08-01.journal.jsonl	1,333,281	Run 3,	80	records.
docs/engine/research/wf-mdmax-capability-*.result.json	1,420,260	Run 2.	18	areas + 3 lenses. No kill audit.
docs/engine/research/wf-mdmax-capability-*.raw.json	1,487,390	Run 2		envelope.
docs/engine/research/wf-mdmax-capability-*.journal.jsonl	1,382,194	Run 2,	78	records.
docs/engine/research/wf-findings-2026-08-01.md	128,449	Run 1.	6	areas, markdown, UNVERIFIED.

#### Demand research (predates the engine programme; 2026-07-12/13)

docs/FRONTMATTER-PRODUCT-PLAN.md	23,538	271	ln	92 pain points, 89 feature requests, 20-app matrix. Ranks the wedges A > C > B.
docs/research/frontmatter-pain-taxonomy.md	27,229	290	ln	T1..Tn severity taxonomy (KILLS ADOPTION > CHURNS USERS > ANNOYS).
docs/research/frontmatter-pain-playbook.md	93,259	539	ln	6 evidence drills, verbatim quotes, effort marks S/M/L, module keys.
docs/research/frontmatter-competitor-gapmap.md	14,339	77	ln	20-app competitor table with prices.
docs/research/frontmatter-segment-strategy.md	25,283	191	ln	8 segment studies + 2 market studies. MARKET-SIZE ROW IS SELF-TAGGED UNVERIFIED (10× divergence across four firms).
docs/research/frontmatter-raw-corpus.json	210,828			Round-1 scraped corpus. NOT under corpus_id.
docs/research/frontmatter-r2-raw-corpus.json	339,303			Round-2 scraped corpus. NOT under corpus_id. F1 (169:95) is measured over these two.
docs/FEATURE-GAP-REPORT.md	9,048	155	ln	Implementation status of the editor, 2026-06-03.

The earlier concept tree ( docs/mdmap/ , 21 files, generated 2026-07-29 at commit 8eb4de2)

```
docs/mdmap/MAP.md 05-agent/MAP.md
docs/mdmap/01-thesis/{MAP,pitch,steelman,recursion,why-now,three-projections,
the-gap-is-the-product,not-a-graph-view}.md
docs/mdmap/02-evidence/{MAP,graphify-hairball}.md
docs/mdmap/03-spec/{MAP,format,gaps}.md
docs/mdmap/04-views/{MAP,rendering,semantic-zoom,doi-focus}.md
docs/mdmap/06-product/MAP.md 07-open/MAP.md
```

Historical. 01-thesis/not-a-graph-view.md is the file that anticipated §3.2's conclusion. Read it for provenance of the thinking, not for current decisions.

The three handoffs (all tracked, all cited by later documents — do not delete)

```
HANDOFF-mdmax-markdown-engine-2026-08-01.md 35,751 420 ln Supersedes the other two. Written at HEAD
798ebbf. §6 = corrections, §11 = open decisions.
HANDOFF-mdz-markdown-format-2026-07-29.md 51,087 829 ln The "next-generation markdown" research thread.
~7.3M subagent tokens, 39 agents. Written at
8eb4de2 with nothing committed.
HANDOFF-graph-engineering-research-2026-07-30.md 47,094 675 ln The graph-engineering thread, reconciled
against docs/engine/PLAN.md v0.2.0.
```

The code

```
package.json frontmatter 0.1.0 · 47 dependencies · 18 devDependencies
AGENTS.md 7,705 B, 181 ln — operational rules for any agent editing this repo.
NOTE: Claude Code reads CLAUDE.md, not AGENTS.md, and no CLAUDE.md exists
here — so this file has never been loaded by the agent it addresses.
src/app/ Next.js App Router: (auth)/login, (public)/[slug], (public)/p, (vault),
(workspace), api/{ai,auth,commit,export,share,vault}
src/modules/ ai · ai-tools · app-shell · auth · drafts · editor · export · graph ·
preview · repository · share · vault (hexagonal: application / domain /
infrastructure / presentation per module — see docs/adr/0001)
src/container/ dependency-container.ts:42 — the hardcoded AUTHOR constant (§6)
src/shared/ application/ports, domain, infrastructure, presentation
test/ ai · ai-tools · api · app-shell · auth · config · drafts · editor · export · ...
docs/adr/0001-adopt-hexagonal-architecture.md 2,667 B — the only ADR.
docs/superpowers/ 13 historical build plans + 1 spec from 2026-05-25 (the `sgnk-md` era).
```

The named files a reader will be sent to

FILE:LINE	WHY IT MATTERS
src/modules/share/infrastructure/share-writer.ts:26	calls matter.stringify while its own header says "splicing" — silent data loss on every Publish
src/modules/preview/presentation/frontmatter.ts:47	calls doc.toString() — 170 of 907 frontmatter blocks throw
src/modules/auth/domain/allowlist.ts:9	the single-user allowlist
src/container/dependency-container.ts:42	the hardcoded AUTHOR constant, fed to four use-cases at lines 66, 70, 75, 80
firestore.rules	declares vaults, memberUids, roles, shares, billing, usage — and contains "comments" zero times
test/preview/html-policy-xss.test.ts	the source of the unreproducible "55 payloads" figure (G9)
docs/engine/PLAN.md:116	the 99.627% row
docs/engine/PLAN.md:80	"294 consecutive real revision pairs" — the sentence with no script behind it

Outside this repository

```
~/Desktop/GitHub/md the primary vault. 756 corpus files, 21,068,853 B, 82.46% of the corpus.
~/Desktop/GitHub/knowledge the L99 knowledge base. 272 corpus files, 3,570,923 B.
~/claude/skills-src 474 *.md / 5,013,124 B — the W1 second corpus (path-list sha256
72b867b02163ece7346e50cbae5cc71a). Still same-operator.
~/sgnk/{gates,traces,baselines,state} AIOS runtime state. A SEPARATE TRACK (D5).
```

13.9 What this appendix does not cover, and how to falsify it

It does not cover.

- The original researcher texts.** Only the post-verification results are on disk. If a verifier misquoted the claim it killed, nobody can detect it from these files. Kill-audit B named this "THE ONE LIMITATION THAT MATTERS."
- Run 1's six areas.** They are inventoried but no number in §13.4 is drawn from them, because they have no adversarial pass.
- The AIOS evidence base.** ~/.claude/CLAUDE.md 's Learned Rules, the ~30 AIOS docs and the 119 skill definitions inform how this programme was run — several of the failures catalogued here are instances of named Learned Rules (#60 an auditor inheriting its subject's blind spots, #62 labelling replayed numbers

SIMULATED, #63 equality-pinned assertions on stochastic quantities, #65 environment-dependent harness results). **They are not evidence for the product** and are deliberately out of scope here (D5).

- **Anything about the future.** Every effort estimate in the research corpus is an estimate.

**How to falsify this appendix.** Each of these is a concrete, cheap experiment. If any comes back differently, this appendix is wrong and must be corrected in the same commit.

1. Run the script in §13.1.4. If it does not print `sha256:3a010b1649899795d79274fc528dbece97fdabf4ff0f81cc02ab619c048c51a4`, the recipe is wrong.
2. Run the script in §13.1.5. If `missing` or `mismatch` is non-zero, the corpus has drifted and **every corpus-tagged number in this plan is suspect until re-run**.
3. Re-parse `headline_verdicts` from run 3. If the counts are not `CONFIRMED 5 · OVERSTATED 10 · REFUTED 1`, §13.2.3 is wrong.
4. Sum `sampled` and count `overturned` across `kill_audit`. If it is not 105 and 10, §13.3 is wrong.
5. Check `git branch -r --contains HEAD`. If it is empty, G8's correction is wrong and the pre-mortem's existential-risk finding is live again.
6. Grep the repository for a script that produces 99.627%. **If you find one, row A1's "reproduced: NO" is wrong and the number can be published.** Two independent searches came back empty; a third would be welcome.

**The one sentence to carry out of this appendix.** *Every number in this plan is either reproducible from a file in this repository, or it is labelled as not reproducible — and the labelling is the product of a four-layer chain (research → adversarial verification → kill audit → this index) whose measured error rate is roughly 10% at each layer above the first, and 100% at the first.* Read the labels.

---

## S14 Verification

Verification record

[← Index](#) · [← §13 Appendix](#)

### 14. Verification record

Four independent fact-checkers audited §1–§13 against the primary sources. All four returned **MINOR\_FIXES** — none demanded a rewrite. They identified **89 defects** and **29 numbers lacking a source**. Every one is listed below with its correction, unapplied, so a reader can audit the audit. **Corrections marked APPLIED have been merged into the body above.**

**APPLIED to the body above:** D1 (claim tally 196→206 / 117-51-29-9), D2 (Hypothes.is — every digit was wrong: 6,281→**20,953** annotations, 27%→**22%** unattachable, per a direct read of arXiv 1512.06195), D3–D4 (171 and 170 are the **same 170 files**, not two sets — kill-audit D re-measured both parsers on the pinned corpus and found an identical set), D6 (test counts — 65/3,604 replaced with **83 real files**; `vitest` collects 247 files / 3,484 tests of which **66.4% are two abandoned worktrees**), D7 (wrong path — the file is `src/modules/preview/presentation/frontmatter.ts`, verified by `find src -name 'frontmatter.ts'`).

**NOT yet applied:** the remaining defects below, including D5 — where the fact-checker found that one of the *verifiers'* kills was itself wrong (the equivalence-fold denominator is stated in the source: 40 files / 1,613 blocks of `~/Desktop/GitHub/knowledge`, 705/1613 = 43.71% strict vs 69/1613 = 4.28% semantic). That is a **false kill**, and it is exactly the failure mode the kill-audit phase exists to catch.

#### 14.1 Defects

##### D1 · `S13-appendix §13.2.3`

- Claim:** "Across all 16 areas, 206 individual claims were adjudicated: 117 CONFIRMED, 51 OVERSTATED, 29 REFUTED, 9 UNSUPPORTED."
- Problem:** Wrong on three of four counts, and contradicted by S13's OWN per-area table two paragraphs above (whose rows I verified individually — all 16 are exact) and by S01 §1.8c. Re-derived from `wf-final-gate-2026-08-01.result.json` over `per_area[].verification.verdicts`.
- Correction:** Replace with: "Across all 16 areas, 206 individual claims were adjudicated: 117 CONFIRMED, 51 OVERSTATED, 29 REFUTED, 9 UNSUPPORTED." Re-derivable with: `python3 -c "import json, collections; d=json.load(open('docs/engine/research/wf-final-gate-2026-08-01.result.json'))"; print(collections.Counter(x.get('verdict') for x in a[d['per_area'] for a in a['verification']]['verdicts']))" → Counter({'CONFIRMED': 117, 'OVERSTATED': 51, 'REFUTED': 29, 'UNSUPPORTED': 9}), total 206. [measured, this session]`

##### D2 · `S02-chronology §2.5 item 6`

- Claim:** "**Hypothes.is** in production with published failure data: **about 22% of 20,953 annotations can no longer attach** [secondary — arXiv 1512.06195, NOT read directly; verify before citing]."
- Problem:** This is the figure the final gate explicitly retired. The `against` area read arXiv 1512.06195 directly at `primary-read` tier and states: "The external-baseline number this program has been carrying is wrong in every digit. I read arXiv 1512.06195 directly; the internal research file reports n=20,953 / 27% / 61% / 3.5%, and the paper says 20,953 / 22% / 53% / 12%." S13 A10 already carries the corrected figure, so S02 and S13 contradict each other inside the same document.
- Correction:** Replace with: "**Hypothes.is** in production with published failure data: of **20,953** highlighted-text annotations, **about 22%** can no longer be attached to their live pages; of those still attached, **53%** are at risk; only about **12%** of orphans are recoverable from web archives. [primary — arXiv 1512.06195 abstract, read directly by the final gate's `against` area; the 20,953 / 27% / 61% / 3.5% figures previously carried in `docs/engine/research/wf-findings-2026-08-01.md:307` are wrong in every digit and must not be repeated] "

##### D3 · `S03-mdmax-capabilities §3.1.4 (Violation 2)`

- Claim:** Under "**Violation 2** — `src/modules/preview/presentation/frontmatter.ts:47`": "Measured against every front-matter block in the pinned corpus: **170 of 907 blocks throw** under gray-matter, and **only 33 survive byte-identical (3.64%).**"
- Problem:** Misattribution: 171/33/3.64% are the measurements of Violation 1 (`share-writer.ts`, gray-matter), already quoted three paragraphs earlier. `frontmatter.ts` uses the `yaml` package (`parseDocument`), never gray-matter — verified by reading the file: line 17 is `import { parseDocument, isMap, type Document } from "yaml";`. The `product-gap` area measured this path separately: "Ported exactly and run on the same corpus\_id: `byte_identical=119`, `refused-and-left-untouched=170`, `changed=618` → 31.86% safe vs 3.64%."
- Correction:** Replace the measurement sentence with: "Measured against every front-matter block in the pinned corpus, this path (ported exactly) leaves **119 of 907 byte-identical, refuses 170, and changes 618 — 31.86% safe against** `share-writer.ts`'s **3.64%**. [measured, corpus\_id sha256:3a010b16..., final-gate product-gap C6] A library swap is therefore not the fix: 31.86% is still a data-loss path. And note the app already ships this preserving path — the Publish path simply does not use it." Keep 171/33/3.64% under Violation 1 only.

##### D4 · `S03-mdmax-capabilities §3.1.5 and §3.1.10`

- Claim:** "**Note the two nearby numbers are not the same number.** 171 is the count of blocks `gray-matter` throws on; 170 is the count `yaml` cannot parse. Different parsers, different failures. Do not merge them." — and §3.1.10's gate: "INCLUDING the 171 that `gray-matter` cannot read and the 170 that `yaml` cannot read".
- Problem:** Contradicted by an audited primary re-measurement. Kill-audit D, on the pinned corpus: "907 of 1084 files carry YAML frontmatter; gray-matter THROWS on **170**; eemeli yaml reports `doc.errors` on **170**; both = 170 — **literally the identical set, not merely the same count.**" The instruction "do not merge them" is therefore backwards, and the gate wording invents two disjoint sets where there is one. (The `product-gap` finding's own text also says "170 of 907 frontmatter files throw in gray-matter/js-yaml; independently, eemeli/yaml ... parses 0 of those 170" — i.e. the same files.)
- Correction:** Replace §3.1.5's note with: "**171 and 170 are the same files, not two sets.** The `product-gap` area reported 171 gray-matter throws; kill-audit D re-measured both parsers on the pinned corpus and found gray-matter throws on **170** and eemeli/ `yaml` reports `doc.errors` on **170** — *literally the identical set*. Quote **170 of 907 (18.74%)** and say 'no standard YAML parser can read them'. [measured, kill audit — PERFECT REPRODUCTION, 4/4 numbers] " And in §3.1.10 change the gate to "...INCLUDING the 170 that no standard YAML parser can read."

##### D5 · `S03-mdmax-capabilities §3.3.4 (killed claim 5 / trailing paragraph)`

- Claim:** "Also unresolved: the **equivalence fold** is claimed to have moved the divergence rate from **43.71% to 4.28%** — but with **no corpus, no artifact, and no denominator**. It is the load-bearing component of the whole design and it is currently a rate without a population."

- **Problem:** The population IS stated in the source and both percentages re-derive exactly from it. `degradation-certificate`'s SELF-CORRECTION evidence reads: "AST-accurate top-level blocks (remark-parse positions) over **40 files / 1,613 blocks of ~/Desktop/GitHub/knowledge**, 6 GFM-class engines: `STRICT_divergent: 705, STRICT_pct: 43.71` vs `SEMANTIC_divergent: 69, SEMANTIC_pct: 4.28`."  $705/1613 = 43.708\%$  and  $69/1613 = 4.278\%$ . S03 is repeating the verifier's "RATE WITHOUT DENOMINATOR" complaint, which is itself defective — a false-kill candidate under the plan's own 0–19% false-kill rule.
- **Correction:** Replace with: "Also flagged: the verifier listed the **equivalence fold** ( $43.71\% \rightarrow 4.28\%$ ) as a 'rate without a denominator'. **That kill is wrong and should be recorded as one** — the area's own evidence names the population: **40 files / 1,613 top-level blocks of ~/Desktop/GitHub/knowledge**, **6 GFM-class engines**, `STRICT_divergent 705` ( $705/1613 = 43.71\%$ ) vs `SEMANTIC_divergent 69` ( $69/1613 = 4.28\%$ ). Both re-derive exactly. `[measured, area evidence; kill overturned this session]` The *legitimate* criticism survives and must travel with the number: that population is a 40-file slice of one root and **carries no corpus\_id**, so re-run the fold over `corpus_id sha256:3a010b16...` before publishing either rate."

#### D6 · S01-orientation §1.6

- **Claim:** "tests | 83 test files under `test/` | `[measured]` | 3,484 tests collected (66.4% of them abandoned worktrees) at last full run `[primary, HANDOFF §10, not re-run this session]`"
- **Problem:** Both numbers are refuted inside this same document. S02 §2.1 (Round 3) records the deep read that caught it: `vitest list --run --filesOnly` → 247 files / 3,484 tests, of which 164 files / 2,314 tests are two abandoned worktrees pinned at `8eb4de2`, and rules **"The measurement governs.** ... Neither '65' nor '3,484' should be repeated." I re-counted live: `find test -type f \ ( -name '*.test.ts' -o -name '*.test.tsx' \ )` = **83**, and each worktree holds 82 ( $83 + 82 + 82 = 247$ , reconciling exactly). Also the HANDOFF citation is wrong: the "83 test files under `test/`, 3,484 tests" line is §0.7 (line 22), not §10.
- **Correction:** Replace the two rows with: "tests | **83 test files under** `test/` | `[measured, this session]` | `vitest list --run --filesOnly` reports **247 files / 3,484 tests**, of which **164 files / 2,314 tests (66.4%) are two abandoned git worktrees pinned at** `8eb4de2` that `vitest.config.ts` does not exclude. `HANDOFF-mdmax-markdown-engine-2026-08-01.md` §0.7's '83 test files under `test/`, 3,484 tests collected (66.4% of them abandoned worktrees)' was already inflated when written; do not repeat either figure. See the chronology section."

#### D7 · S13-appendix §13.8 ("The named files a reader will be sent to")

- **Claim:** Row: " `src/modules/preview/presentation/frontmatter.ts:47` | calls `doc.toString()` — 170 of 907 frontmatter blocks throw"
- **Problem:** Two defects in one row. (a) Wrong path: the file is `src/modules/preview/presentation/frontmatter.ts` — verified by `find src -name 'frontmatter.ts'`, which returns exactly that one path and nothing under `src/modules/editor/`. S03 §3.1.4 has it right. (b) Same misattribution as S03 §3.1.4: the 171 throws are the `gray-matter/ share-writer.ts` figure; the `doc.toString()` path is the `yaml` path measured at 119 byte-identical / 170 refused / 618 changed.
- **Correction:** Replace the row with: " `src/modules/preview/presentation/frontmatter.ts:47` (and 59) | calls `doc.toString()` — this is the `yaml`-backed preserving path; a no-op round trip leaves only **119 of 907 blocks byte-identical (13.12%)**, refuses 170, and changes 618"

#### D8 · S02-chronology §2.4 row 7 ("Changing CommonMark")

- **Claim:** "Re-derived independently in the final gate: spec commits 2019=48, 2020=15, 2021=14, 2022=15, **2023=12, 2024=12**, 2025=7, 2026=6 (partial)"
- **Problem:** Directly violates the governing rule S01 §1.4a sets on the same evidence. Two sources in the record disagree on exactly these two years and neither carries a committed script: the `the-era` verifier bare-cloned the repo and got "2019=48, 2020=15, 2021=14, 2022=15, **2023=13, 2024=11**, 2025=7, 2026=6 — matches '48/15/14/15/13/11/7/6' digit for digit"; the kill auditor re-ran it on `api.github.com` and got "...**2023=12, 2024=12**...". S01 rules: "quote the years both sources agree on (48 in 2019 falling to 7 in 2025) and do not quote 2023 or 2024 individually until a script is committed."
- **Correction:** Replace with: "Re-derived independently in the final gate: spec commits **48 in 2019 falling to 7 in 2025** (2026 = 6 over seven months, partial). The two sources agree on 2019, 2020, 2021, 2022, 2025 and 2026 and both refute the monotonic-decline adjective (2021=14 rises to 2022=15); they split on 2023/2024 (verifier bare-clone 13/11 vs kill-audit API 12/12), so **do not quote 2023 or 2024 individually until a derivation script is committed.** `[contradiction, unresolved]`"

#### D9 · S02-chronology §2.5 (tenth item, the carrier flip)

- **Claim:** "the degradation-certificate area measured the HTML comment carrier **visible in 2 of 12 outputs and payload-destroyed in 10.**"
- **Problem:** Two errors. (a) The denominator is 24 renderer configurations, not 12: "Construct `html-comment` ... **visible 2/24**, payload-in-output 14/24, 8 distinct outputs." (b) "payload destroyed in 10" was **killed by that area's own verifier**: "'payload destroyed in 10 / payload-in-output 14' for the HTML comment (**measured 9 / 15**)." S03 §3.3.3 already uses the corrected "9 distinct outputs", so S02 and S03 disagree.
- **Correction:** Replace with: "the degradation-certificate area measured the HTML comment carrier **visible as literal text in 2 of 24 renderer configurations, with the payload destroyed in 9 and present in the output of 15** (the researcher's '10 destroyed / 14 present' and 'eight distinct outputs' were killed by its own verifier and corrected to 9 / 15 / nine distinct outputs), **and deleted outright by GitHub's blob renderer.**"

#### D10 · S13-appendix §13.6 row 4

- **Claim:** "**Fenced-file count.** `container-thesis` C7: 463 of 1,084 (42.71%). `founder-thesis` verifier: 496 of 1,084 (45.76%). | **Neither governs.** Neither states its fence-detection rule. Do not quote either until one commits a derivation script."
- **Problem:** "Neither states its fence-detection rule" is false — the `founder-thesis` verifier states both rules precisely and adjudicates between them: "'463 (42.71%) contain a 'fence' reproduces only under a **\*\*column-0-only\*\*** match; 'contains' **anywhere** is 496 = 45.76% and the CommonMark 0–3-space-indent fence count is 489 = 45.11%," with the correction "Report the fence-containing rate as 45.76% (or 45.11% CommonMark), not 42.71%." The disagreement is already resolved in the record, and S03 §3.7.5 item 8 uses 496/45.76%.
- **Correction:** Replace the ruling with: "**Both are correct under stated, different rules, and the verifier adjudicated.** 463 (42.71%) is a column-0-only match; 496 (45.76%) is 'contains' anywhere; the CommonMark 0–3-space-indent count is 489 (45.11%). **Quote 45.76% (or 45.11% CommonMark) and name the rule in the same sentence.** `[measured, founder-thesis verifier]`"

#### D11 · S02-chronology §2.1 (Phase 0, discrepancy note)

- **Claim:** "**Nothing dated 2026-07-12 exists in this repository.** The git record governs. If something happened on 2026-07-12 it happened outside version control and outside every transcript in `~/claude/projects/`."
- **Problem:** Refuted by a primary read of the repository. `docs/FRONTMATTER-PRODUCT-PLAN.md` line 3 reads verbatim: "**Date:** 2026-07-12 · **Starting codebase:** this repo (`md`, branch `stabilize/md-uptime`, deployed at `md.sgkn.ai`). `docs/research/frontmatter-competitor-gapmap.md` and `docs/research/frontmatter-pain-taxonomy.md` carry filesystem mtimes of `12 Jul 10:28`. S01 §1.5 and S13 §13.8 both date the demand research 2026-07-12, so this note contradicts two other sections.

- **Correction:** Replace with: "No git object predates `b9ddcfe` at 2026-07-13T16:33:44+05:30. But 2026-07-12 is not fictional: `docs/Frontmatter-Product-Plan.md:3` self-dates to 2026-07-12, and two `docs/research/` files carry 2026-07-12 mtimes. The demand research was done on 2026-07-12 and committed on 2026-07-25 in `8eb4de2`. Where a document's self-declared date and the git record differ, both are reported; the git record fixes when it entered version control, not when the work was done. [primary + measured]"

#### D12 · S13-appendix §13.4 row B5

- **Claim:** "907 files carry YAML frontmatter; 171 throw under the editor's writer; 33 survive byte-identical; 624 remainder."
- **Problem:** The remainder is wrong and the components do not sum. `product-gap`'s evidence: "byte\_identical=33 (3.64% of 907), gray-matter throws=171, byte\_changed=703."  $33 + 171 + 703 = 907$ ;  $33 + 171 + 624 = 828$ . (624 appears to be an accidental import from the `yaml` row's 618-changed figure.) Also "the editor's writer" is imprecise: this is `src/modules/share/infrastructure/share-writer.ts`, the Publish path, which lives in the `share` module.
- **Correction:** Replace with: "907 files carry YAML frontmatter; a no-op write through the Publish path (`src/modules/share/infrastructure/share-writer.ts`, gray-matter) throws on 171, leaves 33 byte-identical (3.64%), and changes 703.  $33 + 171 + 703 = 907$ ."

#### D13 · S03-mdmax-capabilities §3.2.8

- **Claim:** "Do not vendor `dom-anchor-text-quote` as the substrate. ... There is no exact path, no candidate array, no scoring, and the only returns are `null` or one `{start, end}` — it cannot express `AMBIGUOUS`. ... Also: it is not in production at Hypothesis."
- **Problem:** Everything stated is correct, but the section omits the audited correction that materially narrows MDMAX's novelty claim, and by naming only the wrong package it leaves the reader believing the algorithm shape is unclaimed. Kill-audit correction, verbatim: "The right artifact exists and the researcher's DIRECTION survives under a corrected citation. `hypothesis/client/src/annotator/anchoring/match-quote.ts` is 163 lines and does implement MDMAX's shape: an exact-match pass collecting ALL occurrences via an `indexOf` loop, falling back to `approxSearch` (`approx-string-match@2.0.0`, MIT) with `maxErrors = min(256, quote.length/2)`, then a weighted context score — `quoteWeight 50`, `prefixWeight 20`, `suffixWeight 20`, `posWeight 2` ... Cite that, at 163 lines + `approx-string-match`, not `dom-anchor-text-quote`. Note what still is NOT in it: it sorts by score and returns `scoredMatches[0]` unconditionally — no `AMBIGUOUS` state, no margin."
- **Correction:** Add immediately after the `dom-anchor-text-quote` paragraph: "Cite the right artifact instead. `hypothesis/client/src/annotator/anchoring/match-quote.ts` is 163 lines and already implements MDMAX's exact shape in production: an exact-match pass collecting all occurrences via an `indexOf` loop, an `approxSearch` fallback (`approx-string-match@2.0.0`, MIT, `maxErrors = min(256, quote.length/2)`), then a weighted context score — `quoteWeight 50`, `prefixWeight 20`, `suffixWeight 20`, `posWeight 2` with the comment 'Proximity to expected location. Used as a tie-breaker.' [primary, kill audit] The one thing it does not have is the refusal: it sorts by score and returns `scoredMatches[0]` unconditionally — no `AMBIGUOUS` state, no margin. That, and the published false-match rate, is the whole of what MDMAX can own. S1/S2/S3 is not an invention; DELTA is."

#### D14 · S13-appendix §13.4 row G9

- **Claim:** "the "55 payloads" figure cited in `docs/engine/PLAN.md` §15, `docs/engine/README.md` lines 664 and 787 and the dossier does not reproduce"
- **Problem:** The `docs/engine/PLAN.md` citation is wrong. `grep -n "55 payload" docs/engine/PLAN.md` returns nothing; the only two `\b55\b` hits in that file are line 111 ( `55.50%`, an anchoring false-match rate) and line 1273 ( `55,090,364`, a byte count). The figure appears only at `docs/engine/README.md:664`, `docs/engine/README.md:787` and `docs/engine/dossier.html:789`.
- **Correction:** Replace with: "the "55 payloads" figure cited at `docs/engine/README.md:664`, `docs/engine/README.md:787` and `docs/engine/dossier.html:789` does not reproduce: the file has 327 lines, 15 test blocks (14 `it( + 1 test( )`, arrays of 39 and 6 (=45); a broader sweep gives 71. `docs/engine/PLAN.md` does not carry the figure. | live stale number in three published artifacts"

#### D15 · S01-orientation §1.7 (evidence-location table)

- **Claim:** "`docs/mdmap/` | Eight entries — the earlier concept tree ( `01-thesis` ... `07-open` plus `MAP.md` ). Historical; superseded by this plan"
- **Problem:** "Eight entries" counts only the top-level directory entries and contradicts two other sections and the brief. `find docs/mdmap -type f -name '*.md' | wc -l = 21`. S02 §2.1 and §2.3 both say "21 files, 16,906 words" (which I re-measured: `find docs/mdmap -type f -name '*.md' -exec cat {} + | wc -w = 16,906` exactly), and S13 §13.8 lists 21 files.
- **Correction:** Replace with: "`docs/mdmap/` | 21 markdown files across eight top-level entries ( `01-thesis` ... `07-open` plus `MAP.md` ), 16,906 words. The earlier concept tree. Historical; superseded by this plan [measured]"

#### D16 · S03-mdmax-capabilities §3.4.3 / §3.6.6 / §3.4.4

- **Claim:** §3.4.3: "BREAK ... 95, hand-verified real broken links; DRIFT ... 52; GHOST ... 5,777 ... [measured, impact-preview area]" and "A checker that reports 3,442 problems is uninstalled the same day." §3.6.6: "547 of 547 self-declared updated: dates in a real, automation-heavy repo are stale, 32.9% by more than a week. [measured]" §3.4.4: "69,805 rows for 78 MB", "607 KB for a 78 MB vault (0.74%)", "0.254 ms median, 1.567 ms p90".
- **Problem:** Every one of these is a number about our own local files with no `corpus_id` and no named population, which §1.8b defines as "not a measurement". Their real populations are stated in the sources and are all different from the pinned corpus: the impact-preview figures come from "census.py over 3 corpora (md-vault 4,287 files/75.94 MB; knowledge 421/7.39 MB; frontmatter 112/1.54 MB)", and the 547/547 figure comes from "git ls-files '\*.md' in `~/Desktop/GitHub/md` (4,131 tracked .md files)". Both are from the 18-area capability run, whose headlines carry the measured 100% defect rate, and §3.6.6 gives no area key at all.
- **Correction:** Append the population to each. §3.4.3: "[measured, 18-area capability run, impact-preview area – census over three UNPINNED corpora (md-vault 4,287 files / 75.94 MB; knowledge 421 / 7.39 MB; frontmatter 112 / 1.54 MB). \*\*No corpus\_id; not the pinned corpus.\*\* Re-run over sha256:3a010b16... before quoting.]" §3.6.6: "[measured, 18-area capability run – git ls-files '\*.md' over 4,131 tracked files in `~/Desktop/GitHub/md`. \*\*No corpus\_id.\*\* Also check the area's verification block before quoting: that run's headlines have a measured 100% defect rate.]" §3.4.4: add "[measured, markdown-base / impact-preview – a 78 MB vault, not the 25,548,765-byte pinned corpus; no corpus\_id]"

#### D17 · S13-appendix §13.1.4 (the derivation script)

- **Claim:** The ten-line `scripts/derive/corpus-id.py` containing: `print(f"lines={len(lines)} blob_bytes={len('\n'.join(lines).encode())}")`
- **Problem:** That line is a `SyntaxError` on Python 3.11 and earlier: a backslash inside an f-string replacement field only became legal in Python 3.12 (PEP 701). The plan instructs the reader to commit this as a `#!/usr/bin/env python3` script, so on any machine whose `python3` is  $\leq 3.11$  the script that exists to end the corpus-id false-alarm class will itself fail to parse — the exact self-refuting-tool pattern the plan warns about elsewhere. (Everything else in §13.1.4 is correct: I ran the recipe and it printed `lines=1084 blob_bytes=131186` and the declared digest, and the stated first/last sorted lines are exact.)
- **Correction:** Change the print line to avoid the backslash inside the f-string expression, e.g. hoist the blob: replace the two lines

```
digest = "sha256:" + hashlib.sha256("\n".join(lines).encode()).hexdigest()
print(f"lines={len(lines)} blob_bytes={len('\n'.join(lines).encode())}")
```

with

```
blob = "\n".join(lines).encode()
digest = "sha256:" + hashlib.sha256(blob).hexdigest()
print(f"lines={len(lines)} blob_bytes={len(blob)}")
```

Output is unchanged and it parses on every Python 3.x.

#### D18 · S01-orientation §1.4a (the frozen half)

- **Claim:** The four-row CommonMark table (0.29 = 649 @ 2019-04-06, 0.30 = 652 @ 2021-06-19, 0.31.0 = 652, 0.31.2 = 652 @ 2024-01-28) followed immediately by: "→ ZERO net new released conformance examples in five years."
- **Problem:** Read against the table directly above it, the arrow is false: 0.29 → 0.30 adds +3 examples (649 → 652). The source's claim is anchored differently — the the-era falsifier F3 says "CommonMark released examples 652 (0.30, 2021-06-19) → 652 (0.31.2, 2024-01-28)" and the verifier's CONFIRMED verdict is on "CommonMark shipped zero new conformance examples since 2021-06-19". A hostile reader with the table in front of them will catch this.
- **Correction:** Replace the arrow with: "→ ZERO net new released conformance examples since 2021-06-19 — five years and counting. (0.29 → 0.30 added the last three; nothing has been added to a release since.)"

#### D19 · S13-appendix §13.8 ("The plans and the record" / superpowers row)

- **Claim:** "docs/superpowers/ | 13 historical build plans + 1 spec from 2026-05-25 (the sgnk-md era)."
- **Problem:** Off by one and implies 14 files. `find docs/superpowers -type f | wc -l` = 13: docs/superpowers/plans/ holds 12 files (b1-b12 plus -roadmap, which is 11 build plans + 1 roadmap) and docs/superpowers/specs/ holds 1 ( 2026-05-25-sgnk-md-web-editor-design.md ).
- **Correction:** Replace with: " docs/superpowers/ | 13 files: 12 historical build plans (b1-b12 plus a roadmap) + 1 web-editor design spec, all dated 2026-05-25 (the sgnk-md era)."

#### D20 · S02-chronology §2.1 (Round 2, 2026-07-29T21:26:51 IST)

- **Claim:** "2026-07-29T21:26:51 IST — the first four commits. 5ff90a4 (mdmap tree + engine plan v0.2) and 58322f7, which fixed two real bugs in shipped code... Then 33d1b3c (§12 token corrections), 57411a1 (§11a...) and ae81971 (§1.1a...)."
- **Problem:** Five commits are named under a heading that says four. Verified against `git log --format='%h|%ad|%s' --date=iso-strict : 5ff90a4 @ 21:26:51, 58322f7 @ 21:27:11, 33d1b3c @ 21:47:57, 57411a1 @ 21:50:50, ae81971 @ 21:55:28` — five commits, spanning 21:26:51 to 21:55:28, not one timestamp.
- **Correction:** Replace the heading with: "2026-07-29 21:26:51 → 21:55:28 IST — the first five commits."

#### D21 · S02-chronology §2.1 (Round 3, the 1bd4dad file list)

- **Claim:** "src/modules/vault/infrastructure/search-index.ts | 15 +/- 3"
- **Problem:** Wrong on both numbers. `git show --numstat --format=' ' 1bd4dad` gives 11\t4\tsrc/modules/vault/infrastructure/search-index.ts — 11 added, 4 deleted. The "15" is `git show --stat 's` total-changed count (11 + 4), and "3" appears nowhere. The commit's totals are +28,147 / -4, and 4 is the whole commit's deletion count, all from this file.
- **Correction:** Replace with: " src/modules/vault/infrastructure/search-index.ts | 11 + / 4 - (the commit's only deletions)"

#### D22 · S02-chronology §2.1 (Phase 0, the 8eb4de2 session)

- **Claim:** "That commit also landed docs/FRONTMATTER-PRODUCT-PLAN.md (271 lines) and docs/research/ (4 files, 1,097 lines)."
- **Problem:** `git show --stat 8eb4de2 -- docs/` shows docs/research/ landing six files: frontmatter-competitor-gapmap.md (+78), frontmatter-pain-playbook.md (+540), frontmatter-pain-taxonomy.md (+291), frontmatter-segment-strategy.md (+192), plus frontmatter-r2-raw-corpus.json (+3,025) and frontmatter-raw-corpus.json (+3,634). The 4-file / 1,097-line figure is the four .md files at their current `wc -l`. S01 §1.7 correctly says "Six files", so the two sections disagree.
- **Correction:** Replace with: "That commit also landed docs/FRONTMATTER-PRODUCT-PLAN.md (271 lines) and docs/research/ (six files: four .md totalling 1,101 lines as committed, plus the two raw-corpus JSON files, 3,634 and 3,025 lines)."

#### D23 · S02-chronology §2.1 (Round 2, commit eadbf6f )

- **Claim:** " eadbf6f | 04:36:52 | A DOM-verified XSS regression corpus, 55 payloads, proven to fail 28/55 against a deliberately neutered policy"
- **Problem:** Repeats a figure that S13 G9 records as unreproducible, with no caveat — so the two sections state incompatible things about the same file. The market-and-gift area measured test/preview/html-policy-xss.test.ts at 327 lines, 15 test blocks, arrays of 39 and 6 (=45), with a broader sweep giving 71. I re-verified the file is 327 lines with 14 `it( + 1 test(` . Nothing in it yields 55.
- **Correction:** Append to the row: "— but the '55 payloads' figure does not reproduce: the file is 327 lines with 15 test blocks and arrays of 39 and 6 (=45); a broader sweep gives 71. The figure is still live in docs/engine/README.md:664 , :787 and the dossier and must be re-derived or struck. [measured] "

#### D24 · S13-appendix §13.4 row G6

- **Claim:** "20 commits, all by Sagnik Mitra <sagnikmitra123@gmail.com> | premortem, verified | [measured] | reproduced YES"
- **Problem:** Stale against live state, and S13 corrects G8 for exactly this reason but leaves G6 uncorrected. `git rev-list --count HEAD` = 21 today; `git log --format='%an <%ae>' | sort | uniq -c` = "21 Sagnik Mitra sagnikmitra123@gmail.com". The pre-mortem measured 20 because it ran before 1bd4dad .
- **Correction:** Replace with: "21 commits at 1bd4dad, all by Sagnik Mitra <sagnikmitra123@gmail.com> (the pre-mortem measured 20, before 1bd4dad landed) | premortem + live re-check | [measured, 2026-08-01] | reproduced YES"

#### D25 · S01-orientation §1.8e vs S02-chronology §2.1 and S13-appendix §13.2.3

- **Claim:** S01: "The four synthesis lenses in the final gate, at `synthesis[0..3]`: *era-and-vision, product-and-market, technical, the-verdict*." S02 and S13 both name the fourth lens *premortem-integrator*.
- **Problem:** Three sections give two different names for `synthesis[3]`, and a cold reader following S13's name through S01's `python3` recipe will not find it. The artifact's own `lens` field reads: "THE VERDICT – integrating the opposition case and the pre-mortem against all 16 areas, for two founders who start building immediately." S01's rendering matches the artifact; S02/S13 match the workflow brief.
- **Correction:** Standardise on the artifact and note the alias once. In S01 §1.8e, S02 §2.1 (Round 3) and S13 §13.2.3, write: "*era-and-vision, product-and-market, technical, and the-verdict* (the premortem-integrator lens; its `lens` field reads THE VERDICT – integrating the opposition case and the pre-mortem against all 16 areas )."

#### D26 · S01-orientation §1.8d item 3 vs S13-appendix §13.3

- **Claim:** S01: "the unweighted mean across the four auditors is **90.08%**." S13: "the unweighted mean of the four published rates is **90.07%**."
- **Problem:** Two sections publish two values for one derived statistic, and neither matches an exact derivation. From `kill_audit[].upheld/sampled`: 24/29 + 28/29 + 17/21 + 26/26 = 82.7586 + 96.5517 + 80.9524 + 100.0000, mean = **90.066%**. From the four *published* rounded rates (17.24 / 3.4 / 19.0 / 0.0): mean 9.91%, i.e. **90.09%**. Both S01's 90.08% and S13's 90.07% are third values.
- **Correction:** Use one derivation in both sections: "the pooled rate is **95/105 = 90.476%**; the unweighted mean of the four auditors' upheld rates (24/29, 28/29, 17/21, 26/26) is **90.07%**. Neither rounds to the 88% in `docs/mdmax/PLAN.md` v2.0.0 §0. **Quote 90.5% pooled and always state the 0%–19.0% per-auditor range alongside it.**" (S01's §1.4a/§1.8d numbers are otherwise exact: 1 re-derived 29/24/5, 29/28/1, 21/17/4, 26/26/0, pooled 105/95/10 = 9.524%.)

#### D27 · S03-mdmax-capabilities §3.4.2

- **Claim:** "The verifier found the headline census **silently stripped YAML front matter**, removing **3,850 links (29.6% of all wikilinks)** carried by `entities:` (2,283), `related:` (1,407) and `up:` (49) keys."
- **Problem:** One of four keys is dropped, and the three listed sum to 3,739, not 3,850, so the decomposition visibly fails to reconcile. The source lists four: "carried by the keys entities: (2,283), related: (1,407), **relationships: (89)** and up: (49)".
- **Correction:** Replace with: "...removing **3,850 links (29.6% of all wikilinks)** carried by the `entities:` (2,283), `related:` (1,407), `relationships:` (89) and `up:` (49) keys. (Those four sum to 3,828; the source's own decomposition leaves 22 unattributed — quote the 3,850 total and the key list, not a reconstructed sum.)"

#### D28 · S04 §4.2 item 2 and §4.12 ("Consumers must include the four this research could not reach")

- **Claim:** "GitHub Pages was never executed. The gem install of `kramdown` + `kramdown-parser-gfm` into a temporary `GEM_HOME` failed. Every Pages/kramdown cell is a `[primary]` read of `gettalong/kramdown doc/syntax.page` — and §4.12 lists "GitHub Pages via a real `kramdown` + `kramdown-parser-gfm` install" as unreached.
- **Problem:** False for the research corpus as a whole, and it directly contradicts S05 §5.1.2, which tags the github-pages row `[measured, 24-config bench]`. The capability run's `degradation-certificate` area BUILT AND RAN a 24-configuration differential harness that includes `kramdown 2.5.2 + kramdown-parser-gfm 1.1.0` with `input: GFM` (twice: default and GFM), and its verifier independently rebuilt the bench (`/private/tmp/claude-501/mdcert/merged.json`) and reproduced every kramdown row. The Jekyll default was read from primary source (`github/pages-gem lib/github-pages/configuration.rb` lines 15–30). Only the *hold-more* area failed its gem install.
- **Correction:** Rewrite as: "The *hold-more* area could not execute kramdown (its gem install into a temporary `GEM_HOME` failed), so its own Pages cells are `[primary]` reads of `gettalong/kramdown doc/syntax.page` plus `[inference]`. A different area in the same corpus — the capability run's `degradation-certificate` — DID execute `kramdown 2.5.2 + kramdown-parser-gfm 1.1.0` at `input: GFM`, GitHub Pages' verified default (`github/pages-gem lib/github-pages/configuration.rb`:15–30), as 2 of its 24 configurations, and its verifier rebuilt the bench and reproduced the rows. Use those measurements where they exist." Delete GitHub Pages from §4.12's "could not reach" list, or narrow it to "a real Jekyll pipeline (Jekyll sets nine kramdown options and turns `hard_wrap` OFF, which the bench config did not model — verifier correction, `degradation-certificate` killed list)".

#### D29 · S04 §4.4.1 row 2, §4.4.2 row 2, §4.4.3 carrier 2, and S06 §6.6 carrier table

- **Claim:** S04 ranks "Unknown-language fence + info-string meta" as **Tier 1** with the GitHub-Pages cell reading "body visible `[inference]`"; S06 §6.6 ranks "fence META string" as the **"best block-scoped carrier"**.
- **Problem:** Both omit the one measured result that kills the meta channel on a real product surface. `degradation-certificate` measured on `kramdown 2.5.2 + kramdown-parser-gfm 1.1.0` (`input: GFM`): ````js title="a.js" → <p><code>js title="a.js"\nconst a=1;\n</code></p>` — the fence collapses to inline code and the META STRING IS VISIBLE. The verifier reproduced every kramdown measurement byte-exactly and its correction reads verbatim: "kramdown/GFM — GitHub Pages' engine — collapses a fence whose INFO STRING contains whitespace (lang + meta) into inline code, making the meta visible. **This kills fence META as a carrier on github-pages**; it does not touch the unknown-LANG invariant, which holds in 23/24 configs including kramdown/GFM." S04's Pages cell ("body visible") is both the wrong behaviour and the wrong tier.
- **Correction:** S04 §4.4.2 row 2, replace "body visible `[inference]`" with: "**meta VISIBLE, code block destroyed** — ````js title="a.js" → <p><code>js title="a.js"\nconst a=1;\n</code></p>` on `kramdown 2.5.2 + kramdown-parser-gfm 1.1.0` at `input: GFM`. `[measured, capability-run/degradation-certificate, verifier-reproduced]` An unknown language with NO meta survives as `class="language-mdmax-meta"` in 23/24 configs including kramdown/GFM. Add to §4.4.3 carrier 2 a hazard: "*The meta slot is dead on GitHub Pages.* Any whitespace in the info string collapses the fence to inline code there." S06 §6.6, change the fence-META verdict from "best block-scoped carrier" to "best block-scoped carrier **on every surface except GitHub Pages**, where a whitespace-bearing info string leaks the meta as visible text (measured)".

#### D30 · S04 §4.5(a) (" `mode=gfm` **\*\*is the comment box\*\***") and §4.2 item 1; S05 §5.1.1, §5.1.2 (github-comment row), §5.5.1

- **Claim:** S04: "`mode=gfm` **is the comment box**. So a comment written into a document and then pasted into a pull request or an issue is **destroyed silently, in the surface where review actually happens**" and "The comment-box (`gfm` mode) results are solid — that surface *is* what the API serves." S05 carries the same identification into the `github-comment` matrix row and into "Any case-sensitive identifier ... COLLAPSES when a document passes through the comment box."
- **Problem:** The identification is asserted as fact and is contradicted by a verifier in the same corpus. The `degradation-certificate` verifier, verbatim: "labelling `mode=gfm` 'GitHub's comment renderer' is unverified and my data argues against it — `mode=gfm` emits NO heading anchor while `mode=markdown` emits the full `user-content-` anchor machinery that GitHub comments visibly have." That verdict also downgraded "GitHub is at least three renderers" to OVERSTATED, correcting to "TWO distinct rendering surfaces (the render API and the repo blob/contents HTML), plus one documented `mode` flag on the API" (primary read of `github/rest-api-description`: `POST /markdown` has one `mode` property, `enum: ["markdown", "gfm"]`). Every "destroyed at the review surface" conclusion in S04 §4.5(a) and §4.8 row 1, and the S05 "comment box" cells, inherit an unverified premise.

- **Correction:** Replace " `mode=gfm` is the comment box" with: " `mode=gfm` is a documented `mode` flag on `POST /markdown` , not a verified stand-in for the comment box. The `degradation-certificate` verifier argues against that identification — `mode=gfm` emits no heading anchor while `mode=markdown` emits the `user-content-` anchor machinery GitHub comments visibly have. **What is measured is that the HTML comment is deleted in BOTH API modes**, which is enough for the ruling; the extra step to 'the surface where review actually happens' is `[inference]` and should be labelled as one until someone probes a real comment." Apply the same label to S05's `github-comment` row and to §5.5.1's "passes through the comment box" sentence.

**D31** · S04 §4.8 row 2 ("Any appearance-based or `id`-based identity")

- **Claim:** "Separately, byte offset is **62.12% false** and block index **55.50% false** over 41,642 block-versions. `[measured, docs/engine/PLAN.md §1.1a]` "
- **Problem:** Wrong denominator. `docs/engine/PLAN.md §1.1a` states verbatim, immediately above that table: "**Results (denominator = 32,919 surviving block-versions):**" — byte offset 36.93% correct / 62.12% false / 0.95% safe refusal; block index 44.43% / 55.50% / 0.08%. 41,642 is the block-version count the 99.627% figure is measured over, not the denominator for these two schemes.
- **Correction:** "Separately, byte offset is **62.12% false** and block index **55.50% false** over **32,919 surviving block-versions** (the 41,642 figure is the full block-version count attached to the 99.627% result, not this table's denominator). `[measured, docs/engine/PLAN.md §1.1a]` "

**D32** · S04 §4.8 row 12 ("A declared-vocabulary syntax, registry, or library-import mechanism")

- **Claim:** "Given syntax permitting an arbitrary type name, ~850 independent third-party authors invented **zero** types — they used the five GitHub ships and stopped. ... `[measured, area custom-pointers , headline verdict CONFIRMED]` "
- **Problem:** Presented as settled on the strength of the area's CONFIRMED headline, but this specific claim (custom-pointers C2) was verdicted **OVERSTATED** and corrected by ~47x. Verifier verbatim: "847 is a FILE count being presented as an AUTHOR count ... the corpus spans 663 distinct packages, and only 18 of them use the syntax at all ... with n=74 uses the 95% upper bound on the invention rate is 3.97%; with n=18 authors it is 15.33%, and P(0 inventions | 18 authors, true rate 5%) = 0.397, so a null result here is unsurprising even if invention were common." S06 §6.7 correction #1 carries this correction; S04 contradicts S06 in the published text.
- **Correction:** "Of **663 distinct third-party packages** in Corpus B, **18** used GitHub alert syntax at all; across their **74 uses**, zero types outside GitHub's built-in five appeared. 95% upper bound on the invention rate is **3.97% per use** (15.33% per author, n=18). `[measured, area custom-pointers finding C2 – verdict OVERSTATED as originally framed, corrected by its verifier; the zero-invention result survives at the USE level]` Note the null is weak at n=18: P(0 inventions | 18 authors, true rate 5%) = 0.397."

**D33** · S04 §4.10 C-5 (frontmatter prevalence)

- **Claim:** " `custom-pointers` reports **84.73%** with no stated denominator (84.73% of 1,084 would be 918.5 files). **Governing: 907 of 1,084 = 83.67%**, because the numerator is an AST node count over a hash-pinned file list."
- **Problem:** "No stated denominator" is false. `custom-pointers` states its denominator explicitly: "Excluding the 23 reserved `index.md`/`log.md` filenames, 1,061 non-reserved files: 899 (84.73%) carry frontmatter delimiters."  $899 \div 1,061 = 0.847314 = 84.73\%$  exactly, and the verifier reproduced "1,061 non-reserved (EXACT); 899 carry frontmatter delimiters = 84.73% (EXACT)". The two figures measure different populations (AST `yaml` nodes over all 1,084 files vs delimiter-bearing blocks over 1,061 non-reserved files); neither lacks a denominator. This also contradicts S06 §6.6, which publishes 84.73% as "of the pinned corpus".
- **Correction:** "Two figures, two populations, both exact: the AST reports `yaml=907` nodes over all 1,084 files = **83.67%** `[computed: 907 ÷ 1,084]` ; `custom-pointers` reports **899 of 1,061 non-reserved files (84.73%)** carrying frontmatter delimiters, excluding the 23 reserved `index.md` / `log.md` names per OKF v0.2 §11.3 — verifier-reproduced EXACT. **Governing: cite whichever population you mean and always name it.** Use 907/1,084 for corpus-wide AST statements and 899/1,061 for frontmatter-block statements." Fix S06 §6.6 to read "present in 84.73% of the corpus's **non-reserved** files (899 of 1,061)".

**D34** · S04 §4.10 C-6 (frontmatter blocks that are not valid YAML)

- **Claim:** " `docs/mdmax/PLAN.md §5.5` says **170 of 907** (= 18.74% `[computed]` ); `custom-pointers` says **18.91%** (= 171.5 of 907). Immaterial; **quote 170 of 907**, which names its numerator and denominator."
- **Problem:** The arithmetic reverse-engineering is wrong and the governing ruling is backwards. 18.91% is not "171.5 of 907" — it is exactly  $170 \div 899 = 0.189099 = 18.91\%$ , where 899 is the count of frontmatter-delimiter-bearing non-reserved files. The `custom-pointers` verifier reproduced all three numbers on the first attempt with an independently-written script: "899 carry frontmatter delimiters = 84.73% (EXACT); `js-yaml` parses 729 to a mapping = 68.71% (EXACT); 170 throw (EXACT);  $170/899 = 18.91\%$  (EXACT)." This also contradicts S06 §6.7 and §6.10, which correctly publish "170 of 899".
- **Correction:** " `docs/mdmax/PLAN.md §5.5` says **170 of 907**; `custom-pointers` says **170 of 899 = 18.91%**, where 899 is the number of non-reserved files carrying frontmatter delimiters and 170 is the number that throw under `js-yaml`. **Governing: 170 of 899 = 18.91%** — the verifier re-derived the file classification (1,061 non-reserved), the delimiter count (899), the parse count (729) and the throw count (170) all EXACT against `corpus_id sha256:3a010b16...`, and `gray-matter 4.0.3` throws the exact string, line and column quoted. The 907 in §5.5 is the AST `yaml` -node count over all 1,084 files and is the wrong denominator for a parse-failure rate."

**D35** · S04 §4.1 ("Tool versions, so a reader can reproduce")

- **Claim:** "Every one of those except `marked` is already a dependency of this repository ( `package.json` ), so the matrix is re-runnable from a clean checkout."
- **Problem:** Wrong in both directions, verified against the repository this session. `remark-stringify` is NOT in `package.json` — not in `dependencies` , not in `devDependencies` (it resolves only transitively, at `node_modules/remark-stringify` ). And `marked` , the one package named as the exception, IS resolvable locally at **16.4.2** ( `node -e "require('marked/package.json').version" → 16.4.2` ). Declared deps that ARE present: `remark-parse ^11.0.0`, `remark-gfm ^4.0.1`, `remark-frontmatter ^5.0.0`, `remark-rehype ^11.1.2`, `prettier ^3.8.3`.
- **Correction:** "Declared in `package.json` : `remark-parse ^11.0.0`, `remark-gfm ^4.0.1`, `remark-frontmatter ^5.0.0`, `remark-rehype ^11.1.2`, `prettier ^3.8.3`. **Not declared** but resolvable from the current `node_modules` as transitive dependencies: `remark-stringify` and `marked 16.4.2`. A clean `npm ci` reproduces the first five; add `remark-stringify` and `marked` explicitly before treating the matrix as re-runnable from a clean checkout."

**D36** · S04 §4.2 item 7 and §4.9 (options table, option (iii) commentary)

- **Claim:** §4.2 item 7: "The 99.627% re-anchoring figure is quoted twice in this section ... Every use of it below carries that caveat inline." §4.9: "The anchor is measured at **99.627%**, which is plenty good enough to key a sidecar — so this is a genuine risk, not a rhetorical one."
- **Problem:** The number appears three times (§4.7 A1, §4.9, §4.13 OPEN), not twice, and the §4.9 use carries no inline caveat — breaking the promise §4.2 makes two pages earlier. `docs/mdmax/PLAN.md (v1.0.0 §9, committed at 1bd4dad)` lists "Publishing 99.627% before it is re-derived" as a standing prohibition: "nobody has re-derived it, including every area that quoted it. An adversarial 30-line LCS diff baseline sits at 86.658%."
- **Correction:** §4.2 item 7: change "quoted twice" to "quoted three times". §4.9: append the caveat inline — "The anchor is measured at **99.627%** — a BLOCK figure over 41,642 block-versions that no research area has re-derived, that the current plan forbids publishing until it is re-derived on a corpus this team did not

write, and against which a 30-line LCS diff baseline sits at 86.658% (its own verifier reproducing 85.036%) — which is, if it holds, plenty good enough to key a sidecar."

D37 · S04 §4.7 A4 ("What survives and what to write down")

- **Claim:** "Drop "13.2% are not code at all"; the independent estimate is **4–5%**."
- **Problem:** Presented as a settled independent estimate. The final gate's kill audit re-derived it a third time and explicitly warned against treating it that way: "CAVEAT: the verifier's replacement 4.6% is itself an n=30 point estimate and should carry a CI." The auditor's own seed-42 sample of 30 from the ≥2-word pool yielded **1 unambiguous prose span (3.3%)**, at most 4 counting generously (13.3%) — a range that spans the disputed 13.2%. The auditor also re-derived the pools slightly differently: 76,042 fence-aware spans, single-word 54,946 (72.3%), ≥2-word 21,092 (27.7%) vs the verifier's 21,398 / 27.6%.
- **Correction:** "Drop "13.2% are not code at all". Two independent re-samples put it far lower but neither is precise: the verifier's seeded n=30 implies ≈4.6%; the kill audit's independent seed-42 n=30 yields 3.3% (13.3% counting generously). **The honest statement is: the prose share is unmeasured beyond two n=30 point estimates in the 3–13% range, both well below the claimed 13.2%; it needs a labelled sample with a confidence interval before any number is published.** A4 is unaffected — a role recogniser does not depend on the prose share."

D38 · S05 §5.8.4 (table of per-claim verdicts, LangChain row)

- **Claim:** "LangChain's markdown splitter destroys indentation on **34.61%** of fenced-code lines and deletes the Unicode Tag block | **NOT VERIFIED** — headline only | Label [unverified] until re-derived."
- **Problem:** Both cores were CONFIRMED, not unverified. `cap-model-view` verification: "**CONFIRMED** | C5 core: LangChain's MarkdownHeaderTextSplitter applies `line.strip()` and `filter(str.isprintable)` to every line including inside fences; 34.61% of fenced-code lines lose indentation" — with a primary read at `markdown.py` lines 164/167/180-182 and an independent replay giving `code_block_lines` 200,739, indented 69,483, pct 34.614. And "**CONFIRMED** | C6 core: `str.isprintable` deletes the Unicode Tag block, ZWSP/ZWNJ/ZWJ/WJ/BOM/NBSP/bidi/TAB/ideographic space/soft hyphen, while U+FE0F survives" — full truth table re-derived. What failed was the FRAMING ("nobody says so" — REFUTED, the defect was filed as langchain issue #20823 on 2024-04-24) and the SCOPE (OVERSTATED: only `MarkdownHeaderTextSplitter` strips; `MarkdownTextSplitter` and `ExperimentalMarkdownSyntaxTextSplitter` preserve whitespace).
- **Correction:** Replace the row with: "`MarkdownHeaderTextSplitter` destroys indentation on **34.61%** of fenced-code lines and `str.isprintable` deletes the Unicode Tag block | **CONFIRMED** (both cores independently replayed; 200,739 code-block lines, 69,483 indented, 34.614%) | Quote it, with two riders: **name the class**, because two of the three markdown splitters in the same file preserve whitespace and `ExperimentalMarkdownSyntaxTextSplitter` is the official whitespace-preserving replacement; and drop "nobody says so" — it was reported as langchain issue #20823 on 2024-04-24, with issue #22728 proposing deprecation."

D39 · S05 §5.8.4 (table of per-claim verdicts, .md-twin / block-92 row)

- **Claim:** "`.md` twin vs HTML = **17×** bytes; editing block 92 of `docs/engine/README.md` discards **54%** of the cached prefix | **NOT VERIFIED** — headline only | Label [unverified]."
- **Problem:** Two claims with two different verdicts merged into one row, and both verdicts are wrong as stated. The 17× was **CONFIRMED**: "it reproduces on the page I could match: `code.claude.com/docs/en/memory` HTML is 601,419 bytes, the `.md` twin is 35,388 bytes, ratio 16.99" (with the rider "n=1. State the page"). The 54% was **UNSUPPORTED** — a harder verdict than "not verified": "No evidence line was supplied ... it is not reproducible from anything provided — the block model that defines 'block 92' is the unshipped `explain` pass itself. Per LR#62, a number produced by replaying data through code that has never run in production is a prediction, not a measurement, and 'Measured live' is the wrong label for it."
- **Correction:** Split into two rows. Row A: "`.md` twin vs HTML = **17×** bytes | **CONFIRMED, n=1** | Quote as `code.claude.com/docs/en/memory` : 601,419 B HTML vs 35,388 B `.md` = 16.99×. Sample more pages before generalising." Row B: "editing block 92 of `docs/engine/README.md` discards **54%** of the cached prefix | **UNSUPPORTED** | Do not use. No evidence line, no named script, and 'block 92' is defined by the unshipped `explain` pass — it is [SIMULATED] at best and was labelled 'Measured live'."

D40 · S05 §5.9.2 ("The write-back defect")

- **Claim:** "frontmatter's only wired rendered-editing surface is not byte-faithful."
- **Problem:** The final gate's kill audit killed this exact phrasing: "HEADLINE 'the one WIRED rendered-editing surface' — false as stated. `onToggleTask` is wired end to end from the rendered preview: `EditorPane.tsx:652` passes `onToggleTask={handleToggleTask}` to `MarkdownPreviewPane` (declared :643, forwards at :402) → `Markdown.tsx` → `components.tsx:73` `onToggleTask(newContent)` on checkbox change." S05 also contradicts itself: §5.9.1's write-back allowlist lists the task checkbox as a wired affordance that writes to the `.md` from the rendered view, and §5.9.4 analyses it as such.
- **Correction:** "frontmatter's only NON-SPLICING rendered-editing surface is not byte-faithful. (The task checkbox is the other wired rendered-editing surface — `EditorPane.tsx:652` → `MarkdownPreviewPane` → `Markdown.tsx` → `components.tsx:73` — and it is a splice, subject to the two qualifications in §5.9.4.)"

D41 · S05 §5.7 ("THE ARTIFACT CONTRACT", closing paragraph)

- **Claim:** "Note that `pdf-doc.ts:24` pins `KATEX_VERSION = \"0.17.0\"` for the stylesheet, so **the PDF path is half-pinned and the unpinned half is the renderer.**"
- **Problem:** Wrong line number, and the same line is attributed to two different constants inside one section. Verified in the working tree at HEAD 1bd4dad: `src/modules/export/presentation/pdf-doc.ts:23` is `const KATEX_VERSION = \"0.17.0\";` and `:24` is `const MERMAID_VERSION = \"11\";`. §5.1.2 correctly cites :24 for `MERMAID_VERSION`.
- **Correction:** "Note that `pdf-doc.ts:23` pins `KATEX_VERSION = \"0.17.0\"` for the stylesheet while `:24` sets `MERMAID_VERSION = \"11\"`, so the PDF path is half-pinned and the unpinned half is the renderer."

D42 · S05 §5.5.4 ("Blast radius, measured, and it is zero today")

- **Claim:** "across a 4,823-file / 91,394,509-byte corpus, **0 of 24,063 raw-HTML AST nodes carry** `style=` or `hidden=`"
- **Problem:** Conflates two different denominators from the same source. The `integrity` verifier states: "for Tier-A rules D1 and D2 the corpus denominator is exactly ZERO (0 of 24,063 raw-HTML AST nodes across **4,800 files** carry `style=` or `hidden=`). 4,823 / 91,394,509 B is the full integrity corpus; 4,800 is the file count the AST scan covered.
- **Correction:** "across the integrity corpus (4,823 files / 91,394,509 bytes), **0 of 24,063 raw-HTML AST nodes across the 4,800 files the AST scan reached** carry `style=` or `hidden=` [measured, capability-run/integrity verifier] — so rules D1 and D2 were never exercised by the corpus at all, which is a zero blast radius AND an untested rule, not a clean bill of health."

D43 · S05 §5.9.3 ("The table writer")

- **Claim:** "45 of 335 table-bearing files dirtied = 13.4%, touching 199 lines"
- **Problem:** 335 is not the table-bearing file count. The underlying evidence record is verbatim `{files:1084, filesWithTables:338, tables:1240, escPipeTables:7, padTables:89, noLeadPipe:25, editsRun:335, filesDirty:45, totalLinesTouched:199}` — 338 files contain tables; 335 is the number of edit runs performed. (The error is inherited verbatim from the researcher's prose, but this section is a reference table and should not carry it forward.)
- **Correction:** "a NO-OP cell write dirties **45 of the 335 files the edit was run over** (13.4%), touching 199 lines; 338 of 1,084 files contain tables. `[measured, corpus_id sha256:3a010b16...: filesWithTables 338, editsRun 335, filesDirty 45, totalLinesTouched 199]`"

D44 · S05 §5.5.3 ("Our own renderer's formatting policy")

- **Claim:** "The research says **62 allowed elements and 13 denied; I count 61 and 12**. The integrity area's verifier also counted 61. **My measurement governs.**"
- **Problem:** The ruling is correct — I independently parsed `src/modules/preview/presentation/markdown/html-policy.ts` at HEAD `1bd4dad` and counted exactly 12 in `DISALLOWED_RAW_HTML_ELEMENTS` and 61 in `SAFE_HTML_ELEMENTS`, and confirmed `bdi / bdo / wbr / svg` are all absent. But a THIRD figure circulates in the same corpus and is not named, so a reader who meets it will not know which governs: the rendering-frontier verifier's ND1 says KaTeX emits elements "none of which are in that **74-element set**".
- **Correction:** Append: "A third figure circulates — the rendering-frontier verifier's ND1 refers to a '74-element set'. It reproduces neither count. Three independent parses of the file (the integrity verifier's, mine, and a re-parse at HEAD `1bd4dad`) all return **61 safe / 12 denied**; treat 62, 13 and 74 as unreproducible."

D45 · S06 §6.7 ("And our own corpus shows the same field breaking its own parser", closing paragraph)

- **Claim:** "Both methods find **10** distinct `type` values, not 12 — the regex distorts **counts** (`synthesis` 195 → 365), not the distinct-value count."
- **Problem:** This is the one number the final gate's kill audit explicitly instructs nobody to repeat. Kill audit, verbatim on custom-pointers C7: "UPHELD ON THESIS, WRONG NUMBER. ... the distinct-value count is 12 under all six regex × scope combinations I ran (anchored/loose × frontmatter-body/whole-file/first-50) AND under the eemeli-YAML ground truth (12 distinct, `synthesis` 361). The verifier's '10' is unreproducible. The kill still stands because its thesis — the regex distorts COUNTS, not the distinct-value count — is exactly right ... **Do not propagate the 10.**" S06 §6.11 item 9 already lists "the 10-vs-12 distinct values" as needing independent re-derivation, so the section contradicts itself.
- **Correction:** "Both methods find the **same** distinct-value count — the regex distorts **counts**, not distinct values (`synthesis` 195 → 365). The verifier reported that count as 10; the final gate's kill audit could not reproduce 10 under any of six regex × scope combinations and found **12** under all six and under the eemeli-YAML ground truth (12 distinct, `synthesis` 361), and instructs that the 10 not be propagated. **Quote the thesis, not either number, until it is re-derived.**"

D46 · S06 §6.6 ("The carrier, ranked by measured round-trip × render × occupancy", frontmatter row)

- **Claim:** "frontmatter | required anyway | **visible in 23 of 24 renderers** | present in **84.73%** of the pinned corpus | acceptable for a single document-level scalar"
- **Problem:** Quotes an OVERSTATED claim without its caveat, and the load-bearing half was REFUTED. `degradation-certificate` Claim 3 verdict: the 23/24 headline reproduces, but "(a) DISTINCT\_RAW = 8, not 6. (b) The shape claim is false: kramdown ... → a PARAGRAPH, not an h2 ... (c) The interpretive sentence is wrong at the layer the design itself declares. Primary read, `jekyll/jekyll lib/jekyll/convertible.rb read_yaml` ... Jekyll strips frontmatter BEFORE the converter runs ... Hugo, Docusaurus, Astro, Eleventy and GitHub's own blob viewer do the same upstream consumption." And: "The rate is also not a statistic: the denominator is author-chosen and the configs are not independent (3 markdown-it presets, 4 remark configs, 3 cmark-gfm flags)." Separately, "84.73% of the pinned corpus" uses the wrong population (see the §4.10 C-5 defect above): it is 899 of 1,061 **non-reserved** files.
- **Correction:** "frontmatter | required anyway | **visible in 23 of 24 bare markdown ENGINE configurations, 8 distinct outputs — but the deployed TARGETS that matter (GitHub Pages via Jekyll, Hugo, Docusaurus, Astro, Eleventy, GitHub blob) all strip frontmatter above the engine** `[measured; verdict OVERSTATED as originally framed; Jekyll's convertible.rb / document.rb read_primary]`. The 23/24 rate is not a statistic — the denominator is author-chosen and the configs are not independent. | present in **84.73% of non-reserved files (899 of 1,061)** | acceptable for a single document-level scalar"

D47 · S06 §6.4, Correction 3 (the vim `modelinestrict allowlist`)

- **Claim:** "The **allowlist** is **21 options, not 24**. ... The verifier extracted and enumerated all of them: `autoindent, cindent, commentstring, expandtab, filetype, foldcolumn, foldenable, foldmarker, foldmethod, modifiable, readonly, ...`" and "`modifiable` and `readonly` are buffer state."
- **Problem:** The final gate's kill audit re-derived the source and found a specific blemish S06 does not carry: "raw.githubusercontent.com/ciaranm/securemodelines/master/plugin/securemodelines.vim lines 15-30 = 39 strings, which is 21 long-form option names + 18 abbreviations. 24 matches no counting rule. ... **BLEMISH: 'modifiable' is NOT in that allowlist; only 'readonly' is.** Verdict survives on `filetype + readonly`." Note also the enumerated source is a third-party plugin (`ciaranm/securemodelines`), not vim's own `runtime/doc/options.txt`.
- **Correction:** "The **allowlist** is **21 options, not 24** — 39 strings at `ciaranm/securemodelines/plugin/securemodelines.vim` lines 15-30, i.e. 21 long-form names plus 18 abbreviations; 24 matches no counting rule. The kill audit flags one item in the reproduced list: `modifiable` **is not in that allowlist; only `readonly` is.** The 'not all presentation-only' correction survives on `filetype` (vim `runtime/doc/options.txt` :3896, verbatim: 'When this option is set, the FileType autocommand event is triggered') and `readonly`."

D48 · S06 §6.2.3 ("A contradiction between two of our own documents")

- **Claim:** " `HANDOFF-mdmax-markdown-engine-2026-08-01.md:256` states: "Everything markdown-adjacent — Quarto, Sphinx, MkDocs, Pandoc, R Markdown — deliberately **refused** document-declared registry resolution.""
- **Problem:** Off-by-one line citation. Verified in the file: line **255** carries that sentence (as the tail of the LaTeX/Typst paragraph in §7.1); line 256 is blank. The quoted text itself is accurate.
- **Correction:** Change `:256` to `:255`.

D49 · S06 §6.7 ("The three corrections that must travel with the number", correction 1)

- **Claim:** "Corpus B's 847 is a **file** count presented as an **author** count. Resolved to owning packages, the corpus spans **663 distinct packages**, and **only 18 of them used the syntax at all**. The inferential base is **18 authors / 74 uses.**"
- **Problem:** Faithful to the verifier, but a third independent pass in the final gate's kill audit — which UPHELD the kill — does not reproduce three of the four numbers: "Independent walk of `frontmatter/node_modules`: 1,346 \*.md across **664** distinct packages (verifier said 663). Fence-aware GFM alerts = **80** uses in 24 files across **19 PACKAGES**; type histogram NOTE 49 / WARNING 13 / IMPORTANT 10 / TIP 4 / CAUTION 4 = exactly 5 types, zero invented." Only the binomial reproduces to the digit (Clopper-Pearson one-sided 95% upper for 0/18 = 15.33%; 0.95^18 = 0.3972). This is exactly the number-substituting class S06 §6.11 item 9 says to re-derive.

- **Correction:** "Corpus B's 847 is a **file** count presented as an **author** count. Two independent resolutions to owning packages disagree slightly: the verifier gives 663 packages / 18 using the syntax / 74 uses; the final gate's kill audit gives 664 packages / 19 using it / 80 uses across 24 files (histogram NOTE 49, WARNING 13, IMPORTANT 10, TIP 4, CAUTION 4). **Both give exactly 5 types and zero inventions**, and the binomial reproduces to the digit on either base. Quote it as 'fewer than 20 independent packages, ~74–80 uses, 5 types, 0 invented' rather than as a precise pair."

**D50** · Cross-section: S04 §4.1 / S05 §5.0 vs S06 §6.11 item 8 (the corpus\_id)

- **Claim:** S04 §4.1 and S05 §5.0 present `corpus_id sha256:3a010b1649899795d79274fc528dbece97fdabf4ff0f81cc02ab619c048c51a4` as the section's universal citation key, S04 adding that "Any figure about local files ... that does not cite `sha256:3a010b16...` should be treated as unreproducible."
- **Problem:** Neither section discloses that the id itself is not re-derivable, while S06 §6.11 item 8 does — an inconsistency the reader will hit. Two independent confirmations exist: `custom-pointers` ("I attempted 30 serializations of that description ... none reproduces the declared digest") and the `rendering-frontier` verifier ("Two obvious constructions over `root/path\tsha256` lines gave `sha256:0cbbdbd4...` (sorted) and `sha256:2284b06e...` (as-is), neither matching `3a010b16...`"). I confirmed independently that no derivation script is tracked anywhere in the repository. The FILE LIST is sound — 1,084 present, 0 missing, 0 hash mismatches, reproduced by at least three parties.
- **Correction:** Add to S04 §4.1's corpus block and S05 §5.0: "**Caveat on the id itself.** The 1,084-entry file list and every sha256 in it reproduce exactly (verified=1084 sha\_mismatch=0 missing=0, replicated by three independent parties). The `corpus_id` DIGEST does not: 30 serializations of the documented recipe were attempted by one area and two more by another, producing `sha256:0cbbdbd4...` and `sha256:2284b06e...`, and no derivation script is tracked in the repository. **Until a ~15-line derivation script and its captured output are committed beside the manifest, `sha256:3a010b16...` is a label, not a checksum** — what actually anchors every number below is the file-level hash reproduction."

**D51** · Cross-section: S04 §4.1 and §4.10 / S05 §5.13 vs S06 §6.11 item 9 (verifier false-kill rate)

- **Claim:** S04 (twice) and S05 §5.13 state the verifier false-kill rate as "**0–19%**"; S06 §6.11 item 9 states "**5/29 = 17.24%**" with the source `kill_audit.note`.
- **Problem:** Three statements of one quantity, two of them unsourced and inconsistent with the primary artifact. `wf-final-gate-2026-08-01.result.json` `kill_audit` carries "`false_kill_rate`": "**5/29 = 17.24%**" and its note adds: "If the true verdict there is OVERTURNED, the false-kill rate is 6/29 = 20.7%. The published 17.24% is therefore a floor-to-central estimate, not a ceiling." The "0–19%" range appears in no artifact I could locate.
- **Correction:** Harmonise all three sections to: "The verification layer's measured false-kill rate is **5 of 29 sampled kills = 17.24%** [`wf-final-gate-2026-08-01.result.json`, `kill_audit.false_kill_rate`], and the audit states this is a floor-to-central estimate, not a ceiling — one upheld verdict rests on inference and would take it to 6/29 = 20.7%. The pattern is sharp and should govern how each kill is used: kills that **quote a primary source** were **16 of 16 correct**; kills that **re-derive and substitute a number** were **wrong 5 of 13 (38%)**. Accept a source-quoting kill; independently re-derive a number-substituting one."

**D52** · S07 §7.1.3 (last row of the corrections table)

- **Claim:** "[measured] 2026-08-01: package.json is 3,076 bytes, git status --short package.json returns empty, and all 16 scripts plus 43 dependencies and 22 devDependencies are present."
- **Problem:** Three of the four numbers are wrong. Measured 2026-08-01 via `python3 -c "import json; d=json.load(open('package.json'))": scripts = 18, dependencies = 47, devDependencies = 18. Bytes (3,076) and the empty git status reproduce exactly. This also contradicts S09 §9.6, which twice says "18 scripts" and lists all 18 correctly — S09 governs.`
- **Correction:** Replace with: "...and all **18** scripts plus **47** dependencies and **18** devDependencies are present." The 18 scripts are: arch, budget, build, dev, format, format:check, lint, start, tauri, tauri:build, tauri:build:mac-arm, tauri:build:mac-intel, tauri:build:mac-universal, tauri:dev, test, test:watch, typecheck, verify.

**D53** · S07 §7.7.4 ("The measurement") and S08 §8.8 (defensibility table, "Splice-only writing" row)

- **Claim:** S07: "mdast `node.position.offset` splices byte-perfectly on 1,080/1,080 files". S08: "The whole writer is four lines and is byte-identical on **1,080 of 1,080** parseable files." Both presented as settled measurements.
- **Problem:** This is the `against` area's claim C5, whose verdict is **OVERSTATED** and whose evidence was explicitly KILLED: "the identity round-trip is a tautology that returns true for deliberately wrong offsets, absurd offsets and a degenerate range. It cannot go red, so it certifies nothing (LR#68)." The verifier reran it with offsets (2,7) and (0,1) and still got green, because `src.slice(0,s)+src.slice(s,e)+src.slice(e) === src` holds for any  $s \leq e$ . Second, the same verifier measured that a naive four-line `spliceBlock(src, node, newText)` with non-syntax-preserving replacement text **disturbs top-level block structure in 994 of 1,080 files**. S08 additionally uses it to grade splice-only as "~1 day / Nothing" hard-to-get — an estimate built on the killed evidence. S07 also presents the whole `against` headline ("four of six proposed capabilities had working substitutes executed and measured") as "The measurement" when that area's headline\_verdict is OVERSTATED and all four substitute claims (C3 LCS, C5 splice, C7 yaml, C10 OKF) were individually graded OVERSTATED with their load-bearing sentences killed.
- **Correction:** Replace both with: "The identity splice reproduces 1,080/1,080 byte-identical [measured, `against` C5, corpus\_id sha256:3a010b16...] — but that test is a tautology that cannot go red (verifier, verdict OVERSTATED, LR#68), so it certifies the arithmetic and not the offsets. A safe splice must reparse and assert block-count plus sibling-text invariance: the naive four-line version disturbs top-level block structure in **994 of 1,080** files under non-syntax-preserving replacement. Budget it as engine work, not as a four-line function." In S07 §7.7.4, prefix the four-substitutes sentence with "the `against` area (headline\_verdict **OVERSTATED**) argued that..." and note that all four substitute claims were narrowed by their verifier.

**D54** · S08 §8.5, item 7 ("The review loop — the USP"), evidence table

- **Claim:** "measured over 2,332 simulated comment selections ... | byte-offset range 65.31% correct / 34.69% wrong-or-gone | global quote match 85.12% / 12.18% gone / **2.70%** ambiguous | **block-scoped quote match 87.48% / 8.45% + 3.99% / 0.09%** | Block-scoping cuts silent mis-anchoring **30x** ... of the 197 refusals, 196 were blocks genuinely deleted or rewritten, and exactly **1 of 2,332 (0.04%)** was a resolver failure" — all tagged [measured].
- **Problem:** This is the capability-run `collab-primitives` headline, and its own adversarial verifier graded it **REFUTED** on five independent grounds: (1) "2.70%, 0.09% and 30x appear in NO claim in the evidence list" — orphaned from the evidence; (2) it does not reproduce — the verifier's replication on the same two corpora (400 modified-.md revision pairs, 2,400 selections, seed 20260801) gives unscoped 2.58% → block-scoped 0.75%, a **3.44x** reduction, not 30x; (3) the rate rests on ~2 raw events (0.09% × 2,332 = 2.1), Wilson 95% CI [0.024%, 0.312%]; (4) the ratio is quoted without its cost — block-scoping raised refusal/lost from 11.79% to 22.88% (+11.09pp) and dropped clean placement from 85.62% to 76.38%; (5) the byte-offset row is 62.83% in the verifier's replication, not 65.31%. This also directly contradicts S07 §7.4.1 (S3), which correctly says "Use the verification, not the headline" and prints the verifier's numbers. Two sections of one document give different numbers for the same experiment.
- **Correction:** Replace the table with S07 §7.4.1's version and state the verdict: "[measured, `collab-primitives`, headline **REFUTED** by its own verifier — use the verification] Naive byte-offset anchor correct after one real revision: **62.83%** (verifier) vs 65.31% (original), n=2,400 selections over **400** revision pairs. Silent mis-anchoring, unscoped → block-scoped: **2.58% → 0.75%**, a **3.44x reduction**, not 30x. Refusal rate, unscoped → block-scoped: **11.79% → 22.88%** — the cost that doubles alongside the benefit. The 0.09% figure rests on roughly two raw events (Wilson 95% CI [0.024%, 0.312%]) and must not be quoted." Also drop "2,332 real revision pairs" framing entirely — there were 400 revision pairs and 2,332 simulated selections.

D55 · S08 §8.4 ("A naming hazard you will hit within an hour")

- **Claim:** "docs/Frontmatter-Product-Plan.md mislabels these against each other in seven consecutive rows — line 116 uses taxonomy IDs correctly while lines 125–142 use rank numbers, so the offline and collaboration citations are swapped inside a single document. [measured]"
- **Problem:** This is the killed version of the claim. Its verifier graded it OVERSTATED and I re-verified the correction directly: docs/research/frontmatter-pain-taxonomy.md lines 255–256 are its own ranking table — "I 6 | T7 Collaboration impossible in markdown |" and "I 7 | T6 Offline failures |" — so the taxonomy is itself the source of the rank→ID divergence. docs/Frontmatter-Product-Plan.md §1 (lines 35–36) uses a bare # rank column with no T-prefix. Pillar 5 (Collaboration) rows all tag T6 = rank 6 = collaboration; Pillar 4 (offline/mobile) rows tag T7 = rank 7 = offline; and line 116 is "CRDT merge (Yjs/Loro) ... two-device offline edits auto-merge" tagged T1, T7 = rank 7 = offline. Every citation is consistent under rank-numbering. The claim picks the one reading of line 116 that manufactures an internal contradiction.
- **Correction:** Replace with: "The product plan uniformly uses the SHARED RANK numbers (6 = collaboration, 7 = offline) inherited from the taxonomy's own ranking table at frontmatter-pain-taxonomy.md:246–259, but styles them with a 'T' prefix that collides with the taxonomy's section IDs. It is **one consistent document using the wrong sigil, not a document that contradicts itself** [collab-primitives verification, claim 1, verdict OVERSTATED]. Fix by re-prefixing the pillar-table citations to true T-IDs (Pillar 5 T6→T7, Pillar 4 T7→T6, line 116 T7→T6) — or by dropping the T and citing ranks." Keep the rest of §8.4's table, which uses RANK correctly.

D56 · S09 §9.0 ("Warning one")

- **Claim:** "Of ten verified claims in aios-transfer, **four are CONFIRMED, four are OVERSTATED, two are REFUTED** [measured, counted from per\_area[].verification.verdicts]. The two REFUTED claims are the two that most flattered the idea of importing AIOS wholesale."
- **Problem:** The count is wrong. Counted directly from wf-final-gate-2026-08-01.result.json → per\_area[area='aios-transfer'].verification.verdicts: **CONFIRMED 4, OVERSTATED 3, REFUTED 3**. The three REFUTED claims are index 3 ("the AIOS verify ladder transfers to the MDMAX build with zero modification"), index 5 ("CORRECTION TO THE RECORD: the documented environment remedy is backwards"), and index 8 ("markdown is the file type agents fail to edit MOST"). All three are in fact handled correctly elsewhere in the section (§9.6, §9.7.3, §9.0 respectively), so only the tally sentence is defective.
- **Correction:** "Of ten verified claims in aios-transfer, **four are CONFIRMED, three are OVERSTATED, three are REFUTED**. The three REFUTED claims are the ones that most flattered the idea of importing AIOS wholesale — the verify-ladder transfer (§9.6), the run-from-a-fixed-directory remedy (§9.7.3), and the agentic-markdown measurement (below). All three are corrected in place."

D57 · S07 §7.3.2 (competitor matrix, Slite row)

- **Claim:** "Slite does not appear anywhere in the research corpus, the 20-app competitor matrix, or any research area. This row is a hole. It must be researched before the matrix is used externally."
- **Problem:** Refuted by a direct search. grep -ril slite docs/ returns docs/research/frontmatter-r2-raw-corpus.json, which contains Slite twice, including a price: "Notion Plus \$10/user/mo, Business \$20/user/mo (notion.com/pricing); **Slite \$10–\$20/user/mo with no free plan, only a 14-day trial**", and a positioning line: "Outline/Slite/Docmost/BookStack among the self-host-curious". So the price column and a rough positioning are sourced; only the R1–R6 review-loop columns are unresearched. (Related, same row-group: S07 attributes the Confluence quote "Confluence per-seat pricing is the incumbent budget line (exact 2026 price unverified)" to "the segment research"; it is at docs/research/frontmatter-r2-raw-corpus.json:699, not frontmatter-segment-strategy.md.)
- **Correction:** Replace the Slite note with: "Slite is absent from the 20-app competitor gapmap and from all 16 research areas, but it IS in the round-2 demand corpus: **\$10–\$20/user/mo, no free plan, 14-day trial** [secondary, docs/research/frontmatter-r2-raw-corpus.json], positioned there alongside Outline/Docmost/BookStack for the self-host-curious. The price column is sourced; R1–R6 are unresearched and must be filled before external use." And re-cite the Confluence quote to docs/research/frontmatter-r2-raw-corpus.json:699.

D58 · S07 §7.2.2 (editor table, "Wikilinks, transclusion, callouts..." row)

- **Claim:** "19 preview tests including wikilink.test.ts, embeds.test.ts, callout.test.ts, html-policy-xss.test.ts."
- **Problem:** 19 reproduces under neither counting rule. ls test/preview/ | wc -l = **13 files**; grep -c -E '\b(it|test)\(' test/preview/\* sums to **113 test cases**. The four named files all exist. The 13 files are: callout, embedded-note, embeds, frontmatter, html-policy-xss, html-policy, markdown-img-rewrite, markdown, outline, right-pane, table-edit, wikilink-case-insensitive, wikilink.
- **Correction:** "**13** preview test files / **113** test cases, including wikilink.test.ts, embeds.test.ts, callout.test.ts, html-policy-xss.test.ts [measured 2026-08-01, ls test/preview/ and grep -c -E '\b(it|test)\(' ]."

D59 · S07 §7.2.2 (editor table, "Project tree" row)

- **Claim:** "src/modules/vault/presentation/FileTree.tsx plus a nine-file file-tree/ directory: TreeItem.tsx, ContextMenu.tsx, InlineDialog.tsx, ConfirmDialog.tsx, FileTreeActions.tsx, Toast.tsx, path-utils.ts, types.ts."
- **Problem:** Self-inconsistent and wrong: the sentence says nine and then lists eight. ls -1 src/modules/vault/presentation/file-tree/ | wc -l = **8**, and the eight names are exactly the eight listed.
- **Correction:** "...plus an **eight-file** file-tree/ directory: ConfirmDialog.tsx, ContextMenu.tsx, FileTreeActions.tsx, InlineDialog.tsx, Toast.tsx, TreeItem.tsx, path-utils.ts, types.ts."

D60 · S07 §7.2.6 ("The test suite currently proves nothing")

- **Claim:** "vitest.config.ts declares **no exclude key at all** (verified: the file is 16 lines and sets only environment, setupFiles and a @ alias)"
- **Problem:** wc -l vitest.config.ts = **14**, not 16. Everything else in the paragraph reproduces exactly: I ran npx vitest list --filesOnly → 247 total, 164 under .claude/worktrees/, 83 under test/ (164/247 = 66.40%), and the config sets only environment: "node", setupFiles: [".test/setup.ts"] and the @ alias.
- **Correction:** "...(verified: the file is **14** lines and sets only environment, setupFiles and a @ alias)."

D61 · S09 §9.2 ("The measured relationship", the six-row byte table and the 8.3:1 ratio)

- **Claim:** ".md (re-measured, **all**) 18,768,046 / 450 files ... .md (re-measured, **excluding rawl/**) 1,978,406 / 168 files ... AIOS's own markdown is **1,978,406 bytes against 16,465,646 bytes of JSONL — a ratio of 8.3 : 1** [measured]."
- **Problem:** The non-rawl markdown figures do not reproduce and the gap is not append-growth. Measured 2026-08-01 with find ~/.sgnk -name '\*.md' -type f: **479 files / 19,935,834 bytes** (S09: 450 / 18,768,046). ~/.sgnk/rawl/ reproduces **exactly** (282 files / 16,789,640 bytes), so the discrepancy is entirely outside rawl: 479–282 = **197 files**, 19,935,834–16,789,640 = **3,146,194 bytes** (S09: 168 / 1,978,406), giving a ratio of **5.2 : 1**, not 8.3 : 1. Only 2 files / 8,225 bytes are newer than 2026-08-01T00:00:00, so drift does not explain a 1.17 MB / 29-file gap; the largest unexplained block is ~/.sgnk/claude-context-menu/ (32 files,

1,186,454 bytes), which S09's rule appears to have excluded without saying so. JSONL reproduces within 0.02% (16,469,675 vs 16,465,646 — genuine append-growth on a trace log). The section's conclusion (JSONL dominates AIOS state) survives at 5.2:1.

- **Correction:** Re-measure and state the exclusion rule explicitly: ".md all: **19,935,834 bytes / 479 files**; ~/sgnk/rawl/ (web-scraper cache, not AIOS state): 16,789,640 / 282; all other .md : **3,146,194 bytes / 197 files** — against **16,469,675 bytes of JSONL / 275 files**, a ratio of **5.2 : 1** in favour of JSONL [measured 2026-08-01, find ~/sgnk -name '\*.md' -type f -not -path './rawl/\*' ]. The brief's canonical 8.0:1 and this 5.2:1 differ because the exclusion rule differs; both support the same conclusion, and the ratio must be quoted with its rule."

**D62** · S07 §7.5, N4 ("One correctness gate that is easy to miss")

- **Claim:** " [measured] HTML comments appear inside 0.27% of blocks (911 of 337,410 blocks over 5,916 .md files across both corpora)"
- **Problem:** This figure carries no corpus\_id and is explicitly NOT the pinned corpus — 5,916 files against the pinned 1,084. Under the plan's own rule (§7.0: "Every corpus figure in this section refers to corpus\_id sha256:3a010b16...") and the programme's seven-irreproducible-counts hazard, a local-file number on an unnamed 5,916-file selection cannot be tagged [measured] without a caveat. The source (capability-run verification) states the figures exactly and adds a point S07 drops: changing normalize() "changes the identity function under which 99.627% correct / 0.050% false / 0.323% refusal was measured".
- **Correction:** "HTML comments appear inside **0.27% of blocks (911 of 337,410 blocks over 5,916 .md files)** — but note the denominator: that selection is **not the pinned corpus** (1,084 files) and carries **no corpus\_id**, so treat the rate as indicative and re-measure on sha256:3a010b16... before it gates anything. The sharper consequence, from the same source: because §1.1a defines normalize() as 'NFC + collapse whitespace + lowercase', changing it changes the identity function under which the 99.627% / 0.050% / 0.323% result was measured — which is why P7 freezes and versions it first."

**D63** · S07 §7.7.1 ("The measurement" table) and S08 §8.6.2 (second bullet)

- **Claim:** Both quote, as settled [measured] : "10,097 wikilinks ... 72.4% resolving to no file ... 58.9% (638/1,084) of files have zero resolvable outbound links ... 54.6% (592/1,084) have zero backlinks ... p90 out-degree 5."
- **Problem:** Two areas of the same research run measured the same pinned corpus and disagree, and neither section names the conflict. The quoted figures are product-gap NEGATIVE 1 (headline\_verdict CONFIRMED). The the-era thesis re-measured "over all 1,084 files (re-hashed, 0 missing, 0 mismatches, corpus\_id re-derived and MATCHING)" and reports: [ [Name]] resolves **2,754/9,962 = 27.65%** (i.e. 9,962 wikilinks, not 10,097); **597 files (55.1%)** have zero resolved outbound edges (not 638/58.9%); **529 (48.8%)** have zero inbound (not 592/54.6%); plus two facts neither section carries — **502 files (46.3%) are fully isolated** and the graph splits into **511 connected components**, the largest holding 40.1% of the corpus. Median out-degree 0, p90 5 and the ~72.4% unresolved rate agree across both. Per the writing standard's rule 8, a contradiction between two sources must be named and one declared to govern.
- **Correction:** Add to both sections: "**Two measurements of the same pinned corpus disagree and both are in the record.** product-gap (CONFIRMED) reports 10,097 wikilinks, 638/1,084 (58.9%) zero-outbound and 592/1,084 (54.6%) zero-backlinks. the-era, re-hashing all 1,084 files with the corpus\_id re-derived and matching, reports **9,962** wikilinks (2,754 resolved = 27.65%), **597 (55.1%)** zero-outbound and **529 (48.8%)** zero-inbound, and adds **502 files (46.3%) fully isolated** across **511 connected components**. The two agree on median out-degree 0, p90 5 and a ~72% unresolved rate — which is the load-bearing part. the-era's figures govern for file-level counts (it states its re-hash and corpus\_id re-derivation); the discrepancy is a link-resolution rule difference and must be resolved before either ratio is published."

**D64** · S08 §8.10.4 item 4 and §8.8 (defensibility table, "Budgeted projection" row)

- **Claim:** "points do not predict adoption: llms.txt scored **206** points and is now in 9,640 GitHub code-search hits"; and "**Budgeted projection** ... Taken by llms.txt in September 2024: AnswerDotAI/llms-txt, 2,541 stars, created 2024-09-01, mentioned in 1,659 repositories. | taken | **TAKEN**"
- **Problem:** Both uses present llms.txt as an adoption success. The same research run contains the direct countervailing measurement, which neither section names: the-era's 215-repo census (2026-08-01) puts llms.txt at **1/215 = 0.47%** — "llms.txt is measurably dead in the population that matters" — against CONTRIBUTING.md, a 15-year-old convention, at 106/215 = 49.30%, and notes AnswerDotAI/llms-txt has "~100 commits total across 23 months". A 0.47% penetration rate cuts directly against grading the budgeted-projection slot "TAKEN", and against using llms.txt as the example of adoption outrunning HN points. S08 does correctly carry the code-search caveat ("ordinal only; GitHub code search indexes a subset and caps estimates").
- **Correction:** In §8.10.4: "... llms.txt scored **206** points and is in 9,640 GitHub code-search hits [read as ordinal only] — though the same research run measures its real penetration at **1 of the 215 most-starred repos (0.47%)** and calls it 'measurably dead in the population that matters', so it is an argument that points don't predict reach, not that llms.txt won." In §8.8: change the verdict to "**TAKEN AS A NAME, NOT AS ADOPTION** — the slot has a published convention with 2,541 stars and 1,659 mentioning repos, but 0.47% penetration in the top-215 census. Do not enter the slot; do not assume the incumbent is entrenched either."

**D65** · S08 §8.9.1 ("The base rate, given honestly")

- **Claim:** "Of the free artifacts checked, **2 of 9** produced an organic commercial entity for the giver ... **No commercial entity:** prettier, DavidAnson/markdownlint, editorconfig, semver, commonmark/commonmark-spec, repomix, AnswerDotAI/llms-txt."
- **Problem:** The ratio is quoted correctly from the source, but the section's own enumeration cannot reconstruct it. The source's nine are: "mermaid, **Excalidraw (commercial)**, then prettier, DavidAnson/markdownlint, editorconfig, semver, commonmark-spec, repomix, llms-txt (not)." S08 names only **one** of the two commercial outcomes (mermaid) — Excalidraw is missing entirely — and it lists DavidAnson/markdownlint in the no-commercial-entity list while also giving markdownlint its own bullet above ("Zero dollars"), so the section enumerates 8 distinct artifacts and 1 commercial outcome against a stated 2-of-9.
- **Correction:** Add Excalidraw as the second commercial outcome and de-duplicate markdownlint: "**2 of 9:** mermaid (npm 2014-12-02 → Mermaid-Chart org 2022-11-07, 95 months) and **Excalidraw. No commercial entity (7):** prettier, DavidAnson/markdownlint, editorconfig, semver, commonmark/commonmark-spec, repomix, AnswerDotAI/llms-txt." Keep the markdownlint bullet as the closest-in-kind case study, and note it is one of the seven rather than a tenth item.

**D66** · S09 §9.7.2 (bullet list under "the measurement debt") and §9.10(c)

- **Claim:** "**188 of 409 registry items grep source rather than execute it** — the largest block of unverified surface in the system." and "It does not reduce the 188 PROXY items."
- **Problem:** Internally inconsistent with the section's own call-out three paragraphs earlier, which establishes — correctly, and I verified the source — that "The registry file governs. The published headline does not," and that 188 is a **union** over result ∪ result\_prior that "sums to 542 across 409 items". The registry's result field alone gives PROXY = **124**; result\_prior gives PROXY = **64**. Quoting 188 twice as a plain count of registry items repeats exactly the shape the section just diagnosed. (Separately, the source is master-handoff **§8.5 "Test debt"** at line 334, not §8.4 — §8.4 is where the Cohen's-kappa item lives.)
- **Correction:** "**188 registry items grep source rather than execute it** — but note this is the union figure diagnosed above: the registry's result field alone carries PROXY 124 and result\_prior carries 64, and the union double-counts every re-graded item. Read it as 'up to 188, of which 124 are current' [primary, ~/claude/aios-site/testplan/test-registry.json, 409 items; handoff §8.5:334]." Apply the same qualifier in §9.10(c), and correct the section reference from §8.4 to §8.5.

**D67** · S07 §7.5 N8, §7.8 (minute 8); S08 §8.3.2 and §8.6.7

- **Claim:** "the demand corpus's **number-one feature request** is '*Always-visible sync status indicator (what synced, what is pending, when last synced)*'" / "Request #1 is: ..." / "the corpus names as its most concrete trust ask".
- **Problem:** An ordinal position is being presented as a ranking. Verified: that string is the **first record** in `docs/research/frontmatter-raw-corpus.json`'s first slice's `feature_requests` array. The corpus contains **no ranking of feature requests** — the taxonomy ranks the 14 pain *themes*, not the 89 requests. `docs/Frontmatter-Product-Plan.md` uses the defensible phrasing ("the corpus's most concrete trust ask"). Note the surrounding keyword tallies DO reproduce exactly, to the digit: over the 89 requests' `feature + why` fields — sync 62, offline 18, export 17, collab 15, pricing/subscription 14, mobile 13, wysiwyg 7, real-time 7, suggest 5, multiplayer 3, presence 2, vim 1, graph 0.
- **Correction:** Replace "the #1 / number-one feature request" with: "the **first-listed** feature request in the corpus, and the one the product plan calls 'the corpus's most concrete trust ask' — the corpus carries no ranking of its 89 requests, but `sync` leads every keyword tally at 62 [measured, `docs/research/frontmatter-raw-corpus.json`, `feature + why` fields only]."

#### D68 · S09 §9.5, Gate 1; S09 §9.1

- **Claim:** Gate 1: "The theorem is Foster et al., TOPLAS 2007, Lemma 3.9 ... [primary, `docs/engine/PLAN.md:160–163` ]". §9.1: "Note the discrepancy with the master handoff, which publishes '27 gate scripts' in its §7 substrate line — that count was taken at 04:20 IST and the directory was written again at 05:35 IST the same day."
- **Problem:** Two low-severity precision issues. (a) The Foster citation is off: `docs/engine/PLAN.md` line 161 is the "### 1.2 Splice-only writing" header and the Foster sentences run **163–165**; the measurements S09 cites as 165–170 are at **167–170** (line 170 is exactly "**0 of 51 files**", which S09 gets right elsewhere). The file is clean at HEAD so these should be exact. (b) The 04:20/05:35 explanation is an inference presented as fact. Verified inputs: the handoff's own line 3 reads "Written 2026-08-01 04:20 IST", and `~/sgnk/gates` mtime is 2026-08-01 05:35 — but the handoff file's own mtime is **06:01**, i.e. after the directory changed, so a stale-count-at-04:20 is plausible rather than established.
- **Correction:** (a) Cite `docs/engine/PLAN.md:163–165` for Foster et al. and `:167–170` for the measurements. (b) Tag the timing as inference: "...which publishes '27 gate scripts'. The handoff stamps itself `Written 2026-08-01 04:20 IST` (line 3) while `~/sgnk/gates` has mtime 2026-08-01 05:35 [measured], so the likeliest reading is a count taken before the directory changed — **[inference]**; the handoff file itself was last written at 06:01, so this is not established.

#### D69 · S07 §7.2.2 ("PWA and desktop" row) and S07 §7.1.1 (Fact 1, last paragraph of the allowlist block)

- **Claim:** "four `tauri:build:*` scripts in `package.json` " and " `src/config/env.ts:252` **synthesises** a development-bypass actor from the same single string".
- **Problem:** Two minor precision errors. (a) Exactly **three** scripts match `tauri:build:*` — `tauri:build:mac-arm`, `tauri:build:mac-intel`, `tauri:build:mac-universal`; the fourth is plain `tauri:build`. (b) `src/config/env.ts:252` is real and reads `const allowedLogin = process.env["ALLOWED_GH_LOGIN"] ?? "dev";` inside `readDevBypassFlags()` — it *supplies* the login; the synthetic actor is assembled in `src/modules/auth/presentation/session.ts` (whose header comment reads "a synthetic actor whose login matches ALLOWED\_GH\_LOGIN"). The substance — a dev bypass driven off the same single string — is correct.
- **Correction:** (a) "... `tauri:build` plus three `tauri:build:mac-*` scripts in `package.json`." (b) " `src/config/env.ts:252` feeds the same single string into the development bypass ( `process.env["ALLOWED_GH_LOGIN"] ?? "dev"` ), and `src/modules/auth/presentation/session.ts` synthesises an actor from it."

#### D70 · S08 §8.6.4 ("Easier for AI" and 'improves comprehension' are unproven")

- **Claim:** "The '25–168x fewer tokens' promise is an **artifact-size ratio** ... one area measured a headings-only index at **14.9x** corpus-wide (11.7x median per file), and its verifier replied ... that a 211–250 bytes-per-document index yields **94.6–112.1x**, inside the published **61–152x** band."
- **Problem:** Two different "published" bands are used in one paragraph as if they were the same number, without saying so. `docs/mdmax/PLAN.md` §4.4's headline is **25–168x**; the **61–152x** band is the capability-run evidence field's per-corpus full/index range (MD-hand 90.37x, knowledge 61.12x, frontmatter/docs 66.60x, Skills/ 152.55x, Mirrors/ 103.81x). A reader cannot tell that the number being defended (61–152x) is not the number being attacked (25–168x). Everything else reproduces: 14.93x and 11.7x, the outline band 14.82–35.49x, and the +5.2% OKF token loss are all exact in the source.
- **Correction:** "The plan §4.4 promises **25–168x**; the capability run's own evidence field states a per-corpus full/index range of **61–152x** (MD-hand 90.37x, knowledge 61.12x, frontmatter/docs 66.60x, Skills/ 152.55x, Mirrors/ 103.81x). **These are two different published bands and the record never reconciles them.** Against the 61–152x band, the '14.9x corpus-wide (11.7x median per file)' refutation is a tier mismatch — headings-only is the OUTLINE tier (band 14.82–35.49x), and a 211–250 bytes-per-document index yields 94.6–112.1x. Neither band may be quoted as a benefit to an AI consumer until a live model call measures the task; the only such measurement in the record is OKF's, at **+5.2% tokens**, a loss."

#### D71 · S10 §10.1.3

- **Claim:** "The splice writer ... Its correctness is already established: stock `mdast` `position.offset` spliced 107,287 top-level blocks across 1,080 files of the pinned corpus byte-identically, 0 failures [measured]. D7 is therefore not an aspiration; it is a function that exists."
- **Problem:** This is the exact measurement the `against` verifier judged OVERSTATED as a mathematical tautology. Verbatim from `wf-final-gate result.json`, claim C5: "the test is a MATHEMATICAL TAUTOLOGY and cannot fail. `src.slice(0,s) + src.slice(s,e) + src.slice(e) === src` is true for ANY `s <= e`. I ran it with deliberately wrong offsets (2,7), absurd offsets (0,1) and a degenerate range (11,11): all four return true." It also found `spliceBlock` disturbed top-level structure in 994 of 1,080 files on non-syntax-preserving replacements. S12 §12.4 item 13 of this same document records this correctly ("Our splice-fidelity evidence was a tautology"), so S10 and S12 contradict each other on the single most load-bearing engine claim.
- **Correction:** Replace with the sound re-test: "A re-run that CAN fail — substitute one alphabetic character inside the block, reparse, require the block count preserved and every other block's source text unchanged — gave 299/299 files and 2,832 syntax-preserving splices with structure intact, on an explicit subset of the corpus (every 3rd non-empty file <=30 KB, first 12 blocks each) [measured, against-area verifier]. The 1,080/1,080 identity round trip that is often quoted is a tautology and proves nothing; do not cite it. The conclusion — `mdast` offsets are correct — survives on the re-test, and the API needs a reparse-and-verify guard, which is engine work."

#### D72 · S11 §11 preamble (Effort unit)

- **Claim:** "532 insertions into `src/` in the 19 days between the clone and 2026-08-01 ... That is roughly 28 lines of product source per day."
- **Problem:** The premortem verifier explicitly corrected this and the correction is in the cited source: "'19 days since the clone' is wrong: the clone is 0c1b427 'feat: clone md app into frontmatter (product base)', 2026-07-17 — 15 days before 2026-08-01. 19 days is the age of the repo (b9ddcfe, 2026-07-13)." Verified live: `git log` gives 0c1b427 at 2026-07-17 and b9ddcfe at 2026-07-13. It also contradicts S12 §12.3 in this same document, which correctly dates the clone "2026-07-17". 532/19 = 28.0; 532/15 = 35.5.
- **Correction:** "532 insertions into `src/` in the 15 days between the clone ( `0c1b427` , 2026-07-17) and 2026-08-01, across three commits — `8eb4de2` 364, `58322f7` 124, `6ef83a9` 44. [measured, final-gate area premortem N1, with the verifier's date correction applied] That is roughly 35 lines of product source per day."

**D73** · S11 §11.15 (do-not-build table, markdown-as-graph row)

- **Claim:** "46.8% of files sit in zero resolved edges; median out-degree 2 (median document degree 0); 45 transclusions in 25.5 MB; max transitive blast radius 54 files"
- **Problem:** "median out-degree 2" appears in no source and is contradicted by every run in the record. wf-final-gate result.json reports, three separate times: "median out-degree 0 and median in-degree 0"; "median out-degree ZERO, mean 2.27, p90 5"; and the script output "outdeg mean 2.05 / median 0 / max 186 (knowledge.md)". 2.05–2.27 is the MEAN. Reporting a mean as a median in a row whose whole point is that the graph is dust inverts the strongest evidence in the row.
- **Correction:** "46.8% of files sit in zero resolved edges; mean out-degree 2.05–2.27, median out-degree ZERO, p90 5, max 186 ( knowledge.md ); 45 transclusions in 25.5 MB; max transitive blast radius 54 files"

**D74** · S10 §10.10.1 (delete table, folder-compiler row)

- **Claim:** "507 of 1,084 files affect nothing downstream; 530 have no dependents"
- **Problem:** The two labels are attached to the wrong numbers. The source's script output is `orphans(indeg=0)=530 leaves(outdeg=0)=641 isolated=507`, and the final-gate prose says verbatim: "507 of 1,084 files (46.8%) participate in zero resolved edges. 530 (48.9%) affect nothing downstream." S10 §10.7.2 of this same section uses 507 correctly as "files participating in ZERO resolved edges", so §10.10.1 also contradicts §10.7.2.
- **Correction:** "507 of 1,084 files (46.8%) participate in zero resolved edges at all; 530 (48.9%) affect nothing downstream; max blast radius 54; a full cold lex of all 25.5 MB is 1,056 ms"

**D75** · S10 §10.10.1 (delete table, mdmax unpack row)

- **Claim:** " `llm-code-format` ships pack+unpack at 387 downloads/week against repomix's 327,543 — an 846:1 revealed-demand ratio"
- **Problem:** The unit is wrong and it contradicts the same document twice. The source states "387 downloads/month" five times against "downloads/week" three times, and every sentence that carries the 846:1 ratio uses /month: "It gets 387 downloads/month. repomix gets 327,543. That is 846:1." S11 §11.9 and S12 §N6 of this same plan both say "387 downloads/month". A ratio requires the same unit on both sides.
- **Correction:** " `llm-code-format` ships pack+unpack at 387 downloads/month against repomix's 327,543 — an 846:1 revealed-demand ratio against the reverse operation." (Note in passing that repomix's own npm figure is reported elsewhere in the same source as 327,053 monthly downloads for 2026-07; 327,543 is the number the 846:1 was computed from.)

**D76** · S11 §11.4.3

- **Claim:** "publish(file) must not throw — including on the 171 that no YAML parser reads."
- **Problem:** 170 files are unreadable by both YAML parsers, not 171. The product-gap verifier isolated the difference precisely: "the same sentence says 171 and then 'those 170' without reconciling — I isolated the difference, exactly one file (`md/pj.md`) parses fine and throws only at stringify." I re-ran the measurement over the pinned corpus this session and reproduce `jsThrow=170, eeErr=170, bothThrow=170, bothOk=737` of 907 frontmatter blocks. §11.4.1's own body text says "170 of 907 blocks (18.7%) parse under neither YAML library", so §11.4.3 contradicts §11.4.1 — the exact slip the verifier flagged.
- **Correction:** "publish(file) must not throw — including on the 170 that no YAML parser reads, and including `md/pj.md`, which parses but throws inside `matter.stringify` (171 files fail the current write path in total)."

**D77** · S11 §11.6 (and S12 §N4, §12.4 item 8 context)

- **Claim:** "Relay at 176,247 installs (a second area recorded 175,904 ...)" — and in S12 §N4, "Combined installs of every markdown comment plugin ... = 24,569" and "Relay has 175,904 installs"
- **Problem:** The metric label is wrong and the source's verifier corrected it explicitly: "One metric-label error: the plugin-stats field is 'downloads', not installs, and 176,247 is cumulative downloads across all versions. Say 'downloads'." The same applies to the 24,569 / 4,659,822 pair — the independent re-measurement enumerates `community-plugin-stats.json` and reports "187,154 cumulative downloads. dataview = 4,669,090". Cumulative downloads and installs are different quantities, and S12 already flags the age confound that makes the distinction matter.
- **Correction:** S11 §11.6: "Relay at 176,247 cumulative downloads (a second area recorded 175,904 — a fetch gap; quote the range)." S12 §N4: "Combined cumulative downloads of every markdown comment plugin in the largest markdown ecosystem = 24,569, against 4,659,822 for Dataview" and "Relay has 175,904 cumulative downloads."

**D78** · S10 §10.1.3 vs §10.9

- **Claim:** §10.1.3: "107,287 top-level blocks across 1,080 files". §10.9: "total top-level blocks mdast 107317 lezer 107315 (0.0019% apart)"
- **Problem:** Two different totals for the same quantity — mdast top-level blocks over the 1,080 non-empty pinned files — 30 blocks apart, presented in the same section with no reconciliation. Both appear in the record (107,287 from the `against` area's splice run; 107,317/107,315 from the parsing-robustness two-parser run). The document's own standard requires naming the contradiction and stating which governs.
- **Correction:** Add one sentence at §10.9: "Two independent runs over the same 1,080 files give 107,287 (the `against` area's splice pass) and 107,317 (the parsing-robustness two-parser pass) top-level mdast blocks — 30 blocks, 0.028% apart. The difference is unexplained and almost certainly a node-filtering difference (the splice pass skipped nodes without complete `position.offset`). 107,317 governs for segmentation comparisons because it was measured by the run that also produced the Lezer figure; re-derive both before publishing either."

**D79** · S12 §12.3 (Step 2) and §N1

- **Claim:** "21,415 lines of TypeScript in src/ — every one inherited from the md-app clone of 2026-07-17" and "21,415 lines (all inherited from the md app clone)"
- **Problem:** "Every one" / "all" is an inference contradicted by the same corpus and by S11 in this document. The premortem's own measurement, verified: "Only three commits touched src since the product-base import: `8eb4de2 +364, 58322f7 +124, 6ef83a9 +44` = 532 exactly." I verified live that a fourth commit, `1bd4dad`, adds another 11 lines to src/. So at least 543 lines postdate the clone. The load-bearing point — that no MDMAX engine code exists — is separately established by `grep -ril mdmax src test scripts` returning 0 files, which I verified live.
- **Correction:** "21,415 lines of TypeScript in src/, of which 543 postdate the md-app clone of 2026-07-17 (`8eb4de2 +364, 58322f7 +124, 6ef83a9 +44, 1bd4dad +11`) and none is engine work — `grep -ril mdmax src test scripts` returns 0 files [measured, verified live]."

**D80** · S11 §11.8 (review-loop resolver, tier 2 row)

- **Claim:** "86.658% (verifier independently re-derived 85.036%) vs hash-alone 77.297%, on 29,217 anchorable block-versions from the pinned corpus"

- **Problem:** One denominator is attached to two figures measured on different samples. 29,217 anchorable block-versions over 394 consecutive revision pairs is the researcher's run (86.658% / 77.297%). The verifier's 85.036% is a different, larger sample: "my independent run ... all 494 consecutive commit pairs touching a pinned-corpus file across all three roots, 31,149 anchorable block-versions ... gives LCS 85.036% matched ... hash-alone-unique 76.352%." The verifier's hash-alone baseline is 76.352%, not 77.297%.
- **Correction:** "86.658% over 29,217 anchorable block-versions from 394 consecutive revision pairs, against hash-alone 77.297%; independently re-derived by the verifier at 85.036% over 31,149 block-versions from 494 pairs, against hash-alone 76.352% [measured, corpus\_id sha256:3a010b16...]"

#### D81 · S10 §10.8 (Tier A)

- **Claim:** "no CommonMark 0.31.2 spec suite is present in this repository ( `find . -name spec.json` outside `node_modules` returns nothing) and an independent verifier could not check it [measured, this section]"
- **Problem:** Two issues. (a) "Could not check" asserts a failed attempt; the record supports only "did not check" — the spec-suite claim appears in neither the `survived` nor the `killed` list of the parsing-robustness verification, i.e. it was never adjudicated. (b) The numbers are not [secondary]: the researcher fetched the suite live and tagged the result own-measurement — "scratchpad/spec.mjs against <https://spec.commonmark.org/0.31.2/spec.json>: {"total":652,"micromark":{"pass":652,"fail":0},"marked":{"pass":531,"fail":121}}". I confirmed live that no `spec.json` exists outside `node_modules`.
- **Correction:** "The 652/652 and 531/652 figures were measured live against <https://spec.commonmark.org/0.31.2/spec.json> [measured, parsing-robustness area] but were never adjudicated by that area's verifier — the claim appears in neither its `survived` nor its `killed` list — and the suite is not vendored here ( `find . -name spec.json` outside `node_modules` returns nothing [measured, this section]). Treat them as unreplicated single-run measurements. First task in tier A: vendor `spec.json` at a pinned version and re-derive both numbers."

#### D82 · S11 §11.7

- **Claim:** " `market-and-gift` enumerated [markdownguide.org](https://github.com/markdownguide/markdownguide.org)'s `_data/tools.yml`: exactly 28 construct ids, and `frontmatter`, `wikilink`, `callout`, `mermaid`, `math`, `directive`, `embed` and `attribute` appear zero times"
- **Problem:** `attribute` is not one of the strings the source tested. Verbatim: "I fetched [https://raw.githubusercontent.com/mattcone/markdown-guide/master/\\_data/tools.yml](https://raw.githubusercontent.com/mattcone/markdown-guide/master/_data/tools.yml) and enumerated it: exactly 28 construct ids, and the strings "frontmatter", "wikilink", "callout", "mermaid", "math", "katex", "admonition", "directive", "transclu", "embed" appear ZERO times." Three tested strings ( `katex`, `admonition`, `transclu` ) are dropped and one untested string ( `attribute` ) is added.
- **Correction:** "...and the strings `frontmatter`, `wikilink`, `callout`, `mermaid`, `math`, `katex`, `admonition`, `directive`, `transclu` and `embed` appear zero times"

#### D83 · S11 §11.5

- **Claim:** "pain 9 (No browser access / platform gaps) carries severity "Kills at evaluation", with a 63-comment thread that "asks for exactly this.""
- **Problem:** The quoted phrase is a paraphrase presented inside quotation marks. `docs/Frontmatter-Product-Plan.md:38` reads verbatim: "I 9 | No browser access / platform gaps | Kills at evaluation | Obsidian has NO web app; a 63-comment thread begs for exactly what frontmatter is |". The severity string and the count are exact; the quoted clause is not.
- **Correction:** "pain 9 (No browser access / platform gaps) carries severity "Kills at evaluation": "Obsidian has NO web app; a 63-comment thread begs for exactly what frontmatter is" [primary, docs/Frontmatter-Product-Plan.md:38]"

#### D84 · S10 §10.2.2 (cost table, first row)

- **Claim:** "build the `Uint32Array` UTF-16 -> byte prefix table for the largest file | 4 ms, 5.8 MB"
- **Problem:** The row does not name its unit, which is the exact discipline §10.2.1 of this same section establishes three paragraphs earlier ("A sentence about 'the largest file' is ambiguous unless it names its unit"). The source measured `md/graphify-out/GRAPH_REPORT.md`: "build Uint32 UTF16->byte prefix table for largest file 4ms bytes=1466025" — 1,466,025 bytes / 1,457,271 UTF-16 units. §10.2.1 has already established that the largest file by bytes is `md/_Archive/Satinath Content JSON (raw source).md` at 1,491,757 bytes.
- **Correction:** "build the `Uint32Array` UTF-16 -> byte prefix table for `md/graphify-out/GRAPH_REPORT.md` (1,466,025 bytes / 1,457,271 UTF-16 units — the largest file by UTF-16 units) | 4 ms, 5.8 MB"

#### D85 · S11 §11.12.1

- **Claim:** "The verifiers are roughly 88% reliable, not 100%."
- **Problem:** The figure does not follow from the audit it is derived beside. The four kill-audit agents report 5/29, 1/29, 4/21 and 0/26 overturned = 10 of 105 sampled = 9.52%, i.e. ~90.5% reliable; the worst single sample is 19.0%, i.e. 81%. 88% matches neither the pooled rate nor any single sample nor the mean of the four rates (9.91%). It is inherited verbatim from `docs/mdmax/PLAN.md v2.0.0 §0 line 43`, where it is likewise underived. This is a P2 violation inside the section that states P2.
- **Correction:** "Pooled naively that is 10 of 105 = 9.52% false kills, so the verifiers are roughly 90% reliable, not 100% — but the four samples are not verified disjoint and the worst single sample is 19.0%, so the honest working assumption is a false-kill rate somewhere in 0–19%. [measured, wf-final-gate kill\_audit[]]" (S12's preamble already states it this way; align S11 to S12.)

#### D86 · S12 §12.1.3

- **Claim:** "the founder himself authored a 1,409-term controlled vocabulary unprompted, in the form of repeated dangling wikilinks"
- **Problem:** A single-run figure presented as settled, after §12.1.1 of this same section correctly gave the range. The researcher measured 1,409 targets / 5,834 occurrences / 54.0%; the verifier re-derived 1,326 / 5,415 / 53.2%. §12.1.1 states both and says "use the range, not either endpoint"; §12.1.3 then quotes only the flattering endpoint.
- **Correction:** "the founder himself authored a controlled vocabulary of roughly 1,300–1,400 repeated terms unprompted (researcher 1,409 targets / 5,834 occurrences; verifier 1,326 / 5,415), in the form of repeated dangling wikilinks"

#### D87 · S11 §11.8 (review-loop resolver, tier 3 row)

- **Claim:** "content-derived, scoped to blocks of ≤10 tokens, refusing rather than guessing"
- **Problem:** Off by one against the measured strata. The source says "scoped to what my stratification shows it is actually for: blocks under ten tokens", and the buckets are 3–5, 6–9, 10–19, 20+. The 10–19 bucket is already at 0.30–0.34% at-risk, i.e. essentially solved; including 10 in the tier-3 scope pulls in the bucket the tier is not for.

- **Correction:** "content-derived, scoped to blocks of fewer than ten tokens (the 3–5 and 6–9 buckets, at 44.28–47.0% and 8.64–8.72% at-risk respectively), refusing rather than guessing"

#### D88 · S12 §12.1.1 and S11 §11.9

- **Claim:** "container (all 1,084 files packed) 25,548,765 B" / "Container 25,548,765 B vs a one-line-per-doc index 105,538 B — 242x"
- **Problem:** 25,548,765 is the raw byte total of the unpacked corpus, not a packed container. The container-thesis area measured the actual packed artifact at 25,776,700 bytes (+0.892% framing overhead), independently reproduced by its verifier at 25,775,670 (+0.888%). The inherited sloppiness comes from the verdict lens, which writes "the container is 25,548,765 bytes" — but the same document's §12.1.1 already flags the 110x-vs-242x discrepancy, so this is the second, unflagged half of the same conflation.
- **Correction:** "corpus, unpacked 25,548,765 B; packed container 25,776,700 B (+0.892%); one-line-per-doc index 105,538 B → 242x against the corpus, 244x against the packed container"

#### D89 · S10 §10.5.2

- **Claim:** "This plan's own measurement of 68,798 messages found that model-generated markdown is a narrow nine-construct dialect in which 48.3% of list items open with a bold run [secondary]."
- **Problem:** Not wrong, but the record flags this exact anchor as disputed and S10 does not say so. `HANDOFF-mdmax-markdown-engine-2026-08-01.md:131` reads: "18 | AI list items starting with bold | **48.3% vs human 5.3%** | measured (I got 50.0% vs 2.2% on a 7-doc spot check) |", and the final-gate record cites "the plan's own 48.3%/5.3% anchors, which HANDOFF-mdmax §3 line 18 itself flags as disputed". The AI-side figure replicates well (48.3 vs 50.0); the human baseline does not (5.3 vs 2.2), which matters because the sentence is making a contrast claim.
- **Correction:** "...in which 48.3% of list items open with a bold run against a human baseline of 5.3% [secondary; a 7-document spot check in the same record gives 50.0% vs 2.2%, so the AI-side figure replicates and the human baseline does not — quote the contrast as a direction, not a ratio]."

## 14.2 Numbers lacking a source

- S03 §3.6.6 — "547 of 547 self-declared `updated:` dates ... are stale, 32.9% by more than a week", tagged bare `[measured]` : no corpus\_id, no area key, no population. Actual population per the source: `git ls-files '*.md'` over 4,131 tracked files in ~/Desktop/GitHub/md, from the 18-area capability run.
- S03 §3.4.3 — BREAK 95 / DRIFT 52 / GHOST 5,777 and "a checker that reports 3,442 problems": area key given (`impact-preview`) but no corpus\_id and no stated population. Actual: census.py over three unpinned corpora — md-vault 4,287 files / 75.94 MB, knowledge 421 / 7.39 MB, frontmatter 112 / 1.54 MB.
- S03 §3.4.3 — "365 true problems touching only 112 files (13.4%)" over "833 authored files": cites `docs/engine/PLAN.md §4` but no corpus\_id; the 833-file population is not the 1,084-file pinned corpus.
- S03 §3.4.4 — "69,805 rows for 78 MB", "607 KB for a 78 MB vault (0.74%)", "0.254 ms median, 1.567 ms p90": no corpus\_id, no named vault, no script.
- S01 §1.6 — "tests | 83 test files under `test/` | tagged `[measured]` with no command; and "3,484 tests collected (66.4% of them abandoned worktrees) at last full run" with no re-derivation. Live count is 83 test files under `test/`; `vitest list` gives 247 files / 3,484 tests including 164 stale worktree files.
- S03 §3.0.1 — the effort column ("tiers 1–2: 2–3 days · tier 3: 2–3 weeks incl. benchmark", "symbol table 2–3 weeks", "fence-aware split/join ~2 days"): tagged `[inference]` in one cell only; the other cells carry no tag and no basis. Every effort figure in the research corpus is an estimate (S13 §13.9 says so explicitly) — tag the whole column.
- S02 §2.2 — the `~2,400,000` token term inside the 29,946,391 cumulative. S02 does flag it as "the only soft input", but it has no completion notification behind it and no artifact records it; it should carry `[inference]` inline, not only in the surrounding prose.
- S03 §3.5.4 — 82.5% / 59.1% (214/362) / 40.9% (148/362) / median 5,260 B / 63.9% of whole-read bytes: the population is named in prose ("the 1,200 most recent agent transcripts on this machine") but there is no manifest or pinned id for it, so it is not re-runnable. The `aio-s-transfer` area flags exactly this failure for its own transcript numbers: "a MOVING selection ... it needs a corpus id before it is quoted."
- S04 §4.8 row 10: "82.8% of thematic breaks (7,171 of 8,663) immediately precede a heading" — tagged `[measured]` with no corpus\_id, path or area key. It is traceable to the capability run's `missing-constructs` area (8-corpus sample, 4,074 files / 1,293,322 lines) but that sample carries no selection hash anywhere in the section.
- S04 §4.8 row 11: "`<details>` / `<summary>` ... 1,716 / 1,710 uses in the top-300 README corpus" — no area key, no selection hash for the top-300 README corpus. Source is `missing-constructs`.
- S04 §4.8 row 6 and §4.6: "299 forum likes ask for it" (checkbox in a table cell) — no source of any kind. Source is `missing-constructs`, itself unsourced there.
- S04 §4.6 and §4.8 row 6: "4 in 114,762 cells across the 8-corpus sample" and §4.7 A2's "8-corpus, 4,074-file sample" — the 8-corpus sample is used as a denominator six times across S04 and S05 and is never given a selection hash, unlike Corpus B, Corpus D, Corpus E and the `~/claudeskills-src` corpus, which all carry one.
- S04 §4.6: "emoji used as status ..... 15,764 characters" — a raw count with no rate and no denominator; it is not comparable to the other rows in the same block.
- S05 §5.2.2: "`@mdx-js/mdx` 0" published advisories — the other eight counts in that list (dompurify 27, marked 18, mermaid 11, vega 8, katex 5, vega-lite 0, wavedrom 0, viz.js 0) are all traceable to the rendering-frontier area; `@mdx-js/mdx` is not, and I could not locate it in either result JSON.
- S05 §5.8.1: "PDF 9.5% fully accessible" and "85% of TAGGED files defective" — S05 already flags these as UNSOURCED and refuses to use them. Independently confirmed absent: not in either result JSON, not in their journals, not in docs/engine/PLAN.md. Keep the refusal.
- S05 §5.10.4 and S06 §6.6 Rule 3: "Guidance 41%, llama.cpp 39%, XGrammar 28%, Outlines 3%" — sourced to docs/engine/PLAN.md §15.8, which itself gives no citation for the benchmark. Verified present at that path; the underlying source is unnamed anywhere in the corpus.
- S06 §6.2.4: Sonatype figures (>454,600 new malicious packages in 2025; >1.233 million cumulative, +75% YoY; 120,612 blocked in Q4 2025; ">99% of all open source malware now targets npm") — S06 correctly labels them `[secondary]` /search-tier and states nobody read the report; the underlying research says the same. Keep the label.
- S07 §7.5 N4 / S07 §7.7.1 support: "911 of 337,410 blocks (0.27%) over 5,916 .md files across both corpora" — no corpus\_id, and explicitly not the pinned 1,084-file corpus. Present verbatim in the capability-run verification but with no corpus id there either.
- S07 §7.7.2 Count B and S08 §8.3.2/§8.3.3: "2,559 non-URL string leaves", "sync 358", "mobile 245", "realtime/collab 120 broad, 108 tight", "unambiguous review-loop family 25", "conformance 11", "provenance 5". These ARE quoted verbatim from wf-final-gate-2026-08-01.result.json, but the instrument is unstated there and none of them reproduces. My independent walk of both demand corpora gives 2,566–2,571 non-URL leaves, sync 326–328, mobile 141 leaves / 155 occurrences, collab-broad 106, review-loop family 21, `conform` 1, `provenan` 0. By contrast Count A (89 feature requests, feature+why fields) reproduces to the digit. Label Count B "instrument unstated, not independently reproduced".

- S08 §8.3.2: "a strict metadata regex catches 34.2% of the 155 comment-bearing leaves, and 8 of 8 sampled residuals were also metadata" — the regex is not stated, so the 34.2% cannot be re-derived; I count 184–186 comment-bearing non-URL leaves, not 155.
- S08 §8.8.2: the OKF creation-date windows "69 / 66 / 89 / 59" sum to 283 against the stated total\_count of 287. The source reconciles this ("4 repos predate the announcement — internally consistent, not a defect"); S08 prints the windows and the total without the reconciliation, so the arithmetic reads as broken.
- S08 §8.11.1 item 2: "\$1.18B, \$1.35B, \$11.02B and \$17.19B — a 10× divergence". The four figures and the phrase "10× divergence" are quoted verbatim from the source and correctly tagged [secondary, self-tagged unverified], but  $17.19/1.18 = 14.6\times$ , so the label understates its own spread. No claim depends on it.
- S07 §7.6.1 and S07 §7.6.5: the founder's Free/Pro/Max sheet specifics — a five-document Free cap, metered AI credits, and GitHub sync as a Max-tier feature. S07 states plainly that it searched and did not find them, which is correct and verified: my grep across docs/, all HANDOFF-\*.md, polyg.yml, cx.html and the research JSONs finds no such sheet, and docs/mdmap/06-product/MAP.md:57 lists [[pricing]] with status "planned" and no content. This is correctly flagged in-section; listed here only so the reader knows it is the largest unsourced input to §7.6.
- S11 §11.12.1 — "roughly 88% reliable": no derivation anywhere; the kill audit gives  $10/105 = 9.52\%$  false kills (~90.5%), worst single sample 19.0%. Inherited from docs/mdmax/PLAN.md v2.0.0 §0 line 43, where it is also underived.
- S10 §10.5.8 — "Roughly 40 minutes of wall clock on one machine" for the three-arm false-refusal experiment: no basis given, and the section's own measurements (27,390 ms to parse the corpus once, up to 31 s on a single file) argue it is optimistic for five tasks over 1,635 documents.
- S11 §11.1 and §11.3.3 — the founder-day estimates (0.5 / 8–12 / 15–25 / 5–8 / 2 / 20–30 / 3–5): no comparable, no measurement. §11.17 item 1 acknowledges these are [inference], but they appear untagged in the §11.1 table where a reader will act on them.
- S11 §11.11.4 — "Effort for the worktree preflight and HEAD reconciliation: ~3 hours" cited to "aios-transfer effort block"; I could not locate a 3-hour figure in that area's effort field.
- S10 §10.5.6 — "MAX\_LIST\_MARKERS = 20,000 ... 20,000 is a 5x margin": the 5x is stated against "roughly 100,000 items", which is where the flat-list cost crosses 20 s, but  $100,000/20,000 = 5$  only if the threshold is item count; the gate counts list-marker LINES, which is not established to be the same population.
- S10 §10.5.7 — "Cap per-block token count at 4,096" and "Cap any posting list at 512 entries": both constants are asserted; only the 256 candidate cap is derived (from the measured average of 186).